

Oracle® AI Database

Database Globalization Support Guide



26ai
G43222-05
August 2026

ORACLE®

Copyright © 2007, 2026, Oracle and/or its affiliates.

Primary Author: Mary Beth Roeser

Contributors: Dan Chiba, Simon Law, Qianrong Ma, Keni Matsuda, Makoto Tozawa, Sergiusz Wolicki, Michael Yau, Qin Yu, Tim Yu, Weiran Zhang

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software, software documentation, data (as defined in the Federal Acquisition Regulation), or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software," "commercial computer software documentation," or "limited rights data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle®, Java, MySQL, and NetSuite are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

For information about Oracle's commitment to accessibility, visit the Oracle Accessibility Program website at <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=docacc>.

Oracle customer access to and use of Oracle support services will be pursuant to the terms and conditions specified in their Oracle order for the applicable services.

Contents

Preface

Intended Audience	i
Documentation Accessibility	i
Diversity and Inclusion	ii
Related Documentation	ii
Conventions	ii

Changes in Oracle AI Database Globalization Support Guide for Oracle AI Database 26ai

New Features	i
--------------	---

1 Overview of Globalization Support

1.1	Globalization Support Architecture	1
1.1.1	Locale Data on Demand	1
1.1.2	Architecture to Support Multilingual Applications	2
1.1.3	Using Unicode in a Multilingual Database	4
1.2	Globalization Support Features	4
1.2.1	Language Support	5
1.2.2	Territory Support	5
1.2.3	Date and Time Formats	6
1.2.4	Monetary and Numeric Formats	6
1.2.5	Calendar Systems	6
1.2.6	Linguistic Sorting	7
1.2.7	Character Set Support	7
1.2.8	Character Semantics	7
1.2.9	Customization of Locale and Calendar Data	7
1.2.10	Unicode Support	8

2 Choosing a Character Set

2.1	Character Set Encoding	1
2.1.1	What is an Encoded Character Set?	1

2.1.2	Which Characters Are Encoded?	2
2.1.2.1	Phonetic Writing Systems	3
2.1.2.2	Ideographic Writing Systems	3
2.1.2.3	Punctuation, Control Characters, Numbers, and Symbols	3
2.1.2.4	Writing Direction	3
2.1.3	What Characters Does a Character Set Support?	4
2.1.3.1	ASCII Encoding	5
2.1.4	How are Characters Encoded?	6
2.1.4.1	Single-Byte Encoding Schemes	6
2.1.4.2	Multibyte Encoding Schemes	7
2.1.5	Naming Convention for Oracle Database Character Sets	8
2.1.6	Subsets and Supersets	8
2.2	Length Semantics	9
2.3	Choosing an Oracle Database Character Set	11
2.3.1	Choosing Unicode as a Database Character Set	13
2.3.2	Current and Future Language Requirements	13
2.3.3	Client Operating System and Application Compatibility	13
2.3.4	Character Set Conversion Between Clients and the Server	13
2.3.5	Running Single-Byte Client Applications with a Unicode Database	14
2.3.6	Character Set for Data Migrated from an EBCDIC Platform	15
2.3.7	Restrictions on Database Character Sets	15
2.3.7.1	Restrictions on Character Sets Used to Express Names	15
2.3.8	Database Character Set Statement of Direction	16
2.3.9	Choosing a National Character Set	16
2.3.10	Summary of Supported Data Types	17
2.4	Choosing a Database Character Set for a Multitenant Container Database	18
2.5	Changing the Character Set After Database Creation	20
2.6	Monolingual Database Scenario	21
2.6.1	Character Set Conversion in a Monolingual Scenario	22
2.7	Multilingual Database Scenario	23

3 Setting Up a Globalization Support Environment

3.1	Setting NLS Parameters	1
3.2	Choosing a Locale with the NLS_LANG Environment Variable	3
3.2.1	Specifying the Value of NLS_LANG	5
3.2.2	Overriding Language and Territory Specifications	6
3.2.3	Locale Variants	7
3.2.4	Should the NLS_LANG Setting Match the Database Character Set?	8
3.3	Character Set Parameter	8
3.3.1	NLS_OS_CHARSET Environment Variable	8
3.4	NLS Database Parameters	9

3.4.1	NLS Data Dictionary Views	9
3.4.2	NLS Dynamic Performance Views	9
3.4.3	OCINlsGetInfo() Function	10
3.5	Language and Territory Parameters	10
3.5.1	NLS_LANGUAGE	10
3.5.2	NLS_TERRITORY	13
3.5.2.1	Overriding Default Values for NLS_LANGUAGE and NLS_TERRITORY During a Session	15
3.6	Date and Time Parameters	16
3.6.1	Date Formats	16
3.6.1.1	NLS_DATE_FORMAT	17
3.6.1.2	NLS_DATE_LANGUAGE	18
3.6.2	Time Formats	19
3.6.2.1	NLS_TIMESTAMP_FORMAT	20
3.6.2.2	NLS_TIMESTAMP_TZ_FORMAT	20
3.7	Calendar Definitions	21
3.7.1	Calendar Formats	21
3.7.1.1	First Day of the Week	22
3.7.1.2	First Calendar Week of the Year	22
3.7.1.3	Number of Days and Months in a Year	23
3.7.1.4	First Year of Era	24
3.7.2	NLS_CALENDAR	24
3.8	Numeric and List Parameters	25
3.8.1	Numeric Formats	25
3.8.2	NLS_NUMERIC_CHARACTERS	26
3.8.3	NLS_LIST_SEPARATOR	27
3.9	Monetary Parameters	27
3.9.1	Currency Formats	27
3.9.2	NLS_CURRENCY	28
3.9.3	NLS_ISO_CURRENCY	28
3.9.4	NLS_DUAL_CURRENCY	30
3.9.5	Oracle Database Support for the Euro	30
3.9.6	NLS_MONETARY_CHARACTERS	31
3.9.7	NLS_CREDIT	31
3.9.8	NLS_DEBIT	31
3.10	Linguistic Sort Parameters	32
3.10.1	NLS_SORT	32
3.10.2	NLS_COMP	33
3.11	Character Set Conversion Parameter	34
3.11.1	NLS_NCHAR_CONV_EXCP	34
3.12	Length Semantics	34

4 Datetime Data Types and Time Zone Support

4.1	Overview of Datetime and Interval Data Types and Time Zone Support	1
4.2	Datetime and Interval Data Types	1
4.2.1	Datetime Data Types	2
4.2.1.1	DATE Data Type	2
4.2.1.2	TIMESTAMP Data Type	3
4.2.1.3	TIMESTAMP WITH TIME ZONE Data Type	4
4.2.1.4	TIMESTAMP WITH LOCAL TIME ZONE Data Type	5
4.2.1.5	Inserting Values into Datetime Data Types	6
4.2.1.6	Choosing a TIMESTAMP Data Type	9
4.2.2	Interval Data Types	9
4.2.2.1	INTERVAL YEAR TO MONTH Data Type	10
4.2.2.2	INTERVAL DAY TO SECOND Data Type	10
4.2.2.3	Inserting Values into Interval Data Types	11
4.3	Datetime and Interval Arithmetic and Comparisons	11
4.3.1	Datetime and Interval Arithmetic	11
4.3.2	Datetime Comparisons	12
4.3.3	Explicit Conversion of Datetime Data Types	12
4.4	Datetime SQL Functions	12
4.5	Datetime and Time Zone Parameters and Environment Variables	15
4.5.1	Datetime Format Parameters	15
4.5.2	Time Zone Environment Variables	16
4.5.3	Daylight Saving Time Session Parameter	16
4.5.4	Daylight Saving Time Upgrade Parameter	16
4.6	Choosing a Time Zone File	17
4.7	Upgrading the Time Zone File and Timestamp with Time Zone Data	20
4.7.1	Upgrading the Time Zone Data Using the DBMS_DST Package	22
4.7.1.1	Prepare Window	22
4.7.1.2	Upgrade Window	24
4.7.1.3	Upgrade Example	27
4.7.1.4	Upgrade Error Handling	33
4.7.1.5	About the Upgrade History Table	34
4.7.2	Upgrading the Time Zone Data Using the utltz_* Scripts	35
4.7.2.1	Prepare Window	35
4.7.2.2	Upgrade Window	36
4.8	Clients and Servers Operating with Different Versions of Time Zone Files	38
4.9	Setting the Database Time Zone	39
4.10	Setting the Session Time Zone	40
4.11	Converting Time Zones With the AT TIME ZONE Clause	41

4.12	Support for Daylight Saving Time	42
4.12.1	Examples: The Effect of Daylight Saving Time on Datetime Calculations	42

5 Linguistic Sorting and Matching

5.1	Overview of Oracle Database Collation Capabilities	2
5.2	Using Binary Collation	2
5.3	Using Emulated Binary Collations	3
5.4	Using Linguistic Collations	3
5.4.1	Monolingual Collation	4
5.4.2	Multilingual Collation	4
5.4.2.1	Multilingual Collation Levels	5
5.4.3	UCA Collation	6
5.4.3.1	UCA Comparison Levels	7
5.4.3.2	UCA Collation Parameters	8
5.4.3.3	The UCA Japanese IVS Collation	9
5.5	Linguistic Collation Features	11
5.5.1	Base Letters	11
5.5.2	Ignorable Characters	12
5.5.2.1	Primary Ignorable Characters	12
5.5.2.2	Secondary Ignorable Characters	12
5.5.2.3	Tertiary Ignorable Characters	13
5.5.3	Variable Characters and Variable Weighting	13
5.5.4	Contracting Characters	15
5.5.5	Expanding Characters	15
5.5.6	Context-Sensitive Characters	15
5.5.7	Canonical Equivalence	15
5.5.8	Reverse Secondary Sorting	16
5.5.9	Character Rearrangement for Thai and Laotian Characters	17
5.5.10	Special Letters	17
5.5.11	Special Combination Letters	17
5.5.12	Special Uppercase Letters	17
5.5.13	Special Lowercase Letters	18
5.6	Case-Insensitive and Accent-Insensitive Linguistic Collation	18
5.6.1	Examples: Case-Insensitive and Accent-Insensitive Collation	19
5.6.2	Specifying a Case-Insensitive or Accent-Insensitive Collation	20
5.6.3	Examples: Linguistic Collation	21
5.7	Performing Linguistic Comparisons	23
5.7.1	Collation Keys	25
5.7.2	Restricted Precision of Linguistic Comparison	25
5.7.3	Avoiding ORA-12742 Error	26
5.7.4	Examples: Linguistic Comparison	27

5.8	Using Linguistic Indexes	30
5.8.1	Supported SQL Operations and Functions for Linguistic Indexes	31
5.8.2	Linguistic Indexes for Multiple Languages	32
5.8.3	Requirements for Using Linguistic Indexes	33
5.8.3.1	Set NLS_SORT Appropriately	33
5.8.3.2	Specify NOT NULL in a WHERE Clause If the Column Was Not Declared NOT NULL	33
5.8.3.3	Use a Tablespace with an Adequate Block Size	33
5.8.3.4	Example: Setting Up a French Linguistic Index	34
5.9	Searching Linguistic Strings	34
5.10	SQL Regular Expressions in a Multilingual Environment	35
5.10.1	Character Range '[x-y]' in Regular Expressions	35
5.10.2	Collation Element Delimiter '[. .]' in Regular Expressions	36
5.10.3	Character Class '[': :']' in Regular Expressions	36
5.10.4	Equivalence Class '['= =']' in Regular Expressions	36
5.10.5	Examples: Regular Expressions	36
5.11	Column-Level Collation and Case Sensitivity	38
5.11.1	About Data-Bound Collation	39
5.11.2	Default Collations	40
5.11.3	Enabling Data-Bound Collation	41
5.11.4	Specifying a Data-Bound Collation	42
5.11.4.1	Effective Schema Default Collation	42
5.11.4.2	Specifying Data-Bound Collation for a Schema	43
5.11.4.3	Specifying Data-Bound Collation for a Table	44
5.11.4.4	Specifying Data-Bound Collation for a View and a Materialized View	45
5.11.4.5	Specifying Data-Bound Collation for a Column	46
5.11.4.6	Specifying Data-Bound Collation for PL/SQL Units	48
5.11.4.7	Specifying Data-Bound Collation for SQL Expressions	49
5.11.5	Viewing the Data-Bound Collation of a Database Object	52
5.11.6	Case-Insensitive Database	53
5.11.7	Effect of Data-Bound Collation on Other Database Objects	53
5.11.8	Effect of Data-Bound Collation on Distributed Queries and DML Operations	58
5.11.9	Effect of Data-Bound Collation on PL/SQL Types and User-Defined Types	59
5.11.10	Effect of Data-Bound Collation on Oracle XML DB	60

6 Supporting Multilingual Databases with Unicode

6.1	What is the Unicode Standard?	1
6.2	Features of the Unicode Standard	2
6.2.1	Code Points and Supplementary Characters	2
6.2.2	Unicode Encoding Forms	2
6.2.2.1	UTF-8 Encoding Form	3

6.2.2.2	UTF-16 Encoding Form	3
6.2.2.3	UCS-2 Encoding Form	4
6.2.2.4	UTF-32 Encoding Form	4
6.2.2.5	CESU-8 Encoding Form	4
6.2.2.6	Examples: UTF-16, UTF-8, and UCS-2 Encoding	5
6.2.3	Support for the Unicode Standard in Oracle Database	5
6.3	Implementing a Unicode Solution in the Database	6
6.3.1	Enabling Multilingual Support for a Database	7
6.3.2	Enabling Multilingual Support with Unicode Data Types	8
6.3.3	How to Choose Between Unicode Solutions	10
6.4	Unicode Case Studies	10
6.5	Designing Database Schemas to Support Multiple Languages	12
6.5.1	Specifying Column Lengths for Multilingual Data	12
6.5.2	Storing Data in Multiple Languages	13
6.5.3	Storing Documents in Multiple Languages in LOB Data Types	14
6.5.4	Creating Indexes for Searching Multilingual Document Contents	15
6.5.4.1	Creating Multilexers	15
6.5.4.2	Creating Indexes for Documents Stored in the CLOB Data Type	16
6.5.4.3	Creating Indexes for Documents Stored in the BLOB Data Type	16

7 Programming with Unicode

7.1	Overview of Programming with Unicode	1
7.1.1	Database Access Product Stack and Unicode	1
7.2	SQL and PL/SQL Programming with Unicode	3
7.2.1	SQL NCHAR Data Types	3
7.2.1.1	The NCHAR Data Type	4
7.2.1.2	The NVARCHAR2 Data Type	4
7.2.1.3	The NCLOB Data Type	5
7.2.2	Implicit Data Type Conversion Between NCHAR and Other Data Types	5
7.2.3	Exception Handling for Data Loss During Data Type Conversion	5
7.2.4	Rules for Implicit Data Type Conversion	6
7.2.5	SQL Functions for Unicode Data Types	7
7.2.6	Other SQL Functions	8
7.2.7	Unicode String Literals	8
7.2.8	NCHAR String Literal Replacement	9
7.2.9	Using the UTL_FILE Package with NCHAR Data	9
7.3	OCI Programming with Unicode	10
7.3.1	OCIEnvNlsCreate() Function for Unicode Programming	11
7.3.2	OCI Unicode Code Conversion	12
7.3.2.1	Data Integrity	12
7.3.2.2	OCI Performance Implications When Using Unicode	12

7.3.2.3	OCI Unicode Data Expansion	13
7.3.3	Setting UTF-8 to the NLS_LANG Character Set in OCI	14
7.3.4	Binding and Defining SQL CHAR Data Types in OCI	14
7.3.5	Binding and Defining SQL NCHAR Data Types in OCI	15
7.3.6	Handling SQL NCHAR String Literals in OCI	16
7.3.7	Binding and Defining CLOB and NCLOB Unicode Data in OCI	17
7.4	Pro*C/C++ Programming with Unicode	17
7.4.1	Pro*C/C++ Data Conversion in Unicode	18
7.4.2	Using the VARCHAR Data Type in Pro*C/C++	18
7.4.3	Using the NVARCHAR Data Type in Pro*C/C++	19
7.4.4	Using the UVARCHAR Data Type in Pro*C/C++	19
7.5	JDBC Programming with Unicode	20
7.5.1	Binding and Defining Java Strings to SQL CHAR Data Types	20
7.5.2	Binding and Defining Java Strings to SQL NCHAR Data Types	21
7.5.2.1	New JDBC4.0 Methods for NCHAR Data Types	22
7.5.3	Using the SQL NCHAR Data Types Without Changing the Code	22
7.5.4	Using SQL NCHAR String Literals in JDBC	23
7.5.5	Data Conversion in JDBC	23
7.5.5.1	Data Conversion for the OCI Driver	24
7.5.5.2	Data Conversion for Thin Drivers	24
7.5.5.3	Data Conversion for the Server-Side Internal Driver	25
7.5.6	Using oracle.sql.CHAR in Oracle Object Types	25
7.5.6.1	oracle.sql.CHAR	25
7.5.6.2	Accessing SQL CHAR and NCHAR Attributes with oracle.sql.CHAR	26
7.5.7	Restrictions on Accessing SQL CHAR Data with JDBC	27
7.5.7.1	Character Integrity Issues in a Multibyte Database Environment	27
7.6	ODBC and OLE DB Programming with Unicode	28
7.6.1	Unicode-Enabled Drivers in ODBC and OLE DB	28
7.6.2	OCI Dependency in Unicode	28
7.6.3	ODBC and OLE DB Code Conversion in Unicode	29
7.6.3.1	OLE DB Code Conversions	30
7.6.4	ODBC Unicode Data Types	30
7.6.5	OLE DB Unicode Data Types	31
7.6.6	ADO Access	31
7.7	XML Programming with Unicode	32
7.7.1	Writing an XML File in Unicode with Java	32
7.7.2	Reading an XML File in Unicode with Java	33
7.7.3	Parsing an XML Stream in Unicode with Java	33

8 Oracle Globalization Development Kit

8.1	Overview of the Oracle Globalization Development Kit	1
-----	--	---

8.2	Designing a Global Internet Application	1
8.2.1	Deploying a Monolingual Internet Application	2
8.2.2	Deploying a Multilingual Internet Application	4
8.3	Developing a Global Internet Application	5
8.3.1	Locale Determination	6
8.3.2	Locale Awareness	6
8.3.3	Localizing the Content	7
8.4	Getting Started with the Globalization Development Kit	7
8.5	GDK Quick Start	9
8.5.1	Modifying the HelloWorld Application	10
8.6	GDK Application Framework for J2EE	16
8.6.1	Making the GDK Framework Available to J2EE Applications	18
8.6.2	Integrating Locale Sources into the GDK Framework	19
8.6.3	Getting the User Locale From the GDK Framework	20
8.6.4	Implementing Locale Awareness Using the GDK Localizer	22
8.6.5	Defining the Supported Application Locales in the GDK	23
8.6.6	Handling Non-ASCII Input and Output in the GDK Framework	24
8.6.7	Managing Localized Content in the GDK	26
8.6.7.1	Managing Localized Content in JSPs and Java Servlets	26
8.6.7.2	Managing Localized Content in Static Files	27
8.7	GDK Java API	28
8.7.1	Oracle Locale Information in the GDK	28
8.7.2	Oracle Locale Mapping in the GDK	29
8.7.3	Oracle Character Set Conversion in the GDK	30
8.7.4	Oracle Date, Number, and Monetary Formats in the GDK	31
8.7.5	Oracle Binary and Linguistic Sorts in the GDK	31
8.7.6	Oracle Language and Character Set Detection in the GDK	33
8.7.7	Oracle Translated Locale and Time Zone Names in the GDK	34
8.7.8	Using the GDK with E-Mail Programs	34
8.8	The GDK Application Configuration File	36
8.8.1	locale-charset-maps	36
8.8.2	page-charset	37
8.8.3	application-locales	37
8.8.4	locale-determine-rule	38
8.8.5	locale-parameter-name	39
8.8.6	message-bundles	39
8.8.7	url-rewrite-rule	40
8.8.8	Example: GDK Application Configuration File	41
8.9	GDK for Java Supplied Packages and Classes	42
8.9.1	oracle.i18n.lcsd	42
8.9.1.1	LCSScan	43
8.9.2	oracle.i18n.net	44

8.9.3	oracle.i18n.servlet	44
8.9.4	oracle.i18n.text	44
8.9.5	oracle.i18n.util	44
8.10	GDK for PL/SQL Supplied Packages	45
8.11	GDK Error Messages	46

9 SQL and PL/SQL Programming in a Global Environment

9.1	Locale-Dependent SQL Functions with Optional NLS Parameters	1
9.1.1	Default Values for NLS Parameters in SQL Functions	2
9.1.2	Specifying NLS Parameters in SQL Functions	2
9.1.3	Unacceptable NLS Parameters in SQL Functions	4
9.2	Other Locale-Dependent SQL Functions	4
9.2.1	The CONVERT Function	4
9.2.2	SQL Functions for Different Length Semantics	5
9.2.3	LIKE Conditions for Different Length Semantics	7
9.2.4	Character Set SQL Functions	7
9.2.4.1	Converting from Character Set Number to Character Set Name	8
9.2.4.2	Converting from Character Set Name to Character Set Number	8
9.2.4.3	Returning the Length of an NCHAR Column	8
9.2.5	The NLSSORT Function	8
9.2.5.1	NLSSORT Syntax	10
9.2.5.2	Comparing Strings in a WHERE Clause	10
9.2.5.3	Controlling an ORDER BY Clause	11
9.3	Miscellaneous Topics for SQL and PL/SQL Programming in a Global Environment	12
9.3.1	SQL Date Format Masks	12
9.3.2	Calculating Week Numbers	12
9.3.3	SQL Numeric Format Masks	13
9.3.4	Loading External BFILE Data into LOB Columns	13

10 OCI Programming in a Global Environment

10.1	Using the OCI NLS Functions	1
10.2	Specifying Character Sets in OCI	1
10.3	Getting Locale Information in OCI	2
10.4	Mapping Locale Information Between Oracle and Other Standards	3
10.5	Manipulating Strings in OCI	3
10.6	Classifying Characters in OCI	5
10.7	Converting Character Sets in OCI	6
10.8	OCI Messaging Functions	6
10.9	Imsgen Utility	7

11 Character Set Migration

11.1	Overview of Character Set Migration	1
11.1.1	Data Truncation	2
11.1.1.1	Additional Problems Caused by Data Truncation	2
11.1.2	Character Set Conversion Issues	3
11.1.2.1	Replacement Characters that Result from Using the Export and Import Utilities	3
11.1.2.2	Invalid Data That Results from Setting the Client's NLS_LANG Parameter Incorrectly	4
11.1.2.3	Conversion from Single-byte to Multibyte Character Set and Oracle Data Pump	6
11.2	Changing the Database Character Set of an Existing Database	6
11.2.1	Migrating Character Data Using the Database Migration Assistant for Unicode	7
11.2.2	Migrating Character Data Using a Full Export and Import	8
11.3	Repairing Database Character Set Metadata	8
11.3.1	Example: Using CSREPAIR	9
11.4	The Language and Character Set File Scanner	9
11.4.1	Syntax of the LCSSCAN Command	10
11.4.2	Examples: Using the LCSSCAN Command	11
11.4.3	Getting Command-Line Help for the Language and Character Set File Scanner	12
11.4.4	Supported Languages and Character Sets	12
11.4.5	LCSSCAN Error Messages	13

12 EBCDIC Emulation for Mainframe Migration

12.1	Overview of Application Migration from a Mainframe	1
12.1.1	Character Set Differences	1
12.1.2	Migration Strategies	3
12.1.3	Avoiding Dependency on Character Encoding	4
12.2	Choosing a Character Set for the Target Database	4
12.2.1	Requirements for Target Character Sets	4
12.2.2	Migrating to AL32UTF8	5
12.2.3	Migrating to Legacy ASCII-Based Character Sets	5
12.3	Choosing a Source Character Set for Data Migration	6
12.3.1	CDRA-Compatible Character Sets	6
12.3.2	SQL*Loader	6
12.3.3	External Tables	7
12.3.4	DBMS_LOB	7
12.3.5	Character Set Conversion Functions	9
12.3.6	Undefined or Duplicated Codes in a Source Character Set	10
12.4	Emulating EBCDIC Binary Ordering	11
12.4.1	Dependency on Binary Ordering	11

12.4.2	Emulated Binary Collations	12
12.4.3	Deploying Emulated Binary Collations	12

13 Customizing Locale Data

13.1	Overview of the Oracle Locale Builder Utility	1
13.1.1	Configuring Unicode Fonts for the Oracle Locale Builder	1
13.1.2	The Oracle Locale Builder User Interface	2
13.1.3	Oracle Locale Builder Pages and Dialog Boxes	3
13.1.3.1	Existing Definitions Dialog Box	3
13.1.3.2	Session Log Dialog Box	4
13.1.3.3	Preview NLT Tab Page	5
13.1.3.4	Open File Dialog Box	5
13.2	Creating a New Language Definition with Oracle Locale Builder	6
13.3	Creating a New Territory Definition with the Oracle Locale Builder	10
13.4	Displaying a Code Chart with the Oracle Locale Builder	16
13.5	Creating a New Character Set Definition with the Oracle Locale Builder	20
13.5.1	Character Sets with User-Defined Characters	21
13.5.2	Oracle Database Character Set Conversion Architecture	21
13.5.3	Unicode Private Use Area	22
13.5.4	User-Defined Character Cross-References Between Character Sets	22
13.5.5	Guidelines for Creating a New Character Set from an Existing Character Set	23
13.5.6	Example: Creating a New Character Set Definition with the Oracle Locale Builder	24
13.6	Creating a New Linguistic Sort with the Oracle Locale Builder	27
13.6.1	Changing the Sort Order for All Characters with the Same Diacritic	31
13.6.2	Changing the Sort Order for One Character with a Diacritic	34
13.7	Generating and Installing NLB Files	36
13.8	Upgrading Custom NLB Files from Previous Releases of Oracle Database	38
13.9	Deploying Custom NLB Files to Oracle Installations on the Same Platform	38
13.10	Deploying Custom NLB Files to Oracle Installations on Another Platform	39
13.11	Adding Custom Locale Definitions to Java Components with the GINSTALL Utility	39
13.12	Customizing Calendars with the NLS Calendar Utility	40

A Locale Data

A.1	Languages	A-1
A.2	Translated Messages	A-4
A.3	Territories	A-5
A.4	Character Sets	A-7
A.4.1	Recommended Database Character Sets	A-7
A.4.2	Other Character Sets	A-9

A.4.3	Character Sets that Support the Euro Symbol	A-14
A.4.4	Client-Only Character Sets	A-15
A.4.5	Universal Character Sets	A-18
A.4.6	Character Set Conversion Support	A-18
A.4.7	Binary Subset-Superset Pairs	A-19
A.4.8	Mapping EBCDIC to ASCII Character Sets	A-21
A.5	Language and Character Set Detection Support	A-22
A.6	Linguistic Collations	A-24
A.7	Calendar Systems	A-29
A.8	Time Zone Region Names	A-30
A.9	Obsolete Locale Data	A-38
A.9.1	Obsolete Linguistic Sorts	A-38
A.9.2	Obsolete Territories	A-39
A.9.3	Obsolete Languages	A-39
A.9.4	Obsolete Character Sets and Replacement Character Sets	A-39
A.9.5	Updates to the Oracle Database Language and Territory Definition Files	A-40
A.10	Desupported Locale Data	A-41
A.10.1	Desupported Linguistic Sorts	A-41
A.10.2	Desupported Character Sets and Replacement Character Sets	A-42
A.10.3	AL24UTFFSS Character Set Desupported	A-42

B Unicode Character Code Assignments

B.1	Unicode Code Ranges	B-1
B.2	UTF-16 Encoding	B-2
B.3	UTF-8 Encoding	B-2

C Collation Derivation and Determination Rules for SQL Operations

C.1	Collation Derivation	C-1
C.2	Collation Determination	C-4
C.3	SQL Operations and Their Derivation- and Determination-relevant Arguments	C-6

Glossary

Index

Preface

This book describes Oracle globalization support for Oracle AI Database. It explains how to set up a globalization support environment, choose and migrate a character set, customize locale data, do linguistic sorting, program in a global environment, and program with Unicode.

This preface contains these topics:

- [Intended Audience](#)
- [Documentation Accessibility](#)
- [Diversity and Inclusion](#)
- [Related Documentation](#)
- [Conventions](#)

Intended Audience

Oracle AI Database Globalization Support Guide is intended for database administrators, system administrators, and database application developers who perform the following tasks:

- Set up a globalization support environment
- Choose, analyze, or migrate character sets
- Sort data linguistically
- Customize locale data
- Write programs in a global environment
- Use Unicode

To use this document, you must be familiar with relational database concepts, basic Oracle AI Database concepts, and the operating system environment under which you are running Oracle.

Documentation Accessibility

For information about Oracle's commitment to accessibility, visit the Oracle Accessibility Program website at <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=docacc>.

Access to Oracle Support

Oracle customer access to and use of Oracle support services will be pursuant to the terms and conditions specified in their Oracle order for the applicable services.

Diversity and Inclusion

Oracle is fully committed to diversity and inclusion. Oracle respects and values having a diverse workforce that increases thought leadership and innovation. As part of our initiative to build a more inclusive culture that positively impacts our employees, customers, and partners, we are working to remove insensitive terms from our products and documentation. We are also mindful of the necessity to maintain compatibility with our customers' existing technologies and the need to ensure continuity of service as Oracle's offerings and industry standards evolve. Because of these technical constraints, our effort to remove insensitive terms is ongoing and will take time and external cooperation.

Related Documentation

Many of the examples in this book use the sample schemas of the seed database, which is installed by default when you install Oracle. Refer to *Oracle AI Database Sample Schemas* for information on how these schemas were created and how you can use them yourself.

Conventions

The following text conventions are used in this document:

Convention	Meaning
boldface	Boldface type indicates graphical user interface elements associated with an action, or terms defined in text or the glossary.
<i>italic</i>	Italic type indicates book titles, emphasis, or placeholder variables for which you supply particular values.
monospace	Monospace type indicates commands within a paragraph, URLs, code in examples, text that appears on the screen, or text that you enter.

Changes in Oracle AI Database Globalization Support Guide for Oracle AI Database 26ai

The following are changes in *Oracle AI Database Globalization Support Guide* for Oracle AI Database 26ai.

New Features

- [New Features in Oracle AI Database 26ai, Release Update 23.26.3](#)
- [New Features in Oracle AI Database 26ai, Release Update 23.26.2](#)
- [New Features in Oracle AI Database 26ai](#)

New Features in Oracle AI Database 26ai, Release Update 23.26.3

- Support for Unicode 17.0, keeping Oracle Database current with the latest Unicode Standard and expanding multilingual coverage for newly encoded scripts, characters, and symbols.

Unicode 17.0 includes a new currency symbol, the SAUDI RIYAL SIGN (Unicode code point U+20C1), that the Saudi Central Bank introduced in 2025 to represent the currency of Saudi Arabia. The new symbol can be used by configuring the NLS_CURRENCY parameter through the client environment, database session, application-specific logon configuration, or individual SQL formatting functions. To display the symbol correctly, the client character set and application font must support Unicode code point U+20C1.

Saudi Riyal Symbol



See "[Unicode Support](#)".

New Features in Oracle AI Database 26ai, Release Update 23.26.2

- A set of new EBCDIC client-only character sets whose conversion to Unicode strictly follows mappings defined by the IBM Character Data Representation Architecture (CDRA).

See "[Choosing a Source Character Set for Data Migration](#)".

- A set of collations that emulate the binary ordering of the new EBCDIC client-only character sets.

See "[Emulating EBCDIC Binary Ordering](#)".

New Features in Oracle AI Database 26ai

- Support for Unicode 15.0, a major version of the Unicode Standard that supersedes all its previous versions.
See "[Unicode Support](#)".
- Enhanced Ideographic Variation Sequence (IVS) support:
 - A new UCA collation UCA1210_JAPANESE_IVS provides IVS support when processing Japanese text.
See "[The UCA Japanese IVS Collation](#)".
 - String operations that use Unicode composed characters now count an IVS as one complete character. These operations include the SQL functions INSTRC, LENGTHC, and SUBSTRC, and the SQL condition LIKEC.
See "[SQL Functions for Different Length Semantics](#)" and [LIKE Conditions for Different Length Semantics](#).
- A new territory TURKIYE, which supports the official name of the country of Türkiye.
See "[Territories](#)".
- A new linguistic sort ICELANDIC_IST, which implements the Icelandic Standard IST 130:2004.
See "[Linguistic Collations](#)".
- The implementation of the Oracle client character set ZHS32GB18030 is updated to support the latest GB18030-2022 standard.
See "[Client-Only Character Sets](#)".
- The UTL_I18N package provides three new procedures that detect the most likely Oracle character sets and Oracle languages for a given data sample.
See DETECT_CHARSET, DETECT_LANGUAGE, and DETECT_LANGUAGE_CHARSET in *Oracle AI Database PL/SQL Packages and Types Reference*.
- The process of upgrading time zone data using the DBMS_DST package has been enhanced as follows:
 - The upgrade is optimized to take the actual data contents of a table into account. Only tables impacted by a daylight saving time rule change will undergo a data change. See "[Upgrade Window](#)".
 - You can monitor the actions performed on tables during the upgrade process by viewing the upgrade history table. See "[About the Upgrade History Table](#)".
- The new TIME_AT_DBTIMEZONE initialization parameter allows you to specify whether time-dependent database operations use the time zone of the database host system or the time zone of the PDB.
See *Oracle AI Database Reference*.

1

Overview of Globalization Support

This chapter provides an overview of globalization support for Oracle AI Database. This chapter discusses the following topics:

- [Globalization Support Architecture](#)
- [Globalization Support Features](#)

1.1 Globalization Support Architecture

The globalization support in Oracle AI Database enables you to store, process, and retrieve data in native languages. It ensures that database utilities, error messages, sort order, and date, time, monetary, numeric, and calendar conventions automatically adapt to any native language and locale.

In the past, Oracle referred to globalization support capabilities as National Language Support (NLS) features. NLS is actually a subset of globalization support. NLS is the ability to choose a national language and store data in a specific character set. Globalization support enables you to develop multilingual applications and software products that can be accessed and run from anywhere in the world simultaneously. An application can render content of the user interface and process data in the native users' languages and locale preferences.

1.1.1 Locale Data on Demand

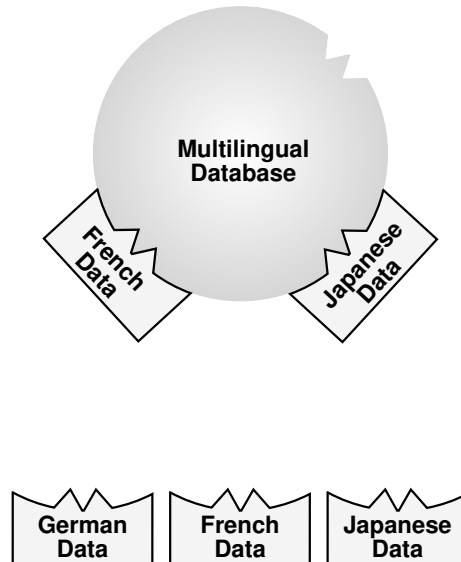
Oracle AI Database globalization support is implemented with the Oracle NLS Runtime Library (NLSRTL). NLSRTL provides a comprehensive suite of language-independent functions that perform proper text and character processing and language-convention manipulations. Behavior of these functions for a specific language and territory is governed by a set of locale-specific data that is identified and loaded at run time.

The locale-specific data is structured as independent sets of data for each locale that Oracle AI supports. The data for a particular locale can be loaded independently of other locale data.

The advantages of this design are as follows:

- You can manage memory consumption by choosing the set of locales that you need.
- You can add and customize locale data for a specific locale without affecting other locales.

The following figure shows how locale-specific data is loaded at run time. In this example, French data and Japanese data are loaded into the multilingual database, but German data is not.

Figure 1-1 Loading Locale-Specific Data to the Database

The locale-specific data is stored in the \$ORACLE_HOME/nls/data directory. The `ORA_NLS10` environment variable should be defined only when you need to change the default directory location for the locale-specific data files, for example, when the system has multiple Oracle AI Database homes that share a single copy of the locale-specific data files.

A boot file is used to determine the availability of the NLS objects that can be loaded. Oracle AI Database supports both system and user boot files. The user boot file gives you the flexibility to tailor what NLS locale objects are available for the database. Also, new locale data can be added and some locale data components can be customized.

① See Also

["Customizing Locale Data"](#)

1.1.2 Architecture to Support Multilingual Applications

Oracle AI Database enables multitier applications and client/server applications to support languages for which the database is configured.

The locale-dependent operations are controlled by several parameters and environment variables on both the client and the database server. On the database server, each session that is started on behalf of a client may run in the same or a different locale as other sessions, and can have the same or different language requirements specified.

Oracle AI Database has a set of session-independent NLS parameters that are specified when you create a database. Two of the parameters specify the database character set and the national character set, which is an alternative Unicode character set that can be specified for `NCHAR`, `NVARCHAR2`, and `NCLOB` data. The parameters specify the character set that is used to store text data in the database. Other parameters, such as language and territory, are used to evaluate and check constraints.

If the client session and the database server specify different character sets, then the database converts character set strings automatically.

From a globalization support perspective, all applications are considered to be clients, even if they run on the same physical machine as the Oracle AI Database instance. For example, when SQL*Plus is started by the UNIX user who owns the Oracle AI Database software from the Oracle home in which the RDBMS software is installed, and SQL*Plus connects to the database through an adapter by specifying the ORACLE_SID parameter, SQL*Plus is considered a client. Its behavior is ruled by client-side NLS parameters.

Another example of an application being considered a client occurs when the middle tier is an application server. The different sessions spawned by the application server are considered to be separate client sessions.

When a client application is started, it initializes the client NLS environment from environment settings. All NLS operations performed locally are executed using these settings. Examples of local NLS operations are:

- Display formatting in Oracle Developer applications
- User OCI code that executes NLS OCI functions with OCI environment handles

When the application connects to a database, a session is created on the server. The new session initializes its NLS environment from NLS instance parameters specified in the initialization parameter file. These settings can be subsequently changed by an ALTER SESSION statement. The statement changes only the session NLS environment. It does not change the local client NLS environment. The session NLS settings are used to process SQL and PL/SQL statements that are executed on the server. For example, use an ALTER SESSION statement to set the NLS_LANGUAGE initialization parameter to Italian:

```
ALTER SESSION SET NLS_LANGUAGE=Italian;
```

Enter a SELECT statement:

```
SQL> SELECT last_name, hire_date, ROUND(salary/8,2) salary FROM employees;
```

You should see results similar to the following:

LAST_NAME	HIRE_DATE	SALARY
...		
Sciarra	30-SET-05	962.5
Urman	07-MAR-06	975
Popp	07-DIC-07	862.5
...		

Note that the month name abbreviations are in Italian.

Immediately after the connection has been established, if the NLS_LANG environment setting is defined on the client side, then an implicit ALTER SESSION statement synchronizes the client and session NLS environments.

① See Also

- ["OCI Programming in a Global Environment"](#)
- ["Setting Up a Globalization Support Environment"](#)

1.1.3 Using Unicode in a Multilingual Database

Unicode, the universal encoded character set, enables you to store information in any language by using a single character set. Unicode provides a unique code value for every character, regardless of the platform, program, or language. Oracle recommends using AL32UTF8 as the database character set. AL32UTF8 is the proper implementation of the UTF-8 encoding form of the Unicode standard.

① Note

Starting with Oracle Database 12c Release 2, if you use Oracle Universal Installer (OUI) or Oracle Database Configuration Assistant (DBCA) to create a database, the default database character set used is the Unicode character set AL32UTF8.

Unicode has the following advantages:

- Simplifies character set conversion and linguistic sort functions.
- Improves performance compared with native multibyte character sets.
- Supports the Unicode data type based on the Unicode standard.

To help you migrate to a Unicode environment, Oracle provides the Database Migration Assistant for Unicode (DMU). The DMU is an intuitive and user-friendly GUI that helps streamline the migration process through an interface that minimizes the workload and ensures that all migration issues are addressed, along with guaranteeing that the data conversion is carried out correctly and efficiently. The DMU offers many advantages over past methods of migrating data, some of which are:

- It guides you through the workflow.
- It offers suggestions for handling certain problems, such as failures during the cleansing of the data.
- It supports selective conversion of data.
- It offers progress monitoring.

① See Also

- ["Supporting Multilingual Databases with Unicode"](#)
- ["Programming with Unicode"](#)
- ["Enabling Multilingual Support with Unicode Data Types"](#)
- *Oracle Database Migration Assistant for Unicode Guide*

1.2 Globalization Support Features

This section provides an overview of the standard globalization features in Oracle AI Database:

- [Language Support](#)
- [Territory Support](#)

- [Date and Time Formats](#)
- [Monetary and Numeric Formats](#)
- [Calendar Systems](#)
- [Linguistic Sorting](#)
- [Character Set Support](#)
- [Character Semantics](#)
- [Customization of Locale and Calendar Data](#)
- [Unicode Support](#)

1.2.1 Language Support

Oracle AI Database enables you to store, process, and retrieve data in native languages. The languages that can be stored in a database are all languages written in scripts that are encoded by Oracle-supported character sets. Through the use of Unicode databases and data types, Oracle AI Database supports most contemporary languages.

Additional support is available for a subset of the languages. The database can, for example, display dates using translated month names, and can sort text data according to cultural conventions.

When this document uses the term *language support*, it refers to the additional language-dependent functionality, and not to the ability to store text of a specific language. For example, language support includes displaying dates or sorting text according to specific locales and cultural conventions. Additionally, for some supported languages, Oracle AI Database provides translated error messages and a translated user interface for the database utilities.

📘 See Also

- "[Setting Up a Globalization Support Environment](#)"
- "[Languages](#)" for the list of Oracle AI Database language names and abbreviations
- "[Translated Messages](#)" for the list of languages into which Oracle AI Database messages are translated

1.2.2 Territory Support

Oracle AI Database supports cultural conventions that are specific to geographical locations. The default local time format, date format, and numeric and monetary conventions depend on the local territory setting. Setting different NLS parameters enables the database session to use different cultural settings. For example, you can set the euro (EUR) as the primary currency and the Japanese yen (JPY) as the secondary currency for a given database session, even when the territory is defined as AMERICA.

📘 See Also

- "[Setting Up a Globalization Support Environment](#)"
- "[Territories](#)" for a list of territories that are supported by Oracle AI Database

1.2.3 Date and Time Formats

Different conventions for displaying the hour, day, month, and year can be handled in local formats. For example, in the United Kingdom, the date is displayed using the DD-MON-YYYY format, while Japan commonly uses the YYYY-MM-DD format.

Time zones and daylight saving support are also available.

See Also

- ["Setting Up a Globalization Support Environment"](#)
- ["Datetime Data Types and Time Zone Support"](#)
- *Oracle AI Database SQL Language Reference*

1.2.4 Monetary and Numeric Formats

Currency, credit, and debit symbols can be represented in local formats. Radix symbols and thousands separators can be defined by locales. For example, in the US, the decimal point is a dot (.), while it is a comma (,) in France. Therefore, the amount \$1,234 has different meanings in different countries.

See Also

- ["Setting Up a Globalization Support Environment"](#)

1.2.5 Calendar Systems

Many different calendar systems are in use around the world. Oracle AI Database supports eight different calendar systems:

- Arabic Hijrah
- English Hijrah
- Ethiopian
- Gregorian
- Japanese Imperial
- Persian
- ROC Official (Republic of China)
- Thai Buddha

See Also

- ["Setting Up a Globalization Support Environment"](#)
- ["Calendar Systems"](#) for more information about supported calendars

1.2.6 Linguistic Sorting

Oracle AI Database provides linguistic definitions for culturally accurate sorting and case conversion. The basic definition treats strings as sequences of independent characters. The extended definition recognizes pairs of characters that should be treated as special cases.

Strings that are converted to upper case or lower case using the basic definition always retain their lengths. Strings converted using the extended definition may become longer or shorter.

① See Also

["Linguistic Sorting and Matching"](#)

1.2.7 Character Set Support

Oracle AI Database supports a large number of single-byte, multibyte, and fixed-width encoding schemes that are based on national, international, and vendor-specific standards.

① See Also

- ["Choosing a Character Set"](#)
- ["Character Sets"](#) for a list of supported character sets

1.2.8 Character Semantics

Oracle AI Database provides character semantics. It is useful for defining the storage requirements for multibyte strings of varying widths in terms of characters instead of bytes.

① See Also

["Length Semantics"](#)

1.2.9 Customization of Locale and Calendar Data

You can customize locale data such as language, character set, territory, or linguistic sort using the Oracle Locale Builder.

You can customize calendars with the NLS Calendar Utility.

① See Also

- ["Customizing Locale Data"](#)
- ["Customizing Calendars with the NLS Calendar Utility"](#)

1.2.10 Unicode Support

Unicode is an industry standard that enables text and symbols from all languages to be consistently represented and manipulated by computers.

Oracle Database has supported the Unicode standard since Oracle 7. Subsequently, Oracle Database 12c Release 2 (12.2) supports Unicode 7.0. Oracle Database 18c and Oracle Database 19c support Unicode 9.0. Oracle Database 21c supports Unicode 12.1. Oracle AI Database 26ai, RU 23.26.2 and earlier, supports Unicode 15.0. Oracle AI Database 26ai, RU 23.26.3 and later, supports Unicode 17.0.

You can store Unicode characters in an Oracle database in two ways:

- You can create a Unicode database that enables you to store UTF-8 encoded characters as SQL CHAR data types VARCHAR2, CHAR, LONG (deprecated), and CLOB.
- You can support multilingual data in specific columns by using SQL NCHAR data types NVARCHAR2, NCHAR, and NCLOB. You can store Unicode characters into columns of the NCHAR data types regardless of how the database character set has been defined. The NCHAR data types are exclusively Unicode data types.

Note

Starting with Oracle Database 12c Release 2 (12.2), if you use Oracle Universal Installer (OUI) or Oracle Database Configuration Assistant (DBCA) to create a database, then the default database character set used is the Unicode character set AL32UTF8.

See Also

"[Supporting Multilingual Databases with Unicode](#)"

2

Choosing a Character Set

This chapter explains how to choose a character set. The following topics are included:

- [Character Set Encoding](#)
- [Length Semantics](#)
- [Choosing an Oracle Database Character Set](#)
- [Choosing a Database Character Set for a Multitenant Container Database](#)
- [Changing the Character Set After Database Creation](#)
- [Monolingual Database Scenario](#)
- [Multilingual Database Scenario](#)

2.1 Character Set Encoding

When computer systems process characters, they use numeric codes instead of the graphical representation of the character. For example, when the database stores the letter A, it actually stores a numeric code that the computer system interprets as the letter. These numeric codes are especially important in a global environment because of the potential need to convert data between different character sets.

This section discusses the following topics:

- [What is an Encoded Character Set?](#)
- [Which Characters Are Encoded?](#)
- [What Characters Does a Character Set Support?](#)
- [How are Characters Encoded?](#)
- [Naming Convention for Oracle Database Character Sets](#)
- [Subsets and Supersets](#)

2.1.1 What is an Encoded Character Set?

You specify an encoded character set when you create a database. Choosing a character set determines what languages can be represented in the database. It also affects:

- How you create the database schema
- How you develop applications that process character data
- How the database works with the operating system
- Database performance
- Storage required for storing character data

A group of characters (for example, alphabetic characters, ideographs, symbols, punctuation marks, and control characters) can be encoded as a character set. An *encoded character set* assigns a unique numeric code to each character in the character set. The numeric codes are

called *code points* or *encoded values*. The following table shows examples of characters that have been assigned a hexadecimal code value in the ASCII character set.

Table 2-1 Encoded Characters in the ASCII Character Set

Character	Description	Hexadecimal Code Value
!	Exclamation Mark	21
#	Number Sign	23
\$	Dollar Sign	24
1	Number 1	31
2	Number 2	32
3	Number 3	33
A	Uppercase A	41
B	Uppercase B	42
C	Uppercase C	43
a	Lowercase a	61
b	Lowercase b	62
c	Lowercase c	63

The computer industry uses many encoded character sets. Character sets differ in the following ways:

- The number of characters available to be used in the set
- The characters that are available to be used in the set (also known as the *character repertoire*)
- The scripts used for writing and the languages that they represent
- The code points or values assigned to each character
- The encoding scheme used to represent a specific character

Oracle Database supports most national, international, and vendor-specific encoded character set standards.

 See Also

"[Character Sets](#)" for a complete list of character sets that are supported by Oracle Database

2.1.2 Which Characters Are Encoded?

The characters that are encoded in a character set depend on the writing systems that are represented. A writing system can be used to represent a language or a group of languages. Writing systems can be classified into two categories:

- [Phonetic Writing Systems](#)
- [Ideographic Writing Systems](#)

This section also includes the following topics:

- [Punctuation, Control Characters, Numbers, and Symbols](#)
- [Writing Direction](#)

2.1.2.1 Phonetic Writing Systems

Phonetic writing systems consist of symbols that represent different sounds associated with a language. Greek, Latin, Cyrillic, and Devanagari are all examples of phonetic writing systems based on alphabets. Note that alphabets can represent multiple languages. For example, the Latin alphabet can represent many Western European languages such as French, German, and English.

Characters associated with a phonetic writing system can typically be encoded in one byte because the character repertoire is usually smaller than 256 characters.

2.1.2.2 Ideographic Writing Systems

Ideographic writing systems consist of ideographs or pictographs that represent the meaning of a word, not the sounds of a language. Chinese and Japanese are examples of ideographic writing systems that are based on tens of thousands of ideographs. Languages that use ideographic writing systems may also use a **syllabary**. Syllabaries provide a mechanism for communicating additional phonetic information. For instance, Japanese has two syllabaries: Hiragana, normally used for grammatical elements, and Katakana, normally used for foreign and onomatopoeic words.

Characters associated with an ideographic writing system typically are encoded in more than one byte because the character repertoire has tens of thousands of characters.

2.1.2.3 Punctuation, Control Characters, Numbers, and Symbols

In addition to encoding the script of a language, other special characters must be encoded:

- Punctuation marks such as commas, periods, and apostrophes
- Numbers
- Special symbols such as currency symbols and math operators
- Control characters such as carriage returns and tabs

2.1.2.4 Writing Direction

Most Western languages are written left to right from the top to the bottom of the page. East Asian languages are usually written top to bottom from the right to the left of the page, although exceptions are frequently made for technical books translated from Western languages. Arabic and Hebrew are written right to left from the top to the bottom.

Numbers reverse direction in Arabic and Hebrew. Although the text is written right to left, numbers within the sentence are written left to right. For example, "I wrote 32 books" would be written as "skoob 32 etorw I". Regardless of the writing direction, Oracle databases store the data in logical order. Logical order means the order that is used by someone typing a language, not how it looks on the screen.

Writing direction does not affect the encoding of a character.

2.1.3 What Characters Does a Character Set Support?

Different character sets support different character repertoires. Because character sets are typically based on a particular writing script, they can support multiple languages. When character sets were first developed, they had a limited character repertoire. Even now there can be problems using certain characters across platforms. The following CHAR and VARCHAR characters are represented in all Oracle database character sets and can be transported to any platform:

- Uppercase and lowercase English characters A through Z and a through z
- Arabic digits 0 through 9
- The following punctuation marks: % ' ' () * + - , . / \ : ; < > = ! _ & ~ { } | ^ ? \$ # @ " ' []
- The following control characters: space, horizontal tab, vertical tab, form feed

If you are using characters outside this set, then take care that your data is supported in the database character set that you have chosen.

At all times, a sequence of bytes comprising a character value—for example, an interactive (keyboard) input to a client program, content of a bind variable buffer, or content of a table column—must be valid in the character set declared for this value. For performance reasons, the database rarely validates provided character data automatically. If a byte sequence is not a valid representation of a string of characters in the corresponding declared character set, the behavior of Oracle software—client APIs, database server, and Oracle utilities—is explicitly undefined. When such a byte sequence is transferred or processed, Oracle database software may, for example, transfer the sequence unmodified, replace bytes that cannot be interpreted with default replacement characters, truncate trailing bytes that do not form a valid character code, or report an error such as "ORA-29275: partial multibyte character". Oracle reserves the right to change this behavior in a future release.

The applicable character set declaration for a character value depends on the given value, as described in the relevant documentation. For example, the input for a client program, such as SQL*Plus, should be in the character set specified in the NLS_LANG variable. The character set for a bind variable in an OCI program may be declared through a bind variable attribute OCI_ATTR_CHARSET_ID, the charset parameter of OCINlsEnvCreate(), or in the NLS_LANG variable, in descending priority. Other client APIs, such as JDBC, may assume all client-side character values to be in Unicode (UTF-8 or UTF-16). The character set of data stored in a database column is either the database character set or the national (NCHAR) character set, depending on the column's data type.

The agreement between the real and declared encodings of a character value is crucial for proper processing of the value, whether in character set conversion or in processing by character functions, such as SUBSTR, UPPER, REGEXP, or ORDER BY.

① See Also

- ["Supporting Multilingual Databases with Unicode"](#)
- *Oracle AI Database SQL Language Reference* for more information about the CONVERT SQL functions
- *Oracle AI Database SQL Language Reference* for more information about the CHR SQL functions
- ["Displaying a Code Chart with the Oracle Locale Builder"](#)

2.1.3.1 ASCII Encoding

[Table 2-2](#) shows how the ASCII character set is encoded. Row and column headings denote hexadecimal digits. To find the encoded value of a character, read the column number followed by the row number. For example, the code value of the character A is 0x41.

Table 2-2 7-Bit ASCII Character Set

-	0	1	2	3	4	5	6	7
0	NUL	DLE	SP	0	@	P	'	p
1	SOH	DC1	!	1	A	Q	a	q
2	STX	DC2	"	2	B	R	b	r
3	ETX	DC3	#	3	C	S	c	s
4	EOT	DC4	\$	4	D	T	d	t
5	ENQ	NAK	%	5	E	U	e	u
6	ACK	SYN	&	6	F	V	f	v
7	BEL	ETB	'	7	G	W	g	w
8	BS	CAN	(	8	H	X	h	x
9	TAB	EM	)	9	I	Y	i	y
A	LF	SUB	*	:	J	Z	j	z
B	VT	ESC	+	;	K	[	k	{
C	FF	FS	,	<	L	\	l	
D	CR	GS	-	=	M	]	m	}
E	SO	RS	.	>	N	^	n	~
F	SI	US	/	?	O	_	o	DEL

As languages evolve to meet the needs of people around the world, new character sets are created to support the languages. Typically, these new character sets support a group of related languages based on the same script. For example, the ISO 8859 character set series was created to support different European languages. [Table 2-3](#) shows the languages that are supported by the ISO 8859 character sets.

Table 2-3 ISO 8859 Character Sets

Standard	Languages Supported
ISO 8859-1	Western European (Albanian, Basque, Breton, Catalan, Danish, Dutch, English, Faeroese, Finnish, French, German, Greenlandic, Icelandic, Irish Gaelic, Italian, Latin, Luxembourgish, Norwegian, Portuguese, Rhaeto-Romanic, Scottish Gaelic, Spanish, Swedish)
ISO 8859-2	Eastern European (Albanian, Croatian, Czech, English, German, Hungarian, Latin, Polish, Romanian, Slovak, Slovenian, Serbian)
ISO 8859-3	Southeastern European (Afrikaans, Catalan, Dutch, English, Esperanto, German, Italian, Maltese, Spanish, Turkish)
ISO 8859-4	Northern European (Danish, English, Estonian, Finnish, German, Greenlandic, Latin, Latvian, Lithuanian, Norwegian, Sámi, Slovenian, Swedish)
ISO 8859-5	Eastern European (Cyrillic-based: Bulgarian, Byelorussian, Macedonian, Russian, Serbian, Ukrainian)

Table 2-3 (Cont.) ISO 8859 Character Sets

Standard	Languages Supported
ISO 8859-6	Arabic
ISO 8859-7	Greek
ISO 8859-8	Hebrew
ISO 8859-9	Western European (Albanian, Basque, Breton, Catalan, Cornish, Danish, Dutch, English, Finnish, French, Frisian, Galician, German, Greenlandic, Irish Gaelic, Italian, Latin, Luxemburgish, Norwegian, Portuguese, Rhaeto-Romanic, Scottish Gaelic, Spanish, Swedish, Turkish)
ISO 8859-10	Northern European (Danish, English, Estonian, Faeroese, Finnish, German, Greenlandic, Icelandic, Irish Gaelic, Latin, Lithuanian, Norwegian, Sámi, Slovenian, Swedish)
ISO 8859-13	Baltic Rim (English, Estonian, Finnish, Latin, Latvian, Norwegian)
ISO 8859-14	Celtic (Albanian, Basque, Breton, Catalan, Cornish, Danish, English, Galician, German, Greenlandic, Irish Gaelic, Italian, Latin, Luxemburgish, Manx Gaelic, Norwegian, Portuguese, Rhaeto-Romanic, Scottish Gaelic, Spanish, Swedish, Welsh)
ISO 8859-15	Western European (Albanian, Basque, Breton, Catalan, Danish, Dutch, English, Estonian, Faroese, Finnish, French, Frisian, Galician, German, Greenlandic, Icelandic, Irish Gaelic, Italian, Latin, Luxemburgish, Norwegian, Portuguese, Rhaeto-Romanic, Scottish Gaelic, Spanish, Swedish)

Historically, character sets have provided restricted multilingual support, which has been limited to groups of languages based on similar scripts. More recently, universal character sets have emerged to enable greatly improved solutions for multilingual support. Unicode is one such universal character set that encompasses most major scripts of the modern world.

① See Also

["Supporting Multilingual Databases with Unicode"](#)

2.1.4 How are Characters Encoded?

Different types of encoding schemes have been created by the computer industry. The character set you choose affects what kind of encoding scheme is used. This is important because different encoding schemes have different performance characteristics. These characteristics can influence your database schema and application development. The character set you choose uses one of the following types of encoding schemes:

- [Single-Byte Encoding Schemes](#)
- [Multibyte Encoding Schemes](#)

2.1.4.1 Single-Byte Encoding Schemes

Single-byte encoding schemes are efficient. They take up the least amount of space to represent characters and are easy to process and program with because one character can be represented in one byte. Single-byte encoding schemes are classified as one of the following types:

- 7-bit encoding schemes

Single-byte 7-bit encoding schemes can define up to 128 characters and normally support just one language. One of the most common single-byte character sets, used since the early days of computing, is ASCII (American Standard Code for Information Interchange).

- 8-bit encoding schemes

Single-byte 8-bit encoding schemes can define up to 256 characters and often support a group of related languages. One example is ISO 8859-1, which supports many Western European languages. The following figure shows the ISO 8859-1 8-bit encoding scheme.

Figure 2-1 ISO 8859-1 8-Bit Encoding Scheme

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
0	NUL	DLE	SP	0	@	P	`	p	NBSP	°	À	Ð	à	ð
1	SOH	DC1	!	1	A	Q	a	q	¡	±	Á	Ñ	á	ñ
2	STX	DC2	“	2	B	R	b	r	¢	²	Â	Ò	â	ò
3	ETX	DC3	#	3	C	S	c	s	£	³	Ã	Ó	ã	ó
4	EOT	DC4	\$	4	D	T	d	t	¤	´	Ä	Ô	ä	ô
5	ENQ	NAK	%	5	E	U	e	u	¥	µ	Å	Ö	å	ö
6	ACK	SYN	&	6	F	V	f	v	¦	¶	Æ	Ø	æ	ø
7	BEL	ETB	'	7	G	W	g	w	§	·	Ç	×	ç	÷
8	BS	CAN	(	8	H	X	h	x	“	¸	È	Ø	è	ø
9	HT	EM	)	9	I	Y	i	y	©	¹	É	Ù	é	ù
A	NL	SUB	*	:	J	Z	j	z	ª	º	Ê	Ú	ê	ú
B	VT	ESC	+	;	K	[	k	{	«	»	Ë	Û	ë	û
C	NP	FS	,	<	L	\	l		¬	¼	Ì	Ü	ì	ü
D	CR	GS	-	=	M	]	m	}	-	½	Í	Ý	í	ý
E	SO	RS	.	>	N	^	n	~	®	¾	Î	Þ	î	þ
F	SI	US	/	?	O	_	o	DEL	¯	¿	Ï	ß	ï	ÿ

2.1.4.2 Multibyte Encoding Schemes

Multibyte encoding schemes are needed to support ideographic scripts used in Asian languages like Chinese or Japanese because these languages use thousands of characters. These encoding schemes use either a fixed number or a variable number of bytes to represent each character.

- Fixed-width multibyte encoding schemes

In a fixed-width multibyte encoding scheme, each character is represented by a fixed number of bytes. The number of bytes is at least two in a multibyte encoding scheme.

- Variable-width multibyte encoding schemes

A variable-width encoding scheme uses one or more bytes to represent a single character. Some multibyte encoding schemes use certain bits to indicate the number of bytes that represents a character. For example, if two bytes is the maximum number of bytes used to represent a character, then the most significant bit can be used to indicate whether that byte is a single-byte character or the first byte of a double-byte character.

- Shift-sensitive variable-width multibyte encoding schemes

Some variable-width encoding schemes use control codes to differentiate between single-byte and multibyte characters with the same code values. A shift-out code indicates that the following character is multibyte. A shift-in code indicates that the following character is single-byte. Shift-sensitive encoding schemes are used primarily on IBM platforms. Note that ISO-2022 character sets cannot be used as database character sets, but they can be used for applications such as a mail server.

2.1.5 Naming Convention for Oracle Database Character Sets

Oracle databases use the following naming convention for its character set names:

<region><number of bits used to represent a character><standard character set name>[S|C]

The parts of the names that appear between angle brackets are concatenated. The optional S or C is used to differentiate character sets that can be used only on the server (S) or only on the client (C).

Note

Keep in mind that:

- You should use the server character set (S) on the Macintosh platform. The Macintosh client character sets are obsolete. On EBCDIC platforms, use the server character set (S) on the server and the client character set (C) on the client.
- UTF8 and UTFE are exceptions to the naming convention.

The following table shows examples of Oracle database character set names.

Table 2-4 Examples of Oracle Database Character Set Names

Oracle Database Character Set Name	Description	Region	Number of Bits Used to Represent a Character	Standard Character Set Name
US7ASCII	U.S. 7-bit ASCII	US	7	ASCII
WE8ISO8859P1	Western European 8-bit ISO 8859 Part 1	WE (Western Europe)	8	ISO8859 Part 1
JA16SJIS	Japanese 16-bit Shifted Japanese Industrial Standard	JA	16	SJIS

2.1.6 Subsets and Supersets

When discussing character set conversion or character set compatibility between databases, Oracle documentation sometimes uses the terms *superset*, *subset*, *binary superset*, or *binary subset* to describe relationship between two character sets. The terms *subset* and *superset*, without the adjective "binary", pertain to character repertoires of two Oracle character sets, that is, to the sets of characters supported (encoded) by each of the character sets. By definition, character set A is a superset of character set B if A supports all characters that B supports. Character set B is a subset of character set A if A is a superset of B.

The terms *binary subset* and *binary superset* restrict the above subset-superset relationship by adding a condition on binary representation (binary codes) of characters of the two character sets. By definition, character set A is a binary superset of character set B if A supports all characters that B supports and all these characters have the same binary representation in A and B. Character set B is a binary subset of character set A if A is a binary superset of B.

When character set A is a binary superset of character set B, any text value encoded in B is at the same time valid in A without need for character set conversion. When A is a non-binary

superset of B, a text value encoded in B can be represented in A without loss of data but may require character set conversion to transform the binary representation.

The database does not maintain a list of all subset-superset pairs, but it does maintain a list of binary subset-superset pairs that it recognizes in various situations, such as checking compatibility of a transportable tablespace or a pluggable database.

① See Also

"[Binary Subset-Superset Pairs](#)" for the list of binary subset-superset pairs recognized by Oracle databases

2.2 Length Semantics

In single-byte character sets, the number of bytes and the number of characters in a string are the same. In multibyte character sets, a character or code point consists of one or more bytes. Calculating the number of characters based on byte lengths can be difficult in a variable-width character set. Calculating column lengths in bytes is called **byte semantics**, while measuring column lengths in characters is called **character semantics**.

Character semantics is useful for defining the storage requirements for multibyte strings of varying widths. For example, in a Unicode database (AL32UTF8), suppose that you need to define a VARCHAR2 column that can store up to five Chinese characters together with five English characters. Using byte semantics, this column requires 15 bytes for the Chinese characters, which are three bytes long, and 5 bytes for the English characters, which are one byte long, for a total of 20 bytes. Using character semantics, the column requires 10 characters.

The following expressions use byte semantics:

- VARCHAR2(20 BYTE)
- SUBSTRB(*string*, 1, 20)

Note the BYTE qualifier in the VARCHAR2 expression and the B suffix in the SQL function name.

The following expressions use character semantics:

- VARCHAR2(10 CHAR)
- SUBSTR(*string*, 1, 10)

Note the CHAR qualifier in the VARCHAR2 expression.

The length semantics of character data type columns, user-defined type attributes, and PL/SQL variables can be specified explicitly in their definitions with the BYTE or CHAR qualifier. This method of specifying the length semantics is recommended as it properly documents the expected semantics in creation DDL statements and makes the statements independent of any execution environment.

If a column, user-defined type attribute or PL/SQL variable definition contains neither the BYTE nor the CHAR qualifier, the length semantics associated with the column, attribute, or variable is determined by the value of the session parameter NLS_LENGTH_SEMANTICS. If you create database objects with legacy scripts that are too large and complex to be updated to include explicit BYTE and/or CHAR qualifiers, execute an explicit ALTER SESSION SET NLS_LENGTH_SEMANTICS statement before running each of the scripts to assure the scripts create objects in the expected semantics.

The NLS_LENGTH_SEMANTICS initialization parameter determines the default value of the NLS_LENGTH_SEMANTICS session parameter for new sessions. Its default value is BYTE. For the sake of compatibility with existing application installation procedures, which may have been written before character length semantics was introduced into Oracle SQL, Oracle recommends that you leave this initialization parameter undefined or you set it to BYTE. Otherwise, created columns may be larger than expected, causing applications to malfunction or, in some cases, cause buffer overflows.

Byte semantics is the default for the database character set. Character length semantics is the default and the only allowable kind of length semantics for NCHAR data types. The user cannot specify the CHAR or BYTE qualifier for NCHAR definitions.

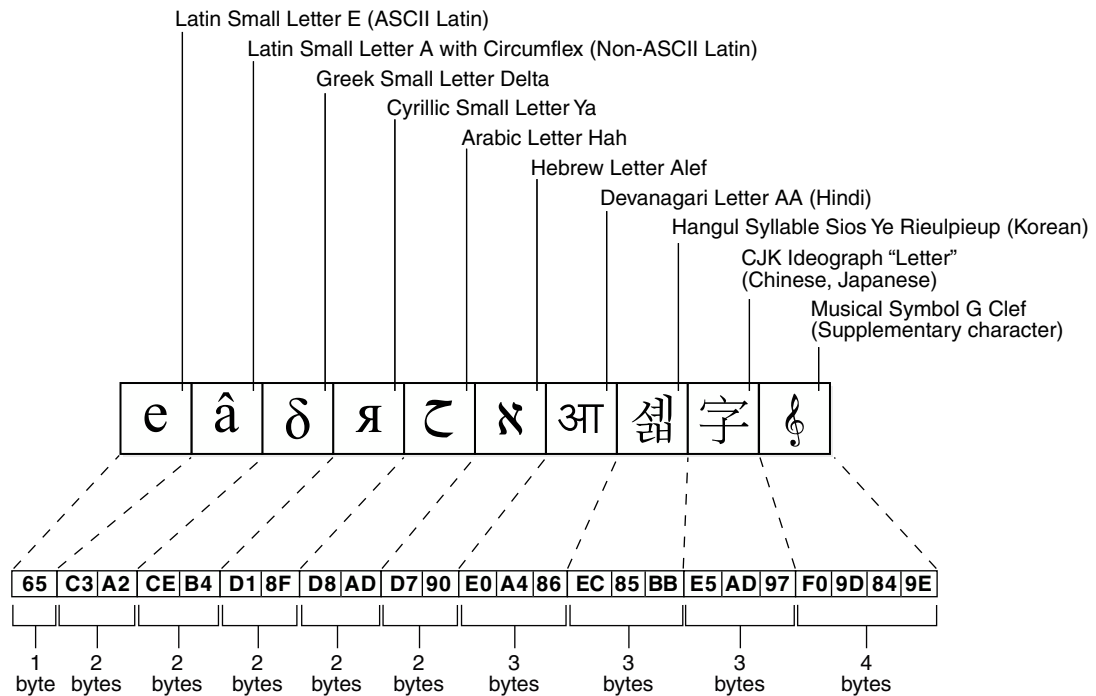
Consider the following example:

```
CREATE TABLE employees

( employee_id NUMBER(4)
, last_name NVARCHAR2(10)
, job_id NVARCHAR2(9)
, manager_id NUMBER(4)
, hire_date DATE
, salary NUMBER(7,2)
, department_id NUMBER(2)
) ;
```

last_name can hold up to 10 Unicode code points, independent of whether the NCHAR character set is AL16UTF16 or UTF8. When the NCHAR character set is AL16UTF16, these stored 10 code points may occupy up to 20 bytes. When the NCHAR character set is UTF8, they may occupy up to 30 bytes.

The following figure shows the number of bytes needed to store different kinds of characters in the UTF-8 character set. The ASCII character requires one byte, the non-ASCII Latin, Greek, Cyrillic, Arabic, and Hebrew characters require two bytes, the Asian characters require three bytes, and the supplementary character requires four bytes of storage.

Figure 2-2 Bytes of Storage for Different Kinds of Characters**① See Also**

- ["SQL Functions for Different Length Semantics"](#) for more information about the SUBSTR and SUBSTRB functions
- ["Length Semantics"](#) for more information about the NLS_LENGTH_SEMANTICS initialization parameter
- ["Supporting Multilingual Databases with Unicode"](#) for more information about Unicode and the NCHAR data type
- *Oracle AI Database SQL Language Reference* for more information about the SUBSTRB and SUBSTR functions and the BYTE and CHAR qualifiers for character data types

2.3 Choosing an Oracle Database Character Set

Oracle uses the database character set for:

- Data stored in SQL CHAR data types (CHAR, VARCHAR2, CLOB, and LONG)
- Identifiers such as table names, column names, and PL/SQL variables
- Entering and storing SQL and PL/SQL source code

The character encoding scheme used by the database is defined as part of the CREATE DATABASE statement. All SQL CHAR data type columns (CHAR, CLOB, VARCHAR2, and LONG), including columns in the data dictionary, have their data stored in the database character set. In addition, the choice of database character set determines which characters can name

objects in the database. SQL NCHAR data type columns (NCHAR, NCHAR2, and NVARCHAR2) use the national character set.

After the database is created, you cannot change the character sets without re-creating the database, except as documented in "[Character Set Migration](#)".

Consider the following questions when you choose an Oracle database character set for the database:

- What languages does the database need to support now?
- What languages does the database need to support in the future?
- Is the character set available on the operating system?
- What character sets are used on clients?
- How well does the application handle the character set?
- What are the performance implications of the character set?
- What are the restrictions associated with the character set?

The Oracle database character sets are listed in "[Character Sets](#)". They are named according to the languages and regions in which they are used. Some character sets that are named for a region are also listed explicitly by language.

If you want to see the characters that are included in a character set, then:

- Check national, international, or vendor product documentation or standards documents
- Use Oracle Locale Builder

This section contains the following topics:

- [Choosing Unicode as a Database Character Set](#)
- [Current and Future Language Requirements](#)
- [Client Operating System and Application Compatibility](#)
- [Character Set Conversion Between Clients and the Server](#)
- [Running Single-Byte Client Applications with a Unicode Database](#)
- [Character Set for Data Migrated from an EBCDIC Platform](#)
- [Restrictions on Database Character Sets](#)
- [Database Character Set Statement of Direction](#)
- [Choosing a National Character Set](#)
- [Summary of Supported Data Types](#)

See Also

- "[UCS-2 Encoding Form](#)"
- "[Choosing a National Character Set](#)"
- "[Changing the Character Set After Database Creation](#)"
- "[Locale Data](#)"
- "[Customizing Locale Data](#)"

2.3.1 Choosing Unicode as a Database Character Set

Oracle recommends using AL32UTF8—Oracle's implementation of the Unicode UTF-8 character encoding—for all new system deployments. Migrating legacy systems to Unicode is also recommended. Deploying your systems today in Unicode offers many advantages in usability, compatibility, and extensibility. An Oracle database enables you to deploy high-performing systems faster and more easily while utilizing the advantages of Unicode. Even if you do not need to support multilingual data today, nor have any requirement for Unicode, it is still likely to be the best choice for a new system in the long run and will ultimately save you time and money as well as give you competitive advantages in the long term.

① See Also

["Supporting Multilingual Databases with Unicode"](#)

2.3.2 Current and Future Language Requirements

If you choose a database character set other than AL32UTF8, for example, due to application compatibility, it is essential to evaluate both current and future language requirements. If your application system may need to support additional languages in the future, select a character set that can accommodate them. Doing so helps avoid the need for character set migration later—a process that can be complex and resource-intensive.

Single-byte character sets typically provide the fastest processing and most efficient storage utilization, but significantly limit language support. Legacy non-Unicode multibyte character sets also offer only limited multilingual capabilities. For broad language support, Oracle strongly recommends AL32UTF8.

2.3.3 Client Operating System and Application Compatibility

The database character set is independent of the operating system because Oracle databases have their own globalization architecture. For example, on an English Windows operating system, you can create and run a database with a Japanese character set. However, when an application in the client operating system accesses the database, the client operating system must be able to support the database character set with appropriate fonts and input methods. For example, you cannot insert or retrieve Japanese data on the English Windows operating system without first installing a Japanese font and input method. Another way to insert and retrieve Japanese data is to use a Japanese operating system remotely to access the database server.

2.3.4 Character Set Conversion Between Clients and the Server

If you choose a database character set that is different from the character set on the client operating system, then the database can convert the operating system character set to the database character set. Character set conversion has the following disadvantages:

- Potential data loss
- Increased overhead

Character set conversions can sometimes cause data loss. For example, if you are converting from character set A to character set B, then the destination character set B must have the same character set repertoire as A. Any characters that are not available in character set B are

converted to a replacement character. The replacement character is often specified as a question mark or as a linguistically related character. For example, ä (a with an umlaut) may be converted to a. If you have distributed environments, then consider using character sets with similar character repertoires to avoid loss of data.

The database character set should always be a superset or equivalent of the native character set of the client's operating system. The character sets used by client applications that access the database usually determine which superset is the best choice.

If all client applications use the same character set, then that character set is usually the best choice for the database character set. When client applications use different character sets, the database character set should be a superset of all the client character sets. This ensures that every character is represented when converting from a client character set to the database character set.

① **See Also**

["Character Set Migration"](#)

2.3.5 Running Single-Byte Client Applications with a Unicode Database

Some applications, due to limitations of the programming language and API calls, can only process single-byte data. For example, consider a C application that processes strings using a simple pointer increment:

```
char *ptr, ch;  
ch = *ptr++;
```

Such applications cannot properly interpret 2-, 3-, and 4-byte character codes present in the AL32UTF8 character set. However, these applications can still access and modify data in a database that uses AL32UTF8. Character set conversion between the client and database ensures correct translation between single-byte and multibyte encodings.

To avoid data corruption and processing errors, the application must adhere to the following restrictions:

- **Data Compatibility:** The application must not select and modify data not supported by its single-byte character set. This restriction can be enforced by ensuring that other applications using different client character sets do not insert or modify the same data. You may implement this using separate tables, views, or Virtual Private Database (VPD) policies.
- **Character Function Usage:** In SQL and PL/SQL statements, do not use character functions that operate on byte offsets and lengths, such as LENGTHB, SUBSTRB, INSTRB, and VSIZE. Use the corresponding character-based functions: LENGTH, SUBSTR, and INSTR.
- **Character Encoding Assumptions:** Do not assume specific binary codes correspond to the same characters in different character sets. For example, calling CHR(228) produces the character 'ä' (a with diaeresis) in client character sets like WE8MSWIN1252 and WE8ISO8859P1, but not in AL32UTF8. Such calls must be replaced with character literals or updated to use AL32UTF8 codes—in this example, use CHR(50084).
- **Hashing and Encryption:** Perform hashing and encryption of character values either entirely on the client side or entirely on the server side. Hashed or encrypted values are not converted between client and server character sets, and a value encrypted on one side

and decrypted on the other will be in the wrong encoding. Similarly, hashes computed on different sides may not match. If you need to process hashed or encrypted values in multiple character set environments, normalize data to AL32UTF8 before hashing or encryption.

- **Length Declarations:** Always declare table columns and PL/SQL variable lengths using character length semantics. For example:

```
last_name VARCHAR2(100 CHAR)
```

You can also use extended data types to ensure support for long columns, for example, `VARCHAR2(4000 CHAR)`. With extended data types enabled, a string that is 2000 bytes in a single-byte character set and might expand to 6000 bytes in AL32UTF8 will not encounter the standard 4000-byte limit.

If the above restrictions are followed, single-byte applications can safely work with a Unicode database. This approach enables migration to Unicode without significant changes to existing applications.

 See Also

["Specifying Column Lengths for Multilingual Data"](#)

2.3.6 Character Set for Data Migrated from an EBCDIC Platform

When migrating an application system from a mainframe EBCDIC-based environment to a modern ASCII-based platform, such as Linux, it is essential to select an Oracle Database character set that supports all characters in the source data. Additionally, the chosen character set should support round-trip conversion between the source (EBCDIC) and target (ASCII) character sets to ensure data integrity, especially when running legacy applications without modifications. For more information, see "[Choosing a Character Set for the Target Database](#)".

2.3.7 Restrictions on Database Character Sets

ASCII-based character sets are supported only on ASCII-based platforms. Similarly, you can use an EBCDIC-based character set only on EBCDIC-based platforms.

The database character set is used to identify SQL and PL/SQL source code. In order to do this, it must have either EBCDIC or 7-bit ASCII as a subset, whichever is native to the platform. Therefore, it is not possible to use a fixed-width, multibyte character set as the database character set. Currently, only the AL16UTF16 character set cannot be used as a database character set.

2.3.7.1 Restrictions on Character Sets Used to Express Names

The following table lists the restrictions on the character sets that can be used to express names.

Table 2-5 Restrictions on Character Sets Used to Express Names

Name	Single-Byte	Variable Width	Comments
Column names	Yes	Yes	-
Schema objects	Yes	Yes	-

Table 2-5 (Cont.) Restrictions on Character Sets Used to Express Names

Name	Single-Byte	Variable Width	Comments
Comments	Yes	Yes	-
Database link names	Yes	No	-
Database names	Yes	No	-
File names (data file, log file, control file, initialization parameter file)	Yes	No	-
Instance names	Yes	No	-
Directory names	Yes	No	-
Keywords	Yes	No	Can be expressed in English ASCII or EBCDIC characters only
Recovery Manager file names	Yes	No	-
Rollback segment names	Yes	No	The ROLLBACK_SEGMENTS parameter does not support NLS
Stored script names	Yes	Yes	-
Tablespace names	Yes	No	-

For a list of supported string formats and character sets, including LOB data (LOB, BLOB, CLOB, and NCLOB), see [Table 2-7](#).

2.3.8 Database Character Set Statement of Direction

A list of character sets has been compiled in [Table A-4](#) and [Table A-5](#) that Oracle strongly recommends for usage as the database character set. Other Oracle-supported character sets that do not appear on this list can continue to be used in this Oracle Database release, but may be desupported in a future release. Starting with Oracle Database 11g Release 1, the choice for the database character set is limited to this list of recommended character sets in common installation paths of Oracle Universal Installer (OUI) and Oracle Database Configuration Assistant (DBCA). Customers are still able to create new databases using custom installation paths and migrate their existing databases even if the character set is not on the recommended list. However, Oracle suggests that customers migrate to a recommended character set as soon as possible. At the top of the list of character sets that Oracle recommends for all new system deployment, is the Unicode character set AL32UTF8.

Note

Starting with Oracle Database 12c Release 2, if you use Oracle Universal Installer (OUI) or Oracle Database Configuration Assistant (DBCA) to create a database, then the default database character set used is AL32UTF8.

2.3.9 Choosing a National Character Set

The term **national character set** refers to an alternative character set that enables you to store Unicode character data in a database that does not have a Unicode database character

set. Another reason for choosing a national character set is that the properties of a different character encoding scheme may be more desirable for extensive character processing operations.

SQL NCHAR, NVARCHAR2, and NCLOB data types support Unicode data only. You can use either the UTF8 or the AL16UTF16 character set. The default is AL16UTF16.

Oracle recommends using SQL CHAR, VARCHAR2, and CLOB data types in AL32UTF8 database to store Unicode character data. Use of SQL NCHAR, NVARCHAR2, and NCLOB should be considered only if you must use a database whose database character set is not AL32UTF8.

① See Also

["Supporting Multilingual Databases with Unicode"](#)

2.3.10 Summary of Supported Data Types

The following table lists the data types that are supported for different encoding schemes.

Table 2-6 SQL Data Types Supported for Encoding Schemes

Data Type	Single Byte	Multibyte Non-Unicode	Multibyte Unicode
CHAR	Yes	Yes	Yes
VARCHAR2	Yes	Yes	Yes
NCHAR	No	No	Yes
NVARCHAR2	No	No	Yes
BLOB	Yes	Yes	Yes
CLOB	Yes	Yes	Yes
LONG	Yes	Yes	Yes
NCLOB	No	No	Yes

① Note

BLOBs process characters as a series of byte sequences. The data is not subject to any NLS-sensitive operations.

The following table lists the SQL data types that are supported for abstract data types.

Table 2-7 Abstract Data Type Support for SQL Data Types

Abstract Data Type	CHAR	NCHAR	BLOB	CLOB	NCLOB
Object	Yes	Yes	Yes	Yes	Yes
Collection	Yes	Yes	Yes	Yes	Yes

You can create an abstract data type with the NCHAR attribute as follows:

```
SQL> CREATE TYPE tp1 AS OBJECT (a NCHAR(10));
Type created.
SQL> CREATE TABLE t1 (a tp1);
Table created.
```

① See Also

- *Oracle AI Database Object-Relational Developer's Guide* for more information about Oracle objects
- *Oracle AI Database PL/SQL Packages and Types Reference* for more information about Oracle collections

2.4 Choosing a Database Character Set for a Multitenant Container Database

Starting with Oracle Database 12c Release 2 (12.2), pluggable databases (PDBs) in a multitenant container database (CDB) can have different *database* character sets and different *national* character sets. The databases or *PDB candidates* that can be plugged into a CDB can be traditional independent databases or existing PDBs unplugged from other CDBs or newly created PDBs in the CDB.

① Note

The character set of the CDB root is considered as the character set of the whole CDB.

The following scenarios may occur depending upon the *database* character set of the PDB candidate that needs to be plugged into a CDB:

- If the PDB candidate is an application PDB to be plugged into an application root:
 - If the database character set of the PDB candidate is the same as the database character set of the application root, the plug-in operation succeeds (as far as the database character set is concerned).
 - If the database character set of the PDB candidate is *plug compatible* with the database character set of the application root, that is, the database character set of the PDB candidate is a binary subset of the database character set of the application root and both are single-byte or both are multibyte, then the database character set of the PDB candidate is automatically changed to the database character set of the application root when the PDB candidate is opened for the first time and the plug-in operation succeeds.
 - If the database character set of the PDB candidate is not *plug compatible* with the database character set of the application root (when none of the above two scenarios apply), then the plug-in operation succeeds. But in this case the newly plugged-in PDB can be opened only in the *restricted* mode for performing administrative tasks and cannot be used for production. Unless you migrate the database character set of the new PDB to the database character set of the application root, the new PDB is unusable.
- If the PDB candidate is to be plugged directly into the CDB root:

- If the database character set of the PDB candidate is the same as the database character set of the CDB, then the plug-in operation succeeds (as far as the database character set is concerned).
- If the database character set of the CDB is AL32UTF8, then the plug-in operation succeeds regardless of the database character set of the PDB candidate.
- If the database character set of the PDB candidate is *plug compatible* with the database character set of the CDB, that is, the database character set of the PDB candidate is a binary subset of the database character set of the CDB and both are single-byte or both are multibyte, then the database character set of the PDB candidate is automatically changed to the database character set of the CDB when the PDB candidate is opened for the first time and the plug-in operation succeeds.
- If the database character set of the PDB candidate is not *plug compatible* with the database character set of the CDB, that is, when none of the last three scenarios mentioned above apply, then the plug-in operation succeeds. But, in this case the newly plugged-in PDB can be opened only in the *restricted* mode for performing administrative tasks and cannot be used for production. Unless you migrate the database character set of the new PDB to the database character set of the CDB, the new PDB is unusable.

See Also

"[Subsets and Supersets](#)" for more information about binary subset and binary superset of a character set.

The following scenarios may occur depending upon the *national* character set of the PDB candidate that needs to be plugged into a CDB:

- If the PDB candidate is an application PDB to be plugged into an application root:
 - If the national character set of the PDB candidate is the same as the national character set of the application root, then the plug-in operation succeeds (as far as the national character set is concerned).
 - If the national character set of the PDB candidate is not the same as the national character set of the application root, then the plug-in operation succeeds. But, in this case the newly plugged-in PDB can be opened only in the *restricted* mode for performing administrative tasks and cannot be used for production. Unless you migrate the national character set of the new PDB to the national character set of the application root, the new PDB is unusable.
- If the PDB candidate is to be plugged directly into the CDB root, then the plug-in operation succeeds (as far as the national character set is concerned).

Note

- When a PDB character set is different from the CDB character set, there may be data truncation, if the column widths of CDB views and V\$ views are not able to accommodate the PDB data that has expanded in length during the character set conversion.
- As UTF8 and AL32UTF8 have different maximum character widths (three versus four bytes per character), the automatic change of UTF8 to AL32UTF8 during plug-in operation will change implicit maximum byte lengths of columns with character length semantics. This change may fail, if there are functional indexes, virtual columns, bitmap join indexes, domain indexes, partitioning keys, sub-partitioning keys, or cluster keys defined on those columns. The plug-in operation may also fail, if a character length semantics column is part of an index key, and the index key exceeds the size limit (around 70% of the index block size) after the character set change. You must make sure that all the offending objects are removed from a database before it is plugged into a CDB. You can recreate those offending objects in the database after the database is plugged into a CDB.

Because of these restrictions, Oracle recommends the following when selecting character sets for CDBs:

- For all new multitenant deployments, use AL32UTF8 as the database character set and AL16UTF16 as the national character set for a CDB.
- Migrate your existing databases to AL32UTF8 database character set before consolidation and then consolidate the databases into one or more AL32UTF8 CDBs, depending on your needs. You can use the *Oracle Database Migration Assistant for Unicode* software to migrate a non-CDB to AL32UTF8 database character set.

See Also

- *Oracle AI Database Concepts* and *Oracle AI Database Multitenant Administrator's Guide* for more information about CDBs, PDBs, and application containers.
- *Oracle Database Migration Assistant for Unicode Guide* for more information about migrating a non-Unicode database character set to a Unicode database character set.

2.5 Changing the Character Set After Database Creation

You may want to change the database character set after the database has been created. For example, you may find that the number of languages that must be supported in your database has increased, and you therefore want to migrate to Unicode character set AL32UTF8.

As character type data in the database must be converted to Unicode, in most cases, you will encounter challenges when you change the database character set to AL32UTF8. For example, CHAR and VARCHAR2 column data may exceed the declared column length. Character data may be lost when it is converted to Unicode if it contains invalid characters.

Before changing the database character set, it is important to identify all problems and carefully plan the data migration. Oracle recommends using the Database Migration Assistant for Unicode to change the database character set to AL32UTF8.

Note

Starting from Oracle Database 12c Release 2, if you use Oracle Universal Installer (OUI) or Oracle Database Configuration Assistant (DBCA) to create a database, then the default database character set used is the Unicode character set AL32UTF8.

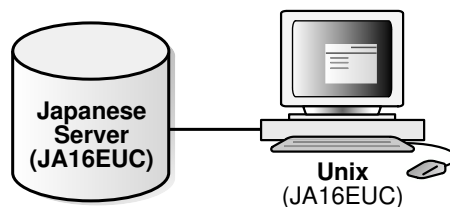
See Also

Oracle Database Migration Assistant for Unicode Guide for more information about how to change character sets

2.6 Monolingual Database Scenario

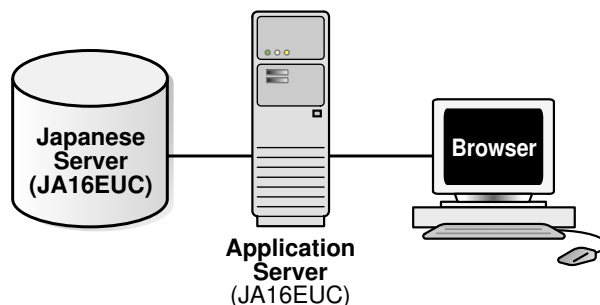
The simplest example of a database configuration is a client and a server that run in the same language environment and use the same character set. This monolingual scenario has the advantage of fast response because the overhead associated with character set conversion is avoided. The following figure shows a database server and a client that use the same character set. The Japanese client and the server both use the JA16EUC character set.

Figure 2-3 Monolingual Database Scenario



You can also use a multitier architecture. The following figure shows an application server between the database server and the client. The application server and the database server use the same character set in a monolingual scenario. The server, the application server, and the client use the JA16EUC character set.

Figure 2-4 Multitier Monolingual Database Scenario

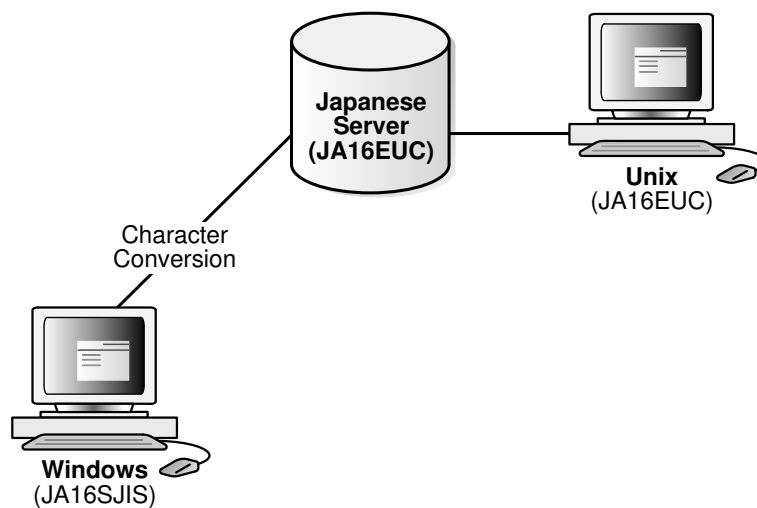


2.6.1 Character Set Conversion in a Monolingual Scenario

Character set conversion may be required in a client/server environment if a client application resides on a different platform than the server and if the platforms do not use the same character encoding schemes. Character data passed between client and server must be converted between the two encoding schemes. Character conversion occurs automatically and transparently through Oracle Net.

You can convert between any two character sets. The following figure shows a server and one client with the JA16EUC Japanese character set. The other client uses the JA16SJIS Japanese character set.

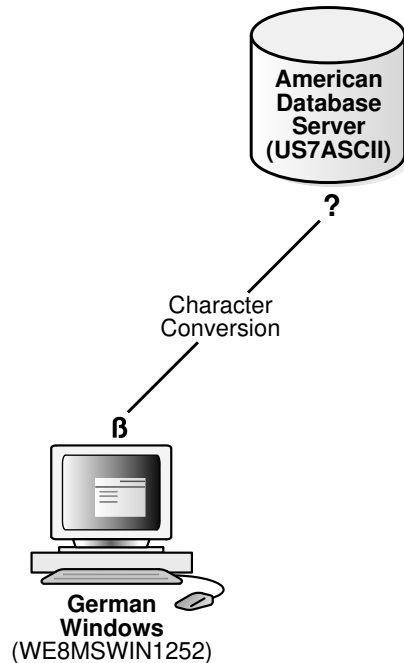
Figure 2-5 Character Set Conversion



When a target character set does not contain all of the characters in the source data, replacement characters are used. If, for example, a server uses US7ASCII and a German client uses WE8ISO8859P1, then the German character ß is replaced with ? and ä is replaced with a.

Replacement characters may be defined for specific characters as part of a character set definition. When a specific replacement character is not defined, a default replacement character is used. To avoid the use of replacement characters when converting from a client character set to a database character set, the server character set should be a superset of all the client character sets.

The following figure shows that data loss occurs when the database character set does not include all of the characters in the client character set. The database character set is US7ASCII. The client's character set is WE8MSWIN1252, and the language used by the client is German. When the client inserts a string that contains ß, the database replaces ß with ?, resulting in lost data.

Figure 2-6 Data Loss During Character Conversion

If German data is expected to be stored on the server, then a database character set that supports German characters should be used for both the server and the client to avoid data loss and conversion overhead.

When one of the character sets is a variable-width multibyte character set, conversion can introduce noticeable overhead. Carefully evaluate your situation and choose character sets to avoid conversion as much as possible.

2.7 Multilingual Database Scenario

If you need multilingual support, then use Unicode AL32UTF8 for the server database character set.

Note

Starting from Oracle Database 12c Release 2, if you use Oracle Universal Installer (OUI) or Oracle Database Configuration Assistant (DBCA) to create a database, then the default database character set used is the Unicode character set AL32UTF8.

Unicode has two major encoding schemes:

- UTF-16: Each character is either 2 or 4 bytes long.
- UTF-8: Each character takes 1 to 4 bytes to store.

Oracle databases provides support for UTF-8 as a database character set and both UTF-8 and UTF-16 as national character sets.

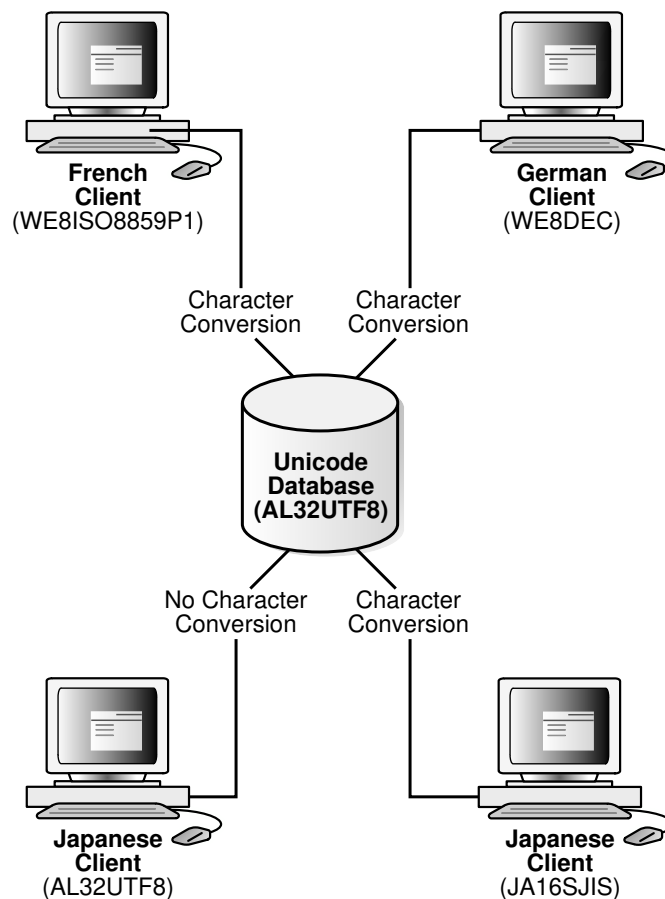
Character set conversion between a UTF-8 database and any single-byte character set introduces very little overhead.

Conversion between UTF-8 and any multibyte character set has some overhead. There is no data loss from conversion, with the following exceptions:

- Some multibyte character sets do not support user-defined characters during character set conversion to and from UTF-8.
- Some Unicode characters are mapped to more than one character in another character set. For example, one Unicode character is mapped to three characters in the JA16SJIS character set. This means that a round-trip conversion may not result in the original JA16SJIS character.

The following figure shows a server that uses the AL32UTF8 Oracle database character set that is based on the Unicode UTF-8 character set.

Figure 2-7 Multilingual Support Scenario in a Client/Server Configuration



There are four clients:

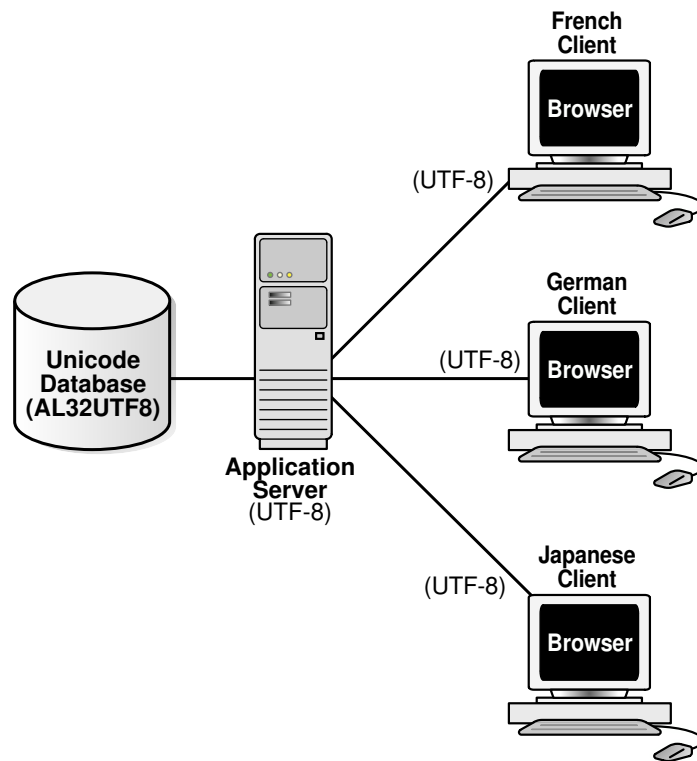
- A French client that uses the WE8ISO8859P1 Oracle database character set
- A German client that uses the WE8DEC character set
- A Japanese client that uses the AL32UTF8 character set
- A Japanese client that used the JA16SJIS character set

Character conversion takes place between each client and the server except for the AL32UTF8 client, but there is no data loss because AL32UTF8 is a universal character set. If

the German client tries to retrieve data from one of the Japanese clients, then all of the Japanese characters in the data are lost during the character set conversion.

The following figure shows a Unicode solution for a multitier configuration.

Figure 2-8 Multitier Multilingual Support Scenario in a Multitier Configuration



The database, the application server, and each client use the AL32UTF8 character set. This eliminates the need for character conversion even though the clients are French, German, and Japanese.

① **See Also**

["Supporting Multilingual Databases with Unicode"](#)

3

Setting Up a Globalization Support Environment

This chapter tells how to set up a globalization support environment. It includes the following topics:

- [Setting NLS Parameters](#)
- [Choosing a Locale with the NLS_LANG Environment Variable](#)
- [Character Set Parameter](#)
- [NLS Database Parameters](#)
- [Language and Territory Parameters](#)
- [Date and Time Parameters](#)
- [Calendar Definitions](#)
- [Numeric and List Parameters](#)
- [Monetary Parameters](#)
- [Linguistic Sort Parameters](#)
- [Character Set Conversion Parameter](#)
- [Length Semantics](#)

3.1 Setting NLS Parameters

NLS (National Language Support) parameters determine the locale-specific behavior on both the client and the server. NLS parameters can be specified in the following ways:

- As initialization parameters on the server
You can include parameters in the initialization parameter file to specify a default session NLS environment. These settings have no effect on the client side; they control only the server's behavior. For example:

```
NLS_TERRITORY = "CZECH REPUBLIC"
```
- As environment variables on the client
You can use NLS environment variables, which may be platform-dependent, to specify locale-dependent behavior for the client and also to override the default values set for the session in the initialization parameter file. For example, on a UNIX system:

```
% setenv NLS_SORT FRENCH
```
- With the ALTER SESSION statement
You can use NLS parameters that are set in an ALTER SESSION statement to override the default values that are set for the session in the initialization parameter file or set by the client with environment variables.

```
SQL> ALTER SESSION SET NLS_SORT = FRENCH;
```

① See Also

Oracle AI Database SQL Language Reference for more information about the ALTER SESSION statement

- In SQL functions

You can use NLS parameters explicitly to hardcode NLS behavior within a SQL function. This practice overrides the default values that are set for the session in the initialization parameter file, set for the client with environment variables, or set for the session by the ALTER SESSION statement. For example:

```
TO_CHAR(hiredate, 'DD/MON/YYYY', 'nls_date_language = FRENCH')
```

① See Also

Oracle AI Database SQL Language Reference for more information about SQL functions, including the TO_CHAR function

[Table 3-1](#) shows the precedence order of the different methods of setting NLS parameters. Higher priority settings override lower priority settings. For example, a value specified in the initialization parameter file can be overridden by a value explicitly set in a SQL function.

Default values have the lowest priority. They are set at the time of database creation and cannot be changed. They can be overridden by any other method, with the following exception: Default values are always used when evaluating expressions in virtual columns, CHECK constraints, and fine-grained auditing (FGA) rules. These expressions must have deterministic results for the duration of their existence and cannot depend on NLS parameter settings that may change.

Table 3-1 Methods of Setting NLS Parameters and Their Priorities

Priority	Method
1 (highest)	Explicitly set in SQL functions
2	Set by an ALTER SESSION statement
3	Set as an environment variable
4	Specified in the initialization parameter file
5 (lowest)	Default value specified when the database was created

[Table 3-2](#) lists the available NLS parameters. Because the SQL function NLS parameters can be specified only with specific functions, the table does not show the SQL function scope. This table shows the following values for Scope:

I = Initialization Parameter File

E = Environment Variable

A = ALTER SESSION

Table 3-2 NLS Parameters

Parameter	Description	Default	Scope
NLS_CALENDAR	Calendar system	Gregorian	I, E, A

Table 3-2 (Cont.) NLS Parameters

Parameter	Description	Default	Scope
NLS_COMP	SQL, PL/SQL operator comparison	BINARY	I, E, A
NLS_CREDIT	Credit accounting symbol	Derived from NLS_TERRITORY	E
NLS_CURRENCY	Local currency symbol	Derived from NLS_TERRITORY	I, E, A
NLS_DATE_FORMAT	Date format	Derived from NLS_TERRITORY	I, E, A
NLS_DATE_LANGUAGE	Language for day and month names	Derived from NLS_LANGUAGE	I, E, A
NLS_DEBIT	Debit accounting symbol	Derived from NLS_TERRITORY	E
NLS_DUAL_CURRENCY	Dual currency symbol	Derived from NLS_TERRITORY	I, E, A
NLS_ISO_CURRENCY	ISO international currency symbol	Derived from NLS_TERRITORY	I, E, A
NLS_LANG	Language, territory, character set	AMERICAN_AMERICA. US7ASCII	E
NLS_LANGUAGE	Language	Derived from NLS_LANG	I, A
NLS_LENGTH_SEMANTICS	How strings are treated	BYTE	I, E, A
NLS_LIST_SEPARATOR	Character that separates items in a list	Derived from NLS_TERRITORY	E
NLS_MONETARY_CHARACTERS	Monetary symbol for dollar and cents (or their equivalents)	Derived from NLS_TERRITORY	E
NLS_NCHAR_CONV_EXCP	Reports data loss during a character type conversion	FALSE	I, A
NLS_NUMERIC_CHARACTERS	Decimal character and group separator	Derived from NLS_TERRITORY	I, E, A
NLS_SORT	Collation	Derived from NLS_LANGUAGE	I, E, A
NLS_TERRITORY	Territory	Derived from NLS_LANG	I, A
NLS_TIMESTAMP_FORMAT	Timestamp	Derived from NLS_TERRITORY	I, E, A
NLS_TIMESTAMP_TZ_FORMAT	Timestamp with time zone	Derived from NLS_TERRITORY	I, E, A

3.2 Choosing a Locale with the NLS_LANG Environment Variable

A **locale** is a linguistic and cultural environment in which a system or program is running. Setting the NLS_LANG environment parameter is the simplest way to specify locale behavior for Oracle database software. It sets the language and territory used by the client application and the database server. It also sets the client's character set, which is the character set for data entered or displayed by a client program.

NLS_LANG is set as an environment variable on UNIX platforms. NLS_LANG is set in the registry on Windows platforms.

The NLS_LANG parameter has three components: language, territory, and character set. Specify it in the following format, including the punctuation:

`NLS_LANG = language_territory.charset`

For example, if the Oracle Universal Installer does not populate NLS_LANG, then its value by default is AMERICAN_AMERICA.US7ASCII. The language is AMERICAN, the territory is AMERICA, and the character set is US7ASCII. The values in NLS_LANG and other NLS parameters are case-insensitive.

Each component of the NLS_LANG parameter controls the operation of a subset of globalization support features:

- *language*
Specifies conventions such as the language used for Oracle database messages, sorting, day names, and month names. Each supported language has a unique name; for example, AMERICAN, FRENCH, or GERMAN. The language argument specifies default values for the territory and character set arguments. If the language is not specified, then the value defaults to AMERICAN.
- *territory*
Specifies conventions such as the default date, monetary, and numeric formats. Each supported territory has a unique name; for example, AMERICA, FRANCE, or CANADA. If the territory is not specified, then the value is derived from the language value.
- *charset*
Specifies the character set used by the client application (normally the Oracle database character set that corresponds to the user's terminal character set or the OS character set). Each supported character set has a unique acronym, for example, US7ASCII, WE8ISO8859P1, WE8DEC, WE8MSWIN1252, or JA16EUC. Each language has a default character set associated with it.

Note

All components of the NLS_LANG definition are optional; any item that is not specified uses its default value. If you specify territory or character set, then you *must* include the preceding delimiter (underscore (_) for territory, period (.) for character set). Otherwise, the value is parsed as a language name.

For example, to set only the territory portion of NLS_LANG, use the following format:
`NLS_LANG=_JAPAN`

The three components of NLS_LANG can be specified in many combinations, as in the following examples:

`NLS_LANG = AMERICAN_AMERICA.WE8MSWIN1252`

`NLS_LANG = FRENCH_CANADA.WE8ISO8859P1`

`NLS_LANG = JAPANESE_JAPAN.JA16EUC`

Note that illogical combinations can be set but do not work properly. For example, the following specification tries to support Japanese by using a Western European character set:


```
NLS_LANG = JAPANESE_JAPAN.WE8ISO8859P1
```

Because the WE8ISO8859P1 character set does not support any Japanese characters, you cannot store or display Japanese data if you use this definition for NLS_LANG.

The rest of this section includes the following topics:

- [Specifying the Value of NLS_LANG](#)
- [Overriding Language and Territory Specifications](#)
- [Locale Variants](#)

① See Also

- "[Locale Data](#)" for a complete list of supported languages, territories, and character sets
- Your operating system documentation for information about additional globalization settings that may be necessary for your platform

3.2.1 Specifying the Value of NLS_LANG

In a UNIX operating system C-shell session, you can specify the value of NLS_LANG by entering a statement similar to the following example:

```
% setenv NLS_LANG FRENCH_FRANCE.WE8ISO8859P1
```

Because NLS_LANG is an environment variable, it is read by the client application at startup time. The client communicates the information defined by NLS_LANG to the server when it connects to the database server.

The following examples show how date and number formats are affected by the NLS_LANG parameter.

Example 3-1 Setting NLS_LANG to American_America.WE8ISO8859P1

Set NLS_LANG so that the language is AMERICAN, the territory is AMERICA, and the Oracle database character set is WE8ISO8859P1:

```
% setenv NLS_LANG American_America.WE8ISO8859P1
```

Enter a SELECT statement:

```
SQL> SELECT last_name, hire_date, ROUND(salary/8,2) salary FROM employees;
```

You should see results similar to the following output:

LAST_NAME	HIRE_DATE	SALARY
...		
Sciarra	30-SEP-05	962.5
Urman	07-MAR-06	975
Popp	07-DEC-07	862.5
...		

Example 3-2 Setting NLS_LANG to French_France.WE8ISO8859P1

Set NLS_LANG so that the language is FRENCH, the territory is FRANCE, and the Oracle database character set is WE8ISO8859P1:

```
% setenv NLS_LANG French_France.WE8ISO8859P1
```

Then the query shown in [Example 3-1](#) returns the following output:

LAST_NAME	HIRE_DATE	SALARY
...		
Sciarra	30/09/05	962,5
Urman	07/03/06	975
Popp	07/12/07	862,5
...		

Note that the date format and the number format have changed. The numbers have not changed, because the underlying data is the same.

3.2.2 Overriding Language and Territory Specifications

The NLS_LANG parameter sets the language and territory environment used by both the server session (for example, SQL command execution) and the client application (for example, display formatting in Oracle database tools). Using this parameter ensures that the language environments of both the database and the client application are automatically the same.

The language and territory components of the NLS_LANG parameter determine the default values for other detailed NLS parameters, such as date format, numeric characters, and linguistic sorting. Each of these detailed parameters can be set in the client environment to override the default values if the NLS_LANG parameter has already been set.

If the NLS_LANG parameter is not set, then the server session environment remains initialized with values of NLS_LANGUAGE, NLS_TERRITORY, and other NLS instance parameters from the initialization parameter file. You can modify these parameters and restart the instance to change the defaults.

You might want to modify the NLS environment dynamically during the session. To do so, you can use the ALTER SESSION statement to change NLS_LANGUAGE, NLS_TERRITORY, and other NLS parameters.

Note

You cannot modify the setting for the client character set with the ALTER SESSION statement.

The ALTER SESSION statement modifies only the session environment. The local client NLS environment is not modified, unless the client explicitly retrieves the new settings and modifies its local environment.

See Also

- ["Overriding Default Values for NLS_LANGUAGE and NLS_TERRITORY During a Session"](#)
- *Oracle AI Database SQL Language Reference*

3.2.3 Locale Variants

Before Oracle Database 10g, Oracle defined language and territory definitions separately. This resulted in the definition of a territory being independent of the language setting of the user. Since Oracle Database 10g, some territories can have different date, time, number, and monetary formats based on the language setting of a user. This type of language-dependent territory definition is called a locale variant.

For the variant to work properly, both NLS_TERRITORY and NLS_LANGUAGE must be specified.

The following table shows the territories that have been enhanced to support variations.

Table 3-3 Oracle Database Locale Variants

Oracle Database Territory	Oracle Database Language
BELGIUM	DUTCH
BELGIUM	FRENCH
BELGIUM	GERMAN
CANADA	FRENCH
CANADA	ENGLISH
DJIBOUTI	FRENCH
DJIBOUTI	ARABIC
FINLAND	FINLAND
FINLAND	SWEDISH
HONG KONG	TRADITIONAL CHINESE
HONG KONG	ENGLISH
INDIA	ENGLISH
INDIA	ASSAMESE
INDIA	BANGLA
INDIA	GUJARATI
INDIA	HINDI
INDIA	KANNADA
INDIA	MALAYALAM
INDIA	MARATHI
INDIA	ORIYA
INDIA	PUNJABI
INDIA	TAMIL
INDIA	TELUGU
LUXEMBOURG	GERMAN
LUXEMBOURG	FRENCH
SINGAPORE	ENGLISH
SINGAPORE	MALAY
SINGAPORE	SIMPLIFIED CHINESE
SINGAPORE	TAMIL
SWITZERLAND	GERMAN

Table 3-3 (Cont.) Oracle Database Locale Variants

Oracle Database Territory	Oracle Database Language
SWITZERLAND	FRENCH
SWITZERLAND	ITALIAN

3.2.4 Should the NLS_LANG Setting Match the Database Character Set?

The NLS_LANG character set should reflect the setting of the operating system character set of the client. For example, if the database character set is AL32UTF8 and the client is running on a Windows operating system, then you should not set AL32UTF8 as the client character set in the NLS_LANG parameter because there are no UTF-8 WIN32 clients. Instead, the NLS_LANG setting should reflect the code page of the client. For example, on an English Windows client, the code page is 1252. An appropriate setting for NLS_LANG is AMERICAN_AMERICA.WE8MSWIN1252.

Setting NLS_LANG correctly enables proper conversion from the client operating system character set to the database character set. When these settings are the same, the database assumes that the data being sent or received is encoded in the same character set as the database character set, so character set validation or conversion may not be performed. This can lead to corrupt data if the client code page and the database character set are different and conversions are necessary.

① See Also

Oracle AI Database Installation Guide for Microsoft Windows for more information about commonly used values of the NLS_LANG parameter in Windows

3.3 Character Set Parameter

Oracle provides an environment variable, NLS_OS_CHARSET, for handling the situation where the client OS character set is different from the Oracle NLS client character set.

3.3.1 NLS_OS_CHARSET Environment Variable

The NLS_OS_CHARSET environment variable should be set on Oracle client installations if the client OS character set is different from the Oracle NLS client character set specified by the NLS_LANG environment variable. The client OS character set is the character set used to represent characters in the OS fields like machine name, program executable name and logged on user name. On UNIX platforms, this is usually the character set specified in the LANG environment variable or the LC_ALL environment variable. An example of setting NLS_OS_CHARSET would be if the locale charset specified in LANG or LC_ALL in a Linux client could be zh_CN (simplified Chinese) and the Oracle client application charset specified in NLS_LANG could be UTF8. In this case, the NLS_OS_CHARSET variable must be set to the equivalent Oracle charset ZHT16GBK.

The NLS_OS_CHARSET environment variable must be set to the Oracle character set name corresponding to the client OS character set.

If NLS_LANG corresponds to the OS character set, NLS_OS_CHARSET does not need to be set. NLS_OS_CHARSET does not need to be set and is ignored on Windows platforms.

3.4 NLS Database Parameters

When a new database is created during the execution of the CREATE DATABASE statement, the NLS-related database configuration is established. The current NLS instance parameters are stored in the data dictionary along with the database and national character sets. The NLS instance parameters are read from the initialization parameter file at instance startup.

You can find the values for NLS parameters by using:

- [NLS Data Dictionary Views](#)
- [NLS Dynamic Performance Views](#)
- [OCINlsGetInfo\(\) Function](#)

3.4.1 NLS Data Dictionary Views

Applications can check the session, instance, and database NLS parameters by querying the following data dictionary views:

- NLS_SESSION_PARAMETERS shows the NLS parameters and their values for the session that is querying the view. It does not show information about the character set.
- NLS_INSTANCE_PARAMETERS shows the current NLS instance parameters that have been explicitly set and the values of the NLS instance parameters.
- NLS_DATABASE_PARAMETERS shows the values of the NLS parameters for the database. The values are stored in the database.

3.4.2 NLS Dynamic Performance Views

Applications can check the following NLS dynamic performance views:

- V\$NLS_VALID_VALUES lists values for the following NLS parameters:

NLS_LANGUAGE
NLS_SORT
NLS_TERRITORY
NLS_CHARACTERSET

- V\$NLS_PARAMETERS shows current values of the following NLS parameters:

NLS_CHARACTERSET
NLS_NCHAR_CHARACTERSET
NLS_NUMERIC_CHARACTERS
NLS_DATE_FORMAT
NLS_DATE_LANGUAGE
NLS_TIME_TZ_FORMAT
NLS_TIMESTAMP_FORMAT
NLS_TIMESTAMP_TZ_FORMAT
NLS_CALENDAR
NLS_LANGUAGE
NLS_CURRENCY
NLS_ISO_CURRENCY

NLS_TERRITORY
NLS_SORT
NLS_COMP
NLS_LENGTH_SEMANTICS
NLS_NCHAR_CONV_EXP

① See Also

Oracle AI Database Reference

3.4.3 OCINlsGetInfo() Function

User applications can query client NLS settings with the `OCINlsGetInfo()` function.

① See Also

"[Getting Locale Information in OCI](#)" for the description of `OCINlsGetInfo()`

3.5 Language and Territory Parameters

This section contains information about the following parameters:

- [NLS_LANGUAGE](#)
- [NLS_TERRITORY](#)

3.5.1 NLS_LANGUAGE

Property	Description
Parameter type	String
Parameter scope	Initialization parameter and ALTER SESSION
Default value	Derived from NLS_LANG
Range of values	Any valid language name

NLS_LANGUAGE specifies the default conventions for the following session characteristics:

- Language for server messages
- Language for day and month names and their abbreviations (specified in the SQL functions `TO_CHAR` and `TO_DATE`)
- Symbols for equivalents of AM, PM, AD, and BC. (A.M., P.M., A.D., and B.C. are valid only if NLS_LANGUAGE is set to AMERICAN.)
- Default sorting sequence for character data when ORDER BY is specified. (GROUP BY uses a binary sort unless ORDER BY is specified.)
- Writing direction
- Affirmative and negative response strings (for example, YES and NO)

The value specified for `NLS_LANGUAGE` in the initialization parameter file is the default for all sessions in that instance. For example, to specify the default session language as French, the parameter should be set as follows:

```
NLS_LANGUAGE = FRENCH
```

Consider the following server message:

```
ORA-00942: table or view does not exist
```

When the language is French, the server message appears as follows:

```
ORA-00942: table ou vue inexistante
```

Messages used by the server are stored in binary-format files that are placed in the `$ORACLE_HOME/product_name/mesg` directory, or the equivalent for your operating system. Multiple versions of these files can exist, one for each supported language, using the following file name convention:

```
<product_id><language_abbrev>.msb
```

For example, the file containing the server messages in French is called `oraf.msb`, because `ORA` is the product ID (`<product_id>`) and `F` is the language abbreviation (`<language_abbrev>`) for French. The `product_name` is `rdbs`, so it is in the `$ORACLE_HOME/rdbs/mesg` directory.

If `NLS_LANG` is specified in the client environment, then the value of `NLS_LANGUAGE` in the initialization parameter file is overridden at connection time.

Messages are stored in these files in one specific character set, depending on the language and the operating system. If this character set is different from the database character set, then message text is automatically converted to the database character set. If necessary, it is then converted to the client character set if the client character set is different from the database character set. Hence, messages are displayed correctly at the user's terminal, subject to the limitations of character set conversion.

The language-specific binary message files that are actually installed depend on the languages that the user specifies during product installation. Only the English binary message file and the language-specific binary message files specified by the user are installed.

The default value of `NLS_LANGUAGE` may be specific to the operating system. You can alter the `NLS_LANGUAGE` parameter by changing its value in the initialization parameter file and then restarting the instance.

① See Also

Your operating system-specific Oracle database documentation for more information about the default value of `NLS_LANGUAGE`

All messages and text should be in the same language. For example, when you run an Oracle Developer application, the messages and boilerplate text that you see originate from three sources:

- Messages from the server
- Messages and boilerplate text generated by Oracle Forms
- Messages and boilerplate text generated by the application

NLS_LANGUAGE determines the language used for the first two kinds of text. The application is responsible for the language used in its messages and boilerplate text.

See Also

["Overriding Default Values for NLS_LANGUAGE and NLS_TERRITORY During a Session"](#) for more information about using the ALTER SESSION statement

The following examples show behavior that results from setting NLS_LANGUAGE to different values.

Example 3-3 NLS_LANGUAGE=ITALIAN

Use the ALTER SESSION statement to set NLS_LANGUAGE to Italian:

```
SQL> ALTER SESSION SET NLS_LANGUAGE=Italian;
```

Enter a SELECT statement:

```
SQL> SELECT last_name, hire_date, ROUND(salary/8,2) salary FROM employees;
```

You should see results similar to the following output:

LAST_NAME	HIRE_DATE	SALARY
...		
Sciarra	30-SET-05	962.5
Urman	07-MAR-06	975
Popp	07-DIC-07	862.5
...		

Note that the month name abbreviations are in Italian.

Example 3-4 NLS_LANGUAGE=GERMAN

Use the ALTER SESSION statement to change the language to German:

```
SQL> ALTER SESSION SET NLS_LANGUAGE=German;
```

Enter the same SELECT statement:

```
SQL> SELECT last_name, hire_date, ROUND(salary/8,2) salary FROM employees;
```

You should see results similar to the following output:

LAST_NAME	HIRE_DATE	SALARY
...		
Sciarra	30-SEP-05	962.5
Urman	07-MRZ-06	975
Popp	07-DEZ-07	862.5
...		

Note that the language of the month abbreviations has changed.

3.5.2 NLS_TERRITORY

Property	Description
Parameter type	String
Parameter scope	Initialization parameter and ALTER SESSION
Default value	Derived from NLS_LANG
Range of values	Any valid territory name

NLS_TERRITORY specifies the conventions for the following default date and numeric formatting characteristics:

- Date format
- Decimal character and group separator
- Local currency symbol
- ISO currency symbol
- Dual currency symbol
- First day of the week
- Credit and debit symbols
- ISO week flag
- List separator

The value specified for NLS_TERRITORY in the initialization parameter file is the default for the instance. For example, to specify the default as France, the parameter should be set as follows:

```
NLS_TERRITORY = FRANCE
```

When the territory is FRANCE, numbers are formatted using a comma as the decimal character.

You can alter the NLS_TERRITORY parameter by changing the value in the initialization parameter file and then restarting the instance. The default value of NLS_TERRITORY can be specific to the operating system.

If NLS_LANG is specified in the client environment, then the value of NLS_TERRITORY in the initialization parameter file is overridden at connection time.

The territory can be modified dynamically during the session by specifying the new NLS_TERRITORY value in an ALTER SESSION statement. Modifying NLS_TERRITORY resets all derived NLS session parameters to default values for the new territory.

To change the territory to France during a session, issue the following ALTER SESSION statement:

```
SQL> ALTER SESSION SET NLS_TERRITORY = France;
```

① See Also

["Overriding Default Values for NLS_LANGUAGE and NLS_TERRITORY During a Session"](#) for more information about using the ALTER SESSION statement

The following examples show behavior that results from different settings of NLS_TERRITORY and NLS_LANGUAGE.

Example 3-5 NLS_LANGUAGE=AMERICAN, NLS_TERRITORY=AMERICA

Enter the following SELECT statement:

```
SQL> SELECT TO_CHAR(salary, 'L99G999D99') salary FROM employees;
```

When NLS_TERRITORY is set to AMERICA and NLS_LANGUAGE is set to AMERICAN, results similar to the following should appear:

```
SALARY
-----
$24,000.00
$17,000.00
$17,000.00
```

Example 3-6 NLS_LANGUAGE=AMERICAN, NLS_TERRITORY=GERMANY

Use an ALTER SESSION statement to change the territory to Germany:

```
SQL> ALTER SESSION SET NLS_TERRITORY = Germany;
Session altered.
```

Enter the same SELECT statement as before:

```
SQL> SELECT TO_CHAR(salary, 'L99G999D99') salary FROM employees;
```

You should see results similar to the following output:

```
SALARY
-----
€24.000,00
€17.000,00
€17.000,00
```

Note that the currency symbol has changed from \$ to €. The numbers have not changed because the underlying data is the same.

Example 3-7 NLS_LANGUAGE=GERMAN, NLS_TERRITORY=GERMANY

Use an ALTER SESSION statement to change the language to German:

```
SQL> ALTER SESSION SET NLS_LANGUAGE = German;
Session wurde geändert.
```

Note that the server message now appears in German.

Enter the same SELECT statement as before:

```
SQL> SELECT TO_CHAR(salary, 'L99G999D99') salary FROM employees;
```

You should see the same results as in [Example 3-6](#):

```
SALARY
-----
€24.000,00
€17.000,00
€17.000,00
```

Example 3-8 NLS_LANGUAGE=GERMAN, NLS_TERRITORY=AMERICA

Use an ALTER SESSION statement to change the territory to America:

```
SQL> ALTER SESSION SET NLS_TERRITORY = America;
Session wurde geändert.
```

Enter the same SELECT statement as in the other examples:

```
SQL> SELECT TO_CHAR(salary, 'L99G999D99') salary FROM employees;
```

You should see results similar to the following output:

```
SALARY
-----
$24,000.00
$17,000.00
$17,000.00
```

Note that the currency symbol changed from € to \$ because the territory changed from Germany to America.

3.5.2.1 Overriding Default Values for NLS_LANGUAGE and NLS_TERRITORY During a Session

Default values for NLS_LANGUAGE and NLS_TERRITORY and default values for specific formatting parameters can be overridden during a session by using the ALTER SESSION statement.

Example 3-9 NLS_LANG=ITALIAN_ITALY.WE8DEC

Set the NLS_LANG environment variable so that the language is Italian, the territory is Italy, and the character set is WE8DEC:

```
% setenv NLS_LANG Italian_Italy.WE8DEC
```

Enter a SELECT statement:

```
SQL> SELECT last_name, hire_date, ROUND(salary/8,2) salary FROM employees;
```

You should see results similar to the following output:

LAST_NAME	HIRE_DATE	SALARY
...		
Sciarra	30-SET-05	962,5
Urman	07-MAR-06	975
Popp	07-DIC-07	862,5
...		

Note the language of the month abbreviations and the decimal character.

Example 3-10 Change Language, Date Format, and Decimal Character

Use ALTER SESSION statements to change the language, the date format, and the decimal character:

```
SQL> ALTER SESSION SET NLS_LANGUAGE=german;
```

Session wurde geändert.

```
SQL> ALTER SESSION SET NLS_DATE_FORMAT='DD.MON.YY';
```

Session wurde geändert.

```
SQL> ALTER SESSION SET NLS_NUMERIC_CHARACTERS='.,';
```

Session wurde geändert.

Enter the SELECT statement shown in [Example 3-9](#):

```
SQL> SELECT last_name, hire_date, ROUND(salary/8,2) salary FROM employees;
```

You should see results similar to the following output:

LAST_NAME	HIRE_DATE	SALARY
...		
Sciarra	30.SEP.05	962.5
Urman	07.MRZ.06	975
Popp	07.DEZ.07	862.5
...		

Note that the language of the month abbreviations is German and the decimal character is a period.

The behavior of the NLS_LANG environment variable implicitly determines the language environment of the database for each session. When a session connects to a database, an ALTER SESSION statement is automatically executed to set the values of the database parameters NLS_LANGUAGE and NLS_TERRITORY to those specified by the language and territory arguments of NLS_LANG. If NLS_LANG is not defined, then no implicit ALTER SESSION statement is executed.

When NLS_LANG is defined, the implicit ALTER SESSION is executed for all instances to which the session connects, for both direct and indirect connections. If the values of NLS parameters are changed explicitly with ALTER SESSION during a session, then the changes are propagated to all instances to which that user session is connected.

3.6 Date and Time Parameters

Oracle databases enable you to control the display of date and time. This section contains the following topics:

- [Date Formats](#)
- [Time Formats](#)

3.6.1 Date Formats

Different Oracle database date formats are shown in the following table.

Table 3-4 Date Formats

Country	Description	Example
Estonia	dd.mm.yyyy	28.02.2003
Germany	dd-mm-rr	28-02-03
Japan	rr-mm-dd	03-02-28
UK	dd-mon-rr	28-Feb-03
US	dd-mon-rr	28-Feb-03

This section includes the following parameters:

- [NLS_DATE_FORMAT](#)
- [NLS_DATE_LANGUAGE](#)

3.6.1.1 NLS_DATE_FORMAT

Property	Description
Parameter type	String
Parameter scope	Initialization parameter, environment variable, and ALTER SESSION
Default value	Derived from NLS_TERRITORY
Range of values	Any valid date format mask

The NLS_DATE_FORMAT parameter defines the default date format to use with the TO_CHAR and TO_DATE functions. The NLS_TERRITORY parameter determines the default value of NLS_DATE_FORMAT. The value of NLS_DATE_FORMAT can be any valid date format mask. For example:

```
NLS_DATE_FORMAT = "MM/DD/YYYY"
```

To add string literals to the date format, enclose the string literal with double quotes. Note that when double quotes are included in the date format, the entire value must be enclosed by single quotes. For example:

```
NLS_DATE_FORMAT = '"Date: "MM/DD/YYYY'
```

The value of NLS_DATE_FORMAT is stored in the internal date format. Each format element occupies two bytes, and each string occupies the number of bytes in the string plus a terminator byte. Also, the entire format mask has a two-byte terminator. For example, "MM/DD/YY" occupies 14 bytes internally because there are three format elements (month, day, and year), two 3-byte strings (the two slashes), and the two-byte terminator for the format mask. The format for the value of NLS_DATE_FORMAT cannot exceed 24 bytes.

You can alter the default value of NLS_DATE_FORMAT by:

- Changing its value in the initialization parameter file and then restarting the instance
- Using an ALTER SESSION SET NLS_DATE_FORMAT statement

See Also

Oracle AI Database SQL Language Reference for more information about date format elements and the ALTER SESSION statement

If a table or index is partitioned on a date column, and if the date format specified by NLS_DATE_FORMAT does not specify the first two digits of the year, then you must use the TO_DATE function with a 4-character format mask for the year. For example:

```
TO_DATE('11-jan-1997', 'dd-mon-yyyy')
```

See Also

Oracle AI Database SQL Language Reference for more information about partitioning tables and indexes and using TO_DATE

Example 3-11 Setting the Date Format to Display Roman Numerals

To set the default date format to display Roman numerals for the month, include the following line in the initialization parameter file:

```
NLS_DATE_FORMAT = "DD RM YYYY"
```

Enter the following SELECT statement:

```
SQL> SELECT TO_CHAR(SYSDATE) currdate FROM DUAL;
```

You should see the following output if today's date is February 12, 1997:

```
CURRDATE  
-----  
12 II 1997
```

3.6.1.2 NLS_DATE_LANGUAGE

Property	Description
Parameter type	String
Parameter scope	Initialization parameter, environment variable, ALTER SESSION, and SQL functions
Default value	Derived from NLS_LANGUAGE
Range of values	Any valid language name

The NLS_DATE_LANGUAGE parameter specifies the language for the day and month names produced by the TO_CHAR and TO_DATE functions. NLS_DATE_LANGUAGE overrides the language that is specified implicitly by NLS_LANGUAGE. NLS_DATE_LANGUAGE has the same syntax as the NLS_LANGUAGE parameter, and all supported languages are valid values.

NLS_DATE_LANGUAGE also determines the language used for:

- Month and day abbreviations returned by the TO_CHAR and TO_DATE functions
- Month and day abbreviations used by the default date format (NLS_DATE_FORMAT)
- Abbreviations for AM, PM, AD, and BC

 See Also

Oracle AI Database SQL Language Reference

Example 3-12 NLS_DATE_LANGUAGE=FRENCH, Month and Day Names

As an example of how to use NLS_DATE_LANGUAGE, set the date language to French:

```
SQL> ALTER SESSION SET NLS_DATE_LANGUAGE = FRENCH;
```

Enter a SELECT statement:

```
SQL> SELECT TO_CHAR(SYSDATE, 'Day:Dd Month yyyy') FROM DUAL;
```

You should see results similar to the following output:

```
TO_CHAR(SYSDATE, 'DAY:DDMONTHYYYY')
```

```
-----  
Vendredi:07 Décembre 2001
```

When numbers are spelled in words using the TO_CHAR function, the English spelling is always used. For example, enter the following SELECT statement:

```
SQL> SELECT TO_CHAR(TO_DATE('12-Oct.-2001'),'Day: ddspth Month') FROM DUAL;
```

You should see results similar to the following output:

```
TO_CHAR(TO_DATE('12-OCT.-2001'),'DAY:DDSPTHMONTH')
```

```
-----  
Vendredi: twelfth Octobre
```

Example 3-13 NLS_DATE_LANGUAGE=FRENCH, Month and Day Abbreviations

Month and day abbreviations are determined by NLS_DATE_LANGUAGE. Enter the following SELECT statement:

```
SQL> SELECT TO_CHAR(SYSDATE, 'Dy:dd Mon yyyy') FROM DUAL;
```

You should see results similar to the following output:

```
TO_CHAR(SYSDATE, 'DY:DDMO
```

```
-----  
Ve:07 Déc. 2001
```

Example 3-14 NLS_DATE_LANGUAGE=FRENCH, Default Date Format

The default date format uses the month abbreviations determined by NLS_DATE_LANGUAGE. For example, if the default date format is DD-MON-YYYY, then insert a date as follows:

```
SQL> INSERT INTO tablename VALUES ('12-Févr.-1997');
```

3.6.2 Time Formats

Different Oracle database time formats are shown in the following table.

Table 3-5 Time Formats

Country	Description	Example
Estonia	hh24:mi:ss	13:50:23
Germany	hh24:mi:ss	13:50:23
Japan	hh24:mi:ss	13:50:23
UK	hh24:mi:ss	13:50:23
US	hh:mi:ssx ff am	1:50:23.555 PM

This section contains information about the following parameters:

- [NLS_TIMESTAMP_FORMAT](#)
- [NLS_TIMESTAMP_TZ_FORMAT](#)

See Also

["Datetime Data Types and Time Zone Support"](#)

3.6.2.1 NLS_TIMESTAMP_FORMAT

Property	Description
Parameter type	String
Parameter scope	Initialization parameter, environment variable, and ALTER SESSION
Default value	Derived from NLS_TERRITORY
Range of values	Any valid datetime format mask

NLS_TIMESTAMP_FORMAT defines the default date format for the TIMESTAMP and TIMESTAMP WITH LOCAL TIME ZONE data types. The following example shows a value for NLS_TIMESTAMP_FORMAT:

```
NLS_TIMESTAMP_FORMAT = 'YYYY-MM-DD HH:MI:SS.FF'
```

You can specify the value of NLS_TIMESTAMP_FORMAT by setting it in the initialization parameter file. You can specify its value for a client as a client environment variable.

You can also alter the value of NLS_TIMESTAMP_FORMAT by:

- Changing its value in the initialization parameter file and then restarting the instance
- Using the ALTER SESSION SET NLS_TIMESTAMP_FORMAT statement

See Also

Oracle AI Database SQL Language Reference for more information about the TO_TIMESTAMP function and the ALTER SESSION statement

Example 3-15 Timestamp Format

```
SQL> SELECT TO_TIMESTAMP('11-nov-2000 01:00:00.336', 'dd-mon-yyyy hh:mi:ss.ff') FROM DUAL;
```

You should see results similar to the following output:

```
TO_TIMESTAMP('11-NOV-200001:00:00.336', 'DD-MON-YYYYHH:MI:SS.FF')
-----
2000-11-11 01:00:00.336000000
```

3.6.2.2 NLS_TIMESTAMP_TZ_FORMAT

Property	Description
Parameter type	String
Parameter scope	Initialization parameter, environment variable, and ALTER SESSION
Default value	Derived from NLS_TERRITORY

Property	Description
Range of values	Any valid datetime format mask

NLS_TIMESTAMP_TZ_FORMAT defines the default date format for the TIMESTAMP and TIMESTAMP WITH LOCAL TIME ZONE data types. It is used with the TO_CHAR and TO_TIMESTAMP_TZ functions.

You can specify the value of NLS_TIMESTAMP_TZ_FORMAT by setting it in the initialization parameter file. You can specify its value for a client as a client environment variable.

You can change the value of NLS_TIMESTAMP_TZ_FORMAT by:

- Changing its value in the initialization parameter file and then restarting the instance
- Using the ALTER SESSION statement.

① See Also

- *Oracle AI Database SQL Language Reference* for more information about the TO_TIMESTAMP_TZ function and the ALTER SESSION statement
- "[Choosing a Time Zone File](#)" for more information about time zones

Example 3-16 Setting NLS_TIMESTAMP_TZ_FORMAT

The format value must be surrounded by quotation marks. For example:

```
NLS_TIMESTAMP_TZ_FORMAT = 'YYYY-MM-DD HH:MI:SS.FF TZH:TZM'
```

The following example of the TO_TIMESTAMP_TZ function uses the format value that was specified for NLS_TIMESTAMP_TZ_FORMAT:

```
SQL> SELECT TO_TIMESTAMP_TZ('2000-08-20, 05:00:00.55 America/Los_Angeles', 'yyyy-mm-dd
hh:mi:ss.ff TZR') FROM DUAL;
```

You should see results similar to the following output:

```
TO_TIMESTAMP_TZ('2000-08-20,05:00:00.55AMERICA/LOS_ANGELES','YYYY-MM-DDHH:M
-----
2000-08-20 05:00:00.550000000 -07:00
```

3.7 Calendar Definitions

This section includes the following topics:

- [Calendar Formats](#)
- [NLS_CALENDAR](#)

3.7.1 Calendar Formats

The following calendar information is stored for each territory:

- [First Day of the Week](#)
- [First Calendar Week of the Year](#)
- [Number of Days and Months in a Year](#)

- [First Year of Era](#)

3.7.1.1 First Day of the Week

Some cultures consider Sunday to be the first day of the week. Others consider Monday to be the first day of the week. A German calendar starts with Monday, as shown in the following table.

Table 3-6 German Calendar Example: March 1998

Mo	Di	Mi	Do	Fr	Sa	So
-	-	-	-	-	-	1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29
30	31	-	-	-	-	-

The first day of the week is determined by the NLS_TERRITORY parameter.

① See Also

["NLS_TERRITORY"](#)

3.7.1.2 First Calendar Week of the Year

Some countries use week numbers for scheduling, planning, and bookkeeping. Oracle databases support this convention. In the ISO standard, the week number can be different from the week number of the calendar year. For example, 1st Jan 1988 is in ISO week number 53 of 1987. An ISO week always starts on a Monday and ends on a Sunday.

- If January 1 falls on a Friday, Saturday, or Sunday, then the ISO week that includes January 1 is the last week of the previous year, because most of the days in the week belong to the previous year.
- If January 1 falls on a Monday, Tuesday, Wednesday, or Thursday, then the ISO week is the first week of the new year, because most of the days in the week belong to the new year.

To support the ISO standard, Oracle provides the IW date format element. It returns the ISO week number.

The following table shows an example in which January 1 occurs in a week that has four or more days in the first calendar week of the year. The week containing January 1 is the first ISO week of 1998.

Table 3-7 First ISO Week of the Year: Example 1, January 1998

Mo	Tu	We	Th	Fr	Sa	Su	ISO Week
-	-	-	1	2	3	4	First ISO week of 1998
5	6	7	8	9	10	11	Second ISO week of 1998

Table 3-7 (Cont.) First ISO Week of the Year: Example 1, January 1998

Mo	Tu	We	Th	Fr	Sa	Su	ISO Week
12	13	14	15	16	17	18	Third ISO week of 1998
19	20	21	22	23	24	25	Fourth ISO week of 1998
26	27	28	29	30	31	-	Fifth ISO week of 1998

The following table shows an example in which January 1 occurs in a week that has three or fewer days in the first calendar week of the year. The week containing January 1 is the 53rd ISO week of 1998, and the following week is the first ISO week of 1999.

Table 3-8 First ISO Week of the Year: Example 2, January 1999

Mo	Tu	We	Th	Fr	Sa	Su	ISO Week
-	-	-	-	1	2	3	Fifty-third ISO week of 1998
4	5	6	7	8	9	10	First ISO week of 1999
11	12	13	14	15	16	17	Second ISO week of 1999
18	19	20	21	22	23	24	Third ISO week of 1999
25	26	27	28	29	30	31	Fourth ISO week of 1999

The first calendar week of the year is determined by the NLS_TERRITORY parameter.

 See Also

["NLS_TERRITORY"](#)

3.7.1.3 Number of Days and Months in a Year

Oracle Database supports six calendar systems in addition to Gregorian, the default:

- Japanese Imperial—uses the same number of months and days as Gregorian, but the year starts with the beginning of each Imperial Era.
- ROC Official—uses the same number of months and days as Gregorian, but the year starts with the founding of the Republic of China.
- Persian—has 31 days for each of the first six months. The next five months have 30 days each. The last month has either 29 days or 30 days (leap year).
- Thai Buddha—uses a Buddhist calendar
- Arabic Hijrah—has 12 months with 354 or 355 days
- English Hijrah—has 12 months with 354 or 355 days
- Ethiopian—has 12 months of 30 days each, then a 13th month that is either five or six days (leap year). The sixth day of the 13th month is added every four years.

The calendar system is specified by the NLS_CALENDAR parameter.

① See Also["NLS_CALENDAR"](#)

3.7.1.4 First Year of Era

The Islamic calendar and the Japanese Imperial calendar are based on the first year of an era.

3.7.1.4.1 Islamic Calendar

The Islamic calendar starts from the year of the Hegira.

3.7.1.4.2 Japanese Imperial Calendar

The Japanese Imperial calendar starts from the beginning of an Emperor's reign. For example:

- January 8, 1989, through December 31, 1989, is the first year of the Heisei era (Heisei 1).
- 2018 is the thirtieth year of the Heisei era (Heisei 30).
- January 1, 2019, through April 30, 2019, is the thirty-first year of the Heisei era (Heisei 31).
- May 1, 2019, through December 31, 2019, is the first year of the Reiwa era (Reiwa 1).
- 2020 is the second year of the Reiwa era (Reiwa 2).

It should be noted that the Gregorian system is also widely understood in Japan. So, for example, both 18 and Heisei 30 can be used to represent 2018.

3.7.2 NLS_CALENDAR

Property	Description
Parameter type	String
Parameter scope	Initialization parameter, environment variable, ALTER SESSION, and SQL functions
Default value	None, implies GREGORIAN
Range of values	Any valid calendar format name

Many different calendar systems are in use throughout the world. NLS_CALENDAR specifies which calendar system an Oracle database uses.

NLS_CALENDAR can have one of the following values:

- Arabic Hijrah
- English Hijrah
- Ethiopian
- GREGORIAN
- Japanese Imperial
- Persian
- ROC Official (Republic of China)

- Thai Buddha

See Also

"[Calendar Systems](#)" for a list of calendar systems, their default date formats, and the character sets in which dates are displayed

Example 3-17 NLS_CALENDAR='English Hijrah'

Set NLS_CALENDAR to English Hijrah.

```
SQL> ALTER SESSION SET NLS_CALENDAR='English Hijrah';
```

Enter a SELECT statement to display SYSDATE:

```
SQL> SELECT SYSDATE FROM DUAL;
```

You should see results similar to the following output:

```
SYSDATE
-----
24 Ramadan      1422
```

3.8 Numeric and List Parameters

This section includes the following topics:

- [Numeric Formats](#)
- [NLS_NUMERIC_CHARACTERS](#)
- [NLS_LIST_SEPARATOR](#)

3.8.1 Numeric Formats

The database must know the number-formatting convention used in each session to interpret numeric strings correctly. For example, the database needs to know whether numbers are entered with a period or a comma as the decimal character (234.00 or 234,00). Similarly, applications must be able to display numeric information in the format expected at the client site.

Examples of numeric formats are shown in the following table.

Table 3-9 Examples of Numeric Formats

Country	Numeric Formats
Estonia	1 234 567,89
Germany	1.234.567,89
Japan	1,234,567.89
UK	1,234,567.89
US	1,234,567.89

Numeric formats are derived from the setting of the NLS_TERRITORY parameter, but they can be overridden by the NLS_NUMERIC_CHARACTERS parameter.

See Also["NLS_TERRITORY"](#)

3.8.2 NLS_NUMERIC_CHARACTERS

Property	Description
Parameter type	String
Parameter scope	Initialization parameter, environment variable, ALTER SESSION, and SQL functions
Default value	Default decimal character and group separator for a particular territory
Range of values	Any two valid numeric characters

This parameter specifies the decimal character and group separator. The group separator is the character that separates integer groups to show thousands and millions, for example. The group separator is the character returned by the G number format mask. The decimal character separates the integer and decimal parts of a number. Setting NLS_NUMERIC_CHARACTERS overrides the values derived from the setting of NLS_TERRITORY.

Any character can be the decimal character or group separator. The two characters specified must be single-byte, and the characters must be different from each other. The characters cannot be any numeric character or any of the following characters: plus (+), hyphen (-), less than sign (<), greater than sign (>). Either character can be a space.

You can change the default value of NLS_NUMERIC_CHARACTERS by:

- Changing the value of NLS_NUMERIC_CHARACTERS in the initialization parameter file and then restarting the instance
- Using the ALTER SESSION statement to change the parameter's value during a session

See Also

Oracle AI Database SQL Language Reference for more information about the ALTER SESSION statement

Example 3-18 Setting NLS_NUMERIC_CHARACTERS

To set the decimal character to a comma and the grouping separator to a period, define NLS_NUMERIC_CHARACTERS as follows:

```
SQL> ALTER SESSION SET NLS_NUMERIC_CHARACTERS = ',.';
```

SQL statements can include numbers represented as numeric or text literals. Numeric literals are not enclosed in quotes. They are part of the SQL language syntax and always use a dot as the decimal character and never contain a group separator. Text literals are enclosed in single quotes. They are implicitly or explicitly converted to numbers, if required, according to the current NLS settings.

The following SELECT statement formats the number 4000 with the decimal character and group separator specified in the ALTER SESSION statement:

```
SQL> SELECT TO_CHAR(4000, '9G999D99') FROM DUAL;
```

You should see results similar to the following output:

```
TO_CHAR(4
-----
4.000,00
```

3.8.3 NLS_LIST_SEPARATOR

Property	Description
Parameter type	String
Parameter scope	Environment variable
Default value	Derived from NLS_TERRITORY
Range of values	Any valid character

NLS_LIST_SEPARATOR specifies the character to use to separate values in a list of values (usually , or . or ; or :). Its default value is derived from the value of NLS_TERRITORY. For example, a list of numbers from 1 to 5 can be expressed as 1,2,3,4,5 or 1.2.3.4.5 or 1;2;3;4;5 or 1:2:3:4:5.

The character specified must be single-byte and cannot be the same as either the numeric or monetary decimal character, any numeric character, or any of the following characters: plus (+), hyphen (-), less than sign (<), greater than sign (>), period (.).

3.9 Monetary Parameters

This section includes the following topics:

- [Currency Formats](#)
- [NLS_CURRENCY](#)
- [NLS_ISO_CURRENCY](#)
- [NLS_DUAL_CURRENCY](#)
- [NLS_MONETARY_CHARACTERS](#)
- [NLS_CREDIT](#)
- [NLS_DEBIT](#)

3.9.1 Currency Formats

Different currency formats are used throughout the world. Some typical ones are shown in the following table.

Table 3-10 Currency Format Examples

Country	Example
Brazil	R\$1,234.56
Germany	1.234,56€
Japan	¥1,234.56
UK	£1,234.56
US	\$1,234.56

3.9.2 NLS_CURRENCY

Property	Description
Parameter type	String
Parameter scope	Initialization parameter, environment variable, ALTER SESSION, and SQL functions
Default value	Derived from NLS_TERRITORY
Range of values	Any valid currency symbol string

NLS_CURRENCY specifies the character string returned by the L number format mask, the local currency symbol. Setting NLS_CURRENCY overrides the setting defined implicitly by NLS_TERRITORY.

You can change the default value of NLS_CURRENCY by:

- Changing its value in the initialization parameter file and then restarting the instance
- Using an ALTER SESSION statement

See Also

Oracle AI Database SQL Language Reference for more information about the ALTER SESSION statement

Example 3-19 Displaying the Local Currency Symbol

Connect to the sample order entry schema:

```
SQL> connect oe/oe
Connected.
```

Enter a SELECT statement similar to the following example:

```
SQL> SELECT TO_CHAR(order_total, 'L099G999D99') "total" FROM orders
       WHERE order_id > 2450;
```

You should see results similar to the following output:

```
total
-----
      $078,279.60
      $006,653.40
      $014,087.50
      $010,474.60
      $012,589.00
      $000,129.00
      $003,878.40
      $021,586.20
```

3.9.3 NLS_ISO_CURRENCY

Property	Description
Parameter type	String

Property	Description
Parameter scope	Initialization parameter, environment variable, ALTER SESSION, and SQL functions
Default value	Derived from NLS_TERRITORY
Range of values	Any valid string

NLS_ISO_CURRENCY specifies the character string returned by the C number format mask, the ISO currency symbol. Setting NLS_ISO_CURRENCY overrides the value defined implicitly by NLS_TERRITORY.

Local currency symbols can be ambiguous. For example, a dollar sign (\$) can refer to US dollars or Australian dollars. ISO specifications define unique currency symbols for specific territories or countries. For example, the ISO currency symbol for the US dollar is USD. The ISO currency symbol for the Australian dollar is AUD.

More ISO currency symbols are shown in the following table.

Table 3-11 ISO Currency Examples

Country	Example
Brazil	1,234,567.89 BRL
Germany	1.234.567,89 EUR
Japan	1,234,567.89 JPY
UK	1,234,567.89 GBP
US	1,234,567.89 USD

NLS_ISO_CURRENCY has the same syntax as the NLS_TERRITORY parameter, and all supported territories are valid values.

You can change the default value of NLS_ISO_CURRENCY by:

- Changing its value in the initialization parameter file and then restarting the instance
- Using an ALTER SESSION statement

① See Also

Oracle AI Database SQL Language Reference for more information about the ALTER SESSION statement

Example 3-20 Setting NLS_ISO_CURRENCY

This example assumes that you are connected as oe/oe in the sample schema.

To specify the ISO currency symbol for France, set NLS_ISO_CURRENCY as follows:

```
SQL> ALTER SESSION SET NLS_ISO_CURRENCY = FRANCE;
```

Enter a SELECT statement:

```
SQL> SELECT TO_CHAR(order_total, 'C099G999D99') "TOTAL" FROM orders
       WHERE customer_id = 146;
```

You should see results similar to the following output:

```
TOTAL
-----
EUR017,848.20
EUR027,455.30
EUR029,249.10
EUR013,824.00
EUR000,086.00
```

3.9.4 NLS_DUAL_CURRENCY

Property	Description
Parameter type	String
Parameter scope	Initialization parameter, environmental variable, ALTER SESSION, and SQL functions
Default value	Derived from NLS_TERRITORY
Range of values	Any valid symbol

Use NLS_DUAL_CURRENCY to override the default dual currency symbol defined implicitly by NLS_TERRITORY.

NLS_DUAL_CURRENCY was introduced to support the euro currency symbol during the euro transition period. See [Table A-8](#) for the character sets that support the euro symbol.

3.9.5 Oracle Database Support for the Euro

Twenty members of the European Monetary Union (EMU) have adopted the euro as their currency. Setting NLS_TERRITORY to correspond to a country in the EMU (Austria, Belgium, Croatia, Cyprus, Estonia, Finland, France, Germany, Greece, Ireland, Italy, Latvia, Lithuania, Luxembourg, Malta, the Netherlands, Portugal, Slovakia, Slovenia, or Spain) results in the default values for NLS_CURRENCY and NLS_DUAL_CURRENCY being set to EUR.

During the transition period (1999 through 2001), support for the euro was provided in Oracle Database 8i and later as follows:

- NLS_CURRENCY was defined as the primary currency of the country
- NLS_ISO_CURRENCY was defined as the ISO currency code of a given territory
- NLS_DUAL_CURRENCY was defined as the secondary currency symbol (usually the euro) for a given territory

Beginning with Oracle Database 9i Release 2, the value of NLS_ISO_CURRENCY results in the ISO currency symbol being set to EUR for EMU member countries who use the euro. For example, suppose NLS_ISO_CURRENCY is set to FRANCE. Enter the following SELECT statement:

```
SQL> SELECT TO_CHAR(order_total, 'C099G999D99') "TOTAL" FROM orders
        WHERE customer_id=116;
```

You should see results similar to the following output:

```
TOTAL
-----
EUR006,394.80
EUR011,097.40
EUR014,685.80
EUR000,129.00
```

Customers who must retain their obsolete local currency symbol can override the default for NLS_DUAL_CURRENCY or NLS_CURRENCY by defining them as parameters in the initialization file on the server and as environment variables on the client.

Note

NLS_LANG must also be set on the client for NLS_CURRENCY or NLS_DUAL_CURRENCY to take effect.

It is not possible to override the ISO currency symbol that results from the value of NLS_ISO_CURRENCY.

3.9.6 NLS_MONETARY_CHARACTERS

Property	Description
Parameter type	String
Parameter scope	Environment variable
Default value	Derived from NLS_TERRITORY
Range of values	Any valid character

NLS_MONETARY_CHARACTERS specifies the character that separates groups of numbers in monetary expressions. For example, when the territory is America, the thousands separator is a comma, and the decimal separator is a period.

3.9.7 NLS_CREDIT

Property	Description
Parameter type	String
Parameter scope	Environment variable
Default value	Derived from NLS_TERRITORY
Range of values	Any string, maximum of 9 bytes (not including null)

NLS_CREDIT sets the symbol that displays a credit in financial reports. The default value of this parameter is determined by NLS_TERRITORY. For example, a space is a valid value of NLS_CREDIT.

This parameter can be specified only in the client environment.

It can be retrieved through the `OCINlsGetInfo()` function.

3.9.8 NLS_DEBIT

Property	Description
Parameter type	String
Parameter scope	Environment variable
Default value	Derived from NLS_TERRITORY

Property	Description
Range of values	Any string, maximum of 9 bytes (not including null)

NLS_DEBIT sets the symbol that displays a debit in financial reports. The default value of this parameter is determined by NLS_TERRITORY. For example, a minus sign (-) is a valid value of NLS_DEBIT.

This parameter can be specified only in the client environment.

It can be retrieved through the `OCINlsGetInfo()` function.

3.10 Linguistic Sort Parameters

You can choose how to sort data by using linguistic sort parameters.

This section includes the following topics:

- [NLS_SORT](#)
- [NLS_COMP](#)

See Also

["Linguistic Sorting and Matching"](#)

3.10.1 NLS_SORT

Property	Description
Parameter type	String
Parameter scope	Initialization parameter, environment variable, ALTER SESSION, and SQL functions
Default value	Derived from NLS_LANGUAGE
Range of values	BINARY or any valid linguistic collation name

NLS_SORT specifies a set of matching and comparison rules for character data. It overrides the default value that is derived from NLS_LANGUAGE.

NLS_SORT contains either of the following values:

NLS_SORT = BINARY | *collation_name*

BINARY specifies the binary collation. *collation_name* specifies a linguistic named collation.

Example 3-21 Setting NLS_SORT

To specify the German linguistic collation, set NLS_SORT as follows:

NLS_SORT = German

Note

When the NLS_SORT parameter is set to BINARY, the optimizer can, in some cases, satisfy the ORDER BY clause without doing a sort operation by choosing an index scan.

When NLS_SORT is set to a linguistic collation, a sort operation is needed to satisfy the ORDER BY clause, if there is no linguistic index for the linguistic collation specified by NLS_SORT.

If a linguistic index exists for the linguistic collation specified by NLS_SORT, then the optimizer can, in some cases, satisfy the ORDER BY clause without doing a sort operation by choosing an index scan.

You can alter the default value of NLS_SORT by:

- Changing its value in the initialization parameter file and then restarting the instance
- Using an ALTER SESSION statement

See Also

- "[Linguistic Sorting and Matching](#)"
- *Oracle AI Database SQL Language Reference* for more information about the ALTER SESSION statement
- "[Linguistic Sorts](#)" for a list of linguistic collation names

3.10.2 NLS_COMP

Property	Description
Parameter type	String
Parameter scope	Initialization parameter, environment variable, and ALTER SESSION
Default value	BINARY
Range of values	BINARY , LINGUISTIC, or ANSI

The value of NLS_COMP affects the comparison behavior of SQL operations whose determined collation is USING_NLS_COMP.

See Also

- "[Using Linguistic Collations](#)"
- "[Using Linguistic Indexes](#)"
- "[Performing Linguistic Comparisons](#)"
- "[About Data-Bound Collation](#)" for more information about the pseudo-collation USING_NLS_COMP

3.11 Character Set Conversion Parameter

This section includes the following topic:

- [NLS_NCHAR_CONV_EXCP](#)

3.11.1 NLS_NCHAR_CONV_EXCP

Property	Description
Parameter type	String
Parameter scope	Initialization parameter and ALTER SESSION
Default value	FALSE
Range of values	TRUE or FALSE

NLS_NCHAR_CONV_EXCP determines whether an error is reported when there is data loss during an implicit or explicit character type conversion between NCHAR/NVARCHAR data and CHAR/VARCHAR2 data. The default value results in no error being reported.

① See Also

"[Character Set Migration](#)" for more information about data loss during character set conversion

3.12 Length Semantics

This section includes the following topic:

- [NLS_LENGTH_SEMANTICS](#)

3.12.1 NLS_LENGTH_SEMANTICS

Property	Description
Parameter type	String
Parameter scope	Environment variable, initialization parameter, and ALTER SESSION
Default value	BYTE
Range of values	BYTE or CHAR

By default, the character data types CHAR and VARCHAR2 are specified in bytes, not characters. Hence, the specification CHAR(20) in a table definition allows 20 bytes for storing character data.

This works well if the database character set uses a single-byte character encoding scheme because the number of characters is the same as the number of bytes. If the database character set uses a multibyte character encoding scheme, then the number of bytes no longer equals the number of characters because a character can consist of one or more bytes. Thus, column widths must be chosen with care to allow for the maximum possible number of bytes

for a given number of characters. You can overcome this problem by switching to character semantics when defining the column size.

NLS_LENGTH_SEMANTICS enables you to create CHAR, VARCHAR2, and LONG columns using either byte or character length semantics. NCHAR, NVARCHAR2, CLOB, and NCLOB columns are always character-based. Existing columns are not affected.

You may be required to use byte semantics in order to maintain compatibility with existing applications.

NLS_LENGTH_SEMANTICS does not apply to tables created in the SYS schema. The data dictionary always uses byte semantics. Tables owned by SYS always use byte semantics if the length qualifier BYTE or CHAR is not specified in the table creation DDL.

Note that if the NLS_LENGTH_SEMANTICS environment variable is not set on the client, then the client session defaults to the value for NLS_LENGTH_SEMANTICS on the database server. This enables all client sessions on the network to have the same NLS_LENGTH_SEMANTICS behavior. Setting the environment variable on an individual client enables the server initialization parameter to be overridden for that client.

Note that if the NLS_LENGTH_SEMANTICS environment variable is not set on the client or the client connects through the Oracle JDBC Thin driver, then the client session defaults to the value for the NLS_LENGTH_SEMANTICS initialization parameter of the instance to which the client connects. For compatibility reasons, Oracle recommends that this parameter be left undefined or set to BYTE.

① Note

Oracle strongly recommends that you do NOT set the NLS_LENGTH_SEMANTICS parameter to CHAR in the instance or server parameter file. This may cause many existing installation scripts to unexpectedly create columns with character length semantics, resulting in run-time errors, including buffer overflows.

① See Also

["Length Semantics"](#)

4

Datetime Data Types and Time Zone Support

This chapter includes the following topics:

- [Overview of Datetime and Interval Data Types and Time Zone Support](#)
- [Datetime and Interval Data Types](#)
- [Datetime and Interval Arithmetic and Comparisons](#)
- [Datetime SQL Functions](#)
- [Datetime and Time Zone Parameters and Environment Variables](#)
- [Choosing a Time Zone File](#)
- [Upgrading the Time Zone File and Timestamp with Time Zone Data](#)
- [Clients and Servers Operating with Different Versions of Time Zone Files](#)
- [Setting the Database Time Zone](#)
- [Setting the Session Time Zone](#)
- [Converting Time Zones With the AT TIME ZONE Clause](#)
- [Support for Daylight Saving Time](#)

4.1 Overview of Datetime and Interval Data Types and Time Zone Support

Businesses conduct transactions across different time zones. Oracle database datetime and interval data types and time zone support make it possible to store consistent information about the time of events and transactions.

Note

This chapter describes Oracle database datetime and interval data types. It does not attempt to describe ANSI data types or other kinds of data types unless noted.

4.2 Datetime and Interval Data Types

The **datetime data types** are DATE, TIMESTAMP, TIMESTAMP WITH TIME ZONE, and TIMESTAMP WITH LOCAL TIME ZONE. Values of datetime data types are sometimes called **datetimes**.

The **interval data types** are INTERVAL YEAR TO MONTH and INTERVAL DAY TO SECOND. Values of interval data types are sometimes called **intervals**.

Both datetimes and intervals are made up of fields. The values of these fields determine the value of the data type. The fields that apply to all Oracle database datetime and interval data types are:

- YEAR
- MONTH
- DAY
- HOUR
- MINUTE
- SECOND

TIMESTAMP WITH TIME ZONE also includes these fields:

- TIMEZONE_HOUR
- TIMEZONE_MINUTE
- TIMEZONE_REGION
- TIMEZONE_ABBR

TIMESTAMP WITH LOCAL TIME ZONE does not store time zone information internally, but you can see local time zone information in SQL output if the TZh:TzM or TZR TzD format elements are specified.

The following sections describe the datetime data types and interval data types in more detail:

- [Datetime Data Types](#)
- [Interval Data Types](#)

① See Also

- *Oracle AI Database SQL Language Reference* for the valid values of the datetime and interval fields
- *Oracle AI Database SQL Language Reference* for information about format elements

4.2.1 Datetime Data Types

This section includes the following topics:

- [DATE Data Type](#)
- [TIMESTAMP Data Type](#)
- [TIMESTAMP WITH TIME ZONE Data Type](#)
- [TIMESTAMP WITH LOCAL TIME ZONE Data Type](#)
- [Inserting Values into Datetime Data Types](#)
- [Choosing a TIMESTAMP Data Type](#)

4.2.1.1 DATE Data Type

The DATE data type stores date and time information. Although date and time information can be represented in both character and number data types, the DATE data type has special associated properties. For each DATE value, the database stores the following information: century, year, month, date, hour, minute, and second.

You can specify a date value by:

- Specifying the date value as a literal
- Converting a character or numeric value to a date value with the `T0_DATE` function

A date can be specified as an ANSI date literal or as an Oracle database date value.

An ANSI date literal contains no time portion and must be specified in exactly the following format:

```
DATE 'YYYY-MM-DD'
```

The following is an example of an ANSI date literal:

```
DATE '1998-12-25'
```

Alternatively, you can specify an Oracle database date value as shown in the following example:

```
T0_DATE('1998-DEC-25 17:30', 'YYYY-MON-DD HH24:MI', 'NLS_DATE_LANGUAGE=AMERICAN')
```

The default date format for an Oracle database date value is derived from the `NLS_DATE_FORMAT` and `NLS_DATE_LANGUAGE` initialization parameters. The date format in the example includes a two-digit number for the day of the month, an abbreviation of the month name, the four digits of the year, and a 24-hour time designation. The specification for `NLS_DATE_LANGUAGE` is included because 'DEC' is not a valid value for `MON` in all locales.

The database automatically converts character values that are in the default date format into date values when they are used in date expressions.

If you specify a date value without a time component, then the default time is midnight. If you specify a date value without a date, then the default date is the first day of the current month.

Oracle Database `DATE` columns always contain fields for both date and time. If your queries use a date format without a time portion, then you must ensure that the time fields in the `DATE` column are set to midnight. You can use the `TRUNC (date)` SQL function to ensure that the time fields are set to midnight, or you can make the query a test of greater than or less than (`<`, `<=`, `>=`, or `>`) instead of equality or inequality (`=` or `!=`). Otherwise, the database may not return the query results you expect.

See Also

- *Oracle AI Database SQL Language Reference* for more information about the `DATE` data type
- ["NLS_DATE_FORMAT"](#)
- ["NLS_DATE_LANGUAGE"](#)
- *Oracle AI Database SQL Language Reference* for more information about literals, format elements such as `MM`, and the `T0_DATE` function

4.2.1.2 TIMESTAMP Data Type

The `TIMESTAMP` data type is an extension of the `DATE` data type. It stores year, month, day, hour, minute, and second values. It also stores fractional seconds, which are not stored by the `DATE` data type.

Specify the `TIMESTAMP` data type as follows:

```
TIMESTAMP [(fractional_seconds_precision)]
```

fractional_seconds_precision is optional and specifies the number of digits in the fractional part of the SECOND datetime field. It can be a number in the range 0 to 9. The default is 6.

For example, '26-JUN-02 09:39:16.78' shows 16.78 seconds. The fractional seconds precision is 2 because there are 2 digits in '78'.

You can specify the TIMESTAMP literal in a format like the following:

```
TIMESTAMP 'YYYY-MM-DD HH24:MI:SS.FF'
```

Using the example format, specify TIMESTAMP as a literal as follows:

```
TIMESTAMP '1997-01-31 09:26:50.12'
```

The value of NLS_TIMESTAMP_FORMAT initialization parameter determines the timestamp format when a character string is converted to the TIMESTAMP data type. NLS_DATE_LANGUAGE determines the language used for character data such as MON.

① See Also

- *Oracle AI Database SQL Language Reference* for more information about the TIMESTAMP data type
- ["NLS_TIMESTAMP_FORMAT"](#)
- ["NLS_DATE_LANGUAGE"](#)

4.2.1.3 TIMESTAMP WITH TIME ZONE Data Type

TIMESTAMP WITH TIME ZONE is a variant of TIMESTAMP that includes a time zone region name or time zone offset in its value. The time zone offset is the difference (in hours and minutes) between local time and UTC (Coordinated Universal Time, formerly Greenwich Mean Time). Specify the TIMESTAMP WITH TIME ZONE data type as follows:

```
TIMESTAMP [(fractional_seconds_precision)] WITH TIME ZONE
```

fractional_seconds_precision is optional and specifies the number of digits in the fractional part of the SECOND datetime field.

You can specify TIMESTAMP WITH TIME ZONE as a literal as follows:

```
TIMESTAMP '1997-01-31 09:26:56.66 +02:00'
```

Two TIMESTAMP WITH TIME ZONE values are considered identical if they represent the same instant in UTC, regardless of the TIME ZONE offsets stored in the data. For example, the following expressions have the same value:

```
TIMESTAMP '1999-01-15 8:00:00 -8:00'  
TIMESTAMP '1999-01-15 11:00:00 -5:00'
```

You can replace the UTC offset with the TZR (time zone region) format element. The following expression specifies America/Los_Angeles for the time zone region:

```
TIMESTAMP '1999-01-15 8:00:00 America/Los_Angeles'
```

To eliminate the ambiguity of boundary cases when the time switches from Standard Time to Daylight Saving Time, use both the TZR format element and the corresponding TZD format element. The TZD format element is an abbreviation of the time zone region with Daylight Saving Time information included. Examples are PST for U. S. Pacific Standard Time and PDT for U. S. Pacific Daylight Time. The following specification ensures that a Daylight Saving Time value is returned:

```
TIMESTAMP '1999-10-29 01:30:00 America/Los_Angeles PDT'
```

If you do not add the TZD format element, and the datetime value is ambiguous, then the database returns an error if you have the `ERROR_ON_OVERLAP_TIME` session parameter set to `TRUE`. If `ERROR_ON_OVERLAP_TIME` is set to `FALSE` (the default value), then the database interprets the ambiguous datetime as Standard Time.

The default date format for the `TIMESTAMP WITH TIME ZONE` data type is determined by the value of the `NLS_TIMESTAMP_TZ_FORMAT` initialization parameter.

① See Also

- *Oracle AI Database SQL Language Reference* for more information about the `TIMESTAMP WITH TIME ZONE` data type
- ["TIMESTAMP Data Type"](#) for more information about fractional seconds precision
- ["Support for Daylight Saving Time"](#)
- ["NLS_TIMESTAMP_TZ_FORMAT"](#)
- *Oracle AI Database SQL Language Reference* for more information about format elements
- *Oracle AI Database SQL Language Reference* for more information about setting the `ERROR_ON_OVERLAP_TIME` session parameter

4.2.1.4 TIMESTAMP WITH LOCAL TIME ZONE Data Type

`TIMESTAMP WITH LOCAL TIME ZONE` is another variant of `TIMESTAMP`. It differs from `TIMESTAMP WITH TIME ZONE` as follows: data stored in the database is normalized to the database time zone, and the time zone offset is not stored as part of the column data. When users retrieve the data, the database returns it in the users' local session time zone. The time zone offset is the difference (in hours and minutes) between local time and UTC (Coordinated Universal Time, formerly Greenwich Mean Time).

Specify the `TIMESTAMP WITH LOCAL TIME ZONE` data type as follows:

```
TIMESTAMP [(fractional_seconds_precision)] WITH LOCAL TIME ZONE
```

fractional_seconds_precision is optional and specifies the number of digits in the fractional part of the `SECOND` datetime field.

There is no literal for `TIMESTAMP WITH LOCAL TIME ZONE`, but `TIMESTAMP` literals and `TIMESTAMP WITH TIME ZONE` literals can be inserted into a `TIMESTAMP WITH LOCAL TIME ZONE` column.

The default date format for `TIMESTAMP WITH LOCAL TIME ZONE` is determined by the value of the `NLS_TIMESTAMP_FORMAT` initialization parameter.

① See Also

- *Oracle AI Database SQL Language Reference* for more information about the `TIMESTAMP WITH LOCAL TIME ZONE` data type
- "[TIMESTAMP Data Type](#)" for more information about fractional seconds precision
- "[NLS_TIMESTAMP_FORMAT](#)"

4.2.1.5 Inserting Values into Datetime Data Types

You can insert values into a datetime column in the following ways:

- Insert a character string whose format is based on the appropriate NLS format value
- Insert a literal
- Insert a literal for which implicit conversion is performed
- Use the `TO_TIMESTAMP`, `TO_TIMESTAMP_TZ`, or `TO_DATE` SQL function

① See Also

"[Datetime SQL Functions](#)" for more information about the `TO_TIMESTAMP` or `TO_TIMESTAMP_TZ` SQL functions

The following examples show how to insert data into datetime data types.

Example 4-1 Inserting Data into a DATE Column

Set the date format.

```
SQL> ALTER SESSION SET NLS_DATE_FORMAT='DD-MON-YYYY HH24:MI:SS';
```

Create a table `table_dt` with columns `c_id` and `c_dt`. The `c_id` column is of `NUMBER` data type and helps to identify the method by which the data is entered. The `c_dt` column is of `DATE` data type.

```
SQL> CREATE TABLE table_dt (c_id NUMBER, c_dt DATE);
```

Insert a date as a character string.

```
SQL> INSERT INTO table_dt VALUES(1, '01-JAN-2003');
```

Insert the same date as a `DATE` literal.

```
SQL> INSERT INTO table_dt VALUES(2, DATE '2003-01-01');
```

Insert the date as a `TIMESTAMP` literal. The database drops the time zone information.

```
SQL> INSERT INTO table_dt VALUES(3, TIMESTAMP '2003-01-01 00:00:00 America/Los_Angeles');
```

Insert the date with the `TO_DATE` function.

```
SQL> INSERT INTO table_dt VALUES(4, TO_DATE('01-JAN-2003', 'DD-MON-YYYY'));
```

Display the data.

```
SQL> SELECT * FROM table_dt;
```

C_ID	C_DT
1	01-JAN-2003 00:00:00
2	01-JAN-2003 00:00:00
3	01-JAN-2003 00:00:00
4	01-JAN-2003 00:00:00

Example 4-2 Inserting Data into a TIMESTAMP Column

Set the timestamp format.

```
SQL> ALTER SESSION SET NLS_TIMESTAMP_FORMAT='DD-MON-YY HH:MI:SSXFF';
```

Create a table `table_ts` with columns `c_id` and `c_ts`. The `c_id` column is of NUMBER data type and helps to identify the method by which the data is entered. The `c_ts` column is of TIMESTAMP data type.

```
SQL> CREATE TABLE table_ts(c_id NUMBER, c_ts TIMESTAMP);
```

Insert a date and time as a character string.

```
SQL> INSERT INTO table_ts VALUES(1, '01-JAN-2003 2:00:00');
```

Insert the same date and time as a TIMESTAMP literal.

```
SQL> INSERT INTO table_ts VALUES(2, TIMESTAMP '2003-01-01 2:00:00');
```

Insert the same date and time as a TIMESTAMP WITH TIME ZONE literal. The database converts it to a TIMESTAMP value, which means that the time zone information is dropped.

```
SQL> INSERT INTO table_ts VALUES(3, TIMESTAMP '2003-01-01 2:00:00 -08:00');
```

Display the data.

```
SQL> SELECT * FROM table_ts;
```

C_ID	C_TS
1	01-JAN-03 02:00:00.000000 AM
2	01-JAN-03 02:00:00.000000 AM
3	01-JAN-03 02:00:00.000000 AM

Note that the three methods result in the same value being stored.

Example 4-3 Inserting Data into the TIMESTAMP WITH TIME ZONE Data Type

Set the timestamp format.

```
SQL> ALTER SESSION SET NLS_TIMESTAMP_TZ_FORMAT='DD-MON-RR HH:MI:SSXFF AM TZR';
```

Set the time zone to '-07:00'.

```
SQL> ALTER SESSION SET TIME_ZONE='-7:00';
```

Create a table `table_tstz` with columns `c_id` and `c_tstz`. The `c_id` column is of NUMBER data type and helps to identify the method by which the data is entered. The `c_tstz` column is of TIMESTAMP WITH TIME ZONE data type.

```
SQL> CREATE TABLE table_tstz (c_id NUMBER, c_tstz TIMESTAMP WITH TIME ZONE);
```

Insert a date and time as a character string.

```
SQL> INSERT INTO table_tstz VALUES(1, '01-JAN-2003 2:00:00 AM -07:00');
```

Insert the same date and time as a `TIMESTAMP` literal. The database converts it to a `TIMESTAMP WITH TIME ZONE` literal, which means that the session time zone is appended to the `TIMESTAMP` value.

```
SQL> INSERT INTO table_tstz VALUES(2, TIMESTAMP '2003-01-01 2:00:00');
```

Insert the same date and time as a `TIMESTAMP WITH TIME ZONE` literal.

```
SQL> INSERT INTO table_tstz VALUES(3, TIMESTAMP '2003-01-01 2:00:00 -8:00');
```

Display the data.

```
SQL> SELECT * FROM table_tstz;
```

C_ID	C_TSTZ
1	01-JAN-03 02:00:00.000000 AM -07:00
2	01-JAN-03 02:00:00.000000 AM -07:00
3	01-JAN-03 02:00:00.000000 AM -08:00

Note that the time zone is different for method 3, because the time zone information was specified as part of the `TIMESTAMP WITH TIME ZONE` literal.

Example 4-4 Inserting Data into the `TIMESTAMP WITH LOCAL TIME ZONE` Data Type

Consider data that is being entered in Denver, Colorado, U.S.A., whose time zone is UTC-7.

```
SQL> ALTER SESSION SET TIME_ZONE='-07:00';
```

Create a table `table_tsltz` with columns `c_id` and `c_tsltz`. The `c_id` column is of `NUMBER` data type and helps to identify the method by which the data is entered. The `c_tsltz` column is of `TIMESTAMP WITH LOCAL TIME ZONE` data type.

```
SQL> CREATE TABLE table_tsltz (c_id NUMBER, c_tsltz TIMESTAMP WITH LOCAL TIME ZONE);
```

Insert a date and time as a character string.

```
SQL> INSERT INTO table_tsltz VALUES(1, '01-JAN-2003 2:00:00');
```

Insert the same data as a `TIMESTAMP WITH LOCAL TIME ZONE` literal.

```
SQL> INSERT INTO table_tsltz VALUES(2, TIMESTAMP '2003-01-01 2:00:00');
```

Insert the same data as a `TIMESTAMP WITH TIME ZONE` literal. The database converts the data to a `TIMESTAMP WITH LOCAL TIME ZONE` value. This means the time zone that is entered (`-08:00`) is converted to the session time zone value (`-07:00`).

```
SQL> INSERT INTO table_tsltz VALUES(3, TIMESTAMP '2003-01-01 2:00:00 -08:00');
```

Display the data.

```
SQL> SELECT * FROM table_tsltz;
```

C_ID	C_TSLTZ
1	01-JAN-03 02.00.00.000000 AM
2	01-JAN-03 02.00.00.000000 AM
3	01-JAN-03 03.00.00.000000 AM

Note that the information that was entered as UTC-8 has been changed to the local time zone, changing the hour from 2 to 3.

4.2.1.6 Choosing a TIMESTAMP Data Type

Use the `TIMESTAMP` data type when you need a datetime value to record the time of an event without the time zone. For example, you can store information about the times when workers punch a time card in and out of their assembly line workstations. Because this is always a local time it is then not needed to store the timezone part. The `TIMESTAMP` data type uses 7 or 11 bytes of storage.

Use the `TIMESTAMP WITH TIME ZONE` data type when the datetime value represents a future local time or the time zone information must be recorded with the value. Consider a scheduled appointment in a local time. The future local time may need to be adjusted if the time zone definition, such as daylight saving rule, changes. Otherwise, the value can become incorrect. This data type is most immune to such impact.

The `TIMESTAMP WITH TIME ZONE` data type requires 13 bytes of storage, or two more bytes of storage than the `TIMESTAMP` and `TIMESTAMP WITH LOCAL TIME ZONE` data types because it stores time zone information. The time zone is stored as a time zone region name or as an offset from UTC. The data is available for display or calculations without additional processing. A `TIMESTAMP WITH TIME ZONE` column cannot be used as a primary key. If an index is created on a `TIMESTAMP WITH TIME ZONE` column, it becomes a function-based index.

The `TIMESTAMP WITH LOCAL TIME ZONE` data type stores the timestamp without time zone information. It normalizes the data to the database time zone every time the data is sent to and from a client. It requires 11 bytes of storage.

The `TIMESTAMP WITH LOCAL TIME ZONE` data type is appropriate when the original time zone is of no interest, but the relative times of events are important and daylight saving adjustment does not have to be accurate. The time zone conversion that this data type performs to and from the database time zone is asymmetrical, due to the daylight saving adjustment. Because this data type does not preserve the time zone information, it does not distinguish values near the adjustment in fall, whether they are daylight saving time or standard time. This confusion between distinct instants can cause an application to behave unexpectedly, especially if the adjustment takes place during the normal working hours of a user.

Note that some regions, such as Brazil and Israel, that update their Daylight Saving Transition dates frequently and at irregular periods, are particularly susceptible to time zone adjustment issues. If time information from these regions is key to your application, you may want to consider using one of the other datetime types.

4.2.2 Interval Data Types

Interval data types store time durations. They are used primarily with analytic functions. For example, you can use them to calculate a moving average of stock prices. You must use interval data types to determine the values that correspond to a particular percentile. You can also use interval data types to update historical tables.

This section includes the following topics:

- [INTERVAL YEAR TO MONTH Data Type](#)
- [INTERVAL DAY TO SECOND Data Type](#)
- [Inserting Values into Interval Data Types](#)

① See Also

Oracle AI Database Data Warehousing Guide for more information about analytic functions, including moving averages and inverse percentiles

4.2.2.1 INTERVAL YEAR TO MONTH Data Type

INTERVAL YEAR TO MONTH stores a period of time using the YEAR and MONTH datetime fields. Specify INTERVAL YEAR TO MONTH as follows:

```
INTERVAL YEAR [(year_precision)] TO MONTH
```

year_precision is the number of digits in the YEAR datetime field. Accepted values are 0 to 9. The default value of *year_precision* is 2.

Interval values can be specified as literals. There are many ways to specify interval literals. The following is one example of specifying an interval of 123 years and 2 months. The year precision is 3.

```
INTERVAL '123-2' YEAR(3) TO MONTH
```

① See Also

Oracle AI Database SQL Language Reference for more information about specifying interval literals with the INTERVAL YEAR TO MONTH data type

4.2.2.2 INTERVAL DAY TO SECOND Data Type

INTERVAL DAY TO SECOND stores a period of time in terms of days, hours, minutes, and seconds. Specify this data type as follows:

```
INTERVAL DAY [(day_precision)] TO SECOND [(fractional_seconds_precision)]
```

day_precision is the number of digits in the DAY datetime field. Accepted values are 0 to 9. The default is 2.

fractional_seconds_precision is the number of digits in the fractional part of the SECOND datetime field. Accepted values are 0 to 9. The default is 6.

The following is one example of specifying an interval of 4 days, 5 hours, 12 minutes, 10 seconds, and 222 thousandths of a second. The fractional second precision is 3.

```
INTERVAL '4 5:12:10.222' DAY TO SECOND(3)
```

Interval values can be specified as literals. There are many ways to specify interval literals.

① See Also

Oracle AI Database SQL Language Reference for more information about specifying interval literals with the INTERVAL DAY TO SECOND data type

4.2.2.3 Inserting Values into Interval Data Types

You can insert values into an interval column in the following ways:

- Insert an interval as a literal. For example:

```
INSERT INTO table1 VALUES (INTERVAL '4-2' YEAR TO MONTH);
```

This statement inserts an interval of 4 years and 2 months.

Oracle Database recognizes literals for other ANSI interval types and converts the values to Oracle Database interval values.

- Use the NUMTODSINTERVAL, NUMTOYMINTERVAL, TO_DSINTERVAL, and TO_YMINTERVAL SQL functions.

① See Also

["Datetime SQL Functions"](#)

4.3 Datetime and Interval Arithmetic and Comparisons

This section includes the following topics:

- [Datetime and Interval Arithmetic](#)
- [Datetime Comparisons](#)
- [Explicit Conversion of Datetime Data Types](#)

4.3.1 Datetime and Interval Arithmetic

You can perform arithmetic operations on date (DATE), timestamp (TIMESTAMP, TIMESTAMP WITH TIME ZONE, and TIMESTAMP WITH LOCAL TIME ZONE) and interval (INTERVAL DAY TO SECOND and INTERVAL YEAR TO MONTH) data. You can maintain the most precision in arithmetic operations by using a timestamp data type with an interval data type.

You can use NUMBER constants in arithmetic operations on date and timestamp values. The database internally converts timestamp values to date values before doing arithmetic operations on them with NUMBER constants. This means that information about fractional seconds is lost during operations that include both date and timestamp values. The database interprets NUMBER constants in datetime and interval expressions as number of days.

Each DATE value contains a time component. The result of many date operations includes a fraction. This fraction means a portion of one day. For example, 1.5 days is 36 hours. These fractions are also returned by Oracle Database built-in SQL functions for common operations on DATE data. For example, the built-in MONTHS_BETWEEN SQL function returns the number of months between two dates. The fractional portion of the result represents that portion of a 31-day month.

The database performs all timestamp arithmetic in UTC time. For TIMESTAMP WITH LOCAL TIME ZONE data, the database converts the datetime value from the database time zone to UTC and converts back to the database time zone after performing the arithmetic. For TIMESTAMP WITH TIME ZONE data, the datetime value is always in UTC, so no conversion is necessary.

① See Also

- *Oracle AI Database SQL Language Reference* for detailed information about datetime and interval arithmetic operations
- "[Datetime SQL Functions](#)" for information about which functions cause implicit conversion to DATE

4.3.2 Datetime Comparisons

When you compare date and timestamp values, the database converts the data to the more precise data type before doing the comparison. For example, if you compare data of `TIMESTAMP WITH TIME ZONE` data type with data of `TIMESTAMP` data type, the database converts the `TIMESTAMP` data to `TIMESTAMP WITH TIME ZONE`, using the session time zone.

The order of precedence for converting date and timestamp data is as follows:

1. `DATE`
2. `TIMESTAMP`
3. `TIMESTAMP WITH LOCAL TIME ZONE`
4. `TIMESTAMP WITH TIME ZONE`

For any pair of data types, the database converts the data type that has a smaller number in the preceding list to the data type with the larger number.

4.3.3 Explicit Conversion of Datetime Data Types

If you want to do explicit conversion of datetime data types, use the `CAST` SQL function. You can explicitly convert `DATE`, `TIMESTAMP`, `TIMESTAMP WITH TIME ZONE`, and `TIMESTAMP WITH LOCAL TIME ZONE` to another data type in the list.

① See Also

Oracle AI Database SQL Language Reference

4.4 Datetime SQL Functions

Datetime functions operate on date (`DATE`), timestamp (`TIMESTAMP`, `TIMESTAMP WITH TIME ZONE`, and `TIMESTAMP WITH LOCAL TIME ZONE`) and interval (`INTERVAL DAY TO SECOND`, `INTERVAL YEAR TO MONTH`) values.

Some of the datetime functions were designed for the Oracle Database `DATE` data type. If you provide a timestamp value as their argument, then the database internally converts the input type to a `DATE` value. The database does not perform internal conversion for the `ROUND` and `TRUNC` functions.

The following table shows the datetime functions that were designed for the Oracle Database `DATE` data type.

Table 4-1 Datetime Functions Designed for the DATE Data Type

Function	Description
ADD_MONTHS	Returns the date <i>d</i> plus <i>n</i> months
LAST_DAY	Returns the last day of the month that contains <i>date</i>
MONTHS_BETWEEN	Returns the number of months between <i>date1</i> and <i>date2</i>
NEW_TIME	Returns the date and time in <i>zone2</i> time zone when the date and time in <i>zone1</i> time zone are <i>date</i> Note: This function takes as input only a limited number of time zones. You can have access to a much greater number of time zones by combining the FROM_TZ function and the datetime expression.
NEXT_DAY	Returns the date of the first weekday named by <i>char</i> that is later than <i>date</i>
ROUND(<i>date</i>)	Returns <i>date</i> rounded to the unit specified by the <i>fmt</i> format model
TRUNC(<i>date</i>)	Returns <i>date</i> with the time portion of the day truncated to the unit specified by the <i>fmt</i> format model

The following table describes additional datetime functions.

Table 4-2 Additional Datetime Functions

Datetime Function	Description
CURRENT_DATE	Returns the current date in the session time zone in a value in the Gregorian calendar, of the DATE data type
CURRENT_TIMESTAMP	Returns the current date and time in the session time zone as a TIMESTAMP WITH TIME ZONE value
DBTIMEZONE	Returns the value of the database time zone. The value is a time zone offset or a time zone region name
EXTRACT (datetime)	Extracts and returns the value of a specified datetime field from a datetime or interval value expression
FROM_TZ	Converts a TIMESTAMP value at a time zone to a TIMESTAMP WITH TIME ZONE value
LOCALTIMESTAMP	Returns the current date and time in the session time zone in a value of the TIMESTAMP data type
NUMTODSINTERVAL	Converts number <i>n</i> to an INTERVAL DAY TO SECOND literal
NUMTOYMINTERVAL	Converts number <i>n</i> to an INTERVAL YEAR TO MONTH literal
SESSIONTIMEZONE	Returns the value of the current session's time zone
SYS_EXTRACT_UTC	Extracts the UTC from a datetime with time zone offset
SYSDATE	Returns the current date and time SYSDATE uses either the time zone of the database host system or the time zone of the PDB, depending on the value of the TIME_AT_DBTIMEZONE initialization parameter. See <i>Oracle AI Database Reference</i> for more information.

Table 4-2 (Cont.) Additional Datetime Functions

Datetime Function	Description
SYSTIMESTAMP	Returns the current date, including fractional seconds SYSTIMESTAMP uses either the time zone of the database host system or the time zone of the PDB, depending on the value of the <code>TIME_AT_DBTIMEZONE</code> initialization parameter. See <i>Oracle AI Database Reference</i> for more information.
TO_CHAR (datetime)	Converts a datetime or interval value of <code>DATE</code> , <code>TIMESTAMP</code> , <code>TIMESTAMP WITH TIME ZONE</code> , or <code>TIMESTAMP WITH LOCAL TIME ZONE</code> data type to a value of <code>VARCHAR2</code> data type in the format specified by the <i>fmt</i> date format
TO_DSINTERVAL	Converts a character string of <code>CHAR</code> , <code>VARCHAR2</code> , <code>NCHAR</code> , or <code>NVARCHAR2</code> data type to a value of <code>INTERVAL DAY TO SECOND</code> data type
TO_NCHAR (datetime)	Converts a datetime or interval value of <code>DATE</code> , <code>TIMESTAMP</code> , <code>TIMESTAMP WITH TIME ZONE</code> , <code>TIMESTAMP WITH LOCAL TIME ZONE</code> , <code>INTERVAL MONTH TO YEAR</code> , or <code>INTERVAL DAY TO SECOND</code> data type from the database character set to the national character set
TO_TIMESTAMP	Converts a character string of <code>CHAR</code> , <code>VARCHAR2</code> , <code>NCHAR</code> , or <code>NVARCHAR2</code> data type to a value of <code>TIMESTAMP</code> data type
TO_TIMESTAMP_TZ	Converts a character string of <code>CHAR</code> , <code>VARCHAR2</code> , <code>NCHAR</code> , or <code>NVARCHAR2</code> data type to a value of the <code>TIMESTAMP WITH TIME ZONE</code> data type
TO_YMINTERVAL	Converts a character string of <code>CHAR</code> , <code>VARCHAR2</code> , <code>NCHAR</code> , or <code>NVARCHAR2</code> data type to a value of the <code>INTERVAL YEAR TO MONTH</code> data type
TZ_OFFSET	Returns the time zone offset that corresponds to the entered value, based on the date that the statement is executed

The following table describes the functions related to the Daylight Saving Time (DST) upgrade process.

Table 4-3 Time Zone Conversion Functions

Time Zone Function	Description
ORA_DST_AFFECTED	Enables you to verify whether the data in a column is affected by upgrading the DST rules from one version to another version
ORA_DST_CONVERT	Enables you to upgrade your TSTZ column data from one version to another
ORA_DST_ERROR	Enables you to verify that there are no errors when upgrading a datetime value

① See Also

- *Oracle AI Database SQL Language Reference* for more information about the Oracle database datetime functions
- "[Support for Daylight Saving Time](#)" for more information about the Daylight Saving Time functionality of Oracle databases
- "[Daylight Saving Time Session Parameter](#)" for information about the session parameter `ERROR_ON_OVERLAP_TIME` related to Daylight Saving Time
- "[Daylight Saving Time Upgrade Parameter](#)" for information about the initialization parameter `DST_UPGRADE_INSERT_CONV` that is used during the Daylight Saving Time upgrade process

4.5 Datetime and Time Zone Parameters and Environment Variables

This section includes the following topics:

- [Datetime Format Parameters](#)
- [Time Zone Environment Variables](#)
- [Daylight Saving Time Session Parameter](#)
- [Daylight Saving Time Upgrade Parameter](#)

4.5.1 Datetime Format Parameters

The following table contains the names and descriptions of the datetime format parameters.

Table 4-4 Datetime Format Parameters

Parameter	Description
<code>NLS_DATE_FORMAT</code>	Defines the default date format to use with the <code>TO_CHAR</code> and <code>TO_DATE</code> functions
<code>NLS_TIMESTAMP_FORMAT</code>	Defines the default timestamp format to use with the <code>TO_CHAR</code> and <code>TO_TIMESTAMP</code> functions
<code>NLS_TIMESTAMP_TZ_FORMAT</code>	Defines the default timestamp with time zone format to use with the <code>TO_CHAR</code> and <code>TO_TIMESTAMP_TZ</code> functions

Their default values are derived from `NLS_TERRITORY`.

You can specify their values by setting them in the initialization parameter file. If you change the values in the initialization parameter file, you must restart the instance for the change to take effect. You can also specify their values for a client as client environment variables. For Java clients, the value of `NLS_TERRITORY` is derived from the default locale of JRE. The values specified in the initialization parameter file are not used for JDBC sessions.

To change their values during a session, use the `ALTER SESSION` statement.

① See Also

- ["Date and Time Parameters"](#) for more information, including examples
- ["NLS_DATE_FORMAT"](#)
- ["NLS_TIMESTAMP_FORMAT"](#)
- ["NLS_TIMESTAMP_TZ_FORMAT"](#)

4.5.2 Time Zone Environment Variables

The time zone environment variables are:

- `ORA_TZFILE`, which enables you to specify a time zone on the client and database server. Note that when you specify `ORA_TZFILE` on a database server, the only time when this environment variable takes effect is during the creation of the database.
- `ORA_SDTZ`, which specifies the default session time zone.

① See Also

- ["Choosing a Time Zone File"](#)
- ["Setting the Session Time Zone"](#)

4.5.3 Daylight Saving Time Session Parameter

`ERROR_ON_OVERLAP_TIME` is a session parameter that determines how the database handles an ambiguous datetime boundary value. Ambiguous datetime values can occur when the time changes between Daylight Saving Time and standard time.

The possible values are `TRUE` and `FALSE`. When `ERROR_ON_OVERLAP_TIME` is `TRUE`, then an error is returned when the database encounters an ambiguous datetime value. When `ERROR_ON_OVERLAP_TIME` is `FALSE`, then ambiguous datetime values are assumed to be the standard time representation for the region. The default value is `FALSE`.

① See Also

- ["Support for Daylight Saving Time"](#)
- *Oracle AI Database SQL Language Reference*

4.5.4 Daylight Saving Time Upgrade Parameter

`DST_UPGRADE_INSERT_CONV` is an initialization parameter that is only used during the upgrade window of the Daylight Saving Time (DST) upgrade process. It is only applicable to tables with `TIMESTAMP WITH TIME ZONE` columns because those are the only ones that are modified during the DST upgrade.

During the upgrade window of the DST patching process (which is described in the `DBMS_DST` package), tables with `TIMESTAMP WITH TIMEZONE` data undergo conversion to the new time zone

version. Columns in tables that have not yet been converted will still have the `TIMESTAMP WITH TIMEZONE` reflecting the previous time zone version. In order to present the data in these columns as though they had been converted to the new time zone version when you issue `SELECT` statements, Oracle adds by default conversion operators over the columns to convert them to the new version. Adding the conversion operator may, however, slow down queries and disable usage of indexes on the `TIMESTAMP WITH TIMEZONE` columns. Hence, Oracle provides a parameter, `DST_UPGRADE_INSERT_CONV`, that specifies whether or not internal operators are allocated on top of `TIMESTAMP WITH TIMEZONE` columns of tables that have not been upgraded. By default, its value is `TRUE`. If users know that the conversion process will not affect the `TIMESTAMP WITH TIMEZONE` columns, then this parameter can be set to `FALSE`.

Oracle strongly recommends that you set this parameter to `TRUE` throughout the DST patching process. By default, this parameter is set to `TRUE`. However, if set to `TRUE`, query performance may be degraded on unconverted tables. Note that this only applies during the upgrade window.

① See Also

- *Oracle AI Database Reference*
- *Oracle AI Database PL/SQL Packages and Types Reference*

4.6 Choosing a Time Zone File

The Oracle database time zone files contain the valid time zone names. The following information is also included for each time zone:

- Offset from Coordinated Universal Time (UTC)
- Transition times for Daylight Saving Time
- Abbreviations for standard time and Daylight Saving Time

Oracle supplies multiple versions of time zone files, and there are two types of file associated with each version: a large file, which contains all the time zones defined in the database, and a small file, which contains only the most commonly used time zones. The large version files are named as `timezlg_version_number.dat` and the small version files are named as `timezone_version_number.dat`, where `version_number` is the version number of the time zone file. The time zone files are stored in the `$ORACLE_HOME/oracore/zoneinfo` directory and the default time zone file is a large time zone file having the highest version number. In Oracle AI Database 26ai, the default time zone file is `$ORACLE_HOME/oracore/zoneinfo/timezlg_43.dat`.

Examples of time zone files are:

```
$ORACLE_HOME/oracore/zoneinfo/timezlg_42.dat    -- large version 42
$ORACLE_HOME/oracore/zoneinfo/timezone_42.dat   -- small version 42
$ORACLE_HOME/oracore/zoneinfo/timezlg_43.dat    -- large version 43
$ORACLE_HOME/oracore/zoneinfo/timezone_43.dat   -- small version 43
```

During the database creation process, you choose the time zone version for the server. This version is fixed, but you can, however, go through the upgrade process to achieve a higher version. You can use different versions of time zone files on the client and server, but Oracle recommends that you do not. This is because there is a performance penalty when a client on one version communicates with a server on a different version. The performance penalty arises

because the `TIMESTAMP WITH TIME ZONE (TSTZ)` data is transferred using a local timestamp instead of UTC.

On the server, the database always uses a large file. On the client, you can use either a large or a small file. If you use a large time zone file on the client, then you should continue to use it unless you are sure that no information will be missing if you switch to a smaller one. If you use a small file, you have to make sure that the client does not query data that is not present in the small time zone file. Otherwise, you get an error.

You can enable the use of a specific time zone file on the client or on the server. If you want to enable a time zone file on the server, there are two situations. One is when you want to upgrade the time zone to the target version. See "[Upgrading the Time Zone File and Timestamp with Time Zone Data](#)" for more information. Another is when you want to create a new database. In this case, you can set the `ORA_TZFILE` environment variable to point to the time zone file of your choice.

To enable a specific time zone file on a client, you can set `ORA_TZFILE` to whatever time zone file you want. If `ORA_TZFILE` is not set, the database automatically picks up and uses the file with the latest time zone version.

See Also

Oracle Call Interface Programmer's Guide for more information on how to upgrade Daylight Saving Time on a client

Note

Oracle database time zone data is derived from the public domain information available on *The IANA Functions* website. Oracle database time zone data may not reflect the most recent data available on this website.

You can obtain a list of time zone names and time zone abbreviations from the time zone file that is installed with your database by entering the following statement:

```
SELECT TZNAME, TZABBREV
       FROM V$TIMEZONE_NAMES
       ORDER BY TZNAME, TZABBREV;
```

For the default time zone file, this statement results in output similar to the following:

TZNAME	TZABBREV
-----	-----
Africa/Abidjan	GMT
Africa/Abidjan	LMT
...	
Africa/Algiers	CEST
Africa/Algiers	CET
Africa/Algiers	LMT
Africa/Algiers	PMT
Africa/Algiers	WEST
Africa/Algiers	WET
...	
WET	LMT
WET	WEST
WET	WET

Zulu UTC

2137 rows selected.

In the above output, 2 time zone abbreviations are associated with the Africa/Abidjan time zone, and 6 abbreviations are associated with the Africa/Algiers time zone. The following table shows some of the time zone abbreviations and their meanings.

Time Zone Abbreviation	Meaning
CEST	Central Europe Summer Time
CET	Central Europe Time
EEST	Eastern Europe Summer Time
EET	Eastern Europe Time
GMT	Greenwich Mean Time
LMT	Local Mean Time
PMT	Paris Mean Time
UTC	Coordinated Universal Time
WEST	Western European Summer Time
WET	Western European Time

Note that an abbreviation can be associated with multiple time zones. For example, CET is associated with both Africa/Algiers and Africa/Casablanca, as well as time zones in Europe.

If you want a list of time zones without repeating the time zone name for each abbreviation, use the following query:

```
SELECT UNIQUE TZNAME
       FROM V$TIMEZONE_NAMES
       ORDER BY TZNAME;
```

For the default time zone file, this statement results in output similar to the following:

```
TZNAME
-----
Africa/Abidjan
Africa/Accra
Africa/Addis_Ababa
...
W-SU
WET
Zulu
```

The default time zone file, that is, the large time zone file contains more than 500 unique time zone names. The small time zone file contains more than 200 unique time zone names.

See Also

- "[Time Zone Region Names](#)" for a list of valid Oracle database time zone names
- \$ORACLE_HOME/oracore/zoneinfo/timexdif.csv provided with your Oracle database software installation for a full list of time zones changed in each version of the time zone file.

4.7 Upgrading the Time Zone File and Timestamp with Time Zone Data

The time zone files that are supplied with the Oracle Database are updated periodically to reflect changes in transition rules for various time zone regions. To find which time zone file your database currently uses, query the `V$TIMEZONE_FILE` view.

① Note

Each Oracle Database release includes a time zone file that is current at the time of the release and a number of older version files. Between Oracle Database releases, new time zone file versions may be provided in patch sets or individual patches to reflect the changes in transition rules for various time zone regions. Older time zone file versions allow you to run upgraded databases without a need to immediately upgrade the time zone file to the most current version.

Daylight Saving Time (DST) Transition Rules Changes

Governments can and do change the rules for when Daylight Saving Time takes effect or how it is handled. When this occurs, Oracle provides a new set of transition rules for handling timestamp with time zone data.

Transition periods for the beginning or ending of Daylight Saving Time can potentially introduce problems (such as data loss) when handling timestamps with time zone data. Oracle has provided the PL/SQL package `DBMS_DST` and the `utltz_*` scripts to deal with this transition.

The changes to DST transition rules may affect existing data of `TIMESTAMP WITH TIME ZONE` data type, because of the way Oracle Database stores this data internally. When users enter timestamps with time zone, Oracle Database converts the data to UTC, based on the transition rules in the time zone file, and stores the data together with the ID of the original time zone on disk. When data is retrieved, the reverse conversion from UTC takes place. For example, in the past, under version 2 transition rules, the value `TIMESTAMP '2007-11-02 12:00:00 America/Los_Angeles'` was stored as UTC value `'2007-11-02 20:00:00'` *plus* the time zone ID for `'America/Los_Angeles'`. The time in Los Angeles was stored as UTC *minus* eight hours (PST). Under version 3 of the transition rules, the offset for the same day is *minus* seven hours (PDT). Beginning with year 2007, the DST has been in effect longer (until the first Sunday of November, which was November 4th in 2007). Now, when users retrieve the same timestamp and the new offset is added to the stored UTC time, they receive `TIMESTAMP '2007-11-02 13:00:00 America/Los_Angeles'`. There is a one hour difference compared to the data previous to version 3 taking effect.

① Note

For any time zone region whose transition rules have been updated, the upgrade process discussed throughout this section affects only timestamps that point to the future relative to the effective date of the corresponding DST rule change. For example, no timestamp before year 2007 is affected by the version 3 change to the `'America/Los_Angeles'` time zone region.

Preparing to Upgrade the Time Zone File and Timestamp with Time Zone Data

Before you actually upgrade any data, that is `TIMESTAMP WITH TIME ZONE (TSTZ)` data in a database, you should verify what the impact of the upgrade is likely to be. In general, you can consider the upgrade process to have two separate sub-processes – prepare and upgrade. To prepare for the upgrade, you start a prepare window, which is the time when you check how much data has to be updated in the database. To upgrade, you start an upgrade window, which is the time when changes to the data actually occur.

While not required, Oracle strongly recommends that you perform the prepare step. In addition to finding out how much data will have to be modified during the upgrade, thus giving you an estimate of how much time the upgrade will take, you will also see any semantic errors that you may encounter.

Upgrading the Time Zone File and Timestamp with Time Zone Data in a Multitenant Environment

The following guidelines apply when upgrading the time zone file and timestamp with time zone data in a multitenant environment:

- Each container in a multitenant environment has its own time zone file. Therefore, to perform a time zone data upgrade across an entire CDB, you must upgrade the CDB root and each PDB separately. Note that Oracle allows different containers to have different time zone file versions, so you have the option of upgrading only a subset of containers in a CDB.
- When performing a time zone data upgrade in a CDB (using either the `DBMS_DST` package or the `utlztz_*` scripts), you must perform the Prepare Window steps and the Upgrade Window steps completely in one container before moving on to the next container.
- A new PDB is always assigned the time zone version of `PDB$SEED`.
- `PDB$SEED` is always assigned the time zone version at the time of CDB creation. The time zone version of `PDB$SEED` cannot be changed.

Methods for Upgrading the Time Zone File and Timestamp with Time Zone Data

You can upgrade the time zone data in your database using either of the following methods:

- [Upgrading the Time Zone Data Using the `DBMS\_DST` Package](#)

This method provides the most control over the individual steps of the time zone data upgrade. Starting with Oracle Database 21c, you have the option of performing this method while the database is online or offline. The online version of this method requires one restart of the database at your convenience and requires minimal locks on tables that need to be upgraded. It allows applications to query all time zone data and to insert and modify time zone data for all tables that can be migrated online. The offline version of this method, which was also available in previous releases, requires the database to be in `UPGRADE` mode during part of the procedure and is more restrictive about when applications can insert and modify time zone data.

- [Upgrading the Time Zone Data Using the `utlztz\_\*` Scripts](#)

This method is easier to perform than the method that uses the `DBMS_DST` package, because it provides a set of wrapper scripts that call the various `DBMS_DST` procedures. However, during the time zone upgrade process, the database is automatically restarted multiple times and applications cannot query or insert time zone data in the database.

4.7.1 Upgrading the Time Zone Data Using the DBMS_DST Package

This section describes how to perform a time zone data upgrade using the DBMS_DST package.

Each container in a multitenant environment has its own time zone file. Therefore, to perform a time zone data upgrade across an entire CDB, you must upgrade the CDB root and each PDB separately. Note that Oracle allows different containers to have different time zone file versions, so you have the option of upgrading only a subset of containers in a CDB.

Perform the steps in the following sections completely in one container before moving on to the next container:

- [Prepare Window](#)
- [Upgrade Window](#)
- [Upgrade Example](#)
- [Upgrade Error Handling](#)

4.7.1.1 Prepare Window

During the prepare window, you can get the information about the data that will be affected during the time zone upgrade process using the following steps:

1. Install the desired version of time zone files to which you will be later migrating into \$ORACLE_HOME/oracore/zoneinfo. If the desired version is *version_number*, then you must add the file *timezlg_version_number.dat*. You can add the file *timezone_version_number.dat* at your discretion later. These files can be found on My Oracle Support. The desired version should be the latest version available, unless the latest version contains relevant DST rule changes that were rolled back by the appropriate government after the version had been released.
2. You can optionally create the following tables:
 - an error table that contains the errors generated during the upgrade process by using the DBMS_DST.CREATE_ERROR_TABLE procedure. If you do not explicitly create this table, then the default table used is `sys.dst$error_table`.
 - a table that contains the affected timestamp with time zone information by using the DBMS_DST.CREATE_AFFECTED_TABLE procedure. If you do not explicitly create this table, then the default table used is `sys.dst$affected_tables`.
 - a trigger table that stores the disabled TSTZ table triggers information by using the DBMS_DST.CREATE_TRIGGER_TABLE procedure. If you do not explicitly create this table, then the default table used is `sys.dst$trigger_table`. Note that during the upgrade window, Oracle Database first disables the triggers on a TSTZ table and then performs the upgrade of its affected TSTZ data. Oracle Database saves the information about those triggers in the `sys.dst$trigger_table` table. After completing the upgrade of the affected TSTZ data in the table, the disabled triggers are enabled by reading their information from the `sys.dst$trigger_table` table and then their information is removed from the `sys.dst$trigger_table` table. If any irrecoverable error occurs, such as an unexpected instance shutdown during the upgrade window, you should check the `sys.dst$trigger_table` table to see if any trigger has not been restored to its previous active state before the upgrade.
3. Purge unneeded data.

The DBMS_SCHEDULER log tables contain a large amount of time zone data. If this data is not needed, then delete it using the following command before you run the upgrade steps.

Stop the main jobs before running this command as it may not delete all of the data from the DBMS_SCHEDULER log tables, if some of the main jobs in a chain of jobs are still running.

```
exec dbms_scheduler.purge_log;
```

The other tables that may contain a large amount of time zone data are the SYS.WRI\$_OPTSTAT_HISTGRM_HISTORY and SYS.WRI\$_OPTSTAT_HISTHEAD_HISTORY tables. In case you do not need this data, then you may delete it using the following commands:

```
-- check the number of rows in the tables
select count(*) from SYS.WRI$_OPTSTAT_HISTGRM_HISTORY;
select count(*) from SYS.WRI$_OPTSTAT_HISTHEAD_HISTORY;

-- check the data retention period of the stats
-- the default value is 31
select systimestamp - dbms_stats.get_stats_history_availability from dual;

-- disable stats retention
exec dbms_stats.alter_stats_history_retention(0);

-- remove all the stats
exec DBMS_STATS.PURGE_STATS(systimestamp);

-- check the result of the purge operation
select count(*) from SYS.WRI$_OPTSTAT_HISTGRM_HISTORY;
select count(*) from SYS.WRI$_OPTSTAT_HISTHEAD_HISTORY;
```

You may set the data retention period back to its original value using the following command once the time zone data upgrade is completed:

```
exec dbms_stats.alter_stats_history_retention(31);
```

4. Execute the procedure DBMS_DST.BEGIN_PREPARE(*new_version*), where *new_version* is the time zone file version you chose in Step 1.
5. Collect information about affected data by executing the procedure DBMS_DST.FIND_AFFECTED_TABLES, optionally passing the names of custom tables created in Step 2 as parameters. Verify the affected columns by querying sys.dst\$affected_tables or the corresponding custom table. Also, it is particularly important to check sys.dst\$affected_tables.error_count or the corresponding error_count column in the custom table for possible errors. If the error count is greater than 0, you can check what kind of errors you might expect during the upgrade by checking sys.dst\$error_table or the corresponding custom error table. See "[Upgrade Error Handling](#)".
6. End the prepare window by executing the procedure DBMS_DST.END_PREPARE.

Note

- Only one DBA should run the prepare window at one time. Also, make sure to correct all errors before running the upgrade.
- You can find the matrix of available patches for updating your time zone files by going to Oracle Support and reading Document ID 412160.1.

① See Also

Oracle AI Database PL/SQL Packages and Types Reference for more information about DBMS_DST package

4.7.1.2 Upgrade Window

① Note

In a multitenant environment, all the PDBs must be shut down before running the DBMS_DST.UPGRADE_DATABASE procedure on a CDB.

During the upgrade window, you can upgrade the time zone data using the following steps:

1. If you have not already done so, download the desired version of `timezlg_version_number.dat` and install it in `$ORACLE_HOME/oracore/zoneinfo`. In addition, you can optionally download `timezone_version_number.dat` from My Oracle Support and put it in the same location.
2. Starting with Oracle Database 21c, you have the option of performing this method while the database is online or offline. The online version of this method requires one restart of the database at your convenience and requires minimal locks on tables that need to be upgraded. It allows applications to query all time zone data and to insert and modify time zone data for all tables that can be migrated online. The offline version of this method, which was also available in previous releases, requires the database to be in UPGRADE mode during part of the procedure and is more restrictive about when applications can insert and modify time zone data.
 - a. If you choose to perform the online version of this procedure:
 - i. Set the `TIMEZONE_VERSION_UPGRADE_ONLINE` initialization parameter to `true`:

```
ALTER SYSTEM SET TIMEZONE_VERSION_UPGRADE_ONLINE = true;
```
 - ii. Proceed to Step 3.
 - b. If you choose to perform the offline version of this procedure:
 - i. Ensure that the `TIMEZONE_VERSION_UPGRADE_ONLINE` initialization parameter is set to its default value of `false`:

```
ALTER SYSTEM SET TIMEZONE_VERSION_UPGRADE_ONLINE = false;
```
 - ii. Shut down the database. In Oracle RAC, you must shut down all instances.
 - iii. Start up the database in the UPGRADE mode. Note that, in Oracle RAC, only one instance should be started. See *Oracle AI Database Upgrade Guide* for more information about the UPGRADE mode.
3. Run the procedure `DBMS_DST.BEGIN_UPGRADE(new_version)`. Optionally, you can set the `error_on_overlap_time` and `error_on_nonexisting_time` parameters to `TRUE` if you do not want to ignore semantic errors during the upgrade of dictionary tables that contain timestamp with time zone data. Examples of such errors could be related to `DBMS_SCHEDULER`, as discussed in "[Upgrade Error Handling](#)". If you specify `TRUE` for either

or both of these parameters, the errors are populated into `sys.dst$error_table`. In this case, you might want to truncate the error table before you run the `BEGIN_UPGRADE` procedure. See *Oracle AI Database PL/SQL Packages and Types Reference* for more information.

The `BEGIN_UPGRADE` procedure modifies time zone information in the database, but does not modify any user table data. While the `BEGIN_UPGRADE` procedure is operating, you cannot add tables containing TSTZ columns to the database, nor can you add TSTZ columns to existing tables. TSTZ columns are columns defined on `TIMESTAMP WITH TIME ZONE` data types or object types containing attributes of `TIMESTAMP WITH TIME ZONE` data types.

After the `BEGIN_UPGRADE` procedure has successfully executed, user tables that contain TSTZ data will be marked with the `UPGRADE IN PROGRESS` property.

4. If the `BEGIN_UPGRADE` procedure fails, the error "ORA-56927: Starting an upgrade window failed" is displayed.

After the `BEGIN_UPGRADE` procedure finishes executing with errors, check `sys.dst$error_table` to determine if the dictionary conversion was successful. If successful, there will not be any rows in the table. If there are errors, correct those errors manually and rerun the `BEGIN_UPGRADE` procedure. See "[Upgrade Error Handling](#)".

5. Truncate the error and trigger tables (by default, `sys.dst$error_table` and `sys.dst$trigger_table`).
6. This step depends on whether you are performing the online or offline version of this procedure:
 - If you are performing the online version of this procedure, then allow the database to continue running until you reach a convenient time to perform a reboot. During this time, the database is still operating with the old time zone version and the TSTZ data has not yet been updated. You are allowed to add tables that contain TSTZ columns to the database and you can add TSTZ columns to existing tables. When it is a convenient time to reboot the database, shut down the database and then restart it in normal mode. In Oracle RAC, you must shut down all instances first before restarting them.
 - If you are performing the offline version of this procedure (that is, you put the database in `UPGRADE` mode in Step 2), then restart the database now in normal mode.
7. Confirm that the new time zone version is the primary version by running the following query:

```
SELECT * FROM V$TIMEZONE_FILE;
```

8. Allow the database to continue running until you reach a convenient time to upgrade the TSTZ data in all tables. A convenient time could be when the database has low usage.

While the database continues running, previously existing TSTZ columns still have the `TIMESTAMP WITH TIMEZONE` reflecting the previous time zone version. In order to present the data in these columns as though they have been converted to the new time zone version when you issue `SELECT` statements, Oracle adds conversion operators over the columns to convert them to the new version. Newly created TSTZ columns will have `TIMESTAMP WITH TIMEZONE` reflecting the new time zone version.

9. When it is a convenient time to upgrade the TSTZ data in all tables, run the procedure `DBMS_DST.UPGRADE_DATABASE`.

Each TSTZ table is upgraded in a separate transaction. For TSTZ base tables with materialized view logs, the base table and materialized view log table are upgraded in one

transaction. During the upgrade, if `TIMEZONE_VERSION_UPGRADE_ONLINE` is set to `true`, the upgrade operation will be performed online whenever possible. For tables containing TSTZ data that cannot be upgraded online, the database will acquire an exclusive DML lock. If `TIMEZONE_VERSION_UPGRADE_ONLINE` is set to `false`, all upgrade operations will acquire an exclusive DML lock on each table and perform the upgrade. All dependent cursors on a table will be invalidated when the upgrade of the table is complete.

Note

Oracle AI Database 26ai introduces the following enhancements to the upgrade process:

- Tables that do not contain any TSTZ data that will be affected by the upgrade are bypassed by the upgrade process.
- Tables that contain only a small amount of TSTZ data that will be affected by the upgrade are converted using discrete DML statements, thereby eliminating the requirement for an exclusive DML lock on the entire table. If you attempt to run a DML statement on such a table while it is being converted, the following error may occur: `ORA-30098: affected TIMESTAMP WITH TIME ZONE data is not allowed at current DST upgrade state`. If you encounter this error, retry the DML statement after the upgrade of the table is completed.
- You can monitor the actions performed on tables during the upgrade process by viewing the `dst$upgrade_history` table. See "[About the Upgrade History Table](#)" for more information.

10. Verify that all tables have been upgraded by querying the `DBA_TSTZ_TABLES` view, as shown in "[Upgrade Example](#)". Then check `sys.dst$error_table` to see if there are any errors. If there are errors, correct the errors and rerun the `DBMS_DST.UPGRADE_TABLE` procedure for the relevant tables. Or, if you do not think those errors are important, then rerun the `DBMS_DST.UPGRADE_TABLE` procedure with the parameters set to ignore errors.
11. End the upgrade window by running the procedure `DBMS_DST.END_UPGRADE`.

Notes

- Tables containing `TIMESTAMP WITH TIME ZONE` columns need to be in a state where they can be updated. Therefore, as an example, the columns cannot have validated and disabled check constraints because this prevents updating.
- Oracle recommends that you use the parallel option if a table size is greater than 2 Gigabytes. Oracle also recommends that you allow Oracle to handle any semantic errors that may arise.
- When you run the `CREATE` statements for error, trigger, or affected tables, you must pass the table name only, not the schema name. This is because the tables are created in the schema from which the `CREATE` statements are run.
- You cannot create materialized views or materialized view logs on tables marked with the `UPGRADE_IN_PROGRESS` property; if you attempt to do so, error `ORA-30403` or `ORA-32426` will be raised. You cannot alter existing materialized view logs on tables marked with the `UPGRADE_IN_PROGRESS` property; if you attempt to do so, error `ORA-32426` will be raised. Existing materialized views on tables that are marked with the `UPGRADE_IN_PROGRESS` property are not allowed to perform a refresh; if a refresh is attempted, error `ORA-30403` will be raised.

See Also

Oracle AI Database PL/SQL Packages and Types Reference for more information about `DBMS_DST` package

4.7.1.3 Upgrade Example

This example illustrates updating DST behavior by using the online method of the procedure. It upgrades an Oracle database to time zone version 14. First, assume that your current database is using time zone version 3, and also assume you have an existing table `t`, which contains timestamp with time zone data.

Connect to the database as the user `scott` and execute the following statements:

```
DROP TABLE t;
CREATE TABLE t (c NUMBER, mark VARCHAR(25), ts TIMESTAMP WITH TIME ZONE);

INSERT INTO t VALUES(1, 'not_affected',
                     to_timestamp_tz('22-sep-2006 13:00:00 america/los_angeles',
                                     'dd-mon-yyyy hh24:mi:ss tztz'));
INSERT INTO t VALUES(4, 'affected_err_exist',
                     to_timestamp_tz('11-mar-2007 00:30:00 america/st_johns',
                                     'dd-mon-yyyy hh24:mi:ss tztz'));
INSERT INTO t VALUES(6, 'affected_no_err',
                     to_timestamp_tz('11-mar-2007 01:30:00 america/st_johns',
                                     'dd-mon-yyyy hh24:mi:ss tztz'));
INSERT INTO t VALUES(14, 'affected_err_dup',
                     to_timestamp_tz('21-sep-2006 23:30:00 egypt',
                                     'dd-mon-yyyy hh24:mi:ss tztz'));

COMMIT;
```

Then, optionally, you can start a prepare window to check the affected data and potential semantic errors where there is an overlap or non-existing time. To do this, you should start a window for preparation to migrate to time zone version 14. It is assumed that you have the necessary privileges. These privileges are controlled with the DBMS_DST package.

① See Also

Oracle AI Database PL/SQL Packages and Types Reference for more information about the DBMS_DST package

As an example, first, prepare the window.

```
connect / as sysdba
set serveroutput on
EXEC DBMS_DST.BEGIN_PREPARE(14);
```

A prepare window has been successfully started.

PL/SQL procedure successfully completed.

Note that the argument 14 causes the time zone version 14 to be used in this statement. After this window is successfully started, you can check the status of the DST in DATABASE_PROPERTIES as shown in the following example:

```
SELECT property_name, SUBSTR(property_value, 1, 30) value
FROM database_properties
WHERE property_name LIKE 'DST_%'
ORDER BY property_name;
```

You will see the output similar to the following:

PROPERTY_NAME	VALUE
DST_PRIMARY_TT_VERSION	3
DST_SECONDARY_TT_VERSION	14
DST_UPGRADE_STATE	PREPARE

Next, you can execute DBMS_DST.FIND_AFFECTED_TABLES to find all the tables in the database that are affected if you upgrade from version 3 to version 14. This table contains the table owner, table name, column name, row count, and error count. Here, you have the choice of using the defaults for error tables (sys.dst\$error_table) and affected tables (sys.dst\$affected_tables) or you can create your own. In this example, we create our own tables by using DBMS_DST.CREATE_ERROR_TABLE and DBMS_DST.CREATE_AFFECTED_TABLE and then pass to FIND_AFFECTED_TABLES as shown below.

Connect to the database as the user SYS and execute the following statements:

```
EXEC DBMS_DST.CREATE_AFFECTED_TABLE('my_affected_tables');
EXEC DBMS_DST.CREATE_ERROR_TABLE('my_error_table');
```

Then, you can execute FIND_AFFECTED_TABLES to see which tables are impacted during the upgrade:

```
connect / as sysdba

BEGIN
```

```

        DBMS_DST.FIND_AFFECTED_TABLES(affected_tables => 'my_affected_tables',
                                      log_errors      => TRUE,
                                      log_errors_table => 'my_error_table');
END;
/

```

Then, check the affected tables:

```
SELECT * FROM my_affected_tables;
```

TABLE_OWNER	TABLE_NAME	COLUMN_NAME	ROW_COUNT	ERROR_COUNT
SCOTT	T	TS	3	2

Then, check the error table:

```
SELECT * FROM my_error_table;
```

TABLE_OWNER	TABLE_NAME	COLUMN_NAME	ROWID	ERROR_NUMBER
SCOTT	T	TS	AAAPW3AABAAANzoAAB	1878
SCOTT	T	TS	AAAPW3AABAAANzoAAE	1883

These errors can be corrected as described in "[Upgrade Error Handling](#)". Then, end the prepare window, as in the following statement:

```
EXEC DBMS_DST.END_PREPARE;
```

A prepare window has been successfully ended.

PL/SQL procedure successfully completed.

After this, you can check the DST status in DATABASE_PROPERTIES:

```

SELECT property_name, SUBSTR(property_value, 1, 30) value
FROM database_properties
WHERE property_name LIKE 'DST_%'
ORDER BY property_name;

```

PROPERTY_NAME	VALUE
DST_PRIMARY_TT_VERSION	3
DST_SECONDARY_TT_VERSION	0
DST_UPGRADE_STATE	NONE

Next, you can use the upgrade window to upgrade the affected data. To do this, first set the TIMEZONE_VERSION_UPGRADE_ONLINE initialization parameter to true:

```
ALTER SYSTEM SET TIMEZONE_VERSION_UPGRADE_ONLINE = true;
```

Execute DBMS_DST.BEGIN_UPGRADE. In Oracle RAC, all instances must be running. BEGIN_UPGRADE upgrades all dictionary tables in one transaction, so the invocation will either succeed or fail as one whole. During the procedure's execution, all tables with TSTZ data are marked as an upgrade in progress. You cannot add tables containing TSTZ columns to the database, nor can you add TSTZ columns to existing tables. See *Oracle AI Database Upgrade Guide* for more information.

So, BEGIN_UPGRADE upgrades system tables that contain TSTZ data and marks user tables (containing TSTZ data) with the UPGRADE_IN_PROGRESS property. This can be checked in DBA_TSTZ_TABLES, and is illustrated later in this example.

There are two parameters in BEGIN_UPGRADE that are for handling semantic errors: error_on_overlap_time (error number ORA-1883) and error_on_nonexisting_time (error number ORA-1878). If the parameters use the default setting of FALSE, Oracle converts the data using a default conversion and does not signal an error. See "[Upgrade Error Handling](#)" for more information regarding what they mean, and how to handle errors.

The following call can automatically correct semantic errors based on some default values when you upgrade the dictionary tables. If you do not ignore semantic errors, and you do have such errors in the dictionary tables, BEGIN_UPGRADE will fail. These semantic errors are populated into my_error_table.

```
EXEC DBMS_DST.BEGIN_UPGRADE(14);
An upgrade window has been successfully started.
```

PL/SQL procedure successfully completed.

After this, you can check the DST status in DATABASE_PROPERTIES, as in the following:

```
SELECT property_name, SUBSTR(property_value, 1, 30) value
FROM database_properties
WHERE property_name LIKE 'DST_%'
ORDER BY property_name;
```

PROPERTY_NAME	VALUE
DST_PRIMARY_TT_VERSION	14
DST_SECONDARY_TT_VERSION	3
DST_UPGRADE_STATE	NEEDRESTART

Then, note that all tables containing TSTZ data display YES for UPGRADE_IN_PROGRESS:

```
SELECT owner, table_name, upgrade_in_progress FROM dba_tstz_tables;
```

OWNER	TABLE_NAME	UPGRADE_IN_PROGRESS
SYS	WRI\$OPTSTAT_AUX_HISTORY	YES
SYS	WRI\$OPTSTAT_OPR	YES
SYS	OPTSTAT_HIST_CONTROL\$	YES
SYS	SCHEDULER\$_JOB	YES
SYS	KET\$_AUTOTASK_STATUS	YES
SYS	AQ\$_ALERT_QT_S	YES
SYS	AQ\$_KUPC\$DATAPUMP_QUETAB_S	YES
DBSNMP	MGMT_DB_FEATURE_LOG	YES
WMSYS	WM\$VERSIONED_TABLES	YES
SYS	WRI\$OPTSTAT_IND_HISTORY	YES
SYS	OPTSTAT_USER_PREFS\$	YES
SYS	FGR\$_FILE_GROUP_FILES	YES
SYS	SCHEDULER\$_WINDOW	YES
SYS	WRR\$_REPLAY_DIVERGENCE	YES
SCOTT	T	YES
...		

Note that the upgrade is in progress for table SCOTT.T. If you access SCOTT.T, the database will transparently apply conversion operators to ensure to properly reflect the new time zone rules on this table while the upgrade is in progress.

After BEGIN_UPGRADE has successfully executed, the database continues to operate with the old time zone file. When it is a convenient time to reboot the database, shut down the database and then restart it in normal mode.

Confirm that the new time zone version is the primary version by running the following query:

```
SELECT * FROM V$TIMEZONE_FILE;
```

FILENAME	VERSION	CON_ID
timezlg_14.dat	14	0

Now you can perform an upgrade of all tables containing TSTZ data by using DBMS_DST.UPGRADE_DATABASE. All tables must be upgraded, otherwise, you will not be able to end the upgrade window using the END_UPGRADE procedure.

Consider the following choices for running the DBMS_DST.UPGRADE_DATABASE procedure:

- Oracle recommends that you use the following code to perform the upgrade. It instructs the database to resolve any errors encountered and relieves you of having to resolve the errors manually.

```
VAR numfail number;
BEGIN
    DBMS_DST.UPGRADE_DATABASE(:numfail);
    DBMS_OUTPUT.PUT_LINE('Number of tables failed to upgrade:' || :numfail);
END;
/
```

- However, if you prefer to view and resolve errors manually, you can use the following code:

```
VAR numfail number;
BEGIN
    DBMS_DST.UPGRADE_DATABASE(:numfail,
        parallel => TRUE,
        log_errors => TRUE,
        log_errors_table => 'my_error_table',
        log_triggers_table => 'SYS.DST$TRIGGER_TABLE',
        error_on_overlap_time => TRUE,
        error_on_nonexisting_time => TRUE);

    DBMS_OUTPUT.PUT_LINE('Number of tables failed to upgrade:' || :numfail);
END;
/
```

If there are any errors, you should correct them and use UPGRADE_TABLE on the individual tables. In that case, you may need to handle tables related to materialized views, such as materialized view base tables, materialized view log tables, and materialized view container tables. There are a couple of considerations to keep in mind when upgrading these tables. First, the base table and its materialized view log table have to be upgraded atomically. Next, the materialized view container table has to be upgraded after all its base tables and the materialized view log tables have been upgraded. In general, Oracle recommends that you handle semantic errors by letting Oracle Database take the default action.

For the sake of this example, let us assume there were some errors in SCOTT.T after you ran UPGRADE_DATABASE. In that case, you can check these errors by using the following query:

```
SELECT * FROM my_error_table;
```

TABLE_OWNER	TABLE_NAME	COLUMN_NAME	ROWID	ERROR_NUMBER
SCOTT	T	TS	AAA02XAABAAANrgAAD	1878
SCOTT	T	TS	AAA02XAABAAANrgAAE	1878

In the output, you can see the errors having number 1878. This error means that an error has occurred for a non-existing time.

To continue with this example, assume that SCOTT.T has a materialized view log scott.mlog\$_t, and that there is a single materialized view on SCOTT.T. Then, assume that this 1878 error has been corrected.

Finally, you can upgrade the table, materialized view log and materialized view as follows:

```
VAR numfail number;
BEGIN
  DBMS_DST.UPGRADE_TABLE(:numfail,
    table_list      => 'SCOTT.t, SCOTT.mlog$_T',
    parallel        => TRUE,
    continue_after_errors => FALSE,
    log_errors      => TRUE,
    log_errors_table => 'my_error_table',
    error_on_overlap_time => FALSE,
    error_on_nonexisting_time => TRUE,
    log_triggers_table => 'SYS.DST$TRIGGER_TABLE',
    atomic_upgrade  => TRUE);

  DBMS_OUTPUT.PUT_LINE('Number of tables failed to upgrade:' || :numfail);
END;
/

VAR numfail number;
BEGIN
  DBMS_DST.UPGRADE_TABLE(:numfail,
    table_list      => 'SCOTT.MYMV_T',
    parallel        => TRUE,
    continue_after_errors => FALSE,
    log_errors      => TRUE,
    log_errors_table => 'my_error_table',
    error_on_overlap_time => FALSE,
    error_on_nonexisting_time => TRUE,
    log_triggers_table => 'SYS.DST$TRIGGER_TABLE',
    atomic_upgrade  => TRUE);

  DBMS_OUTPUT.PUT_LINE('Number of tables failed to upgrade:' || :numfail);
END;
/
```

The atomic_upgrade parameter enables you to combine the upgrade of the table with its materialized view log.

After all the tables are upgraded, you can invoke END_UPGRADE to end an upgrade window as shown below:

```
VAR numfail number;
BEGIN
```

```

DBMS_DST.END_UPGRADE(:numfail);
DBMS_OUTPUT.PUT_LINE('Number of tables failed to upgrade:' || :numfail);
END;
/

```

The upgrade window ends if all the affected tables are upgraded successfully, else the output shows how many tables did not upgrade successfully.

4.7.1.4 Upgrade Error Handling

There are three major semantic errors that can occur during an upgrade. The first is when an existing time becomes a non-existing time, the second is when a time becomes duplicated, and the third is when a duplicate time becomes a non-duplicate time.

As an example of the first case, consider the change from Pacific Standard Time (PST) to Pacific Daylight Saving Time (PDT) in 2007. This change takes place on Mar-11-2007 at 2AM according to version 3 (and any later version up to at least 32) when the clock moves forward one hour to 3AM and produces a gap between 2AM and 3AM. In version 2, this time change occurs on Apr-01-2007. If you upgrade the time zone file from version 2 to version 3, any time in the interval between 2AM and 3AM on Mar-11-2007 is not valid, and raises an error (ORA-1878) if `ERROR_ON_NONEXISTING_TIME` is set to `TRUE`. Therefore, there is a non-existing time problem. When `ERROR_ON_NONEXISTING_TIME` is set to `FALSE`, which is the default value for this parameter, the error is not reported and Oracle preserves UTC time in this case. For example, "Mar-11-2007 02:30 PST" in version 2 becomes "Mar-11-2007 03:30 PDT" in version 3 as they both are translated to the same UTC timestamp.

An example of the second case occurs when changing from PDT to PST. For example, in version 3 for 2007, the change occurs on Nov-04-2007, when the time falls back from 2AM to 1AM. This means that times in the interval <1AM, 2AM> on Nov-04-2007 can appear twice, once with PST and once with PDT. In version 2, this transition occurs on Oct-28-2007 at 2AM. Thus, any timestamp within <1AM, 2AM> on Nov-04-2007, which is uniquely identified in version 2, results in an error (ORA-1883) in version 3, if `ERROR_ON_OVERLAP_TIME` is set to `TRUE`. If you leave this parameter on its default setting of `FALSE`, then the UTC timestamp value is preserved and no error is raised. In this situation, if you change the version from 2 to 3, timestamp "Nov-04-2007 01:30 PST" in version 2 becomes "Nov-04-2007 01:30 PST" in version 3.

The third case happens when a duplicate time becomes a non-duplicate time. Consider the transition from PDT to PST in 2007, for example, where <1AM, 2AM> on Oct-28-2007 in version 2 is the overlapped interval. Then both "Oct-28-2007 01:30 PDT" and "Oct-28-2007 01:30 PST" are valid timestamps in version 2. If `ERROR_ON_OVERLAP_TIME` is set to `TRUE`, an ORA-1883 error is raised during an upgrade from version 2 to version 3. If `ERROR_ON_OVERLAP_TIME` is set to `FALSE` (the default value of this parameter), however, the LOCAL time is preserved and no error is reported. In this case, both "Oct-28-2007 01:30 PDT" and "Oct-28-2007 01:30 PST" in version 2 convert to the same "Oct-28-2007 01:30 PDT" in version 3. Note that setting `ERROR_ON_OVERLAP_TIME` to `FALSE` can potentially cause some time sequences to be reversed. For example, a job (Job A) scheduled at "Oct-28-2007 01:45 PDT" in version 2 is supposed to be executed before a job (Job B) scheduled at "Oct-28-2007 01:30 PST". After the upgrade to version 3, Job A is scheduled at "Oct-28-2007 01:45 PDT" and Job B remains at "Oct-28-2007 01:30 PDT", resulting in Job B being executed before Job A. Even though unchained scheduled jobs are not guaranteed to be executed in a certain order, this issue should be kept in mind when setting up scheduled jobs.

① See Also

Oracle AI Database PL/SQL Packages and Types Reference for more information regarding how to use these parameters

4.7.1.5 About the Upgrade History Table

Starting with Oracle AI Database 26ai, when you perform a time zone data upgrade using the DBMS_DST package, the database automatically creates an upgrade history table, which displays the actions performed on tables during the upgrade process.

The upgrade history table is stored in the SYS schema and is named `dst$upgrade_history`. It contains the following columns:

Column	Datatype	NULL	Description
TABLE_OWNER	VARCHAR2(128)	NOT NULL	Owner of the table
TABLE_NAME	VARCHAR2(128)	NOT NULL	Name of the table
ACTION	VARCHAR2(128)	NOT NULL	Action performed on the table: • UPGRADE (DML - EXCLUSIVE LOCK) • (DML w/ IMPLICIT CHECK CONSTRAINTS) • (SKIPPED - NO ROW AFFECTED) • (SKIPPED - NO (USED) TSTZ COLUMNS) • (SKIPPED BY USER CALL) • UPGRADE (ALTER TABLE MOVE ONLINE) • ADD IMPLICIT CONSTRAINTS • DROP IMPLICIT CONSTRAINTS • CHECK AFFECTED ROWS (AFTER ADDING IMPLICIT CONSTRAINTS)
START_TIME	TIMESTAMP		Time at which the action started
END_TIME	TIMESTAMP		Time at which the action ended
AFFECTED_ROWS	NUMBER		Number of table rows affected by the action
COMMENTS	VARCHAR2(1024)		Optional comments about the action

The following is a sample query of the upgrade history table:

```
SELECT table_name, action, affected_rows, comments
FROM dst$upgrade_history
ORDER BY start_time;
```

TABLE_NAME	ACTION	AFF	COMMENTS
-----	-----	---	-----
MLOG\$_T_MVLOG	ADD IMPLICIT CONSTRAINTS		
MLOG\$_T_MVLOG	CHECK AFFECTED ROWS (AFTER ADDING IMPLICIT CONSTRAINTS)		
MLOG\$_T_MVLOG	UPGRADE (DML w/ IMPLICIT CHECK CONSTRAINTS)	13	
MLOG\$_T_MVLOG	DROP IMPLICIT CONSTRAINTS		
MLOG\$_T_MVLOG_EMPTY	ADD IMPLICIT CONSTRAINTS		
MLOG\$_T_MVLOG_EMPTY	CHECK AFFECTED ROWS (AFTER ADDING IMPLICIT CONSTRAINTS)		
MLOG\$_T_MVLOG_EMPTY	UPGRADE (SKIPPED - NO ROW AFFECTED)	0	leveraged implicit check constraints
MLOG\$_T_MVLOG_EMPTY	DROP IMPLICIT CONSTRAINTS		
MLOG\$_T_MVLOG_POPULATED	ADD IMPLICIT CONSTRAINTS		
MLOG\$_T_MVLOG_POPULATED	CHECK AFFECTED ROWS (AFTER ADDING IMPLICIT CONSTRAINTS)		
MLOG\$_T_MVLOG_POPULATED	UPGRADE (DML w/ IMPLICIT CHECK CONSTRAINTS)	13	
MLOG\$_T_MVLOG_POPULATED	DROP IMPLICIT CONSTRAINTS		

4.7.2 Upgrading the Time Zone Data Using the utltz_* Scripts

This section describes how to perform a time zone data upgrade using the utltz_* scripts.

Each container in a multitenant environment has its own time zone file. Therefore, to perform a time zone data upgrade across an entire CDB, you must upgrade the CDB root and each PDB separately. Note that Oracle allows different containers to have different time zone file versions, so you have the option of upgrading only a subset of containers in a CDB.

Perform the steps in the following sections completely in one container before moving on to the next container:

- [Prepare Window](#)
- [Upgrade Window](#)

4.7.2.1 Prepare Window

During the prepare window, you can get the information about the data that will be affected during the time zone upgrade process using the following steps:

1. Install the desired version of time zone files to which you will be later migrating into \$ORACLE_HOME/oracore/zoneinfo. If the desired version is *version_number*, then you must add the file *timezlg_version_number.dat*. You can add the file *timezone_version_number.dat* at your discretion later. These files can be found on My Oracle Support. The desired version should be the latest version available, unless the latest version contains relevant DST rule changes that were rolled back by the appropriate government after the version had been released.
2. Run any of the following scripts present in the \$ORACLE_HOME/rdbms/admin directory to check how much data will need to be updated in the database during the time zone data upgrade:

- utltz_countstats.sql

This script shows the optimizer statistics of num_rows of all the tables having TIMESTAMP WITH TIME ZONE (TSTZ) data.

Note

Run the utltz_countstats.sql script only when the database optimizer statistics are up to date. Otherwise, run the utltz_countstar.sql script. If you run the utltz_countstats.sql script, then you need not run the utltz_countstar.sql script.

- utltz_countstar.sql

This script shows the result of the count(*) operation for all the tables having TIMESTAMP WITH TIME ZONE (TSTZ) data.

Note

The utltz_countstar.sql script may take a considerable amount of time to complete its execution.

3. Purge unneeded data.

Perform this step as described in the Prepare Window for upgrading the time zone data using the DBMS_DST package. Refer to Step 3 in "[Prepare Window](#)".

4.7.2.2 Upgrade Window

During the upgrade window, you can run the following scripts present in the \$ORACLE_HOME/rdbms/admin directory to upgrade the time zone data in the database:

1. If you have not already done so, download the desired version of `timezlg_version_number.dat` and install it in `$ORACLE_HOME/oracore/zoneinfo`. In addition, you can optionally download `timezone_version_number.dat` from My Oracle Support and put it in the same location.
2. Ensure that the `TIMEZONE_VERSION_UPGRADE_ONLINE` initialization parameter is set to its default value of `false`:

```
ALTER SYSTEM SET TIMEZONE_VERSION_UPGRADE_ONLINE = false;
```

3. Run the `utltz_upg_check.sql` script from the `$ORACLE_HOME` directory:

```
spool utltz_upg_check.log
@utltz_upg_check.sql
spool off
```

The following information is displayed on the screen after successful execution of the script:

```
INFO: A newer RDBMS DST version than the one currently used is found.
INFO: Note that NO DST update was yet done.
INFO: Now run utltz_upg_apply.sql to do the actual RDBMS DST update.
INFO: Note that the utltz_upg_apply.sql script will
INFO: restart the database 2 times WITHOUT any confirmation or prompt.
```

The script also writes the following information in the `alert.log` file:

```
utltz_upg_check successfully found newer RDBMS DSTv new_time_zone_version and
took number_of_minutes minutes to run.
```

If the `utltz_upg_check.sql` script displays the following error, check the previous message displayed on the screen and proceed accordingly.

```
ORA-20xxx: Stopping script - see previous message...
```

4. Run the `utltz_upg_apply.sql` script from the `$ORACLE_HOME` directory after the `utltz_upg_check.sql` script is executed successfully:

```
spool utltz_upg_apply.log
@utltz_upg_apply.sql
spool off
```

Note

The following are the prerequisites for running the `utltz_upg_apply.sql` script:

- In an Oracle RAC environment, the Oracle RAC database must be started as a single database instance.
- In a multitenant environment, all the PDBs must be shut down before running the `utltz_upg_apply.sql` script on the CDB.

Also, note the following:

- The `utltz_upg_apply.sql` script automatically restarts the database multiple times during its execution.
- The `utltz_upg_apply.sql` script generally takes less time to execute than the `utltz_upg_check.sql` script.

The following information is displayed on the screen after successful execution of the `utltz_upg_apply.sql` script:

```
INFO: The RDBMS DST update is successfully finished.  
INFO: Make sure to exit this sqlplus session.  
INFO: Do not use it for timezone related selects.
```

The `TZ_VERSION` column in the `Registry$database` table now gets updated with the new time zone version.

If the script displays the following error message, then check the previous message displayed on the screen and proceed accordingly.

```
ORA-20xxx: Stopping script - see previous message...
```

Note

If you want to see what is happening when the scripts `utltz_upg_check.sql` and `utltz_upg_apply.sql` are being executed, run the following commands:

```
set PAGES 1000

-- query the V$SESSION_LONGOPS view
select TARGET, TO_CHAR(START_TIME, 'HH24:MI:SS - DD-MM-YY'),
       TIME_REMAINING, SOFAR, TOTALWORK, SID, SERIAL#, OPNAME
from V$SESSION_LONGOPS
where sid in
       (select SID from V$SESSION where CLIENT_INFO = 'upg_tzv')
       and
       SOFAR < TOTALWORK
order by START_TIME;

-- query the V$SESSION and V$SQLAREA views
select S.SID, S.SERIAL#, S.SQL_ID, S.PREV_SQL_ID,
       S.EVENT#, S.EVENT, S.P1TEXT, S.P1, S.P2TEXT,
       S.P2, S.P3TEXT, S.P3, S.TIME_REMAINING_MICRO,
       S.SEQ#, S.BLOCKING_SESSION, BS.PROGRAM "Blocking Program",
       Q1.SQL_TEXT "Current SQL", Q2.SQL_TEXT "Previous SQL"
from V$SESSION S, V$SQLAREA Q1, V$SQLAREA Q2, V$SESSION BS
where S.SQL_ID = Q1.SQL_ID(+) and
       S.PREV_SQL_ID = Q2.SQL_ID(+) and
       S.BLOCKING_SESSION = BS.SID(+) and
       S.CLIENT_INFO = 'upg_tzv';
```

4.8 Clients and Servers Operating with Different Versions of Time Zone Files

In Oracle Database 11g, Release 11.2 and later, you can use different versions of time zone file on the client and the server. This mode of operation was not supported in the earlier Oracle Database releases. Both client and server must be Oracle Database 11g, Release 11.2 or later to operate in such a mixed mode.

See Also

Oracle Call Interface Developer's Guide for the ramifications of working in the mixed mode

OCI, JDBC, Pro*C, and SQL*Plus clients can now continue to communicate with the database server without having to update client-side time zone files. For other products, if not explicitly stated in the product-specific documentation, it should be assumed that such clients cannot operate with a database server with a different time zone file than the client. Otherwise, computations on the `TIMESTAMP WITH TIMEZONE` values that are region ID based may give inconsistent results on the client. This is due to different daylight saving time (DST) rules in

effect for the time zone regions affected between the different time zone file versions at the client and on the server.

Note if an application connects to different databases directly or via database links, it is recommended that all databases be on the same time zone file version. Otherwise, computations on the `TIMESTAMP WITH TIMEZONE` values on these different databases may give inconsistent results. This is due to different DST rules in effect for the time zone regions affected between the different time zone file versions across the different database servers.

4.9 Setting the Database Time Zone

Set the database time zone when the database is created by using the `SET TIME_ZONE` clause of the `CREATE DATABASE` statement. If you do not set the database time zone, then it defaults to the time zone of the server's operating system.

The time zone may be set to a named region or an absolute offset from UTC. To set the time zone to a named region, use a statement similar to the following example:

```
CREATE DATABASE db01
...
SET TIME_ZONE='Europe/London';
```

To set the time zone to an offset from UTC, use a statement similar to the following example:

```
CREATE DATABASE db01
...
SET TIME_ZONE='-05:00';
```

The range of valid offsets is -12:00 to +14:00.

Note

The database time zone is relevant only for `TIMESTAMP WITH LOCAL TIME ZONE` columns. Oracle recommends that you set the database time zone to UTC (0:00) to avoid data conversion and improve performance when data is transferred among databases. This is especially important for distributed databases, replication, and exporting and importing.

Note

The `SYSDATE` and `SYSTIMESTAMP` SQL functions use either the time zone of the database host system or the time zone of the PDB, depending on the value of the `TIME_AT_DBTIMEZONE` initialization parameter. See *Oracle AI Database Reference* for more information.

You can change the database time zone by using the `SET TIME_ZONE` clause of the `ALTER DATABASE` statement. For example:

```
ALTER DATABASE SET TIME_ZONE='Europe/London';
ALTER DATABASE SET TIME_ZONE='-05:00';
```

The `ALTER DATABASE SET TIME_ZONE` statement returns an error if the database contains a table with a `TIMESTAMP WITH LOCAL TIME ZONE` column and the column contains data.

The change does not take effect until the database has been shut down and restarted.

You can find out the database time zone by entering the following query:

```
SELECT dbtimezone FROM DUAL;
```

4.10 Setting the Session Time Zone

You can set the default session time zone with the `ORA_SDTZ` environment variable. When users retrieve `TIMESTAMP WITH LOCAL TIME ZONE` data, Oracle Database returns it in the users' session time zone. The session time zone also takes effect when a `TIMESTAMP` value is converted to the `TIMESTAMP WITH TIME ZONE` or `TIMESTAMP WITH LOCAL TIME ZONE` data type.

Note

Setting the session time zone does not affect the value returned by the `SYSDATE` and `SYSTIMESTAMP` SQL functions. These functions return the date and time using the time zone of either the database host system or the PDB, depending on the value of the `TIME_AT_DBTIMEZONE` initialization parameter. See *Oracle AI Database Reference* for more information.

The `ORA_SDTZ` environment variable can be set to the following values:

- Operating system local time zone ('OS_TZ')
- Database time zone ('DB_TZ')
- Absolute offset from UTC (for example, '-05:00')
- Time zone region name (for example, 'Europe/London')

To set `ORA_SDTZ`, use statements similar to one of the following in a UNIX environment (C shell):

```
% setenv ORA_SDTZ 'OS_TZ'  
% setenv ORA_SDTZ 'DB_TZ'  
% setenv ORA_SDTZ 'Europe/London'  
% setenv ORA_SDTZ '-05:00'
```

When setting the `ORA_SDTZ` variable in a Microsoft Windows environment -- in the Registry, among system environment variables, or in a command prompt window -- do not enclose the time zone value in quotes.

The default value of the `ORA_SDTZ` variable, which is used when the variable is not set or it is set to an invalid value, is 'OS_TZ'.

You can change the time zone for a specific SQL session with the `SET TIME_ZONE` clause of the `ALTER SESSION` statement.

`TIME_ZONE` can be set to the following values:

- Default local time zone when the session was started (local)
- Database time zone (dbtimezone)
- Absolute offset from UTC (for example, '+10:00')
- Time zone region name (for example, 'Asia/Hong_Kong')

Use ALTER SESSION statements similar to the following:

```
ALTER SESSION SET TIME_ZONE=local;
ALTER SESSION SET TIME_ZONE=dbtimezone;
ALTER SESSION SET TIME_ZONE='Asia/Hong_Kong';
ALTER SESSION SET TIME_ZONE='+10:00';
```

You can find out the current session time zone by entering the following query:

```
SELECT sessiontimezone FROM DUAL;
```

4.11 Converting Time Zones With the AT TIME ZONE Clause

A datetime SQL expression can be one of the following:

- A datetime column
- A compound expression that yields a datetime value

A datetime expression can include an AT LOCAL clause or an AT TIME ZONE clause. If you include an AT LOCAL clause, then the result is returned in the current session time zone. If you include the AT TIME ZONE clause, then use one of the following settings with the clause:

- Time zone offset: The string '(+|-)HH:MM' specifies a time zone as an offset from UTC. For example, '-07:00' specifies the time zone that is 7 hours behind UTC. For example, if the UTC time is 11:00 a.m., then the time in the '-07:00' time zone is 4:00 a.m.
- DBTIMEZONE: Oracle Database uses the database time zone established (explicitly or by default) during database creation.
- SESSIONTIMEZONE: Oracle Database uses the session time zone established by default or in the most recent ALTER SESSION statement.
- Time zone region name: Oracle Database returns the value in the time zone indicated by the time zone region name. For example, you can specify Asia/Hong_Kong.
- An expression: If an expression returns a character string with a valid time zone format, then Oracle Database returns the input in that time zone. Otherwise, Oracle Database returns an error.

See Also

Oracle AI Database SQL Language Reference

The following example converts the datetime value in the America/New_York time zone to the datetime value in the America/Los_Angeles time zone.

Example 4-5 Converting a Datetime Value to Another Time Zone

```
SELECT FROM_TZ(CAST(TO_DATE('1999-12-01 11:00:00',
    'YYYY-MM-DD HH:MI:SS') AS TIMESTAMP), 'America/New_York')
    AT TIME ZONE 'America/Los_Angeles' "West Coast Time"
FROM DUAL;
```

West Coast Time

```
-----
01-DEC-99 08.00.00.000000 AM AMERICA/LOS_ANGELES
```


4.12 Support for Daylight Saving Time

Oracle Database automatically determines whether Daylight Saving Time is in effect for a specified time zone and returns the corresponding local time. Normally, date/time values are sufficient to allow Oracle Database to determine whether Daylight Saving Time is in effect for a specified time zone. The periods when Daylight Saving Time begins or ends are boundary cases. For example, in the Eastern region of the United States, the time changes from 01:59:59 a.m. to 3:00:00 a.m. when Daylight Saving Time goes into effect. The interval between 02:00:00 and 02:59:59 a.m. does not exist. Values in that interval are invalid. When Daylight Saving Time ends, the time changes from 02:00:00 a.m. to 01:00:01 a.m. The interval between 01:00:01 and 02:00:00 a.m. is repeated. Values from that interval are ambiguous because they occur twice.

To resolve these boundary cases, Oracle Database uses the TZR and TZD format elements. TZR represents the time zone region in datetime input strings. Examples are 'Australia/North', 'UTC', and 'Singapore'. TZD represents an abbreviated form of the time zone region with Daylight Saving Time information. Examples are 'PST' for U. S. Pacific Standard Time and 'PDT' for U. S. Pacific Daylight Time. To see a list of valid values for the TZR and TZD format elements, query the TZNAME and TZABBREV columns of the V\$TIMEZONE_NAMES dynamic performance view.

① See Also

- *Oracle AI Database SQL Language Reference* for more information regarding the session parameter `ERROR_ON_OVERLAP_TIME`
- "[Time Zone Region Names](#)" for a list of valid time zones

4.12.1 Examples: The Effect of Daylight Saving Time on Datetime Calculations

The `TIMESTAMP` data type does not accept time zone values and does not calculate Daylight Saving Time.

The `TIMESTAMP WITH TIME ZONE` and `TIMESTAMP WITH LOCAL TIME ZONE` data types have the following behavior:

- If a time zone region is associated with the datetime value, then the database server knows the Daylight Saving Time rules for the region and uses the rules in calculations.
- Daylight Saving Time is not calculated for regions that do not use Daylight Saving Time.

The rest of this section contains examples that use datetime data types. The examples use the `global_orders` table. It contains the `orderdate1` column of `TIMESTAMP` data type and the `orderdate2` column of `TIMESTAMP WITH TIME ZONE` data type. The `global_orders` table is created as follows:

```
CREATE TABLE global_orders ( orderdate1 TIMESTAMP(0),
                             orderdate2 TIMESTAMP(0) WITH TIME ZONE);
INSERT INTO global_orders VALUES ( '28-OCT-00 11:24:54 PM',
                                   '28-OCT-00 11:24:54 PM America/New_York');
```

Note

If you have created a `global_orders` table for the previous examples, then drop the `global_orders` table before you try [Example 4-7](#) through [Example 4-8](#).

Example 4-6 Comparing Daylight Saving Time Calculations Using TIMESTAMP WITH TIME_ZONE and TIMESTAMP

```
SELECT orderdate1 + INTERVAL '8' HOUR, orderdate2 + INTERVAL '8' HOUR
       FROM global_orders;
```

The following output results:

ORDERDATE1+INTERVAL '8' HOUR	ORDERDATE2+INTERVAL '8' HOUR
-----	-----
29-OCT-00 07.24.54.000000 AM	29-OCT-00 06.24.54.000000 AM AMERICA/NEW_YORK

This example shows the effect of adding 8 hours to the columns. The time period includes a Daylight Saving Time boundary (a change from Daylight Saving Time to standard time). The `orderdate1` column is of `TIMESTAMP` data type, which does not use Daylight Saving Time information and thus does not adjust for the change that took place in the 8-hour interval. The `TIMESTAMP WITH TIME_ZONE` data type does adjust for the change, so the `orderdate2` column shows the time as one hour earlier than the time shown in the `orderdate1` column.

Example 4-7 Comparing Daylight Saving Time Calculations Using TIMESTAMP WITH LOCAL TIME_ZONE and TIMESTAMP

The `TIMESTAMP WITH LOCAL TIME_ZONE` data type uses the value of `TIME_ZONE` that is set for the session environment. The following statements set the value of the `TIME_ZONE` session parameter and create a `global_orders` table. The `global_orders` table has one column of `TIMESTAMP` data type and one column of `TIMESTAMP WITH LOCAL TIME_ZONE` data type.

```
ALTER SESSION SET TIME_ZONE='America/New_York';
CREATE TABLE global_orders ( orderdate1 TIMESTAMP(0),
                             orderdate2 TIMESTAMP(0) WITH LOCAL TIME_ZONE );
INSERT INTO global_orders VALUES ( '28-OCT-00 11:24:54 PM',
                                   '28-OCT-00 11:24:54 PM' );
```

Add 8 hours to both columns.

```
SELECT orderdate1 + INTERVAL '8' HOUR, orderdate2 + INTERVAL '8' HOUR
       FROM global_orders;
```

Because a time zone region is associated with the datetime value for `orderdate2`, the Oracle Database server uses the Daylight Saving Time rules for the region. Thus the output is the same as in [Example 4-6](#). There is a one-hour difference between the two calculations because Daylight Saving Time is not calculated for the `TIMESTAMP` data type, and the calculation crosses a Daylight Saving Time boundary.

Example 4-8 Daylight Saving Time Is Not Calculated for Regions That Do Not Use Daylight Saving Time

Set the time zone region to UTC. UTC does not use Daylight Saving Time.

```
ALTER SESSION SET TIME_ZONE='UTC';
```

Truncate the `global_orders` table.

```
TRUNCATE TABLE global_orders;
```

Insert values into the global_orders table.

```
INSERT INTO global_orders VALUES ( '28-OCT-00 11:24:54 PM',  
                                     TIMESTAMP '2000-10-28 23:24:54 ' );
```

Add 8 hours to the columns.

```
SELECT orderdate1 + INTERVAL '8' HOUR, orderdate2 + INTERVAL '8' HOUR  
FROM global_orders;
```

The following output results.

ORDERDATE1+INTERVAL '8' HOUR	ORDERDATE2+INTERVAL '8' HOUR
-----	-----
29-OCT-00 07.24.54.000000000 AM	29-OCT-00 07.24.54.000000000 AM UTC

The times are the same because Daylight Saving Time is not calculated for the UTC time zone region.

5

Linguistic Sorting and Matching

This chapter explains the mechanism of linguistic sorting and searching of character data or strings in Oracle Database. The process of determining the mutual ordering of strings (character values) is called a [collation](#). For any two strings, the collation defines whether the strings are equal or whether one precedes the other in the sorting order. In the Oracle documentation, the term [sort](#) is often used in place of *collation*.

Determining equality is especially important when a set of strings, such as a table column, is searched for values that equal a specified search term or that match a search pattern. SQL operators and functions used in searching are =, LIKE, REGEXP_LIKE, INSTR, and REGEXP_INSTR. This chapter uses the term *matching* to mean determining the equality of entire strings using the equality operator = or determining the equality of substrings of a string when the string is matched against a pattern using LIKE, REGEXP_LIKE or REGEXP_INSTR. Note that Oracle Text provides advanced full-text searching capabilities for the Oracle Database.

The ordering of strings in a set is called *sorting*. For example, the ORDER BY clause uses collation to determine the ordering of strings to sort the query results, while PL/SQL uses collations to sort strings in associative arrays indexed by VARCHAR2 values, and the functions MIN, MAX, GREATEST, and LEAST use collations to find the smallest or largest character value.

There are many possible collations that can be applied to strings to determine their ordering. Collations that take into consideration the standards and customs of spoken languages are called [linguistic collations](#). They order strings in the same way as dictionaries, phone directories, and other text lists written in a given language. In contrast, [binary collation](#) orders strings based on their binary representation (character encoding), treating each string as a simple sequences of bytes.

See Also

Oracle Text Application Developer's Guide

The following topics explain linguistic sorting and matching:

- [Overview of Oracle Database Collation Capabilities](#)
- [Using Binary Collation](#)
- [Using Emulated Binary Collations](#)
- [Using Linguistic Collations](#)
- [Linguistic Collation Features](#)
- [Case-Insensitive and Accent-Insensitive Linguistic Collation](#)
- [Performing Linguistic Comparisons](#)
- [Using Linguistic Indexes](#)
- [Searching Linguistic Strings](#)
- [SQL Regular Expressions in a Multilingual Environment](#)

- [Column-Level Collation and Case Sensitivity](#)

5.1 Overview of Oracle Database Collation Capabilities

Different languages have different collations. In addition, different cultures or countries that use the same alphabets may sort words differently. For example, in Danish, Æ is after Z, while Y and Ü are considered to be variants of the same letter.

Collation can be case-sensitive or case-insensitive. **Case** refers to the condition of being uppercase or lowercase. For example, in a Latin alphabet, A is the uppercase glyph for a, the lowercase glyph.

Collation can ignore or consider diacritics. A **diacritic** is a mark near or through a character or combination of characters that indicates a different sound than the sound of the character without the diacritic. For example, the cedilla (,) in façade is a diacritic. It changes the sound of c.

Collation order can be phonetic or it can be based on the appearance of the character. For example, collation can be based on the number of strokes in East Asian ideographs. Another common collation issue is combining letters into a single character. For example, in traditional Spanish, ch is a distinct character that comes after c, which means that the correct order is: cerveza, colorado, cheremoya. This means that the letter c cannot be sorted until Oracle Database has checked whether the next letter is an h.

Oracle Database provides the following types of collation:

- Binary
- Emulated binary
- Monolingual
- Multilingual
- Unicode Collation Algorithm (UCA)

While monolingual collation achieves a linguistically correct order for a single language, multilingual collation and UCA collation are designed to handle many languages at the same time. Furthermore, UCA collation conforms to the Unicode Collation Algorithm (UCA) that is a Unicode standard and is fully compatible with the international collation standard ISO 14651. The UCA standard provides a complete linguistic ordering for all characters in Unicode, hence all the languages around the world. With wide deployment of Unicode application, UCA collation is best suited for sorting multilingual data.

An emulated binary collation is based on a specific character set and replicates its sorting order by converting values to the base character set before applying binary collation. Emulated binary collations have been available since Oracle AI Database 26ai, Release Update 23.26.2.

5.2 Using Binary Collation

One way to sort character data is based on the numeric values of the characters defined by the character encoding scheme. This is called a **binary collation**. Binary collation is the fastest type of sort. It produces reasonable results for the English alphabet because the ASCII and EBCDIC standards define the letters A to Z in ascending numeric value.

Note

In the ASCII standard, all uppercase letters appear before any lowercase letters. In the EBCDIC standard, the opposite is true: all lowercase letters appear before any uppercase letters.

When characters used in other languages are present, a binary collation usually does not produce reasonable results. For example, an ascending `ORDER BY` query returns the character strings ABC, ABZ, BCD, ÄBC, when Ä has a higher numeric value than B in the character encoding scheme. A binary collation is not usually linguistically meaningful for Asian languages that use ideographic characters.

5.3 Using Emulated Binary Collations

Emulated binary collations have been implemented for several base character sets, allowing data to be sorted in the same order it would have under the binary collation of a database using that base character set. These collations are particularly useful for preserving sorting order when migrating applications from EBCDIC-based mainframe platforms to modern environments, such as Linux. See "[Emulating EBCDIC Binary Ordering](#)" for more details.

An emulated binary collation converts the data to be sorted into the collation's base character set and then applies binary comparison. Characters from the database character set that are not defined in the base character set are treated as equal to the question mark '?' (the default replacement character). Using emulated binary collations with data containing such characters is not recommended.

For some collation-sensitive functions (for example, `INSTR`), the conversion step is omitted and the standard binary collation is used directly. In most cases, this optimization does not introduce any visible semantic differences. However, if the base character set defines a local code point that can map to two different Unicode characters (via the replacement character mechanism), then those characters compared (in Unicode or other supporting character sets) would be considered equal without the optimization but are considered different with the optimization. Specifically, in the currently supported emulated binary collations, this difference can arise when comparing characters that are not present in the base character set, or when comparing such a character with the question mark '?'.

5.4 Using Linguistic Collations

To produce a collation sequence that matches the alphabetic sequence of characters, another sorting technique must be used that sorts characters independently of their numeric values in the character encoding scheme. This technique is called a **linguistic collation**. A linguistic collation operates by replacing characters with numeric values that reflect each character's proper linguistic order.

This section includes the following topics:

- [Monolingual Collation](#)
- [Multilingual Collation](#)
- [UCA Collation](#)

5.4.1 Monolingual Collation

Oracle Database compares character strings in two steps for monolingual collation. The first step compares the major value of the entire string from a table of major values. Usually, letters with the same appearance have the same major value. The second step compares the minor value from a table of minor values. The major and minor values are defined by Oracle Database. Oracle Database defines letters with diacritic and case differences as having the same major value but different minor values.

Each major table entry contains the **Unicode code point** and major value for a character. The Unicode code point is a 16-bit binary value that represents a character.

The following table illustrates sample values for sorting a, A, ä, Ä, and b.

Table 5-1 Sample Glyphs and Their Major and Minor Sort Values

Glyph	Major Value	Minor Value
a	15	5
A	15	10
ä	15	15
Ä	15	20
b	20	5

Note

Monolingual collation is not available for non-Unicode multibyte database character sets. If a monolingual collation is specified when the database character set is non-Unicode multibyte, then the default sort order is the binary sort order of the database character set. One exception is `UNICODE_BINARY`. This collation is available for all character sets.

See Also

["What is the Unicode Standard?"](#)

5.4.2 Multilingual Collation

Oracle Database provides multilingual collation so that you can sort data in more than one language in one sort. This is useful for regions or languages that have complex sorting rules and for multilingual databases. Note that Oracle Database supports all of the collations defined in the previous releases.

For Asian language data or multilingual data, Oracle Database provides a sorting mechanism based on the ISO 14651 standard. For example, Chinese characters can be ordered by the number of strokes, PinYin, or radicals.

In addition, multilingual collation can handle canonical equivalence and supplementary characters. **Canonical equivalence** is a basic equivalence between characters or sequences

of characters. For example, ç is equivalent to the combination of c and , . **Supplementary characters** are user-defined characters or predefined characters in Unicode that require two code points within a specific code range. You can define up to 1.1 million code points in one multilingual sort.

For example, Oracle Database supports a monolingual French sort (FRENCH), but you can specify a multilingual French collation (FRENCH_M). _M represents the ISO 14651 standard for multilingual sorting. The sorting order is based on the GENERIC_M sorting order and can sort diacritical marks from right to left. Multilingual linguistic sort is usually used if the tables contain multilingual data. If the tables contain only French, then a monolingual French sort might have better performance because it uses less memory. It uses less memory because fewer characters are defined in a monolingual French sort than in a multilingual French sort. There is a trade-off between the scope and the performance of a sort.

 See Also

- ["Canonical Equivalence"](#)
- ["Code Points and Supplementary Characters"](#)

5.4.2.1 Multilingual Collation Levels

Oracle Database evaluates multilingual collation at three levels of precision:

- [Primary Level Collation](#)
- [Secondary Level Collation](#)
- [Tertiary Level Collation](#)

5.4.2.1.1 Primary Level Collation

A primary level collation distinguishes between **base letters**, such as the difference between characters a and b. It is up to individual locales to define whether a is before b, b is before a, or if they are equal. The binary representation of the characters is completely irrelevant. If a character is an ignorable character, then it is assigned a primary level **order** (or weight) of zero, which means it is ignored at the primary level. Characters that are ignorable on other levels are given an order of zero at those levels.

For example, at the primary level, all variations of bat come before all variations of bet. The variations of bat can appear in any order, and the variations of bet can appear in any order:

Bat
bat
BAT
BET
Bet
bet

 See Also

- ["Ignorable Characters"](#)

5.4.2.1.2 Secondary Level Collation

A secondary level collation distinguishes between base letters (the primary level collation) before distinguishing between diacritics on a given base letter. For example, the character Ä differs from the character A only because it has a diacritic. Thus, Ä and A are the same on the primary level because they have the same base letter (A) but differ on the secondary level.

The following list has been sorted on the primary level (resume comes before resumes) and on the secondary level (strings without diacritics come before strings with diacritics):

```
resume
résumé
Résumé
Resumes
resumes
résumés
```

5.4.2.1.3 Tertiary Level Collation

A tertiary level collation distinguishes between base letters (primary level collation), diacritics (secondary level collation), and case (upper case and lower case). It can also include special characters such as +, -, and *.

The following are examples of tertiary level collations:

- Characters a and A are equal on the primary and secondary levels but different on the tertiary level because they have different cases.
- Characters ä and A are equal on the primary level and different on the secondary and tertiary levels.
- The primary and secondary level orders for the dash character - is 0. That is, it is ignored on the primary and secondary levels. If a dash is compared with another character whose primary level weight is nonzero, for example, u, then no result for the primary level is available because u is not compared with anything. In this case, Oracle Database finds a difference between - and u only at the tertiary level.

The following list has been sorted on the primary level (resume comes before resumes) and on the secondary level (strings without diacritics come before strings with diacritics) and on the tertiary level (lower case comes before upper case):

```
resume
Resume
résumé
Résumé
resumes
Resumes
résumés
Résumés
```

5.4.3 UCA Collation

Unicode Collation Algorithm (UCA) is a Unicode standard that is fully compatible with the international collation standard ISO 14651. UCA defines a Default Unicode Collation Element Table (DUCET) that provides a reasonable default ordering for all languages that are not tailored. To achieve the correct ordering for a particular language, DUCET can be tailored to meet the linguistic requirements for that language. There are tailorings of DUCET for various languages provided in the Unicode Common Locale Data Repository.

This Oracle Database release provides UCA collation that fully conforms to UCA 12.1. In addition to the collation based on DUCET, it provides tailored collations for a number of commonly used languages. For example, you can specify UCA collation UCA1210_SCHINESE to sort multilingual data containing Simplified Chinese. The collation will make Simplified Chinese data appear in the PinYin order.

For sorting multilingual data, Oracle recommends the latest supported version of UCA collations.

This section describes the following topics:

- [UCA Comparison Levels](#)
- [UCA Collation Parameters](#)
- [The UCA Japanese IVS Collation](#)

See Also

The Unicode Consortium website for more information about Unicode Collation Algorithm and related terminologies

5.4.3.1 UCA Comparison Levels

Similar to multilingual collation, UCA collations employ a multilevel comparison algorithm to evaluate characters. This can go up to four levels of comparison:

- [Primary Level](#)
- [Secondary Level](#)
- [Tertiary Level](#)
- [Quaternary Level](#)

5.4.3.1.1 Primary Level

The primary level is used to distinguish between base letters, which is similar to the comparison used in the primary level collation of the multilingual collation.

See Also

"[Primary Level Collation](#)" for examples of base letter differences

5.4.3.1.2 Secondary Level

The secondary level is used to distinguish between diacritics if base letters are the same, which is similar to what is used in the secondary level collation of the multilingual collation to distinguish between diacritics.

See Also

"[Secondary Level Collation](#)" for examples of diacritic differences

5.4.3.1.3 Tertiary Level

The tertiary level is used to distinguish between cases on a given base letter with the same diacritic, which is similar to what is used in the tertiary level collation of the multilingual collation to distinguish between cases. Moreover, UCA DUCET collation treats punctuations with primary or quaternary significance based on how variable characters are weighted, which is different from the tertiary level collation of the multilingual collation that treat punctuations with tertiary level of significance.

See Also
"[Tertiary Level Collation](#)" for examples of characters with case differences

5.4.3.1.4 Quaternary Level

The quaternary level is used to distinguish variable characters from other characters, if variable characters are weighted as shifted. It is also used to distinguish Hiragana from Katakana with the same base and case. An example is illustrated in the following figure.

Figure 5-1 Hiragana and Katakana Collation

あ =₃ア (あ and ア are equal on the first three levels)

あ <₄ア (あ is less than ア on the quaternary level)

See Also
"[UCA Collation Parameters](#)"

5.4.3.2 UCA Collation Parameters

The following table illustrates the collation parameters and options that are supported in UCA collations in this release.

Table 5-2 UCA Collation Parameters

Attribute	Options	Collation Modifier
strength	primary	_AI or _S1
	secondary	_CI or _S2
	tertiary	_S3
	quaternary	_S4 (Only applicable when the alternate attribute is set to shifted)

Table 5-2 (Cont.) UCA Collation Parameters

Attribute	Options	Collation Modifier
alternate	non-ignorable	_VN
	shifted	_VS
	blanked	_VB
backwards	on	_BY
	off	_BN
normalization	on	_NY
caseLevel	off	_EN
caseFirst	upper	_FU (Only valid for Danish)
	off	_FN (Only valid for other languages)
hiraganaQuaternary	on	_HY
(Deprecated as of UCA 7.0)	off	_HN
numeric	off	_DN
match-style	minimal	_MN

The parameter strength represents UCA comparison level.

The parameter alternate controls how variable characters are weighted.

The parameter backwards controls if diacritics are to be sorted backward.

The parameter hiraganaQuaternary is applicable to the UCA collations for the Japanese language only. It has no effect on other collations. If it is set to "on" (_HY), then the corresponding Hiragana and Katakana characters have different quaternary weights. Otherwise, they have the same weights. The hiraganaQuaternary parameter is deprecated as of UCA 7.0.

You can configure the preceding four UCA parameters using the options listed in [Table 5-2](#). The options for the other parameters listed in [Table 5-2](#) are currently fixed based on tailored languages and are not configurable.

① See Also

- ["UCA Comparison Levels"](#)
- *The Unicode Consortium* website for a complete description of UCA collation parameters and options

5.4.3.3 The UCA Japanese IVS Collation

An Ideographic Variation Sequence (IVS) is a sequence of two coded characters, the first being a character with the Ideographic property that is not canonically nor compatibly decomposable, the second being a Variation Selector (VS) character in the range U+E0100 to U+E01EF. There are other Variation Selector characters defined in Unicode, specifically, U+180B to U+180D and U+FE00 to U+FE0F. An ideographic character described above followed by any of these characters is not recognized as an IVS.¹

Historically, Oracle has provided UCA collations for the Japanese language, such as the UCA1210_JAPANESE collation. However, these collations take only the ideographic character of an IVS into account when performing linguistic collation on Japanese text.

Starting with Oracle AI Database 26ai, Oracle also offers the UCA1210_JAPANESE_IVS collation, which takes the entire IVS (ideographic character and Variation Selector) into account when performing linguistic collation on Japanese text. This includes linguistic sorting with the ORDER BY clause, as well as linguistic matching with the INSTR function, the LIKE condition, and comparison conditions, such as =, <, and >.

The following example illustrates the difference between the UCA1210_JAPANESE and UCA1210_JAPANESE_IVS collations when performing linguistic matching.

First, specify linguistic comparisons for the database session:

```
ALTER SESSION SET NLS_COMP = LINGUISTIC;
```

Create a table and populate it with three IVSs. Note that the IVSs all use the same ideographic character (U+845B), but have slightly different Variation Selectors (U+E010n):

```
CREATE TABLE test (col1 number, col2 VARCHAR2(20));
INSERT INTO test VALUES (0, UNISTR('\845B\DB40\DD00'));
INSERT INTO test VALUES (1, UNISTR('\845B\DB40\DD01'));
INSERT INTO test VALUES (2, UNISTR('\845B\DB40\DD02'));
```

Specify the UCA1210_JAPANESE collation for the session:

```
ALTER SESSION SET NLS_SORT = UCA1210_JAPANESE;
```

Perform the following SELECT statement on the table. Notice that all rows are selected because only the ideographic character is taken into account when performing linguistic matching.

```
SELECT col1 FROM test
WHERE col2 = UNISTR('\845B\DB40\DD00');
```

```
COL1
-----
0
1
2
```

Now, specify the UCA1210_JAPANESE_IVS collation for the session:

```
ALTER SESSION SET NLS_SORT = UCA1210_JAPANESE_IVS;
```

Perform the same SELECT statement on the table. Notice that only the first row is selected because the entire IVS is taken into account when performing linguistic matching.

```
SELECT col1 FROM test
WHERE col2 = UNISTR('\845B\DB40\DD00');
```

¹ Unicode® Technical Standard #37, "Unicode Ideographic Variation Database" (<http://www.unicode.org/reports/tr37/tr37-14.html>)

COL1

0

① **See Also**

- ["Code Points and Supplementary Characters"](#)
- ["Unicode Encoding Forms"](#)
- [Table A-19](#)

5.5 Linguistic Collation Features

This section contains information about different features that a linguistic collation can have:

- [Base Letters](#)
- [Ignorable Characters](#)
- [Contracting Characters](#)
- [Expanding Characters](#)
- [Context-Sensitive Characters](#)
- [Canonical Equivalence](#)
- [Reverse Secondary Sorting](#)
- [Character Rearrangement for Thai and Laotian Characters](#)
- [Special Letters](#)
- [Special Combination Letters](#)
- [Special Uppercase Letters](#)
- [Special Lowercase Letters](#)

You can customize linguistic collations to include the desired characteristics.

① **See Also**

["Customizing Locale Data"](#)

5.5.1 Base Letters

Base letters are defined in a base letter table, which maps each letter to its base letter. For example, a, A, ä, and Ä all map to a, which is the **base letter**. This concept is particularly relevant for working with Oracle Text.

 See Also*Oracle Text Reference*

5.5.2 Ignorable Characters

In multilingual collation and UCA collation, certain characters may be treated as ignorable.

Ignorable characters are skipped, that is, treated as non-existent, when two character values (strings) containing such characters are compared in a sorting or matching operation. There are three kinds of ignorable characters: primary, secondary, and tertiary.

- [Primary Ignorable Characters](#)
- [Secondary Ignorable Characters](#)
- [Tertiary Ignorable Characters](#)

5.5.2.1 Primary Ignorable Characters

Primary ignorable characters are ignored when the multilingual collation or UCA collation definition applied to the given comparison has the accent-insensitivity modifier `_AI`, for example, `GENERIC_M_AI` or `UCA1210_DUCET_AI`.

Primary ignorable characters are comprised of diacritics (accents) from various alphabets (Latin, Cyrillic, Greek, Devanagari, Katakana, and so on) and also of decorating modifiers, such as an enclosing circle or enclosing square. These characters are non-spacing combining characters, which means they combine with the preceding character to form a complete accented or decorated character ("non-spacing" means that the character occupies the same character position on screen or paper as the preceding character). For example, the character "Latin Small Letter e" followed by the character "Combining Grave Accent" forms a single letter "è", while the character "Latin Capital Letter A" followed by the "Combining Enclosing Circle" forms a single character "(A)". Because non-spacing characters are defined as ignorable for accent-insensitive sorts, these sorts can treat, for example, rôle as equal to role, naïve as equal to naive, and (A)(B)(C) as equal to ABC.

Primary ignorable characters are called non-spacing characters when viewed in a multilingual collation definition in the Oracle Locale Builder utility.

5.5.2.2 Secondary Ignorable Characters

Secondary ignorable characters are ignored when the applied definition has either the accent-insensitivity modifier `_AI` or the case-insensitivity modifier `_CI`.

In multilingual collation, secondary ignorable characters are comprised of punctuation characters, such as the space character, new line control codes, dashes, various quote forms, mathematical operators, dot, comma, exclamation mark, various bracket forms, and so on. In accent-insensitive (`_AI`) and case-insensitive (`_CI`) sorts, these punctuation characters are ignored so that multi-lingual can be treated as equal to multilingual and e-mail can be treated as equal to email.

Secondary ignorable characters are called punctuation characters when viewed in a multilingual collation definition in the Oracle Locale Builder utility.

There are no secondary ignorable characters defined in the UCA DUCET, however. Punctuations are treated as variable characters in the UCA.

5.5.2.3 Tertiary Ignorable Characters

Tertiary ignorable characters are generally ignored in linguistic comparison. They mainly comprise control codes, format characters, variation selectors, and so on. An exception is the UCA1210_JAPANESE_IVS collation, which does not ignore the variation selectors U+E0100 to U+E01EF when they are part of an Ideographic Variation Sequence (IVS).

Primary and secondary ignorable characters are not ignored when a standard, case- and accent-sensitive sort is used. However, they have lower priority when determining the order of strings. For example, `multi-lingual` is sorted after `multilingual` in the `GENERIC_M` sort, but it is still sorted between `multidimensional` and `multinational`. The comparison `d < l < n` of the base letters has higher priority in determining the order than the presence of the secondary ignorable character `HYPHEN (U+002D)`.

You can see the full list of non-spacing characters and punctuation characters in a multilingual collation definition when viewing the definition in the Oracle Locale Builder. Generally, neither punctuation characters nor non-spacing characters are included in monolingual collation definitions. In some monolingual collation definitions, the space character and the tabulator character may be included. The comparison algorithm automatically assigns a minor value to each undefined character. This makes punctuation characters non-ignorable but, as in the case of multilingual collations, considered with lower priority when determining the order of compared strings. The ordering among punctuation characters in monolingual collations is based on their Unicode code points and may not correspond to user expectations.

① See Also

["Case-Insensitive and Accent-Insensitive Linguistic Collation"](#)

5.5.3 Variable Characters and Variable Weighting

There are characters defined with variable collation elements in the UCA. These characters are called variable characters and are comprised of white space characters, punctuations, and certain symbols.

Variable characters can be weighted differently in UCA collations to adjust the effect of these characters in a sorting or comparison, which is called variable weighting. The collation parameter, `alternate`, controls how it works. The following options on variable weighting are supported in UCA collations in this release:

- `blanked`

Variable characters are treated as ignorable characters. For example, `SPACE (U+0020)` is ignored in comparison.

- `non-ignorable`

Variable characters are treated as if they were not ignorable characters. For example, `SPACE (U+0020)` is not ignored in comparison at primary level.

- `shifted`

Variable characters are treated as ignorable characters on the primary, secondary and tertiary levels. In addition, a new quaternary level is used for all characters. The quaternary weight of a character depends on if the character is a variable, ignorable, or other. For example, `SPACE (U+0020)` is assigned a quaternary weight differently from `A (U+0041)`.

because SPACE is a variable character while A is neither a variable nor an ignorable character.

① **See Also**

["UCA Collation Parameters"](#)

Examples of Variable Weighting

This section includes different examples of variable weighting.

Example 5-1 UCA DUCET Order When Variable is Weighed as Blanked

The following list has been sorted using UCA1210_DUCET_VB:

```
blackbird
Blackbird
Black-bird
Black bird
BlackBird
```

Blackbird, Black-bird, and Black bird have the same collation weight because SPACE(U+0020) and HYPHEN(U+002D) are treated as ignorable characters. Selecting only the distinct entries illustrates this behavior (note that only Blackbird is shown in the result):

```
blackbird
Blackbird
BlackBird
```

Blackbird, Black-bird, and Black bird are sorted after blackbird due to case difference on the first letter B (U+0042), but before BlackBird due to case difference at the second b (U+0062).

Example 5-2 UCA DUCET Order When Variable is Weighed as Non-Ignorable

The following list has been sorted using UCA1210_DUCET_VN:

```
Black bird
Black-bird
blackbird
Blackbird
BlackBird
```

Black bird and Black-bird are sorted before blackbird because both SPACE (U+0020) and HYPHEN (U+002D) are not treated as ignorable characters but they are smaller than b (U+0062) at the primary level. Black bird is sorted before Black-bird because SPACE (U+0020) is small than HYPHEN (U+002D) at the primary level.

Example 5-3 UCA DUCET Order When Variable is Weighed as Shifted

The following list has been sorted using UCA1210_DUCET:

```
blackbird
Black bird
Black-bird
Blackbird
BlackBird
```

blackbird is sorted before Black bird and Black-bird because both SPACE (U+0020) and HYPHEN (U+002D) are ignored at the first three levels, and there is a case difference on the first letter b (U+0062). Black-bird is sorted before Blackbird because HYPHEN (U+002D) has a small quaternary weight than the letter b (U+0062) in Blackbird.

5.5.4 Contracting Characters

Collation elements usually consist of a single character, but in some locales, two or more characters in a character string must be considered as a single collation element during sorting. For example, in traditional Spanish, the string *ch* is composed of two characters. These characters are called **contracting characters** in multilingual collation and **special combination letters** in monolingual collation.

Do not confuse a **composed character** with a contracting character. A composed character like *á* can be decomposed into *a* and *'*, each with their own encoding. The difference between a composed character and a contracting character is that a composed character can be displayed as a single character on a terminal, while a contracting character is used only for sorting, and its component characters must be rendered separately.

5.5.5 Expanding Characters

In some locales, certain characters must be sorted as if they were character strings. An example is the German character *ß* (sharp s). It is sorted exactly the same as the string *ss*. Another example is that *ö* sorts as if it were *oe*, after *od* and before *of*. These characters are known as **expanding characters** in multilingual collation and **special letters** in monolingual collation. Just as with contracting characters, the replacement string for an expanding character is meaningful only for sorting.

5.5.6 Context-Sensitive Characters

In Japanese, a prolonged sound mark that resembles an em dash – represents a length mark that lengthens the vowel of the preceding character. The sort order depends on the vowel that precedes the length mark. This is called context-sensitive collation. For example, after the character *ka*, the – length mark indicates a long *a* and is treated the same as *a*, while after the character *ki*, the – length mark indicates a long *i* and is treated the same as *i*. Transliterating this to Latin characters, a sort might look like this:

```
kaa
ka-  -- kaa and ka- are the same
kai  -- kai follows ka- because i is after a
kia  -- kia follows kai because i is after a
kii  -- kii follows kia because i is after a
ki-  -- kii and ki- are the same
```

5.5.7 Canonical Equivalence

Canonical equivalence is an attribute of a multilingual collation and describes how equivalent code point sequences are sorted. If canonical equivalence is applied in a particular multilingual collation, then canonically equivalent strings are treated as equal.

One Unicode code point can be equivalent to a sequence of base letter code points plus diacritic code points. This is called the Unicode canonical equivalence. For example, *ä* equals its base letter *a* and an umlaut. A linguistic flag, `CANONICAL_EQUIVALENCE = TRUE`, indicates that all canonical equivalence rules defined in Unicode need to be applied in a specific multilingual collation. Oracle Database-defined multilingual collations include the appropriate setting for the

canonical equivalence flag. You can set the flag to FALSE to speed up the comparison and ordering functions if all the data is in its composed form.

For example, consider the following strings:

- äa (a umlaut followed by a)
- a"b (a followed by umlaut followed by b)
- äc (a umlaut followed by c)

If CANONICAL_EQUIVALENCE=FALSE, then the sort order of the strings is:

```
a"b
äa
äc
```

This occurs because a comes before ä if canonical equivalence is not applied.

If CANONICAL_EQUIVALENCE=TRUE, then the sort order of the strings is:

```
äa
a"b
äc
```

This occurs because ä and a" are treated as canonically equivalent.

You can use Oracle Locale Builder to view the setting of the canonical equivalence flag in existing multilingual collations. When you create a customized multilingual collation with Oracle Locale Builder, you can set the canonical equivalence flag as desired.

① See Also

["Creating a New Linguistic Sort with the Oracle Locale Builder"](#) for more information about setting the canonical equivalence flag

5.5.8 Reverse Secondary Sorting

In French, sorting strings of characters with diacritics first compares base letters from left to right, but compares characters with diacritics from right to left. For example, by default, a character with a diacritic is placed after its unmarked variant. Thus Édít comes before Edít in a French sort. They are equal on the primary level, and the secondary order is determined by examining characters with diacritics from right to left. Individual locales can request that the characters with diacritics be sorted with the right-to-left rule. Set the REVERSE_SECONDARY linguistic flag to TRUE to enable reverse secondary sorting.

① See Also

["Creating a New Linguistic Sort with the Oracle Locale Builder"](#) for more information about setting the reverse secondary flag

5.5.9 Character Rearrangement for Thai and Laotian Characters

In Thai and Lao, some characters must first change places with the following character before sorting. Normally, these types of characters are symbols representing vowel sounds, and the next character is a consonant. Consonants and vowels must change places before sorting. Set the `SWAP_WITH_NEXT` linguistic flag for all characters that must change places before sorting.

① See Also

"[Creating a New Linguistic Sort with the Oracle Locale Builder](#)" for more information about setting the `SWAP_WITH_NEXT` flag

5.5.10 Special Letters

Special letters is a term used in monolingual collation. They are called **expanding characters** in multilingual collation.

① See Also

"[Expanding Characters](#)"

5.5.11 Special Combination Letters

Special combination letters is the term used in monolingual collations. They are called **contracting letters** in multilingual collation.

① See Also

"[Contracting Characters](#)"

5.5.12 Special Uppercase Letters

One lowercase letter may map to multiple uppercase letters. For example, in traditional German, the uppercase letters for `ß` are `SS`.

These case conversions are handled by the `NLS_UPPER`, `NLS_LOWER`, and `NLS_INITCAP` SQL functions, according to the conventions established by the linguistic collations. The `UPPER`, `LOWER`, and `INITCAP` SQL functions cannot handle these special characters, because their casing operation is based on binary mapping defined for the underlying character set, which is not linguistic sensitive.

The `NLS_UPPER` SQL function returns its first argument string in which all lowercase letters have been mapped to their uppercase equivalents. The following example shows the result of the `NLS_UPPER` function when `NLS_SORT` is set to `XGERMAN`:

```
SELECT NLS_UPPER ('große') "Uppercase" FROM DUAL;
```

```
Upper
-----
GROSSE
```

See Also

Oracle AI Database SQL Language Reference

5.5.13 Special Lowercase Letters

Oracle Database supports special lowercase letters. One uppercase letter may map to multiple lowercase letters. An example is the Turkish uppercase I becoming a small, dotless i.

5.6 Case-Insensitive and Accent-Insensitive Linguistic Collation

An SQL operation in an Oracle Database is generally sensitive to the case and the accents (diacritics) of characters. However, sometimes you may need to perform case-insensitive or accent-insensitive comparison or matching.

In previous versions of the database, case-insensitive queries could be achieved by using the NLS_UPPER and NLS_LOWER SQL functions. The functions change the case of strings based on a specific linguistic collation definition. This enables you to perform case-insensitive searches regardless of the language being used. For example, create a table called test1 as follows:

```
SQL> CREATE TABLE test1(word VARCHAR2(12));
SQL> INSERT INTO test1 VALUES('GROSSE');
SQL> INSERT INTO test1 VALUES('Große');
SQL> INSERT INTO test1 VALUES('große');
SQL> SELECT * FROM test1;
```

```
WORD
-----
GROSSE
Große
große
```

Perform a case-sensitive search for GROSSE as follows:

```
SQL> SELECT word FROM test1 WHERE word='GROSSE';
```

```
WORD
-----
GROSSE
```

Perform a case-insensitive search for GROSSE using the NLS_UPPER function:

```
SELECT word FROM test1
WHERE NLS_UPPER(word, 'NLS_SORT = XGERMAN') = 'GROSSE';
```

```
WORD
-----
GROSSE
Große
große
```

Oracle Database provides case-insensitive and accent-insensitive options for collation. It provides the following types of linguistic collations:

- Linguistic collations that use information about base letters, diacritics, punctuation, and case. These are the standard linguistic collations that are described in "[Using Linguistic Collation](#)".
- Monolingual collations that use information about base letters, diacritics, and punctuation, but not case, and multilingual and UCA collations that use information about base letters and diacritics, but not case or punctuation. This type of sort is called **case-insensitive**.
- Monolingual collations that use information about base letters and punctuation only, and multilingual and UCA collations that use information about base letters only. This type of sort is called **accent-insensitive**. (**Accent** is another word for **diacritic**.) Like case-insensitive sorts, an accent-insensitive sort does not use information about case.

Accent- and case-insensitive multilingual collations ignore punctuation characters as described in "[Ignorable Characters](#)".

The rest of this section contains the following topics:

- [Examples: Case-Insensitive and Accent-Insensitive Collation](#)
- [Specifying a Case-Insensitive or Accent-Insensitive Collation](#)
- [Examples: Linguistic Collation](#)

① See Also

- "[NLS_SORT](#)"
- "[NLS_COMP](#)"

5.6.1 Examples: Case-Insensitive and Accent-Insensitive Collation

The following examples show:

- A collation that uses information about base letters, diacritics, punctuation, and case
- A case-insensitive collation
- An accent-insensitive collation

Example 5-4 Linguistic Collation Using Base Letters, Diacritics, Punctuation, and Case Information

The following list has been sorted using information about base letters, diacritics, punctuation, and case:

```
blackbird
black bird
black-bird
Blackbird
Black-bird
blackbird
bläckbird
```

Example 5-5 Case-Insensitive Linguistic Collation

The following list has been sorted using information about base letters, diacritics, and punctuation, ignoring case:

```
black bird
black-bird
```

```

Black-bird
blackbird
Blackbird
blackbird
blackbird

```

black-bird and Black-bird have the same value in the collation, because the only different between them is case. They could appear interchanged in the list. Blackbird and blackbird also have the same value in the collation and could appear interchanged in the list.

Example 5-6 Accent-Insensitive Linguistic Collation

The following list has been sorted using information about base letters only. No information about diacritics, punctuation, or case has been used.

```

blackbird
blackbird
blackbird
Blackbird
BlackBird
Black-bird
Black bird

```

5.6.2 Specifying a Case-Insensitive or Accent-Insensitive Collation

Use the NLS_SORT session parameter to specify a case-insensitive or accent-insensitive collation:

- Append `_CI` to an Oracle Database collation name for a case-insensitive collation.
- Append `_AI` to an Oracle Database collation name for an accent-insensitive and case-insensitive collation.

For example, you can set NLS_SORT to the following types of values:

```

UCA1210_SPANISH_AI
FRENCH_M_AI
XGERMAN_CI

```

Binary collation can also be case-insensitive or accent-insensitive. When you specify `BINARY_CI` as a value for NLS_SORT, it designates a collation that is accent-sensitive and case-insensitive. `BINARY_AI` designates an accent-insensitive and case-insensitive binary collation. You may want to use a binary collation if the binary collation order of the character set is appropriate for the character set you are using.

For example, with the NLS_LANG environment variable set to `AMERICAN_AMERICA.WE8ISO8859P1`, create a table called test2 and populate it as follows:

```

SQL> CREATE TABLE test2 (letter VARCHAR2(10));
SQL> INSERT INTO test2 VALUES('ä');
SQL> INSERT INTO test2 VALUES('a');
SQL> INSERT INTO test2 VALUES('A');
SQL> INSERT INTO test2 VALUES('Z');
SQL> SELECT * FROM test2;

```

```

LETTER
-----
ä
a
A
Z

```

The default value of NLS_SORT is BINARY. Use the following statement to do a binary collation of the characters in table test2:

```
SELECT * FROM test2 ORDER BY letter;
```

To change the value of NLS_SORT, enter a statement similar to the following:

```
ALTER SESSION SET NLS_SORT=BINARY_CI;
```

The following table shows the collation orders that result from setting NLS_SORT to BINARY, BINARY_CI, and BINARY_AI.

BINARY	BINARY_CI	BINARY_AI
A	a	ä
Z	A	a
a	Z	A
ä	ä	Z

When NLS_SORT=BINARY, uppercase letters come before lowercase letters. Letters with diacritics appear last.

When the collation considers diacritics but ignores case (BINARY_CI), the letters with diacritics appear last.

When both case and diacritics are ignored (BINARY_AI), ä is sorted with the other characters whose base letter is a. All the characters whose base letter is a occur before z.

You can use binary collation for better performance when the character set is US7ASCII or another character set that has the same collation order as the binary collation.

The following table shows the collation orders that result from German collation for the table.

GERMAN	GERMAN_CI	GERMAN_AI
a	a	ä
A	A	a
ä	ä	A
Z	Z	Z

A German collation places lowercase letters before uppercase letters, and ä occurs before Z. When the collation ignores both case and diacritics (GERMAN_AI), ä appears with the other characters whose base letter is a.

5.6.3 Examples: Linguistic Collation

The examples in this section demonstrate a binary collation, a monolingual collation, and a UCA collation. To prepare for the examples, create and populate a table called test3. Enter the following statements:

```
SQL> CREATE TABLE test3 (name VARCHAR2(20));
SQL> INSERT INTO test3 VALUES('Diet');
SQL> INSERT INTO test3 VALUES('Ä voir');
SQL> INSERT INTO test3 VALUES('Freizeit');
```


Example 5-7 Binary Collation

The ORDER BY clause uses a binary collation.

```
SQL> SELECT * FROM test3 ORDER BY name;
```

You should see the following output:

```
Diet
Freizeit
À voir
```

Note that a binary collation results in À voir being at the end of the list.

Example 5-8 Monolingual German Collation

Use the NLSSORT function with the NLS_SORT parameter set to german to obtain a German collation.

```
SQL> SELECT * FROM test3 ORDER BY NLSSORT(name, 'NLS_SORT=german');
```

You should see the following output:

```
À voir
Diet
Freizeit
```

Note that À voir is at the beginning of the list in a German collation.

Example 5-9 Comparing a Monolingual German Collation to a UCA Collation

Insert the character string shown in the following figure into test. It is a D with a crossbar followed by ñ.

Figure 5-2 Example Character String



Perform a monolingual German collation by using the NLSSORT function with the NLS_SORT parameter set to german.

```
SELECT * FROM test2 ORDER BY NLSSORT(name, 'NLS_SORT=german');
```

The output from the German collation shows the new character string last in the list of entries because the characters are not recognized in a German collation.

Perform a UCA collation by entering the following statement:

```
SELECT * FROM test2
ORDER BY NLSSORT(name, 'NLS_SORT=UCA1210_DUCET');
```

The output shows the new character string after Diet, following the UCA order.

① See Also

- ["The NLSSORT Function"](#)
- ["NLS_SORT"](#) for more information about setting and changing the NLS_SORT parameter

5.7 Performing Linguistic Comparisons

Starting with Oracle Database 12c Release 2 (12.2), a collation-sensitive operation determines the collation to use from the collations associated with its arguments.

A collation can be declared for a table column or a view column when the column is created. This associated collation is then passed along the column values to the operations processing the column. An operation applies a set of precedence rules to determine the collation to use based on the collations of its arguments. Similarly, an operation returning a character value derives collation for the return value from the collations of its arguments.

① See Also

["Column-Level Collation and Case Sensitivity"](#) for more information about the collation architecture in Oracle Database.

If a collation-sensitive operation determines that the collation it should apply is the pseudo-collation USING_NLS_COMP, then the NLS_COMP and NLS_SORT parameters are referenced to determine the actual named collation to use. In this case, the collation is determined in the same way as it is determined in Oracle Database 12c Release 1 (12.1) and earlier releases.

The NLS_COMP setting determines how NLS_SORT is handled by the SQL operations. There are three valid values for NLS_COMP:

- **BINARY**
Most SQL operations compare character values using binary collation, regardless of the value set in NLS_SORT. This is the default setting.
- **LINGUISTIC**
All SQL operations compare character values using collation specified in NLS_SORT. For example, NLS_COMP=LINGUISTIC and NLS_SORT=BINARY_CI means the collation-sensitive SQL operations will use binary comparison, but will ignore character case.
- **ANSI**
A limited set of SQL operations honors the NLS_SORT setting. ANSI is available for backward compatibility.

The following table shows how different SQL or PL/SQL operations behave with these different settings.

Table 5-3 Linguistic Comparison Behavior with NLS_COMP Settings

SQL or PL/SQL Operation:	BINARY	LINGUISTIC	ANSI
Set Operators:	-	-	-

Table 5-3 (Cont.) Linguistic Comparison Behavior with NLS_COMP Settings

SQL or PL/SQL Operation:	BINARY	LINGUISTIC	ANSI
UNION, INTERSECT, MINUS	Binary	Honors NLS_SORT	Binary
Scalar Functions:	-	-	-
DECODE	Binary	Honors NLS_SORT	Binary
INSTRx	Binary	Honors NLS_SORT	Binary
LEAST, GREATEST	Binary	Honors NLS_SORT	Binary
MAX, MIN	Binary	Honors NLS_SORT	Binary
NULLIF	Binary	Honors NLS_SORT	Binary
REPLACE	Binary	Honors NLS_SORT	Binary
TRIM, LTRIM, RTRIM	Binary	Honors NLS_SORT	Binary
TRANSLATE	Binary	Honors NLS_SORT	Binary
NLS_INITCAP	Honors NLS_SORT	Honors NLS_SORT	Honors NLS_SORT
NLS_LOWER, NLS_UPPER	Honors NLS_SORT	Honors NLS_SORT	Honors NLS_SORT
NLSSORT	Honors NLS_SORT	Honors NLS_SORT	Honors NLS_SORT
REGEXP_COUNT	Honors NLS_SORT	Honors NLS_SORT	Honors NLS_SORT
REGEXP_INSTR	Honors NLS_SORT	Honors NLS_SORT	Honors NLS_SORT
REGEXP_REPLACE	Honors NLS_SORT	Honors NLS_SORT	Honors NLS_SORT
REGEXP_SUBSTR	Honors NLS_SORT	Honors NLS_SORT	Honors NLS_SORT
Conditions:	-	-	-
=, !=, >, <, >=, <=	Binary	Honors NLS_SORT	Honors NLS_SORT
BETWEEN, NOT BETWEEN	Binary	Honors NLS_SORT	Honors NLS_SORT
IN, NOT IN	Binary	Honors NLS_SORT	Honors NLS_SORT
REGEXP_LIKE	Binary	Honors NLS_SORT	Honors NLS_SORT
LIKE	Binary	Honors NLS_SORT	Binary
CASE Expression:	-	-	-
CASE	Binary	Honors NLS_SORT	Binary
Analytic Function Clauses:	-	-	-
DISTINCT	Honors NLS_SORT	Honors NLS_SORT	Honors NLS_SORT
OVER(ORDER BY)	Honors NLS_SORT	Honors NLS_SORT	Honors NLS_SORT
OVER(PARTITION BY)	Honors NLS_SORT	Honors NLS_SORT	Honors NLS_SORT
Subquery Clauses:	-	-	-
DISTINCT, UNIQUE	Binary	Honors NLS_SORT	Binary
GROUP BY	Binary	Honors NLS_SORT	Binary
ORDER BY	Honors NLS_SORT	Honors NLS_SORT	Honors NLS_SORT

See Also

"[NLS_COMP](#)" and "[NLS_SORT](#)" for more information about these parameters.

5.7.1 Collation Keys

When the comparison conditions =, !=, >, <, >=, <=, BETWEEN, NOT BETWEEN, IN, NOT IN, the query clauses ORDER BY or GROUP BY, or the aggregate function COUNT(DISTINCT) are evaluated according to linguistic rules, the compared argument values are first transformed to binary values called collation keys and then compared byte by byte, like RAW values.

If a monolingual collation is applied, collation keys contain concatenated major values for characters of the source value followed by concatenated minor values for those characters. If a multilingual collation is applied, collation keys contain concatenated primary, then secondary, and then tertiary values. If a UCA collation is applied, collation keys contain concatenated primary, secondary, tertiary, and possibly quaternary values. The case-insensitive and accent-insensitive multilingual and UCA collations may omit quaternary, tertiary, and secondary values. If an emulated binary collation is applied, collation keys contain arguments to be compared converted to the base character set of the collation.

The collation keys are the same values that are returned by the NLSSORT function. That is, activating the linguistic behavior of these SQL operations is equivalent to including their arguments into calls to the NLSSORT function.

① See Also

["The NLSSORT Function"](#)

5.7.2 Restricted Precision of Linguistic Comparison

As collation keys are values of the data type RAW and the maximum length of a RAW value depends on the value of the initialization parameter, MAX_STRING_SIZE, the maximum length of a collation key is controlled by the parameter as well.

When MAX_STRING_SIZE is set to STANDARD, the maximum length of a collation key is restricted to 2000 bytes. If a full source string yields a collation key longer than the maximum length, the collation key generated for this string is calculated for a maximum prefix (initial substring) of the value for which the calculated result does not exceed 2000 bytes.

For monolingual collation, the prefix is typically 1000 characters. For multilingual collation, the prefix is typically 500 characters. For UCA collations, the prefix is typically 300 characters. The exact length of the prefix may be higher or lower and depends on the particular collation and the particular characters contained in the source string. The implication of this method of collation key generation is that SQL operations using the collation keys to implement the linguistic behavior will return results that may ignore trailing parts of long arguments. For example, two strings starting with the same 1000 characters but differing somewhere after the 1000th character will be grouped together by the GROUP BY clause.

When MAX_STRING_SIZE is set to EXTENDED, the maximum length of a collation key is restricted to 32767 bytes. With this setting, collation key generation is switched to precise mode. If a full source string yields a collation key longer than the maximum length, the database raises the ORA-12742 error message instead of generating a truncated key.

5.7.3 Avoiding ORA-12742 Error

In the precise mode, that is, when the initialization parameter `MAX_STRING_SIZE` is set to `EXTENDED`, generation of a collation key may fail with ORA-12742 error, if the buffer reserved for the collation key is too small. This can happen in any of the following two cases:

- The length of the generated key is longer than 32767 bytes
- The expansion ratio used to calculate the collation key length from the source string length is too low for a given combination of collation and source string

The first case may happen for long source strings in any linguistic collation because collation keys are mostly longer than the source strings for which they are created. To avoid ORA-12742 error in this case, make sure that lengths of the collated values are never longer than the following limits:

- 1560 bytes for a UCA collation
- 4094 bytes for a monolingual or multilingual collation
- 21844 bytes for the collation `BINARY_CI`
- 32767 bytes for an emulated binary collation

The second case may happen for strings of any length in all UCA0620 collations and in the collations `UCA0700_DUCET`, `UCA0700_ROOT`, `UCA1210_DUCET`, and `UCA1210_ROOT`. This case happens because the pessimistic expansion ratio for the listed UCA collations is very high. Using the pessimistic expansion ratio for calculation of the pessimistic collation key length would strongly reduce the maximum length of a linguistically indexable column. Therefore, a lower ratio is used for these collations, which works for all source strings except those containing one or more of the four specific rare compatibility characters - one Japanese, one Korean, and two Arabic. The presence of these specific characters in a string may cause the collation key generation for the string to fail with ORA-12742 error.

The UCA0700 collations other than `UCA0700_DUCET` and `UCA0700_ROOT` have been customized to never generate collation keys longer than the chosen expansion ratio. In particular, `UCA0700_ORADUCET` and `UCA0700_ORAR00T` collations are almost identical versions of the corresponding `UCA0700_DUCET` and `UCA0700_ROOT` collations, in which the collation weights for the four problematic characters have been shortened.

Similarly, the UCA1210 collations other than `UCA1210_DUCET` and `UCA1210_ROOT` have been customized to never generate collation keys longer than the chosen expansion ratio. In particular, `UCA1210_ORADUCET` and `UCA1210_ORAR00T` collations are almost identical versions of the corresponding `UCA1210_DUCET` and `UCA1210_ROOT` collations, in which the collation weights for the four problematic characters have been shortened.

Note

Oracle recommends that if you want to use UCA collations, then use only the `UCA1210` collations, except `UCA1210_DUCET` and `UCA1210_ROOT`.

When a character value for which a collation key cannot be generated for a certain collation is inserted into a column, any query comparing or sorting this character value using this collation fails with ORA-12742 error. In certain application configurations, this may cause a denial of service (DoS) attack vulnerability. It is therefore important to follow these guidelines:

- Collate only column values limited in length, not using the problematic UCA collations as described above or
- Dynamically verify that only safe values are inserted into a table or
- Assure that applications are designed in such a way that values entered by one user cannot break queries issued by another user

You can dynamically verify safety of values inserted into a column by creating a CHECK constraint on the column. For example, if you create a table as follows:

```
CREATE TABLE translation_string
(
    id NUMBER,
    string VARCHAR2(32767),
    CONSTRAINT check_string CHECK (VSIZE(NLSSORT(string COLLATE
UCA1210_DUCET)) != -1)
);
```

then any insert or update of a character value in the string column will trigger the collation key generation in the check constraint condition. Problematic values will cause the DML to fail with ORA-12742 error. However, once successfully inserted or updated, the value will never cause ORA-12742 error in a later query.

The check_string constraint in the above example performs a pessimistic check over all the collations. It may be over-pessimistic for many collations. If you know that one or two specific collations will be used with a column, you can modify the check constraint to force generation of collation keys only for those collations. However, in that case, you have to restrict the collations that can be used in your application.

5.7.4 Examples: Linguistic Comparison

The following examples illustrate behavior with different NLS_COMP settings.

Example 5-10 Binary Comparison Binary Collation

The following illustrates behavior with a binary setting:

```
SQL> ALTER SESSION SET NLS_COMP=BINARY;
SQL> ALTER SESSION SET NLS_SORT=BINARY;
SQL> SELECT ename FROM emp1;

ENAME
-----
Mc Calla
MCAfee
McCoye
Mccathye
McCafeé

5 rows selected

SQL> SELECT ename FROM emp1 WHERE ename LIKE 'McC%e';

ENAME
-----
McCoye

1 row selected
```

Example 5-11 Linguistic Comparison Binary Case-Insensitive Collation

The following illustrates behavior with a case-insensitive setting:

```
SQL> ALTER SESSION SET NLS_COMP=LINGUISTIC;
SQL> ALTER SESSION SET NLS_SORT=BINARY_CI;
SQL> SELECT ename FROM emp1 WHERE ename LIKE 'McC%e';
```

```
ENAME
-----
McCoye
Mccathye
```

2 rows selected

Example 5-12 Linguistic Comparison Binary Accent-Insensitive Collation

The following illustrates behavior with an accent-insensitive setting:

```
SQL> ALTER SESSION SET NLS_COMP=LINGUISTIC;
SQL> ALTER SESSION SET NLS_SORT=BINARY_AI;
SQL> SELECT ename FROM emp1 WHERE ename LIKE 'McC%e';
```

```
ENAME
-----
McCoye
Mccathye
McCafeé
```

3 rows selected

Example 5-13 Linguistic Comparisons Returning Fewer Rows

Some operations may return fewer rows after applying linguistic rules. For example, with a binary setting, McAfee and McAfee are different:

```
SQL> ALTER SESSION SET NLS_COMP=BINARY;
SQL> ALTER SESSION SET NLS_SORT=BINARY;
SQL> SELECT DISTINCT ename FROM emp2;
```

```
ENAME
-----
McAfee
McAfee
McCoy
```

3 rows selected

However, with a case-insensitive setting, McAfee and McAfee are the same:

```
SQL> ALTER SESSION SET NLS_COMP=LINGUISTIC;
SQL> ALTER SESSION SET NLS_SORT=BINARY_CI;
SQL> SELECT DISTINCT ename FROM emp2;
```

```
ENAME
-----
McAfee
McCoy
```

2 rows selected

In this example, either McAfee or McAfee could be returned from the DISTINCT operation. There is no guarantee exactly which one will be picked.

Example 5-14 Linguistic Comparisons Using XSPANISH

There are cases where characters are the same using binary comparison but different using linguistic comparison. For example, with a binary setting, the character C in Cindy, Chad, and Clara represents the same letter C:

```
SQL> ALTER SESSION SET NLS_COMP=BINARY;
SQL> ALTER SESSION SET NLS_SORT=BINARY;
SQL> SELECT ename FROM emp3 WHERE ename LIKE 'C%';
```

```
ENAME
-----
Cindy
Chad
Clara
```

3 rows selected

In a database session with the linguistic rule set to traditional Spanish, XSPANISH, ch is treated as one character. So the letter c in Chad is different than the letter C in Cindy and Clara:

```
SQL> ALTER SESSION SET NLS_COMP=LINGUISTIC;
SQL> ALTER SESSION SET NLS_SORT=XSPANISH;
SQL> SELECT ename FROM emp3 WHERE ename LIKE 'C%';
```

```
ENAME
-----
Cindy
Clara
```

2 rows selected

And the letter c in combination ch is different than the c standing by itself:

```
SQL> SELECT REPLACE ('character', 'c', 't') "Changes" FROM DUAL;
```

```
Changes
-----
charatter
```

Example 5-15 Linguistic Comparisons Using UCA1210_TSPANISH

The character ch behaves the same in the traditional Spanish ordering of the UCA collations as that in XSPANISH:

```
SQL> ALTER SESSION SET NLS_COMP = LINGUISTIC;
SQL> ALTER SESSION SET NLS_SORT = UCA1210_TSPANISH;
SQL> SELECT ename FROM emp3 WHERE ename LIKE 'C%';
```

```
ENAME
-----
Cindy
Clara
```

```
SQL> SELECT REPLACE ('character', 'c', 't') "Changes" FROM DUAL;
```

```
Changes
-----
charatter
```


Example 5-16 Emulated Binary Comparison Using BIN_IBM37

This example illustrates the difference in binary ordering between ASCII-based and EBCDIC-based platforms, using the emulated binary collation for the EBCDIC code page IBM37. In ASCII, digits are sorted before letters, and basic Latin (English) uppercase letters are sorted before lowercase letters. In EBCDIC, digits are sorted after basic Latin letters, and basic Latin uppercase letters are sorted after lowercase letters.

```
SQL> ALTER SESSION SET NLS_COMP = LINGUISTIC;
SQL> ALTER SESSION SET NLS_SORT = BINARY;
SQL> SELECT ename FROM emp4 WHERE ename > '0' ORDER BY ename;
```

```
ENAME
-----
Barbara
Cindy
Clara
Eva
de Vil
```

```
SQL> ALTER SESSION SET NLS_SORT = BIN_IBM37;
SQL> SELECT ename FROM emp4 WHERE ename > '0' ORDER BY ename;
```

no rows selected

```
SQL> SELECT ename FROM emp4 WHERE ename > 'a' ORDER BY ename;
ENAME
-----
de Vil
Barbara
Cindy
Clara
Eva
```

5.8 Using Linguistic Indexes

Linguistic collation is language-specific and requires more data processing than binary collation. Using a binary collation for ASCII is accurate and fast because the binary codes for ASCII characters reflect their linguistic order.

When data in multiple languages is stored in the database, you may want applications to collate the data returned from a `SELECT . . . ORDER BY` statement according to different collation sequences depending on the language. You can accomplish this without sacrificing performance by using linguistic indexes. Although a linguistic index for a column slows down inserts and updates, it greatly improves the performance of linguistic collation with the `ORDER BY` clause and the `WHERE` clause.

You can create a function-based index that uses languages other than English. The index does not change the linguistic collation order determined by `NLS_SORT`. The linguistic index simply improves the performance.

The following statement creates an index based on a German collation:

```
CREATE TABLE my_table(name VARCHAR(20) NOT NULL);
CREATE INDEX nls_index ON my_table (NLSSORT(name, 'NLS_SORT = German'));
```

The `NOT NULL` in the `CREATE TABLE` statement ensures that the index is used.

After the index has been created, enter a `SELECT` statement similar to the following example:

```
SELECT * FROM my_table WHERE name LIKE 'Hein%' ORDER BY name;
```

It returns the result much faster than the same SELECT statement without a linguistic index.

When a standard index is created on a column *column* with a named collation *collation* other than BINARY, the created index is implicitly a functional, linguistic index created on the expression:

```
NLSORT(column, 'NLS_SORT=collation')
```

① See Also

- "[Standard Indexes](#)" in the section "Effect of Data-Bound Collation on Other Database Objects" for more information about the effect of column-level collation on indexes
- *Oracle AI Database Concepts* for more information about function-based indexes
- *Oracle AI Database Administrator's Guide* for more information about managing function-based indexes

The rest of this section contains the following topics:

- [Supported SQL Operations and Functions for Linguistic Indexes](#)
- [Linguistic Indexes for Multiple Languages](#)
- [Requirements for Using Linguistic Indexes](#)

5.8.1 Supported SQL Operations and Functions for Linguistic Indexes

Linguistic index support is available for the following collation-sensitive SQL operations and SQL functions:

- Comparison conditions =, !=, >, <, >=, <=
- Range conditions BETWEEN | NOT BETWEEN
- IN | NOT IN
- ORDER BY
- GROUP BY
- LIKE (LIKE, LIKE2, LIKE4, LIKEC)
- DISTINCT
- UNIQUE
- UNION
- INTERSECT
- MINUS

The SQL functions in the following list cannot utilize linguistic index:

- INSTR (INSTR, INSTRB, INSTR2, INSTR4, INSTRC)
- MAX
- MIN
- REPLACE

- TRIM
- LTRIM
- RTRIM
- TRANSLATE

5.8.2 Linguistic Indexes for Multiple Languages

There are four ways to build linguistic indexes for data in multiple languages:

- Build a linguistic index for each language that the application supports. This approach offers simplicity but requires more disk space. For each index, the rows in the language other than the one on which the index is built are collated together at the end of the sequence. The following example builds linguistic indexes for French and German.

```
CREATE INDEX french_index ON employees (NLSSORT(employee_id, 'NLS_SORT=FRENCH'));  
CREATE INDEX german_index ON employees (NLSSORT(employee_id, 'NLS_SORT=GERMAN'));
```

Oracle Database chooses the index based on the NLS_SORT session parameter or the arguments of the NLSSORT function specified in the ORDER BY clause. For example, if the NLS_SORT session parameter is set to FRENCH, then Oracle Database uses french_index. When it is set to GERMAN, Oracle Database uses german_index.

- Build a single linguistic index for all languages. This requires a language column (LANG_COL in "[Example: Setting Up a French Linguistic Index](#)") to be used as a parameter of the NLSSORT function. The language column contains NLS_LANGUAGE values for the data in the column on which the index is built. The following example builds a single linguistic index for multiple languages. With this index, the rows with the same values for NLS_LANGUAGE are sorted together.

```
CREATE INDEX i ON t (LANG_COL, NLSSORT(col, 'NLS_SORT=' || LANG_COL));
```

Queries choose an index based on the argument of the NLSSORT function specified in the ORDER BY clause.

- Build a single linguistic index for all languages using one of the multilingual collations such as GENERIC_M or FRENCH_M. These indexes sort characters according to the rules defined in ISO 14651. For example:

```
CREATE INDEX i ON t (NLSSORT(col, 'NLS_SORT=GENERIC_M'));
```

See Also

"[Multilingual Collation](#)" for more information

- Build a single linguistic index for all languages using one of the UCA collations such as UCA1210_ORADUCET or UCA1210_CFRENCH. These indexes sort characters in the order conforming to ISO 14651 and UCA 12.1. For example:

```
CREATE INDEX i  
ON t (NLSSORT(col, 'NLS_SORT=UCA1210_ORADUCET'));
```

See Also

"[UCA Collation](#)" for more information

5.8.3 Requirements for Using Linguistic Indexes

The following are requirements for using linguistic indexes:

- [Set NLS_SORT Appropriately](#)
- [Specify NOT NULL in a WHERE Clause If the Column Was Not Declared NOT NULL](#)
- [Use a Tablespace with an Adequate Block Size](#)

This section also includes:

- [Example: Setting Up a French Linguistic Index](#)

5.8.3.1 Set NLS_SORT Appropriately

The NLS_SORT parameter should indicate the linguistic definition you want to use for the linguistic collation. If you want a French linguistic collation order, then NLS_SORT should be set to FRENCH. If you want a German linguistic collation order, then NLS_SORT should be set to GERMAN.

There are several ways to set NLS_SORT. You should set NLS_SORT as a client environment variable so that you can use the same SQL statements for all languages. Different linguistic indexes can be used when NLS_SORT is set in the client environment.

① See Also

["NLS_SORT"](#)

5.8.3.2 Specify NOT NULL in a WHERE Clause If the Column Was Not Declared NOT NULL

When you want to use the ORDER BY *column_name* clause with a column that has a linguistic index, include a WHERE clause like the following example:

```
WHERE NLSSORT(column_name) IS NOT NULL
```

This WHERE clause is not necessary if the column has already been defined as a NOT NULL column in the schema.

5.8.3.3 Use a Tablespace with an Adequate Block Size

A collation key created from a character value is usually a few times longer than this value. The actual length expansion depends on the particular collation in use and the content of the source value, with the UCA-based collations expanding the most.

When creating a linguistic index, Oracle Database first calculates the estimated maximum size of the index key by summing up the estimated maximum sizes of the collation keys (NLSSORT results) for each of the character columns forming the index key. In this calculation, the maximum size of a collation key for a character column with the maximum byte length *n* is estimated to be $n \times 21 + 5$ for UCA-based collations and $n \times 8 + 10$ for other collations.

The large expansion ratios can yield large maximum index key sizes, especially for composite (multicolumn) keys. At the same time, the maximum key size of an index cannot exceed

around 70% of the block size of the tablespace containing the index. If it does, an ORA-1450 error is reported. To avoid this error, you should store the linguistic index in a tablespace with an adequate block size, which may be larger than the default block size of your database. A suitable tablespace can be created with the CREATE TABLESPACE statement, provided the initialization parameter DB_nK_CACHE_SIZE corresponding to the required block size *n* has been set appropriately.

See Also

Oracle AI Database Administrator's Guide

5.8.3.4 Example: Setting Up a French Linguistic Index

The following example shows how to set up a French linguistic index. You may want to set NLS_SORT as a client environment variable instead of using the ALTER SESSION statement.

```
ALTER SESSION SET NLS_SORT='FRENCH';
CREATE INDEX test_idx ON test4(NLSSORT(name, 'NLS_SORT=FRENCH'));
SELECT * FROM test4 ORDER BY col;
ALTER SESSION SET NLS_COMP=LINGUISTIC;
SELECT * FROM test4 WHERE name > 'Henri';
```

Note

The SQL functions MAX() and MIN() cannot use linguistic indexes when NLS_COMP is set to LINGUISTIC.

5.9 Searching Linguistic Strings

Searching and collation are related tasks. Organizing data and processing it in a linguistically meaningful order is necessary for proper business processing. Searching and matching data in a linguistically meaningful way depends on what collation order is applied.

For example, searching for all strings greater than c and less than f produces different results depending on the value of NLS_SORT. In an ASCII binary collation, the search finds any strings that start with d or e but excludes entries that begin with upper case D or E or accented e with a diacritic, such as ê. Applying an accent-insensitive binary collation returns all strings that start with d, D, and accented e, such as Ê or ê. Applying the same search with NLS_SORT set to XSPANISH also returns strings that start with ch, because ch is treated as a composite character that collates between c and d in traditional Spanish. This chapter discusses the kinds of collation that Oracle Database offers and how they affect string searches by SQL and SQL regular expressions.

See Also

- ["Linguistic Collation Features"](#)
- ["SQL Regular Expressions in a Multilingual Environment"](#)

5.10 SQL Regular Expressions in a Multilingual Environment

Regular expressions provide a powerful method of identifying patterns of strings within a body of text. Usage ranges from a simple search for a string such as San Francisco to the more complex task of extracting all URLs to finding all words whose every second character is a vowel. SQL and PL/SQL support regular expressions in Oracle Database.

Traditional regular expression engines were designed to address only English text. However, regular expression implementations can encompass a wide variety of languages with characteristics that are very different from western European text. The implementation of regular expressions in Oracle Database is based on the Unicode Regular Expression Guidelines. The REGEXP SQL functions work with all character sets that are supported as database character sets and national character sets. Moreover, Oracle Database enhances the matching capabilities of the POSIX regular expression constructs to handle the unique linguistic requirements of matching multilingual data.

Oracle Database enhancements of the linguistic-sensitive operators are described in the following sections:

- [Character Range '\[x-y\]' in Regular Expressions](#)
- [Collation Element Delimiter '\[. .\]' in Regular Expressions](#)
- [Character Class '\[::\]' in Regular Expressions](#)
- [Equivalence Class '\[= =\]' in Regular Expressions](#)
- [Examples: Regular Expressions](#)

📘 See Also

- *Oracle AI Database Development Guide* for more information about regular expression syntax
- *Oracle AI Database SQL Language Reference* for more information about REGEXP SQL functions

5.10.1 Character Range '[x-y]' in Regular Expressions

According to the POSIX standard, a range in a regular expression includes all collation elements between the start point and the end point of the range in the linguistic definition of the current locale. Therefore, ranges in regular expressions are meant to be linguistic ranges, not byte value ranges, because byte value ranges depend on the platform, and the end user should not be expected to know the ordering of the byte values of the characters. The semantics of the range expression must be independent of the character set. This implies that a range such as [a-d] may include all the letters between a and d plus all of those letters with diacritics, plus any special case collation element such as ch in Traditional Spanish that is sorted as one character.

Oracle Database interprets range expressions as specified by the NLS_SORT parameter to determine the collation elements covered by a given range. For example:

```
Expression:      [a-d]e
NLS_SORT:        BINARY
Does not match:  cheremoya
```

```
NLS_SORT:      XSPANISH
Matches:      >>che<<remoya
```

5.10.2 Collation Element Delimiter '['. ']' in Regular Expressions

This construct is introduced by the POSIX standard to separate collating elements. A **collating element** is a unit of collation and is equal to one character in most cases. However, the collation sequence in some languages may define two or more characters as a collating element. The historical regular expression syntax does not allow the user to define ranges involving multicharacter collation elements. For example, there was no way to define a range from a to ch because ch was interpreted as two separate characters.

By using the collating element delimiter '['. ']', you can separate a multicharacter collation element from other elements. For example, the range from a to ch can be written as [a-[.ch.]]. It can also be used to separate single-character collating elements. If you use '['. ']' to enclose a multicharacter sequence that is not a defined collating element, then it is considered as a semantic error in the regular expression. For example, [ab.] is considered invalid if ab is not a defined multicharacter collating element.

5.10.3 Character Class '[': ']' in Regular Expressions

In English regular expressions, the range expression can be used to indicate a character class. For example, [a-z] can be used to indicate any lowercase letter. However, in non-English regular expressions, this approach is not accurate unless a is the first lowercase letter and z is the last lowercase letter in the collation sequence of the language.

The POSIX standard introduces a new syntactical element to enable specifying explicit character classes in a portable way. The '[': ']' syntax denotes the set of characters belonging to a certain character class. The character class definition is based on the character set classification data.

5.10.4 Equivalence Class '['= ']' in Regular Expressions

Oracle Database also supports equivalence classes through the '['= ']' syntax as recommended by the POSIX standard. A base letter and all of the accented versions of the base constitute an **equivalence class**. For example, the equivalence class [=a=] matches ä as well as â. The current implementation does not support matching of Unicode composed and decomposed forms for performance reasons. For example, ä (a umlaut) does not match 'a followed by umlaut'.

5.10.5 Examples: Regular Expressions

The following examples show regular expression matches.

Example 5-17 Case-Insensitive Match Using the NLS_SORT Value

Case sensitivity in an Oracle Database regular expression match is determined at two levels: the NLS_SORT initialization parameter and the run-time match option. The REGEXP functions inherit the case-sensitive behavior from the value of NLS_SORT by default. The value can also be explicitly overridden by the run-time match option 'c' (case-sensitive) or 'i' (case-insensitive).

```
Expression: catalog(ue)?
NLS_SORT:  GENERIC_M_CI
Matches:
```

```
>>Catalog<<
>>catalogue<<
>>CATALOG<<
```

Oracle Database SQL syntax:

```
SQL> ALTER SESSION SET NLS_SORT='GENERIC_M_CI';
SQL> SELECT col FROM test WHERE REGEXP_LIKE(col, 'catalog(ue)?');
```

Example 5-18 Case Insensitivity Overridden by the Run-time Match Option

Expression: catalog(ue)?
NLS_SORT: GENERIC_M_CI
Match option: 'c'
Matches:

```
>>catalogue<<
```

Does not match:

```
Catalog
CATALOG
```

Oracle Database SQL syntax:

```
SQL> ALTER SESSION SET NLS_SORT='GENERIC_M_CI';
SQL> SELECT col FROM test WHERE REGEXP_LIKE(col, 'catalog(ue)?', 'c');
```

Example 5-19 Matching with the Collation Element Operator [..]

Expression: [^a-[.ch.]]+ /*with NLS_SORT set to xspanish*/
Matches:

```
>>driver<<
```

Does not match:

```
cab
```

Oracle Database SQL syntax:

```
SQL> SELECT col FROM test WHERE REGEXP_LIKE(col, '[^a-[.ch.]]+');
```

Example 5-20 Matching with the Character Class Operator [::]

This expression looks for 6-character strings with lowercase characters. Note that accented characters are matched as lowercase characters.

Expression: [[:lower:]]{6}
Database character set: WE8ISO8859P1
Matches:

```
>>maitre<<
>>mòbile<<
>>pájaro<<
>>zurück<<
```

Oracle Database SQL syntax:

```
SQL> SELECT col FROM test WHERE REGEXP_LIKE(col, '[[:lower:]]{6}');
```


Example 5-21 Matching with the Base Letter Operator [=]

Expression: `r[[]=]sum[[]=]`
Matches:

```
>>resume<<
>>r      <<
>>r      <<
>>res    <<
```

Oracle Database SQL syntax:

```
SQL> SELECT col FROM test WHERE REGEXP_LIKE(col, 'r[[]=]sum[[]=]');
```

① See Also

- *Oracle AI Database Development Guide* for more information about regular expression syntax
- *Oracle AI Database SQL Language Reference* for more information about REGEX SQL functions

5.11 Column-Level Collation and Case Sensitivity

The column-level collation feature specifies a collation for a character column in its definition. This feature applies linguistic processing only where needed and achieves consistent handling of particular column data in all SQL statements. Oracle supports case-insensitive and accent-insensitive collations. By assigning such collation to a column, you can easily force all comparisons of column values to be case-insensitive or accent-insensitive or both.

The collations declared at a column-level are part of the more general data-bound collation architecture, where collation becomes an attribute of data, analogous to the data type. The declared collation is passed along the column to SQL operations and is used together with collations of other operation arguments to determine the collation to use by the operation.

The column-level collation feature is based on the ISO SQL standard and it simplifies application migration to Oracle Database from other database systems that support this feature. This feature is backward-compatible with the mechanism of controlling linguistic behavior for SQL and PL/SQL operations using the session parameters `NLS_COMP` and `NLS_SORT`.

This section contains the following topics:

- [About Data-Bound Collation](#)
- [Default Collations](#)
- [Enabling Data-Bound Collation](#)
- [Specifying a Data-Bound Collation](#)
- [Viewing the Data-Bound Collation of a Database Object](#)
- [Case-Insensitive Database](#)
- [Effect of Data-Bound Collation on Other Database Objects](#)
- [Effect of Data-Bound Collation on Distributed Queries and DML Operations](#)
- [Effect of Data-Bound Collation on PL/SQL Types and User-Defined Types](#)

- [Effect of Data-Bound Collation on Oracle XML DB](#)

5.11.1 About Data-Bound Collation

In Oracle Database 12c Release 1 (12.1) and earlier releases, the two session parameters `NLS_SORT` and `NLS_COMP` determine the rules by which character type data is compared and matched. The collation specified using these two session parameters is called the *session collation*. The value of `NLS_COMP` decides which operations are controlled by the collation specified in the value of `NLS_SORT` and which operations use the `BINARY` collation. All collation-sensitive operations selected by the value of `NLS_COMP` in all SQL and PL/SQL statements executed in the session use the same collation.

Starting with Oracle Database 12c Release 2 (12.2), a new mechanism has been added to apply collations for SQL operations in a much more granular way. A collation is an attribute of data, similar to the data type. A collation can be declared for a character data container, such as table column, and is passed to all SQL operations that operate on that column. Each collation-sensitive operation combines declared collations of its arguments to determine the collation to use for the operation processing. Furthermore, an operation that returns a character value combines collations of its arguments to derive a collation for the result. The operator `COLLATE` allows overriding a collation in any place in an expression.

This type of collation, which is associated with a particular data, is called the *data-bound collation*. A data-bound collation can be applied only to the values of character data types — `VARCHAR2`, `CHAR`, `LONG`, `NVARCHAR2`, `NCHAR`, `CLOB`, and `NCLOB`.

① Note

The data-bound collation of a table column is also used for the following operations that always used binary collation earlier to the Oracle Database 12c Release 2 (12.2):

- Index key ordering for standard (that is, non-functional) indexes on the column, including indexes of primary keys, unique constraints, and bitmap indexes.
- Range, list, and reference partitioning on a column.
- Enforcement of a foreign key constraint on a column that points to a primary key or unique key column in another table.

There are two types of data-bound collations:

- **Named Collation:** This collation is a particular set of collating rules specified by a collation name. Named collations are the same collations that are specified as values for the `NLS_SORT` parameter. A named collation can be either a binary collation or a linguistic collation.
 - Examples of binary named collation are: `BINARY`, `BINARY_CI` (case-insensitive binary collation), and `BINARY_AI` (accent-insensitive and case-insensitive binary collation).
 - Examples of linguistic named collation are: `GENERIC_M`, `GENERIC_M_AI`, `FRENCH`, `POLISH`, `UCA1210_CFRENCH`, and so on.
- **Pseudo-collation:** This collation does not directly specify the collating rules for a character data type. Instead, it instructs collation-sensitive operations to check the values of the `NLS_SORT` and `NLS_COMP` session parameters for the actual named collation to use. Pseudo-collations are the bridge between the new declarative method of specifying collations and the old method that uses session parameters.

The following are the supported pseudo-collations:

- USING_NLS_COMP: Operations that use the USING_NLS_COMP pseudo-collation behave the same as in Oracle Database 12c (12.1) and earlier releases, that is, they use the session collation. The particular named collation applied by the SQL or PL/SQL operation is either BINARY or determined by the value of NLS_SORT, NLS_COMP, and the operation itself.
- USING_NLS_SORT, USING_NLS_SORT_CI, USING_NLS_SORT_AI, and USING_NLS_SORT_CS: If one of these collations is determined as the collation to use for an operation, the operation applies the collation named by the value of NLS_SORT parameter without considering the value of NLS_COMP parameter. Additionally:
 - * If the pseudo-collation is USING_NLS_SORT_CI and the value of NLS_SORT does not end in _CI or _AI, then the name of collation to apply is constructed by appending _CI to the value of NLS_SORT.
 - * If the pseudo-collation is USING_NLS_SORT_AI and the value of NLS_SORT does not end in _CI or _AI, then the name of collation to apply is constructed by appending _AI to the value of NLS_SORT. If the value of NLS_SORT ends in _CI, then the suffix _CI is changed to _AI.
 - * If the pseudo-collation is USING_NLS_SORT_CS and the value of NLS_SORT ends in _CI or _AI, then the name of collation to apply is constructed by stripping this suffix from the NLS_SORT value.
 - * Otherwise, the name of collation to apply is the value of NLS_SORT.

Note

- Suffix _CI stands for case insensitivity. Suffix _AI stands for case and accent insensitivity. Suffix _CS stands for case and accent sensitivity.
- The pseudo-collation USING_NLS_SORT_CI forces the use of the case-insensitive version of the collation specified in the NLS_SORT parameter value.
- The pseudo-collation USING_NLS_SORT_AI forces the use of the case-insensitive and accent-insensitive version of the collation specified in the NLS_SORT parameter value.
- The pseudo-collation USING_NLS_SORT_CS forces the use of the case-sensitive and accent-sensitive version of the collation specified in the NLS_SORT parameter value.
- The only collation supported by CLOB and NCLOB columns is the pseudo-collation USING_NLS_COMP.

5.11.2 Default Collations

Starting with Oracle Database 12c Release 2 (12.2), each table column with a character data type has a declared data-bound collation. If collation for a column is not specified explicitly in the DDL statement that creates the column (in the CREATE TABLE or ALTER TABLE ADD statement), then the containing table's default collation is used for the column. If the DDL statement creating a table does not specify a default collation, then the default collation of the schema owning the table is used as the default collation for the table. Specify default collation for a schema in the CREATE USER statement that creates the owner of the schema. If the CREATE USER statement does not specify the default collation for a schema, then the collation USING_NLS_COMP is used.

Collations are inherited only when database objects are created. For example, changing the table default collation does not change the collations of existing character columns of a table. Only new columns added to the table after the change inherit the new default collation. Similarly, changing the schema default collation does not change the default collations of tables in a schema. Only new tables created in the schema after the change inherit the new default collation.

The session parameter `DEFAULT_COLLATION` overrides the schema default collation as described in the section "[Effective Schema Default Collation](#)".

Note

After upgrading to Oracle Database 12c Release 2 (12.2) or later, all the columns, tables, and schemas in the upgraded database have the `USING_NLS_COMP` collation. This ensures that all the collation-sensitive operations in the database behave the same as before the upgrade, that is, all the operations use *session collation*.

5.11.3 Enabling Data-Bound Collation

To enable the data-bound collation feature, set the following database initialization parameter values:

- `MAX_STRING_SIZE=EXTENDED`
- `COMPATIBLE>=12.2`

Note

- If the data-bound collation feature is not enabled, collations cannot be specified for database objects and value for the `DEFAULT_COLLATION` session parameter cannot be set.
- Until the data-bound collation feature is enabled, all user-defined database objects have the data-bound collation `USING_NLS_COMP`. However, Oracle-supplied database objects are not guaranteed to use only this collation.
- Even if the data-bound collation feature is not enabled, the `COLLATE` operator and the `COLLATION()`, `NLS_COLLATION_ID()`, and `NLS_COLLATION_NAME()` functions can be used in SQL statements.
- Once the data-bound collation feature is enabled, it cannot be disabled, that is, you cannot set the value for the `MAX_STRING_SIZE` parameter back to `STANDARD` and the value for the `COMPATIBLE` parameter back to the earlier Oracle Database release.
- The data-bound collation feature cannot be used in a multitenant container database root (CDB root), because, for a CDB root, the actual value of the `MAX_STRING_SIZE` initialization parameter is ignored and its value is always assumed to be `STANDARD`. However, if the `MAX_STRING_SIZE` parameter value is not specified for a PDB, then the PDB uses the `MAX_STRING_SIZE` parameter value specified for the CDB root.

5.11.4 Specifying a Data-Bound Collation

A data-bound collation can be specified for:

- Table columns
- Cluster columns
- Tables
- Schemas through the owning user
- Views and materialized views
- PL/SQL units, such as procedures, functions, packages, types, and triggers
- SQL expressions

Note

- A collation cannot be specified for a cluster, but it can be specified for key columns in a cluster.
- A collation cannot be specified for a whole database.

5.11.4.1 Effective Schema Default Collation

The *effective schema default collation* is a default collation assigned to a database object created in a particular schema using a DDL statement in a particular user session, when a default collation for the object is not explicitly declared in the DDL statement. The *effective schema default collation* is a combination of the corresponding schema default collation and the value of the `DEFAULT_COLLATION` parameter for the session.

If a value is specified for the `DEFAULT_COLLATION` parameter in a session, then the *effective schema default collation* for that session for a schema is the value of the `DEFAULT_COLLATION` parameter. If a value is *not* specified for the `DEFAULT_COLLATION` parameter in a session, then the *effective schema default collation* for that session is the value of the corresponding schema default collation.

You can specify a value for the parameter `DEFAULT_COLLATION` with the `ALTER SESSION` statement:

```
SQL> ALTER SESSION SET DEFAULT_COLLATION=collation_name;
```

Both named collations and pseudo-collations can be specified as the value for *collation_name*.

You can remove the collation assigned to the `DEFAULT_COLLATION` parameter by assigning it the value `NONE`:

```
SQL> ALTER SESSION SET DEFAULT_COLLATION=NONE;
```

The current value of the `DEFAULT_COLLATION` parameter can be checked in a session by using the statement:

```
SQL> SELECT SYS_CONTEXT('USERENV', 'SESSION_DEFAULT_COLLATION') FROM DUAL;
```

Note

- Oracle recommends that you specify a default collation for a database object during its creation using a DDL statement, when you want the object's default collation to be independent of the default collation of the enclosing schema. You should use the parameter `DEFAULT_COLLATION` only when dealing with legacy scripts that do not specify the collation explicitly.
- A session default collation specified by the `DEFAULT_COLLATION` parameter does not get propagated to any remote sessions connected to the current session using DB links.

5.11.4.2 Specifying Data-Bound Collation for a Schema

You can specify a default data-bound collation for a schema using the `DEFAULT COLLATION` clause in the `CREATE USER` and `ALTER USER` statements. The schema default collation determines the *effective schema default collation* that is assigned as the default collation for all the tables, views, materialized views, PL/SQL units, and user-defined types (UDTs) created in that schema, if these database objects do not have explicitly declared default collations.

If a schema default collation is not specified explicitly in the `CREATE USER` statement, then it is set to `USING_NLS_COMP` collation. You can change the schema default collation with the `ALTER USER` statement. The change does not affect the existing database objects and affects only the database objects that are subsequently created, replaced, or compiled in the schema.

Note

- If the `DEFAULT_COLLATION` parameter is specified for a session, then it overrides the default collation of a schema referenced in that session.
- If a schema has a default collation declaration other than `USING_NLS_COMP`, then PL/SQL units, including user-defined types, can be created in that schema, only if the session parameter `DEFAULT_COLLATION` is set to `USING_NLS_COMP` or the PL/SQL unit creation DDL contains the `DEFAULT COLLATION USING_NLS_COMP` clause.
- A schema default collation cannot be changed for an Oracle-supplied database user.

Example: Applying a default collation to a schema

```
CREATE USER hrsys  
  IDENTIFIED BY password  
  DEFAULT TABLESPACE hr_ts_1  
  DEFAULT COLLATION BINARY  
  ACCOUNT LOCK
```

```
-- the clauses after password can be in any order
/
```

This statement creates a new database user `hrsys` with its schema. The default collation of the schema is set to `BINARY`. All database objects created in the schema that do not contain the `DEFAULT COLLATION` clause have their default collation set to `BINARY`, unless the session parameter `DEFAULT_COLLATION` overrides it.

Example: Changing the default collation of a schema

```
ALTER USER hrsys DEFAULT COLLATION USING_NLS_COMP
/
```

This statement changes the default collation of the `hrsys` schema to the pseudo-collation `USING_NLS_COMP`. After this statement is executed, all the database objects created in the schema that do not contain the `DEFAULT COLLATION` clause have their default collation set to `USING_NLS_COMP`, unless the session parameter `DEFAULT_COLLATION` overrides it. The default collations of the existing database objects are not affected.

You can change the default collation for a schema at any time.

See Also

- ["Effective Schema Default Collation"](#)
- *Oracle AI Database SQL Language Reference* for the syntax of declaring schema default collation in `CREATE USER` and `ALTER USER` statements

5.11.4.3 Specifying Data-Bound Collation for a Table

You can specify a default data-bound collation for a table using the `DEFAULT COLLATION` clause in the `CREATE TABLE` and `ALTER TABLE` statements. The table default collation is assigned to a character column of the table, when an explicit collation is not declared for that column. If a default collation is not explicitly declared for a table in the `CREATE TABLE` statement, then the table collation is set to *effective schema default collation*.

You can change the default collation of a table using the `ALTER TABLE` statement. The change does not affect the existing table columns and affects only those columns that are subsequently added to the table or are updated using the `ALTER TABLE` statement.

Example: Applying a default collation to a table while creating a table

```
CREATE TABLE employees
(
  emp_code   VARCHAR2(10) PRIMARY KEY,
  first_name VARCHAR2(100),
  last_name  VARCHAR2(200),
  job_code   VARCHAR2(5) COLLATE BINARY,
  dep_code   NUMBER
)
DEFAULT COLLATION BINARY_CI
-- other CREATE TABLE clauses
/
```

The columns `emp_code`, `first_name`, and `last_name` inherit the table default collation `BINARY_CI`. The column `job_code` is declared explicitly with the collation `BINARY`. The primary key constraint declared on the column `emp_code` will not allow rows having the `emp_code` values of `abcde123` and `ABCDE123` in the table simultaneously.

Example: Changing the default collation of a table

```
ALTER TABLE employees DEFAULT COLLATION USING_NLS_COMP
/
```

This statement changes the default collation of the table `employees` to the pseudo-collation `USING_NLS_COMP`. Any new `VARCHAR2`, `CHAR`, `NVARCHAR2`, `NCHAR`, and `LONG` columns added to the table after the `ALTER TABLE` statement is executed, inherits the new collation, unless these columns are declared with an explicit collation or belong to a foreign key. The collations of the existing columns are not affected.

The default collation of a table can be changed at any time.

See Also

- ["Effective Schema Default Collation"](#)
- *Oracle AI Database SQL Language Reference* for the syntax of declaring table default collation in the `CREATE TABLE` and `ALTER TABLE` statements

5.11.4.4 Specifying Data-Bound Collation for a View and a Materialized View

You can specify a default data-bound collation for a view and a materialized view by using the `DEFAULT COLLATION` clause in the `CREATE VIEW` and `CREATE MATERIALIZED VIEW` statements respectively.

The default collation of a view or a materialized view is used as the derived collation of all the character literals included in the defining query of that view or materialized view. The default collation of a view or a materialized view can only be changed by recreating that view or materialized view.

Note

- If a default collation is not specified for a view or a materialized view, then it is set to *effective schema default collation*.
- A default collation for a view or a materialized view is not used by the view columns. The collations of the view columns are derived from the view's defining subquery. The `CREATE VIEW` or `CREATE MATERIALIZED VIEW` statement fails with an error or is created invalid, if any of the character columns of that view or materialized view is based on an expression in the defining subquery that has no derived collation.
- The `CREATE VIEW` or `CREATE MATERIALIZED VIEW` statement fails with an error, if its default collation is other than `USING_NLS_COMP`, and the defining query uses a `WITH plsql_declarations` clause.

Example: Applying a collation to a view

```
CREATE VIEW employees_j_polish_sort
( emp_code, first_name, last_name, job_code, dep_code )
DEFAULT COLLATION BINARY
AS
SELECT * FROM employees
WHERE last_name LIKE 'j%'
ORDER BY last_name COLLATE POLISH
/
```

Assuming the definition of the table `employees` is as in the `CREATE TABLE` example above, the view `employees_j_polish_sort` selects all employees with the last name starting with lowercase or uppercase 'j' and sorts them using the named collation `POLISH`. This collation properly orders accented letters for the Polish language. For example, it orders 'ó' between 'o' and 'p'. The `BINARY` and `BINARY_CI` collations order it after 'z'. Without the operator `COLLATE`, the `ORDER BY` clause would order the query result based on the collation of the column `last_name`, which is `BINARY_CI`.

The default collation of the view, which is `BINARY` collation, is used only to derive the collation of the character literal 'j%'. However, collation of a literal has lower priority than collation of a column. The collation of the column `last_name`, which is `BINARY_CI`, takes precedence and is used by the operator `LIKE`.

See Also

["Effective Schema Default Collation"](#)

5.11.4.5 Specifying Data-Bound Collation for a Column

A data-bound collation can be explicitly specified for columns of character data types `VARCHAR2`, `CHAR`, `LONG`, `CLOB`, `NVARCHAR2`, `NCHAR`, and `NCLOB` using:

- The `COLLATE` clause of a standard or a virtual column definition in a `CREATE TABLE` or `ALTER TABLE` statement.
 - If the column collation is not specified explicitly with the `COLLATE` clause for a column, then the default collation of the table is used for that column, except for the cases documented below.
 - If a column has the data type of `CLOB` or `NCLOB`, then its specified collation must be `USING_NLS_COMP`. The default collation of `CLOB` and `NCLOB` columns is always `USING_NLS_COMP` and it does not depend on the table default collation.
 - There are no operators allowed on `LONG` data type values in SQL, other than conversion to `CLOB` data type. Therefore, collation of `LONG` columns is not used in SQL statements. However, the `LONG` data type is identical to `VARCHAR2(32767)` in PL/SQL, and hence needs collation specification in PL/SQL. Therefore, collation specification for `LONG` columns is supported by Oracle, so that it can be passed to PL/SQL units through `%TYPE` and `%ROWTYPE` attributes.

Note

Only the USING_NLS_COMP collation is supported for columns referenced using the %TYPE and %ROWTYPE attributes in PL/SQL units.

- If neither the collation nor the data type is specified explicitly for a virtual column, or a column is created by a CREATE TABLE AS SELECT statement, then the collation is derived from the defining expression of the column, except when the column belongs to a foreign key. If the defining expression of a column has no derived collation, an error is reported.
- If a column belongs to a foreign key, its explicit collation specification must specify the same collation that is declared for the corresponding column of the referenced primary key or unique constraint. If a column belongs to a foreign key and has no explicit collation specification, its collation is assigned from the corresponding column of the referenced primary key or unique constraint. If a column belongs to two or more foreign key constraints referencing primary key or unique constraints with different collation specifications, an error is reported.

Example: Adding a column with collation declaration

```
ALTER TABLE employees ADD gender VARCHAR2(1) COLLATE BINARY_CI  
/
```

This statement adds a new column named gender to the table employees and requests it to be collated using the collation BINARY_CI. Without the COLLATE clause, the column employees.gender would inherit the default collation of the table.

Example: Changing the collation of a column

```
ALTER TABLE employees MODIFY job_code COLLATE BINARY_CI  
/
```

This statement changes the collation of the column employees.job_code to BINARY_CI. The statement would fail, if the column were included in an index key, partitioning key, foreign key, or a virtual column expression.

Note

The COLLATE clause can be applied to a column during its modification only when:

- the column to be modified is of a character data type and is not going to be changed to a non-character data type
- the column to be modified is *not* of a character data type and is going to be changed to a character data type, and the column is one of the following:
 - * a primary key column
 - * a unique key column
 - * a partition key column
 - * a column having a standard index applied to it

- The COLLATE clause of a key column definition in a CREATE CLUSTER statement.

- If the column collation is not specified explicitly with the `COLLATE` clause for a cluster column, then the *effective schema default collation* for the `CREATE CLUSTER` statement is used for that column.
- The collations of cluster key columns must match the collations of the corresponding columns in the tables created in the cluster.

Example: Applying a collation to a column in a cluster

```
CREATE CLUSTER clu1
(
  id          VARCHAR2(10) COLLATE BINARY_CI,
  category    VARCHAR2(20)
)
SIZE 8192 HASHKEYS 1000000
-- other CREATE CLUSTER clauses
/
```

The collation for the column `category` is inherited from the effective schema default collation at the time of `CREATE CLUSTER` execution. Unless the schema containing the cluster `clu1` is defined with a different explicit collation or a different collation is set in the `DEFAULT_COLLATION` session parameter, this effective schema default collation is the pseudo-collation `USING_NLS_COMP`.

A `CREATE TABLE` statement defining a table to be added to the hash cluster `clu1` must specify two of the table's columns in the `CLUSTER` clause. The first column must be of data type `VARCHAR2(10)` and must be declared with the collation `BINARY_CI`, and the second column must be of data type `VARCHAR2(20)` and must be declared with the collation inherited by the cluster column `clu1.category` from the effective schema default collation. The two collations are not used by the hash cluster itself.

Note

- Declared collations of columns involved in creation of various database objects, such as indexes, constraints, clusters, partitions, materialized views, and zone maps undergo certain restrictions that are further described in the section "[Effect of Data-Bound Collation on Other Database Objects](#)".
- The declared collation of a column can be modified with the `ALTER TABLE MODIFY` statement, except for the cases described in the section "[Effect of Data-Bound Collation on Other Database Objects](#)".

5.11.4.6 Specifying Data-Bound Collation for PL/SQL Units

A data-bound collation can be specified for the following PL/SQL units using the `DEFAULT COLLATION` clause in their `CREATE [OR REPLACE]` statement:

- Procedures
- Functions
- Packages
- Types
- Triggers

Varray and nested table types do not have explicitly declared default collations, as they do not have PL/SQL methods or multiple attributes to apply the default collation. Package and type bodies do not have their own collations, and they use the default collations of their specifications.

Starting with Oracle Database 12c Release 2 (12.2), the `CREATE [OR REPLACE] PROCEDURE | FUNCTION | PACKAGE | TYPE | TRIGGER` statement succeeds, only if the *effective schema default collation* is the pseudo-collation `USING_NLS_COMP` or the `DEFAULT COLLATION USING_NLS_COMP` clause in the `CREATE` statement overrides the *effective schema default collation*. This restriction includes varrays and nested tables with scalar elements of character data types.

If an `ALTER COMPILE` statement is issued with the `REUSE SETTINGS` clause, the stored default collation of the database object being compiled is not changed. The compilation of a database object fails, if the object does not satisfy the requirements described in the section [“Effect of Data-Bound Collation on PL/SQL Types and User-Defined Types”](#). For example, the compilation of a database object fails when the stored default collation is not `USING_NLS_COMP` or the `%TYPE` attribute is applied to a column with a named collation in the PL/SQL code.

If an `ALTER COMPILE` statement is issued without the `REUSE SETTINGS` clause, the stored default collation of the database object being compiled is compared with the *effective schema default collation* for the object owner at the time of the execution of the statement. If they are not equal and the PL/SQL unit does not contain the `DEFAULT COLLATION` clause, then an error is reported and the statement fails without compiling the object. If they are equal, then the compilation proceeds. The compilation fails, if the object does not satisfy the requirements described in the section [“Effect of Data-Bound Collation on PL/SQL Types and User-Defined Types”](#).

Starting with Oracle Database 12c Release 2 (12.2), all character data containers in procedures, functions, and methods, such as variables, parameters, and return values, behave as if their data-bound collation is the pseudo-collation `USING_NLS_COMP`. Also, all character attributes behave as if their data-bound collation is the pseudo-collation `USING_NLS_COMP` and all the relational table columns storing object attributes are assigned the pseudo-collation `USING_NLS_COMP`.

Note

If a default collation is not specified for a PL/SQL unit, then it is set to the *effective schema default collation*.

See Also

- [“Effective Schema Default Collation”](#)
- [“Effect of Data-Bound Collation on PL/SQL Types and User-Defined Types”](#)

5.11.4.7 Specifying Data-Bound Collation for SQL Expressions

During an SQL expression evaluation, each character argument to an operator and each character result of an operator has an associated data-bound collation. The collations of an operator's arguments determine the collation used by the operator, if the operator is collation-sensitive. The derived collation of an SQL expression result is relevant for a consumer of the result, which may be another SQL operator in the expression tree or a top-level consumer, such as an SQL statement clause in a `SELECT` statement. You can override the derived

collation of an expression node, such as a simple expression or an operator result, by using the COLLATE operator. The *collation derivation* and *collation determination* rules are used while evaluating an SQL expression.

This section contains the following topics:

- [Collation Derivation](#)
- [Collation Determination](#)
- [Expression Evaluation and the COLLATE Operator](#)
- [COLLATION Function](#)
- [NLS_COLLATION_ID and NLS_COLLATION_NAME Functions](#)

5.11.4.7.1 Collation Derivation

The process of determining the collation of a character result of an SQL operation is called *collation derivation*. Such operation may be an operator, column reference, character literal, bind variable reference, function call, CASE expression, or a query clause.

① See Also

"[Collation Derivation and Determination Rules for SQL Operations](#)" for more information about *collation derivation*.

5.11.4.7.2 Collation Determination

Collation determination is the process of selecting the right collation to apply during the execution of a collation-sensitive operation. A collation-sensitive operation can be an SQL operator, condition, built-in function call, CASE expression or a query clause.

① See Also

"[Collation Derivation and Determination Rules for SQL Operations](#)" for more information about *collation determination*.

5.11.4.7.3 Expression Evaluation and the COLLATE Operator

You can override the derived collation of any expression node, that is, a simple expression or an operator result, with the COLLATE operator. The COLLATE operator does for collations what the CAST operator does for data types. The COLLATE operator must specify a collation or a pseudo-collation by name. Dynamic collation specification in the form of an expression is not allowed. This is different from how collations are specified for the SQL functions NLSSORT, NLS_UPPER, NLS_LOWER, and NLS_INITCAP.

Starting with Oracle Database 12c Release 2 (12.2), the syntax of SQL expressions used in SELECT and DML statements allows changing the collation of a character value expression. The syntax of *compound expression* clause is as follows:

```
{ (expr)
| { + | - | PRIOR } expr
| expr { * | / | + | - | || } expr
```

```
| expr COLLATE collation_name  
}
```

`collation_name` is the name the collation to be assigned to the value of the expression `expr`. The name must be enclosed in double-quotes, if it contains the space character. The `COLLATE` operator overrides the collation that the database derives using the standard collation derivation rules for `expr`. The `COLLATE` operator can be applied only to the expressions of the data types `VARCHAR2`, `CHAR`, `LONG`, `NVARCHAR2`, and `NCHAR`. There is no implicit conversion of the argument of `COLLATE` to a character data type. The `COLLATE` operator has the same precedence as other unary operators, but it is a postfix operator and it is evaluated only after all the prefix operators are evaluated.

See Also

- ["Enabling Data-Bound Collation"](#)
- ["Collation Derivation and Determination Rules for SQL Operations"](#)

5.11.4.7.4 COLLATION Function

Starting with Oracle Database 12c Release 2 (12.2), the function `COLLATION` returns the derived data-bound collation of a character expression.

```
COLLATION( expr );
```

`expr` is an expression of a character data type. The `COLLATION` function returns the name of the derived collation of `expr` as a `VARCHAR2` value. This function returns pseudo-collations as well. The UCA collation names are returned in the long, canonical format with all collation parameters included in the collation name. This function returns `NULL` value, if the collation of the expression is undefined due to any collation conflict in the expression tree.

Note

- The `COLLATION` function returns only the data-bound collations, and not the dynamic collations set by the `NLS_SORT` parameter. Thus, for a column declared as `COLLATE USING_NLS_SORT`, the function returns the character value `"USING_NLS_SORT"`, and not the actual value of the session parameter `NLS_SORT`. You can use the built-in function `SYS_CONTEXT( 'USERENV', 'NLS_SORT' )` to get the actual value of the session parameter `NLS_SORT`.
- The `COLLATION` function used in SQL is evaluated during the compilation of the SQL statement.

5.11.4.7.5 NLS_COLLATION_ID and NLS_COLLATION_NAME Functions

Starting with Oracle Database 12c Release 2 (12.2), the two functions `NLS_COLLATION_ID` and `NLS_COLLATION_NAME` allow numeric collation IDs, as stored in data dictionary, to be translated to collation names and collation names translated to collation IDs.

The syntax for the NLS_COLLATION_ID function is:

```
NLS_COLLATION_ID( expr );
```

expr is an expression that must evaluate to a VARCHAR2 value. The value of expr is taken as a collation name or pseudo-collation name, and the corresponding collation ID is returned by the function. The NULL value is returned, if the collation name is invalid.

The syntax for the NLS_COLLATION_NAME function is:

```
NLS_COLLATION_NAME( expr [, flag] );
```

expr is an expression that must evaluate to a NUMBER value. The value of expr is taken as a collation ID, and the corresponding collation name or pseudo-collation name is returned by the function. The NULL value is returned, if the collation ID is invalid.

The optional parameter flag must evaluate to a VARCHAR2 value. The value of the flag parameter must be 'S', 's', 'L', or 'l'. The default value of the flag parameter is 'L'. This parameter determines the behavior of the function for UCA collations. The flag parameter values 'S' and 's' mean that the UCA collation names are returned in the short format, that is, the format in which all the UCA collation parameters with default values are omitted. The flag parameter values 'L' and 'l' mean that the UCA collation names are returned in the long, canonical format, that is, the format in which all the UCA collation parameters are included, even if they have default values. For example, UCA1210_DUCET and UCA1210_DUCET_S4_VS_BN_NY_EN_FN_HN_DN_MN are short and long names of the same collation respectively.

 See Also

["UCA Collation"](#)

5.11.5 Viewing the Data-Bound Collation of a Database Object

You can view the data-bound collation information for a database object or a column using the following data dictionary views:

Data dictionary views for viewing the default collation of an object

```
DBA|USER_USERS.DEFAULT_COLLATION
```

```
DBA|ALL|USER_TABLES.DEFAULT_COLLATION
```

```
DBA|ALL|USER_VIEWS.DEFAULT_COLLATION
```

```
DBA|ALL|USER_MVIEWS.DEFAULT_COLLATION
```

```
DBA|ALL|USER_OBJECTS.DEFAULT_COLLATION
```

Data dictionary views for viewing the collation of a table, a view, or a cluster column

```
{DBA|ALL|USER}_TAB_COLS.COLLATION
```

```
{DBA|ALL|USER}_TAB_COLUMNS.COLLATION
```

Data dictionary views to view the collation association between a virtual column and an original column

The data dictionary views contain the following columns that show the collation association between a virtual column and an original columns whose linguistic behavior the virtual column implements:

{DBA|ALL|USER}_TAB_COLS.COLLATED_COLUMN_ID

{DBA|ALL|USER}_PART_KEY_COLUMNS.COLLATED_COLUMN_ID

{DBA|ALL|USER}_SUBPART_KEY_COLUMNS.COLLATED_COLUMN_ID

Note

The name of a UCA collation is stored in the data dictionary views in the form of a long canonical format with all its parameters, including the parameters with the default values. For example, the UCA collation UCA1210_DUCET is stored in these views as UCA1210_DUCET_S4_VS_BN_NY_EN_FN_HN_DN_MN.

5.11.6 Case-Insensitive Database

Oracle Database supports case-insensitive collations, such as BINARY_CI, BINARY_AI, GENERIC_M_CI, GENERIC_M_AI, UCA1210_DUCET_CI, and UCA1210_DUCET_AI. By applying such collations to SQL operations, an application can perform string comparisons and matching in a case-insensitive way.

Starting with Oracle Database 12c Release 2 (12.2), you can declare a column to be always compared as case-insensitive by specifying a case-insensitive data-bound collation (collation having suffix _CI or _AI) in the column definition. The column collation, if not specified explicitly, is inherited from the table default collation, which in turn is inherited from the schema default collation. This way, you can easily declare all the character columns in a database as case-insensitive by default, and use explicit collation declarations only for columns that require a case-sensitive collation.

See Also

["About Data-Bound Collation"](#)

5.11.7 Effect of Data-Bound Collation on Other Database Objects

This section describes the affect on the following database objects, when they reference a column implementing a data-bound collation:

- [Persistent Objects](#)
- [Standard Indexes](#)
- [Bitmap Join Indexes](#)
- [Primary and Unique Constraints](#)
- [Foreign Key Constraints](#)
- [Partitioning and Sharding](#)

- [Index-organized Tables \(IOTs\)](#)
- [Clusters](#)
- [Table Clustering and Zone Maps](#)
- [Oracle Text Indexes and Other Domain Indexes](#)
- [Other Specific Table Types](#)

Persistent Objects

A database object with content stored persistently in the database, such as index, partition, primary key, unique key, referential constraint, cluster, or zone map, cannot have its content collated persistently based on transient, possibly changing values of session parameters NLS_COMP and NLS_SORT. Therefore, when a pseudo-collation is declared for a key column of such an object, the values of the column are collated and grouped as described below.

Collation Group	Key Column Collation	Collation Used
Group 1	USING_NLS_COMP	BINARY
	USING_NLS_SORT	
	USING_NLS_SORT_CS	
Group 2	USING_NLS_SORT_CI	BINARY_CI
Group 3	USING_NLS_SORT_AI	BINARY_AI

Standard Indexes

Standard indexes, that is, B-tree indexes defined on a column declared with a collation not from **Group 1**, automatically become functional indexes on the function NLSSORT. This functionality is applicable to bitmap indexes as well. The NLSSORT function uses the collation of the index key column, if it is a named collation, or the collation BINARY_CI or BINARY_AI, as described in the section "[Persistent Objects](#)".

Note

An index defined on a column declared with a collation from **Group 1** is created as a standard binary index.

For example, the SQL statements:

```
CREATE TABLE my_table
(
  my_column VARCHAR2(100) COLLATE POLISH,
  ...
);

CREATE [UNIQUE|BITMAP] INDEX my_index ON my_table(my_column);
```

are equivalent to:

```
CREATE TABLE my_table
(
  my_column VARCHAR2(100) COLLATE POLISH,
```

```

    ...
);

CREATE [UNIQUE|BITMAP] INDEX my_index ON
my_table(NLSSORT(my_column, 'NLS_SORT=POLISH'));

```

A compound index key comprising columns that have collations from **Group 1** as well as not from **Group 1** contains both NLSSORT-based expressions and plain columns.

For example, the SQL statements:

```

CREATE TABLE my_table
(
    id          VARCHAR2(20)  COLLATE USING_NLS_COMP,
    my_column   VARCHAR2(100) COLLATE POLISH,
    ...
);

CREATE [UNIQUE|BITMAP] INDEX my_index ON my_table(id, my_column);

```

are equivalent to:

```

CREATE TABLE my_table
(
    id          VARCHAR2(20)  COLLATE USING_NLS_COMP,
    my_column   VARCHAR2(100) COLLATE POLISH,
    ...
);

CREATE [UNIQUE|BITMAP] INDEX my_index ON my_table(id,
NLSSORT(my_column, 'NLS_SORT=POLISH'));

```

You can change the collation of an index key column with the ALTER TABLE MODIFY statement only among collations of the same group as defined in the section "[Persistent Objects](#)". For example, you can change the collation BINARY to USING_NLS_SORT, but not to USING_NLS_SORT_CI or to any other named collation. To change the collation to another value, the index must be dropped first.

Bitmap Join Indexes

A bitmap join index definition can only reference columns with collations BINARY, USING_NLS_COMP, USING_NLS_SORT, and USING_NLS_SORT_CS. For any of these collations, index keys are collated and the join condition is evaluated using the BINARY collation.

The collation of a bitmap join index key column or a column referenced in the bitmap index join condition can be changed with the ALTER TABLE MODIFY statement only among collations permitted in the index definition.

Primary and Unique Constraints

Primary and unique constraints defined on a column declared with a named collation use that collation to determine the uniqueness of the value to be inserted in that column. In this case, a primary constraint or a unique constraint is implemented by using a unique functional index instead of a binary unique index. Primary and unique constraints on columns declared with any

of the pseudo-collations use a variant of the binary collation as described in the section "[Persistent Objects](#)".

The collation of a primary or a unique key column can be changed with the ALTER TABLE MODIFY statement only among collations of the same group as defined in the section and only if no foreign key constraint references the primary or unique key. To change the collation to another value, the constraint must be dropped first.

Foreign Key Constraints

Foreign key constraints use the collation of the referenced primary or unique key columns when comparing key values. The comparison between a foreign key value and a referenced primary key value is not necessarily binary. Foreign constraints on columns declared with any of the pseudo-collations use a variant of the binary collation as described in the section "[Persistent Objects](#)". The collation of a foreign key column cannot be changed with the ALTER TABLE MODIFY statement. To change the collation, the constraint must be dropped first.

Note

The collation of a foreign key column must be the same as the collation of the referenced column. This requirement is checked when the foreign key constraint is defined.

Partitioning and Sharding

Range, list, hash, and referential partitioning use the collations of the columns building the partitioning key to determine the ordering of values for the purpose of assigning them to proper partitions and sub-partitions, and for partition pruning.

In Oracle Database 18c and later, partitioning and partition set key columns with character data types used as sharding keys must have the collation BINARY, USING_NLS_COMP, USING_NLS_SORT, or USING_NLS_SORT_CS. The same collations are required for partitioning key columns in tables that:

- are of XMLType or
- contain columns of XMLType or
- are defined with the FOR EXCHANGE WITH TABLE clause

The collation of a partitioning key column can be changed with the ALTER TABLE MODIFY statement only among the collations of the same group described in the section "[Persistent Objects](#)".

Note

Data-bound collation does not affect system partitioning.

Index-organized Tables (IOTs)

An index-organized table stores columns of its primary key plus zero or more columns in its primary key index, and the rest of the columns in a heap-organized overflow segment.

Starting with Oracle Database 12c Release 2 (12.2), primary key columns of an IOT must have the collation BINARY, USING_NLS_COMP, USING_NLS_SORT, or USING_NLS_SORT_CS. For all these collations, the index key values are collated with BINARY collation.

The collation of a primary key column of an IOT can be changed with the `ALTER TABLE MODIFY` statement to any of the above mentioned collations only.

Clusters

Oracle Database supports hash clusters and index clusters. Index clusters have an index, and the key value ordering for character key columns in this index is sensitive to collation. Hash clusters are not collation-sensitive in general because table rows are grouped based on a numerical hash function. However, the value of a user-defined hash function may depend on the collations of key columns referenced by the function.

Additionally, the `SORT` clause on hash cluster columns instructs Oracle Database to sort the rows of a cluster on those columns after applying the hash function when performing a DML operation. To ensure that hash and index processing is consistent for all the tables of a cluster, key columns of both hash and index clusters having declared collations must match the collations of corresponding columns of tables stored in that cluster.

Note

- Starting with Oracle Database 12c Release 2 (12.2), creation of an index clusters with key columns declared with a collation other than `BINARY`, `USING_NLS_COMP`, `USING_NLS_SORT`, or `USING_NLS_SORT_CS` is not supported. The same restriction applies to columns of hash clusters that have the `SORT` clause. Key columns of hash clusters without the `SORT` clause can have any collation.
- Hash clusters and index clusters have no default collation. Cluster keys usually have very few columns and new columns cannot be added to a cluster using the `ALTER CLUSTER` command. Therefore, default collations are not useful for clusters. The default collation for a column in a cluster is always derived from the effective schema default collation.
- Collation of a table column corresponding to a cluster key cannot be modified with `ALTER TABLE MODIFY` statement.

See Also

["Effective Schema Default Collation"](#)

Table Clustering and Zone Maps

The data-bound collation feature is not supported for table clustering and zone maps. Clustering and zone maps can only be applied to table columns declared with `BINARY` or `USING_NLS_COMP` collation. For all these collations, column values are clustered based on the `BINARY` collation.

Oracle Text Indexes and Other Domain Indexes

The data-bound collation feature is not supported for Oracle Text indexes and other domain indexes. Domain indexes can be created only on table columns declared with collation `BINARY`, `USING_NLS_COMP`, `USING_NLS_SORT`, or `USING_NLS_SORT_CS`. Oracle Text does not use data-bound collation in its processing. Oracle Text has its own mechanisms to specify matching behavior.

Other Specific Table Types

The default table collation and column collations can be specified for temporary and external tables as well.

① Note

User-defined types (UDTs) support only the pseudo-collation `USING_NLS_COMP`. Therefore, nested tables, which are always based on a user-defined collection type, also support `USING_NLS_COMP` collation only.

① See Also

- ["Specifying Data-Bound Collation for a Table"](#)
- ["Specifying Data-Bound Collation for a Column"](#)

5.11.8 Effect of Data-Bound Collation on Distributed Queries and DML Operations

Distributed queries and DML operations may involve one or more database nodes of different Oracle Database releases, such as, 12.2, 12.1, and earlier. Evaluation of different parts of a query may happen in different nodes and determination of particular nodes evaluating particular operators is subject to optimizer decisions. Moreover, a local node is generally aware only of nodes that it directly accesses through database links. Indirect or multi-hop nodes, which are remote nodes accessed through synonyms referenced in the query and defined in directly connected nodes, are not visible to a local node.

Considering the above scenario and the requirement that query results must be deterministic and cannot depend on optimizer decisions, Oracle defines the following behavior for queries and subqueries:

- If an Oracle Database 12.2 node with the data-bound collation feature enabled connects to another Oracle Database 12.2 node with the data-bound collation feature enabled, all data-bound collation related behavior is supported.
- If an Oracle Database 12.1 node or an earlier Oracle Database release node connects to an Oracle Database 12.2 node, the Oracle Database 12.2 node recognizes that the query is coming from an earlier Oracle Database release. If such a query references columns with a declared collation other than `USING_NLS_COMP`, an error is reported. However, if the remote Oracle Database 12.2 node receives a DML statement, the statement is evaluated, even if it references columns with a declared collation other than `USING_NLS_COMP`.
- If a local Oracle Database 12.2 node connects to a remote database node of earlier Oracle Database release, the local database node assumes that any character data coming from the remote database node has the declared collation of `USING_NLS_COMP`. The local database node makes sure that the new SQL operators, such as `COLLATE` and `NLS_COLLATION_NAME`, are not sent to the remote database node. If an SQL statement has to be executed on the remote node (for example, a DML operation on a remote table), and if it contains the new SQL operators or a reference to a local column with collation other than `USING_NLS_COMP`, then an error is reported.

Note

The above rules are applied recursively when additional databases are accessed through database links defined in remote nodes and referenced through synonyms.

5.11.9 Effect of Data-Bound Collation on PL/SQL Types and User-Defined Types

Oracle Database provides limited data-bound collation support for PL/SQL types and user-defined types (UDTs). Only those features are provided in Oracle Database that are needed to maintain forward compatibility of PL/SQL code, with the possible future extension of data-bound collation architecture to PL/SQL without limiting the use of PL/SQL with database objects that use the data-bound collation feature.

The following features related to PL/SQL units and UDTs are provided in Oracle Database for data-bound collation support:

- A PL/SQL procedure, function, package, trigger, or UDT can be created as a valid object, only if the *effective schema default collation* at the time of its creation is `USING_NLS_COMP`, or its definition contains an explicit `DEFAULT COLLATION USING_NLS_COMP` clause. If the resulting default object collation is different from `USING_NLS_COMP`, the database object is created as invalid with a compilation error.

See Also

["Effective Schema Default Collation"](#)

- The new SQL operators `COLLATE`, `COLLATION`, `NLS_COLLATION_ID`, and `NLS_COLLATION_NAME` used in embedded SQL are accepted and passed to the SQL engine, but their functionality is not available in PL/SQL code.
- The database columns with declared collations other than `USING_NLS_COMP` can be referenced in embedded SQL, but not in PL/SQL expressions.
- The DML row-level triggers cannot reference fields of `OLD`, `NEW`, or `PARENT` pseudo-records, or correlation names that correspond to columns with declared collation other than `USING_NLS_COMP`.
- The PL/SQL variables referenced in embedded SQL statements have the pseudo-collation `USING_NLS_COMP` and the coercibility level 2.
- The `%TYPE` attribute is not allowed on character columns with a declared collation other than the pseudo-collation `USING_NLS_COMP`. Similarly, the `%ROWTYPE` attribute is not allowed on tables, views, cursors, or cursor variables with at least one character column with a declared collation other than `USING_NLS_COMP`. The columns with collations other than `USING_NLS_COMP` can be selected into `INTO` clause variables declared without those attributes. PL/SQL variables always have the default collation `USING_NLS_COMP`. Thus, whatever is the collation of the selected columns, it is always overridden with the pseudo-collation `USING_NLS_COMP` for PL/SQL processing.
- The cursor `FOR LOOP` statements are not allowed on cursors that return result set columns with collation other than the pseudo-collation `USING_NLS_COMP`.
- A relational column created to store an UDT attribute, whether of an object column or of an object table, inherits the attribute's collation property. However, as all UDTs are created

using the pseudo-collation `USING_NLS_COMP`, any relevant columns for UDT attributes are also created with the pseudo-collation `USING_NLS_COMP`.

- A `WHEN` condition in a trigger is evaluated by the SQL engine, and hence, it supports the data-bound collation feature. A `WHEN` condition can reference a column with declared collation other than `USING_NLS_COMP`, and can use the new operators and functions.

5.11.10 Effect of Data-Bound Collation on Oracle XML DB

The XML Query standard XQuery defines features to specify collation for collation-sensitive operators in XML Query expressions. An XQuery collation can be specified for a particular operator, similar to how collation is specified in the second parameter of the Oracle SQL function `NLS_UPPER`, or as a default collation in the static context of an XQuery expression. XQuery does not provide any mechanism to declare collation for a data container or data source. Therefore, the declared collations of any relational database columns passed as arguments in the `PASSING` clause of the `XMLQuery`, `XMExists`, or `XMLTable` operator are ignored by Oracle XML DB.

6

Supporting Multilingual Databases with Unicode

This chapter illustrates how to use the Unicode Standard in an Oracle Database environment. This chapter includes the following topics:

- [What is the Unicode Standard?](#)
- [Features of the Unicode Standard](#)
- [Implementing a Unicode Solution in the Database](#)
- [Unicode Case Studies](#)
- [Designing Database Schemas to Support Multiple Languages](#)

6.1 What is the Unicode Standard?

The Unicode Standard is a character encoding system that defines every character in most of the spoken languages in the world.

To overcome the limitations of existing character encodings, several organizations began working on the creation of a global character set in the late 1980s. The need for this became even greater with the development of the World Wide Web in the mid-1990s. The Internet has changed how companies do business, with an emphasis on the global market that has made a universal character set a major requirement.

A global character set needs to fulfill the following conditions:

- Contain all major living scripts
- Support legacy data and implementations
- Be simple enough that a single implementation of an application is sufficient for worldwide use

A global character set should also have the following capabilities:

- Support multilingual users and organizations
- Conform to international standards
- Enable worldwide interchange of data

The Unicode Standard, which is now in wide use, meets all of the requirements and capabilities of a global character set. It provides a unique code value for every character, regardless of the platform, program, or language. It also defines a number of character properties and processing rules that help implement complex multilingual text processing correctly and consistently. Bi-directional behavior, word breaking, and line breaking are examples of such complex processing.

The Unicode Standard has been adopted by many software and hardware vendors. Many operating systems and browsers now support the standard. The Unicode Standard is required by other standards such as XML, Java, JavaScript, LDAP, and WML. It is also synchronized with the ISO/IEC 10646 standard.

Oracle Database introduced the Unicode Standard character encoding as the now obsolete database character set AL24UTF8S in Oracle Database 7. Since then, incremental improvements have been made in each release to synchronize the support with the new published version of the standard.

① Note

Oracle AI Database 26ai, RU 23.26.2 and earlier, supports Unicode version 15.0.
Oracle AI Database 26ai, RU 23.26.3 and later, supports Unicode version 17.0.

① See Also

The Unicode Consortium website for more information about the Unicode Standard

6.2 Features of the Unicode Standard

This section contains the following topics:

- [Code Points and Supplementary Characters](#)
- [Unicode Encoding Forms](#)
- [Support for the Unicode Standard in Oracle Database](#)

6.2.1 Code Points and Supplementary Characters

The first version of the Unicode Standard was a 16-bit, fixed-width encoding that used two bytes to encode each character. This enabled 65,536 characters to be represented. However, more characters need to be supported, especially additional CJK ideographs that are important for the Chinese, Japanese, and Korean markets.

The current definition of the Unicode Standard assigns a number to each character defined in the standard. These numbers are called code points, and are in the range 0 to 10FFFF hexadecimal. The Unicode notation for representing character code points is the prefix "U+" followed by the hexadecimal code point value. The code point value is left-padded with non-significant zeros to the minimum length of four. Characters with code points U+0000 to U+FFFF are called Basic Multilingual Plane characters. Characters with code points U+10000 to U+10FFFF are called supplementary characters.

Adding supplementary characters has increased the complexity of the Unicode 16-bit, fixed-width encoding form; however, this is still far less complex than managing hundreds of legacy encodings used before Unicode.

6.2.2 Unicode Encoding Forms

The Unicode Standard defines a few encoding forms, which are mappings from Unicode code points to code units. Code units are integer values processed by applications. Code units may have 8, 16, or 32 bits. The standard encoding forms are: UTF-8, UTF-16, and UTF-32. There are also two compatibility encodings mentioned in the standard and its associated technical reports: UCS-2 and CESU-8. Conversion between different Unicode encodings is a simple bit-wise operation that is defined in the standard.

This section contains the following topics:

- [UTF-8 Encoding Form](#)
- [UTF-16 Encoding Form](#)
- [UCS-2 Encoding Form](#)
- [UTF-32 Encoding Form](#)
- [CESU-8 Encoding Form](#)
- [Examples: UTF-16, UTF-8, and UCS-2 Encoding](#)

6.2.2.1 UTF-8 Encoding Form

UTF-8 is the 8-bit encoding form of Unicode. It is a variable-width encoding and a **strict superset** of ASCII. This means that each and every character in the ASCII character set is available in UTF-8 with the same byte representation. One Unicode character can be represented by 1 byte, 2 bytes, 3 bytes, or 4 bytes in the UTF-8 encoding form. Characters from the European and Middle Eastern scripts are represented in either 1 or 2 bytes. Characters from most Asian scripts are represented in 3 bytes. Supplementary characters are represented in 4 bytes.

UTF-8 is the Unicode encoding used for HTML and most Internet browsers.

The benefits of UTF-8 are as follows:

- Compact storage requirement for European scripts because it is a strict superset of ASCII
- Ease of migration between ASCII-based character sets and UTF-8

① See Also

- ["Code Points and Supplementary Characters"](#)
- [Table B-2](#)

6.2.2.2 UTF-16 Encoding Form

UTF-16 is the 16-bit encoding form of Unicode. One character can be represented by either one 16-bit integer value (two bytes) or two 16-bit integer values (four bytes) in UTF-16. All characters from the Basic Multilingual Plane, which are most characters used in everyday text, are represented in two bytes. Supplementary characters are represented in four bytes. The two code units (integer values) encoding a single supplementary character are called a surrogate pair.

UTF-16 is the main Unicode encoding used for internal processing by Java since version J2SE 5.0 and by Microsoft Windows since version 2000.

The benefits of UTF-16 over UTF-8 are as follows:

- More compact storage for Asian scripts because most of the commonly used Asian characters are represented in two bytes.
- Better compatibility with Java and Microsoft clients

① See Also

- ["Code Points and Supplementary Characters"](#)
- [Table B-1](#)

6.2.2.3 UCS-2 Encoding Form

UCS-2 is not an official Unicode encoding form. The name originally comes from older versions of the ISO/IEC 10646 standard, before the introduction of the supplementary characters. Therefore, it is currently used to refer to the UTF-16 encoding form stripped from support for supplementary characters and surrogate pairs. That is, surrogate pairs are processed in UCS-2 as two separate characters. Applications supporting UCS-2 but not UTF-16 should not process text containing supplementary characters, as they may incorrectly split surrogate pairs when dividing text into fragments. They are also generally incapable of displaying such text.

UCS-2 is the Unicode encoding used for internal processing by Java before version J2SE 5.0 and by Microsoft Windows NT.

6.2.2.4 UTF-32 Encoding Form

UTF-32 is the 32-bit encoding form of Unicode. Each Unicode code point is represented by a single 32-bit, fixed-width integer value. It is the simplest encoding form, but very space inefficient. For English text, it quadruples the storage requirements compared to UTF-8 and doubles when compared to UTF-16. Therefore, UTF-32 is sometimes used as an intermediate form in internal text processing, but it is generally not used for information interchange.

In Java, since version J2SE 5.0, selected APIs have been enhanced to operate on characters in the 32-bit form, stored as `int` values.

6.2.2.5 CESU-8 Encoding Form

CESU-8 is not part of the core Unicode Standard. It is described in the Unicode Technical Report #26 published by The Unicode Consortium. CESU-8 is a compatibility encoding form identical to UTF-8 except for its representation of supplementary characters. In CESU-8, supplementary characters are represented as surrogate pairs, as in UTF-16. To obtain the CESU-8 encoding of a supplementary character, encode the character in UTF-16 first and then treat each of the surrogate code units as a code point with the same value. Then, apply the UTF-8 encoding rules (bit transformation) to each of the code points. This will yield two three-byte representations, six bytes in total.

CESU-8 has only two benefits:

- It has the same binary sorting order as UTF-16.
- It uses the same number of codes per character (one or two). This is important for character length semantics in string processing.

In general, the CESU-8 encoding form should be avoided as much as possible.

① See Also

Unicode Technical Report #26 "Compatibility Encoding Scheme for UTF-16: 8-Bit (CESU-8)" published on *The Unicode Consortium* website

6.2.2.6 Examples: UTF-16, UTF-8, and UCS-2 Encoding

The following table shows some characters and their character codes in UTF-16, UTF-8, and UCS-2 encoding. The last character is a treble clef (a music symbol), a supplementary character.

Character	UTF-16	UTF-8	UCS-2
A	0041	41	0041
c	0063	63	0063
Ö	00F6	C3 B6	00F6
亜	4E9C	E4 BA 9C	4E9C
𝄞	D834 DD1E	F0 9D 84 9E	N/A

6.2.3 Support for the Unicode Standard in Oracle Database

Oracle Database began supporting the Unicode character set as a database character set in release 7. [Table 6-1](#) summarizes the Unicode character sets supported by Oracle Database.

Table 6-1 Unicode Character Sets Supported by Oracle Database

Character Set	Supported in RDBMS Release	Unicode Encoding Form	Unicode Version	Database Character Set	National Character Set
AL24UTFFSS	7.2 to 8i	UTF-8	1.1	Yes	No
UTF8	8.0 to 26ai	CESU-8	Oracle Database release 8.0 through Oracle8i Release 8.1.6: 2.1 Oracle8i Database release 8.1.7 and later: 3.0	Yes	Yes (Oracle9i Database and later versions only)
UTFE	8.0 to 26ai	UTF-EBCDIC	Oracle8i Database releases 8.0 through 8.1.6: 2.1 For Oracle8i Database release 8.1.7 and later: 3.0	Yes ¹	No

Table 6-1 (Cont.) Unicode Character Sets Supported by Oracle Database

Character Set	Supported in RDBMS Release	Unicode Encoding Form	Unicode Version	Database Character Set	National Character Set
AL32UTF8	9i to 26ai	UTF-8	Oracle9i Database release 1: 3.0 Oracle9i Database release 2: 3.1 Oracle Database 10g, release 1: 3.2 Oracle Database 10g, release 2: 4.0 Oracle Database 11g: 5.0 Oracle Database 12c, release 1: 6.2 Oracle Database 12c, release 2: 7.0 Oracle Database 18c to Oracle Database 19c: 9.0 Oracle Database 21c: 12.1 Oracle AI Database 26ai, RU 23.26.2 and earlier: 15.0 Oracle AI Database 26ai, RU 23.26.3 and later: 17.0	Yes	No
AL16UTF16	9i to 26ai	UTF-16	Oracle9i Database release 1: 3.0 Oracle9i Database release 2: 3.1 Oracle Database 10g, release 1: 3.2 Oracle Database 10g, release 2: 4.0 Oracle Database 11g: 5.0 Oracle Database 12c, release 1: 6.2 Oracle Database 12c, release 2: 7.0 Oracle Database 18c to Oracle Database 19c: 9.0 Oracle Database 21c: 12.1 Oracle AI Database 26ai, RU 23.26.2 and earlier: 15.0 Oracle AI Database 26ai, RU 23.26.3 and later: 17.0	No	Yes

¹ UTF-EBCDIC is a compatibility encoding form specific to EBCDIC-based systems, such as IBM z/OS or Fujitsu BS2000. It is described in the Unicode Technical Report #16. Oracle character set UTFE is a partial implementation of the UTF-EBCDIC encoding form, supported on EBCDIC-based platforms only. Oracle Database does not support five-byte sequences of this encoding form, limiting the supported code point range to U+0000 - U+3FFFF. The use of the UTFE character set is discouraged.

6.3 Implementing a Unicode Solution in the Database

Unicode characters can be stored in an Oracle database in two ways:

- You can create a database that enables you to store UTF-8 encoded characters as SQL CHAR data types (CHAR, VARCHAR2, CLOB, and LONG).
- You can store Unicode data in either the UTF-16 or CESU-8 encoding form in SQL NCHAR data types (NCHAR, NVARCHAR2, and NCLOB). The SQL NCHAR data types are called Unicode data types because they are used only for storing Unicode data.

Note

You can combine both Unicode solutions, if required by different applications running in a single database.

The following sections explain how to use the two Unicode solutions and how to choose between them:

- [Enabling Multilingual Support for a Database](#)
- [Enabling Multilingual Support with Unicode Data Types](#)
- [How to Choose Between Unicode Solutions](#)

6.3.1 Enabling Multilingual Support for a Database

The database character set specifies the encoding to be used in the SQL CHAR data types as well as the metadata such as table names, column names, and SQL statements. A **Unicode Standard-enabled database** is a database with a Unicode Standard-compliant character set as the database character set. There are two database Oracle character sets that implement the Unicode Standard.

- AL32UTF8

The AL32UTF8 character set implements the UTF-8 encoding form and supports the latest version of the Unicode standard. It encodes characters in one, two, three, or four bytes. Supplementary characters require four bytes. It is for ASCII-based platforms.

AL32UTF8 is the recommended database character set for any new deployment of Oracle Database as it provides the optimal support for multilingual applications, such as Internet websites and applications for multinational companies.

- UTF8

The UTF8 character set implements the CESU-8 encoding form and encodes characters in one, two, or three bytes. It is for ASCII-based platforms.

Supplementary characters inserted into a UTF8 database are stored in the CESU-8 encoding form. Each character is represented by two three-byte codes and hence occupies six bytes of memory in total.

The properties of characters in the UTF8 character set are not guaranteed to be updated beyond version 3.0 of the Unicode Standard.

Oracle recommends that you switch to AL32UTF8 for full support of the supplementary characters and the most recent versions of the Unicode Standard.

Note

- Specify a database character set when you create a database. Oracle recommends using AL32UTF8 as the database character set. AL32UTF8 is the proper implementation of the Unicode encoding UTF-8. Starting with Oracle Database 12c Release 2, AL32UTF8 is used as the default database character set while creating a database using Oracle Universal Installer (OUI) as well as Oracle Database Configuration Assistant (DBCA).
- Do not use UTF8 as the database character set as it is not a proper implementation of the Unicode encoding UTF-8. If the UTF8 character set is used where UTF-8 processing is expected, then data loss and security issues may occur. This is especially true for Web related data, such as XML and URL addresses.
- AL32UTF8 and UTF8 character sets are not compatible with each other as they have different maximum character widths. AL32UTF8 has a maximum character width of 4 bytes, whereas UTF8 has a maximum character width of 3 bytes.
- If the CHARACTER SET clause is not specified in the CREATE DATABASE statement explicitly, then the database character set defaults to US7ASCII (except on EBCDIC platforms).

Example 6-1 Creating a Database with a Unicode Character Set

To create a database with the AL32UTF8 character set, use the CREATE DATABASE statement and include the CHARACTER SET AL32UTF8 clause. For example:

```
CREATE DATABASE sample

CONTROLFILE REUSE
LOGFILE

GROUP 1 ('diskx:log1.log', 'disky:log1.log') SIZE 50K,
GROUP 2 ('diskx:log2.log', 'disky:log2.log') SIZE 50K

MAXLOGFILES 5
MAXLOGHISTORY 100
MAXDATAFILES 10
MAXINSTANCES 2
ARCHIVELOG
CHARACTER SET AL32UTF8
NATIONAL CHARACTER SET AL16UTF16
DATAFILE

'disk1:df1.dbf' AUTOEXTEND ON,
'disk2:df2.dbf' AUTOEXTEND ON NEXT 10M MAXSIZE UNLIMITED

DEFAULT TEMPORARY TABLESPACE temp_ts
UNDO TABLESPACE undo_ts
SET TIME_ZONE = '+00:00';
```

6.3.2 Enabling Multilingual Support with Unicode Data Types

An alternative to storing Unicode data in the database is to use the SQL NCHAR data types (NCHAR, NVARCHAR2, NCLOB). You can store Unicode characters in columns of these data types regardless of how the database character set has been defined. The NCHAR data type is

exclusively a Unicode data type, which means that it stores data encoded in a Unicode encoding form.

Oracle recommends using SQL CHAR, VARCHAR2, and CLOB data types in AL32UTF8 database to store Unicode character data. SQL NCHAR, NVARCHAR2, and NCLOB data types are not supported by some database features. Most notably, Oracle Text and XML DB do not support these data types.

You can create a table using the NVARCHAR2 and NCHAR data types. The column length specified for the NCHAR and NVARCHAR2 columns always equals the number of characters instead of the number of bytes:

```
CREATE TABLE product_information
( product_id          NUMBER(6)
, product_name        NVARCHAR2(100)
, product_description VARCHAR2(1000));
```

The encoding used in the SQL NCHAR data types is the national character set specified for the database. You can specify one of the following Oracle character sets as the national character set:

- AL16UTF16

This is the default character set and recommended for SQL NCHAR data types. This character set encodes Unicode data in the UTF-16 encoding form. It supports supplementary characters, which are stored as four bytes.

- UTF8

When UTF8 is specified for SQL NCHAR data types, the data stored in the SQL data types is in CESU-8 encoding form. The UTF8 character set is deprecated.

You can specify the national character set for the SQL NCHAR data types when you create a database using the CREATE DATABASE statement with the NATIONAL CHARACTER SET clause. The following statement creates a database with WE8ISO8859P1 as the database character set and AL16UTF16 as the national character set.

Example 6-2 Creating a Database with a National Character Set

```
CREATE DATABASE sample

CONTROLFILE REUSE
LOGFILE

GROUP 1 ('diskx:log1.log', 'disky:log1.log') SIZE 50K,
GROUP 2 ('diskx:log2.log', 'disky:log2.log') SIZE 50K

MAXLOGFILES 5
MAXLOGHISTORY 100
MAXDATAFILES 10
MAXINSTANCES 2
ARCHIVELOG
CHARACTER SET WE8ISO8859P1
NATIONAL CHARACTER SET AL16UTF16
DATAFILE

'disk1:df1.dbf' AUTOEXTEND ON,
'disk2:df2.dbf' AUTOEXTEND ON NEXT 10M MAXSIZE UNLIMITED

DEFAULT TEMPORARY TABLESPACE temp_ts
UNDO TABLESPACE undo_ts
SET TIME_ZONE = '+00:00';
```


6.3.3 How to Choose Between Unicode Solutions

Oracle recommends that you deploy all new Oracle databases in the database character set AL32UTF8 and you use SQL VARCHAR2, CHAR, and CLOB data types to store character data. The SQL NVARCHAR2, NCHAR, and NCLOB data types should be considered only if:

- You have an existing database with a non-Unicode database character set and a legacy application, for which the business costs of migrating to Unicode would be unacceptable, and you need to add support for multilingual data in a small part of the application or in a small new module for which a separate database would not make much sense, or
- You need to create an application that has to support multilingual data and which must be installable in any of Oracle database deployed by your customers.

For the database character set in a Unicode Standard-enabled database, always select AL32UTF8. For the national character set, select AL16UTF16. If you consider choosing the deprecated UTF8 because of the lower storage requirements for English character data, first consider other options, such as data compression or increasing disk storage. Later migration to AL16UTF16 may be expensive, if a lot of data accumulates in the database.

Note

- Oracle recommends using AL32UTF8 as the database character set. AL32UTF8 is the proper implementation of the Unicode encoding UTF-8. Starting with Oracle Database 12c Release 2, AL32UTF8 is used as the default database character set while creating a database using Oracle Universal Installer (OUI) as well as Oracle Database Configuration Assistant (DBCA).
- Do not use UTF8 as the database character set as it is not a proper implementation of the Unicode encoding UTF-8. If the UTF8 character set is used where UTF-8 processing is expected, then data loss and security issues may occur. This is especially true for Web related data, such as XML and URL addresses.
- AL32UTF8 and UTF8 character sets are not compatible with each other as they have different maximum character widths. AL32UTF8 has a maximum character width of 4 bytes, whereas UTF8 has a maximum character width of 3 bytes.

6.4 Unicode Case Studies

This section describes typical scenarios for storing Unicode characters in an Oracle database:

- [Scenario 1: Unicode Solution with a Unicode Standard-Enabled Database](#)
- [Scenario 2: Unicode Solution with Unicode Data Types](#)

Scenario 1: Unicode Solution with a Unicode Standard-Enabled Database

An American company running a Java application would like to add German and French support in the next release of the application. They would like to add Japanese support at a later time. The company currently has the following system configuration:

- The existing database has a database character set of US7ASCII.
- All character data in the existing database is composed of ASCII characters.

- PL/SQL stored procedures are used in the database.
- The database is about 300 GB, with very little data stored in CLOB columns.
- There is a nightly downtime of 4 hours.

In this case, a typical solution is to choose AL32UTF8 for the database character set because of the following reasons:

- The database is very large and the scheduled downtime is short. Fast migration of the database to a Unicode character set is vital. Because the database is in US7ASCII, the easiest and fastest way of enabling the database to support the Unicode Standard is to switch the database character set to AL32UTF8 by using the Database Migration Assistant for Unicode (DMU). No data conversion is required for columns other than CLOB because US7ASCII is a subset of AL32UTF8.
- Because most of the code is written in Java and PL/SQL, changing the database character set to AL32UTF8 is unlikely to break existing code. Unicode support is automatically enabled in the application.

Scenario 2: Unicode Solution with Unicode Data Types

A European company that runs its legacy applications mainly on Windows platforms wants to add a new small Windows application written in Visual C/C++. The new application will use the existing database to support Japanese and Chinese customer names. The company currently has the following system configuration:

- The existing database has a database character set of WE8MSWIN1252.
- All character data in the existing database is composed of Western European characters.
- The database is around 500 GB with a lot of CLOB columns.
- Support for full-text search and XML storage is not required in the new application

A typical solution is to take the following actions:

- Use NCHAR and NVARCHAR2 data types to store Unicode characters
- Keep WE8MSWIN1252 as the database character set
- Use AL16UTF16 as the national character set

The reasons for this solution are:

- Migrating the existing database to a Unicode database requires data conversion because the database character set is WE8MSWIN1252 (a Windows Latin-1 character set), which is not a subset of AL32UTF8. Also, a lot of data is stored in CLOB columns. All CLOB values in a database, even if they contain only ASCII characters, must be converted when migrating from a single-byte database character set, such as US7ASCII or WE8MSWIN1252 to AL32UTF8. As a result, there will be a lot of overhead in converting the data to AL32UTF8.
- The additional languages are supported in the new application only. It does not depend on the existing applications or schemas. It is simpler to use the Unicode data type in the new schema and keep the existing schemas unchanged.
- Only customer name columns require Unicode character set support. Using a single NCHAR column meets the customer's requirements without migrating the entire database.
- The new application does not need database features that do not support SQL NCHAR data types.

- The lengths of the SQL NCHAR data types are defined as number of characters. This is the same as how they are treated when using wchar_t strings in Windows C/C++ programs. This reduces programming complexity.
- Existing applications using the existing schemas are unaffected.

6.5 Designing Database Schemas to Support Multiple Languages

In addition to choosing a Unicode solution, the following issues should be taken into consideration when the database schema is designed to support multiple languages:

- [Specifying Column Lengths for Multilingual Data](#)
- [Storing Data in Multiple Languages](#)
- [Storing Documents in Multiple Languages in LOB Data Types](#)
- [Creating Indexes for Searching Multilingual Document Contents](#)

6.5.1 Specifying Column Lengths for Multilingual Data

When you use NCHAR and NVARCHAR2 data types for storing multilingual data, the column size specified for a column is defined in number of characters. (This number of characters means the number of encoded Unicode code points, except that supplementary Unicode characters represented through surrogate pairs count as two characters.)

The following table shows the maximum size of the NCHAR and NVARCHAR2 data types for the AL16UTF16 and UTF8 national character sets.

Table 6-2 Maximum Data Type Size for the AL16UTF16 and UTF8 National Character Sets

National Character Set	Maximum Column Size of NCHAR Data Type	Maximum Column Size of NVARCHAR2 Data Type (When MAX_STRING_SIZE = STANDARD)	Maximum Column Size of NVARCHAR2 Data Type (When MAX_STRING_SIZE = EXTENDED)
AL16UTF16	1000 characters	2000 characters	16383 characters
UTF8	2000 characters	4000 characters	32767 characters

This maximum size in characters is a constraint, not guaranteed capacity of the data type. The maximum capacity is expressed in bytes.

For the NCHAR data type, the maximum capacity is 2000 bytes. For NVARCHAR2, it is 4000 bytes, if the initialization parameter MAX_STRING_SIZE is set to STANDARD, and 32767 bytes, if the initialization parameter MAX_STRING_SIZE is set to EXTENDED

When the national character set is AL16UTF16, the maximum number of characters never occupies more bytes than the maximum capacity, as each character (in an Oracle sense) occupies exactly 2 bytes. However, if the national character set is UTF8, the maximum number of characters can be stored only if all these characters are from the Unicode Basic Latin range, which corresponds to the ASCII standard.

Other Unicode characters occupy more than one byte each in UTF8 and presence of such characters in a 4000 character string makes the string longer than the maximum 4000 bytes. If you want national character set columns to be able to hold the declared number of characters in any national character set, do not declare NCHAR columns longer than 2000/3=666

characters and NVARCHAR2 columns longer than $4000/3=1333$ or $32767/3=10922$ characters, depending on the MAX_STRING_SIZE initialization parameter.

When you use CHAR and VARCHAR2 data types for storing multilingual data, the maximum length specified for each column is, by default, in number of bytes. If the database needs to support Thai, Arabic, or multibyte languages such as Chinese and Japanese, then the maximum lengths of the CHAR, VARCHAR, and VARCHAR2 columns may need to be extended. This is because the number of bytes required to encode these languages in UTF8 or AL32UTF8 may be significantly larger than the number of bytes for encoding English and Western European languages. For example, one Thai character in the Thai character set requires 3 bytes in UTF8 or AL32UTF8. Application designers should consider using an extended character data type or CLOB data type if they need to store data larger than 4000 bytes.

❏ See Also

- *Oracle AI Database SQL Language Reference*
- *Oracle AI Database Reference* for more information about extending character data types by setting MAX_STRING_SIZE to the value of EXTENDED

6.5.2 Storing Data in Multiple Languages

The Unicode character set includes characters of most written languages around the world, but it does not contain information about the language to which a given character belongs. In other words, a character such as ä does not contain information about whether it is a Swedish or German character. In order to provide information in the language a user desires, data stored in a Unicode database should be tagged with the language information to which the data belongs.

There are many ways for a database schema to relate data to a language. The following sections discuss example steps to achieve this goal.

Store Language Information with the Data

For data such as product descriptions or product names, you can add a language column (language_id) of CHAR or VARCHAR2 data type to the product table to identify the language of the corresponding product information. This enables applications to retrieve the information in the desired language. The possible values for this language column are the 3-letter abbreviations of the valid NLS_LANGUAGE values of the database.

❏ See Also

["Locale Data"](#) for a list of NLS_LANGUAGE values and their abbreviations

You can also create a view to select the data of the current language. For example:

```
ALTER TABLE scott.product_information ADD (language_id VARCHAR2(50));
```

```
CREATE OR REPLACE VIEW product AS
  SELECT product_id, product_name
  FROM   product_information
  WHERE  language_id = SYS_CONTEXT('USERENV', 'LANG');
```

Select Translated Data Using Fine-Grained Access Control

Fine-grained access control enables you to limit the degree to which a user can view information in a table or view. Typically, this is done by appending a WHERE clause. When you add a WHERE clause as a fine-grained access policy to a table or view, Oracle automatically appends the WHERE clause to any SQL statements on the table at run time so that only those rows satisfying the WHERE clause can be accessed.

You can use this feature to avoid specifying the desired language of a user in the WHERE clause in every SELECT statement in your applications. The following WHERE clause limits the view of a table to the rows corresponding to the desired language of a user:

```
WHERE language_id = SYS_CONTEXT('userenv', 'LANG')
```

Specify this WHERE clause as a fine-grained access policy for product_information as follows:

```
CREATE FUNCTION func1 (sch VARCHAR2 , obj VARCHAR2 )  
RETURN VARCHAR2(100);  
BEGIN  
RETURN 'language_id = SYS_CONTEXT(''userenv'', ''LANG'')';  
END  
/  
  
DBMS_RLS.ADD_POLICY ('scott', 'product_information', 'lang_policy', 'scott', 'func1',  
'select');
```

Then any SELECT statement on the product_information table automatically appends the WHERE clause.

See Also

Oracle AI Database Development Guide for more information about fine-grained access control

6.5.3 Storing Documents in Multiple Languages in LOB Data Types

You can store documents in multiple languages in CLOB, NCLOB, or BLOB data types and set up Oracle Text to enable content search for the documents.

Data in CLOB columns is stored in the AL16UTF16 character set when the database character set is multibyte, such as UTF8 or AL32UTF8. This means that the storage space required for an English document doubles when the data is converted. Storage for an Asian language document in a CLOB column requires less storage space than the same document in a LONG column using AL32UTF8, typically around 30% less, depending on the contents of the document.

Documents in NCLOB format are also stored in the AL16UTF16 character set regardless of the database character set or national character set. The storage space requirement is the same as for CLOB data. Document contents are converted to UTF-16 when they are inserted into a NCLOB column. If you want to store multilingual documents in a non-Unicode database, then choose NCLOB. However, content search on NCLOB with Oracle Text is not supported.

Documents in BLOB format are stored as they are. No data conversion occurs during insertion and retrieval. However, SQL string manipulation functions (such as LENGTH or SUBSTR) and collation functions (such as NLS_SORT and ORDER BY) cannot be applied to the BLOB data type.

The following table lists the advantages and disadvantages of the CLOB, NCLOB, and BLOB data types when storing documents:

Table 6-3 Comparison of LOB Data Types for Document Storage

Data Types	Advantages	Disadvantages
CLOB	- Content search support with Oracle Text - String manipulation support	- Depends on database character set - Data conversion is necessary for insertion - Cannot store binary documents
NCLOB	- Independent of database character set - String manipulation support	- No content search support - Data conversion is necessary for insertion - Cannot store binary documents
BLOB	- Independent of database character set - Content search support - No data conversion, data stored as is - Can store binary documents, such as Microsoft Word or Microsoft Excel	No string manipulation support

6.5.4 Creating Indexes for Searching Multilingual Document Contents

Oracle Text enables you to build indexes for content search on multilingual documents stored in CLOB format and BLOB format. It uses a language-specific lexer to parse the CLOB or BLOB data and produces a list of searchable keywords.

Create a multilexer to search multilingual documents. The multilexer chooses a language-specific lexer for each row, based on a language column. This section describes the high level steps to create indexes for documents in multiple languages. It contains the following topics:

- [Creating Multilexers](#)
- [Creating Indexes for Documents Stored in the CLOB Data Type](#)
- [Creating Indexes for Documents Stored in the BLOB Data Type](#)

① See Also

Oracle Text Reference

6.5.4.1 Creating Multilexers

The first step in creating the multilexer is the creation of language-specific lexer preferences for each language supported. The following example creates English, German, and Japanese lexers with PL/SQL procedures:

```
ctx_ddl.create_preference('english_lexer', 'basic_lexer');
ctx_ddl.set_attribute('english_lexer', 'index_themes', 'yes');
ctx_ddl.create_preference('german_lexer', 'basic_lexer');
ctx_ddl.set_attribute('german_lexer', 'composite', 'german');
ctx_ddl.set_attribute('german_lexer', 'alternate_spelling', 'german');
ctx_ddl.set_attribute('german_lexer', 'mixed_case', 'yes');
ctx_ddl.create_preference('japanese_lexer', 'JAPANESE_VGRAM_LEXER');
```

After the language-specific lexer preferences are created, they need to be gathered together under a single multilexer preference. First, create the multilexer preference, using the MULTI_LEXER object:

```
ctx_ddl.create_preference('global_lexer','multi_lexer');
```

Now add the language-specific lexers to the multilexer preference using the add_sub_lexer call:

```
ctx_ddl.add_sub_lexer('global_lexer', 'german', 'german_lexer');
ctx_ddl.add_sub_lexer('global_lexer', 'japanese', 'japanese_lexer');
ctx_ddl.add_sub_lexer('global_lexer', 'default', 'english_lexer');
```

This nominates the german_lexer preference to handle German documents, the japanese_lexer preference to handle Japanese documents, and the english_lexer preference to handle everything else, using DEFAULT as the language.

6.5.4.2 Creating Indexes for Documents Stored in the CLOB Data Type

The multilexer decides which lexer to use for each row based on a language column in the table. This is a character column that stores the language of the document in a text column. Use the Oracle language name to identify the language of a document in this column. For example, if you use the CLOB data type to store your documents, then add the language column to the table where the documents are stored:

```
CREATE TABLE globaldoc
(doc_id      NUMBER          PRIMARY KEY,
 language    VARCHAR2(30),
 text        CLOB);
```

To create an index for this table, use the multilexer preference and specify the name of the language column:

```
CREATE INDEX globalx ON globaldoc(text)
  indextype IS ctxsys.context
  parameters ('lexer
               global_lexer
               language
               column
               language');
```

6.5.4.3 Creating Indexes for Documents Stored in the BLOB Data Type

In addition to the language column, the character set and format columns must be added in the table where the documents are stored. The character set column stores the character set of the documents using the Oracle character set names. The format column specifies whether a document is a text or binary document. For example, the CREATE TABLE statement can specify columns called character set and format:

```
CREATE TABLE globaldoc (
  doc_id      NUMBER          PRIMARY KEY,
  language    VARCHAR2(30),
  character set VARCHAR2(30),
  format       VARCHAR2(10),
  text        BLOB
);
```

You can put word-processing or spreadsheet documents into the table and specify binary in the format column. For documents in HTML, XML and text format, you can put them into the table and specify text in the format column.

Because there is a column in which to specify the character set, you can store text documents in different character sets.

When you create the index, specify the names of the format and character set columns:

```
CREATE INDEX globalx ON globaldoc(text)
  indextype is ctxsys.context
  parameters ('filter inso_filter
              lexer global_lexer
              language column language
              format column format
              charset column characteraset');
```

You can use the `charset_filter` if all documents are in text format. The `charset_filter` converts data from the character set specified in the `charset` column to the database character set.

7

Programming with Unicode

This chapter describes how to use programming and access products for Oracle Database with Unicode. This chapter contains the following topics:

- [Overview of Programming with Unicode](#)
- [SQL and PL/SQL Programming with Unicode](#)
- [OCI Programming with Unicode](#)
- [Pro*C/C++ Programming with Unicode](#)
- [JDBC Programming with Unicode](#)
- [ODBC and OLE DB Programming with Unicode](#)
- [XML Programming with Unicode](#)

7.1 Overview of Programming with Unicode

Oracle offers several database access products for inserting and retrieving Unicode data. Oracle offers database access products for commonly used programming environments such as Java and C/C++. Data is transparently converted between the database and client programs, which ensures that client programs are independent of the database character set and national character set. In addition, client programs are sometimes even independent of the character data type, such as NCHAR or CHAR, used in the database.

To avoid overloading the database server with data conversion operations, Oracle always tries to move them to the client side database access products. In a few cases, data must be converted in the database, which affects performance. This chapter discusses details of the data conversion paths.

7.1.1 Database Access Product Stack and Unicode

Oracle offers a comprehensive set of database access products that enable programs from different development environments to access Unicode data stored in the database. These products are listed in the following table.

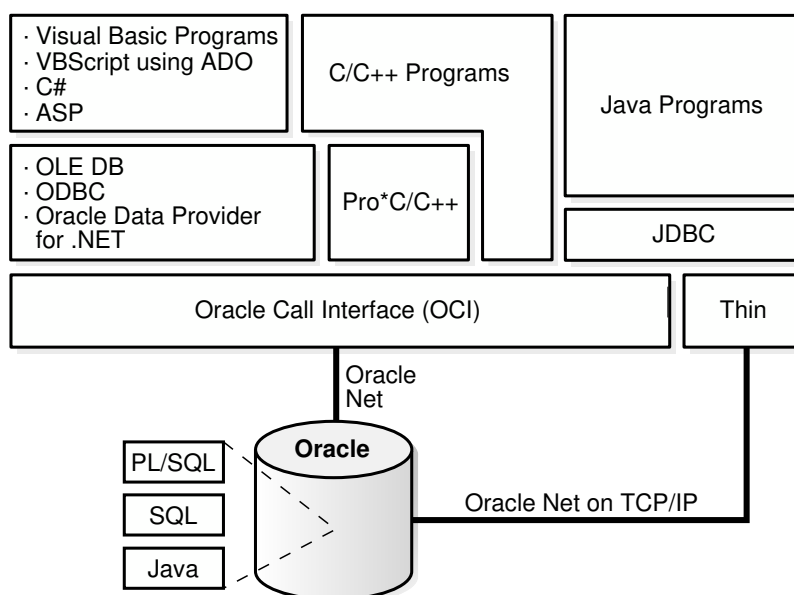
Table 7-1 Oracle Database Access Products

Programming Environment	Oracle Database Access Products
C/C++	Oracle Call Interface (OCI)
	Oracle Pro*C/C++
	Oracle ODBC driver
	Oracle Provider for OLE DB
	Oracle Data Provider for .NET
Java	Oracle JDBC OCI or thin driver
	Oracle server-side thin driver
	Oracle server-side internal driver

Table 7-1 (Cont.) Oracle Database Access Products

Programming Environment	Oracle Database Access Products
PL/SQL	Oracle PL/SQL and SQL
Visual Basic/C#	Oracle ODBC driver Oracle Provider for OLE DB

The following figure shows how the database access products can access the database.

Figure 7-1 Oracle Database Access Products

The Oracle Call Interface (OCI) is the lowest level API that the rest of the client-side database access products use. It provides a flexible way for C/C++ programs to access Unicode data stored in SQL CHAR and NCHAR data types. Using OCI, you can programmatically specify the character set (UTF-8, UTF-16, and others) for the data to be inserted or retrieved. It accesses the database through Oracle Net.

Oracle Pro*C/C++ enables you to embed SQL and PL/SQL in your programs. It uses OCI's Unicode capabilities to provide UTF-16 and UTF-8 data access for SQL CHAR and NCHAR data types.

The Oracle ODBC driver enables C/C++, Visual Basic, and VBScript programs running on Windows platforms to access Unicode data stored in SQL CHAR and NCHAR data types of the database. It provides UTF-16 data access by implementing the SQLWCHAR interface specified in the ODBC standard specification.

The Oracle Provider for OLE DB enables C/C++, Visual Basic, and VBScript programs running on Windows platforms to access Unicode data stored in SQL CHAR and NCHAR data types. It provides UTF-16 data access through wide string OLE DB data types.

The Oracle Data Provider for .NET enables programs running in any .NET programming environment on Windows platforms to access Unicode data stored in SQL CHAR and NCHAR data types. It provides UTF-16 data access through Unicode data types.

Oracle JDBC drivers are the primary Java programmatic interface for accessing an Oracle database. Oracle provides the following JDBC drivers:

- The JDBC OCI driver that is used by Java applications and requires the OCI library
- The JDBC thin driver, which is a pure Java driver that is primarily used by Java applets and supports the Oracle Net protocol over TCP/IP
- The JDBC server-side thin driver, a pure Java driver used inside Java stored procedures to connect to another Oracle server
- The JDBC server-side internal driver that is used inside the Oracle server to access the data in the database

All drivers support Unicode data access to SQL CHAR and NCHAR data types in the database.

The PL/SQL and SQL engines process PL/SQL programs and SQL statements on behalf of client-side programs such as OCI and server-side PL/SQL stored procedures. They allow PL/SQL programs to declare CHAR, VARCHAR2, NCHAR, and NVARCHAR2 variables and to access SQL CHAR and NCHAR data types in the database.

The following sections describe how each of the database access products supports Unicode data access to an Oracle database and offer examples for using those products:

- [SQL and PL/SQL Programming with Unicode](#)
- [OCI Programming with Unicode](#)
- [Pro*C/C++ Programming with Unicode](#)
- [JDBC Programming with Unicode](#)
- [ODBC and OLE DB Programming with Unicode](#)

7.2 SQL and PL/SQL Programming with Unicode

SQL is the fundamental language with which all programs and users access data in an Oracle database either directly or indirectly. PL/SQL is a procedural language that combines the data manipulating power of SQL with the data processing power of procedural languages. Both SQL and PL/SQL can be embedded in other programming languages. This section describes Unicode-related features in SQL and PL/SQL that you can deploy for multilingual applications.

This section contains the following topics:

- [SQL NCHAR Data Types](#)
- [Implicit Data Type Conversion Between NCHAR and Other Data Types](#)
- [Exception Handling for Data Loss During Data Type Conversion](#)
- [Rules for Implicit Data Type Conversion](#)
- [SQL Functions for Unicode Data Types](#)
- [Other SQL Functions](#)
- [Unicode String Literals](#)
- [Using the UTL_FILE Package with NCHAR Data](#)

7.2.1 SQL NCHAR Data Types

There are three SQL NCHAR data types:

- [The NCHAR Data Type](#)

- [The NVARCHAR2 Data Type](#)
- [The NCLOB Data Type](#)

7.2.1.1 The NCHAR Data Type

When you define a table column or a PL/SQL variable as the NCHAR data type, the length is always specified as the number of characters. For example, the following statement creates a column with a maximum length of 30 characters:

```
CREATE TABLE table1 (column1 NCHAR(30));
```

The maximum number of bytes for the column is determined as follows:

maximum number of bytes = (maximum number of characters) x (maximum number of bytes for each character)

For example, if the national character set is UTF8, then the maximum byte length is 30 characters times 3 bytes for each character, or 90 bytes.

The national character set, which is used for all NCHAR data types, is defined when the database is created. The national character set can be either UTF8 or AL16UTF16. The default is AL16UTF16.

The maximum column size allowed is 32000 characters when the national character set is UTF8 and 8000 when it is AL16UTF16. The actual data is subject to the maximum byte limit of 16000. The two size constraints must be satisfied at the same time. In PL/SQL, the maximum length of NCHAR data is 32767 bytes. You can define an NCHAR variable of up to 32767 characters, but the actual data cannot exceed 32767 bytes. If you insert a value that is shorter than the column length, then Oracle pads the value with blanks to whichever length is smaller: maximum character length or maximum byte length.

Note

UTF8 may affect performance because it is a variable-width character set. Excessive blank padding of NCHAR fields decreases performance. Consider using the NVARCHAR2 data type or changing to the AL16UTF16 character set for the NCHAR data type.

7.2.1.2 The NVARCHAR2 Data Type

The NVARCHAR2 data type specifies a variable length character string that uses the national character set. When you create a table with an NVARCHAR2 column, you specify the maximum number of characters for the column. Lengths for NVARCHAR2 are always in units of characters, just as for NCHAR. Oracle subsequently stores each value in the column exactly as you specify it, if the value does not exceed the column's maximum length. Oracle does not pad the string value to the maximum length.

The maximum length for the NVARCHAR2 type is 4000 characters if MAX_STRING_SIZE = STANDARD or 32767 characters if MAX_STRING_SIZE = EXTENDED. These lengths are based on using UTF8; the values are 2000 and 16383 characters when using AL16UTF16.

In PL/SQL, the maximum length for an NVARCHAR2 variable is 32767 bytes. You can define NVARCHAR2 variables up to 32767 characters, but the actual data cannot exceed 32767 bytes.

The following statement creates a table with one NVARCHAR2 column whose maximum length in characters is 2000 and maximum length in bytes is 4000.

```
CREATE TABLE table2 (column2 NVARCHAR2(2000));
```

7.2.1.3 The NCLOB Data Type

NCLOB is a character large object containing Unicode characters, with a maximum size of 4 gigabytes. Unlike the BLOB data type, the NCLOB data type has full transactional support so that changes made through SQL, the DBMS_LOB package, or OCI participate fully in transactions. Manipulations of NCLOB value can be committed and rolled back. Note, however, that you cannot save an NCLOB locator in a PL/SQL or OCI variable in one transaction and then use it in another transaction or session.

NCLOB values are stored in the database in a format that is compatible with UCS-2, regardless of the national character set. Oracle translates the stored Unicode value to the character set requested on the client or on the server, which can be fixed-width or variable-width. When you insert data into an NCLOB column using a variable-width character set, Oracle converts the data into a format that is compatible with UCS-2 before storing it in the database.

📘 See Also

Oracle AI Database SecureFiles and Large Objects Developer's Guide for more information about the NCLOB data type

7.2.2 Implicit Data Type Conversion Between NCHAR and Other Data Types

Oracle supports implicit conversions between SQL NCHAR data types and other Oracle data types, such as CHAR, VARCHAR2, NUMBER, DATE, ROWID, and CLOB. Any implicit conversions for CHAR and VARCHAR2 data types are also supported for SQL NCHAR data types. You can use SQL NCHAR data types the same way as SQL CHAR data types.

Type conversions between SQL CHAR data types and SQL NCHAR data types may involve character set conversion when the database and national character sets are different. Padding with blanks may occur if the target data is either CHAR or NCHAR.

📘 See Also

Oracle AI Database SQL Language Reference

7.2.3 Exception Handling for Data Loss During Data Type Conversion

Data loss can occur during data type conversion when character set conversion is necessary. If a character in the source character set is not defined in the target character set, then a replacement character is used in its place. For example, if you try to insert NCHAR data into a regular CHAR column and the character data in NCHAR (Unicode) form cannot be converted to the database character set, then the character is replaced by a replacement character defined by the database character set. The NLS_NCHAR_CONV_EXCP initialization parameter controls the behavior of data loss during character type conversion. When this parameter is set to TRUE, any SQL statements that result in data loss return an ORA-12713 error and the corresponding operation is stopped. When this parameter is set to FALSE, data loss is not reported and the

unconvertible characters are replaced with replacement characters. The default value is FALSE. This parameter works for both implicit and explicit conversion.

In PL/SQL, when data loss occurs during conversion of SQL CHAR and NCHAR data types, the `LOSSY_CHARSET_CONVERSION` exception is raised for both implicit and explicit conversion.

7.2.4 Rules for Implicit Data Type Conversion

In some cases, conversion between data types is possible in only one direction. In other cases, conversion in both directions is possible. Oracle defines a set of rules for conversion between data types. The following table contains the rules for conversion between data types.

Table 7-2 Rules for Conversion Between Data Types

Statement	Rule
INSERT/UPDATE statement	Values are converted to the data type of the target database column.
SELECT INTO statement	Data from the database is converted to the data type of the target variable.
Variable assignments	Values on the right of the equal sign are converted to the data type of the target variable on the left of the equal sign.
Parameters in SQL and PL/SQL functions	CHAR, VARCHAR2, NCHAR, and NVARCHAR2 are loaded the same way. An argument with a CHAR, VARCHAR2, NCHAR or NVARCHAR2 data type is compared to a formal parameter of any of the CHAR, VARCHAR2, NCHAR or NVARCHAR2 data types. If the argument and formal parameter data types do not match exactly, then implicit conversions are introduced when data is copied into the parameter on function entry and copied out to the argument on function exit.
Concatenation operation or CONCAT function	If one operand is a SQL CHAR or NCHAR data type and the other operand is a NUMBER or other non-character data type, then the other data type is converted to VARCHAR2 or NVARCHAR2. For concatenation between character data types, see " SQL NCHAR data types and SQL CHAR data types ".
SQL CHAR or NCHAR data types and NUMBER data type	Character values are converted to NUMBER data type.
SQL CHAR or NCHAR data types and DATE data type	Character values are converted to DATE data type.
SQL CHAR or NCHAR data types and ROWID data type	Character values are converted to ROWID data type.
SQL NCHAR data types and SQL CHAR data types	Comparisons between SQL NCHAR data types and SQL CHAR data types are more complex because they can be encoded in different character sets. When CHAR and VARCHAR2 values are compared, the CHAR values are converted to VARCHAR2 values. When NCHAR and NVARCHAR2 values are compared, the NCHAR values are converted to NVARCHAR2 values. When there is comparison between SQL NCHAR data types and SQL CHAR data types, character set conversion occurs if they are encoded in different character sets. The character set for SQL NCHAR data types is always Unicode and can be either UTF8 or AL16UTF16 encoding, which have the same character repertoires but are different encodings of the Unicode standard. SQL CHAR data types use the database character set, which can be any character set that Oracle supports. Unicode is a superset of any character set supported by Oracle, so SQL CHAR data types can always be converted to SQL NCHAR data types without data loss.

7.2.5 SQL Functions for Unicode Data Types

SQL NCHAR data types can be converted to and from SQL CHAR data types and other data types using explicit conversion functions. The examples in this section use the table created by the following statement:

```
CREATE TABLE customers
(id NUMBER, name NVARCHAR2(50), address NVARCHAR2(200), birthdate DATE);
```

① See Also

Oracle AI Database SQL Language Reference for more information about explicit conversion functions for SQL NCHAR data types

Example 7-1 Populating the Customers Table Using the TO_NCHAR Function

The TO_NCHAR function converts the data at run time, while the N function converts the data at compilation time.

```
INSERT INTO customers VALUES (1000,
TO_NCHAR('John Smith'),N'500 Oracle Parkway',sysdate);
```

Example 7-2 Selecting from the Customer Table Using the TO_CHAR Function

The following statement converts the values of name from characters in the national character set to characters in the database character set before selecting them according to the LIKE clause:

```
SELECT name FROM customers WHERE TO_CHAR(name) LIKE '%Sm%';
```

You should see the following output:

```
NAME
-----
John Smith
```

Example 7-3 Selecting from the Customer Table Using the TO_DATE Function

Using the N function shows that either NCHAR or CHAR data can be passed as parameters for the TO_DATE function. The data types can mixed because they are converted at run time.

```
DECLARE
  ndatesting NVARCHAR2(20) := N'12-SEP-1975';
  ndstr NVARCHAR2(50);
BEGIN
  SELECT name INTO ndstr FROM customers
  WHERE (birthdate)> TO_DATE(ndatesting, 'DD-MON-YYYY', NLS_DATE_LANGUAGE =
    'AMERICAN');
END;
```

As demonstrated in [Example 7-3](#), SQL NCHAR data can be passed to explicit conversion functions. SQL CHAR and NCHAR data can be mixed together when using multiple string parameters.

7.2.6 Other SQL Functions

Most SQL functions can take arguments of SQL NCHAR data types as well as mixed character data types. The return data type is based on the type of the first argument. If a non-string data type like NUMBER or DATE is passed to these functions, then it is converted to VARCHAR2.

① See Also

Oracle AI Database SQL Language Reference

The following examples use the customer table created in "[SQL Functions for Unicode Data Types](#)".

Example 7-4 INSTR Function

In this example, the string literal 'Sm' is converted to NVARCHAR2 and then scanned by INSTR, to detect the position of the first occurrence of this string in name.

```
SELECT INSTR(name, N'Sm', 1, 1) FROM customers;
```

Example 7-5 CONCAT Function

```
SELECT CONCAT(name,id) FROM customers;
```

id is converted to NVARCHAR2 and then concatenated with name.

Example 7-6 RPAD Function

```
SELECT RPAD(name,100,' ') FROM customers;
```

The following output results:

```
RPAD(NAME,100,' ')
-----
John Smith
```

The space character ' ' is converted to the corresponding character in the NCHAR character set and then padded to the right of name until the total display length reaches 100.

7.2.7 Unicode String Literals

You can input Unicode string literals in SQL and PL/SQL as follows:

- Put a prefix N before a string literal that is enclosed with single quotation marks. This explicitly indicates that the following string literal is an NCHAR string literal. For example, N'r  sum  ' is an NCHAR string literal. For information about limitations of this method, see "[NCHAR String Literal Replacement](#)".
- Use the NCHR(*n*) SQL function, which returns a unit of character code in the national character set, which is AL16UTF16 or UTF8. The result of concatenating several NCHR(*n*) functions is NVARCHAR2 data. In this way, you can bypass the client and server character set conversions and create an NVARCHAR2 string directly. For example, NCHR(32) represents a blank character.

Because `NCHR(n)` is associated with the national character set, portability of the resulting value is limited to applications that run with the same national character set. If this is a concern, then use the `UNISTR` function to remove portability limitations.

- Use the `UNISTR('string')` SQL function. `UNISTR('string')` converts a string to the national character set. To ensure portability and to preserve data, include only ASCII characters and Unicode encoding in the following form: `\xxxx`, where `xxxx` is the hexadecimal value of a character code value in UTF-16 encoding format. For example, `UNISTR('G\0061ry')` represents 'Gary'. The ASCII characters are converted to the database character set and then to the national character set. The Unicode encoding is converted directly to the national character set.

The last two methods can be used to encode any Unicode string literals.

7.2.8 NCHAR String Literal Replacement

This section provides information on how to avoid data loss when performing NCHAR string literal replacement.

Being part of a SQL or PL/SQL statement, the text of any literal, with or without the prefix `N`, is encoded in the same character set as the rest of the statement. On the client side, the statement is in the client character set, which is determined by the client character set defined in `NLS_LANG`, or specified in the `OCIEnvNlsCreate()` call, or predefined as UTF-16 in JDBC. On the server side, the statement is in the database character set.

- When the SQL or PL/SQL statement is transferred from client to the database server, its character set is converted accordingly. It is important to note that if the database character set does not contain all characters used in the text literals, then the data is lost in this conversion. This problem affects NCHAR string literals more than the CHAR text literals. This is because the `N` literals are designed to be independent of the database character set, and should be able to provide any data that the client character set supports.

To avoid data loss in conversion to an incompatible database character set, you can activate the NCHAR literal replacement functionality. The functionality transparently replaces the `N` literals on the client side with an internal format. The database server then decodes this to Unicode when the statement is executed.

- The sections "[Handling SQL NCHAR String Literals in OCI](#)" and "[Using SQL NCHAR String Literals in JDBC](#)" show how to switch on the replacement functionality in OCI and JDBC, respectively. Because many applications, for example, SQL*Plus, use OCI to connect to a database, and they do not control NCHAR literal replacement explicitly, you can set the client environment variable `ORA_NCHAR_LITERAL_REPLACE` to `TRUE` to control the functionality for them. By default, the functionality is switched off to maintain backward compatibility.

7.2.9 Using the UTL_FILE Package with NCHAR Data

The `UTL_FILE` package handles Unicode national character set data of the `NVARCHAR2` data type. NCHAR and NLOB are supported through implicit conversion. The functions and procedures include the following:

- `FOPEN_NCHAR`

This function opens a file in national character set mode for input or output, with the maximum line size specified. Even though the contents of an `NVARCHAR2` buffer may be AL16UTF16 or UTF8 (depending on the national character set of the database), the contents of the file are always read and written in UTF8. See "[Support for the Unicode](#)

[Standard in Oracle Database](#)" for more information. UTL_FILE converts between UTF8 and AL16UTF16 as necessary.

- GET_LINE_NCHAR

This procedure reads text from the open file identified by the file handle and places the text in the output buffer parameter. The file must be opened in national character set mode, and must be encoded in the UTF8 character set. The expected buffer data type is NVARCHAR2. If a variable of another data type, such as NCHAR, NCLOB, or VARCHAR2 is specified, PL/SQL performs standard implicit conversion from NVARCHAR2 after the text is read.

- PUT_NCHAR

This procedure writes the text string stored in the buffer parameter to the open file identified by the file handle. The file must be opened in the national character set mode. The text string will be written in the UTF8 character set. The expected buffer data type is NVARCHAR2. If a variable of another data type is specified, PL/SQL performs implicit conversion to NVARCHAR2 before writing the text.

- PUT_LINE_NCHAR

This procedure is equivalent to PUT_NCHAR, except that the line separator is appended to the written text.

- PUTF_NCHAR

This procedure is a formatted version of a PUT_NCHAR procedure. It accepts a format string with formatting elements \n and %s, and up to five arguments to be substituted for consecutive instances of %s in the format string. The expected data type of the format string and the arguments is NVARCHAR2. If variables of another data type are specified, PL/SQL performs implicit conversion to NVARCHAR2 before formatting the text. Formatted text is written in the UTF8 character set to the file identified by the file handle. The file must be opened in the national character set mode.

The above functions and procedures process text files encoded in the UTF8 character set, that is, in the Unicode CESU-8 encoding. See "[Universal Character Sets](#)" for more information about CESU-8. The functions and procedures convert between UTF8 and the national character set of the database, which can be UTF8 or AL16UTF16, as needed.

① See Also

Oracle AI Database PL/SQL Packages and Types Reference for more information about the UTL_FILE package

7.3 OCI Programming with Unicode

OCI is the lowest-level API for accessing a database, so it offers the best possible performance. When using Unicode with OCI, consider these topics:

- [OCIEnvNlsCreate\(\) Function for Unicode Programming](#)
- [OCI Unicode Code Conversion](#)
- [Setting UTF-8 to the NLS_LANG Character Set in OCI](#)
- [Binding and Defining SQL CHAR Data Types in OCI](#)
- [Binding and Defining SQL NCHAR Data Types in OCI](#)

- [Handling SQL NCHAR String Literals in OCI](#)
- [Binding and Defining CLOB and NCLOB Unicode Data in OCI](#)

① See Also

["OCI Programming in a Global Environment"](#)

7.3.1 OCIEnvNlsCreate() Function for Unicode Programming

The OCIEnvNlsCreate() function is used to specify a SQL CHAR character set and a SQL NCHAR character set when the OCI environment is created. It is an enhanced version of the OCIEnvCreate() function and has extended arguments for two character set IDs. The OCI_UTF16ID UTF-16 character set ID replaces the Unicode mode introduced in Oracle9i release 1 (9.0.1). For example:

```
OCIEnv *envhp;
status = OCIEnvNlsCreate((OCIEnv **)&envhp,
(ub4)0,
(void *)0,
(void (*)(*) ()) 0,
(void (*)(*) ()) 0,
(void(*) ()) 0,
(size_t) 0,
(void **)&0,
(ub2)OCI_UTF16ID, /* Metadata and SQL CHAR character set */
(ub2)OCI_UTF16ID /* SQL NCHAR character set */);
```

The Unicode mode, in which the OCI_UTF16 flag is used with the OCIEnvCreate() function, is deprecated.

When OCI_UTF16ID is specified for both SQL CHAR and SQL NCHAR character sets, all metadata and bound and defined data are encoded in UTF-16. Metadata includes SQL statements, user names, error messages, and column names. Thus, all inherited operations are independent of the NLS_LANG setting, and all metatext data parameters (text*) are assumed to be Unicode text data types (utext*) in UTF-16 encoding.

To prepare the SQL statement when the OCIEnv() function is initialized with the OCI_UTF16ID character set ID, call the OCISmtPrepare() function with a (utext*) string. The following example runs on the Windows platform only. You may need to change wchar_t data types for other platforms.

```
const wchar_t sqlstr[] = L"SELECT * FROM ENAME=:ename";
...
OCISmt* stmthp;
sts = OCIHandleAlloc(envh, (void **)&stmthp, OCI_HTYPE_STMT, 0,
NULL);
status = OCISmtPrepare(stmthp, errhp, (const text*)sqlstr,
wcslen(sqlstr), OCI_NTV_SYNTAX, OCI_DEFAULT);
```

To bind and define data, you do not have to set the OCI_ATTR_CHARSET_ID attribute because the OCIEnv() function has already been initialized with UTF-16 character set IDs. The bind variable names also must be UTF-16 strings.

```
/* Inserting Unicode data */
OCIBindByName(stmthp1, &bnd1p, errhp, (const text*)L":ename",
(sb4)wcslen(L":ename"),
```

```

        (void *) ename, sizeof(ename), SQLT_STR, (void
*)&insname_ind,
        (ub2 *) 0, (ub2 *) 0, (ub4) 0, (ub4 *)0,
OCI_DEFAULT);
OCIAttrSet((void *) bnd1p, (ub4) OCI_HTYPE_BIND, (void *)
&ename_col_len,
        (ub4) 0, (ub4)OCI_ATTR_MAXDATA_SIZE, errhp);
...
/* Retrieving Unicode data */
OCIDefineByPos (stmthp2, &dfn1p, errhp, (ub4)1, (void *)ename,
        (sb4)sizeof(ename), SQLT_STR, (void *)0, (ub2 *)0,
(ub2*)0, (ub4)OCI_DEFAULT);

```

The OCIExecute() function performs the operation.

See Also

["Specifying Character Sets in OCI"](#)

7.3.2 OCI Unicode Code Conversion

Unicode character set conversions take place between an OCI client and the database server if the client and server character sets are different. The conversion occurs on either the client or the server depending on the circumstances, but usually on the client side.

7.3.2.1 Data Integrity

You can lose data during conversion if you call an OCI API inappropriately. If the server and client character sets are different, then you can lose data when the destination character set is a smaller set than the source character set. You can avoid this potential problem if both character sets are Unicode character sets (for example, UTF8 and AL16UTF16).

When you bind or define SQL NCHAR data types, you should set the OCI_ATTR_CHARSET_FORM attribute to SQLCS_NCHAR. Otherwise, you can lose data because the data is converted to the database character set before converting to or from the national character set. This occurs only if the database character set is not Unicode.

7.3.2.2 OCI Performance Implications When Using Unicode

Redundant data conversions can cause performance degradation in your OCI applications. These conversions occur in two cases:

- When you bind or define SQL CHAR data types and set the OCI_ATTR_CHARSET_FORM attribute to SQLCS_NCHAR, data conversions take place from client character set to the national database character set, and from the national character set to the database character set. No data loss is expected, but two conversions happen, even though it requires only one.
- When you bind or define SQL NCHAR data types and do not set OCI_ATTR_CHARSET_FORM, data conversions take place from client character set to the database character set, and from the database character set to the national database character set. In the worst case, data loss can occur if the database character set is smaller than the client's.

To avoid performance problems, you should always set `OCI_ATTR_CHARSET_FORM` correctly, based on the data type of the target columns. If you do not know the target data type, then you should set the `OCI_ATTR_CHARSET_FORM` attribute to `SQLCS_NCHAR` when binding and defining.

The following table contains information about OCI character set conversions.

Table 7-3 OCI Character Set Conversions

Data Types for OCI Client Buffer	OCI_ATTR_CHARSET_FORM	Data Types of the Target Column in the Database	Conversion Between	Comments
utext	SQLCS_IMPLICIT	CHAR, VARCHAR2, CLOB	UTF-16 and database character set in OCI	No unexpected data loss
utext	SQLCS_NCHAR	NCHAR, NVARCHAR2, NCLOB	UTF-16 and national character set in OCI	No unexpected data loss
utext	SQLCS_NCHAR	CHAR, VARCHAR2, CLOB	UTF-16 and national character set in OCI National character set and database character set in database server	No unexpected data loss, but may degrade performance because the conversion goes through the national character set
utext	SQLCS_IMPLICIT	NCHAR, NVARCHAR2, NCLOB	UTF-16 and database character set in OCI Database character set and national character set in database server	Data loss may occur if the database character set is not Unicode
text	SQLCS_IMPLICIT	CHAR, VARCHAR2, CLOB	NLS_LANG character set and database character set in OCI	No unexpected data loss
text	SQLCS_NCHAR	NCHAR, NVARCHAR2, NCLOB	NLS_LANG character set and national character set in OCI	No unexpected data loss
text	SQLCS_NCHAR	CHAR, VARCHAR2, CLOB	NLS_LANG character set and national character set in OCI National character set and database character set in database server	No unexpected data loss, but may degrade performance because the conversion goes through the national character set
text	SQLCS_IMPLICIT	NCHAR, NVARCHAR2, NCLOB	NLS_LANG character set and database character set in OCI Database character set and national character set in database server	Data loss may occur because the conversion goes through the database character set

7.3.2.3 OCI Unicode Data Expansion

Data conversion can result in data expansion, which can cause a buffer to overflow. For binding operations, you must set the `OCI_ATTR_MAXDATA_SIZE` attribute to a large enough size to hold the expanded data on the server. If this is difficult to do, then you must consider changing the table schema. For defining operations, client applications must allocate enough buffer space for the expanded data. The size of the buffer should be the maximum length of the expanded data. You can estimate the maximum buffer length with the following calculation:

1. Get the column data byte size.
2. Multiply it by the maximum number of bytes for each character in the client character set.

This method is the simplest and quickest way, but it may not be accurate and can waste memory. It is applicable to any character set combination. For example, for UTF-16 data binding and defining, the following example calculates the client buffer:

```
ub2 csid = OCI_UTF16ID;
oratext *selstmt = "SELECT ename FROM emp";
counter = 1;
...
OCIStmtPrepare(stmthp, errhp, selstmt, (ub4)strlen((char*)selstmt),
               OCI_NTV_SYNTAX, OCI_DEFAULT);
OCIStmtExecute ( svchp, stmthp, errhp, (ub4)0, (ub4)0,
                 (CONST OCISnapshot*)0, (OCISnapshot*)0,
                 OCI_DESCRIBE_ONLY);
OCIParamGet(stmthp, OCI_HTYPE_STMT, errhp, &myparam, (ub4)counter);
OCIAttrGet((void*)myparam, (ub4)OCI_DTYPE_PARAM, (void*)&col_width,
            (ub4*)0, (ub4)OCI_ATTR_DATA_SIZE, errhp);
...
maxenamelen = (col_width + 1) * sizeof(utext);
cbuf = (utext*)malloc(maxenamelen);
...
OCIDefineByPos(stmthp, &dfnp, errhp, (ub4)1, (void *)cbuf,
               (sb4)maxenamelen, SQLT_STR, (void *)0, (ub2 *)0,
               (ub2*)0, (ub4)OCI_DEFAULT);
OCIAttrSet((void *) dfnp, (ub4) OCI_HTYPE_DEFINE, (void *) &csid,
            (ub4) 0, (ub4)OCI_ATTR_CHARSET_ID, errhp);
OCIStmtFetch(stmthp, errhp, 1, OCI_FETCH_NEXT, OCI_DEFAULT);
...
```

7.3.3 Setting UTF-8 to the NLS_LANG Character Set in OCI

For OCI client applications that support Unicode UTF-8 encoding, use AL32UTF8 to specify the NLS_LANG character set, unless the database character set is UTF8. Use UTF8 if the database character set is UTF8.

Do not set NLS_LANG to AL16UTF16, because AL16UTF16 is the national character set for the server. If you need to use UTF-16, then you should specify the client character set to OCI_UTF16ID, using the OCIAttrSet() function when binding or defining data.

7.3.4 Binding and Defining SQL CHAR Data Types in OCI

To specify a Unicode character set for binding and defining data with SQL CHAR data types, you may need to call the OCIAttrSet() function to set the appropriate character set ID after OCIBind() or OCIDefine() APIs. There are two typical cases:

- Call OCIBind() or OCIDefine() followed by OCIAttrSet() to specify UTF-16 Unicode character set encoding. For example:

```
...
ub2 csid = OCI_UTF16ID;
utext ename[100]; /* enough buffer for ENAME */
...
/* Inserting Unicode data */
OCIBindByName(stmthp1, &bnd1p, errhp, (oratext*)" :ENAME",
              (sb4)strlen((char *)":ENAME"), (void *) ename, sizeof(ename),
              SQLT_STR, (void *)&insname_ind, (ub2 *) 0, (ub2 *) 0, (ub4) 0,
              (ub4 *)0, OCI_DEFAULT);
OCIAttrSet((void *) bnd1p, (ub4) OCI_HTYPE_BIND, (void *) &csid,
```

```

        (ub4) 0, (ub4)OCI_ATTR_CHARSET_ID, errhp);
OCIAttrSet((void *) bnd1p, (ub4) OCI_HTYPE_BIND, (void *) &ename_col_len,
        (ub4) 0, (ub4)OCI_ATTR_MAXDATA_SIZE, errhp);
...
/* Retrieving Unicode data */
OCIDefineByPos (stmthp2, &dfn1p, errhp, (ub4)1, (void *)ename,
        (sb4)sizeof(ename), SQLT_STR, (void *)0, (ub2 *)0,
        (ub2*)0, (ub4)OCI_DEFAULT);
OCIAttrSet((void *) dfn1p, (ub4) OCI_HTYPE_DEFINE, (void *) &csid,
        (ub4) 0, (ub4)OCI_ATTR_CHARSET_ID, errhp);
...

```

If bound buffers are of the utext data type, then you should add a cast (text*) when OCIBind() or OCIDefine() is called. The value of the OCI_ATTR_MAXDATA_SIZE attribute is usually determined by the column size of the server character set because this size is only used to allocate temporary buffer space for conversion on the server when you perform binding operations.

- Call OCIBind() or OCIDefine() with the NLS_LANG character set specified as UTF8 or AL32UTF8.

UTF8 or AL32UTF8 can be set in the NLS_LANG environment variable. You call OCIBind() and OCIDefine() in exactly the same manner as when you are not using Unicode. Set the NLS_LANG environment variable to UTF8 or AL32UTF8 and run the following OCI program:

```

...
oratext ename[100]; /* enough buffer size for ENAME */
...
/* Inserting Unicode data */
OCIBindByName(stmthp1, &bnd1p, errhp, (oratext*)" :ENAME",
        (sb4)strlen((char *)":ENAME"), (void *) ename, sizeof(ename),
        SQLT_STR, (void *)&insname_ind, (ub2 *) 0, (ub2 *) 0,
        (ub4) 0, (ub4 *)0, OCI_DEFAULT);
OCIAttrSet((void *) bnd1p, (ub4) OCI_HTYPE_BIND, (void *) &ename_col_len,
        (ub4) 0, (ub4)OCI_ATTR_MAXDATA_SIZE, errhp);
...
/* Retrieving Unicode data */
OCIDefineByPos (stmthp2, &dfn1p, errhp, (ub4)1, (void *)ename,
        (sb4)sizeof(ename), SQLT_STR, (void *)0, (ub2 *)0, (ub2*)0,
        (ub4)OCI_DEFAULT);
...

```

7.3.5 Binding and Defining SQL NCHAR Data Types in OCI

Oracle recommends that you access SQL NCHAR data types using UTF-16 binding or defining when using OCI. Beginning with Oracle9i, SQL NCHAR data types are Unicode data types with an encoding of either UTF8 or AL16UTF16. To access data in SQL NCHAR data types, set the OCI_ATTR_CHARSET_FORM attribute to SQLCS_NCHAR between binding or defining and execution so that it performs an appropriate data conversion without data loss. The length of data in SQL NCHAR data types is always in the number of Unicode code units.

The following program is a typical example of inserting and fetching data against an NCHAR data column:

```

...
ub2 csid = OCI_UTF16ID;
ub1 cform = SQLCS_NCHAR;
utext ename[100]; /* enough buffer for ENAME */
...
/* Inserting Unicode data */
OCIBindByName(stmthp1, &bnd1p, errhp, (oratext*)" :ENAME",

```



```

        (sb4)strlen((char *)":ENAME"), (void *) ename,
        sizeof(ename), SQLT_STR, (void *)&insname_ind, (ub2 *) 0,
        (ub2 *) 0, (ub4) 0, (ub4 *)0, OCI_DEFAULT);
OCIAttrSet((void *) bnd1p, (ub4) OCI_HTYPE_BIND, (void *) &cform, (ub4) 0,
        (ub4)OCI_ATTR_CHARSET_FORM, errhp);
OCIAttrSet((void *) bnd1p, (ub4) OCI_HTYPE_BIND, (void *) &csid, (ub4) 0,
        (ub4)OCI_ATTR_CHARSET_ID, errhp);
OCIAttrSet((void *) bnd1p, (ub4) OCI_HTYPE_BIND, (void *) &ename_col_len,
        (ub4) 0, (ub4)OCI_ATTR_MAXDATA_SIZE, errhp);
...
/* Retrieving Unicode data */
OCIDefineByPos (stmthp2, &dfn1p, errhp, (ub4)1, (void *)ename,
        (sb4)sizeof(ename), SQLT_STR, (void *)0, (ub2 *)0, (ub2*)0,
        (ub4)OCI_DEFAULT);
OCIAttrSet((void *) dfn1p, (ub4) OCI_HTYPE_DEFINE, (void *) &csid, (ub4) 0,
        (ub4)OCI_ATTR_CHARSET_ID, errhp);
OCIAttrSet((void *) dfn1p, (ub4) OCI_HTYPE_DEFINE, (void *) &cform, (ub4) 0,
        (ub4)OCI_ATTR_CHARSET_FORM, errhp);
...

```

7.3.6 Handling SQL NCHAR String Literals in OCI

By default, the NCHAR literal replacement is not enabled in OCI. You can enable it in OCI by setting the environment variable `ORA_NCHAR_LITERAL_REPLACE` to `TRUE`.

You can also enable literal replacement programmatically in OCI by using the `OCI_NCHAR_LITERAL_REPLACE_ON` and `OCI_NCHAR_LITERAL_REPLACE_OFF` modes in `OCIEnvCreate()` and `OCIEnvNlsCreate()`. For example, `OCIEnvCreate(OCI_NCHAR_LITERAL_REPLACE_ON)` enables NCHAR literal replacement and `OCIEnvCreate(OCI_NCHAR_LITERAL_REPLACE_OFF)` disables it.

As an example, consider the following statement:

```

int main(argc, argv)
{
    OCIEnv *envhp;

    if (OCIEnvCreate((OCIEnv **) &envhp,
        (ub4)OCI_THREADED|OCI_NCHAR_LITERAL_REPLACE_ON,
        (dvoid *)0, (dvoid *)0, (dvoid *)0, (dvoid *)0,
        (dvoid *)0, (dvoid *)0, (dvoid *)0,
        (void *)0, (dvoid *)0, (dvoid *)0,
        (size_t) 0, (dvoid **) 0))
    {
        printf("FAILED: OCIEnvCreate()\n");
        return 1;
    }
    ...
}

```

Note

When NCHAR literal replacement is enabled, `OCIStmtPrepare` and `OCIStmtPrepare2` transform `N'` literals with `U'` literals in the SQL text and store the resulting SQL text in the statement handle. Thus, if an application uses `OCI_ATTR_STATEMENT` to retrieve the SQL text from the OCI statement handle, the SQL text returns `U'` instead of `N'` as specified in the original text.

① See Also

- ["NCHAR String Literal Replacement"](#)
- *Oracle AI Database Administrator's Guide* for information about how to set environment variables

7.3.7 Binding and Defining CLOB and NCLOB Unicode Data in OCI

In order to write (bind) and read (define) UTF-16 data for CLOB or NCLOB columns, the UTF-16 character set ID must be specified as `OCILOBWRITE()` and `OCILOBREAD()`. When you write UTF-16 data into a CLOB column, call `OCILOBWRITE()` as follows:

```
...
ub2 csid = OCI_UTF16ID;
err = OCILOBWRITE (ctx->svchp, ctx->errhp, lobj, &amtp, offset, (void *) buf,
                  (ub4) BUFSIZE, OCI_ONE_PIECE, (void *) 0,
                  (sb4 (*)()) 0, (ub2) csid, (ub1) SQLCS_IMPLICIT);
```

The `amtp` parameter is the data length in number of Unicode code units. The `offset` parameter indicates the offset of data from the beginning of the data column. The `csid` parameter must be set for UTF-16 data.

To read UTF-16 data from CLOB columns, call `OCILOBREAD()` as follows:

```
...
ub2 csid = OCI_UTF16ID;
err = OCILOBREAD(ctx->svchp, ctx->errhp, lobj, &amtp, offset, (void *) buf,
                (ub4)BUFSIZE, (void *) 0, (sb4 (*)()) 0, (ub2)csid,
                (ub1) SQLCS_IMPLICIT);
```

The data length is always represented in the number of Unicode code units. Note one Unicode supplementary character is counted as two code units, because the encoding is UTF-16. After binding or defining a LOB column, you can measure the data length stored in the LOB column using `OCILOBGETLENGTH()`. The returning value is the data length in the number of code units if you bind or define as UTF-16.

```
err = OCILOBGETLENGTH(ctx->svchp, ctx->errhp, lobj, &lenp);
```

If you are using an NCLOB, then you must set `OCI_ATTR_CHARSET_FORM` to `SQLCS_NCHAR`.

7.4 Pro*C/C++ Programming with Unicode

Pro*C/C++ provides the following ways to insert or retrieve Unicode data into or from the database:

- Using the VARCHAR Pro*C/C++ data type or the native C/C++ text data type, a program can access Unicode data stored in SQL CHAR data types of a UTF8 or AL32UTF8 database. Alternatively, a program could use the C/C++ native text type.
- Using the UVARCHAR Pro*C/C++ data type or the native C/C++ utext data type, a program can access Unicode data stored in NCHAR data types of a database.
- Using the NVARCHAR Pro*C/C++ data type, a program can access Unicode data stored in NCHAR data types. The difference between UVARCHAR and NVARCHAR in a Pro*C/C++ program is that the data for the UVARCHAR data type is stored in a utext buffer while the data for the NVARCHAR data type is stored in a text data type.

Pro*C/C++ does not use the Unicode OCI API for SQL text. As a result, embedded SQL text must be encoded in the character set specified in the NLS_LANG environment variable.

This section contains the following topics:

- [Pro*C/C++ Data Conversion in Unicode](#)
- [Using the VARCHAR Data Type in Pro*C/C++](#)
- [Using the NVARCHAR Data Type in Pro*C/C++](#)
- [Using the UVARCHAR Data Type in Pro*C/C++](#)

7.4.1 Pro*C/C++ Data Conversion in Unicode

Data conversion occurs in the OCI layer, but it is the Pro*C/C++ preprocessor that instructs OCI which conversion path should be taken based on the data types used in a Pro*C/C++ program. The following table shows the conversion paths.

Table 7-4 Pro*C/C++ Bind and Define Data Conversion

Pro*C/C++ Data Type	SQL Data Type	Conversion Path
VARCHAR or text	CHAR	NLS_LANG character set to and from the database character set happens in OCI
VARCHAR or text	NCHAR	NLS_LANG character set to and from database character set happens in OCI Database character set to and from national character set happens in database server
NVARCHAR	NCHAR	NLS_LANG character set to and from national character set happens in OCI
NVARCHAR	CHAR	NLS_LANG character set to and from national character set happens in OCI National character set to and from database character set in database server
UVARCHAR or utext	NCHAR	UTF-16 to and from the national character set happens in OCI
UVARCHAR or utext	CHAR	UTF-16 to and from national character set happens in OCI National character set to database character set happens in database server

7.4.2 Using the VARCHAR Data Type in Pro*C/C++

The Pro*C/C++ VARCHAR data type is preprocessed to a struct with a length field and text buffer field. The following example uses the C/C++ text native data type and the VARCHAR Pro*C/C++ data types to bind and define table columns.

```
#include <sqlca.h>
main()
{
    ...
    /* Change to STRING datatype: */
    EXEC ORACLE OPTION (CHAR_MAP=STRING) ;
    text ename[20] ;          /* unsigned short type */
    varchar address[50] ;     /* Pro*C/C++ varchar type */

    EXEC SQL SELECT ename, address INTO :ename, :address FROM emp;
    /* ename is NULL-terminated */
    printf(L"ENAME = %s, ADDRESS = %.*s\n", ename, address.len, address.arr);
    ...
}
```

When you use the VARCHAR data type or native text data type in a Pro*C/C++ program, the preprocessor assumes that the program intends to access columns of SQL CHAR data types

instead of SQL NCHAR data types in the database. The preprocessor generates C/C++ code to reflect this fact by doing a bind or define using the SQLCS_IMPLICIT value for the OCI_ATTR_CHARSET_FORM attribute. As a result, if a bind or define variable is bound to a column of SQL NCHAR data types in the database, then implicit conversion occurs in the database server to convert the data from the database character set to the national database character set and vice versa. During the conversion, data loss occurs when the database character set is a smaller set than the national character set.

7.4.3 Using the NVARCHAR Data Type in Pro*C/C++

The Pro*C/C++ NVARCHAR data type is similar to the Pro*C/C++ VARCHAR data type. It should be used to access SQL NCHAR data types in the database. It tells Pro*C/C++ preprocessor to bind or define a text buffer to the column of SQL NCHAR data types. The preprocessor specifies the SQLCS_NCHAR value for the OCI_ATTR_CHARSET_FORM attribute of the bind or define variable. As a result, no implicit conversion occurs in the database.

If the NVARCHAR buffer is bound against columns of SQL CHAR data types, then the data in the NVARCHAR buffer (encoded in the NLS_LANG character set) is converted to or from the national character set in OCI, and the data is then converted to the database character set in the database server. Data can be lost when the NLS_LANG character set is a larger set than the database character set.

7.4.4 Using the UVARCHAR Data Type in Pro*C/C++

The UVARCHAR data type is preprocessed to a struct with a length field and utext buffer field. The following example code contains two host variables, ename and address. The ename host variable is declared as a utext buffer containing 20 Unicode characters. The address host variable is declared as a uvarchar buffer containing 50 Unicode characters. The len and arr fields are accessible as fields of a struct.

```
#include <sqlca.h>
#include <sqlucs2.h>

main()
{
    ...
    /* Change to STRING datatype */
    EXEC ORACLE OPTION (CHAR_MAP=STRING);
    utext ename[20];          /* unsigned short type */
    uvarchar address[50];     /* Pro*C/C++ uvarchar type */

    EXEC SQL SELECT ename, address INTO :ename, :address FROM emp;

    /* ename is NULL-terminated */
    wprintf(L"ENAME = %s, ADDRESS = %.*s\n", ename, address.len, address.arr);
    ...
}
```

When you use the UVARCHAR data type or native utext data type in Pro*C/C++ programs, the preprocessor assumes that the program intends to access SQL NCHAR data types. The preprocessor generates C/C++ code by binding or defining using the SQLCS_NCHAR value for OCI_ATTR_CHARSET_FORM attribute. As a result, if a bind or define variable is bound to a column of a SQL NCHAR data type, then an implicit conversion of the data from the national character set occurs in the database server. However, there is no data lost in this scenario because the national character set is always a larger set than the database character set.

7.5 JDBC Programming with Unicode

Oracle provides the following JDBC drivers for Java programs to access character data in an Oracle database:

- The JDBC OCI driver
- The JDBC thin driver
- The JDBC server-side internal driver
- The JDBC server-side thin driver

Java programs can insert or retrieve character data to and from columns of SQL CHAR and NCHAR data types. Specifically, JDBC enables Java programs to bind or define Java strings to SQL CHAR and NCHAR data types. Because Java's string data type is UTF-16 encoded, data retrieved from or inserted into the database must be converted from UTF-16 to the database character set or the national character set and vice versa. JDBC also enables you to specify the PL/SQL and SQL statements in Java strings so that any non-ASCII schema object names and string literals can be used.

At database connection time, JDBC sets the server NLS_LANGUAGE and NLS_TERRITORY parameters to correspond to the locale of the Java VM that runs the JDBC driver. This operation ensures that the server and the Java client communicate in the same language. As a result, Oracle error messages returned from the server are in the same language as the client locale.

This section contains the following topics:

- [Binding and Defining Java Strings to SQL CHAR Data Types](#)
- [Binding and Defining Java Strings to SQL NCHAR Data Types](#)
- [Using the SQL NCHAR Data Types Without Changing the Code](#)
- [Using SQL NCHAR String Literals in JDBC](#)
- [Data Conversion in JDBC](#)
- [Using oracle.sql.CHAR in Oracle Object Types](#)
- [Restrictions on Accessing SQL CHAR Data with JDBC](#)

7.5.1 Binding and Defining Java Strings to SQL CHAR Data Types

Oracle JDBC drivers allow you to access SQL CHAR data types in the database using Java string bind or define variables. The following code illustrates how to bind a Java string to a CHAR column.

```
int employee_id = 12345;
String last_name = "Joe";
PreparedStatement pstmt = conn.prepareStatement("INSERT INTO" +
    "employees (last_name, employee_id) VALUES (?, ?)");
pstmt.setString(1, last_name);
pstmt.setInt(2, employee_id);
pstmt.execute();                               /* execute to insert into first row */
employee_id += 1;                             /* next employee number */
last_name = "\uFF2A\uFF4F\uFF45";             /* Unicode characters in name */
pstmt.setString(1, last_name);
pstmt.setInt(2, employee_id);
pstmt.execute();                               /* execute to insert into second row */
```

You can define the target SQL columns by specifying their data types and lengths. When you define a SQL CHAR column with the data type and the length, JDBC uses this information to optimize the performance of fetching SQL CHAR data from the column. The following is an example of defining a SQL CHAR column.

```
OraclePreparedStatement pstmt = (OraclePreparedStatement)
    conn.prepareStatement("SELECT ename, empno from emp");
pstmt.defineColumnType(1, Types.VARCHAR, 3);
pstmt.defineColumnType(2, Types.INTEGER);
ResultSet rest = pstmt.executeQuery();
String name = rset.getString(1);
int id = rset.getInt(2);
```

You must cast PreparedStatement to OraclePreparedStatement to call defineColumnType(). The second parameter of defineColumnType() is the data type of the target SQL column. The third parameter is the length in number of characters.

7.5.2 Binding and Defining Java Strings to SQL NCHAR Data Types

For binding or defining Java string variables to SQL NCHAR data types, Oracle provides an extended PreparedStatement which has the setFormOfUse() method through which you can explicitly specify the target column of a bind variable to be a SQL NCHAR data type. The following code illustrates how to bind a Java string to an NCHAR column.

```
int employee_id = 12345;
String last_name = "Joe"
oracle.jdbc.OraclePreparedStatement pstmt =
    (oracle.jdbc.OraclePreparedStatement)
    conn.prepareStatement("INSERT INTO employees (last_name, employee_id)
        VALUES (?, ?)");
pstmt.setFormOfUse(1, oracle.jdbc.OraclePreparedStatement.FORM_NCHAR);
pstmt.setString(1, last_name);
pstmt.setInt(2, employee_id);
pstmt.execute(); /* execute to insert into first row */
employee_id += 1; /* next employee number */
last_name = "\uFF2A\uFF4F\uFF45"; /* Unicode characters in name */
pstmt.setString(1, last_name);
pstmt.setInt(2, employee_id);
pstmt.execute(); /* execute to insert into second row */
```

You can define the target SQL NCHAR columns by specifying their data types, forms of use, and lengths. JDBC uses this information to optimize the performance of fetching SQL NCHAR data from these columns. The following is an example of defining a SQL NCHAR column.

```
OraclePreparedStatement pstmt = (OraclePreparedStatement)
    conn.prepareStatement("SELECT ename, empno from emp");
pstmt.defineColumnType(1, Types.VARCHAR, 3,
    OraclePreparedStatement.FORM_NCHAR);
pstmt.defineColumnType(2, Types.INTEGER);
ResultSet rest = pstmt.executeQuery();
String name = rset.getString(1);
int id = rset.getInt(2);
```

To define a SQL NCHAR column, you must specify the data type that is equivalent to a SQL CHAR column in the first argument, the length in number of characters in the second argument, and the form of use in the fourth argument of defineColumnType().

You can bind or define a Java string against an NCHAR column without explicitly specifying the form of use argument. This implies the following:

- If you do not specify the argument in the `setString()` method, then JDBC assumes that the bind or define variable is for the SQL CHAR column. As a result, it tries to convert them to the database character set. When the data gets to the database, the database implicitly converts the data in the database character set to the national character set. During this conversion, data can be lost when the database character set is a subset of the national character set. Because the national character set is either UTF8 or AL16UTF16, data loss would happen if the database character set is not UTF8 or AL32UTF8.
- Because implicit conversion from SQL CHAR to SQL NCHAR data types happens in the database, database performance is degraded.

In addition, if you bind or define a Java string for a column of SQL CHAR data types but specify the form of use argument, then performance of the database is degraded. However, data should not be lost because the national character set is always a larger set than the database character set.

7.5.2.1 New JDBC4.0 Methods for NCHAR Data Types

JDBC 11.1 adds support for the new JDBC 4.0 (JDK6) SQL data types NCHAR, NVARCHAR, LONGNVARCHAR, and NLOB. To retrieve a national character value, an application can call one of the following methods:

- `getNString`
- `getNClob`
- `getNCharacterStream`

The `getNClob` method verifies that the retrieved value is indeed an NLOB. Otherwise, these methods are equivalent to corresponding methods without the letter N.

To specify a value for a parameter marker of national character type, an application can call one of the following methods:

- `setNString`
- `setNCharacterStream`
- `setNClob`

These methods are equivalent to corresponding methods without the letter N preceded by a call to `setFormOfUse(..., OraclePreparedStatement.FORM_NCHAR)`.

See Also

Oracle AI Database JDBC Developer's Guide for more information

7.5.3 Using the SQL NCHAR Data Types Without Changing the Code

A Java system property has been introduced in the Oracle JDBC drivers for customers to tell whether the form of use argument should be specified by default in a Java application. This property has the following purposes:

- Existing applications accessing the SQL CHAR data types can be migrated to support the SQL NCHAR data types for worldwide deployment without changing a line of code.
- Applications do not need to call the `setFormOfUse()` method when binding and defining a SQL NCHAR column. The application code can be made neutral and independent of the

data types being used in the back-end database. With this property set, applications can be easily switched from using SQL CHAR or SQL NCHAR.

The Java system property is specified in the command line that invokes the Java application. The syntax of specifying this flag is as follows:

```
java -Doracle.jdbc.defaultNChar=true <application class>
```

With this property specified, the Oracle JDBC drivers assume the presence of the form of use argument for all bind and define operations in the application.

If you have a database schema that consists of both the SQL CHAR and SQL NCHAR columns, then using this flag may have some performance impact when accessing the SQL CHAR columns because of implicit conversion done in the database server.

See Also

["Data Conversion in JDBC"](#) for more information about the performance impact of implicit conversion

7.5.4 Using SQL NCHAR String Literals in JDBC

When using NCHAR string literals in JDBC, there is a potential for data loss because characters are converted to the database character set before processing. See ["NCHAR String Literal Replacement"](#) for more details.

The desired behavior for preserving the NCHAR string literals can be achieved by enabling the property set `oracle.jdbc.convertNcharLiterals`. If the value is true, then this option is enabled; otherwise, it is disabled. The default setting is false. It can be enabled in two ways: a) as a Java system property or b) as a connection property. Once enabled, conversion is performed on all SQL in the VM (system property) or in the connection (connection property). For example, the property can be set as a Java system property as follows:

```
java -Doracle.jdbc.convertNcharLiterals="true" ...
```

Alternatively, you can set this as a connection property as follows:

```
Properties props = new Properties();  
...  
props.setProperty("oracle.jdbc.convertNcharLiterals", "true");  
Connection conn = DriverManager.getConnection(url, props);
```

If you set this as a connection property, it overrides a system property setting.

7.5.5 Data Conversion in JDBC

Because Java strings are always encoded in UTF-16, JDBC drivers transparently convert data from the database character set to UTF-16 or the national character set. The conversion paths taken are different for the JDBC drivers:

- [Data Conversion for the OCI Driver](#)
- [Data Conversion for Thin Drivers](#)
- [Data Conversion for the Server-Side Internal Driver](#)

7.5.5.1 Data Conversion for the OCI Driver

For the OCI driver, the SQL statements are always converted to the database character set by the driver before it is sent to the database for processing. When the database character set is neither US7ASCII nor WE8ISO8859P1, the driver converts the SQL statements to UTF-8 first in Java and then to the database character set in C. Otherwise, it converts the SQL statements directly to the database character set. For Java string bind variables, The following table summarizes the conversion paths taken for different scenarios. For Java string define variables, the same conversion paths, but in the opposite direction, are taken.

Table 7-5 OCI Driver Conversion Path

Form of Use	SQL Data Type	Conversion Path
FORM_CHAR (Default)	CHAR	Conversion between the UTF-16 encoding of a Java string and the database character set happens in the JDBC driver.
FORM_CHAR (Default)	NCHAR	Conversion between the UTF-16 encoding of a Java string and the database character set happens in the JDBC driver. Then, conversion between the database character set and the national character set happens in the database server.
FORM_NCHAR	NCHAR	Conversion between the UTF-16 encoding of a Java string and the national character set happens in the JDBC driver.
FORM_NCHAR	CHAR	Conversion between the UTF-16 encoding of a Java string and the national character set happens in the JDBC driver. Then, conversion between the national character set and the database character set happens in the database server.

7.5.5.2 Data Conversion for Thin Drivers

SQL statements are always converted to either the database character set or to UTF-8 by the driver before they are sent to the database for processing. The driver converts the SQL statement to the database character set when the database character set is one of the following character sets:

- US7ASCII
- WE8ISO8859P1
- WE8DEC
- WE8MSWIN1252

Otherwise, the driver converts the SQL statement to UTF-8 and notifies the database that the statement requires further conversion before being processed. The database, in turn, converts the SQL statement to the database character set. For Java string bind variables, the conversion paths shown in the following table are taken for the thin driver. For Java string define variables, the same conversion paths but in the opposite direction are taken. The four character sets listed earlier are called **selected characters sets** in the table.

Table 7-6 Thin Driver Conversion Path

Form of Use	SQL Data Type	Database Character Set	Conversion Path
FORM_CHAR (Default)	CHAR	One of the selected character sets	Conversion between the UTF-16 encoding of a Java string and the database character set happens in the thin driver.
FORM_CHAR (Default)	NCHAR	One of the selected character sets	Conversion between the UTF-16 encoding of a Java string and the database character set happens in the thin driver. Then, conversion between the database character set and the national character set happens in the database server.
FORM_CHAR (Default)	CHAR	Other than the selected character sets	Conversion between the UTF-16 encoding of a Java string and UTF-8 happens in the thin driver. Then, conversion between UTF-8 and the database character set happens in the database server.
FORM_CHAR (Default)	NCHAR	Other than the selected character sets	Conversion between the UTF-16 encoding of a Java string and UTF-8 happens in the thin driver. Then, conversion from UTF-8 to the database character set and then to the national character set happens in the database server.
FORM_NCHAR	CHAR	Any	Conversion between the UTF-16 encoding of a Java string and the national character set happens in the thin driver. Then, conversion between the national character set and the database character set happens in the database server.
FORM_NCHAR	NCHAR	Any	Conversion between the UTF-16 encoding of a Java string and the national character set happens in the thin driver.

7.5.5.3 Data Conversion for the Server-Side Internal Driver

All data conversion occurs in the database server because the server-side internal driver works inside the database.

7.5.6 Using oracle.sql.CHAR in Oracle Object Types

JDBC drivers support Oracle object types. Oracle objects are always sent from database to client as an object represented in the database character set or national character set. That means the data conversion path in "[Data Conversion in JDBC](#)" does not apply to Oracle object access. Instead, the `oracle.sql.CHAR` class is used for passing SQL CHAR and SQL NCHAR data of an object type from the database to the client.

This section includes the following topics:

- [oracle.sql.CHAR](#)
- [Accessing SQL CHAR and NCHAR Attributes with oracle.sql.CHAR](#)

7.5.6.1 oracle.sql.CHAR

The `oracle.sql.CHAR` class has a special functionality for conversion of character data. The Oracle character set is a key attribute of the `oracle.sql.CHAR` class. The Oracle character set is always passed in when an `oracle.sql.CHAR` object is constructed. Without a known character set, the bytes of data in the `oracle.sql.CHAR` object are meaningless.

The `oracle.sql.CHAR` class provides the following methods for converting character data to strings:

- `getString()`
Converts the sequence of characters represented by the `oracle.sql.CHAR` object to a string, returning a Java string object. If the character set is not recognized, then `getString()` returns a `SQLException`.
- `toString()`
Identical to `getString()`, except that if the character set is not recognized, then `toString()` returns a hexadecimal representation of the `oracle.sql.CHAR` data and does not return a `SQLException`.
- `getStringWithReplacement()`
Identical to `getString()`, except that a default replacement character replaces characters that have no Unicode representation in the character set of this `oracle.sql.CHAR` object. This default character varies among character sets, but it is often a question mark.

You may want to construct an `oracle.sql.CHAR` object yourself (to pass into a prepared statement, for example). When you construct an `oracle.sql.CHAR` object, you must provide character set information to the `oracle.sql.CHAR` object by using an instance of the `oracle.sql.CharacterSet` class. Each instance of the `oracle.sql.CharacterSet` class represents one of the character sets that Oracle supports.

Complete the following tasks to construct an `oracle.sql.CHAR` object:

1. Create a `CharacterSet` instance by calling the static `CharacterSet.make()` method. This method creates the character set class. It requires as input a valid Oracle character set (`OracleId`). For example:

```
int OracleId = CharacterSet.JA16SJIS_CHARSET; // this is character set 832
...
CharacterSet mycharset = CharacterSet.make(OracleId);
```

Each character set that Oracle supports has a unique predefined `OracleId`. The `OracleId` can always be referenced as a character set specified as `Oracle_character_set_name_CHARSET` where `Oracle_character_set_name` is the Oracle character set.

2. Construct an `oracle.sql.CHAR` object. Pass to the constructor a string (or the bytes that represent the string) and the `CharacterSet` object that indicates how to interpret the bytes based on the character set. For example:

```
String mystring = "teststring";
...
oracle.sql.CHAR mychar = new oracle.sql.CHAR(teststring, mycharset);
```

The `oracle.sql.CHAR` class has multiple constructors: they can take a string, a byte array, or an object as input along with the `CharacterSet` object. In the case of a string, the string is converted to the character set indicated by the `CharacterSet` object before being placed into the `oracle.sql.CHAR` object.

The server (database) and the client (or application running on the client) can use different character sets. When you use the methods of this class to transfer data between the server and the client, the JDBC drivers must convert the data between the server character set and the client character set.

7.5.6.2 Accessing SQL CHAR and NCHAR Attributes with `oracle.sql.CHAR`

The following is an example of an object type created using SQL:

```
CREATE TYPE person_type AS OBJECT (
    name VARCHAR2(30), address NVARCHAR2(256), age NUMBER);
CREATE TABLE employees (id NUMBER, person PERSON_TYPE);
```

The Java class corresponding to this object type can be constructed as follows:

```
public class person implement SqlData
{
    oracle.sql.CHAR name;
    oracle.sql.CHAR address;
    oracle.sql.NUMBER age;
    // SqlData interfaces
    getSqlType() {...}
    writeSql(SqlOutput stream) {...}
    readSql(SqlInput stream, String sqltype) {...}
}
```

The `oracle.sql.CHAR` class is used here to map to the NAME attributes of the Oracle object type, which is of VARCHAR2 data type. JDBC populates this class with the byte representation of the VARCHAR2 data in the database and the CharacterSet object corresponding to the database character set. The following code retrieves a person object from the employees table:

```
TypeMap map = ((OracleConnection)conn).getTypeMap();
map.put("PERSON_TYPE", Class.forName("person"));
conn.setTypeMap(map);

.
.
.

ResultSet rs = stmt.executeQuery("SELECT PERSON FROM EMPLOYEES");
rs.next();
person p = (person) rs.getObject(1);
oracle.sql.CHAR sql_name = p.name;
oracle.sql.CHAR sql_address=p.address;
String java_name = sql_name.getString();
String java_address = sql_address.getString();
```

The `getString()` method of the `oracle.sql.CHAR` class converts the byte array from the database character set or national character set to UTF-16 by calling Oracle's Java data conversion classes and returning a Java string. For the `rs.getObject(1)` call to work, the `SqlData` interface has to be implemented in the class `person`, and the `Typemap` map has to be set up to indicate the mapping of the object type `PERSON_TYPE` to the Java class.

7.5.7 Restrictions on Accessing SQL CHAR Data with JDBC

This section contains the following topic:

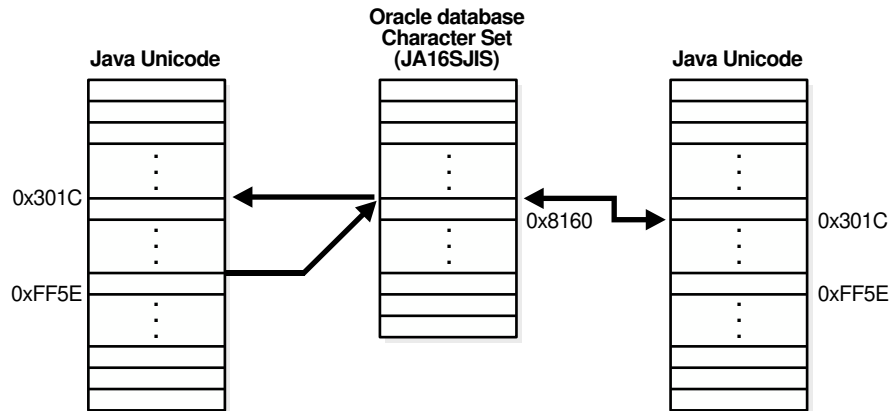
- [Character Integrity Issues in a Multibyte Database Environment](#)

7.5.7.1 Character Integrity Issues in a Multibyte Database Environment

Oracle JDBC drivers perform character set conversions as appropriate when character data is inserted into or retrieved from the database. The drivers convert Unicode characters used by Java clients to Oracle database character set characters, and vice versa. Character data that makes a round trip from the Java Unicode character set to the database character set and back to Java can suffer some loss of information. This happens when multiple Unicode characters are mapped to a single character in the database character set. An example is the Unicode full-width tilde character (0xFF5E) and its mapping to Oracle's JA16SJIS character set. The round-trip conversion for this Unicode character results in the Unicode character 0x301C, which is a wave dash (a character commonly used in Japan to indicate range), not a tilde.

The following figure shows the round-trip conversion of the tilde character.

Figure 7-2 Character Integrity



This issue is not a bug in Oracle's JDBC. It is an unfortunate side effect of the ambiguity in character mapping specifications on different operating systems. Fortunately, this problem affects only a small number of characters in a small number of Oracle character sets such as JA16SJIS, JA16EUC, ZHT16BIG5, and KO16KS5601. The workaround is to avoid making a full round-trip with these characters.

7.6 ODBC and OLE DB Programming with Unicode

You should use the Oracle ODBC driver or Oracle Provider for OLE DB to access the Oracle server when using a Windows platform. This section describes how these drivers support Unicode. It includes the following topics:

- [Unicode-Enabled Drivers in ODBC and OLE DB](#)
- [OCI Dependency in Unicode](#)
- [ODBC and OLE DB Code Conversion in Unicode](#)
- [ODBC Unicode Data Types](#)
- [OLE DB Unicode Data Types](#)
- [ADO Access](#)

7.6.1 Unicode-Enabled Drivers in ODBC and OLE DB

Oracle's ODBC driver and Oracle Provider for OLE DB can handle Unicode data properly without data loss. For example, you can run a Unicode ODBC application containing Japanese data on English Windows if you install Japanese fonts and an input method editor for entering Japanese characters.

Oracle provides ODBC and OLE DB products for Windows platforms only. For UNIX platforms, contact your vendor.

7.6.2 OCI Dependency in Unicode

OCI Unicode binding and defining features are used by the ODBC and OLE DB drivers to handle Unicode data. OCI Unicode data binding and defining features are independent from

NLS_LANG. This means Unicode data is handled properly, irrespective of the NLS_LANG setting on the platform.

See Also

["OCI Programming with Unicode"](#)

7.6.3 ODBC and OLE DB Code Conversion in Unicode

In general, no redundant data conversion occurs unless you specify a different client data type from that of the server. If you bind Unicode buffer SQL_C_WCHAR with a Unicode data column like NCHAR, for example, then ODBC and OLE DB drivers bypass it between the application and OCI layer.

If you do not specify data types before fetching, but call SQLGetData with the client data types instead, then the conversions described in the following table occur.

Table 7-7 ODBC Implicit Binding Code Conversions

Data Types of ODBC Client Buffer	Data Types of the Target Column in the Database	Fetch Conversions	Comments
SQL_C_WCHAR	CHAR, VARCHAR2, CLOB	If the database character set is a subset of the NLS_LANG character set, then the conversions occur in the following order: - Database character set - NLS_LANG - UTF-16 in OCI - UTF-16 in ODBC	No unexpected data loss May degrade performance if database character set is a subset of the NLS_LANG character set
SQL_C_CHAR	CHAR, VARCHAR2, CLOB	If database character set is a subset of NLS_LANG character set: Database character set to NLS_LANG in OCI If database character set is NOT a subset of NLS_LANG character set: Database character set, UTF-16, to NLS_LANG character set in OCI and ODBC	No unexpected data loss May degrade performance if database character set is not a subset of NLS_LANG character set

You must specify the data type for inserting and updating operations.

The data type of the ODBC client buffer is given when you call SQLGetData but not immediately. Hence, SQLFetch does not have the information.

Because the ODBC driver guarantees data integrity, if you perform implicit bindings, then redundant conversion may result in performance degradation. Your choice is the trade-off between performance with explicit binding or usability with implicit binding.

7.6.3.1 OLE DB Code Conversions

Unlike ODBC, OLE DB only enables you to perform implicit bindings for inserting, updating, and fetching data. The conversion algorithm for determining the intermediate character set is the same as the implicit binding cases of ODBC.

Table 7-8 OLE DB Implicit Bindings

Data Types of OLE_DB Client Buffer	Data Types of the Target Column in the Database	In-Binding and Out-Binding Conversions	Comments
DBTYPE_WCHAR	CHAR, VARCHAR2, CLOB	If database character set is a subset of the NLS_LANG character set: Database character set to and from NLS_LANG character set in OCI. NLS_LANG character set to UTF-16 in OLE DB If database character set is NOT a subset of NLS_LANG character set: Database character set to and from UTF-16 in OCI	No unexpected data loss May degrade performance if database character set is a subset of NLS_LANG character set
DBTYPE_CHAR	CHAR, VARCHAR2, CLOB	If database character set is a subset of the NLS_LANG character set: Database character set to and from NLS_LANG in OCI If database character set is not a subset of NLS_LANG character set: Database character set to and from UTF-16 in OCI. UTF-16 to NLS_LANG character set in OLE DB	No unexpected data loss May degrade performance if database character set is not a subset of NLS_LANG character set

7.6.4 ODBC Unicode Data Types

In ODBC Unicode applications, use SQLWCHAR to store Unicode data. All standard Windows Unicode functions can be used for SQLWCHAR data manipulations. For example, wcslen counts the number of characters of SQLWCHAR data:

```
SQLWCHAR sqlStmt[] = L"select ename from emp";
len = wcslen(sqlStmt);
```

Microsoft's ODBC 3.5 specification defines three Unicode data type identifiers for the SQL_C_WCHAR, SQL_C_WVARCHAR, and SQL_WLONGVARCHAR clients; and three Unicode data type identifiers for servers SQL_WCHAR, SQL_WVARCHAR, and SQL_WLONGVARCHAR.

For binding operations, specify data types for both client and server using SQLBindParameter. The following is an example of Unicode binding, where the client buffer Name indicates that Unicode data (SQL_C_WCHAR) is bound to the first bind variable associated with the Unicode column (SQL_WCHAR):

```
SQLBindParameter(StatementHandle, 1, SQL_PARAM_INPUT, SQL_C_WCHAR,
SQL_WCHAR, NameLen, 0, (SQLPOINTER)Name, 0, &Name);
```

The following table represents the data type mappings of the ODBC Unicode data types for the server against SQL NCHAR data types.

Table 7-9 Server ODBC Unicode Data Type Mapping

ODBC Data Type	Oracle Data Type
SQL_WCHAR	NCHAR
SQL_WVARCHAR	NVARCHAR2
SQL_WLONGVARCHAR	NCLOB

According to ODBC specifications, SQL_WCHAR, SQL_WVARCHAR, and SQL_WLONGVARCHAR are treated as Unicode data, and are therefore measured in the number of characters instead of the number of bytes.

7.6.5 OLE DB Unicode Data Types

OLE DB offers the `wchar_t`, `BSTR`, and `OLESTR` data types for a Unicode C client. In practice, `wchar_t` is the most common data type and the others are for specific purposes. The following example assigns a static SQL statement:

```
wchar_t *sqlStmt = OLESTR("SELECT ename FROM emp");
```

The `OLESTR` macro works exactly like an "L" modifier to indicate the Unicode string. If you need to allocate Unicode data buffer dynamically using `OLESTR`, then use the `IMalloc` allocator (for example, `CoTaskMemAlloc`). However, using `OLESTR` is not the normal method for variable length data; use `wchar_t*` instead for generic string types. `BSTR` is similar. It is a string with a length prefix in the memory location preceding the string. Some functions and methods can accept only `BSTR` Unicode data types. Therefore, `BSTR` Unicode string must be manipulated with special functions like `SysAllocString` for allocation and `SysFreeString` for freeing memory.

Unlike ODBC, OLE DB does not allow you to specify the server data type explicitly. When you set the client data type, the OLE DB driver automatically performs data conversion if necessary.

The following table shows the OLE DB data type mapping.

Table 7-10 OLE DB Data Type Mapping

OLE DB Data Type	Oracle Data Type
DBTYPE_WCHAR	NCHAR or NVARCHAR2

If `DBTYPE_BSTR` is specified, then it is assumed to be `DBTYPE_WCHAR` because both are Unicode strings.

7.6.6 ADO Access

ADO is a high-level API to access database with the OLE DB and ODBC drivers. Most database application developers use the ADO interface on Windows because it is easily accessible from Visual Basic, the primary scripting language for Active Server Pages (ASP) for the Internet Information Server (IIS). To OLE DB and ODBC drivers, ADO is simply an OLE DB consumer or ODBC application. ADO assumes that OLE DB and ODBC drivers are Unicode-aware components; hence, it always attempts to manipulate Unicode data.

7.7 XML Programming with Unicode

XML support of Unicode is essential for software development for global markets so that text information can be exchanged in any language. Unicode uniformly supports almost every character and language, which makes it much easier to support multiple languages within XML. To enable Unicode for XML within an Oracle database, the character set of the database must be UTF-8. By enabling Unicode text handling in your application, you acquire a basis for supporting any language. Every XML document is Unicode text and potentially multilingual, unless it is guaranteed that only a known subset of Unicode characters will appear on your documents. Thus Oracle recommends that you enable Unicode for XML. Unicode support comes with Java and many other modern programming environments.

This section includes the following topics:

- [Writing an XML File in Unicode with Java](#)
- [Reading an XML File in Unicode with Java](#)
- [Parsing an XML Stream in Unicode with Java](#)

7.7.1 Writing an XML File in Unicode with Java

A common mistake in reading and writing XML files is using the Reader and Writer classes for character input and output. Using Reader and Writer for XML files should be avoided because it requires character set conversion based on the default character encoding of the run-time environment.

For example, using FileWriter class is not safe because it converts the document to the default character encoding. The output file can suffer from a parsing error or data loss if the document contains characters that are not available in the default character encoding.

UTF-8 is popular for XML documents, but UTF-8 is not usually the default file encoding for Java. Thus using a Java class that assumes the default file encoding can cause problems.

The following example shows how to avoid these problems:

```
import java.io.*;
import oracle.xml.parser.v2.*;

public class I18nSafeXMLFileWritingSample
{
    public static void main(String[] args) throws Exception
    {
        // create a test document
        XMLDocument doc = new XMLDocument();
        doc.setVersion( "1.0" );
        doc.appendChild(doc.createComment( "This is a test empty document." ));
        doc.appendChild(doc.createElement( "root" ));

        // create a file
        File file = new File( "myfile.xml" );

        // create a binary output stream to write to the file just created
        FileOutputStream fos = new FileOutputStream( file );

        // create a Writer that converts Java character stream to UTF-8 stream
        OutputStreamWriter osw = new OutputStreamWriter( fos, "UTF8" );

        // buffering for efficiency
```



```

        Writer w = new BufferedWriter( osw );

        // create a PrintWriter to adapt to the printing method
        PrintWriter out = new PrintWriter( w );

        // print the document to the file through the connected objects
        doc.print( out );
    }
}

```

7.7.2 Reading an XML File in Unicode with Java

Do not read XML files as text input. When reading an XML document stored in a file system, use the parser to automatically detect the character encoding of the document. Avoid using a Reader class or specifying a character encoding on the input stream. Given a binary input stream with no external encoding information, the parser automatically figures out the character encoding based on the byte order mark and encoding declaration of the XML document. Any well-formed document in any supported encoding can be successfully parsed using the following sample code:

```

import java.io.*;
import oracle.xml.parser.v2.*;

public class I18nSafeXMLFileReadingSample
{
    public static void main(String[] args) throws Exception
    {
        // create an instance of the xml file
        File file = new File( "myfile.xml" );

        // create a binary input stream
        FileInputStream fis = new FileInputStream( file );

        // buffering for efficiency
        BufferedInputStream in = new BufferedInputStream( fis );

        // get an instance of the parser
        DOMParser parser = new DOMParser();

        // parse the xml file
        parser.parse( in );
    }
}

```

7.7.3 Parsing an XML Stream in Unicode with Java

When the source of an XML document is not a file system, the encoding information is usually available before reading the document. For example, if the input document is provided in the form of a Java character stream or Reader, its encoding is evident and no detection should take place. The parser can begin parsing a Reader in Unicode without regard to the character encoding.

The following is an example of parsing a document with external encoding information:

```

import java.io.*;
import java.net.*;
import org.xml.sax.*;
import oracle.xml.parser.v2.*;

public class I18nSafeXMLStreamReadingSample

```

```
{
    public static void main(String[] args) throws Exception
    {
        // create an instance of the xml file
        URL url = new URL( "http://myhost/mydocument.xml" );

        // create a connection to the xml document
        URLConnection conn = url.openConnection();

        // get an input stream
        InputStream is = conn.getInputStream();

        // buffering for efficiency
        BufferedInputStream bis = new BufferedInputStream( is );

        /* figure out the character encoding here */
        /* a typical source of encoding information is the content-type header */
        /* we assume it is found to be utf-8 in this example */
        String charset = "utf-8";

        // create an InputSource for UTF-8 stream
        InputSource in = new InputSource( bis );
        in.setEncoding( charset );

        // get an instance of the parser
        DOMParser parser = new DOMParser();

        // parse the xml stream
        parser.parse( in );
    }
}
```

8

Oracle Globalization Development Kit

This chapter includes the following sections:

- [Overview of the Oracle Globalization Development Kit](#)
- [Designing a Global Internet Application](#)
- [Developing a Global Internet Application](#)
- [Getting Started with the Globalization Development Kit](#)
- [GDK Quick Start](#)
- [GDK Application Framework for J2EE](#)
- [GDK Java API](#)
- [The GDK Application Configuration File](#)
- [GDK for Java Supplied Packages and Classes](#)
- [GDK for PL/SQL Supplied Packages](#)
- [GDK Error Messages](#)

8.1 Overview of the Oracle Globalization Development Kit

Designing and developing a globalized application can be a daunting task even for the most experienced developers. This is usually caused by lack of knowledge and the complexity of globalization concepts and APIs. Application developers who write applications using Oracle Database need to understand the Globalization Support architecture of the database, including the properties of the different character sets, territories, languages and linguistic sort definitions. They also need to understand the globalization functionality of their middle-tier programming environment, and find out how it can interact and synchronize with the locale model of the database. Finally, to develop a globalized Internet application, they need to design and write code that is capable of simultaneously supporting multiple clients running on different operating systems, with different character sets and locale requirements.

Oracle Globalization Development Kit (GDK) simplifies the development process and reduces the cost of developing Internet applications that will be used to support a global environment. The GDK includes comprehensive programming APIs for both Java and PL/SQL, code samples, and documentation that address many of the design, development, and deployment issues encountered while creating global applications.

The GDK mainly consists of two parts: GDK for Java and GDK for PL/SQL. GDK for Java provides globalization support to Java applications. GDK for PL/SQL provides globalization support to the PL/SQL programming environment. The features offered in GDK for Java and GDK for PL/SQL are not identical.

8.2 Designing a Global Internet Application

There are two architectural models for deploying a global Web site or a global Internet application, depending on your globalization and business requirements. Which model to

deploy affects how the Internet application is developed and how the application server is configured in the middle-tier. The two models are:

- Multiple instances of monolingual Internet applications

Internet applications that support only one locale in a single binary are classified as monolingual applications. A locale refers to a national language and the region in which the language is spoken. For example, the primary language of the United States and Great Britain is English. However, the two territories have different currencies and different conventions for date formats. Therefore, the United States and Great Britain are considered to be two different locales.

This level of globalization support is suitable for customers who want to support one locale for each instance of the application. Users need to have different entry points to access the applications for different locales. This model is manageable only if the number of supported locales is small.

- Single instance of a multilingual application

Internet applications that support multiple locales simultaneously in a single binary are classified as multilingual applications. This level of globalization support is suitable for customers who want to support several locales in an Internet application simultaneously. Users of different locale preferences use the same entry point to access the application.

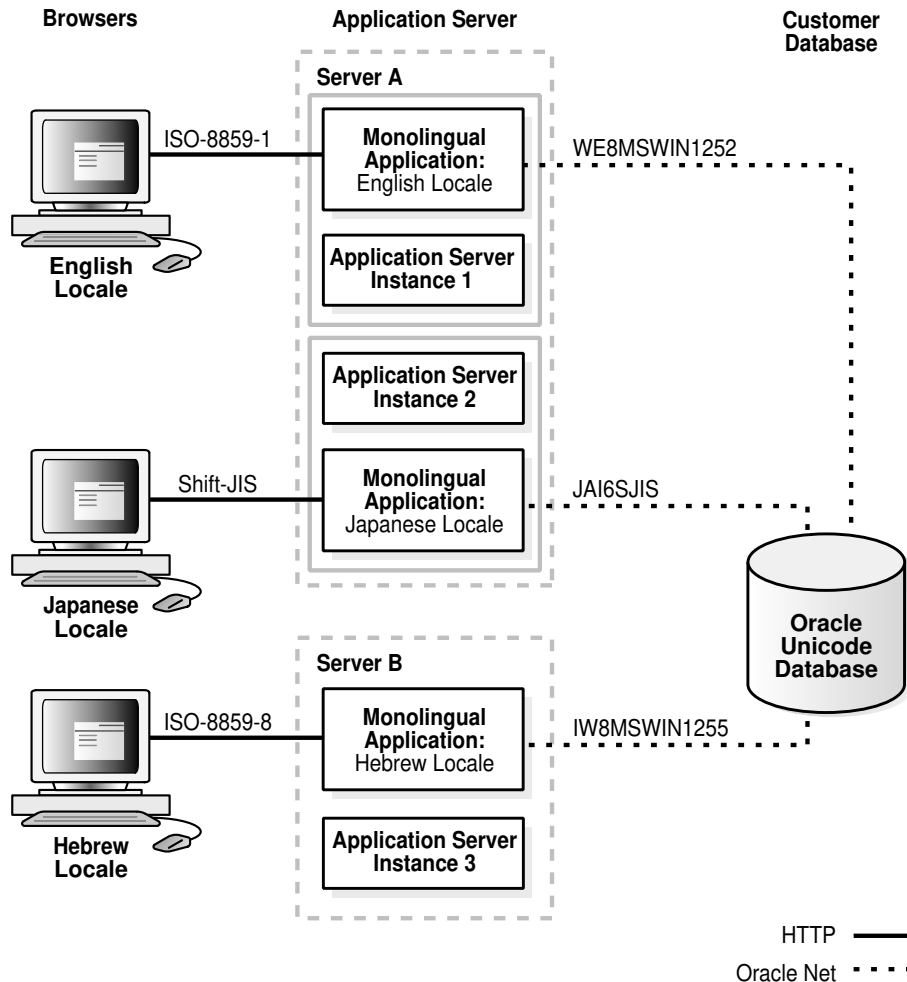
Developing an application using the monolingual model is very different from developing an application using the multilingual model. The Globalization Development Kit consists of libraries, which can assist in the development of global applications using either architectural model.

The rest of this section includes the following topics:

- [Deploying a Monolingual Internet Application](#)
- [Deploying a Multilingual Internet Application](#)

8.2.1 Deploying a Monolingual Internet Application

Deploying a global Internet application with multiple instances of monolingual Internet applications is shown in the following figure.

Figure 8-1 Monolingual Internet Application Architecture

Each application server is configured for the locale that it serves. This deployment model assumes that one instance of an Internet application runs in the same locale as the application in the middle tier.

The Internet applications access a back-end database in the native encoding used for the locale. The following are advantages of deploying monolingual Internet applications:

- The support of the individual locales is separated into different servers so that multiple locales can be supported independently in different locations and that the workload can be distributed accordingly. For example, customers may want to support Western European locales first and then support Asian locales such as Japanese (Japan) later.
- The complexity required to support multiple locales simultaneously is avoided. The amount of code to write is significantly less for a monolingual Internet application than for a multilingual Internet application.

The following are disadvantages of deploying monolingual Internet applications:

- Extra effort is required to maintain and manage multiple servers for different locales. Different configurations are required for different application servers.

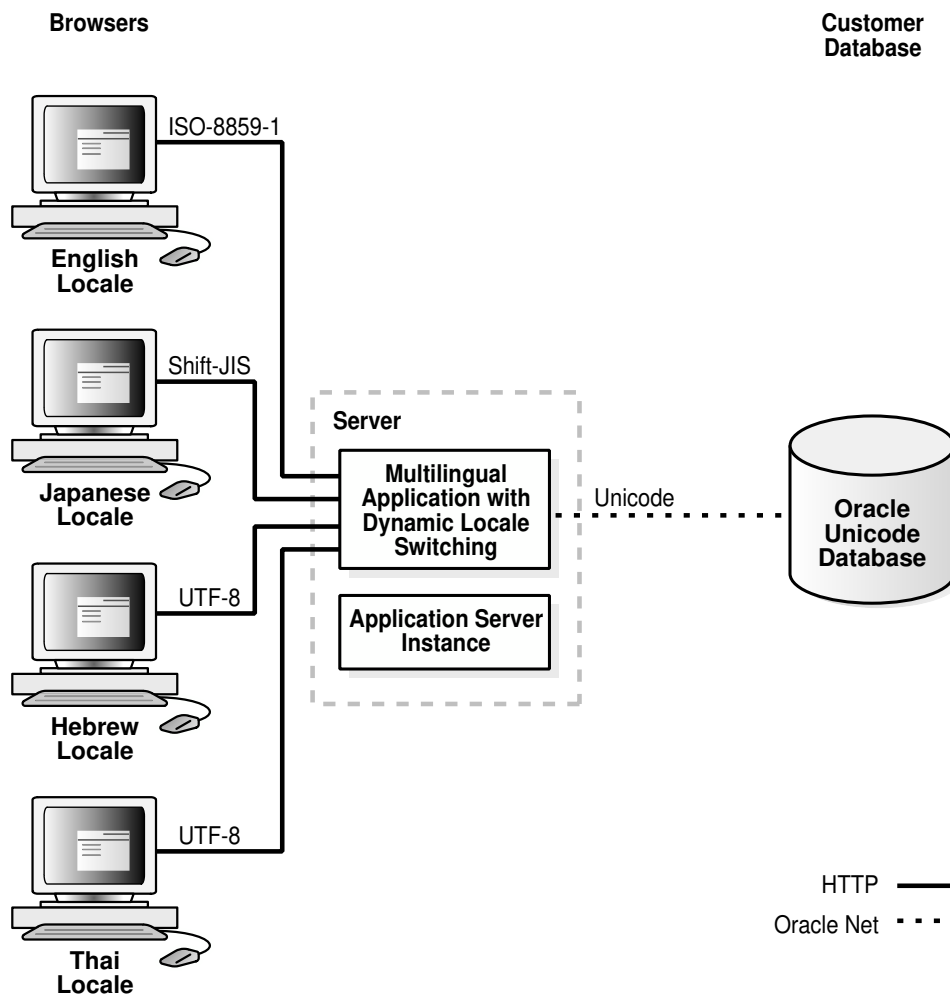
- The minimum number of application servers required depends on the number of locales the application supports, regardless of whether the site traffic will reach the capacity provided by the application servers.
- Load balancing for application servers is limited to the group of application servers for the same locale.
- More QA resources, both human and machine, are required for multiple configurations of application servers. Internet applications running on different locales must be certified on the corresponding application server configuration.
- It is not designed to support multilingual content. For example, a web page containing Japanese and Arabic data cannot be easily supported in this model.

As more and more locales are supported, the disadvantages quickly outweigh the advantages. With the limitation and the maintenance overhead of the monolingual deployment model, this deployment architecture is suitable for applications that support only one or two locales.

8.2.2 Deploying a Multilingual Internet Application

Multilingual Internet applications are deployed to the application servers with a single application server configuration that works for all locales. The following figure shows the architecture of a multilingual Internet application.

Figure 8-2 Multilingual Internet Application Architecture



To support multiple locales in a single application instance, the application may need to do the following:

- Dynamically detect the locale of the users and adapt to the locale by constructing HTML pages in the language and cultural conventions of the locale
- Process character data in Unicode so that data in any language can be supported. Character data can be entered by users or retrieved from back-end databases.
- Dynamically determine the HTML page encoding (or character set) to be used for HTML pages and convert content from Unicode to the page encoding and the reverse.

The following are major advantages of deploying multilingual Internet application:

- Using a single application server configuration for all application servers simplifies the deployment configuration and hence reduces the cost of maintenance.
- Performance tuning and capacity planning do not depend on the number of locales supported by the Web site.
- Introducing additional locales is relatively easy. No extra machines are necessary for the new locales.
- Testing the application across different locales can be done in a single testing environment.
- This model can support multilingual content within the same instance of the application. For example, a web page containing Japanese, Chinese, English and Arabic data can be easily supported in this model.

The disadvantage of deploying multilingual Internet applications is that it requires extra coding during application development to handle dynamic locale detection and Unicode, which is costly when only one or two languages need to be supported.

Deploying multilingual Internet applications is more appropriate than deploying monolingual applications when Web sites support multiple locales.

8.3 Developing a Global Internet Application

Building an Internet application that supports different locales requires good development practices.

For multilingual Internet applications, the application itself must be aware of the user's locale and be able to present locale-appropriate content to the user. Clients must be able to communicate with the application server regardless of the client's locale. The application server then communicates with the database server, exchanging data while maintaining the preferences of the different locales and character set settings. One of the main considerations when developing a multilingual Internet application is to be able to dynamically detect, cache, and provide the appropriate contents according to the user's preferred locale.

For monolingual Internet applications, the locale of the user is always fixed and usually follows the default locale of the run-time environment. Hence, the locale configuration is much simpler.

The following sections describe some of the most common issues that developers encounter when building a global Internet application:

- [Locale Determination](#)
- [Locale Awareness](#)
- [Localizing the Content](#)

8.3.1 Locale Determination

To be locale-aware or locale-sensitive, Internet applications must be able to determine the preferred locale of the user.

Monolingual applications always serve users with the same locale, and that locale should be equivalent to the default run-time locale of the corresponding programming environment.

Multilingual applications can determine a user locale dynamically in three ways. Each method has advantages and disadvantages, but they can be used together in the applications to complement each other. The user locale can be determined in the following ways:

- Based on the user profile information from a LDAP directory server such as the Oracle Internet Directory or other user profile tables stored inside the database

The schema for the user profile should include preferred locale attribute to indicate the locale of a user. This way of determining a locale user does not work if a user has not been logged on before.

- Based on the default locale of the browser

Get the default ISO locale setting from a browser. The default ISO locale of the browser is sent through the Accept-Language HTTP header in every HTTP request. If the Accept-Language header is NULL, then the desired locale should default to English. The drawback of this approach is that the Accept-Language header may not be a reliable source of information for the locale of a user.

- Based on user selection

Allow users to select a locale from a list box or from a menu, and switch the application locale to the one selected.

The Globalization Development Kit provides an application framework that enables you to use these locale determination methods declaratively.

① See Also

["Getting Started with the Globalization Development Kit"](#)

8.3.2 Locale Awareness

To be locale-aware or locale-sensitive, Internet applications need to determine the locale of a user. After the locale of a user is determined, applications should:

- Construct HTML content in the language of the locale
- Use the cultural conventions implied by the locale

Locale-sensitive functions, such as date, time, and monetary formatting, are built into various programming environments such as Java and PL/SQL. Applications may use them to format the HTML pages according to the cultural conventions of the locale of a user. A locale is represented differently in different programming environments. For example, the French (Canada) locale is represented in different environments as follows:

- In the ISO standard, it is represented by fr-CA where fr is the language code defined in the ISO 639 standard and CA is the country code defined in the ISO 3166 standard.

- In Java, it is represented as a Java locale object constructed with `fr`, the ISO language code for French, as the language and `CA`, the ISO country code for Canada, as the country. The Java locale name is `fr_CA`.
- In PL/SQL and SQL, it is represented mainly by the `NLS_LANGUAGE` and `NLS_TERRITORY` session parameters where the value of the `NLS_LANGUAGE` parameter is equal to `CANADIAN FRENCH` and the value of the `NLS_TERRITORY` parameter is equal to `CANADA`.

If you write applications for more than one programming environment, then locales must be synchronized between environments. For example, Java applications that call PL/SQL procedures should map the Java locales to the corresponding `NLS_LANGUAGE` and `NLS_TERRITORY` values and change the parameter values to match the user's locale before calling the PL/SQL procedures.

The Globalization Development Kit for Java provides a set of Java classes to ensure consistency on locale-sensitive behaviors with Oracle databases.

8.3.3 Localizing the Content

For the application to support a multilingual environment, it must be able to present the content in the preferred language and in the locale convention of the user. Hard-coded user interface text must first be externalized from the application, together with any image files, so that they can be translated into the different languages supported by the application. The translation files then must be staged in separate directories, and the application must be able to locate the relevant content according to the user locale setting. Special application handling may also be required to support a fallback mechanism, so that if the user-preferred locale is not available, then the next most suitable content is presented. For example, if Canadian French content is not available, then it may be suitable for the application to switch to the French files instead.

8.4 Getting Started with the Globalization Development Kit

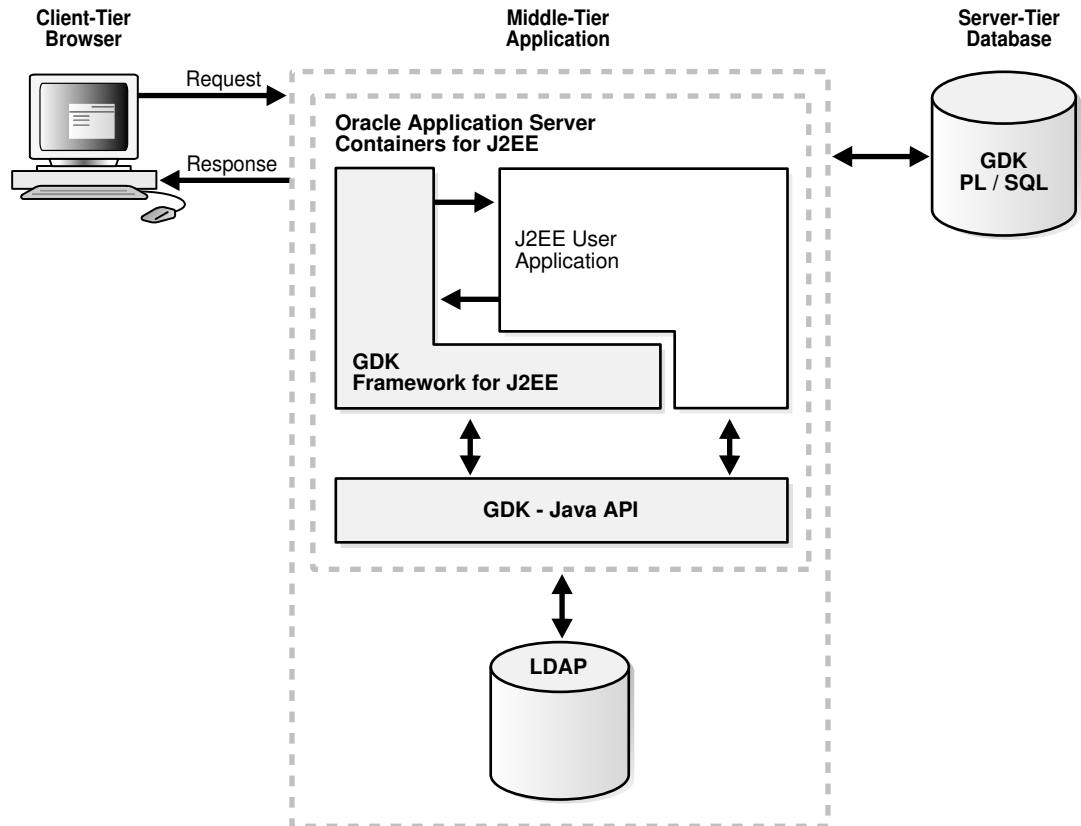
The Globalization Development Kit (GDK) for Java provides a J2EE application framework and Java APIs to develop globalized Internet applications using the best globalization practices and features designed by Oracle. It reduces the complexities and simplifies the code that Oracle developers require to develop globalized Java applications.

GDK for Java complements the existing globalization features in J2EE. Although the J2EE platform already provides a strong foundation for building globalized applications, its globalization functionalities and behaviors can be quite different from Oracle's functionalities. GDK for Java provides synchronization of locale-sensitive behaviors between the middle-tier Java application and the database server.

GDK for PL/SQL contains a suite of PL/SQL packages that provide additional globalization functionalities for applications written in PL/SQL.

The following figure shows major components of the GDK and how they are related to each other. User applications run on the J2EE container of Oracle Application Server in the middle tier. GDK provides the application framework that the J2EE application uses to simplify coding to support globalization. Both the framework and the application call the GDK Java API to perform locale-sensitive tasks. GDK for PL/SQL offers PL/SQL packages that help to resolve globalization issues specific to the PL/SQL environment.

Figure 8-3 GDK Components



The functionalities offered by GDK for Java can be divided into two categories:

- The GDK application framework for J2EE provides the globalization framework for building J2EE-based Internet application. The framework encapsulates the complexity of globalization programming, such as determining user locale, maintaining locale persistency, and processing locale information. It consists of a set of Java classes through which applications can gain access to the framework. These associated Java classes enable applications to code against the framework so that globalization behaviors can be extended declaratively.
- The GDK Java API offers development support in Java applications and provides consistent globalization operations as provided in Oracle database servers. The API is accessible and is independent of the GDK framework so that standalone Java applications and J2EE applications that are not based on the GDK framework are able to access the individual features offered by the Java API. The features provided in the Java API include data and number formatting, sorting, and handling character sets in the same way as the Oracle Database.

Note

The GDK Java API is supported with JDK versions 1.6 and later.

GDK for Java is contained in nine .jar files, all in the form of ora118n*.jar. These files are shipped with the Oracle Database, in the \$ORACLE_HOME/jlib directory. If the application using the GDK is not hosted on the same machine as the database, then the GDK files must be

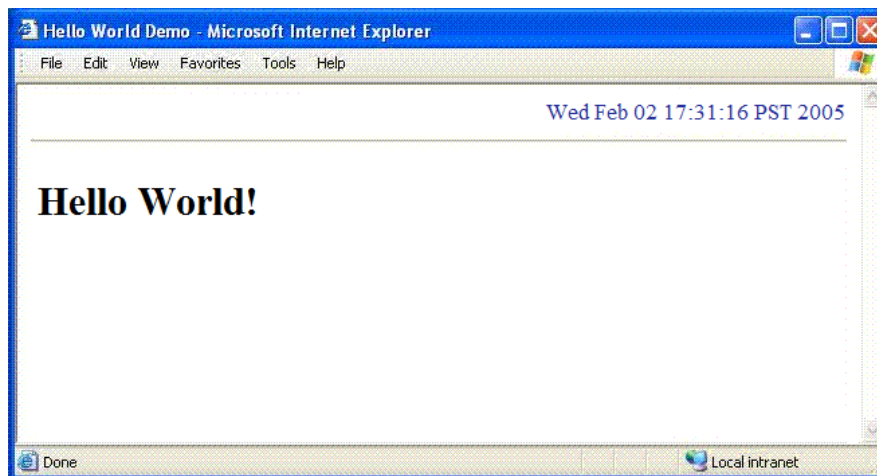
copied to the application server and included into the CLASSPATH to run your application. You do not need to install the Oracle Database into your application server to be able to run the GDK inside your Java application. GDK is a pure Java library that runs on every platform. The Oracle client parameters NLS_LANG and ORACLE_HOME are not required.

8.5 GDK Quick Start

This section explains how to modify a monolingual application to be a global, multilingual application using GDK. The subsequent sections in this chapter provide detailed information on using GDK.

The following figure shows a screenshot from a monolingual Web application.

Figure 8-4 Original HelloWorld Web Page



The initial, non-GDK HelloWorld Web application simply prints a "Hello World!" message, along with the current date and time in the top right hand corner of the page. [Example 8-1](#) shows the original HelloWorld JSP source code for the preceding image.

[Example 8-2](#) shows the corresponding Web application descriptor file for the HelloWorld message.

Example 8-1 HelloWorld JSP Page Code

```
<%@ page contentType="text/html; charset=windows-1252"%>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=windows-1252">
    <title>Hello World Demo</title>
  </head>
  <body>
    <div style="color: blue;" align="right">
      <%= new java.util.Date(System.currentTimeMillis()) %>
    </div>
    <hr/>
    <h1>Hello World!</h1>
  </body>
</html>
```

Example 8-2 HelloWorld web.xml Code

```
<?xml version = '1.0' encoding = 'windows-1252'?>
<!DOCTYPE web-app PUBLIC "-//Sun Microsystems, Inc.//DTD Web Application 2.3//EN"
"http://java.sun.com/dtd/web-app_2_3.dtd">
<web-app>
  <description>web.xml file for the monolingual Hello World</description>
  <session-config>
    <session-timeout>35</session-timeout>
  </session-config>
  <mime-mapping>
    <extension>html</extension>
    <mime-type>text/html</mime-type>
  </mime-mapping>
  <mime-mapping>
    <extension>txt</extension>
    <mime-type>text/plain</mime-type>
  </mime-mapping>
</web-app>
```

The HelloWorld JSP code in [Example 8-1](#) is only for English-speaking users. Some of the problems with this code are as follows:

- There is no locale determination based on user preference or browser setting.
- The title and the heading are included in the code.
- The date and time value is not localized based on any locale preference.
- The character encoding included in the code is for Latin-1.

The GDK framework can be integrated into the HelloWorld code to make it a global, multilingual application. The preceding code can be modified to include the following features:

- Automatic locale negotiation to detect the user's browser locale and serve the client with localized HTML pages. The supported application locales are configured in the GDK configuration file.
- Locale selection list to map the supported application locales. The list can have application locale display names which are the name of the country representing the locale. The list will be included on the Web page so users can select a different locale.
- GDK framework and API for globalization support for the HelloWorld JSP. This involves selecting display strings in a locale-sensitive manner and formatting the date and time value.

8.5.1 Modifying the HelloWorld Application

This section explains how to modify the HelloWorld application to support globalization. The application will be modified to support three locales, Simplified Chinese (zh-CN), Swiss German (de-CH), and American English (en-US). The following rules will be used for the languages:

- If the client locale supports one of these languages, then that language will be used for the application.
- If the client locale does not support one of these languages, then American English will be used for the application.

In addition, the user will be able to change the language by selecting a supported locales from the locale selection list. The following tasks describe how to modify the application:

- [Task 1: Enable the Hello World Application to use the GDK Framework](#)

- [Task 2: Configure the GDK Framework for Hello World](#)
- [Task 3: Enable the JSP or Java Servlet](#)
- [Task 4: Create the Locale Selection List](#)
- [Task 5: Build the Application](#)

Task 1: Enable the Hello World Application to use the GDK Framework

In this task, the GDK filter and a listener are configured in the Web application deployment descriptor file, `web.xml`. This allows the GDK framework to be used with the HelloWorld application. The following example shows the GDK-enabled `web.xml` file.

```
<?xml version = '1.0' encoding = 'windows-1252'?>
<!DOCTYPE web-app PUBLIC "-//Sun Microsystems, Inc.//DTD Web Application 2.3//EN"
"http://java.sun.com/dtd/web-app_2_3.dtd">
<web-app>
  <description>web.xml file for Hello World</description>
  <!-- Enable the application to use the GDK Application Framework.-->
  <filter>
    <filter-name>GDKFilter</filter-name>
    <filter-class>oracle.i18n.servlet.filter.ServletFilter</filter-class>
  </filter>
  <filter-mapping>
    <filter-name>GDKFilter</filter-name>
    <url-pattern>*.jsp</url-pattern>
  </filter-mapping>

  <listener>
    <listener-class>oracle.i18n.servlet.listener.ContextListener</listener-class>
  </listener>

  <session-config>
    <session-timeout>35</session-timeout>
  </session-config>
  <mime-mapping>
    <extension>html</extension>
    <mime-type>text/html</mime-type>
  </mime-mapping>
  <mime-mapping>
    <extension>txt</extension>
    <mime-type>text/plain</mime-type>
  </mime-mapping>
</web-app>
```

The following tags were added to the file:

- `<filter>`
The filter name is `GDKFilter`, and the filter class is `oracle.i18n.servlet.filter.ServletFilter`.
- `<filter-mapping>`
The `GDKFilter` is specified in the tag, as well as the URL pattern.
- `<listener>`
The listener class is `oracle.i18n.servlet.listener.ContextListener`. The default GDK listener is configured to instantiate `GDK ApplicationContext`, which controls application scope operations for the framework.

Task 2: Configure the GDK Framework for Hello World

The GDK application framework is configured with the application configuration file `gdkapp.xml`. The configuration file is located in the same directory as the `web.xml` file. The following example shows the `gdkapp.xml` file.

```

<?xml version="1.0" encoding="UTF-8"?>
<gdkapp xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="gdkapp.xsd">

  <!-- The Hello World GDK Configuration -->
  <page-charset default="yes">UTF-8</page-charset>

  <!-- The supported application locales for the Hello World Application -->

  <application-locals>
    <locale>de-CH</locale>
    <locale default="yes">en-US</locale>
    <locale>zh-CN</locale>
  </application-locals>

  <locale-determine-rule>
    <locale-source>oracle.i18n.servlet.localesource.UserInput</locale-source>
    <locale-source>oracle.i18n.servlet.localesource.HttpAcceptLanguage
    </locale-source>
  </locale-determine-rule>

  <message-bundles>
    <resource-bundle name="default">com.oracle.demo.Messages</resource-bundle>
  </message-bundles>
</gdkapp>

```

The file must be configured for J2EE applications. The following tags are used in the file:

- `<page-charset>`
The page encoding tag specifies the character set used for HTTP requests and responses. The UTF-8 encoding is used as the default because many languages can be represented by this encoding.
- `<application-locals>`
Configuring the application locales in the `gdkapp.xml` file makes a central place to define locales. This makes it easier to add and remove locales without changing source code. The locale list can be retrieved using the GDK API call `ApplicationContext.getSupportedLocales`.
- `<locale-determine-rule>`
The language of the initial page is determined by the language setting of the browser. The user can override this language by choosing from the list. The `locale-determine-rule` is used by GDK to first try the Accept-Language HTTP header as the source of the locale. If the user selects a locale from the list, then the JSP posts a locale parameter value containing the selected locale. The GDK then sends a response with the contents in the selected language.
- `<message-bundles>`
The message resource bundles allow an application access to localized static content that may be displayed on a Web page. The GDK framework configuration file allows an application to define a default resource bundle for translated text for various languages. In the HelloWorld example, the localized string messages are stored in the Java

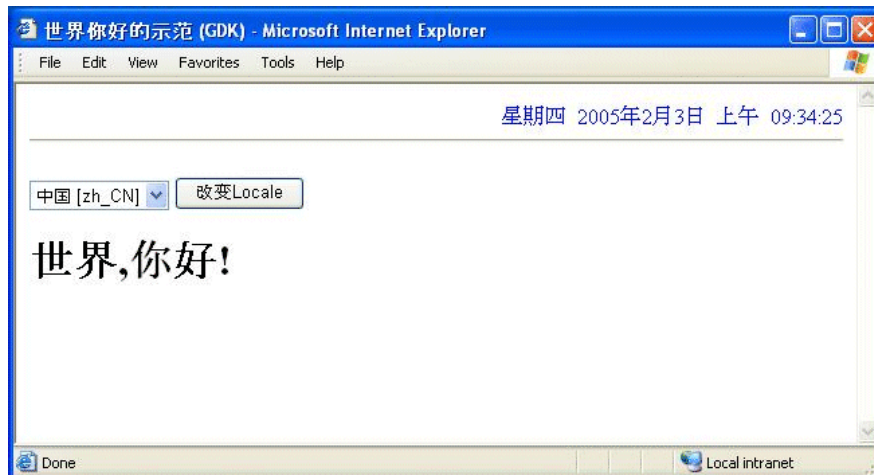
ListResourceBundle bundle named Messages. The Messages bundle consists of base resources for the application which are in the default locale. Two more resource bundles provide the Chinese and German translations. These resource bundles are named Messages_zh_CN.java and Messages_de.java respectively. The HelloWorld application will select the right translation for "Hello World!" from the resource bundle based on the locale determined by the GDK framework. The <message-bundles> tag is used to configure the resource bundles that the application will use.

Task 3: Enable the JSP or Java Servlet

JSPs and Java servlets must be enabled to use the GDK API. The following example shows a JSP that has been modified to enable to use the GDK API and services. This JSP can accommodate any language and locale.

```
. . .
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title><%= localizer.getMessage("helloWorldTitle") %></title>
  </head>
  <body>
    <div style="color: blue;" align="right">
      <% Date currDate= new Date(System.currentTimeMillis()); %>
      <%=localizer.formatDateTime(currDate, OraDateFormat.LONG)%>
    </div>
    <hr/>
    <div align="left">
      <form>
        <select name="locale" size="1">
          <%= getCountryDropDown(request)%>
        </select>
        <input type="submit" value="<%= localizer.getMessage("changeLocale") %>">
      </input>
    </form>
    </div>
    <h1><%= localizer.getMessage("helloWorld") %></h1>
  </body>
</html>
```

The following figure shows the HelloWorld application that has been configured with the zh-CN locale as the primary locale for the browser preference. The HelloWorld string and page title are displayed in Simplified Chinese. In addition, the date is formatted in the zh-CN locale convention. This example allows the user to override the locale from the locale selection list.

Figure 8-5 HelloWorld Localized for the zh-CN Locale

When the locale changes or is initialized using the HTTP Request Accept-Language header or the locale selection list, the GUI behaves appropriately for that locale. This means the date and time value in the upper right corner is localized properly. In addition, the strings are localized and displayed on the HelloWorld page.

The GDK Java Localizer class provides capabilities to localize the contents of a Web page based on the automatic detection of the locale by the GDK framework.

The following code retrieves an instance of the localizer based on the current `HttpServletRequest` object. In addition, several imports are declared for use of the GDK API within the JSP page. The localizer retrieves localized strings in a locale-sensitive manner with fallback behavior, and formats the date and time.

```
<%@page contentType="text/html; charset=UTF-8"%>
<%@page import="java.util.*, oracle.i18n.servlet.*" %>
<%@page import="oracle.i18n.util.*, oracle.i18n.text.*" %>

<%
    Localizer localizer = ServletHelper.getLocalizerInstance(request);
%>
```

The following code retrieves the current date and time value stored in the `currDate` variable. The value is formatted by the localizer `formatDateTime` method. The `OraDateFormat.LONG` parameter in the `formatDateTime` method instructs the localizer to format the date using the locale's long formatting style. If the locale of the incoming request is changed to a different locale with the locale selection list, then the date and time value will be formatted according to the conventions of the new locale. No code changes need to be made to support newly-introduced locales.

```
<div style="color: blue;" align="right">
    <% Date currDate= new Date(System.currentTimeMillis()); %>
    <%=localizer.formatDateTime(currDate, OraDateFormat.LONG)%>
</div>
```

The HelloWorld JSP can be reused for any locale because the HelloWorld string and title are selected in a locale-sensitive manner. The translated strings are selected from a resource bundle.

The GDK uses the `OraResourceBundle` class for implementing the resource bundle fallback behavior. The following code shows how the Localizer picks the HelloWorld message from the resource bundle.

The default application resource bundle Messages is declared in the `gdkapp.xml` file. The localizer uses the message resource bundle to pick the message and apply the locale-specific logic. For example, if the current locale for the incoming request is "de-CH", then the message will first be looked for in the `messages_de_CH` bundle. If it does not exist, then it will look up in the `Messages_de` resource bundle.

```
<h1><%= localizer.getMessage("helloWorld") %></h1>
```

Task 4: Create the Locale Selection List

The locale selection list is used to override the selected locale based on the HTTP Request Accept-Language header. The GDK framework checks the locale parameter passed in as part of the HTTP POST request as a value for the new locale. A locale selected with the locale selection list is posted as the locale parameter value. GDK uses this value for the request locale. All this happens implicitly within the GDK code.

The following code sample displays the locale selection list as an HTML select tag with the name locale. The submit tag causes the new value to be posted to the server. The GDK framework retrieves the correct selection.

```
<form>
  <select name="locale" size="1">
    <%= getCountryDropDown(request)%>
  </select>
  <input type="submit" value="<%= localizer.getMessage("changeLocale") %>">
</input>
</form>
```

The locale selection list is constructed from the HTML code generated by the `getCountryDropDown` method. The method converts the configured application locales into localized country names.

A call is made to the `ServletHelper` class to get the `ApplicationContext` object associated with the current request. This object provides the globalization context for an application, which includes information such as supported locales and configuration information. The `getSupportedLocales` call retrieves the list of locales in the `gdkapp.xml` file. The configured application locale list is displayed as options of the HTML select. The `OraDisplayLocaleInfo` class is responsible for providing localization methods of locale-specific elements such as country and language names.

An instance of this class is created by passing in the current locale automatically determined by the GDK framework. GDK creates requests and response wrappers for HTTP request and responses. The `request.getLocale()` method returns the GDK determined locale based on the locale determination rules.

The `OraDisplayLocaleInfo.getDisplayCountry` method retrieves the localized country names of the application locales. An HTML option list is created in the `ddOptBuffer` string buffer. The `getCountryDropDown` call returns a string containing the following HTML values:

```
<option value="en_US" selected>United States [en_US]</option>
<option value="zh_CN">China [zh_CN]</option>
<option value="de_CH">Switzerland [de_CH]</option>
```

In the preceding values, the en-US locale is selected for the locale. Country names are generated are based on the current locale.

The following example shows the code for constructing the locale selection list.

```
<%!
public String getCountryDropDown(HttpServletRequest request)
{
    StringBuffer ddOptBuffer = new StringBuffer();
    ApplicationContext ctx =
        ServletHelper.getApplicationContextInstance(request);
    Locale[] appLocales = ctx.getSupportedLocales();
    Locale currentLocale = request.getLocale();

    if (currentLocale.getCountry().equals(""))
    {
        // Since the Country was not specified get the Default Locale
        // (with Country) from the GDK
        OraLocaleInfo oli = OraLocaleInfo.getInstance(currentLocale);
        currentLocale = oli.getLocale();
    }

    OraDisplayLocaleInfo odli =
        OraDisplayLocaleInfo.getInstance(currentLocale);
    for (int i=0;i<appLocales.length; i++)
    {
        ddOptBuffer.append("<option value=\"" + appLocales[i] + "\"" +
            (appLocales[i].getLanguage().equals(currentLocale.getLanguage()) ?
            " selected" : "") +
            ">" + odli.getDisplayCountry(appLocales[i]) +
            " [" + appLocales[i] + "]"</option>\n");
    }

    return ddOptBuffer.toString();
}
%>
```

Task 5: Build the Application

In order to build the application, the following files must be specified in the classpath:

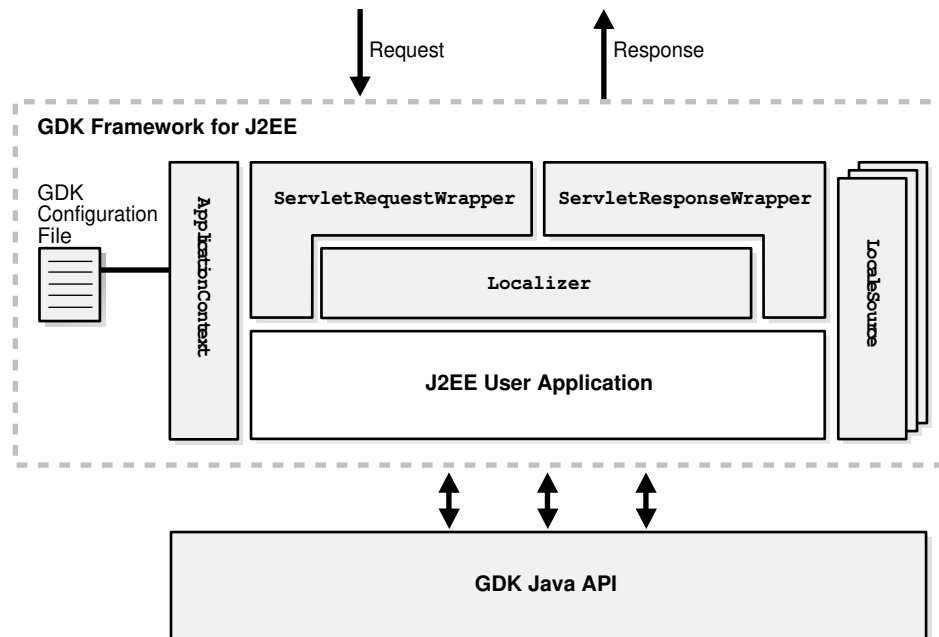
```
ora18n.jar
regexp.jar
```

The ora18n.jar file contains the GDK framework and the API. The regexp.jar file contains the regular expression library. The GDK API also has locale determination capabilities. The classes are supplied by the ora18n-lcsd.jar file.

8.6 GDK Application Framework for J2EE

GDK for Java provides the globalization framework for middle-tier J2EE applications. The framework encapsulates the complexity of globalization programming, such as determining user locale, maintaining locale persistency, and processing locale information. This framework minimizes the effort required to make Internet applications global-ready. The following figure shows the GDK application framework.

Figure 8-6 GDK Application Framework for J2EE



The main Java classes composing the framework are as follows:

- **ApplicationContext** provides the globalization context of an application. The context information includes the list of supported locales and the rule for determining user-preferred locale. The context information is obtained from the GDK application configuration file for the application.
- The set of **LocaleSource** classes can be plugged into the framework. Each **LocaleSource** class implements the **LocaleSource** interface to get the locale from the corresponding source. Oracle bundles several **LocaleSource** classes in GDK. For example, the **DBLocaleSource** class obtains the locale information of the current user from a database schema. You can also write a customized **LocaleSource** class by implementing the same **LocaleSource** interface and plugging it into the framework.
- **ServletRequestWrapper** and **ServletResponseWrapper** are the main classes of the GDK Servlet filter that transforms HTTP requests and HTTP responses. **ServletRequestWrapper** instantiates a **Localizer** object for each HTTP request based on the information gathered from the **ApplicationContext** and **LocaleSource** objects and ensures that forms parameters are handled properly. **ServletResponseWrapper** controls how HTTP response should be constructed.
- **Localizer** is the all-in-one object that exposes the important functions that are sensitive to the current user locale and application context. It provides a centralized set of methods for you to call and make your applications behave appropriately to the current user locale and application context.
- The GDK Java API is always available for applications to enable finer control of globalization behavior.

The GDK application framework simplifies the coding required for your applications to support different locales. When you write a J2EE application according to the application framework, the application code is independent of what locales the application supports, and you control the globalization support in the application by defining it in the GDK application configuration

file. There is no code change required when you add or remove a locale from the list of supported application locales.

The following list gives you some idea of the extent to which you can define the globalization support in the GDK application configuration file:

- You can add and remove a locale from the list of supported locales.
- You can change the way the user locale is determined.
- You can change the HTML page encoding of your application.
- You can specify how the translated resources can be located.
- You can plug a new `LocaleSource` object into the framework and use it to detect a user locale.

This section includes the following topics:

- [Making the GDK Framework Available to J2EE Applications](#)
- [Integrating Locale Sources into the GDK Framework](#)
- [Getting the User Locale From the GDK Framework](#)
- [Implementing Locale Awareness Using the GDK Localizer](#)
- [Defining the Supported Application Locales in the GDK](#)
- [Handling Non-ASCII Input and Output in the GDK Framework](#)
- [Managing Localized Content in the GDK](#)

8.6.1 Making the GDK Framework Available to J2EE Applications

The behavior of the GDK application framework for J2EE is controlled by the GDK application configuration file, `gdkapp.xml`. The application configuration file allows developers to specify the behaviors of globalized applications in one centralized place. One application configuration file is required for each J2EE application using the GDK. The `gdkapp.xml` file should be placed in the `./WEB-INF` directory of the J2EE environment of the application. The file dictates the behavior and the properties of the GDK framework and the application that is using it. It contains locale mapping tables, character sets of content files, and globalization parameters for the configuration of the application. The application administrator can modify the application configuration file to change the globalization behavior in the application, without needing to change the programs and to recompile them.

See Also

["The GDK Application Configuration File"](#)

For a J2EE application to use the GDK application framework defined by the corresponding GDK application configuration file, the GDK Servlet filter and the GDK context listener must be defined in the `web.xml` file of the application. The `web.xml` file should be modified to include the following at the beginning of the file:

```
<web-app>
<!-- Add GDK filter that is called after the authentication -->

<filter>
  <filter-name>gdkfilter</filter-name>
  <filter-class>oracle.i18n.servlet.filter.ServletFilter</filter-class>
```

```

</filter>
<filter-mapping>
  <filter-name>gdkfilter</filter-name>
  <url-pattern>*.jsp</url-pattern>
</filter-mapping>

<!-- Include the GDK context listener -->

<listener>
<listener-class>oracle.i18n.servlet.listener.ContextListener</listener-class>
</listener>
</web-app>

```

Examples of the `gdkapp.xml` and `web.xml` files can be found in the `$ORACLE_HOME/nls/gdk/demo` directory.

The GDK application framework supports Servlet container version 2.3 and later. It uses the Servlet filter facility for transparent globalization operations such as determining the user locale and specifying the character set for content files. The `ContextListener` instantiates GDK application parameters described in the GDK application configuration file. The `ServletFilter` overrides the request and response objects with a GDK request (`ServletRequestWrapper`) and response (`ServletResponseWrapper`) objects, respectively.

If other application filters are used in the application to also override the same methods, then the filter in the GDK framework may return incorrect results. For example, if `getLocale` returns `en_US`, but the result is overridden by other filters, then the result of the GDK locale detection mechanism is affected. All of the methods that are being overridden in the filter of the GDK framework are documented in *Oracle Globalization Development Kit Java API Reference*. Be aware of potential conflicts when using other filters together with the GDK framework.

8.6.2 Integrating Locale Sources into the GDK Framework

Determining the user's preferred locale is the first step in making an application global-ready. The locale detection offered by the J2EE application framework is primitive. It lacks the method that transparently retrieves the most appropriate user locale among locale sources. It provides locale detection by the HTTP language preference only, and it cannot support a multilevel locale fallback mechanism. The GDK application framework provides support for predefined locale sources to complement J2EE. In a web application, several locale sources are available. [Table 8-1](#) summarizes locale sources that are provided by the GDK.

Table 8-1 Locale Resources Provided by the GDK

Locale	Description
HTTP language preference	Locales included in the HTTP protocol as a value of <code>Accept-Language</code> . This is set at the web browser level. A locale fallback operation is required if the browser locale is not supported by the application.
User input locale	Locale specified by the user from a menu or a parameter in the HTTP protocol
User profile locale preference from database	Locale preference stored in the database as part of the user profiles
Application default locale	A locale defined in the GDK application configuration file. This locale is defined as the default locale for the application. Typically, this is used as a fallback locale when the other locale sources are not available.

See Also

["The GDK Application Configuration File"](#) for information about the GDK multilevel locale fallback mechanism

The GDK application framework provides seamless support for predefined locale sources, such as user input locale, HTTP language preference, user profile locale preference in the database, and the application default locale. You can incorporate the locale sources to the framework by defining them under the `<locale-determine-rule>` tag in the GDK application configuration file as follows:

```
<locale-determine-rule>
  <locale-source>
    oracle.i18n.servlet.localesource.UserInput
  </locale-source>
  <locale-source>
    oracle.i18n.servlet.localesource.HTTPAcceptLanguage
  </locale-source>
</locale-determine-rule>
```

The GDK framework uses the locale source declaration order and determines whether a particular locale source is available. If it is available, then it is used as the source, otherwise, it tries to find the next available locale source for the list. In the preceding example, if the `UserInput` locale source is available, it is used first, otherwise, the `HTTPAcceptLanguage` locale source will be used.

Custom locale sources, such as locale preference from an LDAP server, can be easily implemented and integrated into the GDK framework. You must implement the `LocaleSource` interface and specify the corresponding implementation class under the `<locale-determine-rule>` tag in the same way as the predefined locale sources were specified.

The `LocaleSource` implementation not only retrieves the locale information from the corresponding source to the framework but also updates the locale information to the corresponding source when the framework tells it to do so. Locale sources can be read-only or read/write, and they can be cacheable or noncacheable. The GDK framework initiates updates only to read/write locale sources and caches the locale information from cacheable locale sources. Examples of custom locale sources can be found in the `$ORACLE_HOME/nls/gdk/demo` directory.

See Also

Oracle Globalization Development Kit Java API Reference for more information about implementing a `LocaleSource`

8.6.3 Getting the User Locale From the GDK Framework

The GDK offers automatic locale detection to determine the current locale of the user. For example, the following code retrieves the current user locale in Java. It uses a `Locale` object explicitly.

```
Locale loc = request.getLocale();
```

The `getLocale()` method returns the `Locale` that represents the current locale. This is similar to invoking the `HttpServletRequest.getLocale()` method in JSP or Java Servlet code.

However, the logic in determining the user locale is different, because multiple locale sources are being considered in the GDK framework.

Alternatively, you can get a `Localizer` object that encapsulates the `Locale` object determined by the GDK framework. For the benefits of using the `Localizer` object, see "[Implementing Locale Awareness Using the GDK Localizer](#)".

```
Localizer localizer = ServletHelper.getLocalizerInstance(request);
Locale loc = localizer.getLocale();
```

The locale detection logic of the GDK framework depends on the locale sources defined in the GDK application configuration file. The names of the locale sources are registered in the application configuration file. The following example shows the locale determination rule section of the application configuration file. It indicates that the user-preferred locale can be determined from either the LDAP server or from the HTTP Accept-Language header. The `LDAPUserSchema` locale source class should be provided by the application. Note that all of the locale source classes have to be extended from the `LocaleSource` abstract class.

```
<locale-determine-rule>
  <locale-source>LDAPUserSchema</locale-source>
  <locale-source>oracle.i18n.localesource.HTTPAcceptLanguage</locale-source>
</locale-determine-rule>
```

For example, when the user is authenticated in the application and the user locale preference is stored in an LDAP server, then the `LDAPUserSchema` class connects to the LDAP server to retrieve the user locale preference. When the user is anonymous, then the `HttpAcceptLanguage` class returns the language preference of the web browser.

The cache is maintained for the duration of a HTTP session. If the locale source is obtained from the HTTP language preference, then the locale information is passed to the application in the HTTP Accept-Language header and not cached. This enables flexibility so that the locale preference can change between requests. The cache is available in the HTTP session.

The GDK framework exposes a method for the application to overwrite the locale preference information persistently stored in locale sources such as the LDAP server or the user profile table in the database. This method also resets the current locale information stored inside the cache for the current HTTP session. The following is an example of overwriting the preferred locale using the store command.

```
<input type="hidden"
name="<%=appctx.getParameterName(LocaleSource.Parameter.COMMAND)%>"
value="store">
```

To discard the current locale information stored inside the cache, the `clean` command can be specified as the input parameter. The following table shows the list of commands supported by the GDK:

Command	Functionality
store	Updates user locale preferences in the available locale sources with the specified locale information. This command is ignored by the read-only locale sources.
clean	Discards the current locale information in the cache.

Note that the GDK parameter names can be customized in the application configuration file to avoid name conflicts with other parameters used in the application.

8.6.4 Implementing Locale Awareness Using the GDK Localizer

The Localizer object obtained from the GDK application framework is an all-in-one globalization object that provides access to functions that are commonly used in building locale awareness in your applications. In addition, it provides functions to get information about the application context, such as the list of supported locales. The Localizer object simplifies and centralizes the code required to build consistent locale awareness behavior in your applications.

The `oracle.i18n.servlet` package contains the Localizer class. You can get the Localizer instance as follows:

```
Localizer lc = ServletHelper.getLocalizerInstance(request);
```

The Localizer object encapsulates the most commonly used locale-sensitive information determined by the GDK framework and exposes it as locale-sensitive methods. This object includes the following functionalities pertaining to the user locale:

- Format date in long and short formats
- Format numbers and currencies
- Get collation key value of a string
- Get locale data such as language, country and currency names
- Get locale data to be used for constructing user interface
- Get a translated message from resource bundles
- Get text formatting information such as writing direction
- Encode and decode URLs
- Get the common list of time zones and linguistic sorts

For example, when you want to display a date in your application, you may want to call the `Localizer.formatDate()` or `Localizer.formatDateTime()` methods. When you want to determine the writing direction of the current locale, you can call the `Localizer.getWritingDirection()` and `Localizer.getAlignment()` to determine the value used in the `<DIR>` tag and `<ALIGN>` tag respectively.

The Localizer object also exposes methods to enumerate the list of supported locales and their corresponding languages and countries in your applications.

The Localizer object actually makes use of the classes in the GDK Java API to accomplish its tasks. These classes include, but are not limited to, the following: `OraDateFormat`, `OraNumberFormat`, `OraCollator`, `OraLocaleInfo`, `oracle.i18n.util.LocaleMapper`, `oracle.i18n.net.URLEncoder`, and `oracle.i18n.net.URLDecoder`.

The Localizer object simplifies the code you need to write for locale awareness. It maintains caches of the corresponding objects created from the GDK Java API so that the calling application does not need to maintain these objects for subsequent calls to the same objects. If you require more than the functionality the Localizer object can provide, then you can always call the corresponding methods in the GDK Java API directly.

Note

Strings returned by many Localizer methods, such as formatted dates and locale-specific currency symbols, depend on locale data that may be provided by users through URLs or form input. For example, the locale source class `oracle.i18n.servlet.localesource.UserInput` provides various datetime format patterns and the ISO currency abbreviation retrieved from a page URL. A datetime format pattern may include double-quoted literal strings with arbitrary contents. To prevent cross-site script injection attacks, strings returned by Localizer methods must be properly escaped before being displayed as part of an HTML page, for example, by applying the method `encode` of the class `oracle.i18n.net.CharEntityReference`.

See Also

Oracle Globalization Development Kit Java API Reference for detailed information about the Localizer object

8.6.5 Defining the Supported Application Locales in the GDK

The number of locales and the names of the locales that an application needs to support are based on the business requirements of the application. The names of the locales that are supported by the application are registered in the application configuration file. The following example shows the application locales section of the application configuration file. It indicates that the application supports German (de), Japanese (ja), and English for the US (en-US), with English defined as the default fallback application locale. Note that the locale names are based on the IANA convention.

```
<application-locales>
  <locale>de</locale>
  <locale>ja</locale>
  <locale default="yes">en-US</locale>
</application-locales>
```

When the GDK framework detects the user locale, it verifies whether the locale that is returned is one of the supported locales in the application configuration file. The verification algorithm is as follows:

1. Retrieve the list of supported application locales from the application configuration file.
2. Check whether the locale that was detected is included in the list. If it is included in the list, then use this locale as the current client's locale.
3. If there is a variant in the locale that was detected, then remove the variant and check whether the resulting locale is in the list. For example, the Java locale `de_DE_EURO` has a `EURO` variant. Remove the variant so that the resulting locale is `de_DE`.
4. If the locale includes a country code, then remove the country code and check whether the resulting locale is in the list. For example, the Java locale `de_DE` has a country code of `DE`. Remove the country code so that the resulting locale is `de`.
5. If the detected locale does not match any of the locales in the list, then use the default locale that is defined in the application configuration file as the client locale.

By performing steps 3 and 4, the application can support users with the same language requirements but with different locale settings than those defined in the application

configuration file. For example, the GDK can support de-AT (the Austrian variant of German), de-CH (the Swiss variant of German), and de-LU (the Luxembourgian variant of German) locales.

The locale fallback detection in the GDK framework is similar to that of the Java Resource Bundle, except that it is not affected by the default locale of the Java VM. This exception occurs because the Application Default Locale can be used during the GDK locale fallback operations.

If the application-locales section is omitted from the application configuration file, then the GDK assumes that the common locales, which can be returned from the `OracleLocaleInfo.getCommonLocales` method, are supported by the application.

8.6.6 Handling Non-ASCII Input and Output in the GDK Framework

The character set (or character encoding) of an HTML page is a very important piece of information to a browser and an Internet application. The browser needs to interpret this information so that it can use correct fonts and character set mapping tables for displaying pages. The Internet applications need to know so they can safely process input data from a HTML form based on the specified encoding.

The page encoding can be translated as the character set used for the locale to which an Internet application is serving.

In order to correctly specify the page encoding for HTML pages without using the GDK framework, Internet applications must:

1. Determine the desired page input data character set encoding for a given locale.
2. Specify the corresponding encoding name for each HTTP request and HTTP response.

Applications using the GDK framework can ignore these steps. No application code change is required. The character set information is specified in the GDK application configuration file. At run time, the GDK automatically sets the character sets for the request and response objects. The GDK framework does not support the scenario where the incoming character set is different from that of the outgoing character set.

The GDK application framework supports the following scenarios for setting the character sets of the HTML pages:

- A single local character set is dedicated to the whole application. This is appropriate for a monolingual Internet application. Depending on the properties of the character set, it may be able to support more than one language. For example, most Western European languages can be served by ISO-8859-1.
- Unicode UTF-8 is used for all contents regardless of the language. This is appropriate for a multilingual application that uses Unicode for deployment.
- The native character set for each language is used. For example, English contents are represented in ISO-8859-1, and Japanese contents are represented in Shift_JIS. This is appropriate for a multilingual Internet application that uses a default character set mapping for each locale. This is useful for applications that need to support different character sets based on the user locales. For example, for mobile applications that lack Unicode fonts or Internet browsers that cannot fully support Unicode, the character sets must to be determined for each request.

The character set information is specified in the GDK application configuration file. The following is an example of setting UTF-8 as the character set for all the application pages.

```
<page-charset>UTF-8</page-charset>
```

The page character set information is used by the `ServletRequestWrapper` class, which sets the proper character set for the request object. It is also used by the `ContentType` HTTP header specified in the `ServletResponseWrapper` class for output when instantiated. If `page-charset` is set to `AUTO-CHARSET`, then the character set is assumed to be the default character set for the current user locale. Set `page-charset` to `AUTO-CHARSET` as follows:

```
<page-charset>AUTO-CHARSET</page-charset>
```

The default mappings are derived from the `LocaleMapper` class, which provides the default IANA character set for the locale name in the GDK Java API.

[Table 8-2](#) lists the mappings between the common ISO locales and their IANA character sets.

Table 8-2 Mapping Between Common ISO Locales and IANA Character Sets

ISO Locale	NLS_LANGUAGE Value	NLS_TERRITORY Value	IANA Character Set
ar-SA	ARABIC	SAUDI ARABIA	WINDOWS-1256
de-DE	GERMAN	GERMANY	WINDOWS-1252
en-US	AMERICAN	AMERICA	WINDOWS-1252
en-GB	ENGLISH	UNITED KINGDOM	WINDOWS-1252
el	GREEK	GREECE	WINDOWS-1253
es-ES	SPANISH	SPAIN	WINDOWS-1252
fr	FRENCH	FRANCE	WINDOWS-1252
fr-CA	CANADIAN FRENCH	CANADA	WINDOWS-1252
iw	HEBREW	ISRAEL	WINDOWS-1255
ko	KOREAN	KOREA	EUC-KR
ja	JAPANESE	JAPAN	SHIFT_JIS
it	ITALIAN	ITALY	WINDOWS-1252
pt	PORTUGUESE	PORTUGAL	WINDOWS-1252
pt-BR	BRAZILIAN PORTUGUESE	BRAZIL	WINDOWS-1252
tr	TURKISH	TURKIYE	WINDOWS-1254
nl	DUTCH	THE NETHERLANDS	WINDOWS-1252
zh	SIMPLIFIED CHINESE	CHINA	GBK
zh-TW	TRADITIONAL CHINESE	TAIWAN	BIG5

The locale to character set mapping in the GDK can also be customized. To override the default mapping defined in the GDK Java API, a locale-to-character-set mapping table can be specified in the application configuration file.

```
<locale-charset-maps>
  <locale-charset>
    <locale>ja</locale><charset>EUC-JP</charset>
  </locale-charset>
</locale-charset-maps>
```

The previous example shows that for locale Japanese (ja), the GDK changes the default character set from `SHIFT_JIS` to `EUC-JP`.

① See Also

["Oracle Locale Information in the GDK"](#)

8.6.7 Managing Localized Content in the GDK

This section includes the following topics:

- [Managing Localized Content in JSPs and Java Servlets](#)
- [Managing Localized Content in Static Files](#)

8.6.7.1 Managing Localized Content in JSPs and Java Servlets

Resource bundles enable access to localized contents at run time in J2SE. Translatable strings within Java servlets and Java Server Pages (JSPs) are externalized into Java resource bundles so that these resource bundles can be translated independently into different languages. The translated resource bundles carry the same base class names as the English bundles, using the Java locale name as the suffix.

To retrieve translated data from the resource bundle, the `getBundle()` method must be invoked for every request.

```
<% Locale user_locale=request.getLocale();  
    ResourceBundle rb=ResourceBundle.getBundle("resource",user_locale); %>  
<%= rb.getString("Welcome") %>
```

The GDK framework simplifies the retrieval of text strings from the resource bundles. `Localizer.getMessage()` is a wrapper to the resource bundle.

```
<% Localizer.getMessage ("Welcome") %>
```

Instead of specifying the base class name as `getBundle()` in the application, you can specify the resource bundle in the application configuration file, so that the GDK automatically instantiates a `ResourceBundle` object when a translated text string is requested.

```
<message-bundles>  
  <resource-bundle name="default">resource</resource-bundle>  
</message-bundles>
```

This configuration file snippet declares a default resource bundle whose translated contents reside in the "resource" Java bundle class. Multiple resource bundles can be specified in the configuration file. To access a nondefault bundle, specify the name parameter in the `getMessage` method. The message bundle mechanism uses the `OracleResourceBundle` GDK class for its implementation. This class provides the special locale fallback behaviors on top of the Java behaviors. The rules are as follows:

- If the given locale exactly matches the locale in the available resource bundles, it will be used.
- If the resource bundle for Chinese in Singapore (zh_SG) is not found, it will fall back to the resource bundle for Chinese in China (zh_CN) for Simplified Chinese translations.
- If the resource bundle for Chinese in Hong Kong (zh_HK) is not found, it will fall back to the resource bundle for Chinese in Taiwan (zh_TW) for Traditional Chinese translations.
- If the resource bundle for Chinese in Macau (zh_MO) is not found, it will fall back to the resource bundle for Chinese in Taiwan (zh_TW) for Traditional Chinese translations.

- If the resource bundle for any other Chinese (zh_ and zh) is not found, it will fall back to the resource bundle for Chinese in China (zh_CN) for Simplified Chinese translations.
- The default locale, which can be obtained by the `Locale.getDefault()` method, will not be considered in the fallback operations.

For example, assume the default locale is ja_JP and the resource handle for it is available. When the resource bundle for es_MX is requested, and neither resource bundle for es or es_MX is provided, the base resource bundle object that does not have a local suffix is returned.

The usage of the `OraResourceBundle` class is similar to the `java.util.ResourceBundle` class, but the `OraResourceBundle` class does not instantiate itself. Instead, the return value of the `getBundle` method is an instance of the subclass of the `java.util.ResourceBundle` class.

8.6.7.2 Managing Localized Content in Static Files

For a application, which supports only one locale, the URL that has a suffix of `/index.html` typically takes the user to the starting page of the application.

In a globalized application, contents in different languages are usually stored separately, and it is common for them to be staged in different directories or with different file names based on the language or the country name. This information is then used to construct the URLs for localized content retrieval in the application.

The following examples illustrate how to retrieve the French and Japanese versions of the index page. Their suffixes are as follows:

```
/fr/index.html
/ja/index.html
```

By using the `rewriteURL()` method of the `ServletHelper` class, the GDK framework handles the logic to locate the translated files from the corresponding language directories. The `ServletHelper.rewriteURL()` method rewrites a URL based on the rules specified in the application configuration file. This method is used to determine the correct location where the localized content is staged.

The following is an example of the JSP code:

```
">
<a href="<%=ServletHelper.rewriteURL("html/welcome.html", request)%>">
```

The URL rewrite definitions are defined in the GDK application configuration file:

```
<url-rewrite-rule fallback="yes">
  <pattern>(.*)([a-zA-Z0-9_]+).$</pattern>
  <result>$1/$A/$2</result>
</url-rewrite-rule>
```

The pattern section defined in the rewrite rule follows the regular expression conventions. The result section supports the following special variables for replacing:

- `$L` is used to represent the ISO 639 language code part of the current user locale
- `$C` represents the ISO 3166 country code
- `$A` represents the entire locale string, where the ISO 639 language code and ISO 3166 country code are connected with an underscore character (`_`)
- `$1` to `$9` represent the matched substrings

For example, if the current user locale is ja, then the URL for the `welcome.jpg` image file is rewritten as `image/ja/welcome.jpg`, and `welcome.html` is changed to `html/ja/welcome.html`.

Both `ServletHelper.rewriteURL()` and `Localizer.getMessage()` methods perform consistent locale fallback operations in the case where the translation files for the user locale are not available. For example, if the online help files are not available for the `es_MX` locale (Spanish for Mexico), but the `es` (Spanish for Spain) files are available, then the methods will select the Spanish translated files as the substitute.

8.7 GDK Java API

The globalization features and behaviors in Java are not the same as those offered in Oracle Database. For example, J2SE supports a set of locales and character sets that are different from locales and character sets in Oracle Database. This inconsistency can be confusing for users when their application contains data that is formatted based on two different conventions. For example, dates that are retrieved from the database are formatted using Oracle conventions, such as number and date formatting and linguistic sort ordering. However, the static application data is typically formatted using Java locale conventions. The globalization functionalities in Java can also be different depending on the version of the JDK on which the application runs.

Before Oracle Database 10g, when an application was required to incorporate Oracle globalization features, it had to make connections to the database and issue SQL statements. Such operations make the application complicated and generate more network connections to the database.

In Oracle Database 10g and later, the GDK Java API extends the globalization features to the middle tier. By enabling applications to perform globalization logic such as Oracle date and number formatting and linguistic sorting in the middle tier, the GDK Java API enables developers to eliminate expensive programming logic in the database. The GDK Java API also provides standard compliance for XQuery. This improves the overall application performance by reducing the database processing load, and by decreasing unnecessary network traffic between the application tier and the back end.

The GDK Java API also offers advanced globalization features, such as language and character set detection, and the enumeration of common locale data for a territory or a language (for example, all time zones supported in Canada). These features are not available in most programming platforms. Without the GDK Java API, developers must write business logic to handle these processes inside the application.

The key functionalities of the GDK Java API are as follows:

- [Oracle Locale Information in the GDK](#)
- [Oracle Locale Mapping in the GDK](#)
- [Oracle Character Set Conversion in the GDK](#)
- [Oracle Date, Number, and Monetary Formats in the GDK](#)
- [Oracle Binary and Linguistic Sorts in the GDK](#)
- [Oracle Language and Character Set Detection in the GDK](#)
- [Oracle Translated Locale and Time Zone Names in the GDK](#)
- [Using the GDK with E-Mail Programs](#)

8.7.1 Oracle Locale Information in the GDK

Oracle locale definitions, which include languages, territories, linguistic sorts, and character sets, are exposed in the GDK Java API. The naming convention that Oracle uses may be

different from other vendors. Although many of these names and definitions follow industry standards, some are Oracle-specific, tailored to meet special customer requirements.

OraLocaleInfo is an Oracle locale class that includes language, territory, and collator objects. It provides a method for applications to retrieve a collection of locale-related objects for a given locale. Examples include: a full list of the Oracle linguistic sorts available in the GDK, the local time zones defined for a given territory, or the common languages used in a particular territory.

Following are examples of using the OraLocaleInfo class:

```
// All Territories supported by GDK
String[] avterr = OraLocaleInfo.getAvailableTerritories();

// Local TimeZones for a given Territory

OraLocaleInfo oloc = OraLocaleInfo.getInstance("English", "Canada");
TimeZone[] loctz = oloc.getLocalTimeZones();
```

8.7.2 Oracle Locale Mapping in the GDK

The GDK Java API provides the LocaleMapper class. It maps equivalent locales and character sets between Java, IANA, ISO, and Oracle. A Java application may receive locale information from the client that is specified in an Oracle Database locale name or an IANA character set name. The Java application must be able to map to an equivalent Java locale or Java encoding before it can process the information correctly.

The follow example shows using the LocaleMapper class.

```
// Mapping from Java locale to Oracle language and Oracle territory

Locale locale = new Locale("it", "IT");
String oraLang = LocaleMapper.getOraLanguage(locale);
String oraTerr = LocaleMapper.getOraTerritory(locale);

// From Oracle language and Oracle territory to Java Locale

locale = LocaleMapper.getJavaLocale("AMERICAN", "AMERICA");
locale = LocaleMapper.getJavaLocale("TRADITIONAL CHINESE", "");

// From IANA & Java to Oracle Character set

String ocs1 = LocaleMapper.getOraCharacterSet(
    LocaleMapper.IANA, "ISO-8859-1");
String ocs2 = LocaleMapper.getOraCharacterSet(
    LocaleMapper.JAVA, "ISO8859_1");
```

The LocaleMapper class can also return the most commonly used e-mail character set for a specific locale on both Windows and UNIX platforms. This is useful when developing Java applications that need to process e-mail messages.

① See Also

["Using the GDK with E-Mail Programs"](#)

8.7.3 Oracle Character Set Conversion in the GDK

The GDK Java API contains a set of character set conversion classes APIs that enable users to perform Oracle character set conversions. Although Java JDK is already equipped with classes that can perform conversions for many of the standard character sets, they do not support Oracle-specific character sets and Oracle's user-defined character sets.

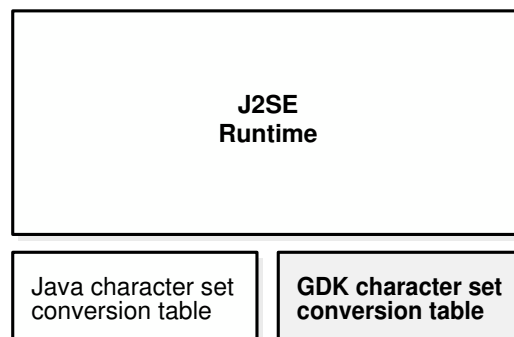
JDK provides an interface for developers to extend Java's character sets. The GDK Java API provides implicit support for Oracle's character sets by using this plug-in feature provided by the JDK package `java.nio.charset`. You can access the J2SE API to obtain Oracle-specific behaviors.

Note

The GDK Java API is supported with JDK versions 1.6 and later.

The following figure shows that the GDK character set conversion tables are plugged into J2SE in the same way as the Java character set tables. With this pluggable framework of J2SE, the Oracle character set conversions can be used in the same way as other Java character set conversions.

Figure 8-7 Oracle Character Set Plug-In



The GDK character conversion classes support all Oracle character sets including user-defined characters sets. It can be used by Java applications to properly convert to and from Java's internal character set, UTF-16.

Oracle's character set names are proprietary. To avoid potential conflicts with Java's own character sets, all Oracle character set names have an X-ORACLE- prefix for all implicit usage through Java's API.

The following is an example of Oracle character set conversion:

```
// Converts the Chinese character "three" from UCS2 to JA16SJIS

String str = "\u4e09";
byte[] barr = str.getBytes("x-oracle-JA16SJIS");
```

Just as with other Java character sets, the character set facility in `java.nio.charset.Charset` is applicable to all of the Oracle character sets. For example, if you want to check whether the specified character set is a superset of another character set, then you can use the `Charset.contains` method as follows:


```
Charset cs1 = Charset.forName("x-oracle-US7ASCII");
Charset cs2 = Charset.forName("x-oracle-WE8WINDOWS1252");
// true if WE8WINDOWS1252 is the superset of US7ASCII, otherwise false.
boolean osc = cs2.contains(cs1);
```

For a Java application that is using the JDBC driver to communicate with the database, the JDBC driver provides the necessary character set conversion between the application and the database. Calling the GDK character set conversion methods explicitly within the application is not required. A Java application that interprets and generates text files based on Oracle's character set encoding format is an example of using Oracle character set conversion classes.

8.7.4 Oracle Date, Number, and Monetary Formats in the GDK

The GDK Java API provides formatting classes that support date, number, and monetary formats using Oracle conventions for Java applications in the `oracle.i18n.text` package.

New locale formats introduced in Oracle Database 10g, such as the short and long date, number, and monetary formats, are also exposed in these format classes.

The following are examples of Oracle date, Oracle number, and Oracle monetary formatting:

```
// Obtain the current date and time in the default Oracle LONG format for
// the locale de_DE (German_Germany)

Locale locale = new Locale("de", "DE");
OraDateFormat odf =
    OraDateFormat.getDateTimeInstance(OraDateFormat.LONG, locale);

// Obtain the numeric value 1234567.89 using the default number format
// for the Locale en_IN (English_India)

locale = new Locale("en", "IN");
OraNumberFormat onf = OraNumberFormat.getNumberInstance(locale);
String nm = onf.format(new Double(1234567.89));

// Obtain the monetary value 1234567.89 using the default currency
// format for the Locale en_US (American_America)

locale = new Locale("en", "US");

onf = OraNumberFormat.getCurrencyInstance(locale);
nm = onf.format(new Double(1234567.89));
```

8.7.5 Oracle Binary and Linguistic Sorts in the GDK

Oracle provides support for binary, monolingual, and multilingual linguistic sorts in the database. In Oracle Database, these sorts provide case-insensitive and accent-insensitive sorting and searching capabilities inside the database. By using the `OraCollator` class, the GDK Java API enables Java applications to sort and search for information based on the latest Oracle binary and linguistic sorting features, including case-insensitive and accent-insensitive options.

Normalization can be an important part of sorting. The composition and decomposition of characters are based on the Unicode standard; therefore, sorting also depends on the Unicode standard. The GDK contains methods to perform composition.

Note

Because each version of the JDK may support a different version of the Unicode standard, the GDK provides an `OraNormalizer` class that is based on the latest version of the Unicode standard, which for Oracle AI Database 26ai is Unicode 15.0 (RU 23.26.2 and earlier) or Unicode 17.0 (RU 23.26.3 and later).

The sorting order of a binary sort is based on the Oracle character set that is being used. Except for the UTFE character set, the binary sorts of all Oracle character sets are supported in the GDK Java API. The only linguistic sort that is not supported in the GDK Java API is JAPANESE, but a similar and more accurate sorting result can be achieved by using JAPANESE_M.

The following example shows string comparisons and string sorting.

Example 8-3 String Comparisons and String Sorting

```
// compares strings using XGERMAN

private static String s1 = "abcSS";
private static String s2 = "abc\u00DF";

String cname = "XGERMAN";
OraCollator ocol = OraCollator.getInstance(cname);
int c = ocol.compare(s1, s2);

// sorts strings using GENERIC_M

private static String[] source =
    new String[]
    {
        "Hochgeschwindigkeitsdrucker",
        "Bildschirmfu\u00DF",
        "Skjermhengsel",
        "DIMM de Mem\u00F3ria",
        "M\u00F3dulo SDRAM com ECC",
    };

cname = "GENERIC_M";
ocol = OraCollator.getInstance(cname);
List result = getCollationKeys(source, ocol);

private static List getCollationKeys(String[] source, OraCollator ocol)
{
    List karr = new ArrayList(source.length);
    for (int i = 0; i < source.length; ++i)
    {
        karr.add(ocol.getCollationKey(source[i]));
    }
    Collections.sort(karr); // sorting operation
    return karr;
}
```

8.7.6 Oracle Language and Character Set Detection in the GDK

The Oracle Language and Character Set Detection Java classes in the GDK Java API provide a high performance, statistically based engine for determining the character set and language for unspecified text. It can automatically identify language and character set pairs from throughout the world. With each text, the language and character set detection engine sets up a series of probabilities, each probability corresponding to a language and character set pair. The most probable pair statistically identifies the dominant language and character set.

The purity of the text submitted affects the accuracy of the language and character set detection. Only plain text strings are accepted, so any tagging must be stripped before hand. The ideal case is literary text with almost no foreign words or grammatical errors. Text strings that contain a mix of languages or character sets, or nonnatural language text like addresses, phone numbers, and programming language code may yield poor results.

The LCSDetector class can detect the language and character set of a byte array, a character array, a string, and an InputStream class. It supports both plain text and HTML file detection. It can take the entire input for sampling or only portions of the input for sampling, when the length or both the offset and the length are supplied. For each input, up to three potential language and character set pairs can be returned by the LCSDetector class. They are always ranked in sequence, with the pair with the highest probability returned first.

① See Also

["Language and Character Set Detection Support"](#) for a list of supported language and character set pairs

The following are examples of using the LCSDetector class to enable language and character set detection:

```
// This example detects the character set of a plain text file "foo.txt" and
// then appends the detected ISO character set name to the name of the text file
```

```
LCSDetector lcsd = new LCSDetector();
File oldfile = new File("foo.txt");
FileInputStream in = new FileInputStream(oldfile);
lcsd.detect(in);
String charset = lcsd.getResult().getIANACharacterSet();
File newfile = new File("foo."+charset+".txt");
oldfile.renameTo(newfile);
```

```
// This example shows how to use the LCSDetector class to detect the language and
// character set of a byte array
```

```
int offset = 0;
LCSDetector led = new LCSDetector();
/* loop through the entire byte array */
while ( true )
{
    bytes_read = led.detect(byte_input, offset, 1024);
    if ( bytes_read == -1 )
        break;
    offset += bytes_read;
}
LCSDResultSet res = led.getResult();
```

```

/* print the detection results with close ratios */
System.out.println("the best guess " );
System.out.println("Langauge " + res.getOraLanguage() );
System.out.println("CharacterSet " + res.getOraCharacterSet() );
int high_hit = res.getHiHitPairs();
if ( high_hit >= 2 )
{
    System.out.println("the second best guess " );
    System.out.println("Langauge " + res.getOraLanguage(2) );
    System.out.println("CharacterSet " +res.getOraCharacterSet(2) );
}
if ( high_hit >= 3 )
{
    System.out.println("the third best guess ");
    System.out.println("Langauge " + res.getOraLanguage(3) );
    System.out.println("CharacterSet " +res.getOraCharacterSet(3) );
}

```

8.7.7 Oracle Translated Locale and Time Zone Names in the GDK

All of the Oracle language names, territory names, character set names, linguistic sort names, and time zone names have been translated into 27 languages including English. They are readily available for inclusion into the user applications, and they provide consistency for the display names across user applications in different languages. `OraDisplayLocaleInfo` is a utility class that provides the translations of locale and attributes. The translated names are useful for presentation in user interface text and for drop-down selection boxes. For example, a native French speaker prefers to select from a list of time zones displayed in French than in English.

The following example shows using `OraDisplayLocaleInfo` to return a list of time zones supported in Canada, using the French translation names.

Example 8-4 Using `OraDisplayLocaleInfo` to Return a Specific List of Time Zones

```

OraLocaleInfo oloc = OraLocaleInfo.getInstance("CANADIAN FRENCH", "CANADA");
OraDisplayLocaleInfo odloc = OraDisplayLocaleInfo.getInstance(oloc);
TimeZone[] loctzs = oloc.getLocaleTimeZones();
String [] disptz = new string [loctzs.length];
for (int i=0; i<loctzs.length; ++i)
{
    disptz [i]= odloc.getDisplayTimeZone(loctzs[i]);
    ...
}

```

8.7.8 Using the GDK with E-Mail Programs

You can use the GDK `LocaleMapper` class to retrieve the most commonly used e-mail character set. Call `LocaleMapper.getIANASCharSetFromLocale`, passing in the locale object. The return value is an array of character set names. The first character set returned is the most commonly used e-mail character set.

The following example illustrates sending an e-mail message containing Simplified Chinese data in the GBK character set encoding.

Example 8-5 Sending E-mail Containing Simplified Chinese Data in GBK Character Set Encoding

```

import oracle.i18n.util.LocaleMapper;
import java.util.Date;
import java.util.Locale;

```

```

import java.util.Properties;
import javax.mail.Message;
import javax.mail.Session;
import javax.mail.Transport;
import javax.mail.internet.InternetAddress;
import javax.mail.internet.MimeMessage;
import javax.mail.internet.MimeUtility;
/**
 * Email send operation sample
 *
 * javac -classpath orai18n.jar:j2ee.jar EmailSampleText.java
 * java -classpath .:orai18n.jar:j2ee.jar EmailSampleText
 */
public class EmailSampleText
{
    public static void main(String[] args)
    {
        send("localhost",                // smtp host name
            "your.address@yourcompany.com", // from email address
            "You",                        // from display email
            "somebody@somecompany.com",   // to email address
            "Subject test zh CN",         // subject
            "Content ~4E02 from Text email", // body
            new Locale("zh", "CN")       // user locale
        );
    }
    public static void send(String smtp, String fromEmail, String fromDispName,
        String toEmail, String subject, String content, Locale locale
    )
    {
        // get the list of common email character sets
        final String[] charset = LocaleMapper.getIANACHarSetFromLocale(LocaleMapper.
EMAIL_WINDOWS,
        locale
    );
        // pick the first one for the email encoding
        final String contentType = "text/plain; charset=" + charset[0];
        try
        {
            Properties props = System.getProperties();
            props.put("mail.smtp.host", smtp);
            // here, set username / password if necessary
            Session session = Session.getDefaultInstance(props, null);
            MimeMessage mimeMessage = new MimeMessage(session);
            mimeMessage.setFrom(new InternetAddress(fromEmail, fromDispName,
                charset[0]
            ));
            mimeMessage.setRecipients(Message.RecipientType.TO, toEmail);
            mimeMessage.setSubject(MimeUtility.encodeText(subject, charset[0], "Q"));
            // body
            mimeMessage.setContent(content, contentType);
            mimeMessage.setHeader("Content-Type", contentType);
            mimeMessage.setHeader("Content-Transfer-Encoding", "8bit");
            mimeMessage.setSentDate(new Date());
            Transport.send(mimeMessage);
        }
        catch (Exception e)
        {
            e.printStackTrace();
        }
    }
}

```

```
}  
}
```

8.8 The GDK Application Configuration File

The GDK application configuration file dictates the behavior and the properties of the GDK application framework and the application that is using it. It contains locale mapping tables and parameters for the configuration of the application. One configuration file is required for each application.

The `gdkapp.xml` application configuration file is an XML document. This file resides in the `./WEB-INF` directory of the J2EE environment of the application.

The following sections describe the contents and the properties of the application configuration file in detail:

- [locale-charset-maps](#)
- [page-charset](#)
- [application-locales](#)
- [locale-determine-rule](#)
- [locale-parameter-name](#)
- [message-bundles](#)
- [url-rewrite-rule](#)
- [Example: GDK Application Configuration File](#)

8.8.1 locale-charset-maps

This section enables applications to override the mapping from the language to the default character set provided by the GDK. This mapping is used when the `page-charset` is set to `AUTO-CHARSET`.

For example, for the `en` locale, the default GDK character set is `windows-1252`. However, if the application requires `ISO-8859-1`, this can be specified as follows:

```
<locale-charset-maps>  
  <locale-charset>  
    <locale>en</locale>  
    <charset>ISO_8859-1</charset>  
  </locale-charset>  
</locale-charset-maps>
```

The locale name is comprised of the language code and the country code, and they should follow the ISO naming convention as defined in ISO 639 and ISO 3166, respectively. The character set name follows the IANA convention.

Optionally, the `user-agent` parameter can be specified in the mapping table to distinguish different clients as follows:

```
<locale-charset>  
  <locale>en,de</locale>  
  <user-agent>^Mozilla/4.0</user-agent>  
  <charset>ISO-8859-1</charset>  
</locale-charset>
```

The previous example shows that if the user-agent value in the HTTP header starts with Mozilla/4.0 (which indicates an older version of Web clients) for English (en) and German (de) locales, then the GDK sets the character set to ISO-8859-1.

Multiple locales can be specified in a comma-delimited list.

See Also

["page-charset"](#)

8.8.2 page-charset

This tag section defines the character set of the application pages. If this is explicitly set to a given character set, then all pages use this character set. The character set name must follow the IANA character set convention, for example:

```
<page-charset>UTF-8</page-charset>
```

However, if the page-charset is set to AUTO-CHARSET, then the character set is based on the default character set of the current user locale. The default character set is derived from the locale to character set mapping table specified in the application configuration file.

If the character set mapping table in the application configuration file is not available, then the character set is based on the default locale name to IANA character set mapping table in the GDK. Default mappings are derived from `OraLocaleInfo` class.

See Also

- ["locale-charset-maps"](#)
- ["Handling Non-ASCII Input and Output in the GDK Framework"](#)

8.8.3 application-locales

This tag section defines a list of the locales supported by the application. For example:

```
<application-locales>
  <locale default="yes">en-US</locale>
  <locale>de</locale>
  <locale>zh-CN</locale>
</application-locales>
```

If the language component is specified with the * country code, then all locale names with this language code qualify. For example, if de-* (the language code for German) is defined as one of the application locales, then this supports de-AT (German- Austria), de (German-Germany), de-LU (German-Luxembourg), de-CH (German-Switzerland), and even irregular locale combination such as de-CN (German-China). However, the application can be restricted to support a predefined set of locales.

It is recommended to set one of the application locales as the default application locale (by specifying `default="yes"`) so that it can be used as a fall back locale for customers who are connecting to the application with an unsupported locale.

8.8.4 locale-determine-rule

This section defines the order in which the preferred user locale is determined. The locale sources should be specified based on the scenario in the application. This section includes the following scenarios:

- Scenario 1: The GDK framework uses the accept language at all times.

```
<locale-source>
  oracle.i18n.servlet.localesource.HTTPAcceptLanguage
</locale-source>
```

- Scenario 2: By default, the GDK framework uses the accept language. After the user specifies the locale, the locale is used for further operations.

```
<locale-source>
  oracle.i18n.servlet.localesource.UserInput
</locale-source>

<locale-source>
  oracle.i18n.servlet.localesource.HTTPAcceptLanguage
</locale-source>
```

- Scenario 3: By default, the GDK framework uses the accept language. After the user is authenticated, the GDK framework uses the database locale source. The database locale source is cached until the user logs out. After the user logs out, the accept language is used again.

```
<db-locale-source
  data-source-name="jdbc/OracleCoreDS"
  locale-source-table="customer"
  user-column="customer_email"
  user-key="userid"
  language-column="nls_language"
  territory-column="nls_territory"
  timezone-column="timezone">

  oracle.i18n.servlet.localesource.DBLocaleSource

</db-locale-source>

<locale-source>
  oracle.i18n.servlet.localesource.HttpAcceptLanguage
</locale-source>
```

Note that Scenario 3 includes the predefined database locale source, `DBLocaleSource`. It enables the user profile information to be specified in the configuration file without writing a custom database locale source. In the example, the user profile table is called "customer". The columns are "customer_email", "nls_language", "nls_territory", and "timezone". They store the unique e-mail address, the Oracle name of the preferred language, the Oracle name of the preferred territory, and the time zone ID of a customer. The user-key is a mandatory attribute that specifies the attribute name used to pass the user ID from the application to the GDK framework.

- Scenario 4: The GDK framework uses the accept language in the first page. When the user inputs a locale, it is cached and used until the user logs into the application. After the user is authenticated, the GDK framework uses the database locale source. The database locale source is cached until the user logs out. After the user logs out, the accept language is used again or the user input is used if the user inputs a locale.

```
<locale-source>
  demo.DatabaseLocaleSource
</locale-source>
<locale-source>
```



```

        oracle.i18n.servlet.localesource.UserInput
    </locale-source>
    <locale-source>
        oracle.i18n.servlet.localesource.HttpAcceptLanguage
    </locale-source>

```

Note that Scenario 4 uses the custom database locale source. If the user profile schema is complex, such as user profile information separated into multiple tables, then the custom locale source should be provided by the application. Examples of custom locale sources can be found in the \$ORACLE_HOME/nls/gdk/demo directory.

8.8.5 locale-parameter-name

This tag defines the name of the locale parameters that are used in the user input so that the current user locale can be passed between requests.

[Table 8-3](#) shows the parameters used in the GDK framework.

Table 8-3 Locale Parameters Used in the GDK Framework

Default Parameter Name	Value
locale	ISO locale where ISO 639 language code and ISO 3166 country code are connected with an underscore (_) or a hyphen (-). For example, zh_CN for Simplified Chinese used in China.
language	Oracle language name. For example, AMERICAN for American English.
territory	Oracle territory name. For example, SPAIN.
timezone	Time zone name. For example, American/Los_Angeles.
iso-currency	ISO 4217 currency code. For example, EUR for the euro.
date-format	Date format pattern mask. For example, DD_MON_RRRR.
long-date-format	Long date format pattern mask. For example, DAY-YYY-MM-DD.
date-time-format	Date and time format pattern mask. For example, DD-MON-RRRR HH24:MI:SS.
long-date-time-format	Long date and time format pattern mask. For example, DAY YYYY-MM-DD HH12:MI:SS AM.
time-format	Time format pattern mask. For example, HH:MI:SS.
number-format	Number format. For example, 9G99G990D00.
currency-format	Currency format. For example, L9G99G990D00.
linguistic-sorting	Linguistic sort order name. For example, JAPANESE_M for Japanese multilingual sort.
charset	Character set. For example, WE8ISO8859P15.
writing-direction	Writing direction string. For example, LTR for left-to-right writing direction or RTL for right-to-left writing direction.
command	GDK command. For example, store for the update operation.

The parameter names are used in either the parameter in the HTML form or in the URL.

8.8.6 message-bundles

This tag defines the base class names of the resource bundles used in the application. The mapping is used in the `Localizer.getMessage` method for locating translated text in the resource bundles.

```
<message-bundles>
  <resource-bundle>Messages</resource-bundle>
  <resource-bundle name="newresource">NewMessages</resource-bundle>
</message-bundles>
```

If the name attribute is not specified or if it is specified as name="default" to the <resource-bundle> tag, then the corresponding resource bundle is used as the default message bundle. To support more than one resource bundle in an application, resource bundle names must be assigned to the nondefault resource bundles. The nondefault bundle names must be passed as a parameter of the getMessage method.

For example:

```
Localizer loc = ServletHelper.getLocalizerInstance(request);
String translatedMessage = loc.getMessage("Hello");
String translatedMessage2 = loc.getMessage("World", "newresource");
```

8.8.7 url-rewrite-rule

This tag is used to control the behavior of the URL rewrite operations. The rewriting rule is a regular expression.

```
<url-rewrite-rule fallback="no">
  <pattern>(.*)([/]+)$</pattern>
  <result>$1/$L/$2</result>
</url-rewrite-rule>
```

① See Also

["Managing Localized Content in the GDK"](#)

If the localized content for the requested locale is not available, then it is possible for the GDK framework to trigger the locale fallback mechanism by mapping it to the closest translation locale. By default, the fallback option is turned off. This can be turned on by specifying fallback="yes".

For example, suppose an application supports only the following translations: en, de, and ja, and en is the default locale of the application. If the current application locale is de-US, then it falls back to de. If the user selects zh-TW as its application locale, then it falls back to en.

A fallback mechanism is often necessary if the number of supported application locales is greater than the number of the translation locales. This usually happens if multiple locales share one translation. One example is Spanish. The application may need to support multiple Spanish-speaking countries and not just Spain, with one set of translation files.

Multiple URL rewrite rules can be specified by assigning the name attribute to nondefault URL rewrite rules. To use the nondefault URL rewrite rules, the name must be passed as a parameter of the rewrite URL method. For example:

```


```

The first rule changes the "images/welcome.gif" URL to the localized welcome image file. The second rule named "flag" changes the "US.gif" URL to the user's country flag image file. The rule definition should be as follows:

```
<url-rewrite-rule fallback="yes">
  <pattern>(.*)/([^\/]*)$</pattern>
  <result>$1/$L/$2</result>
</url-rewrite-rule>
<url-rewrite-rule name="flag">
  <pattern>US.gif</pattern>
  <result>$C.gif</result>
</url-rewrite-rule>
```

8.8.8 Example: GDK Application Configuration File

This section contains an example of an application configuration file with the following application properties:

- The application supports the following locales: Arabic (ar), Greek (el), English (en), German (de), French (fr), Japanese (ja) and Simplified Chinese for China (zh-CN).
- English is the default application locale.
- The page character set for the ja locale is always UTF-8.
- The page character set for the en and de locales when using an Internet Explorer client is windows-1252.
- The page character set for the en, de, and fr locales on other web browser clients is iso-8859-1.
- The page character sets for all other locales are the default character set for the locale.
- The user locale is determined by the following order: user input locale and then Accept-Language.
- The localized contents are stored in their appropriate language subfolders. The folder names are derived from the ISO 639 language code. The folders are located in the root directory of the application. For example, the Japanese file for /shop/welcome.jpg is stored in /ja/shop/welcome.jpg.

```
<?xml version="1.0" encoding="utf-8"?>
<gdkapp
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:noNamespaceSchemaLocation="gdkapp.xsd">
  <!-- Language to Character set mapping -->
  <locale-charset-maps>
    <locale-charset>
      <locale>ja</locale>
      <charset>UTF-8</charset>
    </locale-charset>
    <locale-charset>
      <locale>en,de</locale>
      <user-agent>^Mozilla\[0-9\. ]+\(compatible; MSIE [^;]+; \)
      </user-agent>
      <charset>WINDOWS-1252</charset>
    </locale-charset>
    <locale-charset>
      <locale>en,de,fr</locale>
      <charset>ISO-8859-1</charset>
    </locale-charset>
  </locale-charset-maps>

  <!-- Application Configurations -->
  <page-charset>AUTO-CHARSET</page-charset>
  <application-locales>
    <locale>ar</locale>
    <locale>de</locale>
    <locale>fr</locale>
    <locale>ja</locale>
```

```
<locale>el</locale>
<locale default="yes">en</locale>
<locale>zh-CN</locale>
</application-locales>
<locale-determine-rule>
  <locale-source>
    oracle.i18n.servlet.localesource.UserInput
  </locale-source>
  <locale-source>
    oracle.i18n.servlet.localesource.HttpAcceptLanguage
  </locale-source>
</locale-determine-rule>
<!-- URL rewriting rule -->
<url-rewrite-rule fallback="no">
  <pattern>(.*)/([^\/]*)$</pattern>
  <result>$/L/$1/$2</result>
</url-rewrite-rule>
</gdkapp>
```

8.9 GDK for Java Supplied Packages and Classes

Oracle Globalization Services for Java contains the following packages:

- [oracle.i18n.lcsd](#)
- [oracle.i18n.net](#)
- [oracle.i18n.servlet](#)
- [oracle.i18n.text](#)
- [oracle.i18n.util](#)

See Also

Oracle Globalization Development Kit Java API Reference

8.9.1 oracle.i18n.lcsd

Package `oracle.i18n.lcsd` provides classes to automatically detect and recognize language and character set based on text input. It supports the detection of both plain text and HTML files. Language is based on ISO; encoding is based on IANA or Oracle character sets. It includes the following classes:

- `LCSDetector`: Contains methods to automatically detect and recognize language and character set based on text input.
- `LCSDResultSet`: Stores the result generated by `LCSDetector`. Methods in this class can be used to retrieve specific information from the result.
- `LCSDetectionInputStream`: Transparently detects the language and encoding for a stream object.
- `LCSDetectionReader`: Transparently detects the language and encoding, and converts the input data to Unicode.
- `LCSDetectionHTMLInputStream`: Extends the `LCSDetectionInputStream` class to support the language and encoding detection for input in HTML format.
- `LCSDetectionHTMLReader`: Extends the `LCSDetectionReader` class to support the language and encoding detection for input in HTML format.

8.9.1.1 LCSScan

The Language and Character Set File Scanner (Java Version) is a statistically-based utility for determining the language and character set for unknown file text. Its functionality and usage are similar to the Language and Character Set File Scanner of the "C" Version.

See Also

["The Language and Character Set File Scanner"](#)

8.9.1.1.1 Syntax of the LCSScan Command

Usage: java LCSScan <options>

Example: java LCSScan FILE=test.txt RESULTS=3 SIZE=10000

Keyword	Description	(Default)
-----	-----	-----
RESULTS	number of language and character set pairs to return 1..3	1
SIZE	sampling size of the file in bytes	8192
FORMAT	file format TEXT or HTML	TEXT
RATIO	show result ratio YES or NO	NO
FILE	name of input file	
HELP	show help screen	this screen

8.9.1.1.2 Examples of Using LCSScan

Make sure that the `orai18n-lcsd.jar` and `orai18n-mapping.jar` files are in the CLASSPATH.

Example 8-6 Specifying the File Name in the LCSScan Command

```
java oracle/i18n/lcsd/LCSScan FILE=example.txt
```

In this example, 8192 bytes of `example.txt` file is scanned. One language and character set pair will be returned.

Example 8-7 Specifying the File Name and Sampling Size in the LCSScan Command

```
java oracle/i18n/lcsd/LCSScan FILE=example.txt SIZE=4096
```

In this example, 4096 bytes of `example.txt` file is scanned. One language and character set pair will be returned.

Example 8-8 Specifying the File Name and Number of Language and Character Set Pairs in the LCSScan Command

```
java oracle/i18n/lcsd/LCSScan FILE=example.txt RESULTS=3
```

In this example, 8192 bytes of `example.txt` file is scanned. Three language and character set pairs will be returned.

Example 8-9 Specifying the File Name and Show Result Ratio in the LCSScan Command

```
java oracle/i18n/lcsd/LCSScan FILE=example.txt RATIO=YES
```

In this example, 8192 bytes of `example.txt` file is scanned. One language and character set pair will be returned with the result ratio.

Example 8-10 Specifying the File Name and Format as HTML

```
java oracle.i18n.lcsd.LCSScan FILE=example.html FORMAT=html
```

In this example, 8192 bytes of `example.html` file is scanned. The scan will strip HTML tags before the scan, thus results are more accurate. One language and character set pair will be returned.

8.9.2 oracle.i18n.net

Package `oracle.i18n.net` provides Internet-related data conversions for globalization. It includes the following classes:

- `CharEntityReference`: A utility class to escape or unescape a string into character reference or entity reference form.
- `CharEntityReference.Form`: A form parameter class that specifies the escaped form.

8.9.3 oracle.i18n.servlet

Package `oracle.i18n.Servlet` enables JSP and `JavaServlet` to have automatic locale support and also returns the localized contents to the application. It includes the following classes:

- `ApplicationContext`: Performs application scope operations in the framework.
- `Localizer`: Provides access to the most commonly used globalization information.
- `ServletHelper`: Bridges Java servlets and globalization objects.

8.9.4 oracle.i18n.text

Package `oracle.i18n.text` provides general text data globalization support. It includes the following classes:

- `OraCollationKey`: Represents a `String` under certain rules of a specific `OraCollator` object.
- `OraCollator`: Performs locale-sensitive string comparison, including linguistic collation and binary sorting.
- `OraDateFormat`: Performs formatting and parsing between datetime and string locale. It supports Oracle datetime formatting behavior.
- `OraDecimalFormat`: Performs formatting and parsing between number and string locale. It supports Oracle number formatting behavior.
- `OraDecimalFormatSymbol`: Contains Oracle format symbols used by Oracle number and currency formatting.
- `OraNumberFormat`: Performs formatting and parsing between number and string locale. It supports Oracle number formatting behavior.
- `OraSimpleDateFormat`: Performs formatting and parsing between datetime and string locale. It supports Oracle datetime formatting behavior.

8.9.5 oracle.i18n.util

Package `oracle.i18n.util` provides general utilities for globalization support. It includes the following classes:

- `LocaleMapper`: Provides mappings between Oracle locale elements and equivalent locale elements in other vendors and standards.
- `OraDisplayLocaleInfo`: Provides translations of locale and attributes.
- `OraLocaleInfo`: Contains the language, territory, and collator objects.
- `OraSQLUtil`: Provides useful methods for dealing with SQL.

8.10 GDK for PL/SQL Supplied Packages

The GDK for PL/SQL includes the following PL/SQL packages:

- `UTL_I18N`
- `UTL_LMS`

`UTL_I18N` is a set of PL/SQL services that help developers to build globalized applications. The `UTL_I18N` PL/SQL package provides the following functions:

- String conversion functions for various data types
- Escape and unescape sequences for predefined characters and multibyte characters used by HTML and XML documents
- Functions that map between Oracle, Internet Assigned Numbers Authority (IANA), ISO, and e-mail application character sets, languages, and territories
- A function that returns the Oracle character set name from an Oracle language name
- A function that performs script transliteration
- Functions that return the ISO currency code, local time zones, and local languages supported for a given territory
- Functions that return the most commonly used linguistic sort, a listing of all applicable linguistic sorts, and the local territories supported for a given language
- Functions that map between Oracle full and short language names
- A function that returns the language translation of a given language and territory name
- A function that returns a listing of the most commonly used time zones
- A function that returns the maximum number of bytes for a character of an Oracle character set
- Functions that validate the character encoding of `VARCHAR2`, `NVARCHAR2`, `CLOB`, and `NCLOB` data
- Procedures that detect the most likely Oracle character sets and Oracle languages for a given data sample.

`UTL_LMS` retrieves and formats error messages in different languages.

① See Also

"`UTL_I18N`" and "`UTL_LMS`" in *Oracle AI Database PL/SQL Packages and Types Reference*

8.11 GDK Error Messages

GDK-03001 Invalid or unsupported sorting rule

Cause: An invalid or unsupported sorting rule name was specified.

Action: Choose a valid sorting rule name and check the Globalization Support Guide for the list of sorting rule names.

GDK-03002 The functional-driven sort is not supported.

Cause: A functional-driven sorting rule name was specified.

Action: Choose a valid sorting rule name and check the Globalization Support Guide for the list of sorting rule names.

GDK-03003 The linguistic data file is missing.

Cause: A valid sorting rule was specified, but the associated data file was not found.

Action: Make sure the GDK jar files are correctly installed in the Java application.

GDK-03005 Binary sort is not available for the specified character set .

Cause: Binary sorting for the specified character set is not supported.

Action: Check the Globalization Support Guide for a character set that supports binary sort.

GDK-03006 The comparison strength level setting is invalid.

Cause: An invalid comparison strength level was specified.

Action: Choose a valid comparison strength level from the list -- PRIMARY, SECONDARY or TERTIARY.

GDK-03007 The composition level setting is invalid.

Cause: An invalid composition level setting was specified.

Action: Choose a valid composition level from the list -- NO_COMPOSITION or CANONICAL_COMPOSITION.

GDK-04001 Cannot map Oracle character to Unicode

Cause: The program attempted to use a character in the Oracle character set that cannot be mapped to Unicode.

Action: Write a separate exception handler for the invalid character, or call the withReplacement method so that the invalid character can be replaced with a valid replacement character.

GDK-04002 Cannot map Unicode to Oracle character

Cause: The program attempted to use an Unicode character that cannot be mapped to a character in the Oracle character set.

Action: Write a separate exception handler for the invalid character, or call the withReplacement method so that the invalid character can be replaced with a valid replacement character.

GDK-05000 A literal in the date format is too large.

Cause: The specified string literal in the date format was too long.

Action: Use a shorter string literal in the date format.

GDK-05001 The date format is too long for internal buffer.

Cause: The date format pattern was too long.

Action: Use a shorter date format pattern.

GDK-05002 The Julian date is out of range.

Cause: An invalid date range was specified.

Action: Make sure that date is in the specified range 0 - 3439760.

GDK-05003 Failure in retrieving date/time

Cause: This is an internal error.

Action: Contact Oracle Support Services.

GDK-05010 Duplicate format code found

Cause: The same format code was used more than once in the format pattern.

Action: Remove the redundant format code.

GDK-05011 The Julian date precludes the use of the day of the year.

Cause: Both the Julian date and the day of the year were specified.

Action: Remove either the Julian date or the day of the year.

GDK-05012 The year may only be specified once.

Cause: The year format code appeared more than once.

Action: Remove the redundant year format code.

GDK-05013 The hour may only be specified once.

Cause: The hour format code appeared more than once.

Action: Remove the redundant hour format code.

GDK-05014 The AM/PM conflicts with the use of A.M./P.M.

Cause: AM/PM was specified along with A.M./P.M.

Action: Use either AM/PM or A.M./P.M.; do not use both.

GDK-05015 The BC/AD conflicts with the use of B.C./A.D.

Cause: BC/AD was specified along with B.C./A.D.

Action: Use either BC/AD or B.C./A.D.; do not use both.

GDK-05016 Duplicate month found

Cause: The month format code appeared more than once.

Action: Remove the redundant month format code.

GDK-05017 The day of the week may only be specified once.

Cause: The day of the week format code appeared more than once.

Action: Remove the redundant day of the week format code.

GDK-05018 The HH24 precludes the use of meridian indicator.

Cause: HH24 was specified along with the meridian indicator.

Action: Use either the HH24 or the HH12 with the meridian indicator.

GDK-05019 The signed year precludes the use of BC/AD.

Cause: The signed year was specified along with BC/AD.

Action: Use either the signed year or the unsigned year with BC/AD.

GDK-05020 A format code cannot appear in a date input format.

Cause: A format code appeared in a date input format.

Action: Remove the format code.

GDK-05021 Date format not recognized

Cause: An unsupported format code was specified.

Action: Correct the format code.

GDK-05022 The era format code is not valid with this calendar.

Cause: An invalid era format code was specified for the calendar.

Action: Remove the era format code or use another calendar that supports the era.

GDK-05030 The date format pattern ends before converting entire input string.

Cause: An incomplete date format pattern was specified.

Action: Rewrite the format pattern to cover the entire input string.

GDK-05031 The year conflicts with the Julian date.

Cause: An incompatible year was specified for the Julian date.

Action: Make sure that the Julian date and the year are not in conflict.

GDK-05032 The day of the year conflicts with the Julian date.

Cause: An incompatible day of year was specified for the Julian date.

Action: Make sure that the Julian date and the day of the year are not in conflict.

GDK-05033 The month conflicts with the Julian date.

Cause: An incompatible month was specified for the Julian date.

Action: Make sure that the Julian date and the month are not in conflict.

GDK-05034 The day of the month conflicts with the Julian date.

Cause: An incompatible day of the month was specified for the Julian date.

Action: Make sure that the Julian date and the day of the month are not in conflict.

GDK-05035 The day of the week conflicts with the Julian date.

Cause: An incompatible day of the week was specified for the Julian date.

Action: Make sure that the Julian date and the day of week are not in conflict.

GDK-05036 The hour conflicts with the seconds in the day.

Cause: The specified hour and the seconds in the day were not compatible.

Action: Make sure the hour and the seconds in the day are not in conflict.

GDK-05037 The minutes of the hour conflicts with the seconds in the day.

Cause: The specified minutes of the hour and the seconds in the day were not compatible.

Action: Make sure the minutes of the hour and the seconds in the day are not in conflict.

GDK-05038 The seconds of the minute conflicts with the seconds in the day.

Cause: The specified seconds of the minute and the seconds in the day were not compatible.

Action: Make sure the seconds of the minute and the seconds in the day are not in conflict.

GDK-05039 Date not valid for the month specified

Cause: An invalid date for the month was specified.

Action: Check the date range for the month.

GDK-05040 Input value not long enough for the date format

Cause: Too many format codes were specified.

Action: Remove unused format codes or specify a longer value.

GDK-05041 A full year must be between -4713 and +9999, and not be 0.

Cause: An invalid year was specified.

Action: Specify the year in the specified range.

GDK-05042 A quarter must be between 1 and 4.

Cause: An invalid quarter was specified.

Action: Action: Make sure that the quarter is in the specified range.

GDK-05043 Not a valid month

Cause: An invalid month was specified.

Action: Make sure that the month is between 1 and 12 or has a valid month name.

GDK-05044 The week of the year must be between 1 and 52.

Cause: An invalid week of the year was specified.

Action: Make sure that the week of the year is in the specified range.

GDK-05045 The week of the month must be between 1 and 5.

Cause: An invalid week of the month was specified.

Action: Make sure that the week of the month is in the specified range.

GDK-05046 Not a valid day of the week

Cause: An invalid day of the week was specified.

Action: Make sure that the day of the week is between 1 and 7 or has a valid day name.

GDK-05047 A day of the month must be between 1 and the last day of the month.

Cause: An invalid day of the month was specified.

Action: Make sure that the day of the month is in the specified range.

GDK-05048 A day of year must be between 1 and 365 (366 for leap year).

Cause: An invalid day of the year was specified.

Action: Make sure that the day of the year is in the specified range.

GDK-05049 An hour must be between 1 and 12.

Cause: An invalid hour was specified.

Action: Make sure that the hour is in the specified range.

GDK-05050 An hour must be between 0 and 23.

Cause: An invalid hour was specified.

Action: Make sure that the hour is in the specified range.

GDK-05051 A minute must be between 0 and 59.

Cause: An invalid minute was specified.

Action: Action: Make sure the minute is in the specified range.

GDK-05052 A second must be between 0 and 59.

Cause: An invalid second was specified.

Action: Make sure the second is in the specified range.

GDK-05053 A second in the day must be between 0 and 86399.

Cause: An invalid second in the day was specified.

Action: Make sure second in the day is in the specified range.

GDK-05054 The Julian date must be between 1 and 5373484.

Cause: An invalid Julian date was specified.

Action: Make sure that the Julian date is in the specified range.

GDK-05055 Missing AM/A.M. or PM/P.M.

Cause: Neither AM/A.M. nor PM/P.M. was specified in the format pattern.

Action: Specify either AM/A.M. or PM/P.M.

GDK-05056 Missing BC/B.C. or AD/A.D.

Cause: Neither BC/B.C. nor AD/A.D. was specified in the format pattern.

Action: Specify either BC/B.C. or AD/A.D.

GDK-05057 Not a valid time zone

Cause: An invalid time zone was specified.

Action: Specify a valid time zone.

GDK-05058 Non-numeric character found

Cause: A non-numeric character was found where a numeric character was expected.

Action: Make sure that the character is a numeric character.

GDK-05059 Non-alphabetic character found

Cause: A non-alphabetic character was found where an alphabetic was expected.

Action: Make sure that the character is an alphabetic character.

GDK-05060 The week of the year must be between 1 and 53.

Cause: An invalid week of the year was specified.

Action: Make sure that the week of the year is in the specified range.

GDK-05061 The literal does not match the format string.

Cause: The string literals in the input were not the same length as the literals in the format pattern (with the exception of the leading whitespace).

Action: Correct the format pattern to match the literal. If the "FX" modifier has been toggled on, the literal must match exactly, with no extra whitespace.

GDK-05062 The numeric value does not match the length of the format item.

Cause: The numeric value did not match the length of the format item.

Action: Correct the input date or turn off the FX or FM format modifier. When the FX and FM format codes are specified for an input date, then the number of digits must be exactly the number specified by the format code. For example, 9 will not match the format code DD but 09 will.

GDK-05063 The year is not supported for the current calendar.

Cause: An unsupported year for the current calendar was specified.

Action: Check the Globalization Support Guide to find out what years are supported for the current calendar.

GDK-05064 The date is out of range for the calendar.

Cause: The specified date was out of range for the calendar.

Action: Specify a date that is legal for the calendar.

GDK-05065 Invalid era

Cause: An invalid era was specified.

Action: Make sure that the era is valid.

GDK-05066 The datetime class is invalid.

Cause: This is an internal error.

Action: Contact Oracle Support Services.

GDK-05067 The interval is invalid.

Cause: An invalid interval was specified.

Action: Specify a valid interval.

GDK-05068 The leading precision of the interval is too small.

Cause: The specified leading precision of the interval was too small to store the interval.

Action: Increase the leading precision of the interval or specify an interval with a smaller leading precision.

GDK-05069 Reserved for future use

Cause: Reserved.

Action: Reserved.

GDK-05070 The specified intervals and datetimes were not mutually comparable.

Cause: The specified intervals and datetimes were not mutually comparable.

Action: Specify a pair of intervals or datetimes that are mutually comparable.

GDK-05071 The number of seconds must be less than 60.

Cause: The specified number of seconds was greater than 59.

Action: Specify a value for the seconds to 59 or smaller.

GDK-05072 Reserved for future use

Cause: Reserved.

Action: Reserved.

GDK-05073 The leading precision of the interval was too small.

Cause: The specified leading precision of the interval was too small to store the interval.

Action: Increase the leading precision of the interval or specify an interval with a smaller leading precision.

GDK-05074 An invalid time zone hour was specified.

Cause: The hour in the time zone must be between -12 and 13.

Action: Specify a time zone hour between -12 and 13.

GDK-05075 An invalid time zone minute was specified.

Cause: The minute in the time zone must be between 0 and 59.

Action: Specify a time zone minute between 0 and 59.

GDK-05076 An invalid year was specified.

Cause: A year must be at least -4713.

Action: Specify a year that is greater than or equal to -4713.

GDK-05077 The string is too long for the internal buffer.

Cause: This is an internal error.

Action: Contact Oracle Support Services.

GDK-05078 The specified field was not found in the datetime or interval.

Cause: The specified field was not found in the datetime or interval.

Action: Make sure that the specified field is in the datetime or interval.

GDK-05079 An invalid hh25 field was specified.

Cause: The hh25 field must be between 0 and 24.

Action: Specify an hh25 field between 0 and 24.

GDK-05080 An invalid fractional second was specified.

Cause: The fractional second must be between 0 and 999999999.

Action: Specify a value for fractional second between 0 and 999999999.

GDK-05081 An invalid time zone region ID was specified.

Cause: The time zone region ID specified was invalid.

Action: Contact Oracle Support Services.

GDK-05082 Time zone region name not found

Cause: The specified region name cannot be found.

Action: Contact Oracle Support Services.

GDK-05083 Reserved for future use

Cause: Reserved.

Action: Reserved.

GDK-05084 Internal formatting error

Cause: This is an internal error.

Action: Contact Oracle Support Services.

GDK-05085 Invalid object type

Cause: An invalid object type was specified.

Action: Use a supported object type.

GDK-05086 Invalid date format style

Cause: An invalid format style was specified.

Action: Choose a valid format style.

GDK-05087 A null format pattern was specified.

Cause: The format pattern cannot be null.

Action: Provide a valid format pattern.

GDK-05088 Invalid number format model

Cause: An invalid number format code was specified.

Action: Correct the number format code.

GDK-05089 Invalid number

Cause: An invalid number was specified.

Action: Correct the input.

GDK-05090 Reserved for future use

Cause: Reserved.

Action: Reserved.

GDK-0509 Datetime/interval internal error

Cause: This is an internal error.

Action: Contact Oracle Support Services.

GDK-05098 Too many precision specifiers

Cause: Extra data was found in the date format pattern while the program attempted to truncate or round dates.

Action: Check the syntax of the date format pattern.

GDK-05099 Bad precision specifier

Cause: An invalid precision specifier was specified.

Action: Use a valid precision specifier.

GDK-05200 Missing WE8ISO8859P1 data file

Cause: The character set data file for WE8ISO8859P1 was not installed.

Action: Make sure the GDK jar files are installed properly in the Java application.

GDK-05201 Failed to convert to a hexadecimal value

Cause: An invalid hexadecimal string was included in the HTML/XML data.

Action: Make sure the string includes the hexadecimal character in the form of &x[0-9A-Fa-f]+;.

GDK-05202 Failed to convert to a decimal value

Cause: An invalid decimal string was found in the HTML/XML data.

Action: Make sure the string includes the decimal character in the form of &[0-9]+;.

GDK-05203 Unregistered character entity

Cause: An invalid character entity was found in the HTML/XML data.

Action: Use a valid character entity value in HTML/XML data. See HTML/XML standards for the registered character entities.

GDK-05204 Invalid Quoted-Printable value

Cause: An invalid Quoted-Printable data was found in the data.

Action: Make sure the input data has been encoded in the proper Quoted-Printable form.

GDK-05205 Invalid MIME header format

Cause: An invalid MIME header format was specified.

Action: Check RFC 2047 for the MIME header format. Make sure the input data conforms to the format.

GDK-05206 Invalid numeric string

Cause: An invalid character in the form of %FF was found when a URL was being decoded.

Action: Make sure the input URL string is valid and has been encoded correctly; %FF needs to be a valid hex number.

GDK-05207 Invalid class of the object, key, in the user-defined locale to charset mapping"

Cause: The class of key object in the user-defined locale to character set mapping table was not java.util.Locale.

Action: When you construct the Map object for the user-defined locale to character set mapping table, specify java.util.Locale for the key object.

GDK-05208 Invalid class of the object, value, in the user-defined locale to charset mapping

Cause: The class of value object in the user-defined locale to character set mapping table was not java.lang.String.

Action: When you construct the Map object for the user-defined locale to character set mapping table, specify `java.lang.String` for the value object.

GDK-05209 Invalid rewrite rule

Cause: An invalid regular expression was specified for the match pattern in the rewrite rule.

Action: Make sure the match pattern for the rewriting rule uses a valid regular expression.

GDK-05210 Invalid character set

Cause: An invalid character set name was specified.

Action: Specify a valid character set name.

GDK-0521 Default locale not defined as a supported locale

Cause: The default application locale was not included in the supported locale list.

Action: Include the default application locale in the supported locale list or change the default locale to the one that is in the list of the supported locales.

GDK-05212 The rewriting rule must be a String array with three elements.

Cause: The rewriting rule parameter was not a String array with three elements.

Action: Make sure the rewriting rule parameter is a String array with three elements. The first element represents the match pattern in the regular expression, the second element represents the result pattern in the form specified in the JavaDoc of `ServletHelper.rewriteURL`, and the third element represents the Boolean value "True" or "False" that specifies whether the locale fallback operation is performed or not.

GDK-05213 Invalid type for the class of the object, key, in the user-defined parameter name mapping

Cause: The class of key object in the user-defined parameter name mapping table was not `java.lang.String`.

Action: When you construct the Map object for the user-defined parameter name mapping table, specify `java.lang.String` for the key object.

GDK-05214 The class of the object, value, in the user-defined parameter name mapping, must be of type "java.lang.String".

Cause: The class of value object in the user-defined parameter name mapping table was not `java.lang.String`.

Action: When you construct the Map object for the user-defined parameter name mapping table, specify `java.lang.String` for the value object.

GDK-05215 Parameter name must be in the form [a-z][a-z0-9]*.

Cause: An invalid character was included in the parameter name.

Action: Make sure the parameter name is in the form of `[a-z][a-z0-9]*`.

GDK-05216 The attribute "var" must be specified if the attribute "scope" is set.

Cause: Despite the attribute "scope" being set in the tag, the attribute "var" was not specified.

Action: Specify the attribute "var" for the name of variable.

GDK-05217 The "param" tag must be nested inside a "message" tag.

Cause: The "param" tag was not nested inside a "message" tag.

Action: Make sure the tag "param" is inside the tag "message".

GDK-05218 Invalid \"scope\" attribute is specified.

Cause: An invalid \"scope\" value was specified.

Action: Specify a valid scope as either \"application,\" \"session,\" \"request,\" or \"page\".

GDK-05219 Invalid date format style

Cause: The specified date format style was invalid.

Action: Specify a valid date format style as either \"default,\" \"short,\" or \"long\"

GDK-05220 No corresponding Oracle character set exists for the IANA character set.

Cause: An unsupported IANA character set name was specified.

Action: Specify the IANA character set that has a corresponding Oracle character set.

GDK-05221 Invalid parameter name

Cause: An invalid parameter name was specified in the user-defined parameter mapping table.

Action: Make sure the specified parameter name is supported. To get the list of supported parameter names, call `LocaleSource.Parameter.toArray`.

GDK-05222 Invalid type for the class of the object, key, in the user-defined message bundle mapping.

Cause: The class of key object in the user-defined message bundle mapping table was not \"java.lang.String.\"

Action: When you construct the Map object for the user-defined message bundle mapping table, specify `java.lang.String` for the key object.

GDK-05223 Invalid type for the class of the object, value, in the user-defined message bundle mapping

Cause: The class of value object in the user-defined message bundle mapping table was not \"java.lang.String.\"

Action: When you construct the Map object for the user-defined message bundle mapping table, specify `java.lang.String` for the value object.

GDK-05224 Invalid locale string

Cause: An invalid character was included in the specified ISO locale names in the GDK application configuration file.

Action: Make sure the ISO locale names include only valid characters. A typical name format is an ISO 639 language followed by an ISO 3166 country connected by a dash character; for example, \"en-US\" is used to specify the locale for American English in the United States.

GDK-06001 LCSDetector profile not available

Cause: The specified profile was not found.

Action: Make sure the GDK jar files are installed properly in the Java application.

GDK-06002 Invalid IANA character set name or no corresponding Oracle name found

Cause: The IANA character set specified was either invalid or did not have a corresponding Oracle character set.

Action: Check that the IANA character is valid and make sure that it has a corresponding Oracle character set.

GDK-06003 Invalid ISO language name or no corresponding Oracle name found

Cause: The ISO language specified was either invalid or did not have a corresponding Oracle language.

Action: Check to see that the ISO language specified is valid and has a corresponding Oracle language.

GDK-06004 A character set filter and a language filter cannot be set at the same time.

Cause: A character set filter and a language filter were set at the same time in a LCSDetector object.

Action: Set only one of the two -- character set or language.

GDK-06005 Reset is necessary before LCSDetector can work with a different data source.

Cause: The reset method was not invoked before a different type of data source was used for a LCSDetector object.

Action: Call LCSDetector.reset to reset the detector before switching to detect other types of data source.

ORA-17154 Cannot map Oracle character to Unicode

Cause: The Oracle character was either invalid or incomplete and could not be mapped to an Unicode value.

Action: Write a separate exception handler for the invalid character, or call the withReplacement method so that the invalid character can be replaced with a valid replacement character.

ORA-17155 Cannot map Unicode to Oracle character

Cause: The Unicode character did not have a counterpart in the Oracle character set.

Action: Write a separate exception handler for the invalid character, or call the withReplacement method so that the invalid character can be replaced with a valid replacement character.

9

SQL and PL/SQL Programming in a Global Environment

This chapter contains information useful for SQL programming in a globalization support environment. This chapter includes the following topics:

- [Locale-Dependent SQL Functions with Optional NLS Parameters](#)
- [Other Locale-Dependent SQL Functions](#)
- [Miscellaneous Topics for SQL and PL/SQL Programming in a Global Environment](#)

9.1 Locale-Dependent SQL Functions with Optional NLS Parameters

NLS parameters can be specified for all SQL functions whose behavior depends on globalization support conventions. These functions are:

TO_CHAR
TO_DATE
TO_NUMBER
NLS_UPPER
NLS_LOWER
NLS_INITCAP
NLSSORT

Explicitly specifying the optional NLS parameters for these functions enables the functions to be evaluated independently of the session's NLS parameters. This feature can be important for SQL statements that contain numbers and dates as string literals.

For example, the following query is evaluated correctly if the language specified for dates is AMERICAN:

```
SELECT last_name FROM employees WHERE hire_date > '01-JAN-2005';
```

Such a query can be made independent of the current date language by using a statement similar to the following:

```
SELECT last_name FROM employees
WHERE hire_date > TO_DATE('01-JAN-2005', 'DD-MON-YYYY',
'NLS_DATE_LANGUAGE = AMERICAN');
```

In this way, SQL statements that are independent of the session language can be defined where necessary. Such statements are necessary when string literals appear in SQL statements in views, CHECK constraints, or triggers.

Note

Only SQL statements that must be independent of the session NLS parameter values should explicitly specify optional NLS parameters in locale-dependent SQL functions. Using session default values for NLS parameters in SQL functions usually results in better performance.

All character functions support both single-byte and multibyte characters. Except where explicitly stated, character functions operate character by character, rather than byte by byte.

The rest of this section includes the following topics:

- [Default Values for NLS Parameters in SQL Functions](#)
- [Specifying NLS Parameters in SQL Functions](#)
- [Unacceptable NLS Parameters in SQL Functions](#)

9.1.1 Default Values for NLS Parameters in SQL Functions

When SQL functions evaluate views and triggers, default values from the current session are used for the NLS function parameters. When SQL functions evaluate CHECK constraints, they use the default values that were specified for the NLS parameters when the database was created.

9.1.2 Specifying NLS Parameters in SQL Functions

NLS parameters are specified in SQL functions as follows:

```
'parameter = value'
```

For example:

```
'NLS_DATE_LANGUAGE = AMERICAN'
```

The following NLS parameters can be specified in SQL functions:

```
NLS_DATE_LANGUAGE  
NLS_NUMERIC_CHARACTERS  
NLS_CURRENCY  
NLS_ISO_CURRENCY  
NLS_DUAL_CURRENCY  
NLS_CALENDAR  
NLS_SORT
```

[Table 9-1](#) shows which NLS parameters are valid for specific SQL functions.

Table 9-1 SQL Functions and Their Valid NLS Parameters

SQL Functions	Valid NLS Parameters
TO_DATE	NLS_DATE_LANGUAGE NLS_CALENDAR
TO_NUMBER	NLS_NUMERIC_CHARACTERS NLS_CURRENCY NLS_DUAL_CURRENCY NLS_ISO_CURRENCY
TO_CHAR, TO_NCHAR	NLS_DATE_LANGUAGE NLS_NUMERIC_CHARACTERS NLS_CURRENCY NLS_ISO_CURRENCY NLS_DUAL_CURRENCY NLS_CALENDAR
NLS_UPPER, NLS_LOWER, NLS_INITCAP, NLSSORT	NLS_SORT

The following examples show how to use NLS parameters in SQL functions:

```

TO_DATE ('1-JAN-99', 'DD-MON-YY',
        'nls_date_language = American')

TO_CHAR (hire_date, 'DD/MON/YYYY',
        'nls_date_language = French')

TO_CHAR (SYSDATE, 'DD/MON/YYYY',
        'nls_date_language='Traditional Chinese' ')

TO_NUMBER ('13.000,00', '99G999D99',
        'nls_numeric_characters = ','.')

TO_CHAR (salary, '9G999D99L', 'nls_numeric_characters = ','.'
        nls_currency = 'EUR')

TO_CHAR (salary, '9G999D99C', 'nls_numeric_characters = ','.'
        nls_iso_currency = Japan')

NLS_UPPER (last_name, 'nls_sort = Swiss')

NLSSORT (last_name, 'nls_sort = German')

```

Note

In some languages, some lowercase characters correspond to more than one uppercase character or vice versa. As a result, the length of the output from the NLS_UPPER, NLS_LOWER, and NLS_INITCAP functions can differ from the length of the input.

See Also

- ["Special Uppercase Letters"](#)
- ["Special Lowercase Letters"](#)

9.1.3 Unacceptable NLS Parameters in SQL Functions

The following NLS parameters are not accepted in SQL functions except for NLSSORT:

NLS_LANGUAGE
NLS_TERRITORY
NLS_DATE_FORMAT

NLS_DATE_FORMAT and NLS_TERRITORY_FORMAT are not accepted as parameters because they can interfere with required format masks. A date format must always be specified if an NLS parameter is in a TO_CHAR or TO_DATE function. As a result, NLS_DATE_FORMAT and NLS_TERRITORY_FORMAT are not valid NLS parameters for the TO_CHAR or TO_DATE functions. If you specify NLS_DATE_FORMAT or NLS_TERRITORY_FORMAT in the TO_CHAR or TO_DATE function, then an error is returned.

NLS_LANGUAGE can interfere with the session value of NLS_DATE_LANGUAGE. If you specify NLS_LANGUAGE in the TO_CHAR function, for example, then its value is ignored if it differs from the session value of NLS_DATE_LANGUAGE.

9.2 Other Locale-Dependent SQL Functions

This section includes the following topics:

- [The CONVERT Function](#)
- [SQL Functions for Different Length Semantics](#)
- [LIKE Conditions for Different Length Semantics](#)
- [Character Set SQL Functions](#)
- [The NLSSORT Function](#)

9.2.1 The CONVERT Function

The CONVERT function enables conversion of character data between character sets.

The CONVERT function converts the binary representation of a character string in one character set to another. It uses exactly the same technique as conversion between database and client character sets. Hence, it uses replacement characters and has the same limitations.

See Also
["Character Set Conversion Between Clients and the Server"](#)

The syntax for CONVERT is as follows:

```
CONVERT(char, dest_char_set[, source_char_set])
```

char is the value to be converted. *source_char_set* is the source character set and *dest_char_set* is the destination character set. If the *source_char_set* parameter is not specified, then it defaults to the database character set.

- See Also**
- *Oracle AI Database SQL Language Reference* for more information about the CONVERT function
 - ["Character Set Conversion Support"](#) for character set encodings that are used only for the CONVERT function

9.2.2 SQL Functions for Different Length Semantics

Oracle provides SQL functions that work in accordance with different length semantics. There are three groups of such SQL functions: SUBSTR, LENGTH, and INSTR. Each function in a group is based on a different kind of length semantics and is distinguished by the character or number appended to the function name. For example, SUBSTRB is based on byte semantics.

The SUBSTR functions return a requested portion of a substring. The LENGTH functions return the length of a string. The INSTR functions search for a substring in a string.

The SUBSTR functions calculate the length of a string differently. [Table 9-2](#) summarizes the calculation methods.

Table 9-2 How the SUBSTR Functions Calculate the Length of a String

Function	Calculation Method
SUBSTR	Calculates the length of a string in characters based on the length semantics associated with the character set of the data type. For example, AL32UTF8 characters are calculated in UCS-4 characters. UTF8 and AL16UTF16 characters are calculated in UCS-2 characters. A supplementary character is counted as one character in AL32UTF8 and as two characters in UTF8 and AL16UTF16. Because VARCHAR and NVARCHAR2 may use different character sets, SUBSTR may give different results for different data types even if two strings are identical. If your application requires consistency, then use SUBSTR2 or SUBSTR4 to force all semantic calculations to be UCS-2 or UCS-4, respectively.
SUBSTRB	Calculates the length of a string in bytes.
SUBSTR2	Calculates the length of a string in UCS-2 characters, which is compliant with Java strings and Windows client environments. Characters are represented in UCS-2 or 16-bit Unicode values. Supplementary characters are counted as two characters.
SUBSTR4	Calculates the length of a string in UCS-4 characters. Characters are represented in UCS-4 or 32-bit Unicode values. Supplementary characters are counted as one character.

Table 9-2 (Cont.) How the SUBSTR Functions Calculate the Length of a String

Function	Calculation Method
SUBSTRC	Calculates the length of a string in Unicode composed characters. Supplementary characters and composed characters are counted as one character. Starting with Oracle AI Database 26ai, Ideographic Variation Sequences (IVS) and any other characters having the Mn Unicode general property, together with the preceding base, are also counted as one character.

The LENGTH and INSTR functions calculate string length in the same way, according to the character or number added to the function name.

The following examples demonstrate the differences between SUBSTR and SUBSTRB on a database whose character set is AL32UTF8.

For the string Fußball, the following statement returns a substring that is 4 characters long, beginning with the second character:

```
SELECT SUBSTR ('Fußball', 2 , 4) SUBSTR FROM DUAL;
```

```
SUBS
----
ußba
```

For the string Fußball, the following statement returns a substring 4 bytes long, beginning with the second byte:

```
SELECT SUBSTRB ('Fußball', 2 , 4) SUBSTRB FROM DUAL;
```

```
SUB
---
ußb
```

Starting with Oracle AI Database 26ai, the INSTRC, LENGTHC, and SUBSTRC functions count an Ideographic Variation Sequence (IVS) as one complete character.

For example, consider the following SQL statement, which returns the length of a string containing an IVS. The IVS comprises an ideographic character (U+845B) and a Variation Selector (U+E0100):

```
SELECT LENGTHC(UNISTR('My IVS: \845B\DB40\DD00')) FROM DUAL;
```

Starting with Oracle AI Database 26ai, this statement returns the number 9. In previous releases, this statement returned the number 10.

① See Also

Oracle AI Database SQL Language Reference for more information about the SUBSTR, LENGTH, and INSTR functions

9.2.3 LIKE Conditions for Different Length Semantics

The LIKE conditions specify a test that uses pattern-matching. The equality operator (=) exactly matches one character value to another, but the LIKE conditions match a portion of one character value to another by searching the first value for the pattern specified by the second.

LIKE calculates the length of strings in characters using the length semantics associated with the input character set. The LIKE2, LIKE4, and LIKEC conditions are summarized in [Table 9-3](#).

Table 9-3 LIKE Conditions

Function	Description
LIKE2	Use when characters are represented in UCS-2 semantics. A supplementary character is considered as two characters.
LIKE4	Use when characters are represented in UCS-4 semantics. A supplementary character is considered as one character.
LIKEC	Use when characters are represented in Unicode composed character semantics. Supplementary characters and composed characters are treated as one character. Starting with Oracle AI Database 26ai, Ideographic Variation Sequences (IVSs) and any other characters having the Mn Unicode general property, together with the preceding base, are also treated as one character.

There is no LIKEB condition.

Starting with Oracle AI Database 26ai, the LIKEC condition counts an Ideographic Variation Sequence (IVS) as one complete character.

For example, create a table and insert into the table a string of characters that includes one IVS. The IVS comprises an ideographic character (U+845B) and a Variation Selector (U+E0100):

```
CREATE TABLE test (col1 VARCHAR2(20));
INSERT INTO test VALUES (UNISTR('foo\845B\DB40\DD00bar'));
```

Use the following statement to obtain a count of the number of rows in that table that contain a string of the form foo_bar. Note that the underscore character (_) matches exactly one character in the LIKEC condition.

```
SELECT COUNT(*) FROM test WHERE col1 LIKEC 'foo_bar';
```

Starting with Oracle AI Database 26ai, this SELECT statement returns a count of 1. In previous releases, this statement returned a count of 0.

9.2.4 Character Set SQL Functions

Two SQL functions, NLS_CHARSET_NAME and NLS_CHARSET_ID, can convert between character set ID numbers and character set names. They are used by programs that need to determine character set ID numbers for binding variables through OCI.

Another SQL function, NLS_CHARSET_DECL_LEN, returns the declaration length of a column in number of characters, given the byte length of the column.

This section includes the following topics:

- [Converting from Character Set Number to Character Set Name](#)
- [Converting from Character Set Name to Character Set Number](#)
- [Returning the Length of an NCHAR Column](#)

📘 See Also

Oracle AI Database SQL Language Reference

9.2.4.1 Converting from Character Set Number to Character Set Name

The `NLS_CHARSET_NAME(n)` function returns the name of the character set corresponding to ID number *n*. The function returns NULL if *n* is not a recognized character set ID value.

9.2.4.2 Converting from Character Set Name to Character Set Number

`NLS_CHARSET_ID(text)` returns the character set ID corresponding to the name specified by *text*. *text* is defined as a run-time VARCHAR2 quantity, a character set name. Values for *text* can be NLSRTL names that resolve to character sets that are not the database character set or the national character set.

If the value `CHAR_CS` is entered for *text*, then the function returns the ID of the database character set. If the value `NCHAR_CS` is entered for *text*, then the function returns the ID of the database's national character set. The function returns NULL if *text* is not a recognized name.

📘 Note

The value for *text* must be entered in uppercase characters.

9.2.4.3 Returning the Length of an NCHAR Column

`NLS_CHARSET_DECL_LEN(BYTECNT, CSID)` returns the declaration length of a column in number of characters, given the byte length of the column. *BYTECNT* is the byte length of the column. *CSID* is the character set ID of the column.

9.2.5 The NLSSORT Function

The NLSSORT function enables you to force a specific collation (sort order) for ORDER BY, GROUP BY, comparison conditions, and a number of other collation-sensitive operations. However, starting with Oracle Database 12c Release 2 (12.2), the recommended way to force a specific collation for such operations is to use the COLLATE operator. The COLLATE operator works for all the collation-sensitive operations, including those for which NLSSORT cannot be used, for example MAX, MIN, and INSTR.

📘 See Also

["Expression Evaluation and the COLLATE Operator"](#)

The NLSSORT function calculates a collation key for its character argument. The collation key is a value of data type RAW, which has the following property: when two collation keys created for a given collation for two (possibly different) source character values are compared as binary, their mutual ordering corresponds to the expected mutual ordering of the source character values in this collation, that is, $NLSSORT(c1) < NLSSORT(c2)$, if and only if $c1 < c2$, where both NLSSORT and the character operator $<$ (less-than) use the same collation.

The collations used for ORDER BY, GROUP BY, comparison conditions, and other collation-sensitive operations are determined by the data-bound collation determination rules. If these rules yield a pseudo-collation, the session parameters NLS_COMP and NLS_SORT determine the actual collation.

① See Also

["Collation Determination"](#)

The following example specifies a German collation with the NLS_SORT session parameter. It assumes that the declared collation of column1 is USING_NLS_COMP or USING_NLS_SORT.

```
ALTER SESSION SET NLS_SORT = GERMAN;  
SELECT * FROM table1  
ORDER BY column1;
```

The following example first sets the NLS_SORT session parameter to German, but the NLSSORT function overrides it by specifying a French sort.

```
ALTER SESSION SET NLS_SORT = GERMAN;  
SELECT * FROM table1  
ORDER BY NLSSORT(column1, 'NLS_SORT=FRENCH');
```

The WHERE clause uses binary comparison when NLS_COMP is set to BINARY and the declared collation of referenced columns is USING_NLS_COMP. But, this can be overridden by using the NLSSORT function in the WHERE clause.

① Note

If the Data-bound Collation feature is not used, then all the columns have the declared collation of USING_NLS_COMP.

The following example makes a linguistic comparison using the WHERE clause.

```
ALTER SESSION SET NLS_COMP = BINARY;  
SELECT * FROM table1  
WHERE NLSSORT(column1, 'NLS_SORT=FRENCH') >  
NLSSORT(column2, 'NLS_SORT=FRENCH');
```

Setting the NLS_COMP session parameter to LINGUISTIC causes the NLS_SORT value to be used in the WHERE clause.

Oracle Database may add the NLSSORT function implicitly to SQL expressions in a subquery to implement linguistic behavior for a category of collation-sensitive operations. The implicitly added NLSSORT calls are visible in the execution plan for an SQL statement.

Note

The NLSSORT function, whether called explicitly or implicitly, may report error ORA-12742 under certain conditions. See "[Avoiding ORA-12742 Error](#)" for more details regarding this error.

The rest of this section contains the following topics:

- [NLSSORT Syntax](#)
- [Comparing Strings in a WHERE Clause](#)
- [Controlling an ORDER BY Clause](#)

9.2.5.1 NLSSORT Syntax

There are four ways to use NLSSORT:

- NLSSORT(), which relies on the collation determination rules
- NLSSORT(column1, 'NLS_SORT=xxxx')
- NLSSORT(column1, 'NLS_LANG=xxxx')
- NLSSORT(column1, 'NLS_LANGUAGE=xxxx')

The NLS_LANG parameter of the NLSSORT function is not the same as the NLS_LANG client environment setting. In the NLSSORT function, NLS_LANG specifies the abbreviated language name, such as US for American or PL for Polish. For example:

```
SELECT * FROM table1
ORDER BY NLSSORT(column1, 'NLS_LANG=PL');
```

When a language is specified in an NLSSORT call, the default collation for that language is used by the function.

9.2.5.2 Comparing Strings in a WHERE Clause

NLSSORT enables applications to perform string matching that follows alphabetic conventions. Normally, character strings in a WHERE clause are compared by using the binary values of the characters. One character is considered greater than another character if it has a greater binary value in the database character set. Because the sequence of characters based on their binary values might not match the alphabetic sequence for a language, such comparisons may not follow alphabetic conventions. For example, if a column (column1) contains the values ABC, ABZ, BCD, and ÄBC in the ISO 8859-1 8-bit character set, then the following query returns both BCD and ÄBC because Ä has a higher numeric value than B:

```
SELECT column1 FROM table1 WHERE column1 > 'B';
```

In German, Ä is sorted alphabetically before B, but in Swedish, Ä is sorted after Z. Linguistic comparisons can be made by using NLSSORT in the WHERE clause:

```
WHERE NLSSORT(col) comparison_operator NLSSORT(comparison_string)
```

Note that NLSSORT must be on both sides of the comparison operator. For example:

```
SELECT column1 FROM table1 WHERE NLSSORT(column1) > NLSSORT('B');
```

If a German linguistic sort has been set, then the statement does not return strings beginning with Ä because Ä comes before B in the German alphabet. If a Swedish linguistic sort has been set, then strings beginning with Ä are returned because Ä comes after Z in the Swedish alphabet.

Starting with Oracle Database 12c Release 2 (12.2), the recommended way to make the > (greater-than) operator use linguistic comparison is to add the COLLATE operator to one of the compared values. For example:

```
SELECT column1 FROM table1 WHERE column1 COLLATE USING_NLS_SORT > 'B';
```

When you want to force a particular collation, independent of the session NLS parameters, you can specify it in place of the pseudo-collation USING_NLS_SORT. For example:

```
SELECT column1 FROM table1 WHERE column1 COLLATE GERMAN > 'B';
```

Note

You will get the same result as shown in the preceding examples for the COLLATE operator, if you remove the operator COLLATE and specify the corresponding collation when declaring collation of column1 in table1.

See Also

["Specifying Data-Bound Collation for a Column"](#)

9.2.5.3 Controlling an ORDER BY Clause

If a linguistic sort is in use, then ORDER BY clauses use an implicit NLSSORT on character data. The sort mechanism (linguistic or binary) for an ORDER BY clause is transparent to the application. However, if the NLSSORT function is explicitly specified in an ORDER BY clause, then the implicit NLSSORT is not done.

If a linguistic sort has been defined by the NLS_SORT session parameter, then an ORDER BY clause in an application uses an implicit NLSSORT function. If you specify an explicit NLSSORT function, then it overrides the implicit NLSSORT function.

When the sort mechanism has been defined as linguistic, the NLSSORT function is usually unnecessary in an ORDER BY clause.

When the sort mechanism either defaults or is defined as binary, then a query like the following uses a binary sort:

```
SELECT last_name FROM employees
       ORDER BY last_name;
```

A German linguistic sort can be obtained as follows:

```
SELECT last_name FROM employees
       ORDER BY NLSSORT(last_name, 'NLS_SORT = GERMAN');
```

① See Also

["Using Linguistic Collation"](#)

9.3 Miscellaneous Topics for SQL and PL/SQL Programming in a Global Environment

This section contains the following topics:

- [SQL Date Format Masks](#)
- [Calculating Week Numbers](#)
- [SQL Numeric Format Masks](#)
- [Loading External BFILE Data into LOB Columns](#)

① See Also

Oracle AI Database SQL Language Reference for a complete description of format masks

9.3.1 SQL Date Format Masks

Several format masks are provided with the TO_CHAR, TO_DATE, and TO_NUMBER functions.

The RM (Roman Month) format element returns a month as a Roman numeral. You can specify either upper case or lower case by using RM or rm. For example, for the date 7 Sep 2007, DD-rm-YYYY returns 07-ix-2007 and DD-RM-YYYY returns 07-IX-2007.

Note that the MON and DY format masks explicitly support month and day abbreviations that may not be three characters in length. For example, the abbreviations "Lu" and "Ma" can be specified for the French "Lundi" and "Mardi", respectively.

9.3.2 Calculating Week Numbers

The week numbers returned by the WW format mask are calculated according to the following algorithm: $\text{int}(\text{dayOfYear}+6)/7$. This algorithm does not follow the ISO standard (2015, 1992-06-15).

To support the ISO standard, the IW format element is provided. It returns the ISO week number. In addition, the I, IY, IYY, and IYYY format elements, equivalent in behavior to the Y, YY, YYY, and YYYY format elements, return the year relating to the ISO week number.

In the ISO standard, the year relating to an ISO week number can be different from the calendar year. For example, 1st Jan 1988 is in ISO week number 53 of 1987. A week always starts on a Monday and ends on a Sunday. The week number is determined according the following rules:

- If January 1 falls on a Friday, Saturday, or Sunday, then the week including January 1 is the last week of the previous year, because most of the days in the week belong to the previous year.

- If January 1 falls on a Monday, Tuesday, Wednesday, or Thursday, then the week is the first week of the new year, because most of the days in the week belong to the new year.

For example, January 1, 1991, is a Tuesday, so Monday, December 31, 1990, to Sunday, January 6, 1991, is in week 1. Thus, the ISO week number and year for December 31, 1990, is 1, 1991. To get the ISO week number, use the IW format mask for the week number and one of the IY formats for the year.

9.3.3 SQL Numeric Format Masks

Several additional format elements are provided for formatting numbers:

Element	Description	Purpose
D	Decimal	Returns the decimal point character
G	Group	Returns the group separator
L	Local currency	Returns the local currency symbol
C	International currency	Returns the ISO currency symbol
RN	Roman numeral	Returns the number as its Roman numeral equivalent

For Roman numerals, you can specify either upper case or lower case, using RN or rn, respectively. The number being converted must be an integer in the range 1 to 3999.

9.3.4 Loading External BFILE Data into LOB Columns

The DBMS_LOB PL/SQL package can load external BFILE data into LOB columns. Oracle Database performs character set conversion before loading the binary data into CLOB or NCLOB columns. Thus, the BFILE data does not need to be in the same character set as the database or national character set to work properly. The APIs convert the data from the specified BFILE character set into the database character set for the CLOB data type, or the national character set for the NCLOB data type. The loading takes place on the server because BFILE data is not supported on the client.

- Use DBMS_LOB.LOADBLOBFROMFILE to load BLOB columns.
- Use DBMS_LOB.LOADCLOBFROMFILE to load CLOB and NCLOB columns.

See Also

- *Oracle AI Database PL/SQL Packages and Types Reference*
- *Oracle AI Database SecureFiles and Large Objects Developer's Guide*

10

OCI Programming in a Global Environment

This chapter contains information about OCI programming in a globalized environment. This chapter includes the following topics:

- [Using the OCI NLS Functions](#)
- [Specifying Character Sets in OCI](#)
- [Getting Locale Information in OCI](#)
- [Mapping Locale Information Between Oracle and Other Standards](#)
- [Manipulating Strings in OCI](#)
- [Classifying Characters in OCI](#)
- [Converting Character Sets in OCI](#)
- [OCI Messaging Functions](#)
- [Imsgen Utility](#)

10.1 Using the OCI NLS Functions

Many OCI NLS functions accept one of the following handles:

- The environment handle
- The user session handle

The OCI environment handle is associated with the client NLS environment and initialized with the client NLS environment variables. This environment does not change when ALTER SESSION statements are issued to the server. The character set associated with the environment handle is the client character set.

The OCI session handle is associated with the server session environment. Its NLS settings change when the session environment is modified with an ALTER SESSION statement. The character set associated with the session handle is the database character set.

Note that the OCI session handle does not have any NLS settings associated with it until the first transaction begins in the session. SELECT statements do not begin a transaction.

See Also

Oracle Call Interface Programmer's Guide for detailed information about the OCI NLS functions

10.2 Specifying Character Sets in OCI

Use the `OCIEnvNlsCreate` function to specify client-side database and national character sets when the OCI environment is created. This function enables users to set character set information dynamically in applications, independent of the `NLS_LANG` and `NLS_NCHAR`

initialization parameter settings. In addition, one application can initialize several environment handles for different client environments in the same server environment.

Any Oracle character set ID except AL16UTF16 can be specified through the `OCIEnvNlsCreate` function to specify the encoding of metadata, SQL CHAR data, and SQL NCHAR data. Use `OCI_UTF16ID` in the `OCIEnvNlsCreate` function to specify UTF-16 data.

See Also

Oracle Call Interface Programmer's Guide for more information about the `OCIEnvNlsCreate` function

10.3 Getting Locale Information in OCI

An Oracle locale consists of language, territory, and character set definitions. The locale determines conventions such as day and month names, as well as date, time, number, and currency formats. A globalized application complies with a user's locale setting and cultural conventions. For example, when the locale is set to German, users expect to see day and month names in German.

You can use the `OCINlsGetInfo()` function to retrieve the following locale information:

- Days of the week (translated)
- Abbreviated days of the week (translated)
- Month names (translated)
- Abbreviated month names (translated)
- Yes/no (translated)
- AM/PM (translated)
- AD/BC (translated)
- Numeric format
- Debit/credit
- Date format
- Currency formats
- Default language
- Default territory
- Default character set
- Default linguistic sort
- Default calendar

[Table 10-1](#) summarizes OCI functions that return locale information.

Table 10-1 OCI Functions That Return Locale Information

Function	Description
<code>OCINlsGetInfo()</code>	Returns locale information. See preceding text.
<code>OCINlsCharSetNameToId()</code>	Returns the Oracle character set ID for the specified Oracle character set name
<code>OCINlsCharSetIdToName()</code>	Returns the Oracle character set name from the specified character set ID
<code>OCINlsNumericInfoGet()</code>	Returns specified numeric information such as maximum character size
<code>OCINlsEnvironmentVariableGet()</code>	Returns the character set ID from <code>NLS_LANG</code> or the national character set ID from <code>NLS_NCHAR</code>

① See Also

Oracle Call Interface Programmer's Guide

10.4 Mapping Locale Information Between Oracle and Other Standards

The `OCINlsNameMap` function maps Oracle character set names, language names, and territory names to and from Internet Assigned Numbers Authority (IANA) and International Organization for Standardization (ISO) names.

10.5 Manipulating Strings in OCI

Two types of data structures are supported for string manipulation:

- Native character strings
- Wide character strings

Native character strings are encoded in native Oracle character sets. Functions that operate on native character strings take the string as a whole unit with the length of the string calculated in bytes. Wide character (`wchar`) string functions provide more flexibility in string manipulation. They support character-based and string-based operations with the length of the string calculated in characters.

The wide character data type is Oracle-specific and should not be confused with the `wchar_t` data type defined by the ANSI/ISO C standard. The Oracle wide character data type is always 4 bytes in all platforms, while the size of `wchar_t` depends on the implementation and the platform. The Oracle wide character data type normalizes native characters so that they have a fixed width for easy processing. This guarantees no data loss for round-trip conversion between the Oracle wide character format and the native character format.

String manipulation includes the:

- Conversion of strings between native character format and wide character format
- Character classifications
- Case conversion
- Calculations of display length
- General string manipulation, such as comparison, concatenation, and searching

[Table 10-2](#) summarizes the OCI string manipulation functions.

① Note

The functions and descriptions in [Table 10-2](#) that refer to multibyte strings apply to native character strings.

Table 10-2 OCI String Manipulation Functions

Function	Description
<code>OCIMultiByteToWideChar()</code>	Converts an entire null-terminated string into the <code>wchar</code> format.
<code>OCIMultiByteInSizeToWideChar()</code>	Converts part of a string into the <code>wchar</code> format.
<code>OCIWideCharToMultiByte()</code>	Converts an entire null-terminated wide character string into a multibyte string.
<code>OCIWideCharInSizeToMultiByte()</code>	Converts part of a wide character string into the multibyte format.
<code>OCIWideCharToLower()</code>	Converts the <code>wchar</code> character specified by <code>wc</code> into the corresponding lowercase character if it exists in the specified locale. If no corresponding lowercase character exists, then it returns <code>wc</code> itself.
<code>OCIWideCharToUpper()</code>	Converts the <code>wchar</code> character specified by <code>wc</code> into the corresponding uppercase character if it exists in the specified locale. If no corresponding uppercase character exists, then it returns <code>wc</code> itself.
<code>OCIWideCharStrcmp()</code>	Compares two wide character strings by binary, linguistic, or case-insensitive comparison method. Note: The <code>UNICODE_BINARY</code> sort method cannot be used with <code>OCIWideCharStrcmp()</code> to perform a linguistic comparison of the supplied wide character arguments.
<code>OCIWideCharStrncmp()</code>	Similar to <code>OCIWideCharStrcmp()</code> . Compares two wide character strings by binary, linguistic, or case-insensitive comparison methods. At most <code>len1</code> bytes form <code>str1</code> , and <code>len2</code> bytes form <code>str2</code> . Note: As with <code>OCIWideCharStrcmp()</code> , the <code>UNICODE_BINARY</code> sort method cannot be used with <code>OCIWideCharStrncmp()</code> to perform a linguistic comparison of the supplied wide character arguments.
<code>OCIWideCharStrcat()</code>	Appends a copy of the string pointed to by <code>wsrcstr</code> . Then it returns the number of characters in the resulting string.
<code>OCIWideCharStrncat()</code>	Appends a copy of the string pointed to by <code>wsrcstr</code> . Then it returns the number of characters in the resulting string. At most <code>n</code> characters are appended.
<code>OCIWideCharStrchr()</code>	Searches for the first occurrence of <code>wc</code> in the string pointed to by <code>wstr</code> . Then it returns a pointer to the <code>wchar</code> if the search is successful.
<code>OCIWideCharStrrchr()</code>	Searches for the last occurrence of <code>wc</code> in the string pointed to by <code>wstr</code> .
<code>OCIWideCharStrcpy()</code>	Copies the <code>wchar</code> string pointed to by <code>wsrcstr</code> into the array pointed to by <code>wdststr</code> . Then it returns the number of characters copied.
<code>OCIWideCharStrncpy()</code>	Copies the <code>wchar</code> string pointed to by <code>wsrcstr</code> into the array pointed to by <code>wdststr</code> . Then it returns the number of characters copied. At most <code>n</code> characters are copied from the array.
<code>OCIWideCharStrlen()</code>	Computes the number of characters in the <code>wchar</code> string pointed to by <code>wstr</code> and returns this number.
<code>OCIWideCharStrCaseConversion()</code>	Converts the wide character string pointed to by <code>wsrcstr</code> into the case specified by a flag and copies the result into the array pointed to by <code>wdststr</code> .
<code>OCIWideCharDisplayLength()</code>	Determines the number of column positions required for <code>wc</code> in display.
<code>OCIWideCharMultiByteLength()</code>	Determines the number of bytes required for <code>wc</code> in multibyte encoding.
<code>OCIMultiByteStrcmp()</code>	Compares two multibyte strings by binary, linguistic, or case-insensitive comparison methods.
<code>OCIMultiByteStrncmp()</code>	Compares two multibyte strings by binary, linguistic, or case-insensitive comparison methods. At most <code>len1</code> bytes form <code>str1</code> and <code>len2</code> bytes form <code>str2</code> .
<code>OCIMultiByteStrcat()</code>	Appends a copy of the multibyte string pointed to by <code>srcstr</code> .

Table 10-2 (Cont.) OCI String Manipulation Functions

Function	Description
OCIMultiByteStrncat()	Appends a copy of the multibyte string pointed to by <code>srcstr</code> . At most <i>n</i> bytes from <code>srcstr</code> are appended to <code>dststr</code> .
OCIMultiByteStrncpy()	Copies the multibyte string pointed to by <code>srcstr</code> into an array pointed to by <code>dststr</code> . It returns the number of bytes copied.
OCIMultiByteStrncpy()	Copies the multibyte string pointed to by <code>srcstr</code> into an array pointed to by <code>dststr</code> . It returns the number of bytes copied. At most <i>n</i> bytes are copied from the array pointed to by <code>srcstr</code> to the array pointed to by <code>dststr</code> .
OCIMultiByteStrlen()	Returns the number of bytes in the multibyte string pointed to by <code>str</code> .
OCIMultiByteStrnDisplayLength()	Returns the number of display positions occupied by the complete characters within the range of <i>n</i> bytes.
OCIMultiByteStrCaseConversion()	Converts part of a string from one character set to another.

See Also*Oracle Call Interface Programmer's Guide*

10.6 Classifying Characters in OCI

[Table 10-3](#) shows the OCI character classification functions.

Table 10-3 OCI Character Classification Functions

Function	Description
OCIWideCharIsAlnum()	Tests whether the wide character is an alphabetic letter or decimal digit
OCIWideCharIsAlpha()	Tests whether the wide character is an alphabetic letter
OCIWideCharIsCntrl()	Tests whether the wide character is a control character
OCIWideCharIsDigit()	Tests whether the wide character is a decimal digit
OCIWideCharIsGraph()	Tests whether the wide character is a graph character
OCIWideCharIsLower()	Tests whether the wide character is a lowercase letter
OCIWideCharIsPrint()	Tests whether the wide character is a printable character
OCIWideCharIsPunct()	Tests whether the wide character is a punctuation character
OCIWideCharIsSpace()	Tests whether the wide character is a space character
OCIWideCharIsUpper()	Tests whether the wide character is an uppercase character
OCIWideCharIsXdigit()	Tests whether the wide character is a hexadecimal digit
OCIWideCharIsSingleByte()	Tests whether <code>wc</code> is a single-byte character when converted into multibyte

See Also*Oracle Call Interface Programmer's Guide*

10.7 Converting Character Sets in OCI

Conversion between Oracle character sets and Unicode (16-bit, fixed-width Unicode encoding) is supported. Replacement characters are used if a character has no mapping from Unicode to the Oracle character set. Therefore, conversion back to the original character set is not always possible without data loss.

[Table 10-4](#) summarizes the OCI character set conversion functions.

Table 10-4 OCI Character Set Conversion Functions

Function	Description
<code>OCICharSetToUnicode()</code>	Converts a multibyte string pointed to by <code>src</code> to Unicode into the array pointed to by <code>dst</code>
<code>OCIUnicodeToCharSet()</code>	Converts a Unicode string pointed to by <code>src</code> to multibyte into the array pointed to by <code>dst</code>
<code>OCINlsCharSetConvert()</code>	Converts a string from one character set to another
<code>OCICharSetConversionIsReplacementUsed()</code>	Indicates whether replacement characters were used for characters that could not be converted in the last invocation of <code>OCINlsCharSetConvert()</code> or <code>OCIUnicodeToCharSet()</code>

① See Also

- *Oracle Call Interface Programmer's Guide*
- "[OCI Programming with Unicode](#)"

10.8 OCI Messaging Functions

The user message API provides a simple interface for cartridge developers to retrieve their own messages as well as Oracle messages.

[Table 10-5](#) summarizes the OCI messaging functions.

Table 10-5 OCI Messaging Functions

Function	Description
<code>OCIMessageOpen()</code>	Opens a message handle in a language pointed to by <code>hdl</code>
<code>OCIMessageGet()</code>	Retrieves a message with message number identified by <code>msgno</code> . If the buffer is not zero, then the function copies the message into the buffer specified by <code>msgbuf</code> .
<code>OCIMessageClose()</code>	Closes a message handle pointed to by <code>msg</code> and frees any memory associated with this handle

① See Also

Oracle Call Interface Programmer's Guide

10.9 lmsgen Utility

Purpose

The lmsgen utility converts text-based message files (.msg) into binary format (.msb) so that Oracle messages and OCI messages provided by the user can be returned to OCI functions in the desired language.

Messages used by the server are stored in binary-format files that are placed in the \$ORACLE_HOME/product_name/mesg directory, or the equivalent for your operating system. Multiple versions of these files can exist, one for each supported language, using the following file name convention:

`<product_id><language_abbrev>.msb`

For example, the file containing the server messages in French is called ora`f`.msb, because ORA is the product ID (<product_id>) and F is the language abbreviation (<language_abbrev>) for French. The value for product_name is rdbms, so it is in the \$ORACLE_HOME/rdbms/mesg directory.

Syntax

`LSMGEN text_file product facility [language] [-i indir] [-o outdir]`

text_file is a message text file.

product is the name of the product.

facility is the name of the facility.

language is the optional message language corresponding to the language specified in the NLS_LANG parameter. The language parameter is required if the message file is not tagged properly with language.

indir is the optional directory to specify the text file location.

outdir is the optional directory to specify the output file location.

The output (.msb) file will be generated under the \$ORACLE_HOME/product/mesg/ directory.

Text Message Files

Text message files must follow these guidelines:

- Lines that start with / and // are treated as internal comments and are ignored.
- To tag the message file with a specific language, include a line similar to the following:

```
# CHARACTER_SET_NAME= Japanese_Japan.JA16EUC
```

- Each message contains three fields:

message_number, warning_level, message_text

The message number must be unique within a message file.

The warning level is not currently used. Use 0.

The message text cannot be longer than 511 bytes.

The following example shows an Oracle message text file:

```
/ Copyright (c) 2006 by Oracle. All rights reserved.  
/ This is a test us7ascii message file  
# CHARACTER_SET_NAME= american_america.us7ascii  
/  
00000, 00000, "Export terminated unsuccessfully\n"  
00003, 00000, "no storage definition found for segment(%lu, %lu)"
```

Example: Creating a Binary Message File from a Text Message File

The following table contains sample values for the lmsgen parameters:

Parameter	Value
<i>product</i>	myapp
<i>facility</i>	imp
<i>language</i>	AMERICAN
<i>text_file</i>	impus.msg

One of the lines in the text message file is the following:

```
00128,2, "Duplicate entry %s found in %s"
```

The lmsgen utility converts the text message file (impus.msg) into binary format, resulting in a file called impus.msb. The directory \$ORACLE_HOME/myapp/mesg must already exist.

```
% lmsgen impus.msg myapp imp AMERICAN
```

The following output results:

```
Generating message file impus.msg -->  
$ORACLE_HOME/myapp/mesg/impus.msb
```


Character Set Migration

This chapter discusses character set conversion and character set migration. This chapter includes the following topics:

- [Overview of Character Set Migration](#)
- [Changing the Database Character Set of an Existing Database](#)
- [Repairing Database Character Set Metadata](#)
- [The Language and Character Set File Scanner](#)

11.1 Overview of Character Set Migration

Choosing the appropriate character set for your database is an important decision. When you choose the database character set, consider the following factors:

- The type of data you need to store
- The languages that the database needs to accommodate now and in the future
- The different size requirements of each character set and the corresponding performance implications

Oracle recommends choosing Unicode for its universality and compatibility with contemporary and future technologies and language requirements. The character set defined in the Unicode Standard supports all contemporary written languages with significant use and a few historical scripts. It also supports various symbols, for example, those used in technical, scientific, and musical notations. It is the native or recommended character set of many technologies, such as Java, Windows, HTML, or XML. There is no other character set that is so universal. In addition, Unicode adoption is increasing rapidly with great support from within the industry.

Oracle's implementation of Unicode, AL32UTF8, offers encoding of ASCII characters in 1 byte, characters from European, and Middle East languages in 2 bytes, characters from South and East Asian languages in 3 bytes. Therefore, storage requirements of Unicode are usually higher than storage requirements of a legacy character set for the same language.

A related topic is choosing a new character set for an existing database. Changing the database character set for an existing database is called **character set migration**. In this case, too, Oracle recommends migrating to Unicode for its universality and compatibility. When you migrate from one database character set to another, you should also plan to minimize data loss from the following sources:

- [Data Truncation](#)
- [Character Set Conversion Issues](#)

See Also

["Choosing a Character Set"](#)

11.1.1 Data Truncation

When the database is created using byte semantics, the sizes of the CHAR and VARCHAR2 data types are specified in bytes, not characters. For example, the specification CHAR(20) in a table definition allows 20 bytes for storing character data. When the database character set uses a single-byte character encoding scheme, no data loss occurs when characters are stored because the number of characters is equivalent to the number of bytes. If the database character set uses a multibyte character set, then the number of bytes no longer equals the number of characters because a character can consist of one or more bytes.

During migration to a new character set, it is important to verify the column widths of existing CHAR and VARCHAR2 columns because they may need to be extended to support an encoding that requires multibyte storage. Truncation of data can occur if conversion causes expansion of data.

The following table shows an example of data expansion when single-byte characters become multibyte characters through conversion.

Table 11-1 Single-Byte and Multibyte Encoding

Character	WE8MSWIN 1252 Encoding	AL32UTF8 Encoding
ä	E4	C3 A4
ö	F6	C3 B6
©	A9	C2 A9
€	80	E2 82 AC

The first column of the preceding table shows selected characters. The second column shows the hexadecimal representation of the characters in the WE8MSWIN1252 character set. The third column shows the hexadecimal representation of each character in the AL32UTF8 character set. Each pair of letters and numbers represents one byte. For example, ä (a with an umlaut) is a single-byte character (E4) in WE8MSWIN1252, but it becomes a two-byte character (C3 A4) in AL32UTF8. Also, the encoding for the euro symbol expands from one byte (80) to three bytes (E2 82 AC).

If the data in the new character set requires storage that is greater than the supported byte size of the data types, then you must change your schema. You may need to use CLOB columns.

See Also

["Length Semantics"](#)

11.1.1.1 Additional Problems Caused by Data Truncation

Data truncation can cause the following problems:

- In the database data dictionary, schema object names cannot exceed 30 bytes in length. You must rename schema objects if their names exceed 30 bytes in the new database character set. For example, one Thai character in the Thai national character set requires 1 byte. In AL32UTF8, it requires 3 bytes. If you have defined a table whose name is 11

Thai characters, then the table name must be shortened to 10 or fewer Thai characters when you change the database character set to AL32UTF8.

- If existing Oracle usernames or passwords are created based on characters that change in size in the new character set, then users will have trouble logging in because of authentication failures after the migration to a new character set. This occurs because the encrypted usernames and passwords stored in the data dictionary may not be updated during migration to a new character set. For example, if the current database character set is WE8MSWIN1252 and the new database character set is AL32UTF8, then the length of the username scött (o with an umlaut) changes from 5 bytes to 6 bytes. In AL32UTF8, scött can no longer log in because of the difference in the username. Oracle recommends that usernames and passwords be based on ASCII characters. If they are not, then you must reset the affected usernames and passwords after migrating to a new character set.
- When CHAR data contains characters that expand after migration to a new character set, space padding is not removed during database export by default. This means that these rows will be rejected upon import into the database with the new character set. The workaround is to set the BLANK_TRIMMING initialization parameter to TRUE before importing the CHAR data.

📘 See Also

Oracle AI Database Reference for more information about the BLANK_TRIMMING initialization parameter

11.1.2 Character Set Conversion Issues

This section includes the following topics:

- [Replacement Characters that Result from Using the Export and Import Utilities](#)
- [Invalid Data That Results from Setting the Client's NLS_LANG Parameter Incorrectly](#)
- [Conversion from Single-byte to Multibyte Character Set and Oracle Data Pump](#)

11.1.2.1 Replacement Characters that Result from Using the Export and Import Utilities

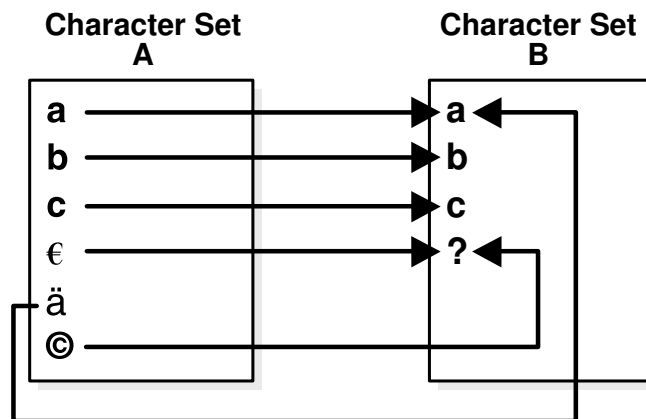
The Export and Import utilities can convert character sets from the original database character set to the new database character set. However, character set conversions can sometimes cause data loss or data corruption. For example, if you are migrating from character set A to character set B, then the destination character set B should be a superset of character set A. The destination character set, B, is a **superset** if it contains all the characters defined in character set A. Characters that are not available in character set B are converted to replacement characters, which are often specified as ? or ¿ or as a character that is related to the unavailable character. For example, ä (a with an umlaut) can be replaced by a. Replacement characters are defined by the target character set.

Note

There is an exception to the requirement that the destination character set B should be a superset of character set A. If your data contains no characters that are in character set A but are not in character set B, then the destination character set does not need to be a superset of character set A to avoid data loss or data corruption.

The following figure shows an example of a character set conversion in which the copyright and euro symbols are converted to ? and ä is converted to a.

Figure 11-1 Replacement Characters in Character Set Conversion



To reduce the risk of losing data, choose a destination character set with a similar character repertoire. Migrating to Unicode may be the best option, because AL32UTF8 contains characters from most legacy character sets.

Note

Starting with Oracle AI Database 26ai, the original Export utility is desupported. Instead, use the Data Pump Export/Import utilities.

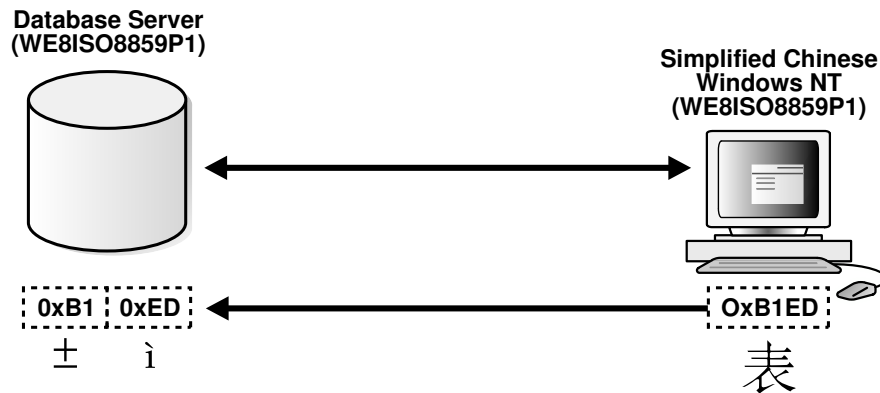
11.1.2.2 Invalid Data That Results from Setting the Client's NLS_LANG Parameter Incorrectly

Another character set migration scenario that can cause the loss of data is migrating a database that contains invalid data. Invalid data usually occurs in a database because the NLS_LANG parameter is not set properly on the client. The NLS_LANG value should reflect the client operating system code page. For example, in an English Windows environment, the code page is WE8MSWIN1252. When the NLS_LANG parameter is set properly, the database can automatically convert incoming data from the client operating system. When the NLS_LANG parameter is not set properly, then the data coming into the database is not converted properly. For example, suppose that the database character set is AL32UTF8, the client is an English Windows operating system, and the NLS_LANG setting on the client is AL32UTF8. Data coming into the database is encoded in WE8MSWIN1252 and is not converted to AL32UTF8 data.

because the NLS_LANG setting on the client matches the database character set. Thus Oracle assumes that no conversion is necessary, and invalid data is entered into the database.

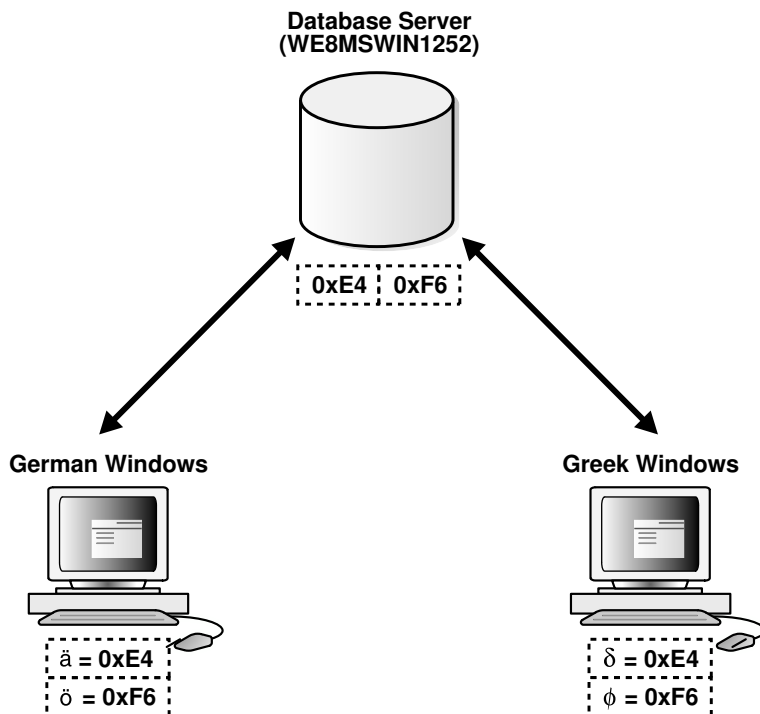
This can lead to two possible data inconsistency problems. One problem occurs when a database contains data from a character set that is different from the database character set but the same code points exist in both character sets. For example, if the database character set is WE8ISO8859P1 and the NLS_LANG setting of the Chinese Windows NT client is SIMPLIFIED CHINESE_CHINA.WE8ISO8859P1, then all multibyte Chinese data (from the ZHS16GBK character set) is stored as multiples of single-byte WE8ISO8859P1 data. This means that Oracle treats these characters as single-byte WE8ISO8859P1 characters. Hence all SQL string manipulation functions such as SUBSTR or LENGTH are based on bytes rather than characters. All bytes constituting ZHS16GBK data are legal WE8ISO8859P1 codes. If such a database is migrated to another character set such as AL32UTF8, then character codes are converted as if they were in WE8ISO8859P1. This way, each of the two bytes of a ZHS16GBK character are converted separately, yielding meaningless values in AL32UTF8. The following figure shows an example of this incorrect character set replacement.

Figure 11-2 Incorrect Character Set Replacement



The second possible problem is having data from mixed character sets inside the database. For example, if the data character set is WE8MSWIN1252, and two separate Windows clients using German and Greek are both using WE8MSWIN1252 as the NLS_LANG character set, then the database contains a mixture of German and Greek characters. The following figure shows how different clients can use different character sets in the same database.

Figure 11-3 Mixed Character Sets



For database character set migration to be successful, both of these cases require manual intervention because Oracle Database cannot determine the character sets of the data being stored. Incorrect data conversion can lead to data corruption, so perform a full backup of the database before attempting to migrate the data to a new character set. Refer to the topic "[Changing the Database Character Set of an Existing Database](#)" for more information about using the Database Migration Assistant for Unicode (DMU) software for handling invalid character data during character set migration to Unicode.

11.1.2.3 Conversion from Single-byte to Multibyte Character Set and Oracle Data Pump

If Oracle Data Pump is being used, and if a character set migration from single-byte to multibyte is performed, then the Data Pump PL/SQL packages must be reloaded.

11.2 Changing the Database Character Set of an Existing Database

Database character set migration is an intricate process that typically involves three stages: data scanning, data cleansing, and data conversion.

Before you change the database character set, you must identify possible database character set conversion problems and truncation of data. This step is called data scanning. Data scanning identifies the amount of effort required to migrate data into the new character encoding scheme before changing the database character set. Some examples of what may be found during a data scan are the number of schema objects where the column widths need to be expanded and the extent of the data that does not exist in the target character repertoire.

This information helps to determine the best approach for converting the database character set.

After the potential data issues are identified, they need to be cleansed properly to ensure the data integrity can be preserved during the data conversion. The data cleansing step could require significant time and effort depending on the scale and complexity of the data issues found. It may take multiple iterations of data scanning and cleansing in order to correctly address all of the data exceptions.

The data conversion is the process by which the character data is converted from the source character set into the target character set representation. Incorrect data conversion can lead to data corruption, so perform a full backup of the database before attempting to migrate the data to a new character set.

There are two approaches for migrating the database character set:

- [Migrating Character Data Using the Database Migration Assistant for Unicode](#)
- [Migrating Character Data Using a Full Export and Import](#)

11.2.1 Migrating Character Data Using the Database Migration Assistant for Unicode

The Database Migration Assistant for Unicode (DMU) offers an intuitive and user-friendly GUI that helps you streamline the migration process to Unicode through an interface that minimizes the manual workload and ensures that the migration tasks are carried out correctly and efficiently.

Some advantages of the DMU are that it does the following:

- Guides you through the workflow

An important advantage of the DMU is that it offers a logical workflow to guide you through the entire process of migrating character sets.
- Offers suggestions for handling certain problems

The DMU can help you when you run into certain problems, such as errors or failures during the scanning or cleansing of the data.
- Supports selective conversion of data

The DMU enables you to convert only the data that must be converted, at the table, column, and row level.
- Offers progress monitoring

The DMU provides a GUI to visualize how the steps are progressing.
- Offers interactive visualization features

The DMU enables you to analyze data and see the results in the GUI in an interactive way. It also enables you to see the data itself in the GUI and cleanse it interactively from identified migration issues.
- Provides the only supported tool for inline conversion

With the DMU, Oracle Database supports inline conversion of database contents. This offers performance and security advantage over other existing conversion methods.
- Allows cleansing actions to be scheduled for later execution during the conversion step

Postponing of cleansing actions, such as data type migration, ensures that the production database and applications are not affected until the actual migration downtime window.

This release of the Database Migration Assistant for Unicode has a few restrictions with respect to what databases it can convert. In particular, it does not convert databases with certain types of convertible data in the data dictionary. The export/import migration methods could be used to overcome these limitations.

In the current database release, the DMU is installed under the \$ORACLE_HOME/dmu directory.

① See Also

Oracle Database Migration Assistant for Unicode Guide

11.2.2 Migrating Character Data Using a Full Export and Import

A full export and import can also be used to convert the database to a new character set. It may be more time-consuming and resource-intensive as a separate target instance must be set up. If you plan to migrate your data to a non-Unicode character set, which Oracle strongly discourages, you can use the DMU to look for invalid character representation issues in the database and use export and import for the data conversion. Note that the DMU will not correctly identify data expansion issues (column and data type limit violations) if the migration is not to Unicode. It will also not identify characters that exist in the source database character set but do not exist in the non-Unicode target character set.

① Note

Starting with Oracle AI Database 26ai, the original Export utility is desupported. Instead, use the Data Pump Export/Import utilities.

① See Also

Oracle AI Database Utilities for more information about the Data Pump Export and Import utilities

11.3 Repairing Database Character Set Metadata

If your database has been in what is commonly called a pass-through configuration, where the client character set is defined (usually through the NLS_LANG client setting) to be equal to the database character set, the character data in your database could be stored in a different character set from the declared database character set. In this scenario, the recommended solution is to migrate your database to Unicode by using the DMU assumed database character set feature to indicate the actual character set for the data. In case migrating to Unicode is not immediately feasible due to business or technical constraints, it would be desirable to at least correct the database character set declaration to match with the database contents.

With Database Migration Assistant for Unicode Release 1.2, you can repair the database character set metadata in such cases using the CSREPAIR script. The CSREPAIR script works in conjunction with the DMU client and accesses the DMU repository. It can be used to change the database character set declaration to the real character set of the data only after the DMU has performed a full database scan by setting the Assumed Database Character Set property

to the target character set and no invalid representation issues have been reported, which verifies that all existing data in the database is defined according to the assumed database character set. Note that CSREPAIR only changes the database character set declaration in the data dictionary metadata and does not convert any database data.

You can find the CSREPAIR script under the admin subdirectory of the DMU installation. The requirements when using the CSREPAIR script are:

1. You must first perform a successful full database scan in the DMU with the Assumed Database Character Set property set to the real character set of the data. In this case, the assumed database character set must be different from the current database character set or else nothing will be done. The CSREPAIR script will not proceed if the DMU reports the existence of invalid data. It will, however, proceed if changeless or convertible data is present from the scan.
2. The target character set in the assumed database character set must be a binary superset of US7ASCII.
3. Only repairing from single-byte to single-byte character sets or multi-byte to multi-byte character sets is allowed as no conversion of CLOB data will be attempted.
4. If you set the assumed character set at the column level, then the value must be the same as the assumed database character set. Otherwise, CSREPAIR will not run.
5. You must have the SYSDBA privilege to run CSREPAIR.

11.3.1 Example: Using CSREPAIR

A typical example is storing WE8MSWIN1252 data in a WE8ISO8859P1 database via the pass-through configuration. To correct the database character set from WE8ISO8859P1 to WE8MSWIN1252, perform the following steps:

1. Set up the DMU and connect to the target WE8ISO8859P1 database.
2. Open the Database Properties tab in the DMU.
3. Set the Assumed Database Character Set property to WE8MSWIN1252.
4. Use the DMU to perform a full database scan.
5. Open the Database Scan Report and verify there is no data reported under the Invalid Representation category.
6. Exit from the DMU client.
7. Start the SQL*Plus utility and connect as a user with the SYSDBA privilege.
8. Run the CSREPAIR script:

```
SQL> @@CSREPAIR.PLB
```

Upon completion, you should get the message:

The database character set has been successfully changed to WE8MSWIN1252. You must restart the database now.

9. Shut down and restart the database.

11.4 The Language and Character Set File Scanner

The Language and Character Set File Scanner (LCSSCAN) is a high-performance, statistically based utility for determining the language and character set for unknown file text. It can automatically identify a wide variety of language and character set pairs. With each text, the

language and character set detection engine sets up a series of probabilities, each probability corresponding to a language and character set pair. The most statistically probable pair identifies the dominant language and character set.

The purity of the text affects the accuracy of the language and character set detection. The ideal case is literary text of one single language with no spelling or grammatical errors. These types of text may require 100 characters of data or more and can return results with a very high factor of confidence. On the other hand, some technical documents can require longer segments before they are recognized. Documents that contain a mix of languages or character sets or text such as addresses, phone numbers, or programming language code may yield poor results. For example, if a document has both French and German embedded, then the accuracy of guessing either language successfully is statistically reduced. Both plain text and HTML files are accepted. If the format is known, you should set the `FORMAT` parameter to improve accuracy.

This section includes the following topics:

- [Syntax of the LCSSCAN Command](#)
- [Examples: Using the LCSSCAN Command](#)
- [Getting Command-Line Help for the Language and Character Set File Scanner](#)
- [Supported Languages and Character Sets](#)
- [LCSSCAN Error Messages](#)

11.4.1 Syntax of the LCSSCAN Command

Start the Language and Character Set File Scanner with the `LCSSCAN` command. Its syntax is as follows:

```
LCSSCAN [RESULTS=number] [FORMAT=file_type] [BEGIN=number] [END=number] FILE=file_name
```

The parameters are described in the rest of this section.

RESULTS

The `RESULTS` parameter is optional.

Property	Description
Default value	1
Minimum value	1
Maximum value	3
Purpose	The number of language and character set pairs that are returned. They are listed in order of probability. The comparative weight of the first choice cannot be quantified. The recommended value for this parameter is the default value of 1.

FORMAT

The `FORMAT` parameter is optional.

Property	Description
Default Value	text

Property	Description
Purpose	This parameter identifies the type of file to be scanned. The possible values are html, text, and auto.

BEGIN

The BEGIN parameter is optional.

Property	Description
Default value	1
Minimum value	1
Maximum value	Number of bytes in file
Purpose	The byte of the input file where LCSSCAN begins the scanning process. The default value is the first byte of the input file.

END

The END parameter is optional.

Property	Description
Default value	End of file
Minimum value	3
Maximum value	Number of bytes in file
Purpose	The last byte of the input file that LCSSCAN scans. The default value is the last byte of the input file.

FILE

The FILE parameter is required.

Property	Description
Default value	None
Purpose	Specifies the name of a text file to be scanned

11.4.2 Examples: Using the LCSSCAN Command

Example 11-1 Specifying Only the File Name in the LCSSCAN Command

```
LCSSCAN FILE=example.txt
```

In this example, the entire example.txt file is scanned because the BEGIN and END parameters have not been specified. One language and character set pair will be returned because the RESULTS parameter has not been specified.

Example 11-2 Specifying the Format as HTML

```
LCSSCAN FILE=example.html FORMAT=html
```

In this example, the entire example.html file is scanned because the BEGIN and END parameters have not been specified. The scan will strip HTML tags before the scan, thus

results are more accurate. One language and character set pair will be returned because the RESULTS parameter has not been specified.

Example 11-3 Specifying the RESULTS and BEGIN Parameters for LCSSCAN

```
LCSSCAN RESULTS=2 BEGIN=50 FILE=example.txt
```

The scanning process starts at the 50th byte of the file and continues to the end of the file. Two language and character set pairs will be returned.

Example 11-4 Specifying the RESULTS and END Parameters for LCSSCAN

```
LCSSCAN RESULTS=3 END=100 FILE=example.txt
```

The scanning process starts at the beginning of the file and ends at the 100th byte of the file. Three language and character set pairs will be returned.

Example 11-5 Specifying the BEGIN and END Parameters for LCSSCAN

```
LCSSCAN BEGIN=50 END=100 FILE=example.txt
```

The scanning process starts at the 50th byte and ends at the 100th byte of the file. One language and character set pair will be returned because the RESULTS parameter has not been specified.

11.4.3 Getting Command-Line Help for the Language and Character Set File Scanner

To obtain a summary of the Language and Character Set File Scanner parameters, enter the following command:

```
LCSSCAN HELP=y
```

The resulting output shows a summary of the Language and Character Set Scanner parameters.

11.4.4 Supported Languages and Character Sets

The Language and Character Set File Scanner supports several character sets for each language.

When the binary values for a language match two or more encodings that have a subset/superset relationship, the subset character set is returned. For example, if the language is German and all characters are 7-bit, then US7ASCII is returned instead of WE8MSWIN1252, WE8ISO8859P15, or WE8ISO8859P1.

When the character set is determined to be UTF-8, the Oracle character set UTF8 is returned by default unless 4-byte characters (supplementary characters) are detected within the text. If 4-byte characters are detected, then the character set is reported as AL32UTF8.

① See Also

["Language and Character Set Detection Support"](#) for a list of supported languages and character sets

11.4.5 LCSSCAN Error Messages

LCD-00001 An unknown error occurred.

Cause: An error occurred accessing an internal structure.

Action: Report this error to Oracle Support.

LCD-00002 NLS data could not be loaded.

Cause: An error occurred accessing \$ORACLE_HOME/nls/data.

Action: Check to make sure \$ORACLE_HOME/nls/data exists and is accessible. If not found check \$ORA_NLS10 directory.

LCD-00003 An error occurred while reading the profile file.

Cause: An error occurred accessing \$ORACLE_HOME/nls/data.

Action: Check to make sure \$ORACLE_HOME/nls/data exists and is accessible. If not found check \$ORA_NLS10 directory.

LCD-00004 The beginning or ending offset has been set incorrectly.

Cause: The beginning and ending offsets must be an integer greater than 0.

Action: Change the offset to a positive number.

LCD-00005 The ending offset has been set incorrectly.

Cause: The ending offset must be greater than the beginning offset.

Action: Change the ending offset to be greater than the beginning offset.

LCD-00006 An error occurred when opening the input file.

Cause: The file was not found or could not be opened.

Action: Check the name of the file specified. Make sure the full file name is specified and that the file is not in use.

LCD-00007 The beginning offset has been set incorrectly.

Cause: The beginning offset must be less than the number of bytes in the file.

Action: Check the size of the file and specify a smaller beginning offset.

LCD-00008 No result was returned.

Cause: Not enough text was inputted to produce a result.

Action: A larger sample of text needs to be inputted to produce a reliable result.

EBCDIC Emulation for Mainframe Migration

This chapter provides guidance for migrating applications and databases from mainframe environments—where EBCDIC character sets are prevalent—to Oracle databases on modern platforms such as Linux. It covers migration strategies, recommendations for character set selection, the use of accurate source-to-target character mappings during data migration, and Oracle features for emulating EBCDIC binary ordering to support legacy application logic without modification.

This chapter includes the following topics:

- [Overview of Application Migration from a Mainframe](#)
- [Choosing a Character Set for the Target Database](#)
- [Choosing a Source Character Set for Data Migration](#)
- [Emulating EBCDIC Binary Ordering](#)

12.1 Overview of Application Migration from a Mainframe

Organizations may choose to migrate from mainframe platforms to modern environments to improve scalability, flexibility, and integration with emerging technologies, to optimize costs, and to support evolving business and regulatory requirements.

This section covers migration challenges and strategies, including:

- [Character Set Differences](#)
- [Migration Strategies](#)
- [Avoiding Dependency on Character Encoding](#)

12.1.1 Character Set Differences

A major challenge in migrating from mainframe platforms is the fundamental difference in character encodings between the source and target environments. Character sets used on mainframe platforms are predominantly based on EBCDIC (Extended Binary Coded Decimal Interchange Code), developed by IBM for their mainframes and midrange computer systems. In contrast, character sets on Linux and other modern operating systems are based on ASCII (American Standard Code for Information Interchange), developed by the American National Standards Institute (ANSI). ASCII is a subset of the Unicode standard, and its characters are encoded identically in UTF-8.

Although EBCDIC and ASCII encode a similar set of basic characters, they assign very different binary codes. The most significant differences are:

- ASCII assigns codes in the range 0-127, while EBCDIC uses the range 0-255. ASCII supersets, such as ISO 8859-1, use codes 128-255 for additional characters, while EBCDIC supersets add codes in unused positions among the basic characters.
- In EBCDIC, digits have numerically higher codes than basic Latin letters and most accented letters. In ASCII, digits have lower codes than any letter.

- In contrast to ASCII, the code range for basic uppercase Latin letters (A-Z) in EBCDIC is not continuous, but divided into three ranges: A-I, J-R, and S-Z. Similarly, lowercase letters (a-z) are divided into ranges a-i, j-r, and s-z.
- Basic Latin lowercase letters have lower codes than their uppercase counterparts in EBCDIC, but higher codes in ASCII.
- The sets of control characters defined in EBCDIC and ASCII differ significantly.

[Table 12-1](#) and [Table 12-2](#) illustrate the basic structure of EBCDIC and ASCII code pages. Empty cells indicate code points reserved in EBCDIC- and ASCII-based character sets for Latin letters with diacritics, non-Latin letters (such as Cyrillic or Greek), or additional punctuation. ASCII-based character sets from the ISO 8859 family also encode additional control codes in the range 0x80-0x9F.

Table 12-1 The Structure of an EBCDIC-Based Character Set (Code Page)

	-0	-1	-2	-3	-4	-5	-6	-7	-8	-9	-A	-B	-C	-D	-E	-F
0-	<i>NUL</i> ¹	<i>SOH</i>	<i>STX</i>	<i>ETX</i>	<i>SEL</i>	<i>HT</i>	<i>RNL</i>	<i>DEL</i>	<i>GE</i>	<i>SPS</i>	<i>RPT</i>	<i>VT</i>	<i>FF</i>	<i>CR</i>	<i>SO</i>	<i>SI</i>
1-	<i>DLE</i>	<i>DC1</i>	<i>DC2</i>	<i>DC3</i>	<i>RES</i>	<i>NL</i>	<i>BS</i>	<i>POC</i>	<i>CAN</i>	<i>EM</i>	<i>UBS</i>	<i>CU1</i>	<i>IFS</i>	<i>IGS</i>	<i>IRS</i>	<i>IUS</i>
2-	<i>DS</i>	<i>SOSI</i>	<i>FS</i>	<i>WUS</i>	<i>BYP</i>	<i>LF</i>	<i>ETB</i>	<i>ESC</i>	<i>SA</i>	<i>SFE</i>	<i>SM/SW</i>	<i>CSP</i>	<i>MFA</i>	<i>ENQ</i>	<i>ACK</i>	<i>BEL</i>
3-			<i>SYN</i>	<i>IR</i>	<i>PP</i>	<i>TRN</i>	<i>NBS</i>	<i>EOT</i>	<i>SBS</i>	<i>IT</i>	<i>RFF</i>	<i>CU3</i>	<i>DC4</i>	<i>NAK</i>		<i>SUB</i>
4-	<i>SP</i>											.	<	(	+	
5-	&										! ²	\$ ²	*	)	;	
6-	-	/										,	%	_	>	?
7-										` ²	:	# ²	@ ²	'	=	"
8-		a	b	c	d	e	f	g	h	i						
9-		j	k	l	m	n	o	p	q	r						
A-		~ ²	s	t	u	v	w	x	y	z						
B-	^ ²										[²	] ²				
C-	{ ²	A	B	C	D	E	F	G	H	I						
D-	} ²	J	K	L	M	N	O	P	Q	R						
E-	\ ²		S	T	U	V	W	X	Y	Z						
F-	0	1	2	3	4	5	6	7	8	9						<i>E0</i>

¹ Names of control codes are italicized.

² These codes are not present in all EBCDIC code pages. They are presented here as defined in EBCDIC code page 37 because they are defined in ASCII.

Table 12-2 The Structure of an ASCII-Based Character Set

	-0	-1	-2	-3	-4	-5	-6	-7	-8	-9	-A	-B	-C	-D	-E	-F
0-	<i>NUL</i> ¹	<i>SOH</i>	<i>STX</i>	<i>ETX</i>	<i>EOT</i>	<i>ENQ</i>	<i>ACK</i>	<i>BEL</i>	<i>BS</i>	<i>HT</i>	<i>LF</i>	<i>VT</i>	<i>FF</i>	<i>CR</i>	<i>SO</i>	<i>SI</i>
1-	<i>DLE</i>	<i>DC1</i>	<i>DC2</i>	<i>DC3</i>	<i>DC4</i>	<i>NAK</i>	<i>SYN</i>	<i>ETB</i>	<i>CAN</i>	<i>EM</i>	<i>SUB</i>	<i>ESC</i>	<i>FS</i>	<i>GS</i>	<i>RS</i>	<i>US</i>
2-	<i>SP</i>	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
3-	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4-	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5-	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_

Table 12-2 (Cont.) The Structure of an ASCII-Based Character Set

	-0	-1	-2	-3	-4	-5	-6	-7	-8	-9	-A	-B	-C	-D	-E	-F
6-	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7-	p	q	r	s	t	u	v	w	x	y	z	{		}	~	<i>DEL</i>
8-																
9-																
A-																
B-																
C-																
D-																
E-																
F-																

¹ Names of control codes are italicized.

12.1.2 Migration Strategies

There are two main strategies for migrating an application system between platforms:

- **Data-Only Migration:** Only the data is migrated to an ASCII-based system, while the mainframe applications continue to run in EBCDIC mode (often via emulation). When migrating to an Oracle Database, this approach requires that the source data is stored in a relational database and accessed with SQL, allowing applications to continue to access data in the same way.
- **Application and Data Migration:** Both applications and data are migrated. Applications are completely rewritten—for example, moving COBOL logic to Java or another language—thereby removing any EBCDIC dependencies and enabling the use of modern character sets and collations.

For migration to Oracle Database, regardless of whether applications process source data in a relational database or directly in files, the main globalization-related tasks are selecting the database character set for the target database and configuring the data loading process to ensure that character codes are properly converted between source and target character sets. See "[Choosing a Character Set for the Target Database](#)" and "[Choosing a Source Character Set for Data Migration](#)" for more information.

Minimal or no modification to applications may require additional configuration of the target database to ensure that, for a given, unmodified SQL query, the database returns the same rows in the same order as the source mainframe database. Refer to "[Emulating EBCDIC Binary Ordering](#)" for more information.

Certain SQL functions cannot be used in queries if those queries are intended to remain unchanged during the migration process. See "[Avoiding Dependency on Character Encoding](#)" for more details.

Because Oracle no longer supports Oracle Database Client software on IBM EBCDIC-based platforms, such as IBM z/OS, unchanged applications require a third-party translation layer to communicate through Oracle Database Client software running on an ASCII-based platform. This layer must convert between the application's EBCDIC code page and the Oracle character set configured for the Oracle Database Client. The conversion should be compatible

with that used for the initial data migration. See "[Choosing a Source Character Set for Data Migration](#)".

If applications are being translated to another programming language and framework, the translation process should include logic to adapt all programming constructs that depend on a specific character encoding to encoding-independent mechanisms, such as a character classification API. A detailed discussion of this requirement is beyond the scope of this guide.

12.1.3 Avoiding Dependency on Character Encoding

The SQL and PL/SQL statements in applications targeted for migration should not assume the use of the original EBCDIC character set. For example, the CHR function should not be called with arguments corresponding to binary codes from an EBCDIC code page. While CHR(67) returns the character 'ä' (a with diaeresis) in the EBCDIC code page 500, it returns the uppercase letter 'C' in all ASCII-based character sets. Such CHR calls must be replaced with character literals—which the translation layer (mentioned in the previous section) can automatically convert to the appropriate database character set—or updated to use target character set's codes (for example, CHR(228) if the target database character set is WE8ISO8859P1, or CHR(50084) if using AL32UTF8).

At a minimum, the following SQL and PL/SQL function calls (or their equivalents in the source mainframe SQL dialect) must be reviewed and typically modified to support running an EBCDIC application against a database with an ASCII-based database character set: CHR, ASCII, UTL_RAW.CAST_TO_VARCHAR2, UTL_RAW.CAST_TO_RAW, and DUMP.

If an application uses hashing and encryption of character values, these operations should be performed entirely on the application side or entirely on the database server side. Hashed or encrypted values do not undergo client-server character set conversion. As a result, a value encrypted on one side and decrypted on the other may be in the wrong encoding, and hashes generated on different sides for the same string may not match.

12.2 Choosing a Character Set for the Target Database

Selecting the appropriate character set for the target Oracle Database is crucial to avoid data loss during migration. This section discusses the requirements for the target character set and outlines specific considerations for selecting the Unicode character set AL32UTF8 or one of the single-byte Oracle character sets.

This section includes the following topics:

- [Requirements for Target Character Sets](#)
- [Migrating to AL32UTF8](#)
- [Migrating to Legacy ASCII-Based Character Sets](#)

12.2.1 Requirements for Target Character Sets

When planning migration, it is essential that the target Oracle Database character set supports every character present in the source mainframe data. The target set does not need to cover all possible characters in the source set—only those that are actually used. For example, control codes other than those corresponding to new line, carriage return, horizontal tab, vertical tab, form feed (new page), and escape are usually not included in business data.

A special case arises when a source character column is used to store binary data, under the data-only migration strategy. In such cases, the data migration process and the target environment must preserve all possible source byte values through round-trip conversion—any

byte sequence stored in the original database must be storable in and retrievable from the target database without any loss of information.

12.2.2 Migrating to AL32UTF8

The Unicode UTF-8 character encoding form, as implemented in the Oracle character set AL32UTF8, is recommended for full character repertoire of any EBCDIC code page. With its extensive set of defined characters—including a range of codes reserved for private use by applications—AL32UTF8 is the only Oracle Database character set capable of representing all characters from any EBCDIC character set. AL32UTF8 can serve as the target character set for any specific EBCDIC source system. It is also suitable for databases that consolidate application systems migrated from multiple different EBCDIC code pages and language groups.

Conversion to and from AL32UTF8 is a round-trip process that preserves all characters, except in cases where an EBCDIC code page maps more than one character code to the same Unicode character or when undefined codes are used to store binary data. For more information on these exceptional cases, see "[Undefined or Duplicated Codes in a Source Character Set](#)".

Because AL32UTF8 is a multibyte character set, single-byte applications running unchanged after migration must meet the requirements outlined in section "[Running Single-Byte Client Applications with a Unicode Database](#)" in order to work correctly with an AL32UTF8 database. To fully benefit from Unicode, applications should be rewritten using a Unicode-capable framework, such as Java or .NET.

12.2.3 Migrating to Legacy ASCII-Based Character Sets

If Unicode is not feasible—due to application limitations or certification requirements—the Oracle Database character set that most closely matches the source EBCDIC code page should be selected to minimize potential data loss during data migration and character set conversions.

[Table A-15](#) lists the closest ASCII-based character sets for several single-byte EBCDIC code pages.

For Western European code pages that do not include the euro sign, WE8ISO8859P1 provides a complete round-trip mapping. For code pages where the euro sign (U+20AC) replaces the general currency symbol (U+00A4), no single-byte ASCII character set offers complete mapping. The available options are:

- **WE8ISO8859P1:** Does not encode the euro sign.
- **WE8ISO8859P15:** Encodes the euro sign, but omits several punctuation characters.
- **WE8MSWIN1252:** Encodes both the euro sign and all needed printable characters, but does not encode many control codes. Most of these control codes are typically not present in character data; however, the character U+0085 (Next Line) may be used.

The target database character set should be chosen based on a thorough analysis of the source data. Characters not supported by the selected character set must be absent from the source data, or you must be able to replace them with other encoded characters without compromising data integrity.

Mapping the Arabic code page IBM-420 to an ASCII-based single-byte character set is particularly challenging. IBM-420 encodes the presentation forms (shapes) of Arabic letters, such as medial or final forms, while Oracle Database character sets support only the characters as they are stored in memory (logical order). Arabic text stored in Oracle Database is expected to be shaped into the correct glyphs by application GUI subsystems.

- For data-only migration, AL32UTF8 is the recommended target character set for Arabic text in presentation forms, as it encodes these presentation forms as Unicode compatibility characters.
- For application and data migration, applications should be re-engineered to process Arabic text using modern approaches, delegating letter shaping to the GUI layer. The data migration process should unify multiple presentation forms of a single character into its logical (memory representation) form.

12.3 Choosing a Source Character Set for Data Migration

This section describes how to select the appropriate character set for loading EBCDIC source data into the target database. It also outlines common methods for loading data and discusses potential issues that may arise during the process.

This section includes the following topics:

- [CDRA-Compatible Character Sets](#)
- [SQL*Loader](#)
- [External Tables](#)
- [DBMS_LOB](#)
- [Character Set Conversion Functions](#)
- [Undefined or Duplicated Codes in a Source Character Set](#)

12.3.1 CDRA-Compatible Character Sets

For accurate data migration, use character sets that map EBCDIC code pages to Unicode according to IBM's Character Data Representation Architecture (CDRA) mappings. [Table A-10](#) lists Oracle character sets that provide these accurate mappings. Earlier Oracle implementations of EBCDIC code pages, such as WE8EBCDIC37, may map certain characters differently. For example, they map both control codes 0x15 (NL) and 0x25 (LF) to the Unicode Line Feed character U+000A, whereas character sets with CDRA-accurate mappings map 0x15 to the Next Line character U+0085.

The character sets listed in [Table A-10](#) are client-only character sets. They can be used, as described in the following topics, to load source data from operating system files into a target Oracle Database.

12.3.2 SQL*Loader

SQL*Loader is the primary Oracle utility for loading data from text files. You specify the source character set for data loading in the control file, and SQL*Loader automatically converts characters from the source character set to the target database character set.

Although you can also specify the source character set for SQL*Loader input files using the NLS_LANG environment variable, do **not** specify EBCDIC character sets in this variable when running on ASCII-based platforms.

① See Also

Oracle AI Database Utilities for more information about SQL*Loader and the syntax of control files

Example 12-1 SQL*Loader Control File

The following control file declares the source character set of text in the file `customer_data.dat` as IBM1140. The remainder of the control file should specify the file structure, such as field names, data types, and separators used as column delimiters.

```
LOAD DATA
CHARACTERSET IBM1140
INFILE 'customer_data.dat'
...
```

12.3.3 External Tables

External tables that use the `ORACLE_LOADER` access driver allow data in text files to be queried using SQL as if the data were rows in a relational table. With this approach, the data remains in the external file and is not physically loaded into the database. This method enables complex transformations to be performed on the data before it is made available to database applications.

① See Also

Oracle AI Database Utilities for more information about external tables that use the `ORACLE_LOADER` access driver

Example 12-2 External Table Definition

The following `CREATE TABLE` statement creates an external table as a proxy for the contents of the file `customer_data.dat`, which is located in the `data_dir` database directory. The character data in the file is encoded using the IBM273 character set. Database directories are created with the SQL statement `CREATE DIRECTORY`, making operating system directories accessible to SQL statements. Column and field definitions are omitted for brevity.

```
CREATE TABLE ext_customer_data (
...
)
ORGANIZATION EXTERNAL (
  TYPE ORACLE_LOADER
  DEFAULT DIRECTORY data_dir
  ACCESS PARAMETERS (
    RECORDS DELIMITED BY NEWLINE
    CHARACTERSET IBM273
    ...
  )
  LOCATION ('customer_data.dat')
)
```

12.3.4 DBMS_LOB

The Oracle-supplied PL/SQL package `DBMS_LOB` provides APIs for loading text files into a CLOB column, either from an operating system file mapped to a BFILE or from a BLOB column.

The function `LOADCLOBFROMFILE` transfers data from a text file represented by a `BFILE` to a `CLOB`, performing necessary character set conversion.

```
DBMS_LOB.LOADCLOBFROMFILE (
  dest_lob      IN OUT NOCOPY  NOCOPY CLOB CHARACTER SET ANY_CS,
  src_bfile     IN              BFILE,
  amount        IN              INTEGER,
  dest_offset   IN OUT         INTEGER,
  src_offset    IN OUT         INTEGER,
  bfile_csid    IN              NUMBER,
  lang_context  IN OUT         INTEGER,
  warning       OUT             INTEGER);
```

The `bfile_csid` parameter lets you specify the numeric ID of the character set used in the file being loaded. The file's contents are converted from the specified character set to the database character set. You can map character set names to their corresponding numeric IDs using the SQL function `NLS_CHARSET_ID`.

The function `CONVERTTOCLOB` copies data from a binary LOB (BLOB) to a character LOB (CLOB) with the necessary character set conversion. This function enables data originally loaded as binary, without interpretation, to be converted to text in the database character set.

```
DBMS_LOB.CONVERTTOCLOB(
  dest_lob      IN OUT NOCOPY  CLOB CHARACTER SET ANY_CS,
  src_blob      IN              BLOB,
  amount        IN              INTEGER,
  dest_offset   IN OUT         INTEGER,
  src_offset    IN OUT         INTEGER,
  blob_csid     IN              NUMBER,
  lang_context  IN OUT         INTEGER,
  warning       OUT             INTEGER);
```

The `blob_csid` parameter lets you specify the numeric ID of the character set used for the text data stored in the BLOB value. The contents of the BLOB are then converted from the specified character set to the database character set.

See Also

Oracle AI Database SecureFiles and Large Objects Developer's Guide for more information about using `DBMS_LOB`

Example 12-3 Loading a File into a CLOB Column

This example demonstrates the procedure `load_product_description`, which accepts a product ID and the name of a file located in the database directory `description_dir`, and loads the file into the product row in the `products` table. The character set of the file is `IBM1140`.

```
CREATE TABLE products
( product_id NUMBER
, product_name VARCHAR2(100 CHAR)
, product_description CLOB
)
```

```

/
CREATE OR REPLACE DIRECTORY description_dir
  AS '/scratch/import/products/descriptions'
/
CREATE OR REPLACE PROCEDURE load_product_description
( p_id NUMBER, p_description_file VARCHAR2 )
AS
  column_clob CLOB;
  descr_file BFILE := BFILENAME('DESCRIPTION_DIR',p_description_file);
  dst_off INTEGER := 1;
  src_off INTEGER := 1;
  lang_ctx INTEGER := 0;
  warn INTEGER := 0;
BEGIN
  UPDATE products
    SET product_description = EMPTY_CLOB()
  WHERE product_id = p_id
  RETURNING product_description INTO column_clob;

  DBMS_LOB.FILEOPEN(descr_file);
  DBMS_LOB.LOADCLOBFROMFILE (
    dest_clob => column_clob,
    src_bfile => descr_file,
    amount => DBMS_LOB.LOBMAXSIZE, -- whole file
    dest_offset => dst_off,
    src_offset => src_off,
    bfile_csid => NLS_CHARSET_ID('IBM1140'),
    lang_context => lang_ctx, -- unused in this example
    warning => warn);
  DBMS_LOB.FILECLOSE(descr_file);
  COMMIT;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    NULL;
  WHEN OTHERS THEN
    IF DBMS_LOB.FILEISOPEN(descr_file) = 1 THEN
      DBMS_LOB.FILECLOSE(descr_file);
    END IF;
    RAISE;
END;
/

```

12.3.5 Character Set Conversion Functions

The Oracle-supplied PL/SQL packages UTL_I18N and UTL_RAW provide functions that allow a value of the data type RAW to be interpreted as a string in a specified character set and then:

- converted to a string in another specified character set and returned as a RAW value, or
- converted to a VARCHAR2 value in the database character set.

These functions can be used with the client-only EBCDIC character sets listed in [Table A-10](#) to implement custom character set conversion processing as needed by applications. For

example, EBCDIC files can be loaded as binary (RAW) values and then processed and transformed as necessary into character values in the database character set.

```
UTL_RAW.CONVERT(  
    r          IN RAW,  
    to_charset  IN VARCHAR2,  
    from_charset IN VARCHAR2)  
RETURN RAW;  
  
UTL_I18N.RAW_TO_CHAR(  
    data      IN RAW,  
    src_charset IN VARCHAR2 DEFAULT NULL)  
RETURN VARCHAR2;
```

The following function can convert a VARCHAR2 value from the database character set to a specified target character set and return it as a RAW value. For example, the returned value can be written to a file using the Oracle-supplied procedure UTL_FILE.PUT_RAW.

```
UTL_I18N.STRING_TO_RAW(  
    data          IN VARCHAR2 CHARACTER SET ANY_CS,  
    dst_charset   IN VARCHAR2 DEFAULT NULL)  
RETURN RAW;
```

The parameters to_charset, from_charset, src_charset, and dst_charset are used in character set conversion functions to specify the names of the source or target character sets, respectively.

① See Also

Oracle AI Database PL/SQL Packages and Types Reference for more information about the UTL_I18N and UTL_RAW packages

12.3.6 Undefined or Duplicated Codes in a Source Character Set

There are two potential issues with source character encoding that must be addressed before migrating data to a target database.

Undefined Codes

The character set (EBCDIC code page) used as the encoding for source data may include undefined codes—bytes that do not correspond to any character. For example, EBCDIC code page 424 (used for Hebrew) does not assign characters to thirty-eight byte values (0x70, 0x72–0x73, 0x75–0x77, 0x80, 0x8C–0x8E, 0x9A–0x9C, 0x9E, 0xAA–0xAE, 0xCB–0xCF, 0xDB–0xDF, 0xEB–0xEF, and 0xFB–0xFE). Applications may still store data containing these bytes, for example, by inappropriately storing binary data in character fields. Character values containing such unassigned bytes cannot be converted to another character set using this code page as the source.

The solution depends on the migration strategy:

- **Data-only migration:** A customized source character set must be defined to allow round-trip conversion of all byte values between the source and target character sets. Typically, the third-party translation layer between the unmodified application and the Oracle

Database Client is the best place to apply such a customized character set. With this approach, you must first use a third-party utility to convert EBCDIC files to the target ASCII-based character set using the customized character set. The resulting files can then be loaded into Oracle using standard utilities, without further character set conversion.

- **Application rewriting:** If applications are rewritten as part of migration, any binary data currently stored in character fields should be moved to binary data types (such as RAW or BLOB) and processed accordingly. A customized source character set may still be required if the unassigned bytes are used for user-specific characters rather than binary data.

Duplicated Codes

A second problem occurs when the EBCDIC source character set maps more than one code to the same Unicode character. For example, EBCDIC code page 918 (used for Arabic) maps both 0x47 and 0x48 to Unicode character U+FE8E. In a conversion cycle to any target database character set and back, only one of these codes will be preserved; the distinction between them is lost. While this may be insignificant for standard character data, it is a critical issue for binary data incorrectly stored in character fields. For applications running unchanged in EBCDIC emulation mode, bytes of binary data must remain unaltered in a round-trip conversion to and from the target database.

The solutions to this duplicated codes issue are essentially the same as those described for undefined codes above.

12.4 Emulating EBCDIC Binary Ordering

This section describes how to use emulated binary collations to ensure that applications observe the same ordering of character values in both the target and source databases.

This section covers the following topics:

- [Dependency on Binary Ordering](#)
- [Emulated Binary Collations](#)
- [Deploying Emulated Binary Collations](#)

12.4.1 Dependency on Binary Ordering

Applications on mainframe systems may use SQL queries with predicates that rely on specific binary ordering of character values—ordering that depends on the underlying character set. For example, the following query is intended to select products with numeric product codes, filtering out those with alphanumeric codes (assuming alphanumeric codes start with a letter):

```
SELECT product_id, product_name
FROM products
WHERE product_code >= '0'
```

This query takes advantage of the fact that, in EBCDIC-based character sets, all basic Latin letters have character codes numerically lower than digits, and thus appear before digits in binary order. As a result, the query returns only products with numeric codes. However, in ASCII-based character sets, letters have character codes higher than those of digits, so the same query would return products with both numeric and alphanumeric codes.

12.4.2 Emulated Binary Collations

To ensure that comparison predicates return the same results in the target Oracle Database as in the source database, you can configure these predicates to use an emulated binary collation corresponding to the source EBCDIC character set.

Emulated binary collations were introduced in Oracle AI Database 26ai, Release Update 23.26.2. For more information, see "[Using Emulated Binary Collations](#)". [Table A-20](#) lists all supported emulated binary collations.

Collation keys generated by emulated binary collations do not exceed the length of the original character values. As a result, linguistic indexes based on these collations have sizes comparable to standard binary indexes. Linguistic indexes—functional indexes created on the expression `NLSSORT(column, 'NLS_SORT=collation_name')`—may be required to support efficient execution plans for predicates that use emulated binary collations. Standard binary indexes cannot be used for this purpose.

An emulated binary collation should be used for data migrated from its base character set. If characters not supported by this character set are added to the database, they should be processed with modernized applications that apply linguistic collations appropriate for user expectations.

12.4.3 Deploying Emulated Binary Collations

There are two primary methods to ensure that query predicates use a given emulated binary collation.

Legacy Session Parameters

Since Oracle Database 10g Release 2 (10.2), you can set the `NLS_COMP` session parameter to `LINGUISTIC` and the `NLS_SORT` session parameter to the name of the desired collation (for example, `BIN_IBM37`). However, this approach applies the selected collation to all character processing in every query and DML statement within the session. Emulated binary collations require more resources than the standard `BINARY` collation. For data where EBCDIC ordering is not required (such as data compared only for equality), the standard `BINARY` collation should be used.

An example of this method is shown in [Example 5-16](#).

Column-Level (Data-Bound) Collations

Available since Oracle Database 12c Release 2 (12.2), column-level collation declarations allow you to apply a specific collation only to character operations on selected columns. This feature, called data-bound collation, enables more granular control. Refer to "[Column-Level Collation and Case Sensitivity](#)" for prerequisites and restrictions regarding data-bound collations.

[Example 12-4](#) demonstrates the use of column-level collation declarations with queries from [Example 5-16](#).

Example 12-4 Declaring Column Collation BIN_IBM37

This example illustrates the difference between binary ordering on ASCII- and EBCDIC-based platforms using the emulated binary collation for the EBCDIC code page IBM37. It

demonstrates how to control collation using column-level data-bound collation declarations. Confirmation messages from SQL*Plus are omitted for brevity.

```
SQL> ALTER SESSION SET NLS_COMP = LINGUISTIC;
SQL> ALTER SESSION SET NLS_SORT = FRENCH;
SQL> CREATE TABLE emp4
  2 (id NUMBER, ename VARCHAR2(100) COLLATE BINARY);
SQL> INSERT INTO emp4 VALUES (1, 'Clara');
SQL> INSERT INTO emp4 VALUES (2, 'Eva');
SQL> INSERT INTO emp4 VALUES (3, 'Cindy');
SQL> INSERT INTO emp4 VALUES (4, 'de Vil');
SQL> INSERT INTO emp4 VALUES (5, 'Barbara');
SQL> REM Data sorted using binary sort order, not FRENCH
SQL> SELECT ename FROM emp4 WHERE ename > '0' ORDER BY ename;
```

ENAME

Barbara
Cindy
Clara
Eva
de Vil

```
SQL> DROP TABLE emp4 PURGE;
SQL> CREATE TABLE emp4
  2 (id NUMBER, ename VARCHAR2(100) COLLATE BIN_IIBM37);
SQL> INSERT INTO emp4 VALUES (1, 'Clara');
SQL> INSERT INTO emp4 VALUES (2, 'Eva');
SQL> INSERT INTO emp4 VALUES (3, 'Cindy');
SQL> INSERT INTO emp4 VALUES (4, 'de Vil');
SQL> INSERT INTO emp4 VALUES (5, 'Barbara');
SQL> REM Data sorted using BIN_IIBM37 sort order, not FRENCH
SQL> SELECT ename FROM emp4 WHERE ename > '0' ORDER BY ename;
```

no rows selected

```
SQL> SELECT ename FROM emp4 WHERE ename > 'a' ORDER BY ename;
```

ENAME

de Vil
Barbara
Cindy
Clara
Eva

13

Customizing Locale Data

This chapter describes how to customize locale data and includes the following topics:

- [Overview of the Oracle Locale Builder Utility](#)
- [Creating a New Language Definition with Oracle Locale Builder](#)
- [Creating a New Territory Definition with the Oracle Locale Builder](#)
- [Displaying a Code Chart with the Oracle Locale Builder](#)
- [Creating a New Character Set Definition with the Oracle Locale Builder](#)
- [Creating a New Linguistic Sort with the Oracle Locale Builder](#)
- [Generating and Installing NLB Files](#)
- [Upgrading Custom NLB Files from Previous Releases of Oracle Database](#)
- [Deploying Custom NLB Files to Oracle Installations on the Same Platform](#)
- [Deploying Custom NLB Files to Oracle Installations on Another Platform](#)
- [Adding Custom Locale Definitions to Java Components with the GINSTALL Utility](#)
- [Customizing Calendars with the NLS Calendar Utility](#)

13.1 Overview of the Oracle Locale Builder Utility

The Oracle Locale Builder offers an easy and efficient way to customize locale data. It provides a graphical user interface through which you can easily view, modify, and define locale-specific data. It extracts data from the text and binary definition files and presents them in a readable format so that you can process the information without worrying about the formats used in these files.

The Oracle Locale Builder manages four types of locale definitions: language, territory, character set, and linguistic sort. It also supports user-defined characters and customized linguistic rules. You can view definitions in existing text and binary definition files and make changes to them, or create your own definitions.

This section contains the following topics:

- [Configuring Unicode Fonts for the Oracle Locale Builder](#)
- [The Oracle Locale Builder User Interface](#)
- [Oracle Locale Builder Pages and Dialog Boxes](#)

13.1.1 Configuring Unicode Fonts for the Oracle Locale Builder

The Oracle Locale Builder uses Unicode characters in many of its functions. For example, it shows the mapping of local character code points to Unicode code points. The Oracle Locale Builder depends on the logical fonts *Serif* and *SansSerif* that are configured in Java Runtime to display the characters. If a character cannot be rendered with the configured fonts, then it is usually displayed as a rectangular box. If you cannot see some characters properly in the

Oracle Locale Builder user interface, then you may need to reconfigure the logical fonts to include additional physical fonts supporting the missing characters.

① Note

The Java Runtime used by the Oracle Locale Builder is located in the `jdk/jre` subdirectory of the Oracle Home directory.

① See Also

The technical note "Font Configuration Files" at <https://docs.oracle.com/javase/8/docs/technotes/guides/intl/fontconfig.html> for more information about the font configuration files used by the Java Runtime.

13.1.2 The Oracle Locale Builder User Interface

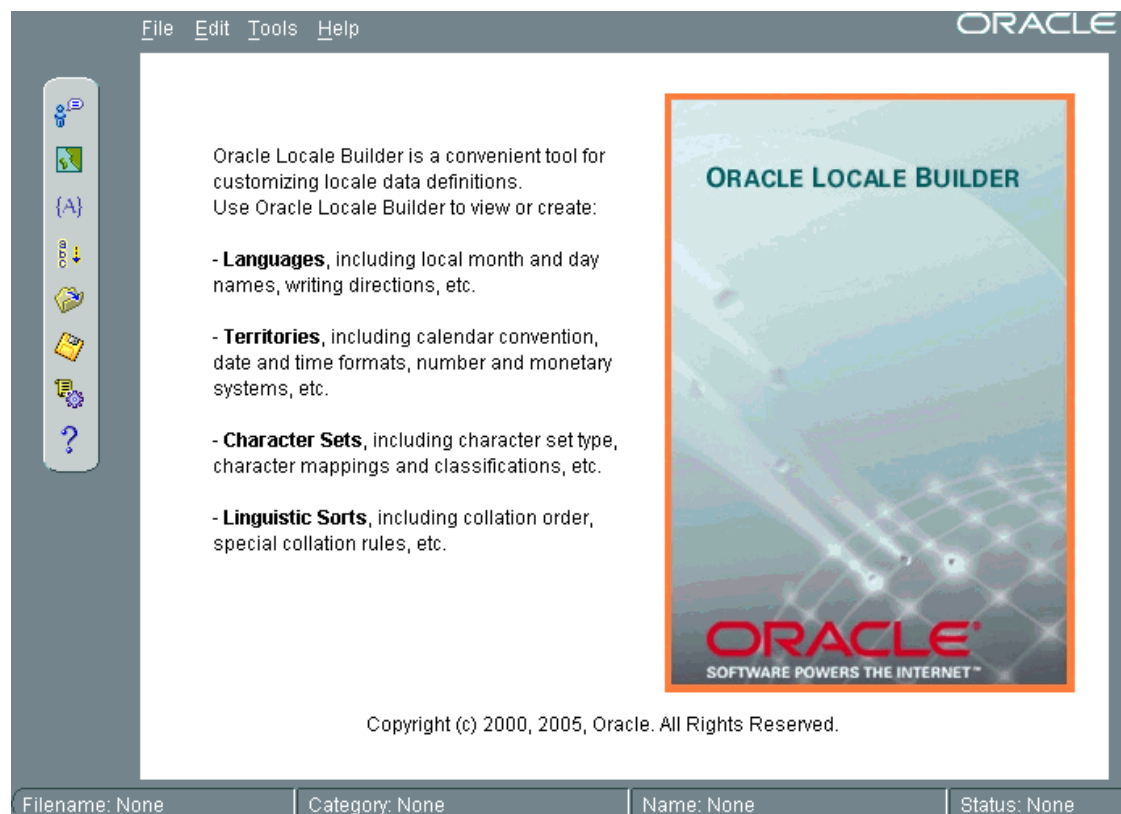
Ensure that the `ORACLE_HOME` parameter is set before starting Oracle Locale Builder.

In the UNIX operating system, start the Oracle Locale Builder by changing into the `$ORACLE_HOME/nls/lbuilder` directory and issuing the following command:

```
% ./lbuilder
```

In a Windows operating system, start the Oracle Locale Builder from the Start menu as follows: **Start > Programs > Oracle-OraHome10 > Configuration and Migration Tools > Locale Builder**. You can also start it from the DOS prompt by entering the `%ORACLE_HOME%\nls\lbuilder` directory and executing the `lbuilder.bat` command.

When you start the Oracle Locale Builder, the following screen appears.

Figure 13-1 Oracle Locale Builder Utility

13.1.3 Oracle Locale Builder Pages and Dialog Boxes

Before using Oracle Locale Builder for a specific task, you should become familiar with the following tab pages and dialog boxes:

- [Existing Definitions Dialog Box](#)
- [Session Log Dialog Box](#)
- [Preview NLT Tab Page](#)
- [Open File Dialog Box](#)

Note

Oracle Locale Builder includes online help.

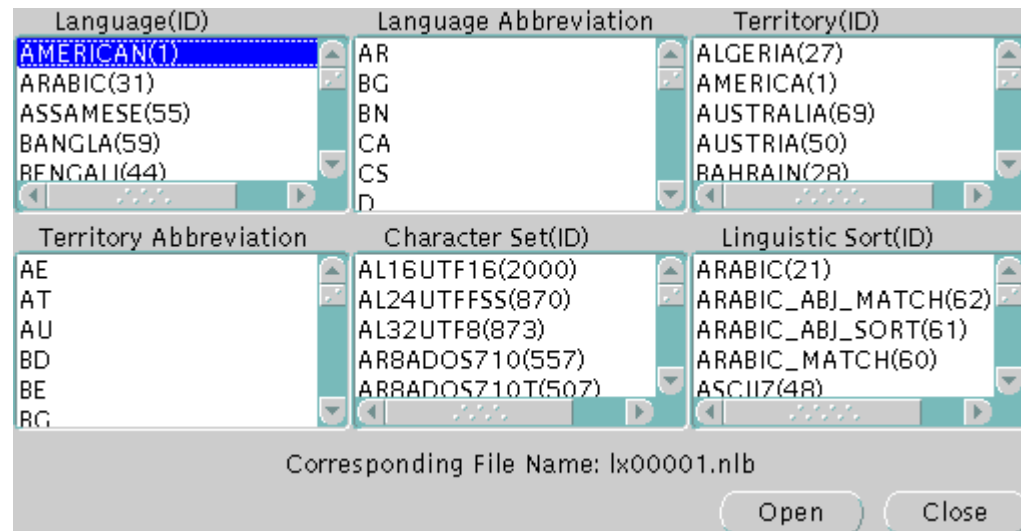
13.1.3.1 Existing Definitions Dialog Box

When you choose **New Language**, **New Territory**, **New Character Set**, or **New Linguistic Sort**, the first tab page that you see is labeled **General**. Click **Show Existing Definitions** to see the Existing Definitions dialog box.

The Existing Definitions dialog box enables you to open locale objects by name. If you know a specific language, territory, linguistic sort (collation), or character set that you want to start with,

then click its displayed name. For example, you can open the AMERICAN language definition file as shown in the following screen.

Figure 13-2 Existing Definitions Dialog Box

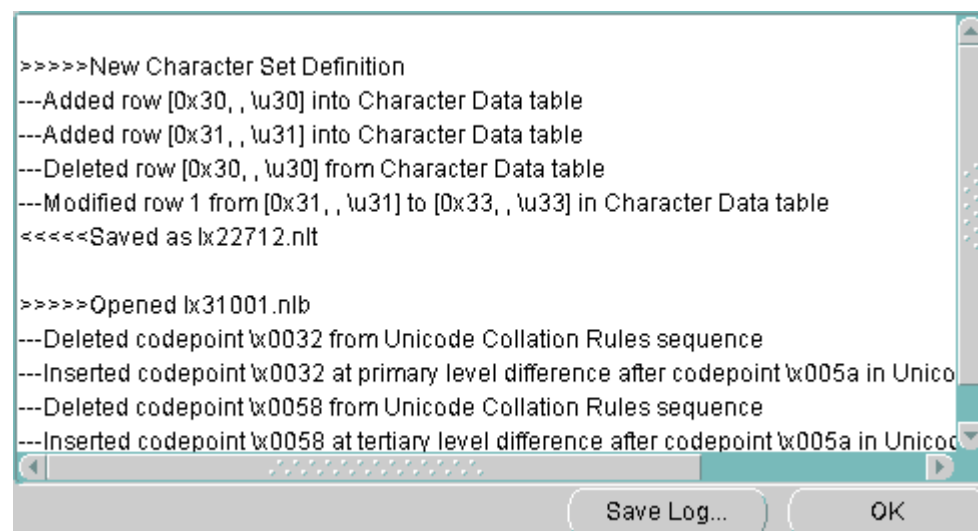


Choosing AMERICAN opens the `l\00001.nlb` file. An NLB file is a binary file that contains the settings for a specific language, territory, character set, or linguistic sort.

Language and territory abbreviations are for reference only and cannot be opened.

13.1.3.2 Session Log Dialog Box

Choose **Tools > View Log** to see the Session Log dialog box. The Session Log dialog box shows what actions have been taken in the current session. Click **Save Log** to keep a record of all changes. The following screen shows an example of a session log.

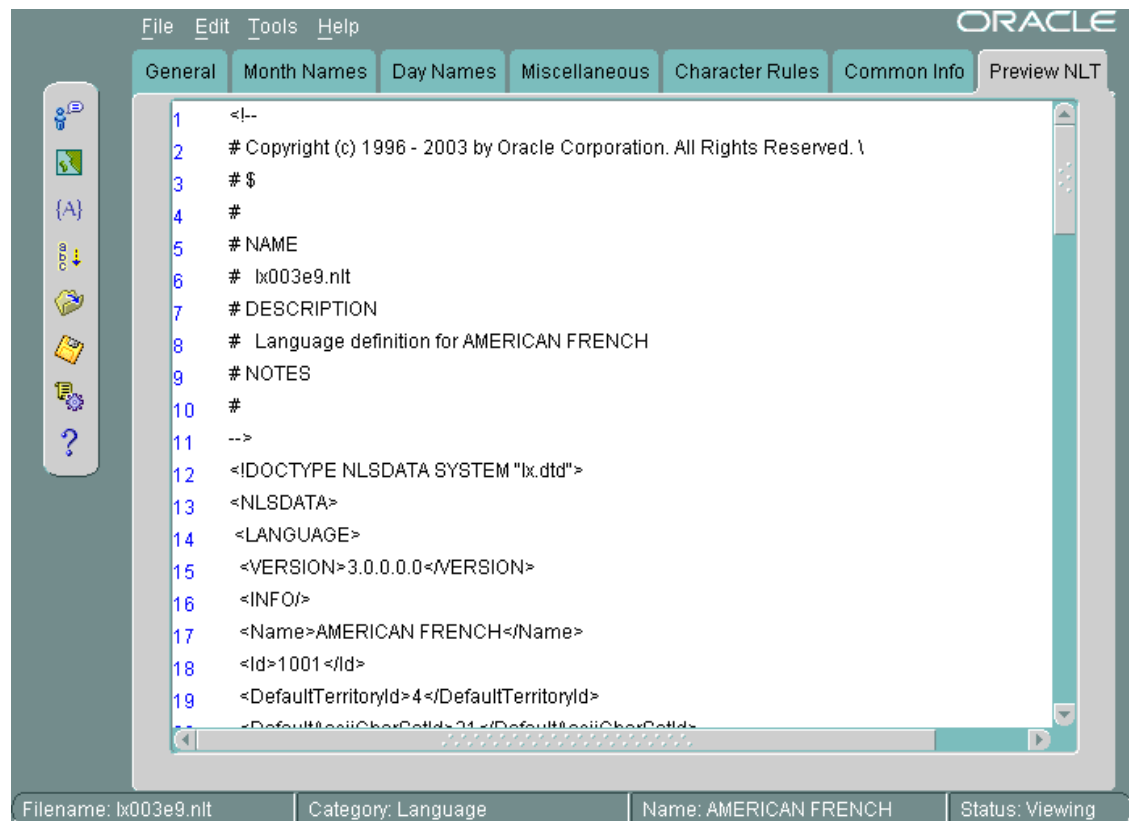
Figure 13-3 Session Log Dialog Box

13.1.3.3 Preview NLT Tab Page

The NLT (National Language Text) file is an XML file with the file extension `.nlt` that stores the settings for a specific language, territory, character set, or linguistic sort. The Preview NLT tab page presents a readable form of the file so that you can see whether the changes you have made are correct. You cannot modify the NLT file from the Preview NLT tab page. You must use the specific tools and procedures available in Oracle Locale Builder to modify the NLT file.

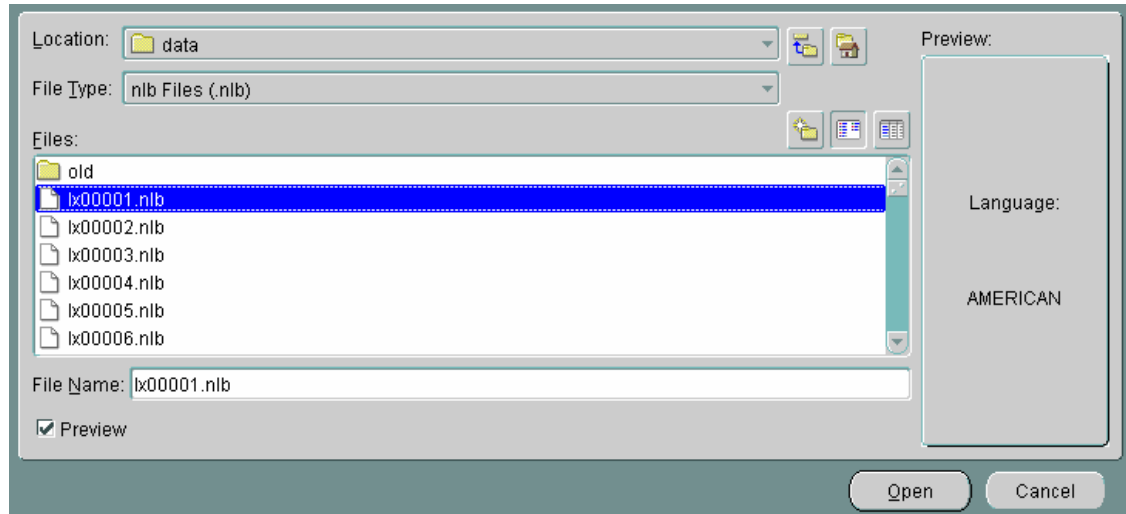
The following screen shows an example of the Preview NLT tab page for a user-defined language called AMERICAN FRENCH.

Figure 13-4 Previewing the NLT File



13.1.3.4 Open File Dialog Box

You can see the Open File dialog box by choosing **File > Open > By File Name**. Then choose the NLB (National Language Binary) file that you want to modify or use as a template. An NLB file is a binary file with the file extension `.nlb` that contains the binary equivalent of the information in the NLT file. The following screen shows the Open File dialog box with the `lx00001.nlb` file selected. The Preview pane shows that this NLB file is for the AMERICAN language.

Figure 13-5 Open File Dialog Box

13.2 Creating a New Language Definition with Oracle Locale Builder

This section shows how to create a new language based on French. This new language is called AMERICAN FRENCH. First, open FRENCH from the Existing Definitions dialog box. Then change the language name to **AMERICAN FRENCH** and the Language Abbreviation to **AF** in the General tab page. Retain the default values for the other fields. The following screen shows the resulting General tab page.

Figure 13-6 Language General Information

The screenshot shows the Oracle Locale Builder interface with the 'General' tab selected. The 'Language Name' is 'AMERICAN FRENCH', 'Language ID' is '1001', 'Language Abbreviation' is 'AF', 'Default Territory' is 'FRANCE', 'Default ASCII Character Set' is 'WE8ISO8859P1', 'Default Ebcidic Character Set' is 'WE8EBCDIC1047', and 'Default Linguistic Definition' is 'FRENCH'. A 'Show Existing Definitions...' button is at the bottom. The status bar at the bottom shows 'Filename: lx00003.nlb', 'Category: Language', 'Name: FRENCH', and 'Status: Editing'.

Language Name:	AMERICAN FRENCH
Language ID:	1001
Language Abbreviation:	AF
Default Territory:	FRANCE
Default ASCII Character Set:	WE8ISO8859P1
Default Ebcidic Character Set:	WE8EBCDIC1047
Default Linguistic Definition:	FRENCH

Show Existing Definitions...

Filename: lx00003.nlb Category: Language Name: FRENCH Status: Editing

The following restrictions apply when choosing names for locale objects such as languages:

- Names must contain only ASCII characters
- Names must start with a letter and cannot have leading or trailing blanks
- Language, territory, and character set names cannot contain underscores or periods

The valid range for the Language ID field for a user-defined language is 1,000 to 10,000. You can accept the value provided by Oracle Locale Builder or you can specify a value within the range.

Note

Only certain ID ranges are valid values for user-defined LANGUAGE, TERRITORY, CHARACTER SET, MONOLINGUAL COLLATION, and MULTILINGUAL COLLATION definitions. The ranges are specified in the sections of this chapter that concern each type of user-defined locale object.

The following screen shows how to set month names using the Month Names tab page.

Figure 13-7 Month Names Tab Page

File Edit Tools Help

General Month Names Day Names Miscellaneous Character Rules Common Info Preview NLT

Capitalize initial letter of month names?

☒ Yes ☐ No (or non-applicable)

	Full Month Names	Abbreviated Month Names
Month 01:	janvier	janv.
Month 02:	février	févr.
Month 03:	mars	mars
Month 04:	avril	avr.
Month 05:	mai	mai
Month 06:	juin	juin
Month 07:	juillet	juil.
Month 08:	août	août
Month 09:	septembre	sept.
Month 10:	octobre	oct.
Month 11:	novembre	nov.
Month 12:	décembre	déc.

Filename: lx00003.nlb Category: Language Name: FRENCH Status: Viewing

All names are shown as they appear in the NLT file. If you choose **Yes** for capitalization, then the month names are capitalized in your application, but they do not appear capitalized in the Month Names tab page.

The following screen shows the Day Names tab page.

Figure 13-8 Day Names Tab Page

File Edit Tools Help

General Month Names Day Names Miscellaneous Character Rules Common Info Preview NLT

Capitalize initial letter of day names?

☒ Yes ☐ No (or non-applicable)

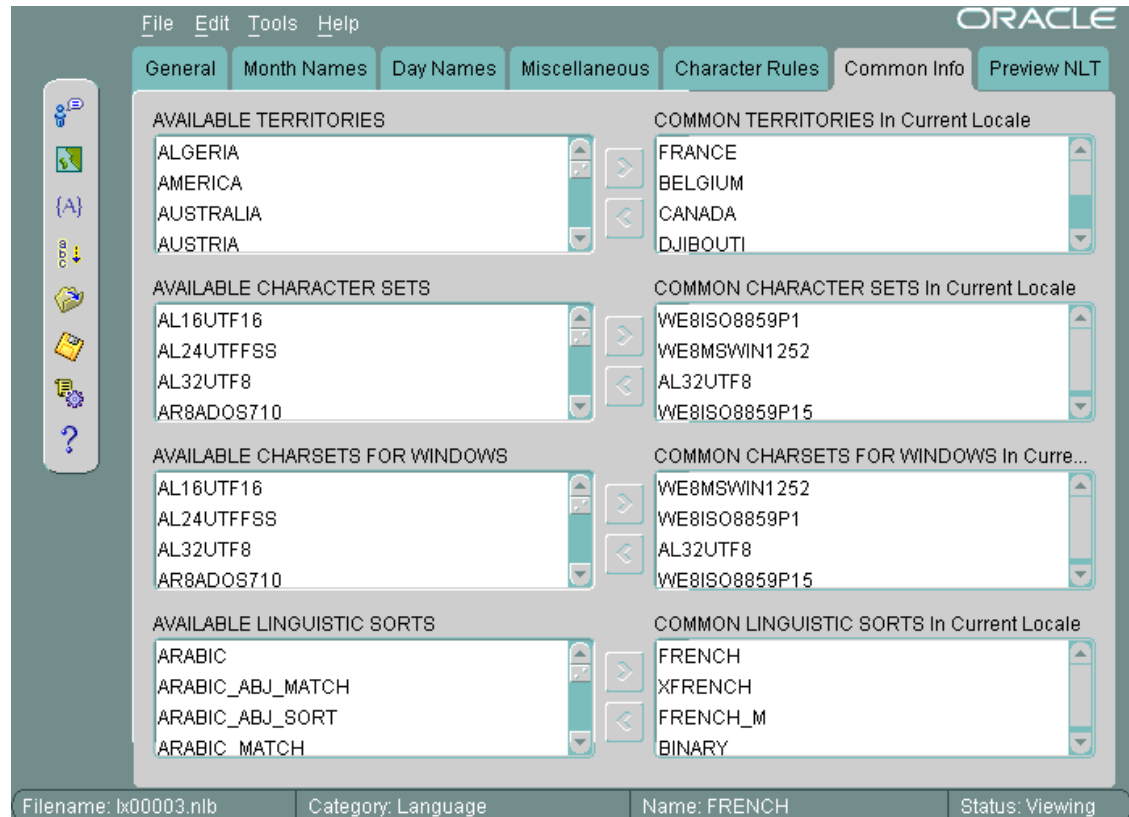
	Full Day Names	Abbreviated Day Names
Sunday:	dimanche	dim.
Monday:	lundi	lun.
Tuesday:	mardi	mar.
Wednesday:	mercredi	mer.
Thursday:	jeudi	jeu.
Friday:	vendredi	ven.
Saturday:	samedi	sam.

Filename: lx00003.nlb Category: Language Name: FRENCH Status: Viewing

You can choose day names for your user-defined language. All names are shown as they appear in the NLT file. If you choose **Yes** for capitalization, then the day names are capitalized in your application, but they do not appear capitalized in the Day Names tab page.

The following screen shows the Common Info tab page.

Figure 13-9 Common Info Tab Page



You can display the territories, character sets, Windows character sets, and linguistic sorts that have associations with the current language. In general, the most appropriate or the most commonly used items are displayed first. For example, with a language of FRENCH, the common territories are FRANCE, BELGIUM, CANADA, and DJIBOUTI, while the character sets for supporting French are WE8ISO8859P1, WE8MSWIN1252, AL32UTF8, and WE8ISO8859P15. As WE8MSWIN1252 is more common than WE8ISO8859P1 in a Windows environment, it is displayed first.

13.3 Creating a New Territory Definition with the Oracle Locale Builder

This section shows how to create a new territory called REDWOOD SHORES and use RS as a territory abbreviation. The new territory is not based on an existing territory definition.

The basic tasks are as follows:

- Assign a territory name
- Choose formats for the calendar, numbers, date and time, and currency

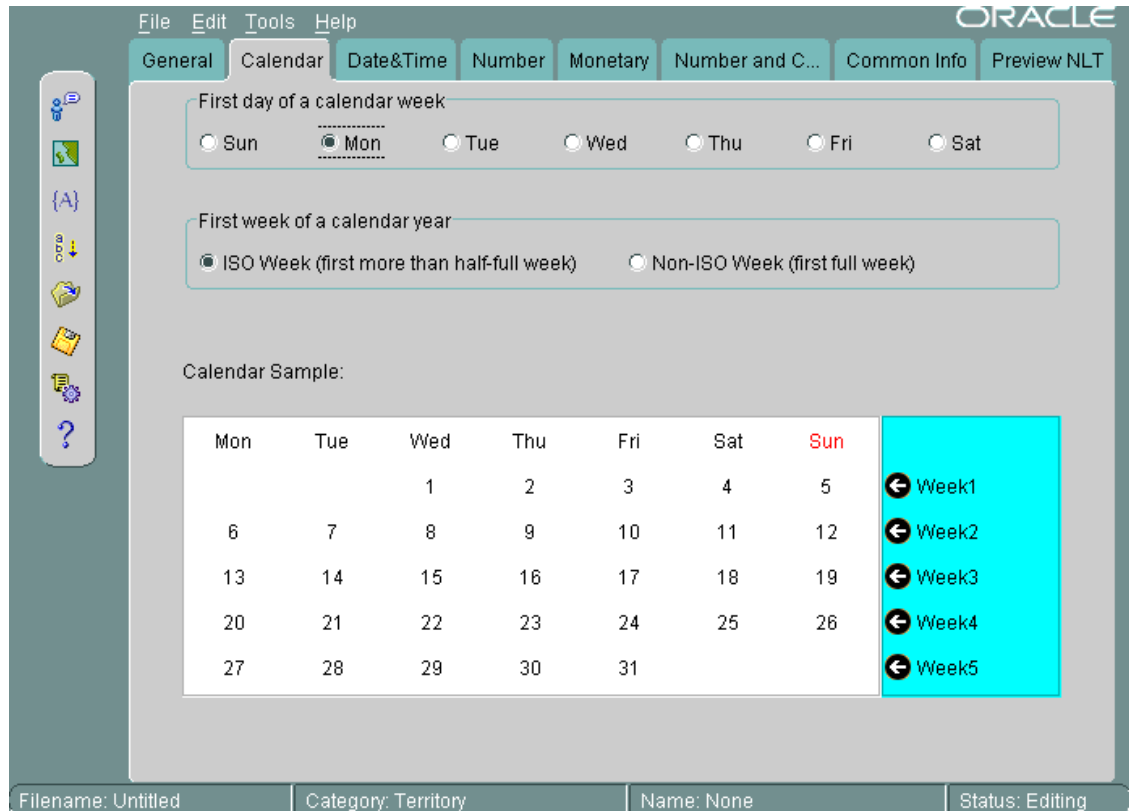
The following screen shows the General tab page with **REDWOOD SHORES** specified as the territory name, **1001** specified as the territory ID, and **RS** specified as the territory abbreviation.

Figure 13-10 General Tab Page for Territories

The screenshot shows the Oracle Locale Builder application window. The title bar includes the Oracle logo and menu items: File, Edit, Tools, Help. The main window has a tabbed interface with the following tabs: General (selected), Calendar, Date&Time, Number, Monetary, Number and C..., Common Info, and Preview NLT. On the left side of the General tab, there is a vertical toolbar with icons for: a folder, a document, a list, a calendar, a currency symbol, a number, a date, a time, a question mark, and a search icon. The main area of the General tab contains four text input fields: "Territory Name:" with the value "REDWOOD SHORES", "Territory ID:" with the value "1001", "Territory Abbreviation:" with the value "RS", and "Territory Variation:" which is empty. Below these fields is a button labeled "Show Existing Definitions...". At the bottom of the window, there is a status bar with four sections: "Filename: Untitled", "Category: Territory", "Name: None", and "Status: Editing".

The valid range for territory ID for a user-defined territory is 1000 to 10000.

The following screen shows settings for calendar formats in the Calendar tab page.

Figure 13-11 Choosing Calendar Formats

Monday is set as the first day of the week, and the first week of the calendar year is set as an ISO week.

See Also

- "[Calendar Formats](#)" for more information about choosing the first day of the week and the first week of the calendar year
- "[Customizing Calendars with the NLS Calendar Utility](#)" for information about customizing calendars themselves

The following screen shows the Date & Time tab page.

Figure 13-12 Choosing Date and Time Formats

The screenshot displays the Oracle Locale Builder application with the 'Date&Time' tab selected. The interface includes a menu bar (File, Edit, Tools, Help) and a toolbar on the left. The main area contains the following settings:

- Short Date Format:** DD-MM-YY
- Short Date Sample:** 20-10-05
- Short Time Format:** HH24:MI:SS
- Short Time Sample:** 15:27:26
- Combined short date&time sample:** 20-10-05 15:27:26
- Oracle Date Format:** DD-MM-YY
- Oracle Date Sample:** 20-10-05
- Long Date Format:** fmDay, Month dd, yyyy
- Long Date Sample:** Thursday, October 20, 2005
- TimeStamp Timezone Format:** (empty)
- TimeStamp Timezone Sample:** (empty)

The bottom status bar indicates: Filename: Untitled, Category: Territory, Name: None, Status: Editing.

When you choose a format from a list, Oracle Locale Builder displays an example of the format. In this case, the Short Date Format is set to **DD-MM-YY**. The Short Time Format is set to **HH24:MI:SS**. The Oracle Date Format is set to **DD-MM-YY**. The Long Date Format is set to **fmDay, Month dd, yyyy**. The TimeStamp Timezone Format is not set.

You can also enter your own formats instead of using the selection from the drop-down menus.

See Also

- ["Date Formats"](#)
- ["Time Formats"](#)
- ["Choosing a Time Zone File"](#)

The following screen shows the Number tab page.

Figure 13-13 Choosing Number Formats

The screenshot shows the Oracle Locale Builder interface with the 'Number' tab selected. The interface includes a menu bar (File, Edit, Tools, Help) and a tab bar (General, Calendar, Date&Time, Number, Monetary, Number and C..., Common Info, Preview NLT). A vertical toolbar on the left contains icons for various functions. The main area displays the following settings:

- Decimal Symbol: .
- Negative Sign Location: ☒ -100 ☐ 100-
- Numeric Group Separator: ,
- Number Grouping: 3
- Number Sample: -1,234.12
- List Separator: ,
- Measurement System: Metric
- Rounding Indicator (value greater than which to round up): 4
- Rounding Sample: 10.4 is rounded to 10 and 10.5 is rounded to 11

At the bottom, a status bar shows: Filename: Untitled | Category: Territory | Name: None | Status: Editing

A period has been chosen for the Decimal Symbol. The Negative Sign Location is specified to be on the left of the number. The Numeric Group Separator is a comma. The Number Grouping is specified as 3 digits. The List Separator is a comma. The Measurement System is metric. The Rounding Indicator is 4.

You can enter your own values instead of using values in the lists.

When you choose a format from a list, Oracle Locale Builder displays an example of the format.

See Also

["Numeric Formats"](#)

The following screen shows settings for currency formats in the Monetary tab page.

Figure 13-14 Choosing Currency Formats

The screenshot shows the Oracle Locale Builder interface with the 'Monetary' tab selected. The settings are as follows:

Field	Value
Local Currency Symbol:	\$
Alternative Currency Symbol:	€
Currency Presentation:	-\$100
Decimal Symbol:	.
Group Separator:	,
Monetary Number Grouping:	3
Monetary Precision:	3
Credit Symbol:	+
Debit Symbol:	-
International Currency Separator:	
International Currency Symbol:	USD

Below the settings, two examples are shown:

- Credit: + \$ 1,234.123
- Debit: - \$ 1,234.123

At the bottom, a preview shows the format: 1,234 USD.

The status bar at the bottom indicates: Filename: Untitled, Category: Territory, Name: None, Status: Editing.

The Local Currency symbol is set to \$. The Alternative Currency Symbol is the euro symbol. The Currency Presentation shows one of several possible sequences of the local currency symbol, the debit symbol, and the number. The Decimal Symbol is the period. The Group Separator is the comma. The Monetary Number Grouping is 3. The Monetary Precision or number of digits after the decimal symbol, is 3. The Credit Symbol is +. The Debit Symbol is -. The International Currency Separator is a blank space, so it is not visible in the field. The International Currency Symbol (ISO currency symbol) is **USD**. Oracle Locale Builder displays examples of the currency formats you have selected.

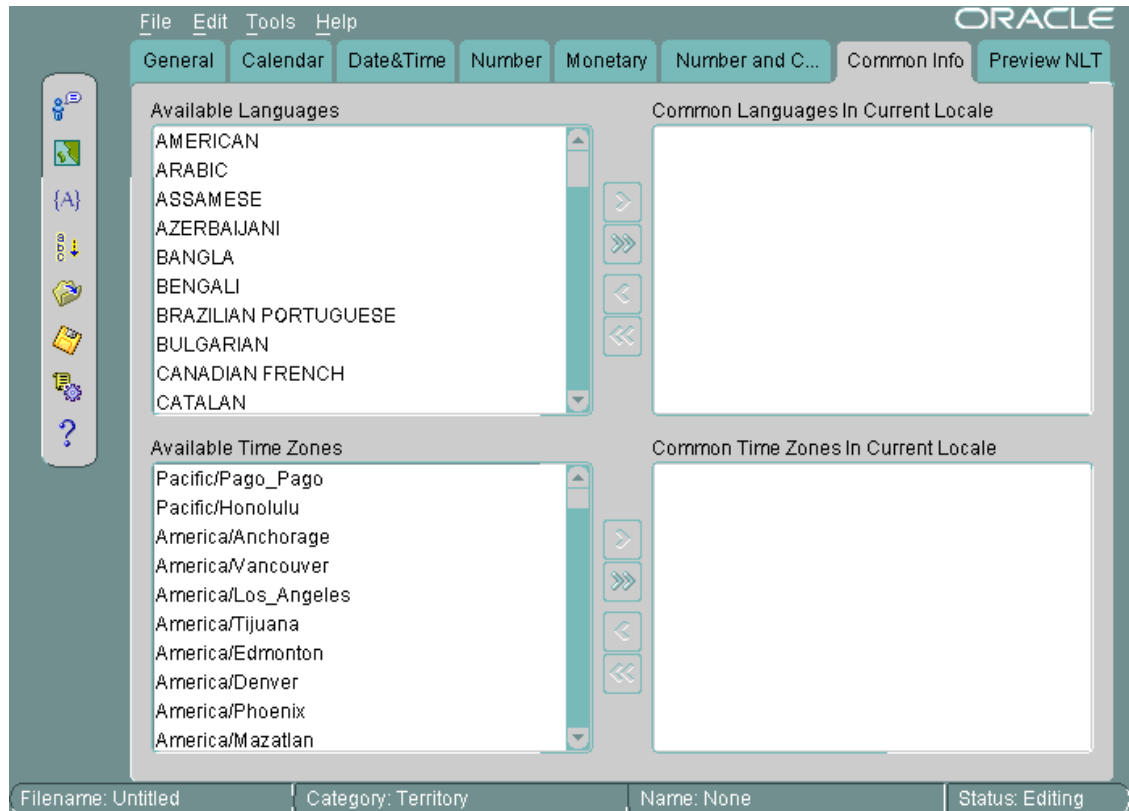
You can enter your own values instead of using the lists.

See Also

["Currency Formats"](#)

The following screen shows the Common Info tab page.

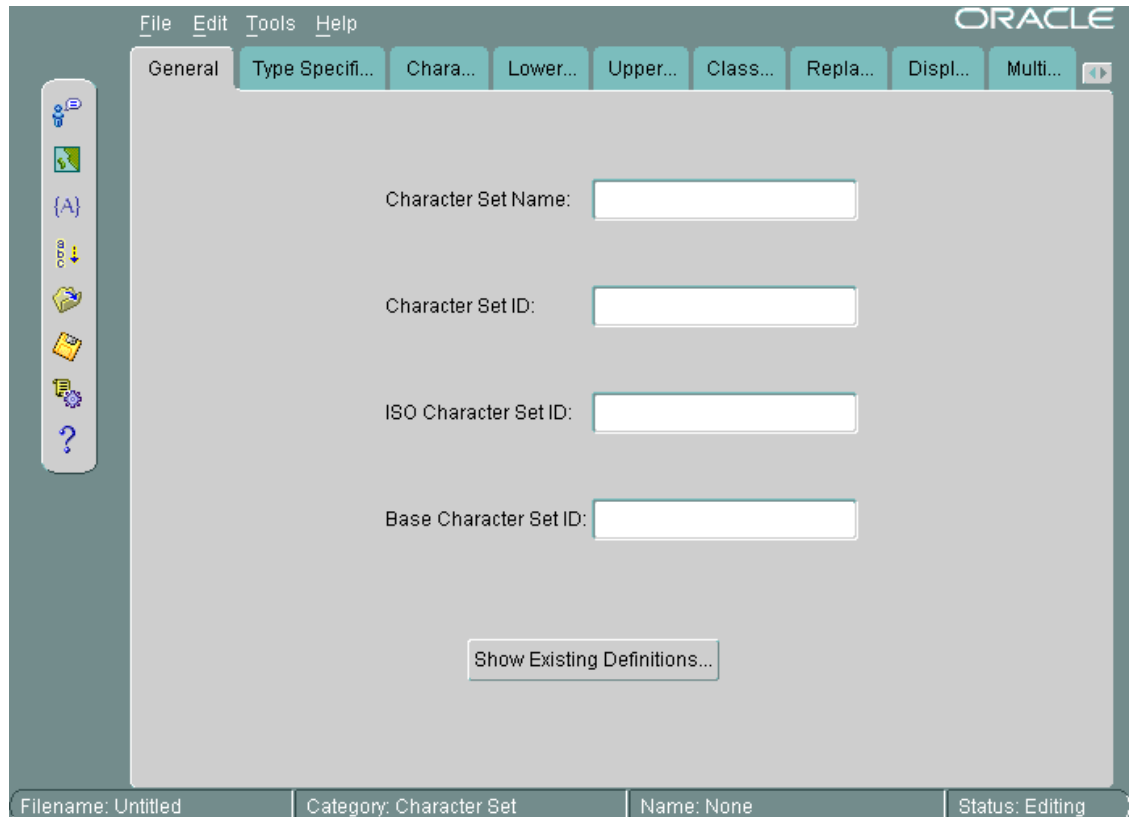
Figure 13-15 Common Info Tab Page



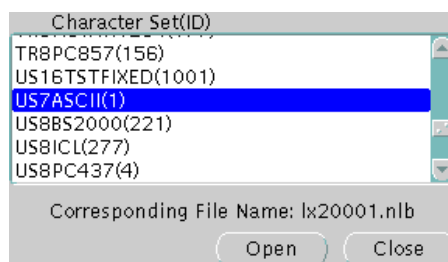
You can display the common languages and time zones for the current territory. For example, with a territory of CANADA, the common languages are ENGLISH, CANADIAN FRENCH, and FRENCH. The common time zones are America/Montreal, America/St_Johns, America/Halifax, America/Winnipeg, America/Regina, America/Edmonton, and America/Vancouver.

13.4 Displaying a Code Chart with the Oracle Locale Builder

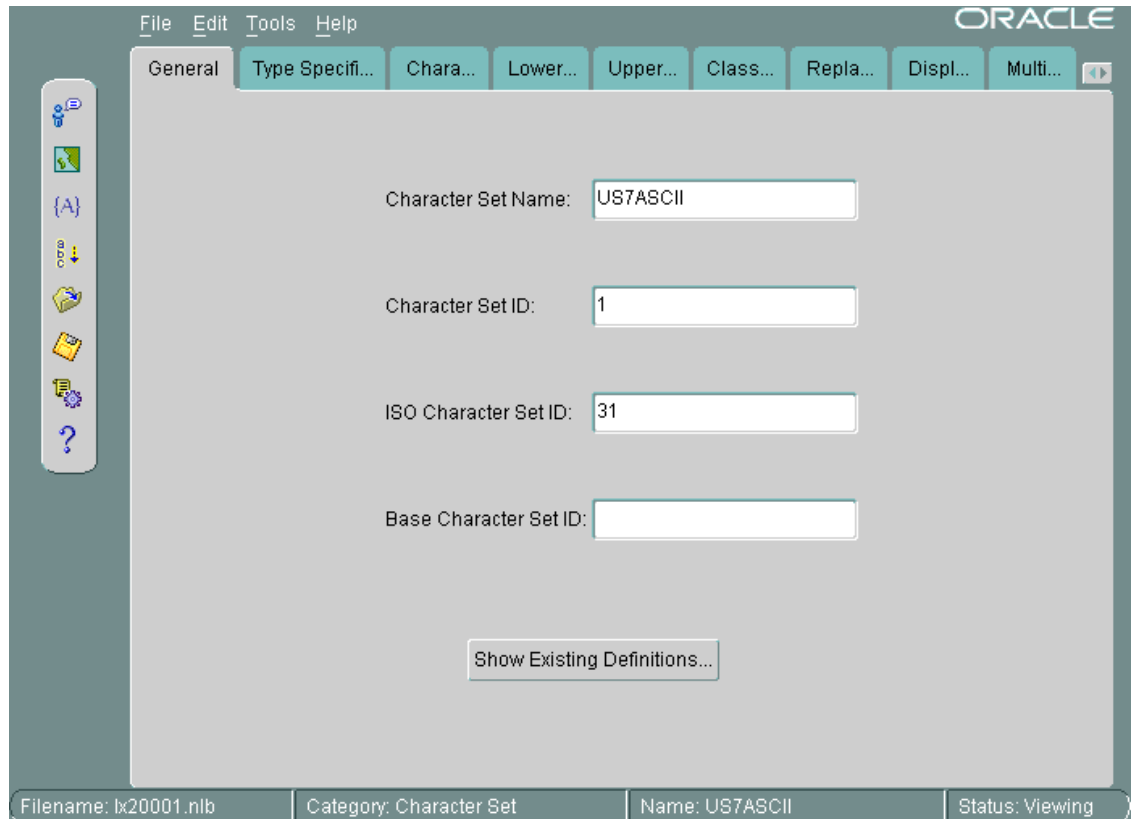
You can display and print the code charts of character sets with the Oracle Locale Builder. From the opening screen for Oracle Locale Builder, choose **File > New > Character Set**. The following screen is displayed.

Figure 13-16 General Tab Page for Character Sets

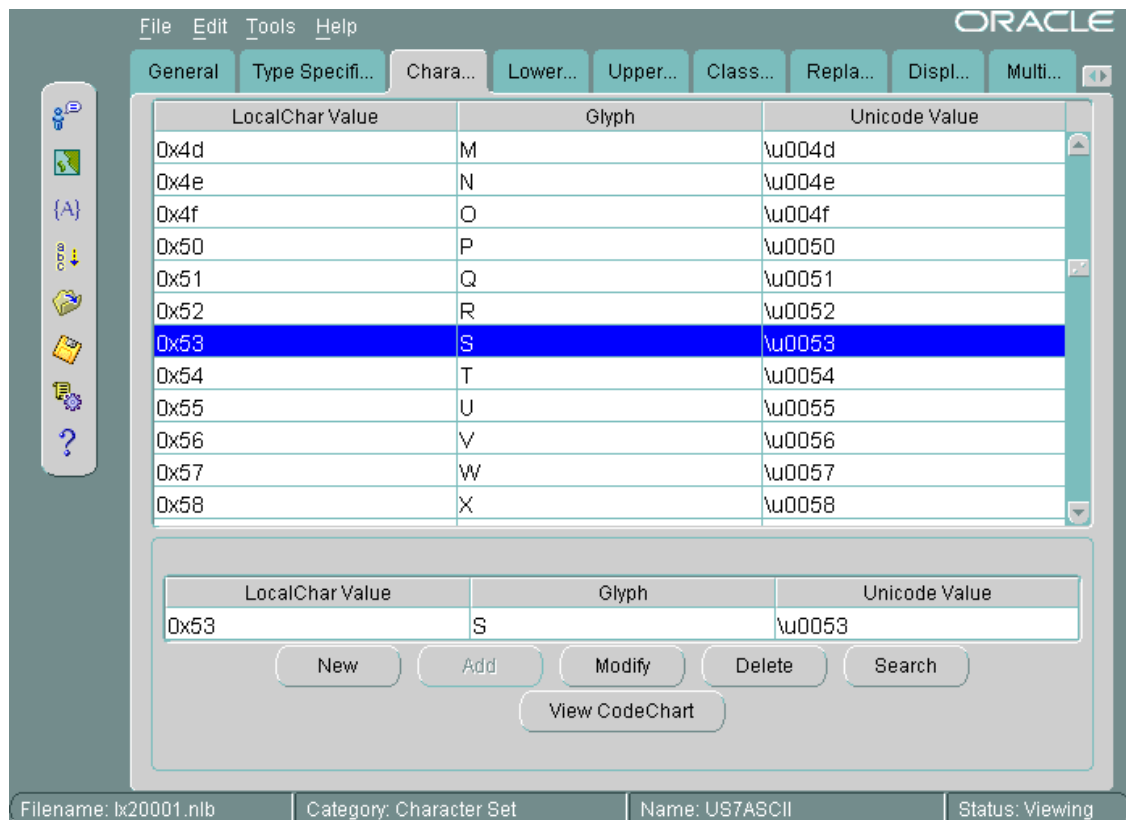
Click **Show Existing Definitions**. Highlight the character set you want to display. The following screen shows the Existing Definitions combo box with US7ASCII highlighted.

Figure 13-17 Choosing US7ASCII in the Existing Definitions Dialog Box

Click **Open** to choose the character set. The following screen shows the General tab page when US7ASCII has been chosen.

Figure 13-18 General Tab Page When US7ASCII Has Been Chosen

Select the Character Data Mapping tab. The following screen shows the Character Data Mapping tab page for US7ASCII.

Figure 13-19 Character Data Mapping Tab Page for US7ASCII

Click **View CodeChart**. The following screen shows the code chart for US7ASCII.

Figure 13-20 US7ASCII Code Chart

0x18	0x19	0x1a	0x1b	0x1c	0x1d	0x1e	0x1f	0x20	0x21	0x22	0x23
□	□	□	□	□	□	□	□		!	"	#
0x0018	0x0019	0x001a	0x001b	0x001c	0x001d	0x001e	0x001f	0x0020	0x0021	0x0022	0x0023
0x24	0x25	0x26	0x27	0x28	0x29	0x2a	0x2b	0x2c	0x2d	0x2e	0x2f
\$	%	&	'	(	)	*	+	,	-	.	/
0x0024	0x0025	0x0026	0x0027	0x0028	0x0029	0x002a	0x002b	0x002c	0x002d	0x002e	0x002f
0x30	0x31	0x32	0x33	0x34	0x35	0x36	0x37	0x38	0x39	0x3a	0x3b
0	1	2	3	4	5	6	7	8	9	:	;
0x0030	0x0031	0x0032	0x0033	0x0034	0x0035	0x0036	0x0037	0x0038	0x0039	0x003a	0x003b
0x3c	0x3d	0x3e	0x3f	0x40	0x41	0x42	0x43	0x44	0x45	0x46	0x47
<	=	>	?	@	A	B	C	D	E	F	G
0x003c	0x003d	0x003e	0x003f	0x0040	0x0041	0x0042	0x0043	0x0044	0x0045	0x0046	0x0047
0x48	0x49	0x4a	0x4b	0x4c	0x4d	0x4e	0x4f	0x50	0x51	0x52	0x53
H	I	J	K	L	M	N	O	P	Q	R	S
0x0048	0x0049	0x004a	0x004b	0x004c	0x004d	0x004e	0x004f	0x0050	0x0051	0x0052	0x0053
0x54	0x55	0x56	0x57	0x58	0x59	0x5a	0x5b	0x5c	0x5d	0x5e	0x5f
T	U	V	W	X	Y	Z	[	\	]	^	_
0x0054	0x0055	0x0056	0x0057	0x0058	0x0059	0x005a	0x005b	0x005c	0x005d	0x005e	0x005f
0x60	0x61	0x62	0x63	0x64	0x65	0x66	0x67	0x68	0x69	0x6a	0x6b
`	a	b	c	d	e	f	g	h	i	j	k
0x0060	0x0061	0x0062	0x0063	0x0064	0x0065	0x0066	0x0067	0x0068	0x0069	0x006a	0x006b
0x6c	0x6d	0x6e	0x6f	0x70	0x71	0x72	0x73	0x74	0x75	0x76	0x77
l	m	n	o	p	q	r	s	t	u	v	w
0x006c	0x006d	0x006e	0x006f	0x0070	0x0071	0x0072	0x0073	0x0074	0x0075	0x0076	0x0077
0x78	0x79	0x7a	0x7b	0x7c	0x7d	0x7e	0x7f	0x80	0x81	0x82	0x83
x	y	z	{		}	~	□	□	□	□	□
0x0078	0x0079	0x007a	0x007b	0x007c	0x007d	0x007e	0x007f	0x0080	0x0081	0x0082	0x0083

It shows the encoded value of each character in the local character set, the glyph associated with each character, and the Unicode value of each character in the local character set.

If you want to print the code chart, then click **Print Page**.

13.5 Creating a New Character Set Definition with the Oracle Locale Builder

You can customize a character set to meet specific user needs. You can extend an existing encoded character set definition. User-defined characters are often used to encode special characters that represent the following language elements:

- Proper names
- Historical Han characters that are not defined in an existing character set standard
- Vendor-specific characters
- New symbols or characters that you define

This section describes how Oracle Database supports user-defined characters. It includes the following topics:

- [Character Sets with User-Defined Characters](#)
- [Oracle Database Character Set Conversion Architecture](#)
- [Unicode Private Use Area](#)
- [User-Defined Character Cross-References Between Character Sets](#)

- [Guidelines for Creating a New Character Set from an Existing Character Set](#)
- [Example: Creating a New Character Set Definition with the Oracle Locale Builder](#)

13.5.1 Character Sets with User-Defined Characters

User-defined characters are typically supported within East Asian character sets. These East Asian character sets have at least one range of reserved code points for user-defined characters. For example, Japanese Shift-JIS preserves 1880 code points for user-defined characters. They are shown in [Table 13-1](#).

Table 13-1 Shift JIS User-Defined Character Ranges

Japanese Shift JIS User-Defined Character Range	Number of Code Points
F040-F07E, F080-F0FC	188
F140-F17E, F180-F1FC	188
F240-F27E, F280-F2FC	188
F340-F37E, F380-F3FC	188
F440-F47E, F480-F4FC	188
F540-F57E, F580-F5FC	188
FF640-F67E, F680-F6FC	188
F740-F77E, F780-F7FC	188
F840-F87E, F880-F8FC	188
F940-F97E, F980-F9FC	188

The Oracle Database character sets listed in [Table 13-2](#) contain predefined ranges that support user-defined characters.

Table 13-2 Oracle Database Character Sets with User-Defined Character Ranges

Character Set Name	Number of Code Points Available for User-Defined Characters
JA16DBCS	4370
JA16EBDIC930	4370
JA16SJIS	1880
JA16SJISYEN	1880
KO16DBCS	1880
KO16MSWIN949	1880
ZHS16DBCS	1880
ZHS16GBK	2149
ZHT16DBCS	6204
ZHT16MSWIN950	6217

13.5.2 Oracle Database Character Set Conversion Architecture

The code point value that represents a particular character can vary among different character sets. A Japanese kanji character is shown in the following figure.

Figure 13-21 Japanese Kanji Character

The following table shows how the character is encoded in different character sets.

Unicode Encoding	JA16SJIS Encoding	JA16EUC Encoding	JA16DBCS Encoding
4E9C	889F	B0A1	4867

Oracle Database defines all character sets with respect to Unicode code points. That is, each character is defined as a Unicode code value. Character conversion takes place transparently by using Unicode as the intermediate form. For example, when a JA16SJIS client connects to a JA16EUC database, the Japanese kanji character shown in the above figure has the code point value 889F when it is entered from the JA16SJIS client. It is internally converted to Unicode (with code point value 4E9C), and then converted to JA16EUC (code point value B0A1).

13.5.3 Unicode Private Use Area

In Unicode, the range of code points E000-F8FF is reserved for the Private Use Area (PUA). The PUA is intended for end users' or vendors' private use character definition.

User-defined characters can be converted between two Oracle Database character sets by using Unicode PUA as the intermediate form, which is the same as for the standard characters.

13.5.4 User-Defined Character Cross-References Between Character Sets

Cross-references between different character sets are required when registering user-defined characters across operating systems. Cross-references ensure that the user-defined characters can be converted correctly across the different character sets when they are mapped to a Unicode PUA value.

For example, when registering a user-defined character on both a Japanese Shift-JIS operating system and a Japanese IBM Host operating system, you may want to assign the F040 code point on the Shift-JIS operating system and the 6941 code point on the IBM Host operating system for this character. This is so that Oracle Database can map this character correctly between the character sets JA16SJIS and JA16DBCS.

User-defined character cross-reference information can be found by viewing the character set definitions using the Oracle Locale Builder. For example, you can determine that both the Shift-JIS UDC value F040 and the IBM Host UDC value 6941 are mapped to the same Unicode PUA value E000.

① See Also

["Unicode Character Code Assignments"](#)

13.5.5 Guidelines for Creating a New Character Set from an Existing Character Set

By default, the Oracle Locale Builder generates the next available character set ID for you. You can also choose your own character set ID. Use the following format for naming character set definition NLT files:

`1x2dddd.nlt`

`dddd` is the 4-digit character set ID in hex.

When you modify a character set, observe the following guidelines:

- Do not remap existing characters.
- All character mappings must be unique.
- New characters should be mapped into the Unicode private use range of e000-f4ff.

Note

The actual Unicode private use range is e000-f8ff. However, Oracle Database reserves f500-f8ff for its own private use.

- No line in the character set definition file can be longer than 80 characters.

Note

When you create a new multibyte character set from an existing character set, use an 8-bit or multibyte character set as the original character set.

If you derive a new character set from an existing Oracle Database character set, then Oracle recommends using the following character set naming convention:

`<Oracle_character_set_name><organization_name>EXT<version>`

For example, if a company such as Sun Microsystems adds user-defined characters to the JA16EUC character set, then the following character set name is appropriate:

`JA16EUCSUNWEXT1`

The character set name contains the following parts:

- JA16EUC is the character set name defined by Oracle Database
- SUNW represents the organization name (company stock trading abbreviation for Sun Microsystems)
- EXT specifies that this character set is an extension to the JA16EUC character set
- 1 specifies the version

13.5.6 Example: Creating a New Character Set Definition with the Oracle Locale Builder

This section shows how to create a new character set called MYCHARSET with 10001 for its character set ID. The example uses the WE8ISO8859P1 character set and adds 10 Chinese characters.

The following screen shows the General tab page for MYCHARSET character set.

Figure 13-22 General Tab Page for MYCHARSET

The screenshot shows the Oracle Locale Builder application window. The title bar includes the Oracle logo and menu items: File, Edit, Tools, Help. Below the menu is a tab bar with the following tabs: General (selected), Type Specifi..., Chara..., Lower..., Upper..., Class..., Repla..., Displ..., and Multi... On the left side of the window is a vertical toolbar with icons for various functions. The main area contains the following fields and buttons:

- Character Set Name: MYCHARSET
- Character Set ID: 10001
- ISO Character Set ID: (empty field)
- Base Character Set ID: (empty field)
- Show Existing Definitions... (button)

At the bottom of the window is a status bar with the following information:

- Filename: lx2001f.nlb
- Category: Character Set
- Name: WE8ISO8859P1
- Status: Editing

Click **Show Existing Definitions** and choose the WE8ISO8859P1 character set from the Existing Definitions dialog box.

The ISO Character Set ID and Base Character Set ID fields are optional. The Base Character Set ID is used for inheriting values so that the properties of the base character set are used as a template. The character set ID is automatically generated, but you can override it. The valid range for a user-defined character set ID is 8000 to 8999 or 10000 to 20000.

Note

If you are using Pro*COBOL, then choose a character set ID between 8000 and 8999.

The ISO Character Set ID remains blank for user-defined character sets.

In this example, the Base Character Set ID remains blank. However, you can specify a character set to use as a template. The settings in the Type Specification tab page must match the type settings of the base character set that you enter in the Base Character Set ID field. If the type settings do not match, then you will receive an error when you generate your custom character set.

The following screen shows the Type Specification tab page.

Figure 13-23 Type Specification Tab Page

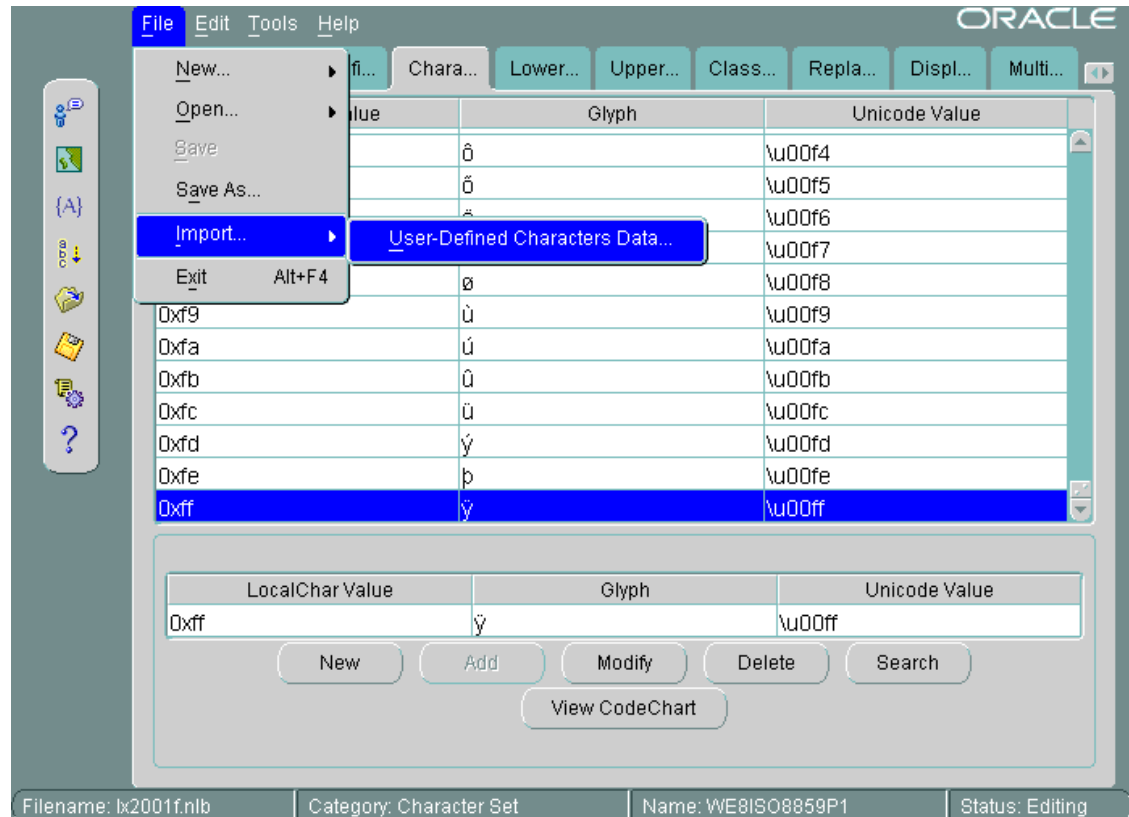
The screenshot shows the Oracle Locale Builder interface with the 'Type Specification' tab selected. The 'Character Set Category' section has three radio buttons: 'ASCII_BASED' (selected), 'EBCDIC_BASED', and 'FIXED_WIDTH'. The 'Additional Flags' section has three checkboxes: 'DISPLAY' (unchecked), 'SHIFT' (unchecked), and 'BYTE_UNIQUE' (checked). The 'Special Characters (When FIXED_WIDTH is set)' section has three rows for 'Pad Character', 'Underscore Character', and 'Percent Character', each with 'LocalChar Value' and 'Glyph' input fields. The 'Shift Characters (When SHIFT is set)' section has two rows for 'Shift Out' and 'Shift In', each with 'LocalChar Value' and 'Glyph' input fields. The '7 bit (When DISPLAY is set)' section has two radio buttons: 'True' (unchecked) and 'False' (checked). The bottom status bar shows 'Filename: Untitled', 'Category: Character Set', 'Name: None', and 'Status: Editing'.

The Character Set Category is **ASCII_BASED**. The **BYTE_UNIQUE** option is checked.

When you have chosen an existing character set, the fields for the Type Specification tab page should already be set to appropriate values. You should keep these values unless you have a specific reason for changing them. If you need to change the settings, then use the following guidelines:

- **FIXED_WIDTH** is used to identify character sets whose characters have a uniform length.
- **BYTE_UNIQUE** means that the single-byte range of code points is distinct from the multibyte range. The code in the first byte indicates whether the character is single-byte or multibyte. An example is JA16EUC.
- **DISPLAY** identifies character sets that are used only for display on clients and not for storage. Some Arabic, Devanagari, and Hebrew character sets are display character sets.
- **SHIFT** is used for character sets that require extra shift characters to distinguish between single-byte characters and multibyte characters.

The following screen shows how to add user-defined characters.

Figure 13-24 Importing User-Defined Character Data

Open the Character Data Mapping tab page. Highlight the character that you want to add characters after in the character set. In this example, the 0xff local character value is highlighted.

You can add one character at a time or use a text file to import a large number of characters. In this example, a text file is imported. The first column is the local character value. The second column is the Unicode value. The file contains the following character values:

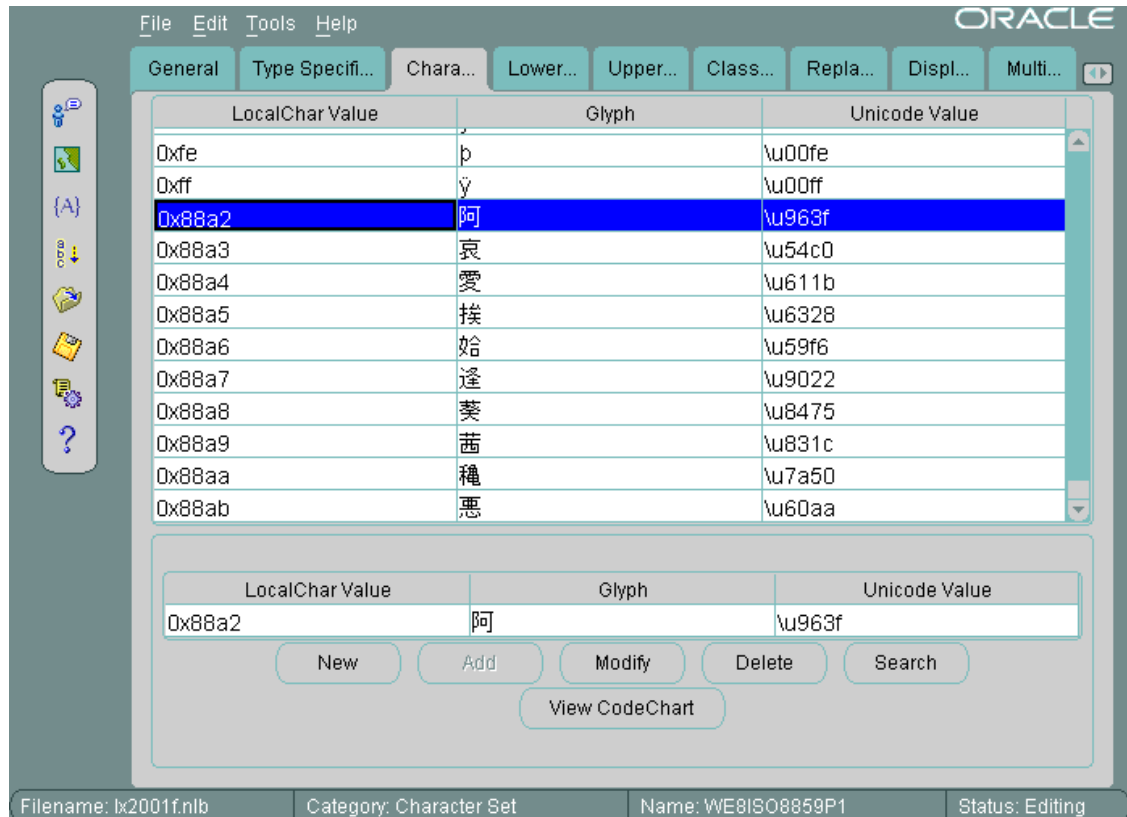
```

88a2 963f
88a3 54c0
88a4 611b
88a5 6328
88a6 59f6
88a7 9022
88a8 8475
88a9 831c
88aa 7a50
88ab 60aa

```

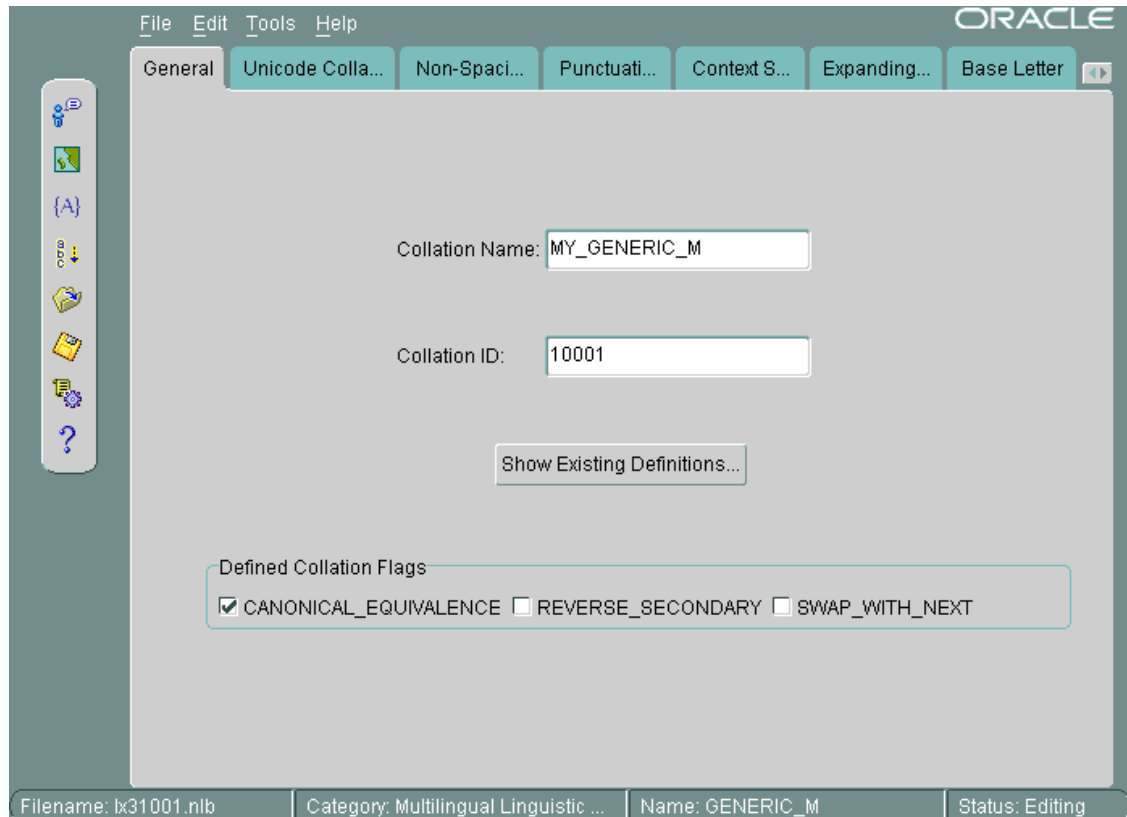
Choose **File > Import > User-Defined Characters Data**.

The following screen shows that the imported characters are added after 0xff in the character set.

Figure 13-25 New Characters in the Character Set

13.6 Creating a New Linguistic Sort with the Oracle Locale Builder

This section shows how to create a new multilingual linguistic sort called MY_GENERIC_M with a collation ID of 10001. The GENERIC_M linguistic sort is used as the basis for the new linguistic sort. The following screen shows how to begin.

Figure 13-26 General Tab Page for Collation

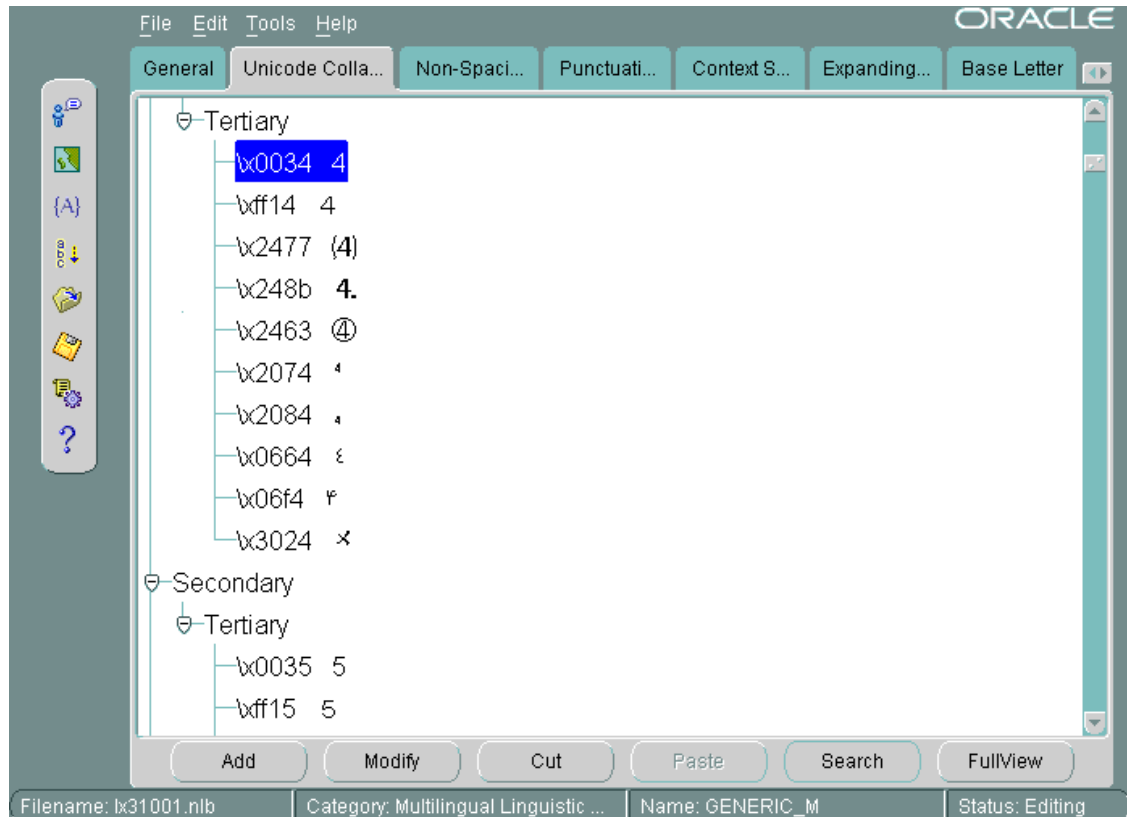
Settings for the flags are automatically derived. **SWAP_WITH_NEXT** is relevant for Thai and Lao sorts. **REVERSE_SECONDARY** is for French sorts. **CANONICAL_EQUIVALENCE** determines whether canonical rules are used. In this example, **CANONICAL_EQUIVALENCE** is checked.

The valid range for Collation ID (sort ID) for a user-defined sort is 1000 to 2000 for monolingual collation and 10000 to 11000 for multilingual collation.

See Also

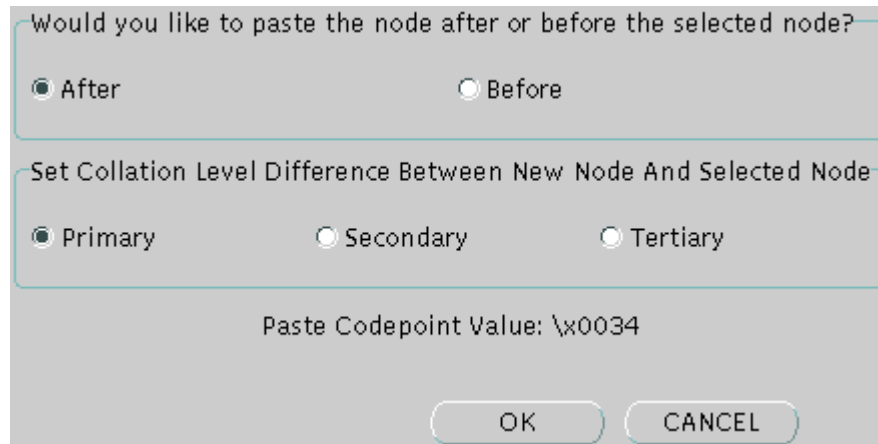
- [Figure 13-30](#) for more information about canonical rules
- ["Linguistic Sorting and Matching"](#)

The following screen shows the Unicode Collation Sequence tab page.

Figure 13-27 Unicode Collation Sequence Tab Page

This example customizes the linguistic sort by moving digits so that they sort after letters. Complete the following steps:

1. Highlight the Unicode value that you want to move. In the preceding screen, the `\x0034` Unicode value is highlighted. Its location in the Unicode Collation Sequence is called a *node*.
2. Click **Cut**. Select the location where you want to move the node.
3. Click **Paste**. Clicking **Paste** opens the Paste Node dialog box as shown in the following screen.

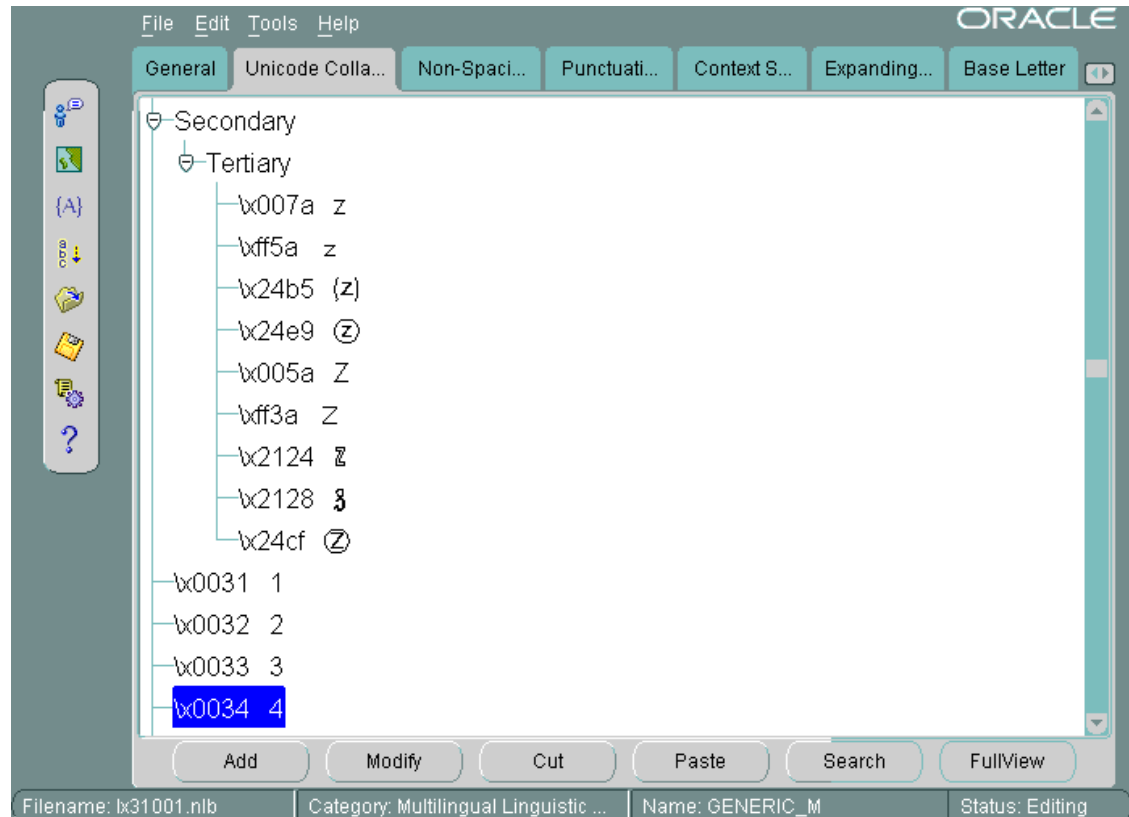
Figure 13-28 Paste Node Dialog BoxThe image shows a 'Paste Node Dialog Box' with a light gray background. It contains two sections. The first section is titled 'Would you like to paste the node after or before the selected node?' and has two radio buttons: 'After' (which is selected) and 'Before'. The second section is titled 'Set Collation Level Difference Between New Node And Selected Node' and has three radio buttons: 'Primary' (selected), 'Secondary', and 'Tertiary'. Below these sections, the text 'Paste Codepoint Value: \x0034' is displayed. At the bottom right, there are two buttons: 'OK' and 'CANCEL'.

The Paste Node dialog box enables you to choose whether to paste the node after or before the location you have selected. It also enables you to choose the level (Primary, Secondary, or Tertiary) of the node in relation to the node that you want to paste it next to.

4. Select the position and the level at which you want to paste the node. In the preceding screen, **After** and **Primary** options are selected.
5. Click **OK** to paste the node.

Use similar steps to move other digits to a position after the letters a through z.

The following screen shows the resulting Unicode Collation Sequence tab page after the digits 0 through 4 have been moved to a position after the letters a through z.

Figure 13-29 Unicode Collation Sequence After Modification

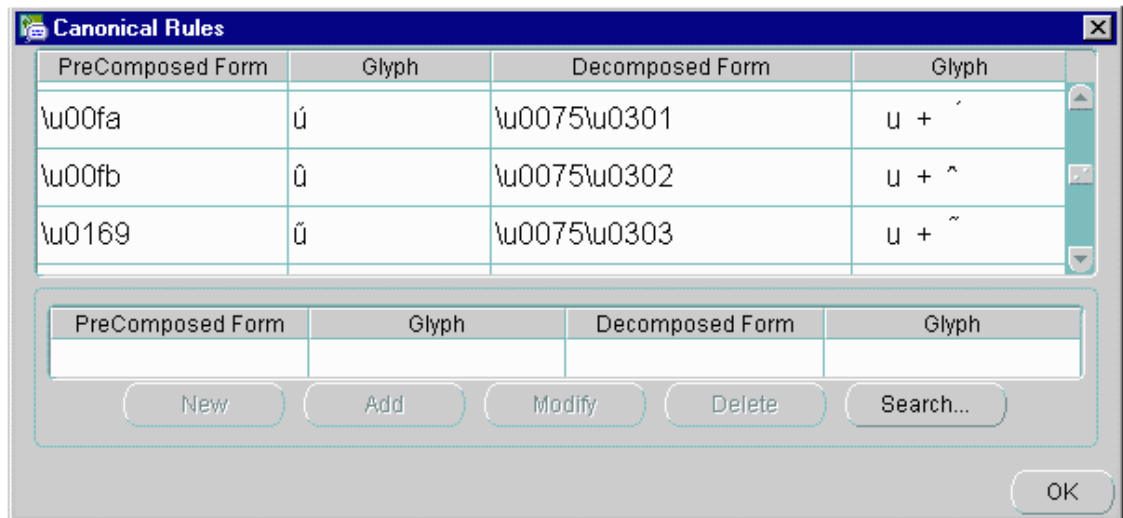
The rest of this section contains the following topics:

- [Changing the Sort Order for All Characters with the Same Diacritic](#)
- [Changing the Sort Order for One Character with a Diacritic](#)

13.6.1 Changing the Sort Order for All Characters with the Same Diacritic

This example shows how to change the sort order for characters with diacritics. You can do this by changing the sort for all characters containing a particular diacritic or by changing one character at a time. This example changes the sort of each character with a circumflex (for example, û) to be after the same character containing a tilde.

Verify the current sort order by choosing **Tools > Canonical Rules**. This opens the Canonical Rules dialog box as shown in the following screen.

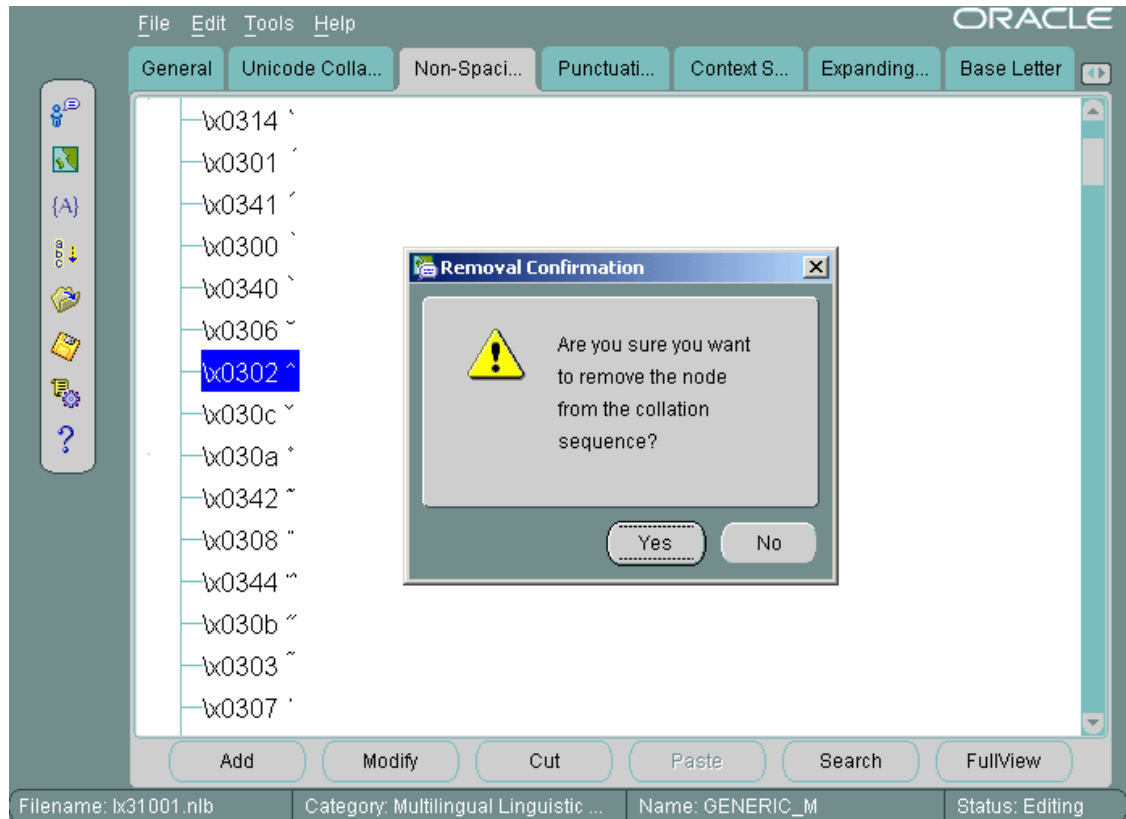
Figure 13-30 Canonical Rules Dialog Box

This dialog box shows how characters are decomposed into their canonical equivalents and their current sorting orders. For example, û is represented as u plus ^.

① See Also

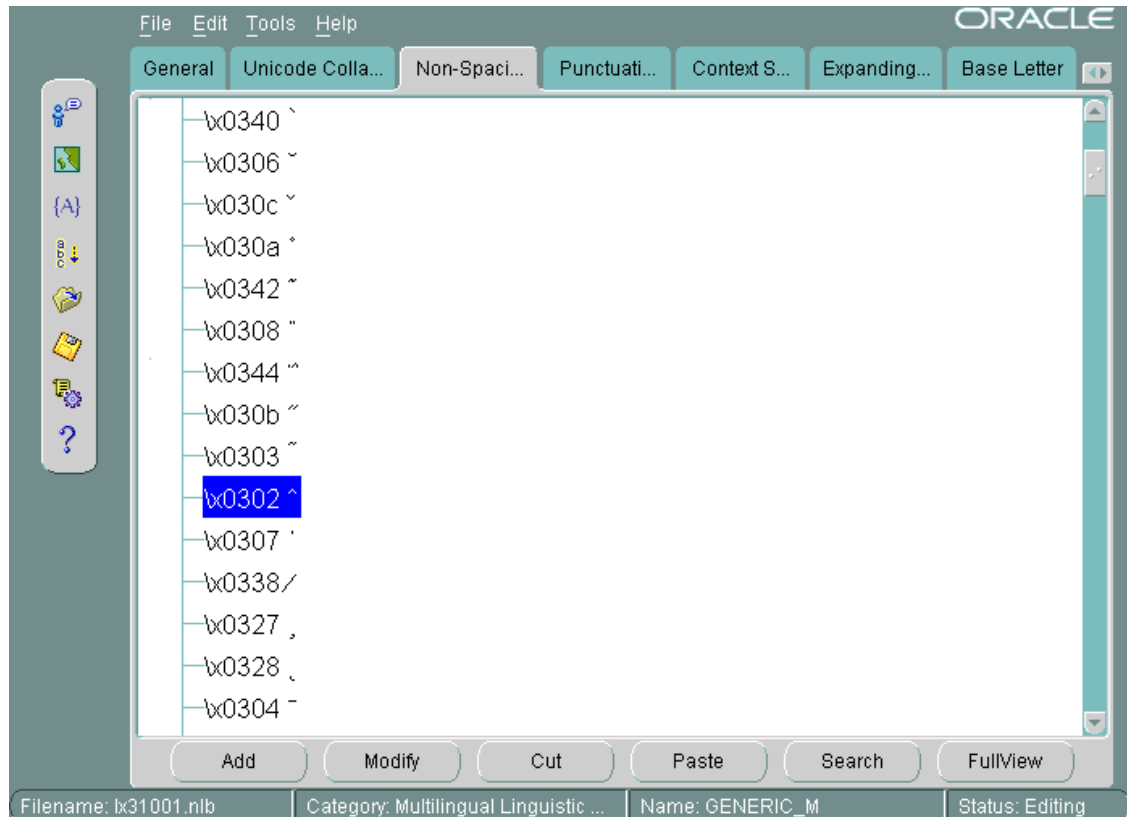
"[Linguistic Sorting and Matching](#)" for more information about canonical rules

In the Oracle Locale Builder collation window (shown in [Figure 13-26](#)), select the Non-Spacing Characters tab. If you use the Non-Spacing Characters tab page, then changes for diacritics apply to all characters. The following screen shows the Non-Spacing Characters tab page.

Figure 13-31 Changing the Sort Order for All Characters with the Same Diacritic

Select the circumflex and click **Cut**. Click **Yes** in the Removal Confirmation dialog box. Select the tilde and click **Paste**. Choose **After** and **Secondary** in the Paste Node dialog box and click **OK**.

The following screen shows the new sort order.

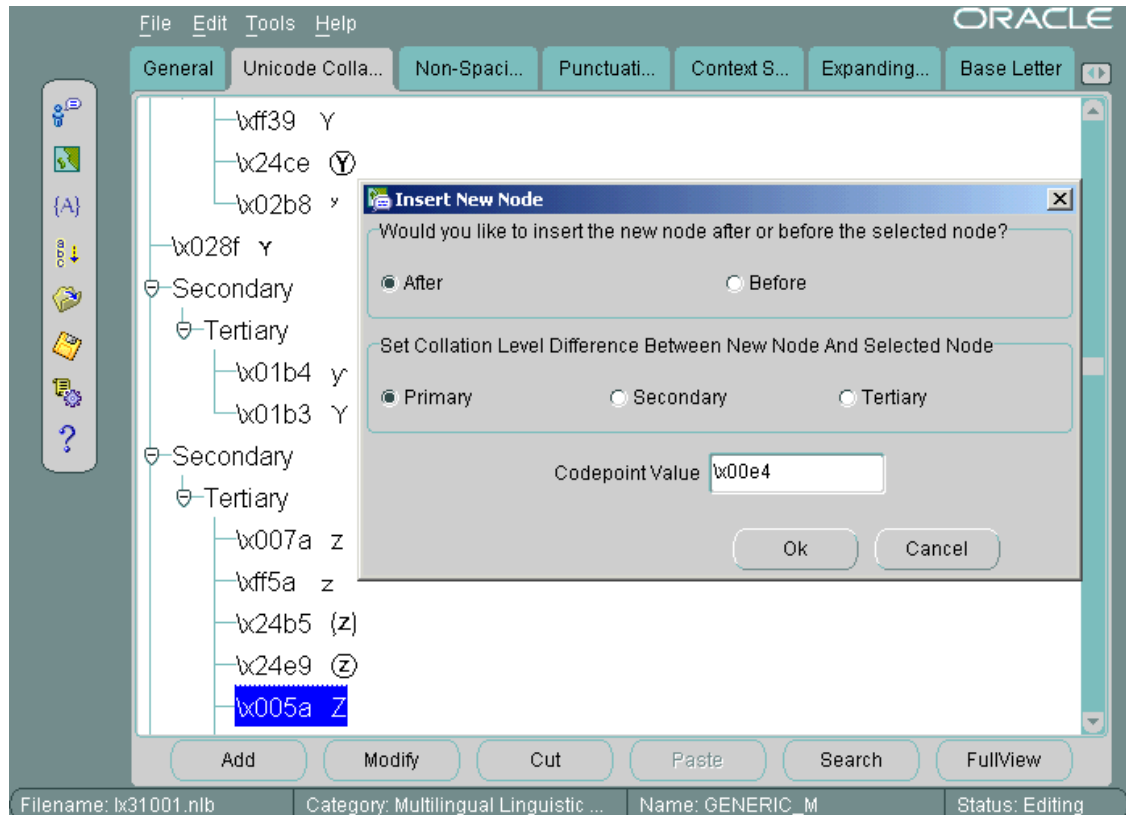
Figure 13-32 The New Sort Order for Characters with the Same Diacritic

13.6.2 Changing the Sort Order for One Character with a Diacritic

To change the order of a specific character with a diacritic, insert the character directly into the appropriate position. Characters with diacritics do not appear in the **Unicode Collation Sequence** tab page, so you cannot cut and paste them into the new location.

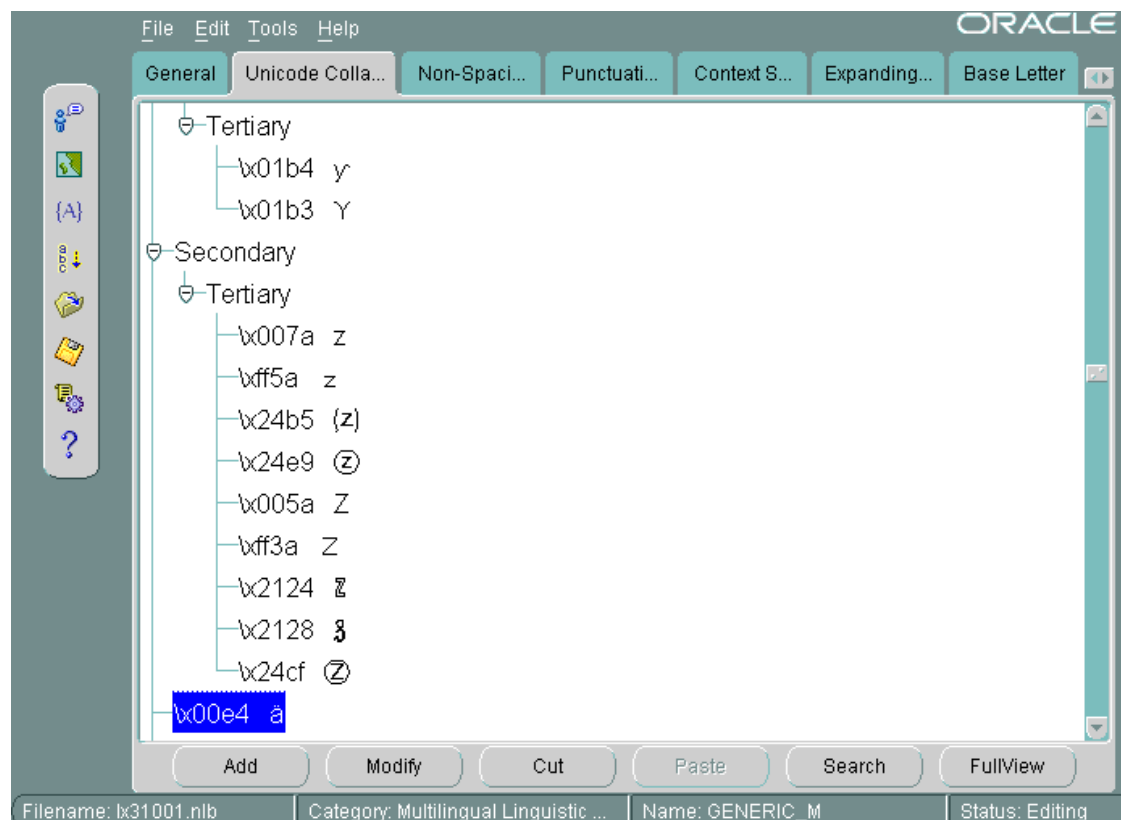
This example changes the sort order for ä so that it sorts after Z.

Select the Unicode Collation tab. Highlight the character, Z, that you want to put ä next to. Click **Add**. The Insert New Node dialog box appears, as shown in the following screen.

Figure 13-33 Changing the Sort Order of One Character with a Diacritic

Choose **After** and **Primary** in the Insert New Node dialog box. Enter the Unicode code point value of ä. The code point value is \x00e4. Click **OK**.

The following screen shows the resulting sort order.

Figure 13-34 New Sort Order After Changing a Single Character

13.7 Generating and Installing NLB Files

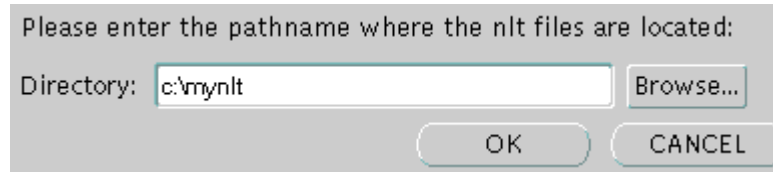
After you have defined a new language, territory, character set, or linguistic sort, generate new NLB files from the NLT files as follows.

1. As the user who owns the files (typically user `oracle`), back up the NLS installation boot file (`lx0boot.nlb`) and the NLS system boot file (`lx1boot.nlb`) in the `ORA_NLS10` directory. On a UNIX platform, enter commands similar to the following example:

```
% setenv ORA_NLS10 $ORACLE_HOME/nls/data
% cd $ORA_NLS10
% cp -p lx0boot.nlb lx0boot.nlb.orig
% cp -p lx1boot.nlb lx1boot.nlb.orig
```

Note that the `-p` option preserves the timestamp of the original file.

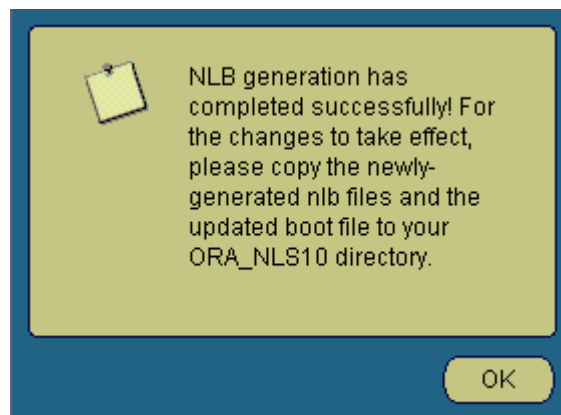
2. In Oracle Locale Builder, choose **Tools > Generate NLB** or click the **Generate NLB** icon in the left side bar.
3. Click **Browse** to find the directory where the NLT file is located. The location dialog box is shown below.

Figure 13-35 Location Dialog Box

Do not try to specify an NLT file. Oracle Locale Builder generates an NLB file for each NLT file.

4. Click **OK** to generate the NLB files.

The following screen shows the final notification about successfully generated NLB files for all the NLT files in the directory.

Figure 13-36 NLB Generation Success Dialog Box

5. Copy the `lx1boot.nlb` file into the path that is specified by the `ORA_NLS10` environment variable. For example, on a UNIX platform, enter a command similar to the following example:

```
% cp /directory_name/lx1boot.nlb $ORA_NLS10/lx1boot.nlb
```

6. Copy the new NLB files into the `ORA_NLS10` directory. For example, on a UNIX platform, enter commands similar to the following example:

```
% cp /directory_name/lx22710.nlb $ORA_NLS10
% cp /directory_name/lx52710.nlb $ORA_NLS10
```

Note

Oracle Locale Builder generates NLB files in the directory where the NLT files reside

7. Restart the database to use the newly created locale data.
8. To use the new locale data on the client side, exit the client and re-invoke the client after installing the NLB files.

① See Also

["Locale Data on Demand"](#) for more information about the `ORA_NLS10` environment variable

13.8 Upgrading Custom NLB Files from Previous Releases of Oracle Database

Locale definition files are database release-dependent. For example, NLB files from Oracle Database 19c and Oracle Database 21c are not directly supported in an Oracle AI Database 26ai installation, and so forth. Even a patch set may introduce a small change to the NLB file format, if it is necessary to fix a bug. Installation of a patch set or a patch set update (PSU) may overwrite your customizations, if any Oracle-supplied NLB files have been modified in the patch set.

In order to migrate your locale customization files from your current release of the database to a new release or patch set, perform the following steps:

1. Create a directory and copy your customized NLB or NLT files there.
2. Install the new database release, patch set or patch set update into the existing Oracle Home or into a new Oracle Home, as appropriate.
3. Use the Oracle Locale Builder from the new or updated Oracle Home to open each of the files copied in step (1) and save them in the NLT format to the source directory.
4. Repeat the NLB generation and installation steps as described in the section "[Generating and Installing NLB Files](#)", still using the new version of the Oracle Locale Builder and the same source directory.

Note: Oracle Locale Builder can read and process previous versions of the NLT and NLB files, as well as read and process these files from different platforms. However, Oracle Locale Builder always saves NLT files and generates NLB files in the latest format for the release of Oracle Database that you have installed.

Note: It is the user's responsibility to maintain and deploy custom NLB files after database upgrade and patching operations. Custom locale objects are not supported in Oracle Exadata family of products and Oracle-managed database services such as Oracle Exadata Cloud@Customer (ExaCC) and Oracle Autonomous Database (ADB).

13.9 Deploying Custom NLB Files to Oracle Installations on the Same Platform

To add your customizations to another Oracle Home with exactly the same database release and patch configuration and on the same platform as the Oracle Home used to generate the original set of customizations, perform the following steps:

1. In the target Oracle Home, perform step 1 from "[Generating and Installing NLB Files](#)".
2. If the target Oracle Home is on another machine, copy your customized NLB files and the generated `lx1boot.nlb` file to the target machine, using any method preserving the binary integrity of the files, such as FTP in binary mode, copy to a remotely mounted file system, `scp` utility, and so on.

3. On the target machine, perform steps 5-8 from "[Generating and Installing NLB Files](#)" using the directory containing your customized NLB files and the `lx1boot.nlb` file as `directory_name`.

13.10 Deploying Custom NLB Files to Oracle Installations on Another Platform

While being release-dependent, NLB files are platform-independent. Platform-dependent differences in the binary format (such as 32-bit versus 64-bit, big-endian versus little-endian, ASCII versus EBCDIC) are processed transparently during NLB loading. Therefore, when deploying your locale customization files into other Oracle Database installations running with the same Oracle Database release and patch configuration, but under a different operating system platform, you can choose one of the following two options:

1. Copy over the custom `.NLT` files to your new platform and repeat the NLB generation and installation steps as described in "[Generating and Installing NLB Files](#)".
2. Copy over the entire set of `.NLB` files (both Oracle-supplied NLB files and custom NLB files) to your new platform.

Note that option (2) may introduce some overhead at NLB loading time due to the transparent platform processing required. However, this overhead should be negligible, because each NLB file is usually loaded only once after an Oracle Database instance or an Oracle Client application is started and it is cached until the instance or application is shut down. Moreover, NLB files are loaded on demand. So, in most installations, only a small subset of all available NLB files is ever loaded into memory.

Option (2) is especially useful to customize files for platforms on which Oracle Locale Builder is not supported.

To copy over the entire set of NLB files to your new platform, perform the following steps:

1. Shut down all Oracle Database instances and Oracle Client applications using the target Oracle Home.
2. As the user who owns the files (typically `user oracle`), move all NLB files from the `ORA_NLS10` directory of the target Oracle Home to a backup directory. On a UNIX platform, enter commands similar to the following example:

```
% setenv ORA_NLS10 $ORACLE_HOME/nls/data
% cd $ORA_NLS10
% mkdir orig
% mv *.nlb orig
```
3. Copy all NLB files from the source Oracle Home NLB directory to the target Oracle Home NLB (`$ORA_NLS10`) directory. Use any remote copy method preserving the binary integrity of the files, such as FTP in binary mode, copy to a remotely mounted file system, `rcp` utility, and so on.
4. Restart the database instances and/or applications, as desired.

13.11 Adding Custom Locale Definitions to Java Components with the GINSTALL Utility

The `Ginstall` utility adds custom character sets, language, territory, and linguistic sorts to Java components in your applications. You use Locale Builder to define your custom character sets, language, territory, and linguistic sort. Locale Builder generates NLT files, which contain the

custom definitions. Then to add the custom definitions to the Java components, you run Ginstall to generate gdk_custom.jar.

To add custom definitions for character set, language, territory, and linguistic sort:

1. Generate the NLT file using Oracle Locale Builder.

If you are upgrading custom NLB files from a previous release, follow the procedure described in "[Upgrading Custom NLB Files from Previous Releases of Oracle Database](#)".

2. Run Ginstall with -add or -a option to generate gdk_custom.jar.

```
java -classpath $ORACLE_HOME/jlib/orai18n.jar:$ORACLE_HOME/lib/xmlparserv2.jar
Ginstall -[add | a] <Name of NLT file>
```

To generate multiple NLT files:

```
java -classpath $ORACLE_HOME/jlib/orai18n.jar:$ORACLE_HOME/lib/xmlparserv2.jar
Ginstall -[add | a] <NLT file1>
<NLT file2>
<NLT file3>
```

3. Copy gdk_custom.jar to the same directory as orai18n.jar or orai18n-mapping.jar.

To remove a custom definition:

- Run Ginstall as follows.

```
java -classpath $ORACLE_HOME/jlib/orai18n.jar:$ORACLE_HOME/lib/xmlparserv2.jar
Ginstall -[remove | r] <Name of NLT file>
```

To update a custom definition:

- Run Ginstall as follows.

```
java -classpath $ORACLE_HOME/jlib/orai18n.jar:$ORACLE_HOME/lib/xmlparserv2.jar
Ginstall -[update | u] <Name of NLT file>
```

13.12 Customizing Calendars with the NLS Calendar Utility

Oracle Database supports several calendars. Some of them may require the addition of ruler eras in the future and some may require tailoring to local needs through addition or subtraction of deviation days. To add the required information to your Oracle implementation, you can use external files that are automatically loaded when the calendar functions are executed.

Calendar data is first defined in a text file. The text definition file must be converted into binary format. You can use the NLS Calendar Utility (lxegen) to convert the text definition file into binary format.

The name of the text definition file and its location for the lxegen utility are hard-coded and depend on the platform. On UNIX platforms, the file name is lxecal.nlt. It is located in the \$ORACLE_HOME/nls directory. A sample text definition file is included in the \$ORACLE_HOME/nls/demo directory.

Depending on the number of different calendars referenced in the text definition file, the lxegen utility produces one or more binary files. The names of the binary files are also hard-coded and depend on the platform. On UNIX platforms, the names of the binary files are lxecal.h.nlb (deviation days for the Arabic Hijrah calendar), lxecal.eh.nlb (deviation days for the English Hijrah calendar), and lxecal.ji.nlb (ruler eras for the Japanese Imperial calendar). The binary files are generated in the same directory as the text file and overwrite existing binary files.

After the binary files have been generated, they are automatically loaded during system initialization. Do not move or rename the files. Unlike files generated by Oracle Locale Builder,

calendar customization binary files are not platform-independent. You should generate them for each combination of Oracle software release and platform separately.

Invoke the calendar utility from the command line as follows:

```
% lxcgen
```

See Also

- ["Calendar Systems"](#)
- *Oracle AI Database Examples Installation Guide* for more information regarding how to install demo files
- Operating system documentation for the location of the files on your system

A

Locale Data

This appendix lists the languages, territories, character sets, and other locale data supported by Oracle Database. This appendix contains the following topics:

- [Languages](#)
- [Translated Messages](#)
- [Territories](#)
- [Character Sets](#)
- [Language and Character Set Detection Support](#)
- [Linguistic Collations](#)
- [Calendar Systems](#)
- [Time Zone Region Names](#)
- [Obsolete Locale Data](#)
- [Desupported Locale Data](#)

You can obtain information about character sets, languages, territories, and linguistic sorts by querying the `V$NLS_VALID_VALUES` dynamic performance view.

See Also

Oracle AI Database Reference for more information about the `V$NLS_VALID_VALUES` view

A.1 Languages

Languages in [Table A-1](#) provide support for locale-sensitive information such as:

- Day and month names and their abbreviations
- Symbols for equivalent expressions for A.M., P.M., A.D., and B.C.
- Default sorting sequence for character data when the `ORDER BY SQL` clause is specified
- Writing direction (left to right or right to left)
- Affirmative and negative response strings (for example, YES and NO)

By using Unicode databases and data types, you can store, process, and retrieve data for almost all contemporary languages, including many that do not appear in [Table A-1](#).

Table A-1 Oracle Database Supported Languages

Language Name	Language Abbreviation	Default Sort
ALBANIAN	sq	GENERIC_M
AMERICAN	us	binary
AMHARIC	am	GENERIC_M
ARABIC	ar	ARABIC
ARMENIAN	hy	GENERIC_M
ASSAMESE	as	binary
AZERBAIJANI	az	AZERBAIJANI
BANGLA	bn	binary
BASQUE	eu	GENERIC_M
BELARUSIAN	be	RUSSIAN
BRAZILIAN PORTUGUESE	ptb	WEST_EUROPEAN
BULGARIAN	bg	BULGARIAN
BURMESE	my	GENERIC_M
CANADIAN FRENCH	fr	CANADIAN FRENCH
CATALAN	ca	CATALAN
CROATIAN	hr	CROATIAN
CYRILLIC KAZAKH	ckk	GENERIC_M
CYRILLIC SERBIAN	csr	GENERIC_M
CYRILLIC UZBEK	cuz	GENERIC_M
CZECH	cs	CZECH
DANISH	dk	DANISH
DARI	prs	GENERIC_M
DIVEHI	dv	GENERIC_M
DUTCH	nl	DUTCH
EGYPTIAN	eg	ARABIC
ENGLISH	gb	binary
ESTONIAN	et	ESTONIAN
FINNISH	sf	FINNISH
FRENCH	f	FRENCH
GEORGIAN	ka	GENERIC_M
GERMAN DIN	din	GERMAN
GERMAN	d	GERMAN
GREEK	el	GREEK
GUJARATI	gu	binary
HEBREW	iw	HEBREW
HINDI	hi	binary
HUNGARIAN	hu	HUNGARIAN
ICELANDIC	is	ICELANDIC

Table A-1 (Cont.) Oracle Database Supported Languages

Language Name	Language Abbreviation	Default Sort
INDONESIAN	in	INDONESIAN
IRISH	ga	binary
ITALIAN	i	WEST_EUROPEAN
JAPANESE	ja	binary
KANNADA	kn	binary
KHMER	km	GENERIC_M
KOREAN	ko	binary
KYRGYZ	ky	GENERIC_M
LAO	lo	GENERIC_M
LATIN AMERICAN SPANISH	esa	SPANISH
LATIN BOSNIAN	lbs	GENERIC_M
LATIN SERBIAN	lsr	binary
LATIN UZBEK	luz	GENERIC_M
LATVIAN	lv	LATVIAN
LITHUANIAN	lt	LITHUANIAN
MACEDONIAN	mk	binary
MALAY	ms	MALAY
MALAYALAM	ml	binary
MALTESE	mt	GENERIC_M
MARATHI	mr	binary
MEXICAN SPANISH	esm	WEST_EUROPEAN
NEPALI	ne	GENERIC_M
NORWEGIAN	n	NORWEGIAN
ORIYA	or	binary
PERSIAN	fa	GENERIC_M
POLISH	pl	POLISH
PORTUGUESE	pt	WEST_EUROPEAN
PUNJABI	pa	binary
ROMANIAN	ro	ROMANIAN
RUSSIAN	ru	RUSSIAN
SIMPLIFIED CHINESE	zhs	binary
SINHALA	si	GENERIC_M
SLOVAK	sk	SLOVAK
SLOVENIAN	sl	SLOVENIAN
SPANISH	e	SPANISH
SWAHILI	sw	GENERIC_M
SWEDISH	s	SWEDISH
TAMIL	ta	binary

Table A-1 (Cont.) Oracle Database Supported Languages

Language Name	Language Abbreviation	Default Sort
TELUGU	te	binary
THAI	th	THAI_DICTIONARY
TRADITIONAL CHINESE	zht	binary
TURKISH	tr	TURKISH
TURKMEN	tk	GENERIC_M
UKRAINIAN	uk	UKRAINIAN
URDU	ur	GENERIC_M
VIETNAMESE	vn	VIETNAMESE

A.2 Translated Messages

Oracle Database error messages have been translated into the languages which are listed in [Table A-2](#).

Table A-2 Oracle Database Supported Messages

Name	Abbreviation
ARABIC	ar
BRAZILIAN PORTUGUESE	ptb
CATALAN	ca
CZECH	cs
DANISH	dk
DUTCH	nl
FINNISH	sf
FRENCH	f
GERMAN	d
GREEK	el
HEBREW	iw
HUNGARIAN	hu
ITALIAN	i
JAPANESE	ja
KOREAN	ko
NORWEGIAN	n
POLISH	pl
PORTUGUESE	pt
ROMANIAN	ro
RUSSIAN	ru
SIMPLIFIED CHINESE	zhs
SLOVAK	sk

Table A-2 (Cont.) Oracle Database Supported Messages

Name	Abbreviation
SPANISH	e
SWEDISH	s
THAI	th
TRADITIONAL CHINESE	zht
TURKISH	tr

A.3 Territories

[Table A-3](#) lists the territories that Oracle Database supports.

Table A-3 Oracle Database Supported Territories

Territory	Territory	Territory
AFGHANISTAN	GEORGIA	NORWAY
ALBANIA	GERMANY	OMAN
ALGERIA	GHANA	PAKISTAN
AMERICA	GREECE	PANAMA
ANGOLA	GRENADA	PARAGUAY
ANTIGUA AND BARBUDA	GUATEMALA	PERU
ARGENTINA	GUYANA	PHILIPPINES
ARMENIA	HAITI	POLAND
ARUBA	HONDURAS	PORTUGAL
AUSTRALIA	HONG KONG	PUERTO RICO
AUSTRIA	HUNGARY	QATAR
AZERBAIJAN	ICELAND	ROMANIA
BAHAMAS	INDIA	RUSSIA
BAHRAIN	INDONESIA	SAINT KITTS AND NEVIS
BANGLADESH	IRAN	SAINT LUCIA
BARBADOS	IRAQ	SAUDI ARABIA
BELARUS	IRELAND	SENEGAL
BELGIUM	ISRAEL	SERBIA
BELIZE	ITALY	SIERRA LEONE
BERMUDA	IVORY COAST	SINGAPORE
BOLIVIA	JAMAICA	SLOVAKIA
BOSNIA AND HERZEGOVINA	JAPAN	SLOVENIA
BOTSWANA	JORDAN	SOMALIA
BRAZIL	KAZAKHSTAN	SOUTH AFRICA
BULGARIA	KENYA	SOUTH SUDAN
CAMBODIA	KOREA	SPAIN
CAMEROON	KUWAIT	SRI LANKA
CANADA	KYRGYZSTAN	SUDAN
CATALONIA	LAOS	SURINAME
CAYMAN ISLANDS	LATVIA	SWAZILAND
CHILE	LEBANON	SWEDEN
CHINA	LIBYA	SWITZERLAND
COLOMBIA	LIECHTENSTEIN	SYRIA
CONGO BRAZZAVILLE	LITHUANIA	TAIWAN
CONGO KINSHASA	LUXEMBOURG	TANZANIA
COSTA RICA	MACAO	THAILAND
CROATIA	MALAWI	THE NETHERLANDS
CURACAO	MALAYSIA	TRINIDAD AND TOBAGO
CYPRUS	MALDIVES	TUNISIA
CZECH REPUBLIC	MALTA	TURKEY
DENMARK	MAURITANIA	TURKIYE ¹
DJIBOUTI	MAURITIUS	TURKMENISTAN
DOMINICA	MEXICO	UGANDA
DOMINICAN REPUBLIC	MOLDOVA	UKRAINE
ECUADOR	MONTENEGRO	UNITED ARAB EMIRATES
EGYPT	MOROCCO	UNITED KINGDOM
EL SALVADOR	MOZAMBIQUE	URUGUAY
ESTONIA	MYANMAR	UZBEKISTAN
ETHIOPIA	NAMIBIA	VENEZUELA
FINLAND	NEPAL	VIETNAM
FRANCE	NEW ZEALAND	YEMEN
FYR MACEDONIA	NICARAGUA	ZAMBIA
GABON	NIGERIA	ZIMBABWE

¹ The TURKIYE territory is introduced in Oracle AI Database 26ai to support the new official name of the country of Türkiye. The TURKEY territory, which has been available in previous Oracle Database releases, continues to be supported.

The TURKIYE and TURKEY territories provide identical functionality in the database. They specify the same cultural conventions, such as date and time formats, and numeric and monetary conventions.

A.4 Character Sets

The character sets that Oracle Database supports are listed in the following sections according to three broad categories.

- [Recommended Database Character Sets](#)
- [Other Character Sets](#)
- [Client-Only Character Sets](#)

In addition, common character set subset/superset combinations are listed. Some character sets can only be used with certain data types. For example, the AL16UTF16 character set can only be used as an NCHAR character set, and not as a database character set.

Also documented in the comment section are other unique features of the character set that may be important to users or your database administrator. For example, the information includes whether the character set supports the euro currency symbol, whether user-defined characters are supported, and whether the character set is a strict superset of ASCII. (You can use the Database Migration Assistant for Unicode to migrate an existing database to a new character set, only if all of the schema data is a strict subset of the new character set.)

The key for the comment column of the character set tables is:

SB: single-byte encoding
 MB: multibyte encoding
 FIXED: fixed-width multibyte encoding
 ASCII: strict superset of ASCII
 EURO: euro symbol supported
 UDC: user-defined characters supported

Oracle Database does not document individual code page layouts. For specific details about a particular character set, its character repertoire, and code point values, you can use Oracle Locale Builder. Otherwise, you should refer to the actual national, international, or vendor-specific standards.

See Also

- *Oracle Database Migration Assistant for Unicode Guide*
- ["Customizing Locale Data"](#)

A.4.1 Recommended Database Character Sets

[Table A-4](#) lists the recommended and most commonly used ASCII-based Oracle Database character sets. The list is ordered alphabetically within their respective language group.

Table A-4 Recommended ASCII Database Character Sets

Language Group	Character Set	Description	Comments
Asian	JA16EUC	EUC 24-bit Japanese	MB, ASCII

Table A-4 (Cont.) Recommended ASCII Database Character Sets

Language Group	Character Set	Description	Comments
Asian	JA16EUCTILDE	The same as JA16EUC except for the way that the wave dash and the tilde are mapped to and from Unicode.	MB, ASCII
Asian	JA16SJIS	Shift-JIS 16-bit Japanese	MB, ASCII, UDC
Asian	JA16SJISTILDE	The same as JA16SJIS except for the way that the wave dash and the tilde are mapped to and from Unicode.	MB, ASCII, UDC
Asian	KO16MSWIN949	MS Windows Code Page 949 Korean	MB, ASCII, UDC
Asian	TH8TISASCII	Thai Industrial Standard 620-2533 - ASCII 8-bit	SB, ASCII, EURO
Asian	VN8MSWIN1258	MS Windows Code Page 1258 8-bit Vietnamese	SB, ASCII, EURO
Asian	ZHS16GBK	GBK 16-bit Simplified Chinese	MB, ASCII, UDC
Asian	ZHT16HKSCS	MS Windows Code Page 950 with Hong Kong Supplementary Character Set HKSCS-2001 (character set conversion to and from Unicode is based on Unicode 3.0)	MB, ASCII, EURO
Asian	ZHT16MSWIN950	MS Windows Code Page 950 Traditional Chinese	MB, ASCII, UDC
Asian	ZHT32EUC	EUC 32-bit Traditional Chinese	MB, ASCII
European	BLT8ISO8859P13	ISO 8859-13 Baltic	SB, ASCII
European	BLT8MSWIN1257	MS Windows Code Page 1257 8-bit Baltic	SB, ASCII, EURO
European	CL8ISO8859P5	ISO 8859-5 Latin/Cyrillic	SB, ASCII
European	CL8MSWIN1251	MS Windows Code Page 1251 8-bit Latin/Cyrillic	SB, ASCII, EURO
European	EE8ISO8859P2	ISO 8859-2 East European	SB, ASCII
European	EL8ISO8859P7	ISO 8859-7 Latin/Greek	SB, ASCII, EURO
European	EL8MSWIN1253	MS Windows Code Page 1253 8-bit Latin/Greek	SB, ASCII, EURO
European	EE8MSWIN1250	MS Windows Code Page 1250 8-bit East European	SB, ASCII, EURO
European	NE8ISO8859P10	ISO 8859-10 North European	SB, ASCII
European	NEE8ISO8859P4	ISO 8859-4 North and North-East European	SB, ASCII
European	WE8ISO8859P15	ISO 8859-15 West European	SB, ASCII, EURO
European	WE8MSWIN1252	MS Windows Code Page 1252 8-bit West European	SB, ASCII, EURO
Middle Eastern	AR8ISO8859P6	ISO 8859-6 Latin/Arabic	SB, ASCII
Middle Eastern	AR8MSWIN1256	MS Windows Code Page 1256 8-Bit Latin/Arabic	SB, ASCII, EURO

Table A-4 (Cont.) Recommended ASCII Database Character Sets

Language Group	Character Set	Description	Comments
Middle Eastern	IW8ISO8859P8	ISO 8859-8 Latin/Hebrew	SB, ASCII
Middle Eastern	IW8MSWIN1255	MS Windows Code Page 1255 8-bit Latin/Hebrew	SB, ASCII, EURO
Middle Eastern	TR8MSWIN1254	MS Windows Code Page 1254 8-bit Turkish	SB, ASCII, EURO
Middle Eastern	WE8ISO8859P9	ISO 8859-9 West European & Turkish	SB, ASCII
Universal	AL32UTF8	Unicode 15.0 Universal Character Set (UCS), UTF-8 encoding scheme (Oracle AI Database 26ai, RU 23.26.2 and earlier) Unicode 17.0 Universal Character Set (UCS), UTF-8 encoding scheme (Oracle AI Database 26ai, RU 23.26.3 and later)	MB, ASCII, EURO

[Table A-5](#) lists the recommended EBCDIC-based Oracle Database character sets that are supported on Fujitsu BS2000, which is the only EBCDIC-based platform currently supported by Oracle Database.

Table A-5 Recommended EBCDIC Database Character Sets

Language Group	Character Set	Description	Comments
European	CE8BS2000	Siemens EBCDIC.DF.04 8-bit Central European	SB
European	CL8BS2000	Siemens EBCDIC.EHC.LC 8-bit Cyrillic	SB
European	EE8BS2000	Siemens EBCDIC.DF.04 8-bit East European	SB
European	WE8BS2000	Siemens EBCDIC.DF.04 8-bit West European	SB
European	WE8BS2000E	Siemens EBCDIC.DF.04 8-bit West European	SB, EURO
European	WE8BS2000L5	Siemens EBCDIC.DF.L5 8-bit West European/Turkish	SB

A.4.2 Other Character Sets

[Table A-6](#) lists the other ASCII-based Oracle Database character sets. The list is ordered alphabetically within their language groups.

Table A-6 Other ASCII-based Database Character Sets

Language Group	Character Set	Description	Comments
Asian	BN8BSCII	Bangladesh National Code 8-bit BSCII	SB, ASCII
Asian	IN8ISCI	Multiple-Script Indian Standard 8-bit Latin/Indian Languages	SB, ASCII
Asian	JA16VMS	JVMS 16-bit Japanese	MB, ASCII
Asian	KO16KSC5601	KSC5601 16-bit Korean	MB, ASCII
Asian	KO16KSCCS	KSCCS 16-bit (Johab) Korean	MB, ASCII

Table A-6 (Cont.) Other ASCII-based Database Character Sets

Language Group	Character Set	Description	Comments
Asian	TH8MACTHAIS	Mac Server 8-bit Latin/Thai	SB, ASCII
Asian	VN8VN3	VN3 8-bit Vietnamese	SB, ASCII
Asian	ZHT16BIG5	BIG5 16-bit Traditional Chinese	MB, ASCII
Asian	ZHT16CCDC	HP CCDC 16-bit Traditional Chinese	MB, ASCII
Asian	ZHS16CGB231280	CGB2312-80 16-bit Simplified Chinese	MB, ASCII
Asian	ZHT16HKSCS31	MS Windows Code Page 950 with Hong Kong Supplementary Character Set HKSCS-2001 (character set conversion to and from Unicode is based on Unicode 3.1)	MB, ASCII, EURO
Asian	ZHT32SOPS	SOPS 32-bit Traditional Chinese	MB, ASCII
Asian	ZHT32TRIS	TRIS 32-bit Traditional Chinese	MB, ASCII
European	BG8MSWIN	MS Windows 8-bit Bulgarian Cyrillic	SB, ASCII
European	BG8PC437S	IBM-PC Code Page 437 8-bit (Bulgarian Modification)	SB, ASCII
European	BLT8CP921	Latvian Standard LVS8-92(1) Windows/Unix 8-bit Baltic	SB, ASCII
European	BLT8PC775	IBM-PC Code Page 775 8-bit Baltic	SB, ASCII
European	CDN8PC863	IBM-PC Code Page 863 8-bit Canadian French	SB, ASCII
European	CEL8ISO8859P14	ISO 8859-14 Celtic	SB, ASCII
European	CL8ISOIR111	ISOIR111 Cyrillic	SB, ASCII
European	CL8KOI8R	RELCOM Internet Standard 8-bit Latin/Cyrillic	SB, ASCII
European	CL8KOI8U	KOI8 Ukrainian Cyrillic	SB, ASCII
European	CL8MACCYRILLICS	Mac Server 8-bit Latin/Cyrillic	SB, ASCII
European	EE8MACCES	Mac Server 8-bit Central European	SB, ASCII
European	EE8MACCROATIANS	Mac Server 8-bit Croatian	SB, ASCII
European	EE8PC852	IBM-PC Code Page 852 8-bit East European	SB, ASCII
European	EL8DEC	DEC 8-bit Latin/Greek	SB, ASCII
European	EL8MACGREEKS	Mac Server 8-bit Greek	SB, ASCII
European	EL8PC437S	IBM-PC Code Page 437 8-bit (Greek modification)	SB, ASCII
European	EL8PC851	IBM-PC Code Page 851 8-bit Greek/Latin	SB, ASCII
European	EL8PC869	IBM-PC Code Page 869 8-bit Greek/Latin	SB, ASCII
European	ET8MSWIN923	MS Windows Code Page 923 8-bit Estonian	SB, ASCII
European	HU8ABMOD	Hungarian 8-bit Special AB Mod	SB, ASCII
European	HU8CWI2	Hungarian 8-bit CWI-2	SB, ASCII
European	IS8PC861	IBM-PC Code Page 861 8-bit Icelandic	SB, ASCII
European	LA8ISO6937	ISO 6937 8-bit Coded Character Set for Text Communication	SB, ASCII
European	LA8PASSPORT	German Government Printer 8-bit All-European Latin	SB, ASCII
European	LT8MSWIN921	MS Windows Code Page 921 8-bit Lithuanian	SB, ASCII
European	LT8PC772	IBM-PC Code Page 772 8-bit Lithuanian (Latin/Cyrillic)	SB, ASCII
European	LT8PC774	IBM-PC Code Page 774 8-bit Lithuanian (Latin)	SB, ASCII
European	LV8PC8LR	Latvian Version IBM-PC Code Page 866 8-bit Latin/Cyrillic	SB, ASCII
European	LV8PC1117	IBM-PC Code Page 1117 8-bit Latvian	SB, ASCII

Table A-6 (Cont.) Other ASCII-based Database Character Sets

Language Group	Character Set	Description	Comments
European	LV8RST104090	IBM-PC Alternative Code Page 8-bit Latvian (Latin/Cyrillic)	SB, ASCII
European	N8PC865	IBM-PC Code Page 865 8-bit Norwegian	SB, ASCII
European	RU8BESTA	BESTA 8-bit Latin/Cyrillic	SB, ASCII
European	RU8PC855	IBM-PC Code Page 855 8-bit Latin/Cyrillic	SB, ASCII
European	RU8PC866	IBM-PC Code Page 866 8-bit Latin/Cyrillic	SB, ASCII
European	SE8ISO8859P3	ISO 8859-3 South European	SB, ASCII
European	US7ASCII	ASCII 7-bit American	SB, ASCII
European	US8PC437	IBM-PC Code Page 437 8-bit American	SB, ASCII
European	WE8DEC	DEC 8-bit West European	SB, ASCII
European	WE8DG	DG 8-bit West European	SB, ASCII
European	WE8ISO8859P1	ISO 8859-1 West European	SB, ASCII
European	WE8MACROMAN8S	Mac Server 8-bit Extended Roman8 West European	SB, ASCII
European	WE8NCR4970	NCR 4970 8-bit West European	SB, ASCII
European	WE8NEXTSTEP	NeXTSTEP PostScript 8-bit West European	SB, ASCII
European	WE8PC850	IBM-PC Code Page 850 8-bit West European	SB, ASCII
European	WE8PC858	IBM-PC Code Page 858 8-bit West European	SB, ASCII, EURO
European	WE8PC860	IBM-PC Code Page 860 8-bit West European	SB, ASCII
European	WE8ROMAN8	HP Roman8 8-bit West European	SB, ASCII
Middle Eastern	AR8ADOS710	Arabic MS-DOS 710 Server 8-bit Latin/Arabic	SB, ASCII
Middle Eastern	AR8ADOS720	Arabic MS-DOS 720 Server 8-bit Latin/Arabic	SB, ASCII
Middle Eastern	AR8APTEC715	APTEC 715 Server 8-bit Latin/Arabic	SB, ASCII
Middle Eastern	AR8ARABICMACS	Mac Server 8-bit Latin/Arabic	SB, ASCII
Middle Eastern	AR8ASMO8X	ASMO Extended 708 8-bit Latin/Arabic	SB, ASCII
Middle Eastern	AR8MUSSAD768	Mussa'd Alarabi/2 768 Server 8-bit Latin/Arabic	SB, ASCII
Middle Eastern	AR8NAFITHA711	Nafitha Enhanced 711 Server 8-bit Latin/Arabic	SB, ASCII
Middle Eastern	AR8NAFITHA721	Nafitha International 721 Server 8-bit Latin/Arabic	SB, ASCII
Middle Eastern	AR8SAKHR706	SAKHR 706 Server 8-bit Latin/Arabic	SB, ASCII
Middle Eastern	AR8SAKHR707	SAKHR 707 Server 8-bit Latin/Arabic	SB, ASCII
Middle Eastern	AZ8ISO8859P9E	ISO 8859-9 Latin Azerbaijani	SB, ASCII

Table A-6 (Cont.) Other ASCII-based Database Character Sets

Language Group	Character Set	Description	Comments
Middle Eastern	IN8ISCI	Multiple-Script Indian Standard 8-bit Latin/Indian Languages	SB, ASCII
Middle Eastern	IW8MACHEBREWS	Mac Server 8-bit Hebrew	SB, ASCII
Middle Eastern	IW8PC1507	IBM-PC Code Page 1507/862 8-bit Latin/Hebrew	SB, ASCII
Middle Eastern	LA8ISO6937	ISO 6937 8-bit Coded Character Set for Text Communication	SB, ASCII
Middle Eastern	TR8DEC	DEC 8-bit Turkish	SB, ASCII
Middle Eastern	TR8MACTURKISHS	Mac Server 8-bit Turkish	SB, ASCII
Middle Eastern	TR8PC857	IBM-PC Code Page 857 8-bit Turkish	SB, ASCII
Universal	UTF8	Unicode 3.0 Universal character set, CESU-8 encoding scheme	MB, ASCII, EURO

[Table A-7](#) lists the other EBCDIC-based Oracle Database character sets. The list is ordered alphabetically within their language groups.

Table A-7 Other EBCDIC-based Database Character Sets

Language Group	Character Set	Description	Comments
Asian	JA16DBCS	IBM EBCDIC 16-bit Japanese	MB, UDC
Asian	JA16EBCDIC930	IBM DBCS Code Page 290 16-bit Japanese	MB, UDC
Asian	KO16DBCS	IBM EBCDIC 16-bit Korean	MB, UDC
Asian	TH8TISEBCDIC	Thai Industrial Standard 620-2533 - EBCDIC 8-bit	SB
Asian	TH8TISEBCDICS	Thai Industrial Standard 620-2533-EBCDIC Server 8-bit	SB
Asian	ZHS16DBCS	IBM EBCDIC 16-bit Simplified Chinese	MB, UDC
Asian	ZHT16DBCS	IBM EBCDIC 16-bit Traditional Chinese	MB, UDC
European	BLT8EBCDIC1112	EBCDIC Code Page 1112 8-bit Server Baltic Multilingual	SB
European	BLT8EBCDIC1112S	EBCDIC Code Page 1112 8-bit Server Baltic Multilingual	SB
European	CL8EBCDIC1025	EBCDIC Code Page 1025 8-bit Cyrillic	SB
European	CL8EBCDIC1025C	EBCDIC Code Page 1025 Client 8-bit Cyrillic	SB
European	CL8EBCDIC1025R	EBCDIC Code Page 1025 Server 8-bit Cyrillic	SB
European	CL8EBCDIC1025S	EBCDIC Code Page 1025 Server 8-bit Cyrillic	SB
European	CL8EBCDIC1025X	EBCDIC Code Page 1025 (Modified) 8-bit Cyrillic	SB
European	CL8EBCDIC1158	EBCDIC Code Page 1158 8-bit Cyrillic	SB
European	CL8EBCDIC1158R	EBCDIC Code Page 1158 Server 8-bit Cyrillic	SB
European	D8BS2000	Siemens 9750-62 EBCDIC 8-bit German	SB
European	D8EBCDIC273	EBCDIC Code Page 273/1 8-bit Austrian German	SB

Table A-7 (Cont.) Other EBCDIC-based Database Character Sets

Language Group	Character Set	Description	Comments
European	D8EBCDIC1141	EBCDIC Code Page 1141 8-bit Austrian German	SB, EURO
European	DK8BS2000	Siemens 9750-62 EBCDIC 8-bit Danish	SB
European	DK8EBCDIC277	EBCDIC Code Page 277/1 8-bit Danish	SB
European	DK8EBCDIC1142	EBCDIC Code Page 1142 8-bit Danish	SB, EURO
European	E8BS2000	Siemens 9750-62 EBCDIC 8-bit Spanish	SB
European	EL8EBCDIC423R	IBM EBCDIC Code Page 423 for RDBMS server-side	SB
European	EE8EBCDIC870	EBCDIC Code Page 870 8-bit East European	SB
European	EE8EBCDIC870C	EBCDIC Code Page 870 Client 8-bit East European	SB
European	EE8EBCDIC870S	EBCDIC Code Page 870 Server 8-bit East European	SB
European	EL8EBCDIC875	EBCDIC Code Page 875 8-bit Greek	SB
European	EL8EBCDIC875R	EBCDIC Code Page 875 Server 8-bit Greek	SB
European	EL8GCOS7	Bull EBCDIC GCOS7 8-bit Greek	SB
European	F8BS2000	Siemens 9750-62 EBCDIC 8-bit French	SB
European	F8EBCDIC297	EBCDIC Code Page 297 8-bit French	SB
European	F8EBCDIC1147	EBCDIC Code Page 1147 8-bit French	SB, EURO
European	I8EBCDIC280	EBCDIC Code Page 280/1 8-bit Italian	SB
European	I8EBCDIC1144	EBCDIC Code Page 1144 8-bit Italian	SB, EURO
European	S8BS2000	Siemens 9750-62 EBCDIC 8-bit Swedish	SB
European	S8EBCDIC278	EBCDIC Code Page 278/1 8-bit Swedish	SB
European	S8EBCDIC1143	EBCDIC Code Page 1143 8-bit Swedish	SB, EURO
European	US8BS2000	Siemens 9750-62 EBCDIC 8-bit American	SB
European	US8ICL	ICL EBCDIC 8-bit American	SB
European	WE8EBCDIC37	EBCDIC Code Page 37 8-bit West European	SB
European	WE8EBCDIC284	EBCDIC Code Page 284 8-bit Latin American/Spanish	SB
European	WE8EBCDIC285	EBCDIC Code Page 285 8-bit West European	SB
European	WE8EBCDIC500	EBCDIC Code Page 500 8-bit West European	SB
European	WE8EBCDIC500C	EBCDIC Code Page 500 8-bit Oracle/c	SB
European	WE8EBCDIC871	EBCDIC Code Page 871 8-bit Icelandic	SB
European	WE8EBCDIC924	Latin 9 EBCDIC 924	SB, EBCDIC
European	WE8EBCDIC1047	EBCDIC Code Page 1047 8-bit West European	SB
European	WE8EBCDIC1047E	Latin 1/Open Systems 1047	SB, EBCDIC, EURO
European	WE8EBCDIC1140	EBCDIC Code Page 1140 8-bit West European	SB, EURO
European	WE8EBCDIC1140C	EBCDIC Code Page 1140 8-bit West European	SB, EURO
European	WE8EBCDIC1145	EBCDIC Code Page 1145 8-bit West European	SB, EURO
European	WE8EBCDIC1146	EBCDIC Code Page 1146 8-bit West European	SB, EURO
European	WE8EBCDIC1148	EBCDIC Code Page 1148 8-bit West European	SB, EURO
European	WE8EBCDIC1148C	EBCDIC Code Page 1148 Client 8-bit West European	SB, EURO

Table A-7 (Cont.) Other EBCDIC-based Database Character Sets

Language Group	Character Set	Description	Comments
European	WE8GCOS7	Bull EBCDIC GCOS7 8-bit West European	SB
European	WE8ICL	ICL EBCDIC 8-bit West European	SB
Middle Eastern	AR8EBCDIC420S	EBCDIC Code Page 420 Server 8-bit Latin/Arabic	SB
Middle Eastern	AR8EBCDICX	EBCDIC XBASIC Server 8-bit Latin/Arabic	SB
Middle Eastern	IW8EBCDIC424	EBCDIC Code Page 424 8-bit Latin/Hebrew	SB
Middle Eastern	IW8EBCDIC424S	EBCDIC Code Page 424 Server 8-bit Latin/Hebrew	SB
Middle Eastern	IW8EBCDIC1086	EBCDIC Code Page 1086 8-bit Hebrew	SB
Middle Eastern	TR8EBCDIC1026	EBCDIC Code Page 1026 8-bit Turkish	SB
Middle Eastern	TR8EBCDIC1026S	EBCDIC Code Page 1026 Server 8-bit Turkish	SB
Middle Eastern	WE8EBCDIC37C	EBCDIC Code Page 37 8-bit Oracle/c	SB
Universal	UTFE	Unicode 3.0 Universal character set, UTF-EBCDIC encoding scheme	MB, EURO

A.4.3 Character Sets that Support the Euro Symbol

[Table A-8](#) lists the character sets that support the Euro symbol.

Table A-8 Character Sets that Support the Euro Symbol

Character Set Name	Hexadecimal Code Value of the Euro Symbol
AL16UTF16	20AC
AL32UTF8	E282AC
AR8MSWIN1256	80
BLT8MSWIN1257	80
CL8EBCDIC1158	E1
CL8EBCDIC1158R	9F
CL8MSWIN1251	88
D8EBCDIC1141	9F
DK8EBCDIC1142	5A
EE8MSWIN1250	80
EL8EBCDIC423R	FD
EL8EBCDIC875R	DF
EL8ISO8859P7	A4
EL8MSWIN1253	80
F8EBCDIC1147	9F
I8EBCDIC1144	9F
IW8MSWIN1255	80
KO16KSC5601	A2E6
KO16KSCCS	D9E6

Table A-8 (Cont.) Character Sets that Support the Euro Symbol

Character Set Name	Hexadecimal Code Value of the Euro Symbol
KO16MSWIN949	A2E6
S8EBCDIC1143	5A
TH8TISASCII	80
TR8MSWIN1254	80
UTF8	E282AC
UTFE	CA4653
VN8MSWIN1258	80
WE8BS2000E	9F
WE8EBCDIC1047E	9F
WE8EBCDIC1140	9F
WE8EBCDIC1140C	9F
WE8EBCDIC1145	9F
WE8EBCDIC1146	9F
WE8EBCDIC1148	9F
WE8EBCDIC1148C	9F
WE8EBCDIC924	9F
WE8ISO8859P15	A4
WE8MACROMAN8	DB
WE8MACROMAN8S	DB
WE8MSWIN1252	80
WE8PC858	DF
ZHS32GB18030	A2E3
ZHT16HKSCS	A3E1
ZHT16HKSCS31	A3E1
ZHT16MSWIN950	A3E1

A.4.4 Client-Only Character Sets

[Table A-9](#) and [Table A-10](#) list the Oracle Database character sets that are supported as client-only character sets. Character sets are listed alphabetically within their respective language groups.

[Table A-10](#) presents the EBCDIC-based client-only character sets added in Oracle AI Database 26ai, Release Update 23.26.2, to support mainframe platform migration. These character sets are designed to convert characters to Unicode—and, by extension, to ASCII-based character sets—precisely following the mappings defined by the IBM Character Data Representation Architecture (CDRA). See "[EBCDIC Emulation for Mainframe Migration](#)" for more information.

Table A-9 Client-Only Character Sets

Language Group	Character Set	Description	Comments
Asian	JA16EUCYEN	EUC 24-bit Japanese with 'I' mapped to the Japanese yen character	MB
Asian	JA16MACSJIS	Mac client Shift-JIS 16-bit Japanese	MB
Asian	JA16SJISYEN	Shift-JIS 16-bit Japanese with 'I' mapped to the Japanese yen character	MB, UDC
Asian	TH8MACTHAI	Mac Client 8-bit Latin/Thai	SB
Asian	ZHS16MACCGB231280	Mac client CGB2312-80 16-bit Simplified Chinese	MB
Asian	ZHS32GB18030	GB18030-2022 32-bit Simplified Chinese	MB
European	CH7DEC	DEC VT100 7-bit Swiss (German/French)	SB
European	CL8MACCYRILLIC	Mac Client 8-bit Latin/Cyrillic	SB
European	D7SIEMENS9780X	Siemens 97801/97808 7-bit German	SB
European	D7DEC	DEC VT100 7-bit German	SB
European	DK7SIEMENS9780X	Siemens 97801/97808 7-bit Danish	SB
European	EEC8EUROASCI	EEC Targon 35 ASCII West European/Greek	SB
European	EEC8EUROPA3	EEC EUROPA3 8-bit West European/Greek	SB
European	EE8MACCROATIAN	Mac Client 8-bit Croatian	SB
European	EE8MACCE	Mac Client 8-bit Central European	SB
European	EL8PC737	IBM-PC Code Page 737 8-bit Greek/Latin	SB
European	EL8MACGREEK	Mac Client 8-bit Greek	SB
European	E7DEC	DEC VT100 7-bit Spanish	SB
European	E7SIEMENS9780X	Siemens 97801/97808 7-bit Spanish	SB
European	F7DEC	DEC VT100 7-bit French	SB
European	F7SIEMENS9780X	Siemens 97801/97808 7-bit French	SB
European	I7DEC	DEC VT100 7-bit Italian	SB
European	I7SIEMENS9780X	Siemens 97801/97808 7-bit Italian	SB
European	IS8MACICELANDICS	Mac Server 8-bit Icelandic	SB
European	IS8MACICELANDIC	Mac Client 8-bit Icelandic	SB
European	NL7DEC	DEC VT100 7-bit Dutch	SB
European	NDK7DEC	DEC VT100 7-bit Norwegian/Danish	SB
European	N7SIEMENS9780X	Siemens 97801/97808 7-bit Norwegian	SB
European	SF7DEC	DEC VT100 7-bit Finnish	SB
European	S7SIEMENS9780X	Siemens 97801/97808 7-bit Swedish	SB
European	S7DEC	DEC VT100 7-bit Swedish	SB
European	SF7ASCII	ASCII 7-bit Finnish	SB
European	WE8ISOICLUK	ICL special version ISO8859-1	SB
European	WE8MACROMAN8	Mac Client 8-bit Extended Roman8 West European	SB
European	WE8HP	HP LaserJet 8-bit West European	SB
European	YUG7ASCII	ASCII 7-bit Yugoslavian	SB

Table A-9 (Cont.) Client-Only Character Sets

Language Group	Character Set	Description	Comments
Middle Eastern	AR8ARABICMAC	Mac Client 8-bit Latin/Arabic	SB
Middle Eastern	IW7IS960	Israeli Standard 960 7-bit Latin/Hebrew	SB
Middle Eastern	IW8MACHEBREW	Mac Client 8-bit Hebrew	SB
Middle Eastern	TR7DEC	DEC VT100 7-bit Turkish	SB
Middle Eastern	TR8MACTURKISH	Mac Client 8-bit Turkish	SB

Table A-10 Client-Only EBCDIC Character Sets with IBM CDRA Unicode Mappings

Language Group	Character Set	Description	Comments
European	IBM37	IBM EBCDIC Code Page 37 for Western Europe	SB
European	IBM273	IBM EBCDIC Code Page 273 for German	SB
European	IBM277	IBM EBCDIC Code Page 277 for Danish and Norwegian	SB
European	IBM278	IBM EBCDIC Code Page 278 for Finnish and Swedish	SB
European	IBM280	IBM EBCDIC Code Page 280 for Italian	SB
European	IBM284	IBM EBCDIC Code Page 284 for Spanish	SB
European	IBM285	IBM EBCDIC Code Page 285 for United Kingdom	SB
European	IBM297	IBM EBCDIC Code Page 297 for French	SB
European	IBM500	IBM EBCDIC Code Page 500 International Latin 1	SB
European	IBM875	IBM EBCDIC Code Page 875 for Greek	SB
European	IBM1140	IBM EBCDIC Code Page 1140 (IBM-37 + euro)	SB, EURO
European	IBM1141	IBM EBCDIC Code Page 1141 (IBM-273 + euro)	SB, EURO
European	IBM1142	IBM EBCDIC Code Page 1142 (IBM-277 + euro)	SB, EURO
European	IBM1143	IBM EBCDIC Code Page 1143 (IBM-278 + euro)	SB, EURO
European	IBM1144	IBM EBCDIC Code Page 1144 (IBM-280 + euro)	SB, EURO
European	IBM1145	IBM EBCDIC Code Page 1145 (IBM-284 + euro)	SB, EURO
European	IBM1146	IBM EBCDIC Code Page 1146 (IBM-285 + euro)	SB, EURO
European	IBM1147	IBM EBCDIC Code Page 1147 (IBM-297 + euro)	SB, EURO
European	IBM1148	IBM EBCDIC Code Page 1148 (IBM-500 + euro)	SB, EURO
Middle Eastern	IBM420	IBM EBCDIC Code Page 420 for Arabic	SB
Middle Eastern	IBM1026	IBM EBCDIC Code Page 1026 for Turkish	SB

A.4.5 Universal Character Sets

[Table A-11](#) lists the Oracle Database character sets that provide universal language support. They attempt to support all languages of the world, including, but not limited to, Asian, European, and Middle Eastern languages.

Table A-11 Universal Character Sets

Name	Description	Comments
AL16UTF16	Unicode 15.0 Universal character set, UTF-16BE encoding scheme (Oracle AI Database 26ai, RU 23.26.2 and earlier) Unicode 17.0 Universal character set, UTF-16BE encoding scheme (Oracle AI Database 26ai, RU 23.26.3 and later)	MB, EURO, FIXED
AL32UTF8	Unicode 15.0 Universal character set, UTF-8 encoding scheme	MB, ASCII, EURO
UTF8	Unicode 3.0 Universal character set, CESU-8 encoding scheme	MB, ASCII, EURO
UTFE	Unicode 3.0 Universal character set, UTF-EBCDIC encoding scheme	MB, EURO

Note

CESU-8 defines an encoding scheme for Unicode that is identical to UTF-8 except for its representation of supplementary characters. In CESU-8, supplementary characters are represented as six-byte sequences that result from the transformation of each UTF-16 surrogate code unit into an eight-bit form that is similar to the UTF-8 transformation, but without first converting the input surrogate pairs to a scalar value.

See Also

- ["Supporting Multilingual Databases with Unicode"](#)
- Unicode Technical Report #26 "Compatibility Encoding Scheme for UTF-16: 8-Bit (CESU-8)" published on *The Unicode Consortium* website

A.4.6 Character Set Conversion Support

The following character set encodings are supported for conversion only. They cannot be used as database character set or national character set.

Table A-12 Character Set Encodings Supported for Conversion Only

Character Set Encoding	Description
UTF-16 (little-endian)	This is a 16-bit little-endian encoding form of Unicode. The corresponding Oracle Database character set is AL16UTF16LE.

Table A-12 (Cont.) Character Set Encodings Supported for Conversion Only

Character Set Encoding	Description
ISO2022-CN	This encoding covers a variety of Chinese character encodings. They supports both simplified and traditional characters using both GB and CNS/BIG5. Being distinguished by escape sequences and shift characters, these character sets include ASCII, GB 2312-80, CNS 11643-1992 Planes 1 and 2. The corresponding Oracle Database character sets are ZHS16CGB231280, ZHS16GBK and ZHT32TRIS.
ISO2022-JP	This encoding is a mixture of ASCII, JIS-Romann, JIS C 6226-1978 and JIS X 0208:1997. It is a subset of Oracle Database character set JA16EUC and can be converted to JA16EUC character set after some calculation.
ISO2022-KR	This encoding covers ASCII and KSC 5601 character sets. The corresponding Oracle Database character sets are KO16MSWIN949 and KO16KSC5601.
HZ-GB-2312	This encoding covers GB 2312-80, ASCII and GB-Roman. The corresponding Oracle Database character set is ZHS16CGB231280.

You can use the Oracle Database character sets related to these encodings as the values for the CONVERT function parameters `source_char_set` and `dest_char_set`.

① See Also

- ["The CONVERT Function"](#)
- *Oracle AI Database SQL Language Reference* for more information about the CONVERT function

A.4.7 Binary Subset-Superset Pairs

Oracle Database does not maintain a list of all subset-superset pairs of its character sets but it does maintain a list of binary subset-superset pairs that it recognizes when checking compatibility of two character sets.

[Table A-13](#) lists all binary subset-superset relationships recognized by Oracle Database.

Table A-13 Binary Subset-Superset Pairs

Subset	Superset
AR8ARABICMACT	AR8ARABICMAC
AR8ISO8859P6	AR8ASMO8X
BLT8CP921	BLT8ISO8859P13
BLT8CP921	LT8MSWIN921
D7DEC	D7SIEMENS9780X
D7SIEMENS9780X	D7DEC
DK7SIEMENS9780X	N7SIEMENS9780X
I7DEC	I7SIEMENS9780X
I7SIEMENS9780X	IW8EBCDIC424
IW8EBCDIC424	IW8EBCDIC1086

Table A-13 (Cont.) Binary Subset-Superset Pairs

Subset	Superset
KO16KSC5601	KO16MSWIN949
LT8MSWIN921	BLT8ISO8859P13
LT8MSWIN921	BLT8CP921
N7SIEMENS9780X	DK7SIEMENS9780X
US7ASCII	See " Binary Supersets of US7ASCII "
UTF8	AL32UTF8
WE8DEC	TR8DEC
WE8DEC	WE8NCR4970
WE8ISO8859P1	WE8MSWIN1252
WE8ISO8859P9	TR8MSWIN1254
WE8NCR4970	TR8DEC
WE8NCR4970	WE8DEC
WE8PC850	WE8PC858

US7ASCII is a special case because so many other character sets are supersets of it.

Binary Supersets of US7ASCII

The following is a list of all the character sets that are binary supersets of US7ASCII that are recognized by Oracle Database. These character sets are listed in the alphabetical order.

Table A-14 Character Sets That Are Binary Supersets of US7ASCII

Character Set	Character Set	Character Set	Character Set
AL32UTF8	CL8MACCYRILLICS	JA16VMS	VN8MSWIN1258
AR8ADOS710	CL8MSWIN1251	KO16KSC5601	VN8VN3
AR8ADOS720	EE8ISO8859P2	KO16KSCCS	WE8DEC
AR8APTEC715	EE8MACCES	KO16MSWIN949	WE8DG
AR8ARABICMACS	EE8MACCROATIANS	LA8ISO6937	WE8ISO8859P1
AR8ASMO8X	EE8MSWIN1250	LA8PASSPORT	WE8ISO8859P15
AR8ISO8859P6	EE8PC852	LT8MSWIN921	WE8ISO8859P9
AR8MSWIN1256	EL8DEC	LT8PC772	WE8MACROMAN8S
AR8MUSSAD768	EL8ISO8859P7	LT8PC774	WE8MSWIN1252
AR8NAFITHA711	EL8MACGREEKS	LV8PC1117	WE8NCR4970
AR8NAFITHA721	EL8MSWIN1253	LV8PC8LR	WE8NEXTSTEP
AR8SAKHR706	EL8PC437S	LV8RST104090	WE8PC850
AR8SAKHR707	EL8PC851	N8PC865	WE8PC858
AZ8ISO8859PE	EL8PC869	NE8ISO8859P10	WE8PC860
BG8MSWIN	ET8MSWIN923	NEE8ISO8859P4	WE8ROMAN8
BG8PC437S	HU8ABMOD	RU8BESTA	ZHS16CGB231280
BLT8CP921	HU8CW12	RU8PC855	ZHS16GBK
BLT8ISO8859P13	IN8SCII	RU8PC866	ZHS32GB18030
BLT8MSWIN1257	IS8PC861	SE8ISO8859P3	ZHT16BIG5
BLT8PC775	IW8ISO8859P8	TH8MACTHAIS	ZHT16CCDC
BN8BCSII	IW8MACHEBREWS	TH8TISASCII	ZHT16DBT
CDN8PC863	IW8MSWIN1255	TR8DEC	ZHT16HKSCS
CEL8ISO8859P14	IW8PC1507	TR8MACTURKISHS	ZHT16MSWIN950
CL8ISO8859P5	JA16EUC	TR8MSWIN1254	ZHT32EUC
CL8ISOIR111	JA16EUCTILDE	TR8PC857	ZHT32SOPS
CL8KOI8R	JA16SJIS	US8PC437	ZHT32TRIS
CL8KOI8U	JA16SJISTILDE	UTF8	

See Also

"[Subsets and Supersets](#)" for discussion of what subsets and supersets of a character set are

A.4.8 Mapping EBCDIC to ASCII Character Sets

[Table A-15](#) lists single-byte ASCII-based character sets that most closely match selected single-byte IBM EBCDIC character sets (code pages). This table is intended to help you select a target database character set when migrating data from a mainframe platform to an ASCII-based platform, such as Linux. See "[Choosing a Character Set for the Target Database](#)" for more details.

Table A-15 Best ASCII-Based Character Set to Match EBCDIC-Based Code Pages

Language Group	EBCDIC Code Page	Matching ASCII-Based Character Sets	Comments
European	IBM-37	WE8ISO8859P1	Full round-trip conversion
European	IBM-273	WE8ISO8859P1	Full round-trip conversion
European	IBM-277	WE8ISO8859P1	Full round-trip conversion
European	IBM-278	WE8ISO8859P1	Full round-trip conversion
European	IBM-280	WE8ISO8859P1	Full round-trip conversion
European	IBM-284	WE8ISO8859P1	Full round-trip conversion
European	IBM-285	WE8ISO8859P1	Full round-trip conversion
European	IBM-297	WE8ISO8859P1	Full round-trip conversion
European	IBM-500	WE8ISO8859P1	Full round-trip conversion
European	IBM-875	EL8ISO8859P7	Missing U+00B4 Acute Accent and U+0387 Greek Ano Teleia; U+00B4 can be mapped to U+0384; U+0387 may be mapped to U+00B7
European	IBM-1140	WE8ISO8859P1	Missing U+20AC Euro Sign
		WE8ISO8859P15	Missing U+00A6 Broken Bar, U+00A8 Diaeresis, U+00B4 Acute Accent, U+00B8 Cedilla, U+00BC Vulgar Fraction One Quarter, U+00BD Vulgar Fraction One Half, and U+00BE Vulgar Fraction Three Quarters
		WE8MSWIN1252	Missing control codes U+0080, U+0082—U+008C, U+008E, U+0091—U+009C, U+009E, and U+009F; the lack of the character U+0085 Next Line may be significant
European	IBM-1141	WE8ISO8859P1 WE8ISO8859P15 WE8MSWIN1252	See comments for IBM-1140
European	IBM-1142	WE8ISO8859P1 WE8ISO8859P15 WE8MSWIN1252	See comments for IBM-1140
European	IBM-1143	WE8ISO8859P1 WE8ISO8859P15 WE8MSWIN1252	See comments for IBM-1140

Table A-15 (Cont.) Best ASCII-Based Character Set to Match EBCDIC-Based Code Pages

Language Group	EBCDIC Code Page	Matching ASCII-Based Character Sets	Comments
European	IBM-1144	WE8ISO8859P1 WE8ISO8859P15 WE8MSWIN1252	See comments for IBM-1140
European	IBM-1145	WE8ISO8859P1 WE8ISO8859P15 WE8MSWIN1252	See comments for IBM-1140
European	IBM-1146	WE8ISO8859P1 WE8ISO8859P15 WE8MSWIN1252	See comments for IBM-1140
European	IBM-1147	WE8ISO8859P1 WE8ISO8859P15 WE8MSWIN1252	See comments for IBM-1140
European	IBM-1148	WE8ISO8859P1 WE8ISO8859P15 WE8MSWIN1252	See comments for IBM-1140
Middle Eastern	IBM-420	AR8ISO8859P6	Missing U+00A2 Cent Sign, U+00A6 Broken Bar, U+00AC Not Sign, U+00D7 Multiplication Sign, U+00F7 Division Sign, U+200B Zero-Width Space, U+0660—U+0669 Arabic-Indic Digit Zero to Nine, and 37 presentation forms (initial, medial, final, ligatured) of Arabic letters; Oracle single-byte database character sets for Arabic do not support presentation forms, only AL32UTF8 does
Middle Eastern	IBM-1026	WE8ISO8859P9	Full round-trip conversion

A.5 Language and Character Set Detection Support

[Table A-16](#) displays the languages and character sets that are supported by the Language and Character Set Detection utility (LCSSCAN) and the Globalization Development Kit (GDK).

Each language has several character sets that can be detected.

When the binary values for a language match two or more encodings that have a subset/superset relationship, the subset character set is returned. For example, if the language is German and all characters are 7-bit, then US7ASCII is returned instead of WE8MSWIN1252, WE8ISO8859P15, or WE8ISO8859P1.

When the character set is determined to be UTF-8, the Oracle Database character set UTF8 is returned by default unless 4-byte characters (supplementary characters) are detected within the text. If 4-byte characters are detected, then the character set is reported as AL32UTF8.

Table A-16 Languages and Character Sets Supported by LCSSCAN and GDK

Language	Character Sets
Arabic	AL16UTF16, AL32UTF8, AR8ISO8859P6, AR8MSWIN1256, UTF8
Bulgarian	AL16UTF16, AL32UTF8, CL8ISO8859P5, CL8MSWIN1251, UTF8

Table A-16 (Cont.) Languages and Character Sets Supported by LCSSCAN and GDK

Language	Character Sets
Catalan	AL16UTF16, AL32UTF8, US7ASCII, UTF8, WE8ISO8859P1, WE8ISO8859P15, WE8MSWIN1252
Croatian	AL16UTF16, AL32UTF8, EE8ISO8859P2, EE8MSWIN1250, UTF8
Czech	AL16UTF16, AL32UTF8, EE8ISO8859P2, EE8MSWIN1250, UTF8
Danish	AL16UTF16, AL32UTF8, US7ASCII, UTF8, WE8ISO8859P1, WE8ISO8859P15, WE8MSWIN1252
Dutch	AL16UTF16, AL32UTF8, US7ASCII, UTF8, WE8ISO8859P1, WE8ISO8859P15, WE8MSWIN1252
English	AL16UTF16, AL32UTF8, US7ASCII, UTF8, WE8ISO8859P1, WE8ISO8859P15, WE8MSWIN1252
Estonian	AL16UTF16, AL32UTF8, NEE8ISO8859P4, UTF8
Finnish	AL16UTF16, AL32UTF8, US7ASCII, UTF8, WE8ISO8859P1, WE8ISO8859P15, WE8MSWIN1252
French	AL16UTF16, AL32UTF8, US7ASCII, UTF8, WE8ISO8859P1, WE8ISO8859P15, WE8MSWIN1252
German	AL16UTF16, AL32UTF8, US7ASCII, UTF8, WE8ISO8859P1, WE8ISO8859P15, WE8MSWIN1252
Greek	AL16UTF16, AL32UTF8, EL8ISO8859P7, EL8MSWIN1253, UTF8
Hebrew	AL16UTF16, AL32UTF8, IW8ISO8859P8, IW8MSWIN1255, UTF8
Hindi	AL16UTF16, AL32UTF8, IN8ISCII, UTF8
Hungarian	AL16UTF16, AL32UTF8, EE8ISO8859P2, EE8MSWIN1250, UTF8
Indonesian	AL16UTF16, AL32UTF8, US7ASCII, UTF8, WE8ISO8859P1, WE8ISO8859P15, WE8MSWIN1252
Italian	AL16UTF16, AL32UTF8, US7ASCII, UTF8, WE8ISO8859P1, WE8ISO8859P15, WE8MSWIN1252
Japanese	AL16UTF16, AL32UTF8, ISO2022-JP, JA16EUC, JA16SJIS, UTF8
Korean	AL16UTF16, AL32UTF8, ISO2022-KR, KO16KSC5601, KO16MSWIN949, UTF8
Latvian	AL16UTF16, AL32UTF8, NEE8ISO8859P4, UTF8
Lithuanian	AL16UTF16, AL32UTF8, NEE8ISO8859P4, UTF8
Malay	AL16UTF16, AL32UTF8, US7ASCII, UTF8, WE8ISO8859P1, WE8ISO8859P15, WE8MSWIN1252
Norwegian	AL16UTF16, AL32UTF8, US7ASCII, UTF8, WE8ISO8859P1, WE8ISO8859P15, WE8MSWIN1252
Persian	AL16UTF16, AL32UTF8, AR8MSWIN1256, UTF8
Polish	AL16UTF16, AL32UTF8, EE8ISO8859P2, EE8MSWIN1250, UTF8
Portuguese	AL16UTF16, AL32UTF8, US7ASCII, UTF8, WE8ISO8859P1, WE8ISO8859P15, WE8MSWIN1252
Romanian	AL16UTF16, AL32UTF8, EE8ISO8859P2, EE8MSWIN1250, UTF8
Russian	AL16UTF16, AL32UTF8, CL8ISO8859P5, CL8KOI8R, CL8MSWIN1251, RU8PC866, UTF8
Serbian	AL16UTF16, AL32UTF8, CL8ISO8859P5, CL8MSWIN1251, UTF8
Simplified Chinese	AL16UTF16, AL32UTF8, HZ-GB-2312, UTF8, ZHS16GBK, ZHS16CGB231280
Slovak	AL16UTF16, AL32UTF8, EE8ISO8859P2, EE8MSWIN1250, UTF8

Table A-16 (Cont.) Languages and Character Sets Supported by LCSSCAN and GDK

Language	Character Sets
Slovenian	AL16UTF16, AL32UTF8, EE8ISO8859P2, EE8MSWIN1250, UTF8
Spanish	AL16UTF16, AL32UTF8, US7ASCII, UTF8, WE8ISO8859P1, WE8ISO8859P15, WE8MSWIN1252
Swedish	AL16UTF16, AL32UTF8, US7ASCII, UTF8, WE8ISO8859P1, WE8ISO8859P15, WE8MSWIN1252
Thai	AL16UTF16, AL32UTF8, TH8TISASCII, UTF8
Traditional Chinese	AL16UTF16, AL32UTF8, UTF8, ZHT16MSWIN950
Turkish	AL16UTF16, AL32UTF8, TR8MSWIN1254, UTF8, WE8ISO8859P9
Ukrainian	AL16UTF16, AL32UTF8, CL8ISO8859P5, CL8MSWIN1251, UTF8
Vietnamese	AL16UTF16, AL32UTF8, VN8VN3, UTF8

A.6 Linguistic Collations

Oracle Database provides four kinds of linguistic collations: monolingual collations, multilingual collations, Unicode Collation Algorithm (UCA) collations, and emulated binary collations. Emulated binary collations were introduced with Oracle AI Database 26ai, Release Update 23.26.2.

A monolingual collation is usually created to sort character data in a single language and is named after the corresponding language. Some languages have multiple collations implementing multiple sorting standards for each language. Some monolingual collations have an extended version that handles special linguistic cases. The name of the extended version is prefixed with the letter X. These special cases typically mean that one character is sorted like a sequence of two characters or a sequence of two characters is sorted as one character. For example, *ch* and *ll* are treated as a single character in XSPANISH. Extended monolingual collations may also define special language-specific uppercase and lowercase rules that override standard rules of a character set.

All the linguistic collations can additionally be specified as case-insensitive or accent-insensitive by appending `_CI` or `_AI` to the linguistic collation name respectively.

[Table A-17](#) lists the monolingual linguistic collations supported by Oracle Database.

See Also

[Table A-1, "Oracle Database Supported Languages"](#) for a list of the default collation for each language

Table A-17 Monolingual Linguistic Collations

Basic Name	Extended Name	Special Cases
ARABIC	-	-
ARABIC_MATCH	-	-
ARABIC_ABJ_SORT	-	-

Table A-17 (Cont.) Monolingual Linguistic Collations

Basic Name	Extended Name	Special Cases
ARABIC_ABJ_MATCH	-	-
ASCII7	-	-
AZERBAIJANI	XAZERBAIJANI	i, I, lowercase i without dot, uppercase I with dot
BENGALI	-	-
BIG5 ¹	-	-
BINARY	-	-
BULGARIAN	-	-
CATALAN	XCATALAN	æ, AE, ß
CROATIAN	XCROATIAN	D, L, N, d, l, n, ß
CZECH	XCZECH	ch, CH, Ch, ß
CZECH_PUNCTUATION	XCZECH_PUNCTUATION	ch, CH, Ch, ß
DANISH	XDANISH	A, ß, Å, å
DUTCH	XDUTCH	ij, IJ
EBCDIC	-	-
EEC_EURO	-	-
EEC_EUROPA3	-	-
ESTONIAN	-	-
FINNISH	-	-
FRENCH	XFRENCH	-
GBK ¹	-	-
GERMAN	XGERMAN	ß
GERMAN	XGERMAN_S	ß, uppercase ß
GERMAN_DIN	XGERMAN_DIN	ß, ä, ö, ü, Ä, Ö, Ü
GERMAN_DIN	XGERMAN_DIN_S	ß, ä, ö, ü, uppercase ß, Ä, Ö, Ü
GREEK	-	-
HEBREW	-	-
HKSCS ¹	-	-
HUNGARIAN	XHUNGARIAN	cs, gy, ny, sz, ty, zs, ß, CS, Cs, GY, Gy, NY, Ny, SZ, Sz, TY, Ty, ZS, Zs
ICELANDIC	-	-
ICELANDIC_IST	-	-
INDONESIAN	-	-
ITALIAN	-	-
LATIN	-	-
LATVIAN	-	-
LITHUANIAN	-	-
MALAY	-	-

Table A-17 (Cont.) Monolingual Linguistic Collations

Basic Name	Extended Name	Special Cases
NORWEGIAN	-	-
POLISH	-	-
PUNCTUATION	XPUNCTUATION	-
ROMANIAN	-	-
RUSSIAN	-	-
SLOVAK	XSLOVAK	dz, DZ, Dz, ß (<i>caron</i>)
SLOVENIAN	XSLOVENIAN	ß
SPANISH	XSPANISH	ch, ll, CH, Ch, LL, Ll
SWEDISH	-	-
SWISS	XSWISS	ß
TURKISH	XTURKISH	æ, AE, ß
UKRAINIAN	-	-
UNICODE_BINARY	-	-
VIETNAMESE	-	-
WEST_EUROPEAN	XWEST_EUROPEAN	ß

¹ The collations BIG5, GBK, and HKSCS are implemented using the multilingual collation format and emulate the binary orders of the ZHT16BIG5, ZHS16GBK, and ZHT16HKSCS character sets, respectively.

[Table A-18](#) lists the multilingual linguistic collations available in Oracle Database. All of them include GENERIC_M (an ISO standard for sorting Latin-based characters) as a base. Multilingual linguistic collations are used for a specific primary language together with Latin-based characters. For example, KOREAN_M sorts Korean and Latin-based characters, but it does not sort Chinese, Thai, or Japanese characters.

Table A-18 Multilingual Linguistic Collations

Collation Name	Description
CANADIAN_M	Canadian French collation supports reverse secondary, special expanding characters
DANISH_M	Danish collation supports sorting uppercase characters before lowercase characters
FRENCH_M	French collation supports reverse sort for secondary
GENERIC_M	Generic sorting order which is based on ISO14651 and Unicode canonical equivalence rules but excluding compatible equivalence rules
JAPANESE_M	Japanese collation supports SJIS character set order and EUC characters which are not included in SJIS
KOREAN_M	Korean collation: Hangul characters are based on Unicode binary order. Hanja characters based on pronunciation order. All Hangul characters are before Hanja characters
SPANISH_M	Traditional Spanish collation supports special contracting characters
THAI_M	Thai collation supports swap characters for some vowels and consonants

Table A-18 (Cont.) Multilingual Linguistic Collations

Collation Name	Description
SCHINESE_RADICAL_M	Simplified Chinese collation based on radical as primary order and number of strokes order as secondary order
SCHINESE_STROKE_M	Simplified Chinese collation uses number of strokes as primary order and radical as secondary order
SCHINESE_PINYIN_M	Simplified Chinese PinYin sorting order
TCHINESE_RADICAL_M	Traditional Chinese collation based on radical as primary order and number of strokes order as secondary order
TCHINESE_STROKE_M	Traditional Chinese collation uses number of strokes as primary order and radical as secondary order. It supports supplementary characters.

 See Also

["Linguistic Sorting and Matching"](#)

[Table A-19](#) illustrates UCA collations.

Table A-19 UCA Collations

Collation Name	UCA Version	Language	Collation Type	Default Setting for Collation Parameters
UCA1210_DUCET	12.1	All	DUCET	_S4_VS_BN_NY_EN_FN_HN_DN_MN
UCA1210_ROOT	12.1	All (CLDR root)	standard	_S4_VS_BN_NY_EN_FN_HN_DN_MN
UCA1210_ORADUCET	12.1	All (Oracle tailored)	ORADUCE T	_S4_VS_BN_NY_EN_FN_HN_DN_MN
UCA1210_ORAROOT	12.1	All (CLDR root, Oracle tailored)	ORAROOT	_S4_VS_BN_NY_EN_FN_HN_DN_MN
UCA1210_CFRENCH	12.1	Canadian French	standard	_S4_VS_BY ¹ _NY_EN_FN_HN_DN_MN
UCA1210_DANISH	12.1	Danish	standard	_S4_VS_BN_NY_EN_FU ² _HN_DN_MN
UCA1210_JAPANESE	12.1	Japanese	standard	_S4_VS_BN_NY_EN_FN_HY ³ _DN_MN
UCA1210_JAPANESE_IVS	12.1	Japanese	IVS	_S4_VS_BN_NY_EN_FN_HY ³ _DN_MN
UCA1210_KOREAN	12.1	Korean	standard	_S4_VS_BN_NY_EN_FN_HN_DN_MN
UCA1210_SPANISH	12.1	Spanish	standard	_S4_VS_BN_NY_EN_FN_HN_DN_MN
UCA1210_TSPANISH	12.1	Spanish	traditional	_S4_VS_BN_NY_EN_FN_HN_DN_MN
UCA1210_THAI	12.1	Thai	standard	_S4_VS_BN_NY_EN_FN_HN_DN_MN
UCA1210_SCHINESE	12.1	Simplified Chinese	pinyin	_S4_VS_BN_NY_EN_FN_HN_DN_MN
UCA1210_SCHINESE1	12.1	Simplified Chinese	radical	_S4_VS_BN_NY_EN_FN_HN_DN_MN
UCA1210_SCHINESE2	12.1	Simplified Chinese	stroke	_S4_VS_BN_NY_EN_FN_HN_DN_MN
UCA1210_TCHINESE	12.1	Traditional Chinese	stroke	_S4_VS_BN_NY_EN_FN_HN_DN_MN
UCA1210_TCHINESE1	12.1	Traditional Chinese	radical	_S4_VS_BN_NY_EN_FN_HN_DN_MN
UCA0700_DUCET	7.0	All	DUCET	_S4_VS_BN_NY_EN_FN_HN_DN_MN

Table A-19 (Cont.) UCA Collations

Collation Name	UCA Version	Language	Collation Type	Default Setting for Collation Parameters
UCA0700_ROOT	7.0	All	CLDR root	_S4_VS_BN_NY_EN_FN_HN_DN_MN
UCA0700_ORADUCET	7.0	All	DUCET	_S4_VS_BN_NY_EN_FN_HN_DN_MN
UCA0700_ORAROOT	7.0	All	CLDR root	_S4_VS_BN_NY_EN_FN_HN_DN_MN
UCA0700_CFRENCH	7.0	Canadian French	standard	_S4_VS_ BY ⁴ _NY_EN_FN_HN_DN_MN
UCA0700_DANISH	7.0	Danish	standard	_S4_VS_BN_NY_EN_ FU ⁵ _HN_DN_MN
UCA0700_JAPANESE	7.0	Japanese	standard	_S4_VS_BN_NY_EN_FN_ HY ⁶ _DN_MN
UCA0700_KOREAN	7.0	Korean	standard	_S4_VS_BN_NY_EN_FN_HN_DN_MN
UCA0700_SPANISH	7.0	Spanish	standard	_S4_VS_BN_NY_EN_FN_HN_DN_MN
UCA0700_TSPANISH	7.0	Spanish	traditional	_S4_VS_BN_NY_EN_FN_HN_DN_MN
UCA0700_THAI	7.0	Thai	standard	_S4_VS_BN_NY_EN_FN_HN_DN_MN
UCA0700_SCHINESE	7.0	Simplified Chinese	pinyin	_S4_VS_BN_NY_EN_FN_HN_DN_MN
UCA0700_SCHINESE1	7.0	Simplified Chinese	radical	_S4_VS_BN_NY_EN_FN_HN_DN_MN
UCA0700_SCHINESE2	7.0	Simplified Chinese	stroke	_S4_VS_BN_NY_EN_FN_HN_DN_MN
UCA0700_TCHINESE	7.0	Traditional Chinese	stroke	_S4_VS_BN_NY_EN_FN_HN_DN_MN
UCA0700_TCHINESE1	7.0	Traditional Chinese	radical	_S4_VS_BN_NY_EN_FN_HN_DN_MN
UCA0620_DUCET	6.2	All	DUCET	_S4_VS_BN_NY_EN_FN_HN_DN_MN
UCA0620_ROOT	6.2	All	CLDR root	_S4_VS_BN_NY_EN_FN_HN_DN_MN
UCA0620_CFRENCH	6.2	Canadian French	standard	_S4_VS_ BY ⁷ _NY_EN_FN_HN_DN_MN
UCA0620_DANISH	6.2	Danish	standard	_S4_VS_BN_NY_EN_ FU ⁸ _HN_DN_MN
UCA0620_JAPANESE	6.2	Japanese	standard	_S4_VS_BN_NY_EN_FN_ HY ⁹ _DN_MN
UCA0620_KOREAN	6.2	Korean	standard	_S4_VS_BN_NY_EN_FN_HN_DN_MN
UCA0620_SPANISH	6.2	Spanish	standard	_S4_VS_BN_NY_EN_FN_HN_DN_MN
UCA0620_TSPANISH	6.2	Spanish	traditional	_S4_VS_BN_NY_EN_FN_HN_DN_MN
UCA0620_THAI	6.2	Thai	standard	_S4_VS_BN_NY_EN_FN_HN_DN_MN
UCA0620_SCHINESE	6.2	Simplified Chinese	pinyin	_S4_VS_BN_NY_EN_FN_HN_DN_MN
UCA0620_SCHINESE1	6.2	Simplified Chinese	radical	_S4_VS_BN_NY_EN_FN_HN_DN_MN
UCA0620_SCHINESE2	6.2	Simplified Chinese	stroke	_S4_VS_BN_NY_EN_FN_HN_DN_MN
UCA0620_TCHINESE	6.2	Traditional Chinese	stroke	_S4_VS_BN_NY_EN_FN_HN_DN_MN
UCA0620_TCHINESE1	6.2	Traditional Chinese	radical	_S4_VS_BN_NY_EN_FN_HN_DN_MN

¹ Default setting of **_BY** is unique for UCA1210_CFRENCH collation. For all other UCA 12.1 collations, the default setting is **_BN**.

² Default setting of **_FU** is unique for UCA1210_DANISH collation. For all other UCA 12.1 collations, the default setting is **_FN**.

³ Default setting of **_HY** is unique for UCA1210_JAPANESE and UCA1210_JAPANESE_IVS collations. For all other UCA 12.1 collations, the default setting is **_HN**.

⁴ Default setting of **_BY** is unique for UCA0700_CFRENCH collation. For all other UCA 7.0 collations, the default setting is **_BN**.

⁵ Default setting of **_FU** is unique for UCA0700_DANISH collation. For all other UCA 7.0 collations, the default setting is **_FN**.

⁶ Default setting of **_HY** is unique for UCA0700_JAPANESE collation. For all other UCA 7.0 collations, the default setting is **_HN**.

⁷ Default setting of **_BY** is unique for UCA0620_CFRENCH collation. For all other UCA 6.2 collations, the default setting is **_BN**.

⁸ Default setting of **_FU** is unique for UCA0620_DANISH collation. For all other UCA 6.2 collations, the default setting is **_FN**.

⁹ Default setting of **_HY** is unique for UCA0620_JAPANESE collation. For all other UCA 6.2 collations, the default setting is **_HN**.

Note

Oracle recommends that you do not use UCA 6.2 and 7.0 collations, nor the UCA1210_DUCET and UCA1210_R00T collations. See "[Avoiding ORA-12742 Error](#)" for information about the issues affecting these collations.

[Table A-20](#) lists the available emulated binary collations. The collations are named using the pattern `BIN_<charset>`, where `<charset>` represents the name of the base character set. The emulated binary collations preserve the binary ordering of their corresponding character sets.

Table A-20 Emulated Binary Collations

Collation Name	Description
BIN_IBM37	Binary collation for character set IBM37 (Western European)
BIN_IBM273	Binary collation for character set IBM273 (German)
BIN_IBM277	Binary collation for character set IBM277 (Danish and Norwegian)
BIN_IBM278	Binary collation for character set IBM278 (Finnish and Swedish)
BIN_IBM280	Binary collation for character set IBM280 (Italian)
BIN_IBM284	Binary collation for character set IBM284 (Spanish)
BIN_IBM285	Binary collation for character set IBM285 (United Kingdom)
BIN_IBM297	Binary collation for character set IBM297 (French)
BIN_IBM420	Binary collation for character set IBM420 (Arabic)
BIN_IBM500	Binary collation for character set IBM500 (International Latin 1)
BIN_IBM875	Binary collation for character set IBM875 (Greek)
BIN_IBM1026	Binary collation for character set IBM1026 (Turkish)
BIN_IBM1140	Binary collation for character set IBM1140 (Western European + euro)
BIN_IBM1141	Binary collation for character set IBM1141 (German + euro)
BIN_IBM1142	Binary collation for character set IBM1142 (Danish and Norwegian + euro)
BIN_IBM1143	Binary collation for character set IBM1143 (Finnish and Swedish + euro)
BIN_IBM1144	Binary collation for character set IBM1144 (Italian + euro)
BIN_IBM1145	Binary collation for character set IBM1145 (Spanish + euro)
BIN_IBM1146	Binary collation for character set IBM1146 (United Kingdom + euro)
BIN_IBM1147	Binary collation for character set IBM1147 (French + euro)
BIN_IBM1148	Binary collation for character set IBM1148 (International Latin 1 + euro)

A.7 Calendar Systems

By default, most territory definitions use the Gregorian calendar system. [Table A-21](#) lists the other calendar systems supported by Oracle Database.

Table A-21 Supported Calendar Systems

Name	Default Date Format	Character Set Used For Default Date Format
Japanese Imperial	EEYYMMDD	JA16EUC
ROC Official	EEyymmdd	ZHT32EUC
Thai Buddha	dd month EE yyyy	TH8TISASCII
Persian	DD Month YYYY	AR8ASMO8X
Arabic Hijrah	DD Month YYYY	AR8ISO8859P6
English Hijrah	DD Month YYYY	US7ASCII
Ethiopian	Month DD YYYY	AL32UTF8

The Arabic Hijrah and English Hijrah calendars implemented in the Oracle Database are a variant of the tabular Islamic calendar in which the leap years are the 2nd, 5th, 7th, 10th, 13th, 16th, 18th, 21st, 24th, 26th, and 29th in the 30-years cycle and in which the 1st of Muharram 1 AH corresponds to the 16th of July 622 AD. Users can apply deviation days to modify the calendar to suit their requirements, for example, by following an alternative set of leap years. See "[Customizing Calendars with the NLS Calendar Utility](#)" for more details about defining deviation days. The only difference between Arabic Hijrah and English Hijrah calendars are month names, which are written, correspondingly, in Arabic and in English transliteration.

The following example shows how July 11, 2019, appears in Japanese Imperial.

Figure A-1 Japanese Imperial Example

```
SQL> alter session set NLS_CALENDAR = 'Japanese Imperial';

Session altered.

SQL> select sysdate from dual;

SYSDATE
-----
令和01年07月11日
```

A.8 Time Zone Region Names

[Table A-22](#) shows the time zone region names in the time zone files for version 11 that are supplied with the Oracle Database. See "[Datetime Data Types and Time Zone Support](#)" for more information regarding time zone files.

You can see the time zone region names by issuing the following statement:

```
SELECT DISTINCT(TZNAME) FROM V$TIMEZONE_NAMES;
```

Table A-22 Time Zone Region Names

Time Zone Name	In the Smaller Time Zone File?	Time Zone Name	In the Smaller Time Zone File?
Africa/Abidjan	No	Asia/Qatar	No
Africa/Accra	No	Asia/Qyzylorda	No
Africa/Addis_Ababa	No	Asia/Rangoon	No
Africa/Algiers	No	Asia/Riyadh	Yes
Africa/Asmara	No	Asia/Saigon	No
Africa/Asmera	No	Asia/Sakhalin	No
Africa/Bamako	No	Asia/Samarkand	No
Africa/Bangui	No	Asia/Seoul	Yes
Africa/Banjul	No	Asia/Shanghai	Yes
Africa/Bissau	No	Asia/Singapore	Yes
Africa/Blantyre	No	Asia/Taipei	Yes
Africa/Brazzaville	No	Asia/Tashkent	No
Africa/Bujumbura	No	Asia/Tbilisi	No
Africa/Cairo	Yes	Asia/Tehran	Yes
Africa/Casablanca	No	Asia/Tel_Aviv	Yes
Africa/Ceuta	No	Asia/Thimbu	No
Africa/Conakry	No	Asia/Thimphu	No
Africa/Dakar	No	Asia/Tokyo	Yes
Africa/Dar_es_Salaam	No	Asia/Ujung_Pandang	No
Africa/Djibouti	No	Asia/Ulaanbaator	No
Africa/Doula	No	Asia/Ulan_Bator	No
Africa/El_Aaiun	No	Asia/Urumqi	No
Africa/Freetown	No	Asia/Vientiane	No
Africa/Gaborone	No	Asia/Vladivostok	No
Africa/Harare	No	Asia/Yakutsk	No
Africa/Johannesburg	No	Asia/Yetaterinburg	No
Africa/Kampala	No	Asia/Yerevan	No
Africa/Khartoum	No	Atlantic/Azores	No
Africa/Kigali	No	Atlantic/Bermuda	No
Africa/Kinshasa	No	Atlantic/Canary	No
Africa/Lagos	No	Atlantic/Cape_Verde	No
Africa/Libreville	No	Atlantic/Faeroe	No
Africa/Lome	No	Atlantic/Faroe	No
Africa/Luanda	No	Atlantic/Jan_Mayen	No
Africa/Lubumbashi	No	Atlantic/Madeira	No
Africa/Lusaka	No	Atlantic/Reykjavik	Yes
Africa/Malabo	No	Atlantic/South_Georgia	No
Africa/Maputo	No	Atlantic/St_Helena	No

Table A-22 (Cont.) Time Zone Region Names

Time Zone Name	In the Smaller Time Zone File?	Time Zone Name	In the Smaller Time Zone File?
Africa/Maseru	No	Atlantic/Stanley	No
Africa/Mbabane	No	Australia/ACT	Yes
Africa/Mogadishu	No	Australia/Adelaide	Yes
Africa/Monrovia	No	Australia/Brisbane	Yes
Africa/Nairobi	No	Australia/Broken_Hill	Yes
Africa/Ndjamena	No	Australia/Canberra	Yes
Africa/Niamey	No	Australia/Currie	No
Africa/Nouakchott	No	Australia/Darwin	Yes
Africa/Ouagadougou	No	Australia/Eucla	No
Africa/Porto-Novo	No	Australia/Hobart	Yes
Africa/Sao_Tome	No	Australia/LHI	Yes
Africa/Timbuktu	No	Australia/Lindeman	Yes
Africa/Tripoli	Yes	Australia/Lord_Howe	Yes
Africa/Tunis	No	Australia/Melbourne	Yes
Africa/Windhoek	No	Australia/NSW	Yes
America/Adak	Yes	Australia/North	Yes
America/Anchorage	Yes	Australia/Perth	Yes
America/Anguilla	No	Australia/Queensland	Yes
America/Antigua	No	Australia/South	Yes
America/Araguaina	No	Australia/Sydney	Yes
America/Argentina/ Buenos_Aires	No	Australia/Tasmania	Yes
America/Argentina/ Catamarca	No	Australia/Victoria	Yes
America/Argentina/ ComodRivadavia	No	Australia/West	Yes
America/Argentina/Cordoba	No	Australia/Yancowinna	Yes
America/Argentina/Jujuy	No	Brazil/Acre	Yes
America/Argentina/La_Rioja	Yes	Brazil/DeNoronha	Yes
America/Argentina/Mendoza	No	Brazil/East	Yes
America/Argentina/ Rio_Gallegos	Yes	Brazil/West	Yes
America/Argentina/Salta	No	CET	Yes
America/Argentina/San_Juan	Yes	CST	Yes
America/Argentina/San_Luis	No	CST6CDT	Yes
America/Argentina/Tucuman	Yes	Canada/Atlantic	Yes
America/Argentina/Ushuaia	Yes	Canada/Central	Yes
America/Aruba	No	Canada/East-Saskatchewan	Yes
America/Asuncion	No	Canada/Eastern	Yes

Table A-22 (Cont.) Time Zone Region Names

Time Zone Name	In the Smaller Time Zone File?	Time Zone Name	In the Smaller Time Zone File?
America/Atikokan	No	Canada/Mountain	Yes
America/Atka	Yes	Canada/Newfoundland	Yes
America/Bahia	No	Canada/Pacific	Yes
America/Barbados	No	Canada/Saskatchewan	Yes
America/Belem	No	Canada/Yukon	Yes
America/Belize	No	Chile/Continental	Yes
America/Blanc-Sablon	No	Chile/EasterIsland	Yes
America/Boa_Vista	No	Cuba	Yes
America/Bogota	No	EET	Yes
America/Boise	No	EST	Yes
America/Buenos_Aires	No	EST5EDT	Yes
America/Cambridge_Bay	No	Egypt	Yes
America/Campo_Grande	No	Eire	Yes
America/Cancun	No	Etc/GMT	Yes
America/Caracas	No	Etc/GMT+0	Yes
America/Catamarca	No	Etc/GMT+1	Yes
America/Cayenne	No	Etc/GMT+10	Yes
America/Cayman	No	Etc/GMT+11	Yes
America/Chicago	Yes	Etc/GMT+12	Yes
America/Chihuahua	No	Etc/GMT+2	Yes
America/Coral_Harbour	No	Etc/GMT+3	Yes
America/Cordoba	No	Etc/GMT+4	Yes
America/Costa_Rica	No	Etc/GMT+5	Yes
America/Cuiaba	No	Etc/GMT+6	Yes
America/Curacao	No	Etc/GMT+7	Yes
America/Danmarkshavn	No	Etc/GMT+8	Yes
America/Dawson	No	Etc/GMT+9	Yes
America/Dawson_Creek	No	Etc/GMT-0	Yes
America/Denver	Yes	Etc/GMT-1	Yes
America/Detroit	Yes	Etc/GMT-10	Yes
America/Dominica	No	Etc/GMT-11	Yes
America/Edmonton	Yes	Etc/GMT-12	Yes
America/Eirunepe	Yes	Etc/GMT-13	Yes
America/El_Salvador	No	Etc/GMT-14	Yes
America/Ensenada	Yes	Etc/GMT-2	Yes
America/Fort_Wayne	Yes	Etc/GMT-3	Yes
America/Fortaleza	No	Etc/GMT-4	Yes
America/Glace_Bay	No	Etc/GMT-5	Yes

Table A-22 (Cont.) Time Zone Region Names

Time Zone Name	In the Smaller Time Zone File?	Time Zone Name	In the Smaller Time Zone File?
America/Godthab	No	Etc/GMT-6	yes
America/Goose_Bay	No	Etc/GMT-7	Yes
America/Grand_Turk	No	Etc/GMT-8	Yes
America/Grenada	No	Etc/GMT-9	Yes
America/Guadeloupe	No	Etc/GMT0	Yes
America/Guatemala	No	Etc/Greenwich	Yes
America/Guayaquil	No	Europe/Amsterdam	No
America/Guyana	No	-	-
America/Halifax	Yes	Europe/Andorra	No
America/Havana	Yes	Europe/Athens	No
America/Hermosillo	No	Europe/Belfast	Yes
America/Indiana/Indianapolis	Yes	Europe/Belgrade	No
America/Indiana/Knox	No	Europe/Berlin	No
America/Indiana/Marengo	No	Europe/Bratislava	No
America/Indiana/Petersburg	No	Europe/Brussels	No
America/Indiana/Tell_City	No	Europe/Bucharest	No
America/Indiana/Vevay	No	Europe/Budapest	No
America/Indiana/Vincennes	No	Europe/Chisinau	No
America/Indiana/Winamac	No	Europe/Copenhagen	No
America/Indianapolis	Yes	Europe/Dublin	Yes
America/Inuvik	No	Europe/Gibraltar	No
America/Iqaluit	No	Europe/Guernsey	Yes
America/Jamaica	Yes	Europe/Helsinki	No
America/Jujuy	No	Europe/Isle_of_Man	Yes
America/Juneau	No	Europe/Istanbul	Yes
America/Kentucky/Louisville	No	Europe/Jersey	Yes
America/Kentucky/Monticello	No	Europe/Kaliningrad	No
America/Knox_IN	No	Europe/Kiev	No
America/La_Paz	No	Europe/Lisbon	Yes
America/Lima	No	Europe/Ljubljana	No
America/Los_Angeles	Yes	Europe/London	Yes
America/Louisville	No	Europe/Luxembourg	No
America/Maceio	No	Europe/Madrid	No
America/Managua	No	Europe/Malta	No
America/Manaus	Yes	Europe/Mariehamn	No
America/Marigot	No	Europe/Minsk	No
America/Martinique	No	Europe/Monaco	No
America/Mazatlan	Yes	Europe/Moscow	Yes

Table A-22 (Cont.) Time Zone Region Names

Time Zone Name	In the Smaller Time Zone File?	Time Zone Name	In the Smaller Time Zone File?
America/Mendoza	No	Europe/Nicosia	No
America/Menominee	No	Europe/Oslo	No
America/Merida	No	Europe/Paris	No
America/Mexico_City	Yes	Europe/Podgorica	No
America/Miquelon	No	Europe/Prague	No
America/Moncton	No	Europe/Riga	No
America/Monterrey	Yes	Europe/Rome	No
America/Montevidео	No	Europe/Samara	No
America/Montreal	Yes	Europe/San_Marino	No
America/Montserrat	No	Europe/Sarajevo	No
America/Nassau	No	Europe/Simferopol	No
America/New_York	Yes	Europe/Skopje	No
America/Nipigon	No	Europe/Sofia	No
America/Nome	No	Europe/Stockholm	No
America/Noronha	Yes	Europe/Tallinn	No
America/North_Dakota/ Center	No	Europe/Tirane	No
America/North_Dakota/ New_Salem	No	Europe/Tiraspol	No
America/Panama	No	Europe/Uzhgorod	No
America/Pangnirtung	No	Europe/Vaduz	No
America/Paramaribo	No	Europe/Vatican	No
America/Phoenix	Yes	Europe/Vienna	No
America/Port-au-Prince	No	Europe/Vilnius	No
America/Port_of_Spain	No	Europe/Volgograd	No
America/Porto_Acre	No	Europe/Warsaw	Yes
America/Porto_Velho	No	Europe/Zagreb	No
America/Port_of_Spain	No	Europe/Zaporozhye	No
America/Porto_Acre	No	Europe/Zurich	No
America/Porto_Velho	No	GB	Yes
America/Puerto_Rico	No	GB-Eire	Yes
America/Rainy_River	No	GMT	Yes
America/Rankin_Inlet	No	GMT+0	Yes
America/Recife	No	GMT-0	Yes
America/Regina	Yes	GMT0	Yes
America/Resolute	No	Greenwich	Yes
America/Rio_Branco	Yes	HST	Yes
America/Rosario	No	Hongkong	Yes
America/Santiago	Yes	Iceland	Yes

Table A-22 (Cont.) Time Zone Region Names

Time Zone Name	In the Smaller Time Zone File?	Time Zone Name	In the Smaller Time Zone File?
America/Santo_Domingo	No	Indian/Antananarivo	No
America/Sao_Paulo	Yes	Indian/Chagos	No
America/Scoresbysund	No	Indian/Christmas	No
America/Shiprock	Yes	Indian/Cocos	No
America/St_Barthelemy	No	Indian/Comoro	No
America/St_Johns	Yes	Indian/Kerguelen	No
America/St_Kitts	No	Indian/Mahe	No
America/St_Lucia	No	Indian/Maldives	No
America/St_Thomas	No	Indian/Mauritius	No
America/St_Vincent	No	Indian/Mayotte	No
America/Swift_Current	No	Indian/Reunion	No
America/Tegucigalpa	No	Iran	Yes
America/Thule	No	Israel	Yes
America/Thunder_Bay	No	Jamaica	Yes
America/Tijuana	Yes	Japan	Yes
America/Tortola	No	Kwajalein	Yes
America/Vancouver	Yes	Libya	Yes
America/Virgin	No	MET	Yes
America/Whitehorse	Yes	MST	Yes
America/Winnipeg	Yes	MST7MDT	Yes
America/Yakutat	No	Mexico/BajaNorte	Yes
America/Yellowknife	No	Mexico/BajaSur	Yes
Antarctica/Casey	No	Mexico/General	Yes
Antarctica/Davis	No	NZ	Yes
Antarctica/DumontDURville	No	NZ-CHAT	Yes
Antarctica/Mawson	No	Navajo	Yes
Antarctica/McMurdo	No	PRC	Yes
Antarctica/Palmer	No	PST	Yes
Antarctica/South_Pole	No	PST8PDT	Yes
Antarctica/Syowa	No	Pacific/Apia	No
Arctic/Longyearbyen	No	Pacific/Auckland	Yes
Asia/Aden	No	Pacific/Chatham	Yes
Asia/Almaty	No	Pacific/Easter	Yes
Asia/Amman	No	Pacific/Efate	No
Asia/Anadyr	No	Pacific/Enderbury	No
Asia/Aqtau	No	Pacific/Fakaofo	No
Asia/Aqtobe	No	Pacific/Fiji	No
Asia/Ashgabat	No	Pacific/Funafuji	No

Table A-22 (Cont.) Time Zone Region Names

Time Zone Name	In the Smaller Time Zone File?	Time Zone Name	In the Smaller Time Zone File?
Asia/Ashkhabad	No	Pacific/Galapagos	No
Asia/Baghdad	No	Pacific/Gambier	No
Asia/Bahrain	No	Pacific/Guadalcanal	No
Asia/Baku	No	Pacific/Guam	No
Asia/Bangkok	No	Pacific/Honolulu	Yes
Asia/Beirut	No	Pacific/Johnston	No
Asia/Bishkek	No	Pacific/Kiritimati	No
Asia/Brunei	No	Pacific/Kosrae	No
Asia/Calcutta	Yes	Pacific/Kwajalein	Yes
Asia/Choibalsan	No	Pacific/Majuro	No
Asia/Chongqing	No	Pacific/Marquesas	No
Asia/Chungking	No	Pacific/Midway	No
Asia/Colombo	No	Pacific/Nauru	No
Asia/Dacca	No	Pacific/Niue	No
Asia/Damascus	No	Pacific/Norfolk	No
Asia/Dhaka	No	Pacific/Noumea	No
Asia/Dili	No	Pacific/Pago_Pago	Yes
Asia/Dubai	No	Pacific/Palau	No
Asia/Dushanbe	No	Pacific/Pitcairn	No
Asia/Gaza	No	Pacific/Ponape	No
Asia/Harbin	No	Pacific/Rarotonga	No
Asia/Ho_Chi_Minh	No	Pacific/Rarotonga	No
Asia/Hong_Kong	Yes	Pacific/Saipan	No
Asia/Hovd	No	Pacific/Samoa	Yes
Asia/Irkutsk	No	Pacific/Tahiti	No
Asia/Istanbul	Yes	Pacific/Tarawa	No
Asia/Jakarta	No	Pacific/Tongatapu	No
Asia/Jayapura	No	Pacific/Truk	No
Asia/Jerusalem	Yes	Pacific/Wake	No
Asia/Kabul	No	Pacific/Wallis	No
Asia/Kamchatka	No	Pacific/Yap	No
Asia/Karachi	No	Poland	Yes
Asia/Kashgar	No	Portugal	Yes
Asia/Kathmandu	No	ROC	Yes
Asia/Katmandu	No	ROK	Yes
Asia/Kolkata	No	Singapore	Yes
Asia/Krasnoyarsk	No	Turkey	Yes
Asia/Kuala_Lumpur	No	US/Alaska	Yes

Table A-22 (Cont.) Time Zone Region Names

Time Zone Name	In the Smaller Time Zone File?	Time Zone Name	In the Smaller Time Zone File?
Asia/Kuching	No	US/Aleutian	Yes
Asia/Kuwait	No	US/Arizona	Yes
Asia/Macao	No	US/Central	Yes
Asia/Macau	No	US/East-Indiana	Yes
Asia/Magadan	No	US/Eastern	Yes
Asia/Makassar	No	US/Hawaii	Yes
Asia/Manila	No	US/Indiana-Starke	No
Asia/Muscat	No	US/Michigan	Yes
Asia/Nicosia	No	US/Mountain	Yes
Asia/Novosibirsk	No	US/Pacific	Yes
Asia/Omsk	No	US/Pacific-New	Yes
Asia/Oral	No	US/Samoa	Yes
Asia/Phnom_Penh	No	UTC	No
Asia/Pontianak	No	W-SU	Yes
Asia/Pyongyang	No	WET	Yes

See Also["Choosing a Time Zone File"](#)

A.9 Obsolete Locale Data

This section contains information about obsolete linguistic sorts, character sets, languages, and territories. The obsolete linguistic sort, language, and territory definitions are still available. However, they are supported for backward compatibility only; they may be desupported in a future release. You can obtain a listing of the obsolete character sets, languages, territories, and linguistic sorts for the current database release by querying the V\$NLS_VALID_VALUES view.

A.9.1 Obsolete Linguistic Sorts

[Table A-23](#) contains linguistic sorts that have been obsoleted starting with Oracle Database 10g.

Table A-23 Obsolete Linguistic Sorts

Obsolete Sort Name	Replacement Sort
THAI_TELEPHONE	THAI_M
THAI_DICTIONARY	THAI_M
CANADIAN FRENCH	CANADIAN_M
JAPANESE	JAPANESE_M

A.9.2 Obsolete Territories

[Table A-24](#) contains territories that have been obsoleted starting with Oracle Database 10g.

Table A-24 Obsolete Territories

Obsolete Territory Name	Replacement Territory
CIS	RUSSIA
MACEDONIA	FYR MACEDONIA
YUGOSLAVIA	BOSNIA AND HERZEGOVINA, SERBIA, or MONTENEGRO
SERBIA AND MONTENEGRO	SERBIA or MONTENEGRO
CZECHOSLOVAKIA	CZECH REPUBLIC or SLOVAKIA

A.9.3 Obsolete Languages

[Table A-25](#) contains languages that have been obsoleted starting with Oracle Database 10g.

Table A-25 Obsolete Languages

Obsolete Language Name	Replacement Language
BENGALI	BANGLA

A.9.4 Obsolete Character Sets and Replacement Character Sets

[Table A-26](#) lists the obsolete character sets. If you reference any of these character sets in your code, then replace them with the new character set.

Table A-26 Obsolete Character Sets and Their Replacements

Obsolete Character Set	Replacement Character Set
AR8ADOS710T	AR8ISO8859P6, AR8MSWIN1256, and AL32UTF8
AR8ADOS720T	AR8ISO8859P6, AR8MSWIN1256, and AL32UTF8
AR8APTEC715T	AR8ISO8859P6, AR8MSWIN1256, and AL32UTF8
AR8ASMO708PLUS	AR8ISO8859P6, AR8MSWIN1256, and AL32UTF8
AR8HPARABIC8T	AR8ISO8859P6, AR8MSWIN1256, and AL32UTF8
AR8MSAWIN	AR8MSWIN1256
AR8MUSSAD768T	AR8ISO8859P6, AR8MSWIN1256, and AL32UTF8
AR8NAFITHA711T	AR8ISO8859P6, AR8MSWIN1256, and AL32UTF8
AR8NAFITHA721T	AR8ISO8859P6, AR8MSWIN1256, and AL32UTF8
AR8SAKHR707T	AR8ISO8859P6, AR8MSWIN1256, and AL32UTF8
AR8XBASIC	AR8EBCDIC420S
CL8EBCDIC875S	CL8EBCDIC875R
CL8MSWINDOW31	CL8MSWIN1251
EL8EBCDIC875S	EL8EBCDIC875R

Table A-26 (Cont.) Obsolete Character Sets and Their Replacements

Obsolete Character Set	Replacement Character Set
JA16DBCSFIXED	AL16UTF16 and UTF8
JA16EUCFIXED	AL16UTF16 and UTF8 See the note following this table
JA16SJISFIXED	AL16UTF16 and UTF8 See the note following this table
KO16DBCSFIXED	AL16UTF16 and UTF8
KO16KSC5601FIXED	AL16UTF16 and UTF8
ZHS16CGB231280FIXED	AL16UTF16 and UTF8
ZHS16DBCSFIXED	AL16UTF16 and UTF8
ZHS16GBKFIXED	AL16UTF16 and UTF8
ZHS32EUCFIXED	AL16UTF16 and UTF8
ZHT16BIG5FIXED	AL16UTF16 and UTF8
ZHT16DBCSFIXED	AL16UTF16 and UTF8
ZHT16DBT	AL32UTF8
ZHT32TRISFIXED	AL16UTF16 and UTF8

Note

The character sets JA16EUCFIXED (1830) and JA16SJISFIXED (1832) are supported on the database client side using:

- NLS_NCHAR client environment variable
- *ncharset* parameter of the `OCIEnvNlsCreate()` call
- `OCI_ATTR_CHARSET_ID` attribute of a bind or a define handle

A.9.5 Updates to the Oracle Database Language and Territory Definition Files

Changes have been made to the content in some of the language and territory definition files since Oracle Database 10g. These updates are necessary to correct the legacy definitions that no longer meet the local conventions in some of the languages and territories that Oracle Database supports. These changes include modifications to the currency symbols, month names, and group separators. One example is the local currency symbol for Brazil. This was updated from Cr\$ to R\$ in Oracle Database 10g.

Please refer to the "Oracle Database Language and Territory Definition Changes" table documented in the `$ORACLE_HOME/nls/data/old/data_changes.html` file for a detailed list of the changes.

You should review your existing application code to make sure that the latest locale definition files that are included in this Oracle Database release are being used. If you are not able to make locale-specific code changes to support your applications, then you may use the Oracle9i locale definition files that are included in this Oracle Database release.

To revert back to the Oracle9i language and territory behavior:

1. Shut down the database.
2. Run the script `cr9idata.pl` from the `$ORACLE_HOME/nls/data/old` directory.
3. Set the `ORA_NLS10` environment variable to the newly created `$ORACLE_HOME/nls/data/9idata` directory.
4. Restart the database.

Steps 2 and 3 will need to be repeated for all the Oracle Database clients that need to revert back to the Oracle9i definition files.

Note

Oracle strongly recommends that you use the latest locale definition files included in this Oracle Database release. Oracle9i locale definition files will be desupported in a future release.

A.10 Desupported Locale Data

This section contains information about desupported linguistic sorts and character sets. Oracle will no longer fix bugs related to these features and Oracle can choose to remove the code required to use these features.

A.10.1 Desupported Linguistic Sorts

[Table A-27](#) contains the UCA 6.1 collations that have been desupported starting with Oracle Database 21c. Oracle recommends to migrate schema objects, such as linguistic indexes and data-bound collations created using the UCA 6.1 collations, to the UCA 12.1 collations. UCA 12.1 has incorporated all enhancements and upgrades on UCA since version 6.1. It also has proper weight assignment for all new characters introduced to the Unicode standard since Unicode 6.1.

Table A-27 Desupported UCA 6.1 Collations

Desupported Sort Name	Replacement Sort
UCA0610_ROOT	UCA1210_ROOT
UCA0610_DUCET	UCA1210_DUCET
UCA0610_SPANISH	UCA1210_SPANISH
UCA0610_TSPANISH	UCA1210_TSPANISH
UCA0610_CFRENCH	UCA1210_CFRENCH
UCA0610_DANISH	UCA1210_DANISH
UCA0610_THAI	UCA1210_THAI
UCA0610_JAPANESE	UCA1210_JAPANESE
UCA0610_KOREAN	UCA1210_KOREAN
UCA0610_SCHINESE	UCA1210_SCHINESE
UCA0610_SCHINESE1	UCA1210_SCHINESE1

Table A-27 (Cont.) Desupported UCA 6.1 Collations

Desupported Sort Name	Replacement Sort
UCA0610_SCHINESE2	UCA1210_SCHINESE2
UCA0610_TCHINESE	UCA1210_TCHINESE
UCA0610_TCHINESE1	UCA1210_TCHINESE1

A.10.2 Desupported Character Sets and Replacement Character Sets

[Table A-28](#) lists the desupported character sets. If you reference any of these character sets in your code, then replace them with the new character set.

Table A-28 Desupported Character Sets and Their Replacements

Desupported Character Set	Replacement Character Set
CGB2312-80	ZHS16CGB231280
CNS 11643-86	ZHT32EUC
JDBCS	JA16DBCS
JEUC	JA16EUC
JVMS	JA16VMS
KDBCS	KO16DBCS
KSC5601	KO16KSC5601
SJIS	JA16SJIS

A.10.3 AL24UTFFSS Character Set Desupported

The Unicode character set AL24UTFFSS was introduced in Oracle Database version 7 to support the UTF-8 encoding scheme and was based on the Unicode standard 1.1. AL24UTFFSS was desupported in Oracle9i. Oracle Database began offering the Unicode database character set UTF8 in Oracle8 and AL32UTF8 in Oracle9i. The AL32UTF8 character set has been updated to conform to Unicode 7.0 in Oracle Database 12c Release 2 (12.2), Unicode 9.0 in Oracle Database 18c and Oracle Database 19c, Unicode 12.1 in Oracle Database 21c, Unicode 15.0 in Oracle AI Database 26ai (RU 23.26.2 and earlier), and Unicode 17.0 in Oracle AI Database 26ai (RU 23.26.3 and later).

The migration path for an existing AL24UTFFSS database is to upgrade to UTF8 prior to upgrading to Oracle Database 9i or later. You can use the Character Set Scanner for data analysis in Oracle8 before attempting to migrate your existing database character set to UTF8.

B

Unicode Character Code Assignments

This appendix provides an introduction to Unicode character assignments. This appendix contains the following topics:

- [Unicode Code Ranges](#)
- [UTF-16 Encoding](#)
- [UTF-8 Encoding](#)

B.1 Unicode Code Ranges

[Table B-1](#) contains code ranges that have been allocated in Unicode for UTF-16 character codes.

Table B-1 Unicode Character Code Ranges for UTF-16 Character Codes

Types of Characters	First 16 Bits	Second 16 Bits
ASCII	0000-007F	-
European (except ASCII), Arabic, Hebrew	0080-07FF	-
Indic, Thai, certain symbols (such as the euro symbol), Chinese, Japanese, Korean	0800-0FFF 1000 - CFFF D000 - D7FF F900 - FFFF	-
Private Use Area #1	E000 - EFFF F000 - F8FF	-
Supplementary characters: Additional Chinese, Japanese, and Korean characters; historic characters; musical symbols; mathematical symbols	D800 - D8BF D8C0 - DABF DAC0 - DB7F	DC00 - DFFF DC00 - DFFF DC00 - DFFF
Private Use Area #2	DB80 - DBBF DBC0 - DBFF	DC00 - DFFF DC00 - DFFF

[Table B-2](#) contains code ranges that have been allocated in Unicode for UTF-8 character codes.

Table B-2 Unicode Character Code Ranges for UTF-8 Character Codes

Types of Characters	First Byte	Second Byte	Third Byte	Fourth Byte
ASCII	00 - 7F	-	-	-
European (except ASCII), Arabic, Hebrew	C2 - DF	80 - BF	-	-
Indic, Thai, certain symbols (such as the euro symbol), Chinese, Japanese, Korean	E0 E1 - EC ED EF	A0 - BF 80 - BF 80 - 9F A4 - BF	80 - BF 80 - BF 80 - BF 80 - BF	-

Table B-2 (Cont.) Unicode Character Code Ranges for UTF-8 Character Codes

Types of Characters	First Byte	Second Byte	Third Byte	Fourth Byte
Private Use Area #1	EE	80 - BF	80 - BF	-
	EF	80 - A3	80 - BF	
Supplementary characters: Additional Chinese, Japanese, and Korean characters; historic characters; musical symbols; mathematical symbols	F0	90 - BF	80 - BF	80 - BF
	F1 - F2	80 - BF	80 - BF	80 - BF
	F3	80 - AF	80 - BF	80 - BF
Private Use Area #2	F3	B0 - BF	80 - BF	80 - BF
	F4	80 - 8F	80 - BF	80 - BF

Note

Blank spaces represent nonapplicable code assignments. Character codes are shown in hexadecimal representation.

B.2 UTF-16 Encoding

As shown in [Table B-1](#), UTF-16 character codes for some characters (Additional Chinese/Japanese/Korean characters and Private Use Area #2) are represented in two units of 16-bits. These are supplementary characters. A supplementary character consists of two 16-bit values. The first 16-bit value is encoded in the range from 0xD800 to 0xDBFF. The second 16-bit value is encoded in the range from 0xDC00 to 0xDFFF. With supplementary characters, UTF-16 character codes can represent more than one million characters. Without supplementary characters, only 65,536 characters can be represented. The AL16UTF16 character set in Oracle Database supports supplementary characters.

See Also

["Code Points and Supplementary Characters"](#)

B.3 UTF-8 Encoding

The UTF-8 character codes in [Table B-2](#) show that the following conditions are true:

- ASCII characters use 1 byte
- European (except ASCII), Arabic, and Hebrew characters require 2 bytes
- Indic, Thai, Chinese, Japanese, and Korean characters as well as certain symbols such as the euro symbol require 3 bytes
- Characters in the Private Use Area #1 require 3 bytes
- Supplementary characters require 4 bytes
- Characters in the Private Use Area #2 require 4 bytes

In Oracle Database, the AL32UTF8 character set supports 1-byte, 2-byte, 3-byte, and 4-byte values. In Oracle Database, the UTF8 character set supports 1-byte, 2-byte, and 3-byte values, but not 4-byte values.

C

Collation Derivation and Determination Rules for SQL Operations

This appendix describes collation derivation and determination rules for SQL operations. This appendix contains the following topics:

- [Collation Derivation](#)
- [Collation Determination](#)
- [SQL Operations and Their Derivation- and Determination-relevant Arguments](#)

C.1 Collation Derivation

The process of determining the collation of a character result of an SQL operation is called collation derivation. Such operation may be an operator, column reference, character literal, bind variable reference, function call, CASE expression, or a query clause.

Each character value in an SQL expression has a *derived collation* and a *derived coercibility level*.

The *derived collation* and *coercibility level* of the basic expressions is described in the following table.

Table C-1 Derived Collation and Derived Coercibility Level of Various Expression Types

Type of Expression	Derived Collation	Derived Coercibility Level
Result of the COLLATE operator	The named collation or the pseudo-collation specified in the COLLATE operator	0
Data container reference such as table, view, or materialized view column reference	The declared named collation or the pseudo-collation of the data container	2
Result of a PL/SQL function call or a user-defined operator	USING_NLS_COMP collation	2
Character literal	USING_NLS_COMP collation, if included in a top-level statement; else default collation of a view, a materialized view, or a PL/SQL unit, if included in its source	4
Character bind variable reference when the OCI_ATTR_COLLATION_ID attribute is <i>not set</i> on the corresponding bind variable handle	USING_NLS_COMP collation	4
Character bind variable reference when the OCI_ATTR_COLLATION_ID attribute is <i>set</i> on the corresponding bind variable handle	Collation with ID passed as the attribute value	0

Note

- Coercibility level 1 corresponds to no collation assigned
- Coercibility level 3 is reserved for future use

The derived collation and coercibility level of an operation's result is based on the collations and coercibility levels of the operation's arguments. A *derivation-relevant* character argument of an operation is an argument used to derive the collation of the operator's result. An operator may have zero or more derivation-relevant character arguments, and zero or more other character arguments, such as flags or other control information not directly interacting with the derivation-relevant arguments. An argument is considered *derivation-relevant*, if its value is included in the result, either after some transformation or without undergoing any transformation.

An argument that is a format model, a pattern, a flag string, or a key into a virtual table of system information is not considered a derivation-relevant argument. For example, the built-in function `T0_CHAR(arg1, arg2, arg3)` has no derivation-relevant arguments, as the main argument `arg1` is not of a character data type. The two character arguments `arg2` and `arg3` are not derivation-relevant arguments as they only define the format and parameters for the conversion of the main argument `arg1`.

The derived collation and coercibility level of the result of an operation without derivation-relevant arguments are the same as when a character literal would have been put in that expression in the place of the operation.

The following are the collation derivation rules for operations that return character values and have derivation-relevant arguments. These rules are applied recursively in an expression tree. These rules are based on the SQL standard version ISO/IEC 9075-2:1999.

The derived collation of a result of an operation with derivation-relevant character arguments *arg1, arg2, ..., argn* is:

- If at least one argument has the coercibility level 0, then all the arguments with coercibility level 0 must have the same collation, which is the derived collation of the result. The coercibility level of the result is 0. If two arguments with coercibility level 0 have different collations, then an error is reported.
- Otherwise, if at least one argument has the coercibility level 1, then the expression result has the coercibility level 1 and no collation is assigned to it.
- Otherwise, if LCL is the *numerically lowest coercibility level* of the arguments, then:
 - If all the arguments with LCL have the same collation, then this collation is the derived collation of the result, and the coercibility level of the result is LCL.
 - Otherwise, the result of the expression has the coercibility level 1 and no collation is assigned to it.

Note

Set operators have arguments that are expression lists. For set operators, collation derivation is performed separately on corresponding elements of each of the arguments of the expression list. For example, in the query:

```
SELECT expr1, expr2 FROM t1
UNION
SELECT expr3, expr4 FROM t2
```

the collation is derived separately for the first and the second column of the result set. For the first column, the collation derivation rules are applied to expr1 and expr3. For the second column, the rules are applied to expr2 and expr4.

See Also

["SQL Operations and Their Derivation- and Determination-relevant Arguments"](#)

Collation Derivation for Bind Variable References

In OCI, you can pass a collation for a bind variable in a query or a DML statement using the value of the `OCI_ATTR_COLLATION_ID` attribute. The `OCI_ATTR_COLLATION_ID` attribute can be set on a bind variable handle to any of the supported collation IDs using the `OCIAttrSet()` function. The IDs of both named collations and pseudo-collations are allowed. In this case, the derived coercibility level of the bind variable reference is 0.

When the `OCI_ATTR_COLLATION_ID` attribute value is set to `OCI_COLLATION_NONE` (the default value) on a bind variable handle, the collation of the bind variable is `USING_NLS_COMP` and the derived coercibility level of the bind variable reference is 4.

OCI does not check whether a collation is valid for a given data type of a bind variable. If the `OCI_ATTR_COLLATION_ID` attribute value is set for a non-character data type variable, it is ignored by the database server.

Collation of bind variables is currently ignored in PL/SQL expressions. For forward compatibility reasons, the `OCI_ATTR_COLLATION_ID` attribute should not be set for bind variables passed to an anonymous PL/SQL block, unless the variables are referenced exclusively in SQL statements.

See Also

Oracle Call Interface Programmer's Guide for more information about the `OCI_ATTR_COLLATION_ID` attribute.

C.2 Collation Determination

Collation determination is the process of selecting the right collation to apply during the execution of a collation-sensitive operation. A collation-sensitive operation can be an SQL operator, condition, built-in function call, CASE expression or a query clause.

For Oracle Database releases earlier to 12.2, collation to be applied by an operation is determined by only the NLS_SORT and NLS_COMP session parameters.

① Note

The optional second parameters to NLS_UPPER, NLS_LOWER, NLS_INITCAP, and NLSSORT are exceptions.

Starting from Oracle Database 12.2, collation to be applied by an operation is determined by the derived data-bound collations of its arguments. Once a pseudo-collation is determined as the collation to use, NLS_SORT and NLS_COMP session parameters are checked to provide the actual named collation to apply.

① Note

The collation determination does not have to apply to the same operation to which collation derivation applies. For example, TO_CHAR function is not collation-sensitive, so it does not need collation determination. But, TO_CHAR function returns a character result that needs a collation declaration, hence collation derivation applies to it. Conversely, INSTR function needs to match characters and needs a collation determined for this match operation. However, the result of INSTR function is a number, hence no collation derivation is required for it.

The *determination-relevant* character argument of an operation is an argument used to determine the collation to be used by the operation. A collation-sensitive operation may have one or more determination-relevant character arguments and zero or more other character arguments, such as flags or other control information not directly interacting with the determination-relevant arguments.

An argument is considered determination-relevant, if its value is compared during the evaluation of an operation. An argument that is a format model, a flag string, or a key into a virtual table of system information is not considered a determination-relevant argument. However, a pattern argument can be a determination-relevant argument. For example, two of the three arguments of the LIKE predicate – *argument* and *pattern* – are determination-relevant arguments. The third argument – *the escape character* – is not considered determination-relevant argument. Another example is the built-in function REGEXP_COUNT, which has four arguments – *source_char*, *pattern*, *position*, and *match_param*. The determination-relevant arguments are *source_char* and *pattern*, which contain the strings to be compared. The non-determination-relevant character argument are *position*, which is numeric, and *match_param*, which provides parameters for the matching operation.

The following are the collation determination rules to determine the collation to use for an operation with determination-relevant character arguments *arg1*, *arg2*, ..., *argn*. These rules are based on the SQL standard version ISO/IEC 9075-2:1999.

- If operation is the equality condition and is used to enforce a foreign key constraint, then the collation to be used is the declared collation of the primary or unique key column being referenced. This declared collation must be the same as the declared collation of the foreign key column.
- Otherwise, if at least one argument has the derived coercibility level 0, then all the arguments with coercibility level 0 must have the same collation, and this collation is used by the operation. If two arguments with coercibility level 0 have different collations, then an error is reported.
- Otherwise, if at least one argument has the derived coercibility level 1, then an error is reported.
- Otherwise, if LCL is the *numerically lowest coercibility level* of the arguments, then:
 - If all arguments with LCL have the same collation, then that collation is used by the operation.
 - Otherwise, an error is reported.

When the determined collation is a pseudo-collation, then the affected operation must refer to the session or database settings `NLS_SORT` or `NLS_COMP` or both to determine the actual named collation to apply. The database settings are used for expressions in virtual columns, `CHECK` constraints, and fine grained auditing (FGA) rules.

The collation determination rules for an operation involving a `CLOB` or an `NCLOB` data type value must result in the pseudo-collation `USING_NLS_COMP`, otherwise an error is reported.

Note

Some conditions, set operators, and query clauses have arguments which are expression lists. In this case, collation determination is performed on the corresponding compared elements of each of the arguments in the expression list. For example, in the condition:

```
(expr1, expr2) IN (SELECT expr3, expr4 FROM t1)
```

the collation is determined separately for the pairs of compared elements. First, the collation determination rules are applied to expr1 and expr3. Then, the rules are applied to expr2 and expr4. When the condition is evaluated, values of expr1 are compared to values of expr3 using the first determined collation and values of expr2 are compared to values of expr4 using the second determined collation. Similarly, in the query:

```
SELECT expr1, expr2 FROM t1
MINUS
SELECT expr3, expr4 FROM t2
```

the collation determination rules are first applied to expr1 and expr3, then to expr2 and expr4. When the MINUS operator is evaluated, values of expr1 are compared to values of expr3 using the first determined collation and values of expr2 are compared to values of expr4 using the second determined collation.

In the query:

```
SELECT * FROM t1 ORDER BY expr1, expr2, expr3
```

rows are sorted first on values of expr1 using the derived collation expr1, then ties are broken by sorting on values of expr2 using the derived collation expr2, and then on values of expr3 using the derived collation expr3. Each position in the ORDER BY list is treated like a separate comparison operator for row values.

See Also

["SQL Operations and Their Derivation- and Determination-relevant Arguments"](#)

C.3 SQL Operations and Their Derivation- and Determination-relevant Arguments

The following table lists all the SQL operations that return a character value or are collation-sensitive or both. For each operation returning a character value, the table lists operation's derivation-relevant arguments. If the operation has no such arguments, the fixed collation of the operation's result is shown instead. The term *Literal Collation* means that the collation derived for the operation's result is the collation of a character literal put in place of the operation in an expression; this is either USING_NLS_COMP for top-level SQL statements or the default collation of a view, materialized view, or a PL/SQL stored unit containing the expression

in its source. For each collation-sensitive operation, the following table lists the operation's determination-relevant arguments.

Table C-2 Derivation- and Determination-relevant Arguments for SQL Operations

Operation Type	Operation Name	Operation Description	Derivation-relevant Arguments or Fixed Collation	Determination-relevant Arguments
Pseudo-column	VERSIONS_OPERATION	Operation type in a flashback version query	Literal collation	—
Pseudo-column	COLUMN_VALUE	Value of nested table element of character data type	USING_NLS_COMP	—
Operator	$a_1 a_2$	Character Value Concatenation	a_1, a_2	—
Operator	PRIOR a_1	Hierarchical query parent value	a_1	—
Operator	CONNECT_BY_ROOT a_1	Hierarchical query root value	a_1	—
Operator	SELECT $a_{11},$ $a_{21}, \dots, a_{m1}$ FROM ... UNION ALL SELECT $a_{12},$ $a_{22}, \dots, a_{m2}$ FROM ...	Non-distinct union of two row sets	$a_{11}, a_{12}, a_{21},$ $a_{22}, \dots, a_{m1}, a_{m2}$ Collation for each column of the resulting row set is derived separately by combining collations of columns from each of the two argument row sets. <i>Special case:</i> if an argument a_{i2} ($1 \leq i \leq m$) belongs to a recursive member in a WITH clause and it is calculated recursively, then the collation is derived from the corresponding argument a_{i1} of the anchor member.	—

Table C-2 (Cont.) Derivation- and Determination-relevant Arguments for SQL Operations

Operation Type	Operation Name	Operation Description	Derivation-relevant Arguments or Fixed Collation	Determination-relevant Arguments
Operator	SELECT $a_{11},$ $a_{21}, \dots a_{m1}$ FROM ...	Distinct union of two row sets	$a_{11}, a_{12}, a_{21},$ $a_{22}, \dots a_{m1}, a_{m2}$	$a_{11}, a_{12}, a_{21},$ $a_{22}, \dots a_{m1}, a_{m2}$
	UNION		Collation for each column of the resulting row set is derived separately by combining collations of columns from each of the two argument row sets.	Collation for comparison of each column of the argument row set is determined separately by combining collations of columns from each of the two argument row sets.
	SELECT $a_{12},$ $a_{22}, \dots a_{m2}$ FROM ...			
Operator	SELECT $a_{11},$ $a_{21}, \dots a_{m1}$ FROM ...	Distinct intersection of two row sets	$a_{11}, a_{12}, a_{21},$ $a_{22}, \dots a_{m1}, a_{m2}$	$a_{11}, a_{12}, a_{21},$ $a_{22}, \dots a_{m1}, a_{m2}$
	INTERSECT		Collation for each column of the resulting row set is derived separately by combining collations of columns from each of the two argument row sets.	Collation for comparison of each column of the argument row set is determined separately by combining collations of columns from each of the two argument row sets.
	SELECT $a_{12},$ $a_{22}, \dots a_{m2}$ FROM ...			
Operator	SELECT $a_{11},$ $a_{21}, \dots a_{m1}$ FROM ...	Distinct subtraction of row sets	$a_{11}, a_{12}, a_{21},$ $a_{22}, \dots a_{m1}, a_{m2}$	$a_{11}, a_{12}, a_{21},$ $a_{22}, \dots a_{m1}, a_{m2}$
	MINUS		Collation for each column of the resulting row set is derived separately by combining collations of columns from each of the two argument row sets.	Collation for comparison of each column of the argument row set is determined separately by combining collations of columns from each of the two argument row sets.
	SELECT $a_{12},$ $a_{22}, \dots a_{m2}$ FROM ...			
Expression	CASE WHEN c_1 THEN r_1 WHEN c_2 THEN r_2 ... WHEN c_n THEN r_n ELSE r_{n+1} END	Searched case expression	$r_1, r_2, \dots r_n, r_{n+1}$	Each condition $c_1, \dots c_n$ has independent collation determination.

Table C-2 (Cont.) Derivation- and Determination-relevant Arguments for SQL Operations

Operation Type	Operation Name	Operation Description	Derivation-relevant Arguments or Fixed Collation	Determination-relevant Arguments
Expression	CASE v WHEN s ₁ THEN r ₁ WHEN s ₂ THEN r ₂ ... WHEN s _n THEN r _n ELSE r _{n+1} END	Simple case expression; equivalent to: CASE WHEN v=s ₁ THEN r ₁ WHEN v=s ₂ THEN r ₂ ... WHEN v=s _n THEN r _n ELSE r _{n+1} END	r ₁ , r ₂ , ..., r _n , r _{n+1}	V, s ₁ , s ₂ , ..., s _n If collation of v does not dominate over collations of: s ₁ , s ₂ , ..., s _n then simple case is transformed to searched case internally.
Expression	Object Access Expression	Reference to an object method	USING_NLS_COMP	—
Expression	:name	Bind variable reference	Literal collation	—
Expression	(a ₁ , ..., a _n)	Expression list	Each list element has its collation derived separately and independently.	When two lists are compared, the collation determination is performed separately and independently for each of the two character data type elements at the same index in both the lists.
Condition	a ₁ = a ₂ a ₁ <> a ₂ a ₁ < a ₂ a ₁ > a ₂ a ₁ >= a ₂ a ₁ <= a ₂	Simple comparison conditions	—	a ₁ , a ₂ If a ₁ and a ₂ are lists, then see <i>Expression list</i> above.
Condition	a ₁ = ANY (a ₂ , ..., a _n) a ₁ <> ANY (a ₂ , ..., a _n) a ₁ < ANY (a ₂ , ..., a _n) a ₁ > ANY (a ₂ , ..., a _n) a ₁ >= ANY (a ₂ , ..., a _n) a ₁ <= ANY (a ₂ , ..., a _n) (ANY may be replaced by SOME or ALL)	List comparison condition; equivalent to: a ₁ <op> a ₂ AND OR a ₁ <op> a ₃ AND OR ... a ₁ <op> a _n	—	a ₁ , a ₂ a ₁ , a ₃ ... a ₁ , a _n Collations are determined separately for each pair. If a ₁ to a _n are lists, then see <i>Expression list</i> above.

Table C-2 (Cont.) Derivation- and Determination-relevant Arguments for SQL Operations

Operation Type	Operation Name	Operation Description	Derivation-relevant Arguments or Fixed Collation	Determination-relevant Arguments
Condition	$a_1 = \text{ANY}$ (SELECT a_2 FROM ...)	Query comparison conditions	—	a_1, a_2 If a_1 and a_2 are lists, then see <i>Expression list</i> above.
	$a_1 <> \text{ANY}$ (SELECT a_2 FROM ...)			
	$a_1 < \text{ANY}$ (SELECT a_2 FROM ...)			
	$a_1 > \text{ANY}$ (SELECT a_2 FROM ...)			
	$a_1 \geq \text{ANY}$ (SELECT a_2 FROM ...)			
	$a_1 \leq \text{ANY}$ (SELECT a_2 FROM ...)			
	(ANY may be replaced by SOME or ALL)			
Condition	a_1 [NOT] LIKE [a_2 a_3] a_2 ESCAPE a_3	Check if pattern a_2 matches string a_1 using a_3 as escape character in a_2	—	a_1, a_2
Condition	REGEXP_LIKE($a_1, a_2, [a_3]$)	Check if regular expression a_2 matches string a_1 according to flags in a_3	—	a_1, a_2
Condition	a_1 [NOT] BETWEEN a_2 AND a_3	Range comparison; equivalent to: $a_1 \geq a_2$ AND $a_1 \leq a_3$	—	a_1, a_2 a_1, a_3 Collation is determined separately for each comparison.
Condition	a_1 [NOT] IN ($a_2, a_3, \dots a_n$)	Membership comparison; equivalent to: $a_1 =$ ANY($a_2, a_3, \dots a_n$)	—	See =ANY above

Table C-2 (Cont.) Derivation- and Determination-relevant Arguments for SQL Operations

Operation Type	Operation Name	Operation Description	Derivation-relevant Arguments or Fixed Collation	Determination-relevant Arguments
Function	APPROX_COUNT_DISTINCT(a_1)	Approximate count of distinct values of a_1 in the result set	—	a_1
Function	ASCIIISTR(a_1)	Escape non-ASCII characters in a_1 with Unicode escapes	a_1	—
Function	CAST(a_1 AS <i><character data type></i>)	Cast value a_1 to a character data type	a_1 , if a_1 is of character data type; literal collation otherwise.	—
Function	CHR(a_1)	Convert numeric code a_1 to character and return as a VARCHAR2 string	Literal collation	—
Function	COALESCE($a_1, a_2, \dots, a_n$)	First non-null value among: $a_1, a_2, \dots, a_n$ COALESCE(a_1, a_2) is equivalent to: CASE WHEN a_1 IS NOT NULL THEN a_1 ELSE a_2 END; COALESCE($a_1, a_2, \dots, a_n$) is equivalent to: CASE WHEN a_1 IS NOT NULL THEN a_1 ELSE COALESCE ($a_2, \dots, a_n$) END;	$a_1, a_2, \dots, a_n$	—
Function	COLLATION(a_1)	Return name of derived collation of a_1 as string	Literal collation	—

Table C-2 (Cont.) Derivation- and Determination-relevant Arguments for SQL Operations

Operation Type	Operation Name	Operation Description	Derivation-relevant Arguments or Fixed Collation	Determination-relevant Arguments
Function	COLLECT([DISTINCT] a_1 ORDER BY a_2)	Aggregate into a nested table	—	a_1 for DISTINCT a_2 for ORDER BY
Function	COMPOSE(a_1)	Normalize a_1 to Unicode NFC	a_1	—
Function	CONCAT(a_1, a_2)	Concatenate strings a_1 and a_2	a_1, a_2	—
Function	CONVERT($a_1[, a_2[, a_3]]$)	Convert character set of a_1 from a_3 to a_2	a_1	—
Function	COUNT(DISTINCT a_1)	Count distinct values of a_1 in the result set	—	a_1
Function	CORR_K(a_1, a_2, a_3)	Kendall's tau-b correlation coefficient	—	a_1, a_2
Function	CORR_S(a_1, a_2, a_3)	Spearman's rho correlation coefficient	—	a_1, a_2
Function	CUBE_TABLE(. . .)	OLAP cube or hierarchy to relational table	Literal collation (for each character data type column in the generated table)	—
Function	CV($[a_1]$)	Current dimension value in a model clause	Collation of the dimension column to which CV() call corresponds, a_1 or implicit	—
Function	DBTIMEZONE	Database time zone as string	Literal collation	—

Table C-2 (Cont.) Derivation- and Determination-relevant Arguments for SQL Operations

Operation Type	Operation Name	Operation Description	Derivation-relevant Arguments or Fixed Collation	Determination-relevant Arguments
Function	DECODE($v_1, s_1, r_1, s_2, r_2, \dots, s_n, r_n, r_{n+1}$)	Value selection	$r_1, r_2, \dots, r_n, r_{n+1}$	$v_1, s_1, s_2, \dots, s_n$
Function	DECOMPOSE(a_1, a_2)	Unicode normalization (NFD, NFKD); a_2 is the requested normalization form	a_1	—
Function	DENSE_RANK($[a_1, a_2, \dots, a_n]$)	Dense rank of a value in a group of values	—	Ranking is based on collation of the elements in function's ORDER BY clause.
Function	DUMP($a_1[, a_2[, a_3[, a_4]]]$)	Debugging dump of a_4 bytes of value a_1 in format a_2 from position a_3	Literal collation	—
Function	EMPTY_CLOB	Empty CLOB locator	USING_NLS_COMP	—
Function	EXTRACT(TIMEZONE_REGION TIMEZONE_ABBR FROM a_1)	Extract time zone information from the datetime value a_1	Literal collation	—
Function	EXTRACTVALUE($a_1, a_2[, a_3]$)	Extract element value from XMLType	Literal collation	—
Function	FIRST_VALUE(a_1)	First value of a_1 from a set of rows	a_1	—
Function	GREATEST($a_1, a_2, \dots, a_n$)	Largest value among $a_1, \dots, a_n$	$a_1, a_2, \dots, a_n$	$a_1, a_2, \dots, a_n$
Function	INITCAP(a_1)	Capitalize initial letters of a_1	a_1	—
Function	INSTR[B 2 4 C] ($a_1, a_2[, a_3[, a_4]]$)	Position of a_4 -th occurrence of string a_2 in string a_1 starting at position a_3	—	a_1, a_2

Table C-2 (Cont.) Derivation- and Determination-relevant Arguments for SQL Operations

Operation Type	Operation Name	Operation Description	Derivation-relevant Arguments or Fixed Collation	Determination-relevant Arguments
Function	JSON_QUERY($a_1, a_2, \dots$)	Retrieve fragment of the JSON object a_1 described by the JSON path expression a_2 as a string	Literal collation	—
Function	JSON_TABLE($a_1, a_2, \dots$)	Present fragment of the JSON object a_1 described by the JSON path expression a_2 as a virtual relational table	Literal collation (for each character data type column in the generated table)	—
Function	JSON_VALUE($a_1, a_2, \dots$)	Retrieve a scalar value from the JSON object a_1 described by the JSON path expression a_2 as an SQL scalar value	Literal collation	—
Function	LAG($a_1[, a_2[, a_3]]$)	Value of a_1 at row offset a_2 , or a_3 , if outside of window	a_1	—
Function	LAST_VALUE(a_1)	Last value of a_1 from a set of rows	a_1	—
Function	LEAD($a_1[, a_2[, a_3]]$)	Value of a_1 at row offset a_2 , or a_3 , if outside of window	a_1	—
Function	LEAST($a_1, a_2, \dots, a_n$)	Smallest value among $a_1, \dots, a_n$	$a_1, a_2, \dots, a_n$	$a_1, a_2, \dots, a_n$
Function	LISTAGG($a_1[, a_2]$)	Aggregate values of a_1 from multiple rows into a list; a_2 - separator	a_1 , if a_1 is of character data type, otherwise literal collation if not RAW	—
Function	LOWER(a_1)	Lowercase a_1	a_1	—
Function	LPAD($a_1, a_2[, a_3]$)	Pad string a_1 with string a_3 on the left up to display length a_2	a_1	—
Function	LTRIM($a_1[, a_2]$)	Remove characters from the beginning of a_1 as long as they can be found in string a_2	a_1	a_1
Function	MAX(a_1)	Maximum value of a_1 in the result set	a_1	a_1

Table C-2 (Cont.) Derivation- and Determination-relevant Arguments for SQL Operations

Operation Type	Operation Name	Operation Description	Derivation-relevant Arguments or Fixed Collation	Determination-relevant Arguments
Function	MIN(a_1)	Minimum value of a_1 in the result set	a_1	a_1
Function	NCHR(a_1)	Convert numeric code a_1 to character and return as a NVARCHAR2 string	Literal collation	—
Function	NLS_CHARSET_NAME(a_1)	Name of the character set with ID a_1	Literal collation	—
Function	NLS_COLLATION_NAME(a_1)	Name of the collation with ID a_1	Literal collation	—
Function	NLS_INITCAP(a_1 [, a_2])	Capitalize initial letters of a_1 optionally using collation specified in a_2	a_1	a_1 Collation specified with NLS_SORT in a_2 overrides collation of a_1 , but only at the execution time. COLLATION(NLS_INITCAP(a_1 , a_2)) returns collation of a_1
Function	NLS_LOWER(a_1 [, a_2])	Lowercase a_1 optionally using collation specified in a_2	a_1	a_1 Collation specified with NLS_SORT in a_2 overrides collation of a_1 , but only at the execution time. COLLATION(NLS_LOWER(a_1 , a_2)) returns collation of a_1
Function	NLS_UPPER(a_1 [, a_2])	Capitalize a_1 optionally using collation specified in a_2	a_1	a_1 Collation specified with NLS_SORT in a_2 overrides collation of a_1 , but only at the execution time. COLLATION(NLS_UPPER(a_1 , a_2)) returns collation of a_1
Function	NLSSORT(a_1 [, a_2])	Generate collation key for a_1 optionally using collation specified in a_2	—	a_1 Collation specified with NLS_SORT in a_2 overrides the collation of a_1 , but only at the execution time.

Table C-2 (Cont.) Derivation- and Determination-relevant Arguments for SQL Operations

Operation Type	Operation Name	Operation Description	Derivation-relevant Arguments or Fixed Collation	Determination-relevant Arguments
Function	$NTH_VALUE(a_1, n)$	The n -th value of a_1 from a set of rows	a_1	—
Function	$NULLIF(a_1, a_2)$	NULL, if $a_1=a_2$, otherwise a_1 This is equivalent to: <pre> CASE WHEN $a_1=a_2$ THEN NULL ELSE a_1 END; </pre>	a_1	a_1, a_2
Function	$NVL(a_1, a_2)$	a_1 , if a_1 is not NULL, otherwise a_2	a_1, a_2	—
Function	$NVL2(a_1, a_2, a_3)$	a_2 , if a_1 is not NULL, otherwise a_3 .	a_2, a_3	—
Function	ORA_INVOKING_USER	Invoking user name	Literal collation	—
Function	PATH(a_1)	Path to a resource	Literal collation	—
Function	$PERCENT_RANK([a_1, a_2, \dots, a_n]) \dots$	Percent rank of a value in a group of values	—	Ranking is based on collation of the elements in function's ORDER BY clause.
Function	PREDICTION	Data mining prediction	Literal collation	—
Function	$PRESENTNNV(a_1, a_2, a_3)$	If the cell reference a_1 exists before execution of the enclosing model clause and is not null when the function is evaluated, then a_2 else a_3	a_2, a_3	—
Function	$PRESENTV(a_1, a_2, a_3)$	If the cell reference a_1 exists before execution of the enclosing model clause, then a_2 , else a_3 .	a_2, a_3	—

Table C-2 (Cont.) Derivation- and Determination-relevant Arguments for SQL Operations

Operation Type	Operation Name	Operation Description	Derivation-relevant Arguments or Fixed Collation	Determination-relevant Arguments
Function	PREVIOUS(a_1)	Value of the cell reference a_1 at the beginning of an iteration in a model clause	a_1	—
Function	RANK($[a_1, a_2, \dots, a_n]$)	Rank of a value in a group of values	—	Ranking is based on collation of the elements in function's ORDER BY clause.
Function	RAWTOHEX(a_1)	Convert the RAW value a_1 to its hexadecimal representation in a VARCHAR2 string	Literal collation	—
Function	RAWTONHEX(a_1)	Convert the RAW value a_1 to its hexadecimal representation in a NVARCHAR2 string	Literal collation	—
Function	REGEXP_COUNT($a_1, a_2[, a_3[, a_4]]$)	Number of times regular expression a_2 matches substrings of string a_1 according to flags a_4 starting matching at position a_3	—	a_1, a_2
Function	REGEXP_INSTR($a_1, a_2[, a_3[, a_4[, a_5[, a_6[, a_7]]]]$)	Minimal position in a_1 at which regular expression a_2 matches substring of string a_1 for the a_4 -th time according to flags a_6 starting matching at position a_3 ; a_5 and a_7 control which position is actually returned	—	a_1, a_2
Function	REGEXP_REPLACE($a_1, a_2[, a_3[, a_4[, a_5[, a_6]]]]$)	Replace with string a_3 all matches or the a_5 -th match of regular expression a_2 with a substring of string a_1 according to flags a_6 starting matching at position a_4	a_1	a_1, a_2
Function	REGEXP_SUBSTR($a_1, a_2[, a_3[, a_4[, a_5[, a_6]]]]$)	Return the a_4 -th matching substring of regular expression a_2 in string a_1 according to flags a_5 starting matching at position a_3 . if a_6 is specified, it is the index of sub-expression to return in place of the whole matching substring.	a_1	a_1, a_2

Table C-2 (Cont.) Derivation- and Determination-relevant Arguments for SQL Operations

Operation Type	Operation Name	Operation Description	Derivation-relevant Arguments or Fixed Collation	Determination-relevant Arguments
Function	<code>REPLACE(a₁, a₂[, a₃])</code>	a ₁ with every occurrence of a ₂ replaced with a ₃	a ₁	a ₁ , a ₂
Function	<code>ROWIDTOCHAR(a₁)</code>	Convert the rowid a ₁ to a VARCHAR2 string	Literal collation	—
Function	<code>ROWIDTONCHAR(a₁)</code>	Convert the rowid a ₁ to a NVARCHAR2 string	Literal collation	—
Function	<code>RPAD(a₁, a₂[, a₃])</code>	Pad string a ₁ with string a ₃ on the right up to display length a ₂	a ₁	—
Function	<code>RTRIM(a₁[, a₂])</code>	Remove characters from the end of a ₁ as long as they can be found in string a ₂	a ₁	a ₁
Function	<code>SESSIONTIMEZONE</code>	Database time zone as string	Literal collation	—
Function	<code>SOUNDEX(a₁)</code>	Soundex representation of a ₁ (for phonetic comparison)	a ₁	—
Function	<code>STATS_BINOMIAL_TEST(a₁, a₂, a₃[, a₄])</code>	Exact probability test of dichotomous variables a ₁ and a ₂	—	a ₁
Function	<code>STATS_CROSSTAB(a₁, a₂[, a₃])</code>	Crosstab analysis of a ₁ and a ₂	—	a ₁ , a ₂ Collation is determined independently for each argument.
Function	<code>STATS_F_TEST(a₁, a₂[, a₃[, a₄]])</code>	Variance analysis of a ₁ and a ₂	—	a ₁

Table C-2 (Cont.) Derivation- and Determination-relevant Arguments for SQL Operations

Operation Type	Operation Name	Operation Description	Derivation-relevant Arguments or Fixed Collation	Determination-relevant Arguments
Function	STATS_KS_TEST(a_1 , a_2 [, a_3])	Kolmogorov-Smirnov function	—	a_1 , a_2 Collation is determined independently for each argument.
Function	STATS_MODE(a_1)	Most frequent value of a_1 in the result set	a_1	a_1
Function	STATS_MW_TEST(a_1 , a_2 [, a_3])	Mann Whitney test	—	a_1 , a_2 Collation is determined independently for each argument.
Function	STATS_ONE_WAY_ANOVA(a_1 , a_2 [, a_3])	One-way analysis of variance	—	a_1
Function	STATS_T_TEST_INDEP(a_1 , a_2 [, a_3 [, a_4]])	T-test of independent groups with same variance	—	a_1
Function	STATS_T_TEST_INDEPU(a_1 , a_2 [, a_3 [, a_4]])	T-test of independent groups with unequal variance	—	a_1
Function	SUBSTR[B 2 4 C] (a_1 , a_2 [, a_3])	Substring of a_1 starting at position a_2 of length a_3	a_1	—
Function	SYS_CONNECT_BY_PATH(a_1 , a_2)	Path of value a_1 from root to node, with column values separated by character a_2	a_1	—
Function	SYS_CONTEXT(a_1 , a_2 [, a_3])	Context parameter a_2 of length a_3 from namespace a_1	Literal collation	—

Table C-2 (Cont.) Derivation- and Determination-relevant Arguments for SQL Operations

Operation Type	Operation Name	Operation Description	Derivation-relevant Arguments or Fixed Collation	Determination-relevant Arguments
Function	TO_CHAR(<i>a</i> ₁) /*character*/	Convert <i>a</i> ₁ from data type CLOB, NCHAR, NVARCHAR2, or NCLOB to VARCHAR2	<i>a</i> ₁	—
Function	TO_CHAR(<i>a</i> ₁ [, <i>a</i> ₂ [, <i>a</i> ₃]]) /*datetime*/	Convert <i>a</i> ₁ from a datetime data type to VARCHAR2 with optional format <i>a</i> ₂ and NLS environment <i>a</i> ₃	Literal collation	—
Function	TO_CHAR(<i>a</i> ₁ [, <i>a</i> ₂ [, <i>a</i> ₃]]) /*number*/	Convert <i>a</i> ₁ from a numeric data type to VARCHAR2 with optional format <i>a</i> ₂ and NLS environment <i>a</i> ₃	Literal collation	—
Function	TO_CLOB(<i>a</i> ₁)	Convert <i>a</i> ₁ from data type CHAR, VARCHAR2, CLOB, NCHAR, NVARCHAR2, or NCLOB to CLOB	<i>a</i> ₁ (must yield USING_NLS_COMP)	—
Function	TO_LOB(<i>a</i> ₁) /*long*/	Convert <i>a</i> ₁ from data type LONG to CLOB	<i>a</i> ₁ (must yield USING_NLS_COMP)	—
Function	TO_MULTI_BYTE(<i>a</i> ₁)	Map normal-width characters in <i>a</i> ₁ to full-width characters	<i>a</i> ₁	—
Function	TO_NCHAR(<i>a</i> ₁) /*character*/	Convert <i>a</i> ₁ from data type NCLOB, CHAR, VARCHAR2, or CLOB to NVARCHAR2	<i>a</i> ₁	—
Function	TO_NCHAR(<i>a</i> ₁ [, <i>a</i> ₂ [, <i>a</i> ₃]])) /*datetime*/	Convert <i>a</i> ₁ from a datetime data type to NVARCHAR2 with optional format <i>a</i> ₂ and NLS environment <i>a</i> ₃	Literal collation	—
Function	TO_NCHAR(<i>a</i> ₁ [, <i>a</i> ₂ [, <i>a</i> ₃]])) /*number*/	Convert <i>a</i> ₁ from a numeric data type to NVARCHAR2 with optional format <i>a</i> ₂ and NLS environment <i>a</i> ₃	Literal collation	—

Table C-2 (Cont.) Derivation- and Determination-relevant Arguments for SQL Operations

Operation Type	Operation Name	Operation Description	Derivation-relevant Arguments or Fixed Collation	Determination-relevant Arguments
Function	TO_NCLOB(<i>a</i> ₁)	Convert <i>a</i> ₁ from data type CHAR, VARCHAR2, CLOB, NCHAR, NVARCHAR2, or NCLOB to NCLOB	<i>a</i> ₁ (must yield USING_NLS_COMP)	—
Function	TO_SINGLE_BYTE(<i>a</i> ₁)	Map full-width characters in <i>a</i> ₁ to normal-width characters	<i>a</i> ₁	—
Function	TRANSLATE(<i>a</i> ₁ , <i>a</i> ₂ , <i>a</i> ₃)	Transform <i>a</i> ₁ by mapping characters in <i>a</i> ₂ to corresponding characters in <i>a</i> ₃	<i>a</i> ₁	<i>a</i> ₁
Function	TRANSLATE(<i>a</i> ₁ USING CHAR_CS NCHAR_CS)	Convert <i>a</i> ₁ from one character set form to another (roughly equivalent to: TO_CHAR TO_NCHAR / *character*/)	<i>a</i> ₁	—
Function	TRIM([[LEADING TRAILING BOTH] [<i>a</i> ₁] FROM] <i>a</i> ₂)	Remove all occurrences of character <i>a</i> ₁ at the beginning and/or at the end of <i>a</i> ₂	<i>a</i> ₂	<i>a</i> ₂
Function	TZ_OFFSET(<i>a</i> ₁)	Offset for the time zone <i>a</i> ₁	Literal collation	—
Function	UNISTR	Transform string <i>a</i> ₁ into an NVARCHAR2 string interpreting Unicode escapes	<i>a</i> ₁	—
Function	UPPER(<i>a</i> ₁)	Capitalize string <i>a</i> ₁	<i>a</i> ₁	—
Function	USER	Login user name	Literal collation	—
Function	USERENV(<i>a</i> ₁)	USERENV context parameter <i>a</i> ₁	Literal collation	—
Function	XMLCAST(<i>a</i> ₁ AS < <i>data type</i> >)	Cast result of XMLQuery to data type	Literal collation	—

Table C-2 (Cont.) Derivation- and Determination-relevant Arguments for SQL Operations

Operation Type	Operation Name	Operation Description	Derivation-relevant Arguments or Fixed Collation	Determination-relevant Arguments
Function	XMLSERIALIZE(... a_1 [AS VARCHAR2 CLOB]...)	Serialize XML document a_1 to a string	Literal collation	—
Function	XMLTABLE(... COLUMNS col_1 <data type> ... col_n <data type>...)	Present content of an XML object as a virtual relational table	Literal collation (for each character data type column in the generated table)	—
Clause	OVER(PARTITION BY $a_1, a_2, \dots a_n$)	Analytic clause partitioning	—	a_1 a_2 ... a_n Collation is determined separately for each character argument in the clause.
Clause	OVER(ORDER BY a_1 , $a_2, \dots a_n$)	Analytic clause ordering	—	a_1 a_2 ... a_n Collation is determined separately for each character argument in the clause.
Clause	ORDER BY a_1 , $a_2, \dots a_n$	Aggregate function ordering	—	a_1 a_2 ... a_n Collation is determined separately for each character argument in the clause.

Table C-2 (Cont.) Derivation- and Determination-relevant Arguments for SQL Operations

Operation Type	Operation Name	Operation Description	Derivation-relevant Arguments or Fixed Collation	Determination-relevant Arguments
Clause	ORDER BY a_1 , a_2 , ... a_n	Query result ordering	—	a_1 a_2 ... a_n Collation is determined separately for each character argument in the clause.
Clause	GROUP BY a_1 , a_2 , ... a_n	Query row grouping	—	a_1 a_2 ... a_n Collation is determined separately for each character argument in the clause.

Glossary

accent

A mark that changes the sound of a character. Because the common meaning of the word **accent** is associated with the stress or prominence of the character's sound, the preferred word in *Oracle AI Database Globalization Support Guide* is **diacritic**.

See also [diacritic](#).

accent-insensitive linguistic sort

A linguistic sort that uses information only about base letters, not diacritics or case.

See also [linguistic collation](#), [base letter](#), [diacritic](#), [case](#).

AL16UTF16

The default Oracle Database character set for the SQL NCHAR data type, which is used for the national character set. It encodes Unicode data in the UTF-16BE (big endian) encoding scheme.

See also [national character set](#), [UTF-16](#).

AL32UTF8

An Oracle Database character set for the SQL CHAR data type, which is used for the database character set. It encodes Unicode data in the UTF-8 encoding scheme.

Starting from Oracle Database 12c Release 2, if you use Oracle Universal Installer (OUI) or Oracle Database Configuration Assistant (DBCA) to create a database, then the default database character set used is AL32UTF8.

See also [database character set](#).

ASCII

American Standard Code for Information Interchange. A common encoded 7-bit character set for English. ASCII includes the letters A-Z and a-z, as well as digits, punctuation symbols, and control characters. The Oracle Database character set name is US7ASCII.

base letter

A character stripped of its diacritics and case. For example, the base letter for a, A, ä, and Ä is a.

See also [diacritic](#).

binary collation

A type of collation that orders strings based on their binary representation (character encoding), treating each string as a simple sequences of bytes.

See also [collation](#), [linguistic collation](#), [monolingual linguistic collation](#), [multilingual linguistic collation](#), [accent-insensitive linguistic sort](#), [case-insensitive linguistic collation](#).

binary sorting

Ordering character strings using the binary collation.

byte semantics

Treatment of strings as a sequence of bytes. Offsets into strings and string lengths are expressed in bytes.

See also [character semantics](#) and [length semantics](#).

canonical equivalence

A Unicode Standard term for describing that two characters or sequences of characters are to be semantically considered as the same character. Canonically equivalent characters cannot be distinguished when they are correctly rendered. For example, the precomposed character ñ (U+00F1 Latin Small Letter N With Tilde) is canonically equivalent to the sequence n (U+006E Latin Small Letter N) followed by ~ (U+0303 Combining Tilde).

case

Refers to the condition of being uppercase or lowercase. For example, in a Latin alphabet, A is the uppercase form for a, which is the lowercase form.

case conversion

Changing a character from uppercase to lowercase or vice versa.

case-insensitive linguistic collation

A linguistic collation that uses information about base letters and diacritics but not case but not when determining the ordering of strings.

See also [base letter](#), [case](#), [diacritic](#), [linguistic collation](#).

character

A character is an abstract element of text. A character is different from a glyph, which is a specific representation of a character. For example, the first character of the English uppercase alphabet can be displayed as monospaced A, proportional italic AA, cursive (longhand) A, and so on. These forms are different glyphs that represent the same character. A character, a character code, and a glyph are related as follows:

character --(encoding)--> character code --(font)--> glyph

For example, the first character of the English uppercase alphabet is represented in computer memory as a number. The number is called the **encoding** or the **character code**. The character code for the first character of the English uppercase alphabet is 0x41 in the ASCII encoding scheme. The character code is 0xc1 in the EBCDIC encoding scheme.

You must choose a font to display or print the character. The available fonts depend on which encoding scheme is being used. Each font will usually use a different shape, that is, a different glyph to represent the same character.

See also [character code](#) and [glyph](#).

character classification

Information that provides details about the type of character associated with each character code. For example, a character can be uppercase, lowercase, punctuation, or control character.

character code

A character code is a sequence of bytes that represents a specific character. The sequence depends on the character encoding scheme. For example, the character code of the first character of the English uppercase alphabet is 0x41 in the ASCII encoding scheme, but it is 0xc1 in the EBCDIC encoding scheme.

See also [character](#).

character encoding form

A rule that assigns numbers to all characters in a character set.

character encoding scheme

A rule that maps numbers assigned by the character encoding form to particular sequences of bytes (character codes). For example, the UTF-16 encoding form has the big-endian encoding scheme (UTF-16BE) and the little-endian encoding scheme (UTF-16LE).

Most encoding forms have only one encoding scheme. Therefore, encoding form, encoding scheme, and encoding are often used interchangeably.

Oracle character sets correspond to character encoding schemes. For example, AL16UTF16 is the Oracle name for the UTF-16BE encoding scheme.

character repertoire

The characters that are available to be used, or encoded, in a specific character set.

character semantics

Treatment of strings as a sequence of characters. Offsets into strings and string lengths are expressed in characters (character codes).

See also [byte semantics](#) and [length semantics](#).

character set

A collection of elements that represent textual information for a specific language or group of languages. One language can be represented by more than one character set.

A character set does not always imply a specific character encoding scheme. A character encoding scheme is the assignment of a character code to each character in a character set.

In this manual, a character set usually does imply a specific character encoding scheme. Therefore, a character set is the same as an encoded character set in this manual.

character set migration

Changing the character set of an existing database.

character string

A sequence of characters.

A character string can also contain no characters. In this case, the character string is called a **null string**. The number of characters in a null string is 0 (zero).

client character set

The encoded character set used by the database client. A client character set can differ from the database character set. The database character set is sometimes called the **server character set**. If the client character set is different from the database character set, then character set conversion must occur.

See also [database character set](#).

code point

The numeric representation of a character in a character set. For example, the code point of A in the ASCII character set is 0x41. The code point of a character is also called the **encoded value** of a character.

See also [Unicode code point](#).

code unit

The unit of encoded text for processing and interchange. The size of the code unit varies depending on the character encoding scheme. In most character encodings, a code unit is 1 byte. Important exceptions are UTF-16 and UCS-2, which use 2-byte code units, and wide character, which uses 4 bytes.

See also [character encoding form](#).

collation

Ordering of character strings according to rules about sorting characters that are associated with a language in a specific locale. Also called **linguistic sort**.

See also [linguistic collation](#), [monolingual linguistic collation](#), [multilingual linguistic collation](#), [accent-insensitive linguistic sort](#), [case-insensitive linguistic collation](#).

data scanning

The process of identifying potential problems with character set conversion and truncation of data before migrating the database character set.

database character set

The encoded character set that is used to store text in the database. This includes CHAR, VARCHAR2, LONG, and fixed-width CLOB column values and all SQL and PL/SQL text.

Database Migration Assistant for Unicode (DMU)

An intuitive and user-friendly GUI tool to migrate your character set. It helps you streamline the migration process through an interface that minimizes the workload and ensures that all migration issues are addressed.

diacritic

A mark near or through a character or combination of characters that indicates a different sound than the sound of the character without the diacritical mark. For example, the cedilla in façade is a diacritic. It changes the sound of c.

EBCDIC

Extended Binary Coded Decimal Interchange Code. EBCDIC is a family of encoded character sets used mostly on IBM mainframe systems.

encoded character set

A character set with an associated character encoding scheme. An encoded character set specifies the byte sequence (character code) that is assigned to each character.

See also [character encoding form](#).

encoded value

The numeric representation of a character in a character set. For example, the code point of A in the ASCII character set is 0x41. The encoded value of a character is also called the **code point** of a character.

font

An ordered collection of character glyphs that provides a graphical representation of characters in a character set.

globalization

The process of making software suitable for different linguistic and cultural environments. Globalization should not be confused with localization, which is the process of preparing software for use in one specific locale (for example, translating error messages or user interface text from one language to another).

glyph

A glyph (font glyph) is a specific representation (shape) of a character. A character can have many different glyphs.

See also [character](#).

ideograph

A symbol that represents an idea. Some writing systems use ideographs to represent words through their meaning instead of using letters to represent words through their sound. Chinese is an example of an ideographic writing system.

ISO

International Organization for Standardization. A worldwide federation of national standards bodies from 130 countries. The mission of ISO is to develop and promote standards in the world to facilitate the international exchange of goods and services.

ISO 8859

A family of 8-bit encoded character sets. The most common one is ISO 8859-1 (also known as ISO Latin1), and is used for Western European languages.

ISO 14651

A multilingual linguistic collation standard that is designed for almost all languages of the world.

See also [multilingual linguistic collation](#).

ISO/IEC 10646

A universal character set standard that defines the characters of most major scripts used in the modern world. ISO/IEC 10646 is kept synchronized with the Unicode Standard as far as character repertoire is concerned but it defines fewer properties and fewer text processing algorithms than the Unicode Standard.

ISO currency

The 3-letter abbreviation used to denote a local currency, based on the ISO 4217 standard. For example, USD represents the United States dollar.

ISO Latin1

The ISO 8859-1 character set standard. It is an 8-bit extension to ASCII that adds 128 characters that include the most common Latin characters used in Western Europe. The Oracle Database character set name is WE8ISO8859P1.

See also [ISO 8859](#).

length semantics

Length semantics determines how you treat the length of a character string. The length can be expressed as a number of characters (character codes) or as a number of bytes in the string.

See also [character semantics](#) and [byte semantics](#).

linguistic collation

A type of collation that takes into consideration the standards and customs of spoken languages.

See also [collation](#), [linguistic sorting](#), [monolingual linguistic collation](#), [multilingual linguistic collation](#), [accent-insensitive linguistic sort](#), [case-insensitive linguistic collation](#).

linguistic index

An index built on a linguistic sort order.

linguistic sorting

Ordering character strings using a linguistic binary collation.

See also [multilingual linguistic collation](#) and [monolingual linguistic collation](#).

locale

A collection of information about the linguistic and cultural preferences from a particular region. Typically, a locale consists of language, territory, character set, linguistic, and calendar information defined in NLS data files.

localization

The process of providing language-specific or culture-specific information for software systems. Translation of an application's user interface is an example of localization. Localization should not be confused with globalization, which is the making software suitable for different linguistic and cultural environments.

monolingual linguistic collation

An Oracle Database collation that has two levels of comparison for strings. String are first ordered based on major values for their characters and if they are found equal in this comparison, they are further ordered based on minor values of their characters. Major values correspond roughly to base letters while minor values correspond to diacritics and case. Most European languages can be sorted with a monolingual collation, but monolingual collations are inadequate for Asian languages and for multilingual text.

See also [multilingual linguistic collation](#).

monolingual support

Support for only one language.

multibyte

Two or more bytes.

When character codes are assigned to all characters in a specific language or a group of languages, one byte (8 bits) can represent 256 different characters. Two bytes (16 bits) can represent up to 65,536 different characters. Two bytes are not enough to represent all the characters for many languages. Some characters require 3 or 4 bytes.

One example is the UTF-8 Unicode encoding form. In UTF-8, there are many 2-byte and 3-byte characters.

Another example is Traditional Chinese, used in Taiwan. It has more than 80,000 characters. Some character encoding schemes that are used in Taiwan use 4 bytes to encode characters.

See also [single byte](#).

multibyte character

A character whose character code consists of two or more bytes under a certain character encoding scheme.

Note that the same character may have different character codes under different encoding schemes. Oracle Database cannot tell whether a character is a multibyte character without

knowing which character encoding scheme is being used. For example, Japanese Hankaku-Katakana (half-width Katakana) characters are one byte in the JA16SJIS encoded character set, two bytes in JA16EUC, and three bytes in AL32UTF8.

See also [single-byte character](#).

multibyte character string

A character string encoded in a multibyte character encoding scheme.

multibyte character encoding scheme

A character encoding scheme in which character codes may have more than one byte.

See also multibyte fixed-width character encoding scheme, multibyte varying-width character encoding scheme.

multibyte fixed-width character encoding scheme

A character encoding scheme in which each character code has the same fixed number of bytes, greater than one. AL16UTF16 is a multibyte fixed-width character set.

multibyte varying-width character encoding scheme

A character encoding scheme in which each character code has a number of bytes from a given range. The range is one to the maximum character width of the character set. Depending on the encoding scheme, the maximum character width of the character set may be 2, 3, or 4 bytes. For example, ZHT16BIG5 has character codes with one or two bytes. UTF8 has character codes with one, two, or three bytes. AL32UTF8 has character codes with one, two, three, or four bytes. Oracle does not support encoding schemes with more than 4 bytes per character code.

multilingual linguistic collation

An Oracle Database collation that evaluates strings on three levels. Asian languages require a multilingual linguistic collation even if data exists in only one language. Multilingual linguistic collations are also used when data exists in several languages.

In multilingual collations, strings are first ordered based on primary weights, then, if necessary, secondary weights, then tertiary weights. For letters, primary weights correspond to base letters, secondary weights to diacritics, and tertiary weights to case and specific decoration, such as circle around the character. For ideographic scripts weights may represent other character variations.

national character set

An alternate character set from the database character set that can be specified for NCHAR, NVARCHAR2, and NLOB columns. National character sets are AL16UTF16 and UTF8 only.

NLB files

Binary files used by the Locale Builder to define locale-specific data. They define all of the locale definitions that are shipped with a specific release of Oracle Database. You can create user-defined NLB files with Oracle Locale Builder.

See also [Oracle Locale Builder](#) and [NLT files](#).

NLS

National Language Support. NLS enables users to interact with the database in their native languages. It also enables applications to run in different linguistic and cultural environments. The term has been replaced by the terms globalization and localization.

NLSRTL

National Language Support Runtime Library. This library is responsible for providing locale-independent algorithms for internationalization. The locale-specific information (that is, NLSDATA) is read by the NLSRTL library during run-time.

NLT files

Text files used by the Locale Builder to define locale-specific data. Because they are in text, you can view the contents.

null string

A character string that contains no characters.

Oracle Locale Builder

A GUI utility that offers a way to view, modify, or define locale-specific data.

replacement character

A character used during character conversion when the source character is not available in the target character set. For example, ? (question mark) is often used as the default replacement character in Oracle character sets.

restricted multilingual support

Multilingual support that is restricted to a group of related languages. Western European languages can be represented with ISO 8859-1, for example, but the use of ISO 8859-1 restricts the multilingual support. Thai or Chinese could not be added to the group.

SQL CHAR data types

Includes CHAR, VARCHAR, VARCHAR2, CLOB, and LONG data types.

SQL NCHAR data types

Includes NCHAR, NVARCHAR2, and NCL0B data types.

script

A particular system of writing. A collection of related graphic symbols that are used in a writing system. Some scripts can represent multiple languages, and some languages use multiple scripts. Examples of scripts include Latin, Arabic, and Han.

single byte

One byte. One byte usually consists of 8 bits. When character codes are assigned to all characters for a specific language, one byte (8 bits) can represent 256 different characters.

See also [multibyte](#).

single-byte character

A single-byte character is a character whose character code consists of one byte under a specific character encoding scheme. Note that the same character may have different character codes under different encoding schemes. Oracle Database cannot tell which character is a single-byte character without knowing which encoding scheme is being used. For example, the euro currency symbol is one byte in the WE8MSWIN1252 encoded character set, two bytes in AL16UTF16, and three bytes in UTF8.

See also [multibyte character](#).

single-byte character string

A single-byte character string is a string encoded in a single-byte character encoding scheme. The term may also be used to describe a multibyte varying-width character string that happens to consist only of single-byte character codes. See also multibyte varying-width character encoding scheme.

sort

An ordering of strings. This can be based on requirements from a locale instead of the binary representation of the strings, which is called a linguistic sort, or based on binary coded values, which is called a binary sort.

See also [multilingual linguistic collation](#) and [monolingual linguistic collation](#).

supplementary characters

The first version of the Unicode Standard was a 16-bit, fixed-width encoding that used two bytes to encode each character. This enabled 65,536 characters to be represented. However, more characters need to be supported because of the large number of Asian ideograms.

Unicode Standard version 3.1 defined supplementary characters to meet this need by extending the numbering range for characters from 0000-FFFF hexadecimal to 0000-10FFFF

hexadecimal. Unicode 3.1 began using two 16-bit code units (also known as **surrogate pairs**) to represent a single supplementary character in the UTF-16 form. This enabled an additional 1,048,576 characters to be defined. The Unicode 3.1 standard added the first group of 44,944 supplementary characters. More were added with subsequent versions of the Unicode Standard.

surrogate pairs

See also [supplementary characters](#).

syllabary

Provide a mechanism for communicating phonetic information along with the ideographic characters used by languages such as Japanese.

UCS-2

An obsolete form for an ISO/IEC 10646 standard character set encoding form. Currently used to mean the UTF-16 encoding form without support for surrogate pairs.

UCS-4

An obsolete name for an ISO/IEC 10646 standard encoding form, synonymous with UTF-32.

Unicode Standard

Unicode Standard is a universal encoded character set that enables information from any language to be stored by using a single character set. Unicode Standard provides a unique code value for every character, regardless of the platform, program, or language.

Unicode Standard also defines various text processing algorithms and related character properties to aid in complex script processing of scripts such as Arabic or Devanagari (Hindi).

Unicode database

A database whose database character set is AL32UTF8 or UTF8.

Unicode code point

A value in the Unicode codespace, which ranges from 0 to 0x10FFFF. Unicode assigns a unique code point to every character.

Unicode data type

A SQL NCHAR data type (NCHAR, NVARCHAR2, and NCL0B). You can store Unicode characters in columns of these data types even if the database character set is not based on the Unicode Standard.

unrestricted multilingual support

The ability to use as many languages as desired. A universal character set, such as Unicode Standard, helps to provide unrestricted multilingual support because it supports a very large character repertoire, encompassing most modern languages of the world.

UTFE

An Oracle character set implementing a 4-byte subset of the Unicode UTF-EBCDIC encoding form, used only on EBCDIC platforms and deprecated.

UTF8

The UTF8 Oracle character set encodes characters in one, two, or three bytes. The UTF8 character set supports Unicode 3.0 and implements the CESU-8 encoding scheme. Although specific supplementary characters were not assigned code points in Unicode until version 3.1, the code point range was allocated for supplementary characters in Unicode 3.0.

Supplementary characters are treated as two separate, user-defined characters that occupy 6 bytes. UTF8 is deprecated.

wide character

A multibyte fixed-width character format that is useful for extensive text processing because it enables data to be processed in consistent, fixed-width chunks. Multibyte varying-width character values may be internally converted to the wide character format for faster processing.

UTF-8

The 8-bit encoding form and scheme of the Unicode Standard. It is a multibyte varying-width encoding. One Unicode character can be 1 byte, 2 bytes, 3 bytes, or 4 bytes in the UTF-8 encoding. Characters from the European scripts are represented in either 1 or 2 bytes. Characters from most Asian scripts are represented in 3 bytes. Supplementary characters are represented in 4 bytes. The Oracle Database character set that implements UTF-8 is AL32UTF8.

UTF-16

The 16-bit encoding form of Unicode. One Unicode character can be one or two 2-code units in the UTF-16 encoding. Characters (including ASCII characters) from European scripts and most Asian scripts are represented by one code unit (2 bytes). Supplementary characters are represented by two code units (4 bytes). The Oracle Database character sets that implement UTF-16 are AL16UTF16 and AL16UTF16LE. AL16UTF16 implements the big-endian encoding scheme of the UTF-16 encoding form (more significant byte of each code unit comes first in memory). AL16UTF16 is a valid national character set. AL16UTF16LE implements the little-endian UTF-16 encoding scheme. It is a conversion-only character set, valid only in character set conversion functions such as SQL CONVERT or PL/SQL UTL_I18N.STRING_TO_RAW. Note that

most SQL string processing functionality treats each UTF-16 code unit in AL16UTF16 as a separate character. The functions INSTR4, SUBSTR4, and LENGTH4 are an exception.

Index

Numerics

7-bit encoding schemes, [6](#), [7](#)

8-bit encoding schemes, [7](#)

A

abbreviations

languages, [A-1](#)

abstract data type

creating as NCHAR, [17](#)

accent-insensitive linguistic sort, [18](#)

ADD_MONTHS SQL function, [13](#)

ADO interface and Unicode, [31](#)

AL16UTF16 character set, [5](#), [A-18](#)

AL24UTFFSS character set, [5](#)

AL32UTF8 character set, [5](#), [7](#), [A-18](#)

ALTER SESSION statement

SET NLS_CURRENCY clause, [28](#)

SET NLS_LANGUAGE clause, [15](#)

SET NLS_NUMERIC_CHARACTERS clause,
[26](#)

SET NLS_TERRITORY clause, [15](#)

application-locales, [37](#)

ASCII encoding, [5](#)

B

base letters, [5](#), [11](#)

BFILE data

loading into LOBs, [13](#)

binary sorts, [2](#)

case-insensitive and accent-insensitive, [20](#)

example, [22](#)

binding and defining CLOB and NCLOB data in
OCI, [17](#)

binding and defining SQL CHAR datatypes in
OCI, [14](#)

binding and defining SQL NCHAR datatypes in
OCI, [15](#)

BLANK_TRIMMING parameter, [3](#)

BLOBs

creating indexes, [16](#)

byte semantics, [9](#), [34](#)

C

C number format mask, [28](#)

Calendar Utility, [40](#)

calendars

customizing, [40](#)

parameter, [21](#)

supported, [A-29](#)

canonical equivalence, [4](#), [15](#)

case, [2](#)

case-insensitive linguistic sort, [18](#)

CESU-8 compliance, [A-18](#)

character data

converting with CONVERT SQL function, [4](#)

character data conversion

database character set, [7](#)

character data scanning

before character set migration, [6](#)

character rearrangement, [17](#)

character repertoire, [1](#)

character semantics, [9](#), [34](#)

character set

conversion, [21](#)

data loss

during conversion, [13](#)

detecting with Globalization Development Kit,
[33](#)

national, [4](#)

character set conversion

between OCI client and database server, [12](#)

parameters, [34](#)

character set definition

customizing, [24](#)

guidelines for editing files, [23](#)

naming files, [23](#)

character set encodings for conversion only, [A-18](#)

character set migration

identifying character data conversion

problems, [6](#)

scanning character data, [6](#)

character sets

AL16UTF16, [5](#)

AL24UTFFSS, [5](#)

AL32UTF8, [5](#)

ASCII, [A-7](#)

changing after database creation, [20](#)

- character sets (*continued*)
 - choosing, [1](#)
 - conversion, [13](#), [22](#), [4](#)
 - conversion using OCI, [6](#)
 - customizing, [20](#)
 - data loss, [3](#)
 - EBCDIC, [A-7](#), [A-21](#)
 - encoding, [1](#)
 - ISO 8859 series, [5](#)
 - migration, [1](#), [2](#)
 - naming, [8](#)
 - national, [9](#), [4](#)
 - other ASCII-based, [A-9](#)
 - other EBCDIC-based, [A-12](#)
 - restrictions on character sets used to express names, [15](#)
 - supersets and subsets, [A-19](#)
 - supported, [A-7](#)
 - supporting different character repertoires, [4](#)
 - universal, [A-18](#)
 - UTFE, [5](#)
 - character snational, [16](#)
 - character type conversion
 - error reporting, [34](#)
 - characters
 - available in all Oracle database character sets, [4](#)
 - context-sensitive, [15](#)
 - contracting, [15](#)
 - user-defined, [21](#)
 - choosing a character set, [1](#)
 - client operating system
 - character set compatibility with applications, [13](#)
 - client-only character sets, [A-15](#)
 - CLOB and NCLOB data
 - binding and defining in OCI, [17](#)
 - CLOBs
 - creating indexes, [16](#)
 - code chart
 - displaying and printing, [16](#)
 - code point, [1](#)
 - collation
 - column-level collation, [38](#)
 - customizing, [27](#)
 - data-bound collation, [39](#)
 - Hiragana and Katakana, [8](#)
 - linguistic collation, [3](#)
 - monolingual, [4](#)
 - multilingual, [4](#)
 - UCA Japanese IVS collation, [9](#)
 - Unicode Collation Algorithm, [6](#)
 - comparisons
 - linguistic, [23](#)
 - compatibility
 - client operating system and application character sets, [13](#)
 - composed characters, [15](#)
 - context-sensitive characters, [15](#)
 - contracting characters, [15](#)
 - contracting letters, [17](#)
 - control characters, encoding, [3](#)
 - conversion
 - between character set ID number and character set name, [7](#)
 - CONVERT SQL function, [4](#)
 - convert time zones, [41](#)
 - converting character data
 - CONVERT SQL function, [4](#)
 - converting character data between character sets, [4](#)
 - Coordinated Universal Time, [4](#), [5](#)
 - creating a database with Unicode datatypes, [8](#)
 - creating a Unicode database, [8](#)
 - CSREPAIR script, [8](#)
 - currencies
 - formats, [27](#)
 - CURRENT_DATE SQL function, [13](#)
 - CURRENT_TIMESTAMP SQL function, [13](#)
- ## D
-
- data conversion
 - in Pro*C/C++, [18](#)
 - OCI driver, [24](#)
 - ODBC and OLE DB drivers, [29](#)
 - thin driver, [24](#)
 - Unicode Java strings, [23](#)
 - data expansion
 - during character set migration, [2](#)
 - during data conversion, [13](#)
 - data inconsistencies causing data loss, [5](#)
 - data loss
 - caused by data inconsistencies, [5](#)
 - during character set conversion, [13](#)
 - during character set migration, [3](#)
 - during datatype conversion exceptions, [5](#)
 - during OCI Unicode character set conversion, [12](#)
 - from mixed character sets, [5](#)
 - Data Pump PL/SQL packages and character set migration, [6](#)
 - data truncation, [2](#)
 - restrictions, [2](#)
 - data types
 - abstract, [17](#)
 - datetime, [1](#)
 - inserting values into datetime data types, [6](#)
 - inserting values into interval data types, [11](#)

data types (*continued*)
 interval, [1](#), [9](#)
 supported, [17](#)
 data-bound collation
 collation derivation, [C-1](#)
 collation derivation and determination rules for
 SQL operations, [C-1](#)
 collation determination, [C-1](#)
 database character set
 character data conversion, [7](#)
 choosing, [11](#)
 compatibility between client operating system
 and applications, [13](#)
 migrated from an EBCDIC platform, [15](#)
 Database Migration Assistant for Unicode (DMU),
 [7](#)
 database schemas
 designing for multiple languages, [12](#)
 database time zone, [39](#)
 datatype conversion
 data loss and exceptions, [5](#)
 implicit, [6](#)
 SQL functions, [7](#)
 date and time parameters, [16](#)
 date formats, [16](#), [17](#), [12](#)
 and partition bound expressions, [17](#)
 dates
 ISO standard, [22](#), [12](#)
 NLS_DATE_LANGUAGE parameter, [18](#)
 datetime data types, [1](#)
 inserting values, [6](#)
 datetime format parameters, [15](#)
 Daylight Saving Time
 Oracle support, [42](#)
 rules, [20](#)
 Daylight Saving Time Upgrade parameter, [16](#)
 days
 format element, [19](#)
 language of names, [19](#)
 DBTIMEZONE SQL function, [13](#)
 detecting language and character sets
 Globalization Development Kit, [33](#)
 detection
 supported languages and character sets,
 [A-22](#)
 diacritic, [2](#)
 DMU
 Database Migration Assistant for Unicode, [7](#)
 DST_UPGRADE_INSERT_CONV initialization
 parameter, [16](#)
 DUCET (Default Unicode Collation Element
 Table), [6](#)

E

encoding
 control characters, [3](#)
 ideographic writing systems, [3](#)
 numbers, [3](#)
 phonetic writing systems, [3](#)
 punctuation, [3](#)
 symbols, [3](#)
 encoding schemes
 7-bit, [6](#), [7](#)
 8-bit, [7](#)
 fixed-width, [7](#)
 multibyte, [7](#)
 shift-sensitive variable-width, [7](#)
 shift-sensitive variable-width multibyte, [7](#)
 single-byte, [6](#)
 variable-width, [7](#)
 environment variables
 ORA_SDTZ, [16](#)
 ORA_TZFILE, [16](#)
 error messages
 languages, [A-4](#)
 translation, [A-4](#)
 euro
 Oracle support, [30](#)
 expanding characters, [17](#)
 characters
 expanding, [15](#)
 EXTRACT (datetime) SQL function, [13](#)

F

fixed-width multibyte encoding schemes, [7](#)
 fonts
 Unicode, [1](#)
 format elements, [13](#)
 C, [13](#)
 D, [13](#)
 day, [19](#)
 G, [13](#)
 IW, [13](#)
 IY, [13](#)
 L, [13](#)
 month, [19](#)
 RM, [12](#)
 RN, [13](#)
 format masks, [26](#), [12](#)
 formats
 currency, [27](#)
 date, [17](#), [15](#)
 numeric, [25](#)
 time, [19](#)
 FROM_TZ SQL function, [13](#)

G

GDK application configuration file, [36](#)
 example, [41](#)
 GDK application framework for J2EE, [16](#)
 GDK components, [7](#)
 GDK error messages, [46](#)
 GDK Java API, [28](#)
 GDK Java supplied packages and classes, [42](#)
 GDK Localizer object, [22](#)
 Globalization Development Kit, [1](#)
 application configuration file, [36](#)
 character set conversion, [30](#)
 components, [7](#)
 defining supported application locales, [23](#)
 e-mail programs, [34](#)
 error messages, [46](#)
 framework, [16](#)
 integrating locale sources, [19](#)
 Java API, [28](#)
 Java supplied packages and classes, [42](#)
 locale detection, [20](#)
 Localizer object, [22](#)
 managing localized content in static files, [27](#)
 managing strings in JSPs and Java servlets, [26](#)
 non_ASCII input and output in an HTML page, [24](#)
 Oracle binary and linguistic sorts, [31](#)
 Oracle date, number, and monetary formats, [31](#)
 Oracle language and character set detection, [33](#)
 Oracle locale information, [28](#)
 Oracle locale mapping, [29](#)
 Oracle translated locale and time zone names, [34](#)
 supported locale resources, [19](#)
 globalization features, [4](#)
 globalization support
 architecture, [1](#)
 Greenwich Mean Time, [4, 5](#)
 guessing the language or character set, [9](#)

H

Hiragana, [8](#)

I

IANA character sets
 mapping with ISO locales, [24](#)
 Ideographic Variation Sequence, [9, 5, 7](#)
 ideographic writing systems, encoding, [3](#)
 ignorable characters, [12](#)
 implicit datatype conversion, [6](#)

indexes

 creating for documents stored as CLOBs, [16](#)
 creating for multilingual document search, [15](#)
 creating indexes for documents stored as BLOBs, [16](#)
 linguistic, [30](#)
 initialization parameter
 DST_UPGRADE_INSERT_CONV, [16](#)
 initialization parameters
 NLS_DATE_FORMAT, [15](#)
 NLS_TIMESTAMP_FORMAT, [15](#)
 NLS_TIMESTAMP_TZ_FORMAT, [15](#)
 INSTR SQL functions, [8](#)
 Internet application
 locale
 determination, [6](#)
 monolingual, [2](#)
 multilingual, [2, 4](#)
 interval data types, [1, 9](#)
 inserting values, [11](#)
 ISO 8859 character sets, [5](#)
 ISO locales
 mapping with IANA character sets, [24](#)
 ISO standard
 date format, [12](#)
 ISO standard date format, [22, 12](#)
 ISO week number, [12](#)
 IVS
 See Ideographic Variation Sequence
 IW format element, [13](#)
 IY format element, [13](#)

J

Java
 Unicode data conversion, [23](#)
 Java strings
 binding and defining in Unicode, [20](#)
 JDBC OCI driver
 and Unicode, [3](#)
 JDBC programming
 Unicode, [20](#)
 JDBC Server Side internal driver
 and Unicode, [3](#)
 JDBC Server Side thin driver
 and Unicode, [3](#)
 JDBC thin driver
 and Unicode, [3](#)

K

Katakana, [8](#)

L

language
 detecting with Globalization Development Kit, [33](#)
 language abbreviations, [A-1](#)
 Language and Character Set File Scanner, [9](#)
 language definition
 customizing, [6](#)
 overriding, [6](#)
 language support, [5](#)
 languages
 error messages, [A-4](#)
 languages and character sets
 supported by LCSSCAN, [A-22](#)
 LAST_DAY SQL function, [13](#)
 LCSSCAN
 error messages, [13](#)
 LCSSCAN, [9](#)
 supported languages and character sets, [12](#), [A-22](#)
 LCSSCAN command
 BEGIN parameter, [11](#)
 END parameter, [11](#)
 examples, [11](#)
 FILE parameter, [11](#)
 HELP parameter, [12](#)
 online help, [12](#)
 RESULTS parameter, [10](#)
 length semantics, [9](#), [34](#)
 LIKE conditions in SQL statements, [7](#)
 linguistic collation
 accent-insensitive collation, [18](#)
 case-insensitive collation, [18](#)
 overview, [3](#)
 linguistic collation definitions
 supported, [A-24](#)
 linguistic comparisons, [23](#)
 linguistic indexes, [30](#)
 linguistic sorts
 BINARY, [21](#)
 BINARY_AI, linguistic sorts
 BINARY_CI, [21](#)
 controlling, [11](#)
 customizing, [27](#)
 characters with diacritics, [31](#), [34](#)
 levels, [5](#)
 list of defaults, [A-2–A-4](#)
 parameters, [22](#)
 list parameter, [25](#)
 lmsgen utility, [7](#)
 loading external BFILE data into LOBs, [13](#)
 LOBs
 loading external BFILE data, [13](#)
 storing documents in multiple languages, [14](#)

locale, [3](#)
 dependencies, [7](#)
 of Internet application
 determining, [6](#)
 variant, [7](#)
 locale information
 mapping between Oracle and other
 standards, [3](#)
 locale-charset-map, [36](#)
 locale-determine-rule, [38](#)
 locale-parameter-name, [39](#)
 LOCALTIMESTAMP SQL function, [13](#)

M

message-bundles, [39](#)
 migration
 character sets, [1](#)
 mixed character sets
 causing data loss, [5](#)
 monetary parameters, [27](#)
 monolingual Internet application, [2](#)
 monolingual linguistic collations
 supported, [A-24](#)
 monolingual linguistic sorts
 example, [22](#)
 months
 format element, [19](#)
 language of names, [19](#)
 MONTHS_BETWEEN SQL function, [13](#)
 multibyte encoding schemes, [7](#)
 fixed-width, [7](#)
 shift-sensitive variable-width, [7](#)
 variable-width, [7](#)
 multilexers
 creating, [15](#)
 multilingual data
 specifying column lengths, [12](#)
 multilingual document search
 creating indexes, [15](#)
 multilingual Internet application, [4](#)
 multilingual linguistic collations
 supported, [A-26](#)
 multilingual linguistic sorts
 example, [22](#)
 multiple languages
 designing database schemas, [12](#)
 storing data, [13](#)
 storing documents in LOBs, [14](#)

N

national character set, [16](#), [9](#), [4](#)
 NCHAR data type
 creating abstract data type, [17](#)
 NCLOB datatype, [5](#)

NEW_TIME SQL function, [13](#)
 NEXT_DAY SQL function, [13](#)
 NLB data
 transportable, [39](#)
 NLB file, [4](#)
 NLB files, [1](#)
 generating and installing, [36](#)
 NLS Calendar Utility, [40](#)
 NLS parameters
 default values in SQL functions, [2](#)
 list, [1](#)
 setting, [1](#)
 specifying in SQL functions, [2](#)
 unacceptable in SQL functions, [4](#)
 NLS Runtime Library, [1](#)
 NLS_CALENDAR parameter, [24](#)
 NLS_COMP parameter, [33](#)
 NLS_CREDIT parameter, [31](#)
 NLS_CURRENCY parameter, [28](#)
 NLS_DATE_FORMAT initialization parameter, [15](#)
 NLS_DATE_FORMAT parameter, [17](#)
 NLS_DATE_LANGUAGE parameter, [18](#)
 NLS_DEBIT parameter, [31](#)
 NLS_DUAL_CURRENCY parameter, [30](#)
 NLS_ISO_CURRENCY parameter, [28](#)
 NLS_LANG parameter, [3](#)
 choosing a locale, [3](#)
 client setting, [8](#)
 examples, [5](#)
 OCI client applications, [14](#)
 specifying, [5](#)
 UNIX client, [8](#)
 Windows client, [8](#)
 NLS_LENGTH_SEMANTICS initialization
 parameter, [10](#)
 NLS_LENGTH_SEMANTICS session parameter,
 [10](#)
 NLS_LIST_SEPARATOR parameter, [34](#)
 NLS_MONETARY_CHARACTERS parameter, [31](#)
 NLS_NCHAR_CONV_EXCP parameter, [34](#)
 NLS_NUMERIC_CHARACTERS parameter, [26](#)
 NLS_SORT parameter, [32](#)
 NLS_TERRITORY parameter, [13](#)
 NLS_TIMESTAMP_FORMAT initialization
 parameter, [15](#)
 NLS_TIMESTAMP_TZ_FORMAT initialization
 parameter, [15](#)
 NLSRTL, [1](#)
 NLSSORT SQL function, [8](#)
 syntax, [10](#)
 NLT files, [1](#)
 numbers, encoding, [3](#)
 numeric formats, [25](#)
 SQL masks, [13](#)
 numeric parameters, [25](#)
 NUMTODSINTERVAL SQL function, [13](#)

NUMTOYMINTERVAL SQL function, [13](#)
 NVARCHAR datatype
 Pro*C/C++, [19](#)

O

obsolete character sets, [A-39](#)
 OCI
 binding and defining CLOB and NCLOB data
 in OCI, [17](#)
 binding and defining SQL NCHAR datatypes,
 [15](#)
 SQL CHAR datatypes, [14](#)
 OCI and Unicode, [2](#)
 OCI character set conversion, [12](#)
 data loss, [12](#)
 performance, [12](#)
 OCI client applications
 using Unicode character sets, [14](#)
 OCI data conversion
 data expansion, [13](#)
 OCI_UTF16ID character set ID, [11](#)
 OCICharSetConvert(), [6](#)
 OCINlsCharSetIdToName(), [2](#)
 OCINlsCharSetNameToId(), [2](#)
 OCINlsEnvironmentVariableGet(), [2](#)
 OCINlsGetInfo(), [2](#)
 OCINlsNumericInfoGet(), [2](#)
 OCIWideCharToUpper(), [6](#)
 ODBC Unicode applications, [30](#)
 OLE DB Unicode datatypes, [31](#)
 operating system
 character set compatibility with applications,
 [13](#)
 ORA_DST_AFFECTED SQL function, [14](#)
 ORA_DST_CONVERT SQL function, [14](#)
 ORA_DST_ERROR SQL function, [14](#)
 ORA_SDTZ environment variable, [16](#)
 ORA_TZFILE environment variable, [16](#)
 Oracle Call Interface and Unicode, [2](#)
 Oracle Data Provider for .NET and Unicode, [2](#)
 Oracle Data Pump and character set conversion,
 [6](#)
 Oracle Language and Character Set Detection
 Java classes, [33](#)
 Oracle Locale Builder
 choosing a calendar format, [11](#)
 choosing currency formats, [14](#)
 choosing date and time formats, [12](#)
 displaying code chart, [16](#)
 Existing Definitions dialog box, [3](#)
 Open File dialog box, [5](#)
 Preview NLT screen, [5](#)
 restrictions on names for locale objects, [7](#)
 Session Log dialog box, [4](#)
 starting, [2](#)

Oracle ODBC driver and Unicode, [2](#)
 Oracle OLE DB driver and Unicode, [2](#)
 Oracle Pro*C/C++ and Unicode, [2](#)
 ORDER BY clause, [11](#)
 overriding language and territory definitions, [6](#)

P

page-charset, [37](#)
 parameters
 BLANK_TRIMMING, [3](#)
 calendar, [21](#)
 character set conversion, [34](#)
 linguistic sorts, [32](#)
 methods of setting, [1](#)
 monetary, [27](#)
 NLS_CALENDAR, [24](#)
 NLS_COMP, [33](#)
 NLS_CREDIT, [31](#)
 NLS_CURRENCY, [28](#)
 NLS_DATE_FORMAT, [17](#)
 NLS_DATE_LANGUAGE, [18](#)
 NLS_DEBIT, [31](#)
 NLS_DUAL_CURRENCY, [30](#)
 NLS_ISO_CURRENCY, [28](#)
 NLS_LANG, [3](#)
 NLS_LIST_SEPARATOR, [34](#)
 NLS_MONETARY_CHARACTERS, [31](#)
 NLS_NCHAR_CONV_EXCP, [34](#)
 NLS_NUMERIC_CHARACTERS, [26](#)
 NLS_SORT, [32](#)
 NLS_TERRITORY, [13](#)
 numeric, [25](#)
 time and date, [16](#)
 performance
 during OCI Unicode character set conversion, [12](#)
 phonetic writing systems, encoding, [3](#)
 PL/SQL and SQL and Unicode, [3](#)
 primary level sort, [5](#)
 Private Use Area, [22](#)
 Pro*C/C++
 data conversion, [18](#)
 NVARCHAR datatype, [19](#)
 VARCHAR datatype, [18](#)
 punctuation, encoding, [3](#)

R

regular expressions
 character class, [36](#)
 character range, [35](#)
 collation element delimiter, [36](#)
 equivalence class, [36](#)
 examples, [36](#)
 multilingual environment, [35](#)

replacement characters
 CONVERT SQL function, [4](#)
 restrictions
 data truncation, [2](#)
 passwords, [3](#)
 space padding during export, [3](#)
 usernames, [3](#)
 reverse secondary sorting, [16](#)
 ROUND (date) SQL function, [13](#)
 RPAD SQL function, [8](#)

S

searching multilingual documents, [15](#)
 searching string, [34](#)
 secondary level sort, [6](#)
 session time zone, [40](#)
 SESSIONTIMEZONE SQL function, [13](#)
 shift-sensitive variable-width multibyte encoding
 schemes, [7](#)
 single-byte encoding schemes, [6](#)
 sorting
 reverse secondary, [16](#)
 specifying nondefault linguistic sorts, [32](#)
 space padding
 during export, [3](#)
 special combination letters, [15, 17](#)
 special letters, [15, 17](#)
 special lowercase letters, [18](#)
 special uppercase letters, [17](#)
 SQL CHAR datatypes, [11](#)
 OCI, [14](#)
 SQL function
 ORA_DST_AFFECTED, [14](#)
 ORA_DST_CONVERT, [14](#)
 ORA_DST_ERROR, [14](#)
 SQL functions, [14](#)
 ADD_MONTHS, [13](#)
 CONVERT, [4](#)
 CURRENT_DATE, [13](#)
 CURRENT_TIMESTAMP, [13](#)
 datatype conversion, [7](#)
 DBTIMEZONE, [13](#)
 default values for NLS parameters, [2](#)
 EXTRACT (datetime), [13](#)
 FROM_TZ, [13](#)
 INSTR, [8](#)
 LAST_DAY, [13](#)
 LOCALTIMESTAMP, [13](#)
 MONTHS_BETWEEN, [13](#)
 NEW_TIME, [13](#)
 NEXT_DAY, [13](#)
 NLSSORT, [8](#)
 NUMTODSINTERVAL, [13](#)
 NUMTOYMINTERVAL, [13](#)
 ROUND (date), [13](#)

SQL functions (*continued*)

- RPAD, [8](#)
- SESSIONTIMEZONE, [13](#)
- specifying NLS parameters, [2](#)
- SYS_EXTRACT_UTC, [13](#)
- SYSDATE, [13](#)
- SYSTIMESTAMP, [14](#)
- TO_CHAR (datetime), [14](#)
- TO_DSINTERVAL, [14](#)
- TO_TIMESTAMP_TZ, [14](#)
- TO_YMINTERVAL, [14](#)
- TRUNC (date), [13](#)
- TZ_OFFSET, [14](#)
- unacceptable NLS parameters, [4](#)

SQL NCHAR datatypes

- binding and defining in OCI, [15](#)

SQL statements

- LIKE conditions, [7](#)

strict superset, [3](#)

string comparisons

- WHERE clause, [10](#)

string literals

- Unicode, [8](#)

string manipulation using OCI, [3](#)

strings

- searching, [34](#)

superset, strict, [3](#)

supersets and subsets, [A-19](#)

supplementary characters, [4](#)

- linguistic collation support, [A-27](#)

supported datatypes, [17](#)

supported territories, [A-5](#)

syllabary, [3](#)

symbols, encoding, [3](#)

SYS_EXTRACT_UTC SQL function, [13](#)

SYSDATE SQL function, [13](#)

SYSTIMESTAMP SQL function, [14](#)

T

territory

- dependencies, [7](#)

territory definition, [13](#)

- customizing, [10](#)
- overriding, [6](#)

territory support, [5](#), [A-5](#)

territory variant, [7](#)

tertiary level sort, [6](#)

Thai and Laotian character rearrangement, [17](#)

tilde, [27](#)

time and date parameters, [16](#)

time zone

- abbreviations, [17](#)
- data source, [17](#)
- database, [39](#)
- environment variables, [16](#)

time zone (*continued*)

- file, [17](#)
- names, [17](#)
- session, [40](#)

time zone file

- choosing, [17](#)
- upgrading, [20](#)

time zones

- converting, [41](#)
- upgrading time zone file, [20](#)

TIMESTAMP data type

- when to use, [9](#)

TIMESTAMP data types

- choosing, [9](#)

timestamp format, [20](#)

TIMESTAMP WITH LOCAL TIME ZONE data type

- when to use, [9](#)

TIMESTAMP WITH TIME ZONE data type

- when to use, [9](#)

TO_CHAR (datetime) SQL function, [14](#)

TO_CHAR SQL function

- format masks, [12](#)
- group separator, [26](#)
- spelling of days and months, [18](#)

TO_DATE SQL function

- format masks, [12](#)
- spelling of days and months, [18](#)

TO_DSINTERVAL SQL function, [14](#)

TO_NUMBER SQL function

- format masks, [12](#)

TO_TIMESTAMP, [14](#)

TO_TIMESTAMP SQL function, [14](#)

TO_TIMESTAMP_TZ SQL function, [14](#)

TO_YMINTERVAL SQL function, [14](#)

transportable NLB data, [39](#)

TRUNC (date) SQL function, [13](#)

TZ_OFFSET SQL function, [14](#)

TZABBREV, [17](#)

TZNAME, [17](#)

U

UCS-2 encoding, [4](#)

Unicode, [1](#)

- binding and defining Java strings, [20](#)
- character code assignments, [B-1](#)
- character set conversion between OCI client and database server, [12](#)
- code ranges for UTF-16 characters, [B-1](#)
- code ranges for UTF-8 characters, [B-1](#)
- data conversion in Java, [23](#)
- encoding, [2](#)
- fonts, [1](#)
- JDBC OCI driver, [3](#)
- JDBC programming, [20](#)
- JDBC Server Side internal driver, [3](#)

Unicode (continued)

- JDBC Server Side thin driver, [3](#)
 - JDBC thin driver, [3](#)
 - mode, [11](#)
 - ODBC and OLE DB programming, [28](#)
 - Oracle Call Interface, [2](#)
 - Oracle Data Provider for .NET, [2](#)
 - Oracle ODBC driver, [2](#)
 - Oracle OLE DB driver, [2](#)
 - Oracle Pro*C/C++, [2](#)
 - Oracle support, [5](#)
 - parsing an XML stream with Java, [33](#)
 - PL/SQL and SQL, [3](#)
 - Private Use Area, [22](#)
 - programming, [1](#)
 - reading an XML file with Java, [33](#)
 - string literals, [8](#)
 - UCS-2 encoding, [4](#)
 - UTF-16 encoding, [3](#)
 - UTF-8 encoding, [3](#)
 - writing an XML file with Java, [32](#)
 - XML programming, [32](#)
- Unicode database, [7](#)
- case study, [10](#)
- Unicode datatypes, [8](#)
- case study, [11](#)
- upgrade
- Daylight Saving Time, [16](#)
 - time zone data upgrade overview, [20](#)
 - time zone data upgrade using the DBMS_DST package, [22](#)
 - time zone data upgrade using the utlzl_* scripts, [35](#)

upgrade (continued)

- url-rewrite-rule, [40](#)
- US7ASCII
- supersets, [A-20](#)
- user-defined characters, [21](#)
- adding to a character set definition, [26](#)
 - cross-references between character sets, [22](#)
- UTC, [4](#), [5](#)
- UTF-16 encoding, [3](#), [B-2](#)
- UTF-8 encoding, [3](#), [B-2](#)
- UTF8 character set, [7](#), [A-18](#)
- UTFE character set, [5](#), [A-18](#)

V**VARCHAR datatype**

- Pro*C/C++, [18](#)
- variable-width multibyte encoding schemes, [7](#)

W

- wave dash, [27](#)
- WHERE clause
- string comparisons, [10](#)

X**XML**

- parsing in Unicode with Java, [33](#)
 - reading in Unicode with Java, [33](#)
 - writing in Unicode with Java, [32](#)
- XML programming
- Unicode, [32](#)